

langchain

57개 문서

목차

LangChain Academy

Agents

- Core components
- Invocation
- Streaming
- Configure the harness
- Component categories
- Common patterns
- Learn more

Context engineering in agents

- Overview
- Model context
- Tool context
- Life-cycle context
- Best practices
- Related resources

Build a data analysis agent from scratch

- What you will learn
- Setup

Local: Ollama must be running on your machine

Cloud: Set your Ollama API key for hosted inference

- Add context management
- Add skills
- Add a visualization subagent
- What you built

Deployment

- LangSmith Cloud

Event streaming

- What you can stream
- Agent messages
- Reasoning content
- Tool calls
- Streaming sub-agents
- State and final output
- Multiple projections
- Custom updates
- Related

Get help

- Learning resources
- Community support

- Professional support
- Contribute
- Stay connected
- Guardrails
 - Built-in guardrails
 - Custom guardrails
- Check for banned keywords
- Use the custom guardrail
- This request will be blocked before any processing
- Use a model to evaluate safety
- Use the safety guardrail
 - Additional resources
- Human-in-the-loop
 - Interrupt decision types
 - Configuring interrupts
 - Conditional interrupts
 - Responding to interrupts
 - Streaming with human-in-the-loop
 - Execution lifecycle
 - Custom HITL logic
- Install LangChain
- Requires Python 3.10+
- Requires Node.js 22+
- Requires Node.js 22+
- Installing the OpenAI integration
- Installing the Anthropic integration
- Installing the OpenAI integration
- Installing the Anthropic integration
- Installing the OpenAI integration
- Installing the Anthropic integration
- Build a semantic search engine with LangChain
 - Overview
 - Setup
 - Create documents
 - Generate embeddings
 - Index documents
 - Query the vector store
 - Use retrievers
 - Next steps
- Long-term memory
 - Usage
 - Memory storage
 - Read long-term memory in tools

Write long-term memory from tools

Messages

Basic usage

Message types

System message - Tells the model how to behave and provide context for interactions

Human message - Represents user input and interactions with the model

AI message - Responses generated by the model, including text content, tool calls, and metadata

Message content

From URL

From base64 data

From provider-managed File ID

From base64 data

From provider-managed File ID

Serialization

Use with chat models

Models

Basic usage

Parameters

Invocation

The

The sky

The sky is

The sky is typically

The sky is typically blue

...

```
[{"type": "text", "text": "The sky is typically blue..."}]
```

Bind (potentially multiple) tools to the model

Step 1: Model generates tool calls

Step 2: Execute tools and collect results

Step 3: Pass results back to model for final response

"The current weather in Boston is 72°F and sunny."

The model may generate multiple tool calls

```
[  
{'name': 'get_weather', 'args': {'location': 'Boston'}, 'id': 'call_1'},  
{'name': 'get_weather', 'args': {'location': 'Tokyo'}, 'id': 'call_2'},  
]
```

Execute all tools (can be done in parallel with async)

Output:

Tool: get_weather

ID: call_SvMIU1TVIZugrFLckFE2ceRE

Args: {"lo

Args: catio

Args: n": "B

Args: osto

Args: n"}

Tool: get_weather

ID: call_QMZdy6qInx13oWKE7KhuhOLR

Args: {"lo

Args: catio

Args: n": "T

Args: okyo

Args: "}

Structured output

Advanced topics

[

{"type": "text", "text": "Here's a picture of a cat"},

{"type": "image", "base64": "...", "mime_type": "image/jpeg"},

]

LangSmith Observability

Prerequisites

Enable tracing

Quickstart

LangChain overview

pip install -qU langchain "langchain[openai]"

pip install -qU langchain "langchain[anthropic]"

pip install -qU langchain langchain-fireworks

pip install -qU langchain langchain-ollama

pip install -qU langchain langchain-aws

US cross-region inference profile; use global.anthropic.claude-sonnet-4-6 for worldwide routing.

Install Python 3.11+ separately if needed.

Requires Node.js 22+

Requires Node.js 22+

Set up API keys

Local: Ollama must be running (<https://ollama.com>)

Cloud: Set your Ollama API key for hosted inference

Build a basic agent

Capabilities

Capabilities

Trace agent calls

Next steps

Retrieval

Runtime

Overview

Access

Short-term memory

Overview

- Usage
 - Customizing agent memory
 - Common patterns
 - Access memory
- Build a SQL agent
 - Overview
 - Setup
 - Build your SQL agent
 - Next steps
- Streaming
 - Overview
 - Supported stream modes
 - Agent progress
 - Custom updates
 - Common patterns
 - v2 streaming format
- Unified format — no more tuple unpacking
 - Related
- Structured output
 - Response format
 - Response format
 - Provider strategy
- ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')
{'name': 'John Doe', 'email': 'john@example.com', 'phone': '(555) 123-4567'}
- ProductReview(rating=5, sentiment='positive', key_points=['fast shipping', 'expensive'])
{'rating': 5, 'sentiment': 'positive', 'key_points': ['fast shipping', 'expensive']}
- ProductReview(rating=5, sentiment='positive', key_points=['fast shipping', 'expensive'])
- LangSmith Studio
- Tools
 - Create tools
 - Access context
- Access memory
- Update memory
- First session: save user info
- Second session: get user info
- Here is the user info for user with ID "abc123":
 - Name: Foo
 - Age: 25
 - Email: foo@langchain.dev
- Tool execution
 - Dynamic tool selection
 - Headless tools
 - Prebuilt tools

- Tools from MCP servers

- Server-side tool use

- Agent Chat UI

- Build a voice agent with LangChain

- Overview

- Setup

- 1. Speech-to-text

- 2. LangChain agent

- 3. Text-to-speech

- Putting it all together

- INVALID_PROMPT_INPUT

- Troubleshooting

- INVALID_TOOL_RESULTS

- Common causes

- Troubleshooting

- MESSAGE_COERCION_FAILURE

- Accepted message formats

- Troubleshooting

- MODEL_AUTHENTICATION

- Troubleshooting

- MODEL_NOT_FOUND

- Troubleshooting

- MODEL_RATE_LIMIT

- Troubleshooting

- OUTPUT_PARSING_FAILURE

- Troubleshooting

- Authentication

- Bearer token

- OAuth authentication

- Per-server authentication

- Per-user authentication

- See also

- Bearer token

- OAuth authentication

- Handle authentication errors

- Header precedence

- Per-user authentication

- See also

- Connections

- Choose a pattern

- Connection lifecycle

- Multiple servers

- Scale a deployment

Caching

Protocol eras

See also

Model Context Protocol (MCP)

Install

Quickstart

Transports

Next steps

Tools

Use MCP tools in an agent

Handle tool outputs

Tool metadata

Human-in-the-loop

Server requests during tool execution

See also

Prebuilt middleware

Provider-agnostic middleware

Provider-specific middleware

Custom middleware

Hooks

State updates

Create middleware

Custom state schema

Custom stream transformers

Custom context

Execution order

Agent jumps

Configure tracing

Best practices

Examples

Additional resources

Middleware overview

The agent loop

Use middleware inside a LangGraph workflow

Additional resources

Custom workflow

Key characteristics

When to use

Basic implementation

Example: RAG pipeline

Build customer support with handoffs

Setup

1. Define custom state

2. Create tools that manage workflow state
3. Define step configurations
4. Create step-based middleware
5. Create the agent
6. Test the workflow
7. Understanding state transitions
8. Manage message history
9. Add flexibility: Go back

Complete example

Next steps

Handoffs

Key characteristics

When to use

Basic implementation

Implementation approaches

Implementation considerations

Multi-agent

Why multi-agent?

Context management: Provide specialized knowledge without overwhelming the model's context window. If context were infinite and latency zero, you could dump all knowledge into a single prompt—but since it's not, you need patterns to selectively surface relevant information.

Distributed development: Allow different teams to develop and maintain capabilities independently, composing them into a larger system with clear boundaries.

Patterns

Performance comparison

Build a multi-source knowledge base with routing

Overview

Setup

1. Define state
2. Define tools for each vertical
3. Create specialized agents
4. Build the router workflow
5. Compile the workflow
6. Use the router
7. Understanding the architecture
8. Complete working example
9. Advanced: Stateful routers
10. Key takeaways

Next steps

Router

Key characteristics

When to use

Basic implementation

Stateless vs. stateful

Stateless

Stateful

Build a SQL assistant with on-demand skills

How it works

Setup

1. Define skills
2. Create skill loading tool
3. Build skill middleware
4. Create the agent with skill support
5. Test progressive disclosure
6. Advanced: Add constraints with custom state

Complete example

Implementation variations

Progressive disclosure and context engineering

Next steps

Skills

Key characteristics

When to use

Basic implementation

Extending the pattern

Build a personal assistant with subagents

Overview

Setup

2. Create specialized sub-agents
3. Wrap sub-agents as tools
4. Create the supervisor agent
5. Use the supervisor
6. Add human-in-the-loop review
7. Advanced: Control information flow
8. Key takeaways

Next steps

Subagents

Key characteristics

When to use

Basic implementation

Design decisions

Sync vs. async

Tool patterns

Context engineering

Checkpointing and state inspection

Agent Evals

Install AgentEvals

- Trajectory match evaluator

- LLM-as-judge evaluator

- Run evals in LangSmith

Test

Integration testing

- Separate unit and integration tests

- Manage API keys

- Assert on structure, not content

- Use custom test matchers

- Reduce cost and latency

- Record and replay HTTP calls

- Next steps

Unit testing

- Mock chat model

- InMemorySaver checkpointer

- Mock chat model with fakeModel

- Next steps

langchain

LangChain Academy

langchain/academy.mdx

Agents

langchain/agents.mdx

An agent is a model calling tools in a loop until a given task is complete.

 Core agent loop diagram

A harness is everything around that loop: the prompt, the tools, and any middleware that shapes the model's behavior.

참고

Agent = Model + Harness

The job of a harness: get the model the right context at the right time for the given task.

@[`create_agent`] is a highly configurable harness. At its simplest, you can create one with:

python

js

Building on that, you can configure the basics directly with the `model=`, `tools=`, and `system_prompt=` parameters. For more advanced capabilities, extend the harness with [middleware](#).

팁

[Deep Agents](#) builds on `create_agent` and comes with commonly useful capabilities already assembled, such as planning, file system tools, subagents, and memory. Use `create_agent` when you need to configure the harness yourself.

Core components

 Agent model and harness components diagram

Model

Pass a model identifier string (`"provider:model"`) or an initialized model instance to select the model for your agent. See [Models](#) for parameters, provider setup, and dynamic model selection.

python

```
js
```

Tools

To provide the agent with tools, pass any Python callable, LangChain tool, or tool dict. See [Tools](#) for tool definition, context access, and dynamic tool selection.

```
python
```

```
js
```

System prompt

Shape how the agent approaches tasks. The system prompt parameter accepts a string or `SystemMessage`. For dynamic prompts at runtime, use [middleware](#).

```
python
```

```
js
```

Structured output

Return a validated schema from the agent using `response_format=`. See [Structured output](#) for strategies and examples.

```
python
```

```
js
```

Agent state

```
python
```

Every agent manages its execution context through `@[AgentState]`, a typed dictionary that holds the current conversation history and any custom fields your tools and middleware need.

The built-in field is:

Field	Type	Description
<code>messages</code>	<code>list[BaseMessage]</code>	The full conversation history for the current thread. Append-only: new messages are added, never replaced.

`AgentState` is also the type signature for every node-style middleware hook (`before_model` , `after_model` , and similar). Hooks receive the current state and can return a dict of updates to merge back into it.

To add custom fields (for example, a `user_id` or a counter), subclass `AgentState` and pass the subclass to `create_agent` via `state_schema=` :

For full details, examples, and middleware-level state schemas, see [Short-term memory](#) and [Custom middleware](#).

js

Every agent manages its execution context through an `AgentState` object that holds the current conversation history and any custom fields your tools and middleware need.

The built-in field is:

Field	Type	Description
<code>messages</code>	<code>BaseMessage[]</code>	The full conversation history for the current thread. Append-only: new messages are added, never replaced.

`AgentState` is also the type passed to every node-style middleware hook (`beforeModel` , `afterModel` , and similar). Hooks receive the current state and can return an object of updates to merge back into it.

To add custom fields, define a state schema on your middleware using `stateSchema` with a `StateSchema` or Zod object:

For full details, examples, and middleware-level state schemas, see [Short-term memory](#) and [Custom middleware](#).

Invocation

팁

Trace each step of this loop, debug tool calls, and evaluate agent outputs with [LangSmith](#). Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

python

You can invoke an agent with a message. Behind the scenes that passes an update to the agent's `State` . All agents include a [sequence of messages](#) in their state; to invoke the agent,

pass a new message along with a `thread_id` so the agent can persist and resume conversation history:

js

You can invoke an agent with a message. Behind the scenes that passes an update to the agent's `state`. All agents include a [sequence of messages](#) in their state; to invoke the agent, pass a new message along with a `thread_id` so the agent can persist and resume conversation history:

python

js

참고

Persisting conversation history with `thread_id` requires the agent to be configured with a [checkpointer](#). When deployed on [LangSmith](#), a checkpointer is provisioned automatically. Locally, pass one explicitly, for example `create_agent(..., checkpointer=InMemorySaver())`.

python

If you also need to pass per-run configuration (such as a user ID, API keys, or feature flags) to tools and middleware, pass it as `context` alongside `config`. Define the shape of that data with `context_schema` and access it through `runtime.context`:

`thread_id` scopes the *conversation* (message history, checkpoints), while `context` carries *per-run* data your tools and middleware read at invocation time. Both are commonly passed together. See [tool context](#) and [Runtime](#) for more.

js

If you also need to pass per-run configuration (such as a user ID, API keys, or feature flags) to tools and middleware, pass it as `context` alongside the config. Define the shape of that data with `contextSchema` and access it through `runtime.context`:

`thread_id` scopes the *conversation* (message history, checkpoints), while `context` carries *per-run* data your tools and middleware read at invocation time. Both are commonly passed together. See [tool context](#) and [Runtime](#) for more.

Streaming

`invoke` returns the final response at the end of a run. If an agent executes multiple tool calls, users often need progress updates before completion. Use streaming to surface intermediate messages and tool activity as they happen.

python

js

팁

For streaming modes, event types, and UI patterns, see [Streaming](#).

Configure the harness

`create_agent` is highly extensible. Middleware is the primitive for customization: each piece handles one concern, hooks into the agent loop at the right moment, and composes freely with any other. Take exactly what your use case needs and skip the rest.

Common patterns are prebuilt as first-class middleware. You can build anything else as [custom middleware](#).



Agent harness capabilities by category

As agents take on complex work, they need support across a few key areas. The middleware ecosystem provides:

[Execution environment](#)

Tools, filesystem, sandboxes, and code execution

[Context management](#)

Summarization, memory, skills, and prompt caching

[Planning and delegation](#)

Todo lists and subagents for parallel, isolated work

[Fault tolerance](#)

Retries, fallbacks, and call limits

[Guardrails](#)

PII detection and content controls

Steering

Human-in-the-loop approval before high-impact actions

팁

`create_deep_agent` pre-assembles this stack for long-running coding and research tasks (filesystem, summarization, subagents, and prompt caching included by default). See [Deep Agents](#) for the full prebuilt harness.

Execution environment

Agents are especially useful when they can take action rather than just generate text. The execution environment gives the agent a workspace: tools it can call, a filesystem for reading and writing files across turns, and code execution for running scripts or shell commands.

python

js

See @`[FilesystemMiddleware]`, [Sandboxes](#), [Interpreters](#).

참고

This example imports from the `deepagents` package. Install it with:

python

```
```bash pip pip install deepagents
```

```
```bash uv
uv add deepagents
```

js

```
```bash npm npm install deepagents
```

```
```bash yarn
yarn add deepagents
```

```
```bash pnpm pnpm add deepagents
```

```
</div>
```

```
</div>
```

### Context management

Every model call has a fixed context window. As an agent runs, that window fills with a cumulating history, tool results, and intermediate steps. Summarization compresses history before overflow hits; memory loads persistent instructions at startup so knowledge carries across sessions; skills surface domain knowledge on demand rather than loading everything upfront.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
</div>
```

See [@\[`SummarizationMiddleware`\]](#), [@\[`MemoryMiddleware`\]](#), [\[Skills\]\(/oss/langchain/multi-agent/skills\)](#), [\[Context engineering\]\(/oss/deepagents/context-engineering\)](#).

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

This example imports from the `deepagents` package. Install it with:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```bash pip
pip install deepagents
```

```
```bash uv uv add deepagents
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```



```
```bash npm
npm install deepagents
```

```
```bash yarn yarn add deepagents
```

```
```bash pnpm
pnpm add deepagents
```

Planning and delegation

Complex tasks often exceed what one context window can handle. Delegation lets the main agent break work into pieces, hand them to subagents that each run in their own isolated context, and stay focused on coordination rather than execution. Work can run in parallel; the main agent's context stays clean.

python

js

See [Subagents](#).

참고

This example imports from the `deepagents` package. Install it with:

python

```
```bash pip pip install deepagents
```

```
```bash uv
uv add deepagents
```

js

```
```bash npm npm install deepagents
```

```
```bash yarn
yarn add deepagents
```

```
```bash pnpm pnpm add deepagents
```

</div>

</div>

### ### Name your agent

Optionally use an identifier for the agent. This is especially useful when embedding the agent as a subgraph in [multi-agent](/oss/langchain/multi-agent) systems.

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

</div>

### ### Fault tolerance

Agents in production encounter failures that rarely appear in development: rate limits, model timeouts, transient API errors. Fault tolerance middleware handles these at the infrastructure level so your tools and business logic don't need try/catch around every call.

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

</div>

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

See @[`ModelRetryMiddleware`], @[`ToolRetryMiddleware`], [Prebuilt middleware](/oss/langchain/middleware/built-in).

</div>

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

See `@[`modelRetryMiddleware`]`, `@[`toolRetryMiddleware`]`, `[Prebuilt middleware](/oss/langchain/middleware/built-in)`.

```
</div>
```

### ### Guardrails

Some policies can't live in a prompt—they need to be enforced deterministically regardless of what the model does. Guardrails intercept data as it flows through the agent loop, applying compliance rules or content policies before tool results reach the model's context.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
</div>
```

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

See `@[`PIIMiddleware`]`, `[Prebuilt middleware](/oss/langchain/middleware/built-in)`.

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

See `@[`piiMiddleware`]`, `[Prebuilt middleware](/oss/langchain/middleware/built-in)`.

```
</div>
```

### ### Steering

Full autonomy isn't always appropriate. Steering lets you place humans at specific decision points—before destructive writes, expensive API calls, or anything requiring judgment—without restructuring your agent. The agent pauses and waits; a human approves, edits, or rejects; execution continues.

```

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

</div>

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

See @[`HumanInTheLoopMiddleware`], [Human-in-the-loop](/oss/langchain/human-in-the-loop).

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

See @[`humanInTheLoopMiddleware`], [Human-in-the-loop](/oss/langchain/human-in-the-loop).

</div>

Middleware resources

[Middleware overview](/oss/langchain/middleware/overview)

How the middleware stack works and when hooks fire

[Prebuilt middleware](/oss/langchain/middleware/built-in)

Full reference with configuration examples

[Custom middleware](/oss/langchain/middleware/custom)

Write your own hooks for business logic, PII scrubbing, and more

<div class="page-break"></div>

Component architecture

<p class="source">langchain/component-architecture.mdx</p>

```

LangChain's power comes from how its components work together to create sophisticated AI applications. This page provides diagrams showcasing the relationships between different components.

### ## Core component ecosystem

The diagram below shows how LangChain's major components connect to form complete AI applications:

```
```mermaid
graph TD
    %% Input processing
    subgraph "📄 Input processing"
        A[Text input] --> B[Document loaders]
        B --> C[Text splitters]
        C --> D[Documents]
    end

    %% Embedding & storage
    subgraph "📚 Embedding & storage"
        D --> E[Embedding models]
        E --> F[Vectors]
        F --> G[(Vector stores)]
    end

    %% Retrieval
    subgraph "🔍 Retrieval"
        H[User Query] --> I[Embedding models]
        I --> J[Query vector]
        J --> K[Retrievers]
        K --> G
        G --> L[Relevant context]
    end

    %% Generation
    subgraph "🗨️ Generation"
        M[Chat models] --> N[Tools]
        N --> O[Tool results]
        O --> M
        L --> M
        M --> P[AI response]
    end

    %% Orchestration
    subgraph "🎮 Orchestration"
        Q[Agents] --> M
        Q --> N
        Q --> K
        Q --> R[Memory]
    end
```
```

```

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef output fill:#EBD0F0,stroke:#885270,stroke-width:2px,color:#441E33
classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68

class A,H trigger
class B,C,E,I,K,M,N,Q process
class D,F,J,L,O,P,R neutral
class G output

```

## How components connect

Each component layer builds on the previous ones:

1. **Input processing** – Transform raw data into structured documents
2. **Embedding & storage** – Convert text into searchable vector representations
3. **Retrieval** – Find relevant information based on user queries
4. **Generation** – Use AI models to create responses, optionally with tools
5. **Orchestration** – Coordinate everything through agents and memory systems

## Component categories

LangChain organizes components into these main categories:

| Category                            | Purpose                     | Key Components                      | Use Cases                                          |
|-------------------------------------|-----------------------------|-------------------------------------|----------------------------------------------------|
| <a href="#">Models</a>              | AI reasoning and generation | Chat models, LLMs, Embedding models | Text generation, reasoning, semantic understanding |
| <a href="#">Tools</a>               | External capabilities       | APIs, databases, etc.               | Web search, data access, computations              |
| <a href="#">Agents</a>              | Orchestration and reasoning | ReAct agents, tool calling agents   | Nondeterministic workflows, decision making        |
| <a href="#">Memory</a>              | Context preservation        | Message history, custom state       | Conversations, stateful interactions               |
| <a href="#">Retrievers</a>          | Information access          | Vector retrievers, web retrievers   | RAG, knowledge base search                         |
| <a href="#">Document processing</a> | Data ingestion              | Loaders, splitters, transformers    | PDF processing, web scraping                       |
| <a href="#">Vector Stores</a>       | Semantic search             | Chroma, Pinecone, FAISS             | Similarity search, embeddings storage              |

## Common patterns

### RAG (Retrieval-Augmented generation)

```
graph LR
 A[User question] --> B[Retriever]
 B --> C[Relevant docs]
 C --> D[Chat model]
 A --> D
 D --> E[Informed response]

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68

class A trigger
class B,D process
class C,E neutral
```

### Agent with tools

```
graph LR
 A[User request] --> B[Agent]
 B --> C{Need tool?}
 C -->|Yes| D[Call tool]
 D --> E[Tool result]
 E --> B
 C -->|No| F[Final answer]

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef decision fill:#FDF3FF,stroke:#7E65AE,stroke-width:2px,color:#504B5F
classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68

class A trigger
class B,D process
class C decision
class E,F neutral
```

### Multi-agent system

```
graph LR
 A[Complex Task] --> B[Supervisor agent]
 B --> C[Specialist agent 1]
 B --> D[Specialist agent 2]
 C --> E[Results]
 D --> E
```

```
E --> B
B --> F[Coordinated response]
```

```
classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68
```

```
class A trigger
class B,C,D process
class E,F neutral
```

## Learn more

---

- [Creating agents](#)
- [Working with tools](#)
- [Browse integrations](#)



# Context engineering in agents

langchain/context-engineering.mdx

## Overview

The hard part of building agents (or any LLM application) is making them reliable enough. While they may work for a prototype, they often fail in real-world use cases.

### Why do agents fail?

When agents fail, it's usually because the LLM call inside the agent took the wrong action / didn't do what we expected. LLMs fail for one of two reasons:

1. The underlying LLM is not capable enough
2. The "right" context was not passed to the LLM

More often than not - it's actually the second reason that causes agents to not be reliable.

**Context engineering** is providing the right information and tools in the right format so the LLM can accomplish a task. This is the number one job of AI Engineers. This lack of "right" context is the number one blocker for more reliable agents, and LangChain's agent abstractions are uniquely designed to facilitate context engineering.

#### 팁

New to context engineering? Start with the [conceptual overview](#) to understand the different types of context and when to use them.

## The agent loop

A typical agent loop consists of two main steps:

1. **Model call** - calls the LLM with a prompt and available tools, returns either a response or a request to execute tools
2. **Tool execution** - executes the tools that the LLM requested, returns tool results



Core agent loop diagram

This loop continues until the LLM decides to finish.

## What you can control

To build reliable agents, you need to control what happens at each step of the agent loop, as well as what happens between steps.

| Context Type                       | What You Control                                                                     | Transient or Persistent |
|------------------------------------|--------------------------------------------------------------------------------------|-------------------------|
| <a href="#">Model Context</a>      | What goes into model calls (instructions, message history, tools, response format)   | Transient               |
| <a href="#">Tool Context</a>       | What tools can access and produce (reads/writes to state, store, runtime context)    | Persistent              |
| <a href="#">Life-cycle Context</a> | What happens between model and tool calls (summarization, guardrails, logging, etc.) | Persistent              |

## Transient context

What the LLM sees for a single call. You can modify messages, tools, or prompts without changing what's saved in state.

## Persistent context

What gets saved in state across turns. Life-cycle hooks and tool writes modify this permanently.

## Data sources

Throughout this process, your agent accesses (reads / writes) different sources of data:

| Data Source            | Also Known As        | Scope               | Examples                                                                   |
|------------------------|----------------------|---------------------|----------------------------------------------------------------------------|
| <b>Runtime Context</b> | Static configuration | Conversation-scoped | User ID, API keys, database connections, permissions, environment settings |
| <b>State</b>           | Short-term memory    | Conversation-scoped | Current messages, uploaded files, authentication status, tool results      |
| <b>Store</b>           | Long-term memory     | Cross-conversation  | User preferences, extracted insights, memories, historical data            |

## How it works

LangChain [middleware](#) is the mechanism under the hood that makes context engineering practical for developers using LangChain.

Middleware allows you to hook into any step in the agent lifecycle and:

- Update context
- Jump to a different step in the agent lifecycle

Throughout this guide, you'll see frequent use of the middleware API as a means to the context engineering end.

## Model context

---

Control what goes into each model call - instructions, available tools, which model to use, and output format. These decisions directly impact reliability and cost.

### [System Prompt](#)

Base instructions from the developer to the LLM.

### [Messages](#)

The full list of messages (conversation history) sent to the LLM.

### [Tools](#)

Utilities the agent has access to for taking actions.

### [Model](#)

The actual model (including configuration) to be called.

### [Response Format](#)

Schema specification for the model's final response.

All of these types of model context can draw from **state** (short-term memory), **store** (long-term memory), or **runtime context** (static configuration).

## System Prompt

The system prompt sets the LLM's behavior and capabilities. Different users, contexts, or conversation stages need different instructions. Successful agents draw on memories, preferences, and configuration to provide the right instructions for the current state of the conversation.

### State

Access message count or conversation context from state:

#### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

@dynamic_prompt
def state_aware_prompt(request: ModelRequest) -> str:
 # request.messages is a shortcut for request.state["messages"]
 message_count = len(request.messages)

 base = "You are a helpful assistant."
```

```

if message_count > 10:
 base += "\nThis is a long conversation - be extra concise."

return base

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[state_aware_prompt]
)

```

**js**

```

import { createAgent } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 middleware: [
 dynamicSystemPromptMiddleware((state) => {
 // Read from State: check conversation length
 const messageCount = state.messages.length;

 let base = "You are a helpful assistant.";

 if (messageCount > 10) {
 base += "\nThis is a long conversation - be extra concise.";
 }

 return base;
 }),
],
});

```

## Store

Access user preferences from long-term memory:

**python**

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:

```

```

 user_id: str

@dynamic_prompt
def store_aware_prompt(request: ModelRequest) -> str:
 user_id = request.runtime.context.user_id

 # Read from Store: get user preferences
 store = request.runtime.store
 user_prefs = store.get(("preferences",), user_id)

 base = "You are a helpful assistant."

 if user_prefs:
 style = user_prefs.value.get("communication_style", "balanced")
 base += f"\nUser prefers {style} responses."

 return base

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[store_aware_prompt],
 context_schema=Context,
 store=InMemoryStore()
)

```

js

```

import * as z from "zod";
import { createAgent, dynamicSystemPromptMiddleware } from "langchain";

const contextSchema = z.object({
 userId: z.string(),
});

type Context = z.infer<typeof contextSchema>;

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 contextSchema,
 middleware: [
 dynamicSystemPromptMiddleware<Context>(async (state, runtime) => {
 const userId = runtime.context.userId;

 // Read from Store: get user preferences
 const store = runtime.store;
 const userPrefs = await store.get(["preferences"], userId);
 })
]
});

```

```

let base = "You are a helpful assistant.";

if (userPrefs) {
 const style = userPrefs.value?.communicationStyle || "balanced";
 base += `\nUser prefers ${style} responses.`;
}

 return base;
}),
],
});

```

## Runtime Context

Access user ID or configuration from Runtime Context:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

@dataclass
class Context:
 user_role: str
 deployment_env: str

@dynamic_prompt
def context_aware_prompt(request: ModelRequest) -> str:
 # Read from Runtime Context: user role and environment
 user_role = request.runtime.context.user_role
 env = request.runtime.context.deployment_env

 base = "You are a helpful assistant."

 if user_role == "admin":
 base += "\nYou have admin access. You can perform all operations."
 elif user_role == "viewer":
 base += "\nYou have read-only access. Guide users to read operations only."

 if env == "production":
 base += "\nBe extra careful with any data modifications."

 return base

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[context_aware_prompt],

```

```
context_schema=Context
)
```

js

```
import * as z from "zod";
import { createAgent, dynamicSystemPromptMiddleware } from "langchain";

const contextSchema = z.object({
 userRole: z.string(),
 deploymentEnv: z.string(),
});

type Context = z.infer<typeof contextSchema>;

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 contextSchema,
 middleware: [
 dynamicSystemPromptMiddleware<Context>((state, runtime) => {
 // Read from Runtime Context: user role and environment
 const userRole = runtime.context.userRole;
 const env = runtime.context.deploymentEnv;

 let base = "You are a helpful assistant.";

 if (userRole === "admin") {
 base += "\nYou have admin access. You can perform all operations.";
 } else if (userRole === "viewer") {
 base += "\nYou have read-only access. Guide users to read operations only.";
 }

 if (env === "production") {
 base += "\nBe extra careful with any data modifications.";
 }

 return base;
 }),
],
});
```

## Messages

Messages make up the prompt that is sent to the LLM. It's critical to manage the content of messages to ensure that the LLM has the right information to respond well.

## State

Inject uploaded file context from State when relevant to current query:

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def inject_file_context(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Inject context about files user has uploaded this session."""
 # Read from State: get uploaded files metadata
 uploaded_files = request.state.get("uploaded_files", []) # [!code highlight]

 if uploaded_files:
 # Build context about available files
 file_descriptions = []
 for file in uploaded_files:
 file_descriptions.append(
 f"- {file['name']} ({file['type']}): {file['summary']}"
)

 file_context = f"""Files you have access to in this conversation:
{chr(10).join(file_descriptions)}

Reference these files when answering questions."""

 # Inject file context before recent messages
 messages = [# [!code highlight]
 *request.messages,
 {"role": "user", "content": file_context},
]
 request = request.override(messages=messages) # [!code highlight]

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[inject_file_context]
)
```

### js



```

import { createMiddleware, HumanMessage } from "langchain";

const injectFileContext = createMiddleware({
 name: "InjectFileContext",
 wrapModelCall: (request, handler) => {
 // request.state is a shortcut for request.state.messages
 const uploadedFiles = request.state.uploadedFiles || []; // [!code highlight]

 if (uploadedFiles.length > 0) {
 // Build context about available files
 const fileDescriptions = uploadedFiles.map(file =>
 `- ${file.name} (${file.type}): ${file.summary}`
);

 const fileContext = `Files you have access to in this conversation:
${fileDescriptions.join("\n")}`;

 Reference these files when answering questions.`;

 // Inject file context before recent messages
 const messages = [// [!code highlight]
 ...request.messages, // Rest of conversation
 new HumanMessage(fileContext)
];
 request = { ...request, messages }; // [!code highlight]
 }

 return handler(request);
 },
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 middleware: [injectFileContext],
});

```

## Store

Inject user's email writing style from Store to guide drafting:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable
from langgraph.store.memory import InMemoryStore

```

```

@dataclass
class Context:
 user_id: str

@wrap_model_call
def inject_writing_style(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Inject user's email writing style from Store."""
 user_id = request.runtime.context.user_id # [!code highlight]

 # Read from Store: get user's writing style examples
 store = request.runtime.store # [!code highlight]
 writing_style = store.get(("writing_style",), user_id) # [!code highlight]

 if writing_style:
 style = writing_style.value
 # Build style guide from stored examples
 style_context = f"""Your writing style:
- Tone: {style.get('tone', 'professional')}
- Typical greeting: "{style.get('greeting', 'Hi')}}"
- Typical sign-off: "{style.get('sign_off', 'Best')}}"
- Example email you've written:
{style.get('example_email', '')}"""

 # Append at end - models pay more attention to final messages
 messages = [
 *request.messages,
 {"role": "user", "content": style_context}
]
 request = request.override(messages=messages) # [!code highlight]

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[inject_writing_style],
 context_schema=Context,
 store=InMemoryStore()
)

```

js

```

import * as z from "zod";
import { createMiddleware, HumanMessage } from "langchain";

```

```

const contextSchema = z.object({
 userId: z.string(),
});

const injectWritingStyle = createMiddleware({
 name: "InjectWritingStyle",
 contextSchema,
 wrapModelCall: async (request, handler) => {
 const userId = request.runtime.context.userId; // [!code highlight]

 // Read from Store: get user's writing style examples
 const store = request.runtime.store; // [!code highlight]
 const writingStyle = await store.get(["writing_style"], userId); // [!code highlight]

 if (writingStyle) {
 const style = writingStyle.value;
 // Build style guide from stored examples
 const styleContext = `Your writing style:
- Tone: ${style.tone || 'professional'}
- Typical greeting: "${style.greeting || 'Hi'}"
- Typical sign-off: "${style.signOff || 'Best'}"
- Example email you've written:
${style.exampleEmail || ''}`;

 // Append at end - models pay more attention to final messages
 const messages = [
 ...request.messages,
 new HumanMessage(styleContext)
];
 request = { ...request, messages }; // [!code highlight]
 }

 return handler(request);
 },
});

```

## Runtime Context

Inject compliance rules from Runtime Context based on user's jurisdiction:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

```

```

@dataclass
class Context:
 user_jurisdiction: str
 industry: str
 compliance_frameworks: list[str]

@wrap_model_call
def inject_compliance_rules(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Inject compliance constraints from Runtime Context."""
 # Read from Runtime Context: get compliance requirements
 jurisdiction = request.runtime.context.user_jurisdiction # [!code highlight]
 industry = request.runtime.context.industry # [!code highlight]
 frameworks = request.runtime.context.compliance_frameworks # [!code highlight]

 # Build compliance constraints
 rules = []
 if "GDPR" in frameworks:
 rules.append("- Must obtain explicit consent before processing personal data")
 rules.append("- Users have right to data deletion")
 if "HIPAA" in frameworks:
 rules.append("- Cannot share patient health information without authorization")
 rules.append("- Must use secure, encrypted communication")
 if industry == "finance":
 rules.append("- Cannot provide financial advice without proper disclaimers")

 if rules:
 compliance_context = f"""Compliance requirements for {jurisdiction}:
{chr(10).join(rules)}"""

 # Append at end - models pay more attention to final messages
 messages = [
 *request.messages,
 {"role": "user", "content": compliance_context}
]
 request = request.override(messages=messages) # [!code highlight]

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[inject_compliance_rules],
 context_schema=Context
)

```

```

import * as z from "zod";
import { createMiddleware, HumanMessage } from "langchain";

const contextSchema = z.object({
 userJurisdiction: z.string(),
 industry: z.string(),
 complianceFrameworks: z.array(z.string()),
});

type Context = z.infer<typeof contextSchema>;

const injectComplianceRules = createMiddleware<Context>({
 name: "InjectComplianceRules",
 contextSchema,
 wrapModelCall: (request, handler) => {
 // Read from Runtime Context: get compliance requirements
 const { userJurisdiction, industry, complianceFrameworks } = request.runtime.co
ntext; // [!code highlight]

 // Build compliance constraints
 const rules = [];
 if (complianceFrameworks.includes("GDPR")) {
 rules.push("- Must obtain explicit consent before processing personal data");
 rules.push("- Users have right to data deletion");
 }
 if (complianceFrameworks.includes("HIPAA")) {
 rules.push("- Cannot share patient health information without authorization");
 rules.push("- Must use secure, encrypted communication");
 }
 if (industry === "finance") {
 rules.push("- Cannot provide financial advice without proper disclaimers");
 }

 if (rules.length > 0) {
 const complianceContext = `Compliance requirements for ${userJurisdiction}:
${rules.join("\n")}`;

 // Append at end - models pay more attention to final messages
 const messages = [
 ...request.messages,
 new HumanMessage(complianceContext)
];
 request = { ...request, messages }; // [!code highlight]
 }

 return handler(request);
 },
});

```

## 참고

### Transient vs Persistent Message Updates:

The examples above use `wrap_model_call` to make **transient** updates - modifying what messages are sent to the model for a single call without changing what's saved in state.

For **persistent** updates that modify state, you can:

#### python

- Return a `@[ExtendedModelResponse]` with a `@[Command]` from `wrap_model_call` to inject state updates from the model call layer.
- Use life-cycle hooks like `before_model`, `after_model`, or `wrap_tool_call` (for tool returns) to update the conversation history. See the [middleware documentation](#) for more details.

#### js

- Return a `@[Command]` directly from `wrapModelCall` to inject state updates from the model call layer.
- Use life-cycle hooks like `beforeModel`, `afterModel`, or `wrapToolCall` (for tool returns) to update the conversation history. See the [middleware documentation](#) for more details.

See [State updates](#) for more information.

## Tools

Tools let the model interact with databases, APIs, and external systems. How you define and select tools directly impacts whether the model can complete tasks effectively.

### Defining tools

Each tool needs a clear name, description, argument names, and argument descriptions. These aren't just metadata—they guide the model's reasoning about when and how to use the tool.

#### python

```
from langchain.tools import tool

@tool(parse_docstring=True)
```

```
def search_orders(
 user_id: str,
 status: str,
 limit: int = 10
) -> str:
 """Search for user orders by status.

 Use this when the user asks about order history or wants to check
 order status. Always filter by the provided status.

 Args:
 user_id: Unique identifier for the user
 status: Order status: 'pending', 'shipped', or 'delivered'
 limit: Maximum number of results to return
 """
 # Implementation here
 pass
```

js

```
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const searchOrders = tool(
 async ({ userId, status, limit }) => {
 // Implementation here
 },
 {
 name: "search_orders",
 description: `Search for user orders by status.

 Use this when the user asks about order history or wants to check
 order status. Always filter by the provided status.`,
 schema: z.object({
 userId: z.string().describe("Unique identifier for the user"),
 status: z.enum(["pending", "shipped", "delivered"]).describe("Order status to
 filter by"),
 limit: z.number().default(10).describe("Maximum number of results to retur
 n"),
 }),
 }
);
```

## Selecting tools

Not every tool is appropriate for every situation. Too many tools may overwhelm the model (overload context) and increase errors; too few limit capabilities. Dynamic tool

selection adapts the available toolset based on authentication state, user permissions, feature flags, or conversation stage.

## State

Enable advanced tools only after certain conversation milestones:

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def state_based_tools(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Filter tools based on conversation State."""
 # Read from State: check if user has authenticated
 state = request.state # [!code highlight]
 is_authenticated = state.get("authenticated", False) # [!code highlight]
 message_count = len(state["messages"])

 # Only enable sensitive tools after authentication
 if not is_authenticated:
 tools = [t for t in request.tools if t.name.startswith("public_")]
 request = request.override(tools=tools) # [!code highlight]
 elif message_count < 5:
 # Limit tools early in conversation
 tools = [t for t in request.tools if t.name != "advanced_search"]
 request = request.override(tools=tools) # [!code highlight]

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[public_search, private_search, advanced_search],
 middleware=[state_based_tools]
)
```

### js

```
import { createMiddleware } from "langchain";

const stateBasedTools = createMiddleware({
 name: "StateBasedTools",
 wrapModelCall: (request, handler) => {
```



```

// Read from State: check authentication and conversation length
const state = request.state; // [!code highlight]
const isAuthenticated = state.authenticated || false; // [!code highlight]
const messageCount = state.messages.length;

let filteredTools = request.tools;

// Only enable sensitive tools after authentication
if (!isAuthenticated) {
 filteredTools = request.tools.filter(t => t.name.startsWith("public_")); // [!code highlight]
} else if (messageCount < 5) {
 filteredTools = request.tools.filter(t => t.name !== "advanced_search"); // [!code highlight]
}

return handler({ ...request, tools: filteredTools }); // [!code highlight]
},
});

```

## Store

Filter tools based on user preferences or feature flags in Store:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
 user_id: str

@wrap_model_call
def store_based_tools(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Filter tools based on Store preferences."""
 user_id = request.runtime.context.user_id

 # Read from Store: get user's enabled features
 store = request.runtime.store
 feature_flags = store.get(("features",), user_id)

```

```

if feature_flags:
 enabled_features = feature_flags.value.get("enabled_tools", [])
 # Only include tools that are enabled for this user
 tools = [t for t in request.tools if t.name in enabled_features]
 request = request.override(tools=tools)

return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, analysis_tool, export_tool],
 middleware=[store_based_tools],
 context_schema=Context,
 store=InMemoryStore()
)

```

js

```

import * as z from "zod";
import { createMiddleware } from "langchain";

const contextSchema = z.object({
 userId: z.string(),
});

const storeBasedTools = createMiddleware({
 name: "StoreBasedTools",
 contextSchema,
 wrapModelCall: async (request, handler) => {
 const userId = request.runtime.context.userId; // [!code highlight]

 // Read from Store: get user's enabled features
 const store = request.runtime.store; // [!code highlight]
 const featureFlags = await store.get(["features"], userId); // [!code highlight]

 let filteredTools = request.tools;

 if (featureFlags) {
 const enabledFeatures = featureFlags.value?.enabledTools || [];
 filteredTools = request.tools.filter(t => enabledFeatures.includes(t.name)); // [!code highlight]
 }

 return handler({ ...request, tools: filteredTools }); // [!code highlight]
 },
});

```

## Runtime Context

Filter tools based on user permissions from Runtime Context:

## python

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@dataclass
class Context:
 user_role: str

@wrap_model_call
def context_based_tools(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Filter tools based on Runtime Context permissions."""
 # Read from Runtime Context: get user role
 user_role = request.runtime.context.user_role

 if user_role == "admin":
 # Admins get all tools
 pass
 elif user_role == "editor":
 # Editors can't delete
 tools = [t for t in request.tools if t.name != "delete_data"]
 request = request.override(tools=tools)
 else:
 # Viewers get read-only tools
 tools = [t for t in request.tools if t.name.startswith("read_")]
 request = request.override(tools=tools)

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[read_data, write_data, delete_data],
 middleware=[context_based_tools],
 context_schema=Context
)
```

## js

```
import * as z from "zod";
import { createMiddleware } from "langchain";
```

```

const contextSchema = z.object({
 userRole: z.string(),
});

const contextBasedTools = createMiddleware({
 name: "ContextBasedTools",
 contextSchema,
 wrapModelCall: (request, handler) => {
 // Read from Runtime Context: get user role
 const userRole = request.runtime.context.userRole; // [!code highlight]

 let filteredTools = request.tools;

 if (userRole === "admin") {
 // Admins get all tools
 } else if (userRole === "editor") {
 filteredTools = request.tools.filter(t => t.name !== "delete_data"); // [!code highlight]
 } else {
 filteredTools = request.tools.filter(t => t.name.startsWith("read_")); // [!code highlight]
 }

 return handler({ ...request, tools: filteredTools }); // [!code highlight]
 },
});

```

See [Dynamic tools](#) for both filtering pre-registered tools and registering tools at runtime (e.g., from MCP servers).

## Model

Different models have different strengths, costs, and context windows. Select the right model for the task at hand, which might change during an agent run.

## State

Use different models based on conversation length from State:

### python

```

from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.chat_models import init_chat_model
from typing import Callable

Initialize models once outside the middleware
large_model = init_chat_model("claude-sonnet-4-6")

```

```

standard_model = init_chat_model("gpt-5.5")
efficient_model = init_chat_model("gpt-5.4-mini")

@wrap_model_call
def state_based_model(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Select model based on State conversation length."""
 # request.messages is a shortcut for request.state["messages"]
 message_count = len(request.messages) # [!code highlight]

 if message_count > 20:
 # Long conversation - use model with larger context window
 model = large_model
 elif message_count > 10:
 # Medium conversation
 model = standard_model
 else:
 # Short conversation - use efficient model
 model = efficient_model

 request = request.override(model=model) # [!code highlight]

 return handler(request)

agent = create_agent(
 model="gpt-5.4-mini",
 tools=[...],
 middleware=[state_based_model]
)

```

js

```

import { createMiddleware, initChatModel } from "langchain";

// Initialize models once outside the middleware
const largeModel = initChatModel("claude-sonnet-4-6");
const standardModel = initChatModel("gpt-5.5");
const efficientModel = initChatModel("gpt-5.4-mini");

const stateBasedModel = createMiddleware({
 name: "StateBasedModel",
 wrapModelCall: (request, handler) => {
 // request.messages is a shortcut for request.state.messages
 const messageCount = request.messages.length; // [!code highlight]
 let model;

 if (messageCount > 20) {

```

```

 model = largeModel;
} else if (messageCount > 10) {
 model = standardModel;
} else {
 model = efficientModel;
}

 return handler({ ...request, model }); // [!code highlight]
},
});

```

## Store

Use user's preferred model from Store:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.chat_models import init_chat_model
from typing import Callable
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
 user_id: str

Initialize available models once
MODEL_MAP = {
 "gpt-5.5": init_chat_model("gpt-5.5"),
 "gpt-5.4-mini": init_chat_model("gpt-5.4-mini"),
 "claude-sonnet": init_chat_model("claude-sonnet-4-6"),
}

@wrap_model_call
def store_based_model(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Select model based on Store preferences."""
 user_id = request.runtime.context.user_id

Read from Store: get user's preferred model
store = request.runtime.store
user_prefs = store.get(("preferences",), user_id)

if user_prefs:

```

```

 preferred_model = user_prefs.value.get("preferred_model")
 if preferred_model and preferred_model in MODEL_MAP:
 request = request.override(model=MODEL_MAP[preferred_model])

return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[store_based_model],
 context_schema=Context,
 store=InMemoryStore()
)

```

js

```

import * as z from "zod";
import { createMiddleware, initChatModel } from "langchain";

const contextSchema = z.object({
 userId: z.string(),
});

// Initialize available models once
const MODEL_MAP = {
 "gpt-5.5": initChatModel("gpt-5.5"),
 "gpt-5.4-mini": initChatModel("gpt-5.4-mini"),
 "claude-sonnet": initChatModel("claude-sonnet-4-6"),
};

const storeBasedModel = createMiddleware({
 name: "StoreBasedModel",
 contextSchema,
 wrapModelCall: async (request, handler) => {
 const userId = request.runtime.context.userId; // [!code highlight]

 // Read from Store: get user's preferred model
 const store = request.runtime.store; // [!code highlight]
 const userPrefs = await store.get(["preferences"], userId); // [!code highlight]

 let model = request.model;

 if (userPrefs) {
 const preferredModel = userPrefs.value?.preferredModel;
 if (preferredModel && MODEL_MAP[preferredModel]) {
 model = MODEL_MAP[preferredModel]; // [!code highlight]
 }
 }
 }
})

```

```

 return handler({ ...request, model }); // [!code highlight]
 },
});

```

## Runtime Context

Select model based on cost limits or environment from Runtime Context:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.chat_models import init_chat_model
from typing import Callable

@dataclass
class Context:
 cost_tier: str
 environment: str

Initialize models once outside the middleware
premium_model = init_chat_model("claude-sonnet-4-6")
standard_model = init_chat_model("gpt-5.5")
budget_model = init_chat_model("gpt-5.4-mini")

@wrap_model_call
def context_based_model(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Select model based on Runtime Context."""
 # Read from Runtime Context: cost tier and environment
 cost_tier = request.runtime.context.cost_tier
 environment = request.runtime.context.environment

 if environment == "production" and cost_tier == "premium":
 # Production premium users get best model
 model = premium_model
 elif cost_tier == "budget":
 # Budget tier gets efficient model
 model = budget_model
 else:
 # Standard tier
 model = standard_model

 request = request.override(model=model)

```



```

return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[context_based_model],
 context_schema=Context
)

```

js

```

import * as z from "zod";
import { createMiddleware, initChatModel } from "langchain";

const contextSchema = z.object({
 costTier: z.string(),
 environment: z.string(),
});

// Initialize models once outside the middleware
const premiumModel = initChatModel("claude-sonnet-4-6");
const standardModel = initChatModel("gpt-5.5");
const budgetModel = initChatModel("gpt-5.4-mini");

const contextBasedModel = createMiddleware({
 name: "ContextBasedModel",
 contextSchema,
 wrapModelCall: (request, handler) => {
 // Read from Runtime Context: cost tier and environment
 const costTier = request.runtime.context.costTier; // [!code highlight]
 const environment = request.runtime.context.environment; // [!code highlight]

 let model;

 if (environment === "production" && costTier === "premium") {
 model = premiumModel;
 } else if (costTier === "budget") {
 model = budgetModel;
 } else {
 model = standardModel;
 }

 return handler({ ...request, model }); // [!code highlight]
 },
});

```

See [Dynamic model](#) for more examples.

## Response format

Structured output transforms unstructured text into validated, structured data. When extracting specific fields or returning data for downstream systems, free-form text isn't sufficient.

**How it works:** When you provide a schema as the response format, the model's final response is guaranteed to conform to that schema. The agent runs the model / tool calling loop until the model is done calling tools, then the final response is coerced into the provided format.

### Defining formats

Schema definitions guide the model. Field names, types, and descriptions specify exactly what format the output should adhere to.

#### python

```
from pydantic import BaseModel, Field

class CustomerSupportTicket(BaseModel):
 """Structured ticket information extracted from customer message."""

 category: str = Field(
 description="Issue category: 'billing', 'technical', 'account', or 'product'"
)
 priority: str = Field(
 description="Urgency level: 'low', 'medium', 'high', or 'critical'"
)
 summary: str = Field(
 description="One-sentence summary of the customer's issue"
)
 customer_sentiment: str = Field(
 description="Customer's emotional tone: 'frustrated', 'neutral', or 'satisfied'"
)
```

#### js

```
import { z } from "zod";

const customerSupportTicket = z.object({
 category: z.enum(["billing", "technical", "account", "product"]).describe(
 "Issue category"
),
 priority: z.enum(["low", "medium", "high", "critical"]).describe(
```

```

 "Urgency level"
),
 summary: z.string().describe(
 "One-sentence summary of the customer's issue"
),
 customerSentiment: z.enum(["frustrated", "neutral", "satisfied"]).describe(
 "Customer's emotional tone"
),
}).describe("Structured ticket information extracted from customer message");

```

## Selecting formats

Dynamic response format selection adapts schemas based on user preferences, conversation stage, or role—returning simple formats early and detailed formats as complexity increases.

## State

Configure structured output based on conversation state:

### python

```

from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from pydantic import BaseModel, Field
from typing import Callable

class SimpleResponse(BaseModel):
 """Simple response for early conversation."""
 answer: str = Field(description="A brief answer")

class DetailedResponse(BaseModel):
 """Detailed response for established conversation."""
 answer: str = Field(description="A detailed answer")
 reasoning: str = Field(description="Explanation of reasoning")
 confidence: float = Field(description="Confidence score 0-1")

@wrap_model_call
def state_based_output(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Select output format based on State."""
 # request.messages is a shortcut for request.state["messages"]
 message_count = len(request.messages) # [!code highlight]

 if message_count < 3:

```

```

 # Early conversation - use simple format
 request = request.override(response_format=SimpleResponse) # [!code highlight]
else:
 # Established conversation - use detailed format
 request = request.override(response_format=DetailedResponse) # [!code highlight]

return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[state_based_output]
)

```

js

```

import { createMiddleware } from "langchain";
import { z } from "zod";

const simpleResponse = z.object({
 answer: z.string().describe("A brief answer"),
});

const detailedResponse = z.object({
 answer: z.string().describe("A detailed answer"),
 reasoning: z.string().describe("Explanation of reasoning"),
 confidence: z.number().describe("Confidence score 0-1"),
});

const stateBasedOutput = createMiddleware({
 name: "StateBasedOutput",
 wrapModelCall: (request, handler) => {
 // request.state is a shortcut for request.state.messages
 const messageCount = request.messages.length; // [!code highlight]

 let responseFormat;
 if (messageCount < 3) {
 // Early conversation - use simple format
 responseFormat = simpleResponse; // [!code highlight]
 } else {
 // Established conversation - use detailed format
 responseFormat = detailedResponse; // [!code highlight]
 }

 return handler({ ...request, responseFormat });
 },
});

```

## Store

Configure output format based on user preferences in Store:

### python

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from pydantic import BaseModel, Field
from typing import Callable
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
 user_id: str

class VerboseResponse(BaseModel):
 """Verbose response with details."""
 answer: str = Field(description="Detailed answer")
 sources: list[str] = Field(description="Sources used")

class ConciseResponse(BaseModel):
 """Concise response."""
 answer: str = Field(description="Brief answer")

@wrap_model_call
def store_based_output(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Select output format based on Store preferences."""
 user_id = request.runtime.context.user_id

 # Read from Store: get user's preferred response style
 store = request.runtime.store
 user_prefs = store.get(("preferences",), user_id)

 if user_prefs:
 style = user_prefs.value.get("response_style", "concise")
 if style == "verbose":
 request = request.override(response_format=VerboseResponse)
 else:
 request = request.override(response_format=ConciseResponse)

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
```

```

tools=[...],
middleware=[store_based_output],
context_schema=Context,
store=InMemoryStore()
)

```

js

```

import * as z from "zod";
import { createMiddleware } from "langchain";

const contextSchema = z.object({
 userId: z.string(),
});

const verboseResponse = z.object({
 answer: z.string().describe("Detailed answer"),
 sources: z.array(z.string()).describe("Sources used"),
});

const conciseResponse = z.object({
 answer: z.string().describe("Brief answer"),
});

const storeBasedOutput = createMiddleware({
 name: "StoreBasedOutput",
 wrapModelCall: async (request, handler) => {
 const userId = request.runtime.context.userId; // [!code highlight]

 // Read from Store: get user's preferred response style
 const store = request.runtime.store; // [!code highlight]
 const userPrefs = await store.get(["preferences"], userId); // [!code highlight]

 const style = userPrefs?.value?.responseStyle || "concise";
 const responseFormat =
 style === "verbose" ? verboseResponse : conciseResponse; // [!code highlight]

 return handler({
 ...request,
 responseFormat,
 });
 },
});

```

## Runtime Context

Configure output format based on Runtime Context like user role or environment:

## python

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from pydantic import BaseModel, Field
from typing import Callable

@dataclass
class Context:
 user_role: str
 environment: str

class AdminResponse(BaseModel):
 """Response with technical details for admins."""
 answer: str = Field(description="Answer")
 debug_info: dict = Field(description="Debug information")
 system_status: str = Field(description="System status")

class UserResponse(BaseModel):
 """Simple response for regular users."""
 answer: str = Field(description="Answer")

@wrap_model_call
def context_based_output(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Select output format based on Runtime Context."""
 # Read from Runtime Context: user role and environment
 user_role = request.runtime.context.user_role
 environment = request.runtime.context.environment

 if user_role == "admin" and environment == "production":
 # Admins in production get detailed output
 request = request.override(response_format=AdminResponse)
 else:
 # Regular users get simple output
 request = request.override(response_format=UserResponse)

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[context_based_output],
 context_schema=Context
)
```

js

```
import * as z from "zod";
import { createMiddleware } from "langchain";

const contextSchema = z.object({
 userRole: z.string(),
 environment: z.string(),
});

const adminResponse = z.object({
 answer: z.string().describe("Answer"),
 debugInfo: z.record(z.any()).describe("Debug information"),
 systemStatus: z.string().describe("System status"),
});

const userResponse = z.object({
 answer: z.string().describe("Answer"),
});

const contextBasedOutput = createMiddleware({
 name: "ContextBasedOutput",
 wrapModelCall: (request, handler) => {
 // Read from Runtime Context: user role and environment
 const userRole = request.runtime.context.userRole; // [!code highlight]
 const environment = request.runtime.context.environment; // [!code highlight]

 let responseFormat;
 if (userRole === "admin" && environment === "production") {
 responseFormat = adminResponse; // [!code highlight]
 } else {
 responseFormat = userResponse; // [!code highlight]
 }

 return handler({ ...request, responseFormat });
 },
});
```

## Tool context

---

Tools are special in that they both read and write context.

In the most basic case, when a tool executes, it receives the LLM's request parameters and returns a tool message back. The tool does its work and produces a result.

Tools can also fetch important information for the model that allows it to perform and complete tasks.



## Reads

Most real-world tools need more than just the LLM's parameters. They need user IDs for database queries, API keys for external services, or current session state to make decisions. Tools read from state, store, and runtime context to access this information.

### State

Read from State to check current session information:

#### python

```
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@tool
def check_authentication(
 runtime: ToolRuntime
) -> str:
 """Check if user is authenticated."""
 # Read from State: check current auth status
 current_state = runtime.state
 is_authenticated = current_state.get("authenticated", False)

 if is_authenticated:
 return "User is authenticated"
 else:
 return "User is not authenticated"

agent = create_agent(
 model="gpt-5.5",
 tools=[check_authentication]
)
```

#### js

```
import * as z from "zod";
import { createAgent, tool, type ToolRuntime } from "langchain";

const checkAuthentication = tool(
 async (_, runtime: ToolRuntime) => {
 // Read from State: check current auth status
 const currentState = runtime.state;
 const isAuthenticated = currentState.authenticated || false;

 if (isAuthenticated) {
 return "User is authenticated";
 } else {
```

```

 return "User is not authenticated";
 }
},
{
 name: "check_authentication",
 description: "Check if user is authenticated",
 schema: z.object({}),
}
);

```

## Store

Read from Store to access persisted user preferences:

### python

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
 user_id: str

@tool
def get_preference(
 preference_key: str,
 runtime: ToolRuntime[Context]
) -> str:
 """Get user preference from Store."""
 user_id = runtime.context.user_id

 # Read from Store: get existing preferences
 store = runtime.store
 existing_prefs = store.get(("preferences",), user_id)

 if existing_prefs:
 value = existing_prefs.value.get(preference_key)
 return f"{preference_key}: {value}" if value else f"No preference set for {preference_key}"
 else:
 return "No preferences found"

agent = create_agent(
 model="gpt-5.5",
 tools=[get_preference],
 context_schema=Context,

```

```
store=InMemoryStore()
)
```

js

```
import * as z from "zod";
import { createAgent, tool, type ToolRuntime } from "langchain";

const contextSchema = z.object({
 userId: z.string(),
});

const getPreference = tool(
 async ({ preferenceKey }, runtime: ToolRuntime) => {
 const userId = runtime.context.userId;

 // Read from Store: get existing preferences
 const store = runtime.store;
 const existingPrefs = await store.get(["preferences"], userId);

 if (existingPrefs) {
 const value = existingPrefs.value?.[preferenceKey];
 return value ? `${preferenceKey}: ${value}` : `No preference set for ${preferenceKey}`;
 } else {
 return "No preferences found";
 }
 },
 {
 name: "get_preference",
 description: "Get user preference from Store",
 schema: z.object({
 preferenceKey: z.string(),
 }),
 }
);
```

## Runtime Context

Read from Runtime Context for configuration like API keys and user IDs:

python

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
```

```

class Context:
 user_id: str
 api_key: str
 db_connection: str

@tool
def fetch_user_data(
 query: str,
 runtime: ToolRuntime[Context]
) -> str:
 """Fetch data using Runtime Context configuration."""
 # Read from Runtime Context: get API key and DB connection
 user_id = runtime.context.user_id
 api_key = runtime.context.api_key
 db_connection = runtime.context.db_connection

 # Use configuration to fetch data
 results = perform_database_query(db_connection, query, api_key)

 return f"Found {len(results)} results for user {user_id}"

agent = create_agent(
 model="gpt-5.5",
 tools=[fetch_user_data],
 context_schema=Context
)

Invoke with runtime context
result = agent.invoke(
 {"messages": [{"role": "user", "content": "Get my data"}]},
 context=Context(
 user_id="user_123",
 api_key="sk-...",
 db_connection="postgresql://..."
)
)

```

js

```

import * as z from "zod";
import { tool } from "@langchain/core/tools";
import { createAgent } from "langchain";

const contextSchema = z.object({
 userId: z.string(),
 apiKey: z.string(),
 dbConnection: z.string(),
});

```

```

const fetchUserData = tool(
 async ({ query }, runtime: ToolRuntime<any, typeof contextSchema>) => {
 // Read from Runtime Context: get API key and DB connection
 const { userId, apiKey, dbConnection } = runtime.context;

 // Use configuration to fetch data
 const results = await performDatabaseQuery(dbConnection, query, apiKey);

 return `Found ${results.length} results for user ${userId}`;
 },
 {
 name: "fetch_user_data",
 description: "Fetch data using Runtime Context configuration",
 schema: z.object({
 query: z.string(),
 }),
 }
);

const agent = createAgent({
 model: "gpt-5.5",
 tools: [fetchUserData],
 contextSchema,
});

```

## Writes

Tool results can be used to help an agent complete a given task. Tools can both return results directly to the model and update the memory of the agent to make important context available to future steps.

## State

Write to State to track session-specific information using Command:

### python

```

from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent
from langgraph.types import Command

@tool
def authenticate_user(
 password: str,
 runtime: ToolRuntime
) -> Command:
 """Authenticate user and update State."""
 # Perform authentication (simplified)

```

```

 if password == "correct":
 # Write to State: mark as authenticated using Command
 return Command(
 update={"authenticated": True},
)
 else:
 return Command(update={"authenticated": False})

agent = create_agent(
 model="gpt-5.5",
 tools=[authenticate_user]
)

```

js

```

import * as z from "zod";
import { tool } from "@langchain/core/tools";
import { createAgent } from "langchain";
import { Command } from "@langchain/langgraph";

const authenticateUser = tool(
 async ({ password }) => {
 // Perform authentication
 if (password === "correct") {
 // Write to State: mark as authenticated using Command
 return new Command({
 update: { authenticated: true },
 });
 } else {
 return new Command({ update: { authenticated: false } });
 }
 },
 {
 name: "authenticate_user",
 description: "Authenticate user and update State",
 schema: z.object({
 password: z.string(),
 }),
 }
);

```

## Store

Write to Store to persist data across sessions:

python

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
 user_id: str

@tool
def save_preference(
 preference_key: str,
 preference_value: str,
 runtime: ToolRuntime[Context]
) -> str:
 """Save user preference to Store."""
 user_id = runtime.context.user_id

 # Read existing preferences
 store = runtime.store
 existing_prefs = store.get(("preferences",), user_id)

 # Merge with new preference
 prefs = existing_prefs.value if existing_prefs else {}
 prefs[preference_key] = preference_value

 # Write to Store: save updated preferences
 store.put(("preferences",), user_id, prefs)

 return f"Saved preference: {preference_key} = {preference_value}"

agent = create_agent(
 model="gpt-5.5",
 tools=[save_preference],
 context_schema=Context,
 store=InMemoryStore()
)

```

js

```

import * as z from "zod";
import { createAgent, tool, type ToolRuntime } from "langchain";

const savePreference = tool(
 async ({ preferenceKey, preferenceValue }, runtime: ToolRuntime<any, typeof contextSchema>) => {
 const userId = runtime.context.userId;

 // Read existing preferences

```

```

const store = runtime.store;
const existingPrefs = await store.get(["preferences"], userId);

// Merge with new preference
const prefs = existingPrefs?.value || {};
prefs[preferenceKey] = preferenceValue;

// Write to Store: save updated preferences
await store.put(["preferences"], userId, prefs);

 return `Saved preference: ${preferenceKey} = ${preferenceValue}`;
 },
 {
 name: "save_preference",
 description: "Save user preference to Store",
 schema: z.object({
 preferenceKey: z.string(),
 preferenceValue: z.string(),
 }),
 }
);

```

See [Tools](#) for comprehensive examples of accessing state, store, and runtime context in tools.

## Life-cycle context

Control what happens **between** the core agent steps - intercepting data flow to implement cross-cutting concerns like summarization, guardrails, and logging.

As you've seen in [Model Context](#) and [Tool Context](#), [middleware](#) is the mechanism that makes context engineering practical. Middleware allows you to hook into any step in the agent lifecycle and either:

1. **Update context** - Modify state and store to persist changes, update conversation history, or save insights
2. **Jump in the lifecycle** - Move to different steps in the agent cycle based on context (e.g., skip tool execution if a condition is met, repeat model call with modified context)



Middleware hooks in the agent loop

### Example: Summarization

One of the most common life-cycle patterns is automatically condensing conversation history when it gets too long. Unlike the transient message trimming shown in [Model](#)



[Context](#), summarization **persistently updates state** - permanently replacing old messages with a summary that's saved for all future turns.

LangChain offers built-in middleware for this:

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger={"tokens": 4000},
 keep=("messages", 20),
),
],
)
```

### js

```
import { createAgent, summarizationMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 middleware: [
 summarizationMiddleware({
 model: "gpt-5.4-mini",
 trigger: { tokens: 4000 },
 keep: { messages: 20 },
 }),
],
});
```

When the conversation exceeds the token limit, `SummarizationMiddleware` automatically: 1. Summarizes older messages using a separate LLM call 2. Replaces them with a summary message in State (permanently) 3. Keeps recent messages intact for context

The summarized conversation history is permanently updated - future turns will see the summary instead of the original messages.

### 참고

For a complete list of built-in middleware, available hooks, and how to create custom middleware, see the [Middleware documentation](#).

## Best practices

---

1. **Start simple** - Begin with static prompts and tools, add dynamics only when needed
2. **Test incrementally** - Add one context engineering feature at a time
3. **Monitor performance** - Track model calls, token usage, and latency
4. **Use built-in middleware** - Leverage [SummarizationMiddleware](#) , [LLMToolSelectorMiddleware](#) , etc.
5. **Document your context strategy** - Make it clear what context is being passed and why
6. **Understand transient vs persistent**: Model context changes are transient (per-call), while life-cycle context changes persist to state

## Related resources

---

- [Context conceptual overview](#) - Understand context types and when to use them
- [Middleware](#) - Complete middleware guide
- [Tools](#) - Tool creation and context access
- [Memory](#) - Short-term and long-term memory patterns
- [Agents](#) - Core agent concepts

# Build a data analysis agent from scratch

langchain/deep-agent-from-scratch.mdx

This guide builds a data analysis agent from first principles using `create_agent` and Deep Agents middleware.

Both `create_agent` and `create_deep_agent` provide you with fine-grained control over tools, memory, and more. The main difference between both is that Deep Agents comes with a range of commonly useful capabilities already built in, such as planning, file system tools, and subagents. If the Deep Agents default harness does not fit your needs, this guide shows you how to start with `create_agent` and assemble the harness one piece at a time, so you can see exactly what each component adds and swap in only what your use case needs.

Follow this guide to build an agent that:

1. Accepts a CSV file for analysis
2. Writes and executes Python code in an isolated sandbox
3. Delegates visualization work to a specialized subagent
4. Loads data analysis patterns from a skills file

The final stack mirrors what `create_deep_agent` assembles by default.

## What you will learn

Each step adds one capability to the same data analysis agent:

| Step                 | Problem without it                     | What you add                                                   |
|----------------------|----------------------------------------|----------------------------------------------------------------|
| Minimal agent        | —                                      | Baseline loop: model + tools, no harness                       |
| Sandbox + filesystem | Agent cannot read CSVs or run Python   | Isolated <a href="#">backend</a> + file and execute tools      |
| Summarization        | Long sessions hit context limits       | Automatic history compression                                  |
| Skills               | Domain rules bloat the system prompt   | On-demand expertise via <a href="#">progressive disclosure</a> |
| Subagent             | Chart iteration crowds the main thread | Isolated worker + parallel delegation                          |

## Setup

### Install packages

Install the packages for this tutorial:

## python

```
pip install deepagents langsmith
```

## js

```
npm install deepagents langsmith
```

### Set up LangSmith API keys

This tutorial uses @[ LangSmithSandbox ], which provisions sandboxes through SandboxClient . That client authenticates with LangSmith using LANGSMITH\_API\_KEY from your environment, so an API key is required to run the tutorial. Setting up LangSmith also allows you to see traces of what happens when your agent runs.

1. [Sign up for a free account](#). You can use Google, GitHub, or email.
2. [Create an API key](#) in **Settings > API Keys**.
3. Export the LangSmith API key:

```
export LANGSMITH_API_KEY=...
```

4. Enable tracing to inspect tool calls, middleware steps, and subagent delegation as you add each piece:

```
export LANGSMITH_TRACING=true
```

### Add a model provider API key

Export the API key for the model provider you use in the code samples:

```
```bash Google export GOOGLE_API_KEY=...
```

```
```bash OpenAI
export OPENAI_API_KEY=...
```

```
```bash Anthropic export ANTHROPIC_API_KEY=...
```

```
```bash OpenRouter
export OPENROUTER_API_KEY=...
```

```
```bash Fireworks export FIREWORKS_API_KEY=...
```

```
```bash Baseten
export BASETEN_API_KEY=...
```

```
```bash Ollama
```

Local: Ollama must be running on your machine

Cloud: Set your Ollama API key for hosted inference

```
export OLLAMA_API_KEY=...
```

```
## Build the agent
```

```
## Create the minimal agent
```

A data analysis agent needs more than a chat loop, but to begin with, start with the baseline: only a model and a loop.

Use `@[`create_agent`]` and specify the model that you want to use:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
</div>
```

This runs, but the agent has no filesystem and no way to execute code. If you ask it to analyze a CSV, it can only guess from the prompt. The next steps add real file access and code execution.

```
## Add a sandbox backend
```

To analyze data efficiently, the agent needs to run code on files. This requires two things:

- An isolated [sandbox](/oss/deepagents/sandboxes) where the agent can place files and run code on the files without giving the agent access to your host machine.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

- A [backend](/oss/deepagents/backends) which provides the file system tools to work with the sandbox (`read\_file`, `write\_file`, `edit\_file`, `delete`, `glob`, `grep`) using the `@[`FilesystemMiddleware`]`. Because the `LangSmithSandbox` backend implements the sandbox protocol, `@[`FilesystemMiddleware`]` also adds the `execute` tool, which allows the agent to run shell commands.

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

- A [backend](/oss/deepagents/backends) which provides the file system tools to work with the sandbox (`read\_file`, `write\_file`, `edit\_file`, `glob`, `grep`) using the `@[`FilesystemMiddleware`]`. Because the `LangSmithSandbox` backend implements the sandbox protocol, `@[`FilesystemMiddleware`]` also adds the `execute` tool, which allows the agent to run shell commands.

```
</div>
```

@[`LangSmithSandbox`] is where files live and commands run. @[`FilesystemMiddleware`] is what exposes that environment to the model as tools. The same middleware works with other backends if you swap the backend later.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

@[`LangSmithSandbox`] gives the agent an isolated environment with a filesystem and an `execute` tool for running shell commands. With it, the agent can install packages, write scripts, and run them without touching the host. To boot from a custom image instead of the default runtime, pass `snapshot\_name` or `snapshot\_id` to `create\_sandbox`; see [Sandbox snapshots](/langsmith/sandbox-snapshots).

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

@[`LangSmithSandbox`] gives the agent an isolated environment with a filesystem and an `execute` tool for running shell commands. With it, the agent can install packages, write scripts, and run them without touching the host. To boot from a custom image instead

of the default runtime, pass `snapshotId` to `LangSmithSandbox.create()`; see [Sandbox snapshots](/langsmith/sandbox-snapshots).

</div>

Replace the agent from the previous step with one that includes @[`FilesystemMiddleware`]:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

</div>

The sandbox filesystem is separate from your laptop. You must upload the files you need to it before you invoke the agent:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

</div>

<div class="callout note" markdown="1">

<p class="callout-label">참고</p>

With @[`LangSmithSandbox`], upload paths must be absolute POSIX paths (for example, `/sales.csv`). Relative paths such as `sales.csv` are rejected with `invalid\_path` and the file is not written to the sandbox.

</div>

Combine the code from the previous steps into one script and run it:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

```
```bash
python analyze_sales.py
```

js

```
npx tsx analyze-sales.ts
```

On the first run, LangSmith provisions a sandbox (this can take a few seconds). The script uploads `sales.csv`, streams the agent run, and prints assistant messages as they arrive. You should see an analysis of the sample sales data: product-level revenue, which widgets sold most, and brief trend notes. Exact wording varies by model run.

Open the run in [LangSmith](#) and watch the agent use filesystem tools (`read_file`, and `execute` if it runs Python in the sandbox) before it replies.

## Add context management

---

After step 2, every tool result stays in the message history. A real analysis session (multiple plots, failed scripts, large `read_file` output) fills the context window quickly.

`@[SummarizationMiddleware]` compresses older turns when history grows too large, so the agent keeps working without you manually trimming messages. This matters less on the first `sales.csv` question and more on follow-ups such as "Now segment by product and plot monthly trends."

Update your agent from step 2 by adding `@[SummarizationMiddleware]` to the middleware list:

python

js

Run a multi-turn session to see summarization in action. After the initial analysis, ask follow-up questions that trigger more file reads or script runs. In LangSmith, look for a summarization step before later model calls. For more information, [Context engineering](#).

## Add skills

---

[Skills](#) provide a way to give an agent on-demand domain knowledge when needed using progressive disclosure. Skills can include multi-step workflows, rules, and conventions. By placing this information in a skill, it isn't added to the system prompt by default which ensures the tokens are only used when the information from the skill is needed for a task.



When the agent starts, it sees only lightweight metadata about each skill. When a task needs a skill, the agent loads the full skill file on demand.

Create a skill file in a skills directory:

```
skills/
 pandas-patterns/
 SKILL.md
```

```

name: pandas-patterns
description: Common pandas and matplotlib patterns for data analysis and visualization

Data loading
Use `pd.read_csv()` for CSV files. Always check `df.info()` and `df.describe()` first.

Visualization
Use `matplotlib` for bar charts, `seaborn` for statistical plots.
Save figures with `plt.savefig("output.png", dpi=150, bbox_inches="tight")`.

Reporting
Write a markdown summary to `report.md` alongside any generated charts.
```

This skill contains information on how the visualization should be done.

With `@[LangSmithSandbox]`, skill paths resolve on the sandbox filesystem, not your local machine. Upload your local `skills/` directory before configuring `@[SkillsMiddleware]`:

**python**

**js**

Then create your agent with your skills by adding `@[SkillsMiddleware]`:

**python**

**js**

You can try a prompt such as "Analyze sales.csv using our pandas patterns." The agent will load the skill when it needs plotting or reporting guidance. If you ask a different question that does not need the skill, the agent will not load it.

## Add a visualization subagent

Some tasks produce large intermediate output (script drafts, failed runs, file reads) that would crowd the main agent's context if kept in one thread. A `subagent` runs in its own context window so the supervisor sees only the final result, not every tool call along the way. That keeps the main analysis focused and leaves room for follow-up questions.

One example where using a subagent makes sense is chart generation. Plotting often means iterating on Python scripts, installing packages, and reading error output before a figure is ready. The following `visualizer` subagent can handle that work in isolation while the main agent continues planning and analysis. With `@[ TodoListMiddleware ]`, the main agent can also delegate that chart work in parallel instead of blocking on each plot.

Update your agent from step 4 by adding `@[ TodoListMiddleware ]` and `@[ SubAgentMiddleware ]`:

```
python
js
```

Try a prompt such as "Analyze sales.csv, then create a bar chart of revenue by product." The main agent handles analysis and planning and delegates chart generation to the `visualizer` subagent via the `task` tool.

If you enabled tracing in [Setup](#), open the run in [LangSmith](#). You should see a `task` call to `visualizer`, a separate sub-run with its own tool loop, and a short result returned to the supervisor.

## What you built

You've built a customized agent with the following middleware:

Middleware	What it adds
<code>@[ FilesystemMiddleware ] + LangSmithSandbox</code>	Isolated filesystem + <code>execute</code> tool
<code>@[ SummarizationMiddleware ]</code>	Automatic context compression
<code>@[ SkillsMiddleware ]</code>	Domain knowledge loaded on demand
<code>@[ TodoListMiddleware ] + @[ SubAgentMiddleware ]</code>	Parallel visualization subagent

```
python
```

This is the same foundation as `@[ create_deep_agent ]`: assembled manually so you control exactly what's included.

**js**

This is the same foundation as `@[ createDeepAgent ]`: assembled manually so you control exactly what's included.

The possibilities don't end here: see [Prebuilt middleware](#) for the full list of composable capabilities, and the `@[ create_agent ]` reference for all configuration options.

**python**

To work with the pre-assembled version, see [Customize Deep Agents](#). For the full data analysis example using `create_deep_agent`, see [Data analysis](#).

**js**

To work with the pre-assembled version, see [Customize Deep Agents](#). For the full data analysis example using `createDeepAgent`, see [Data analysis](#).

# Deployment

---

langchain/deploy.mdx

When you are ready to deploy your LangChain agent to production, choose a hosting model that fits your stack. [LangSmith Cloud](#) provides fully managed infrastructure for stateful, long-running agents with persistent state and background execution.

## js

You can also deploy on **JavaScript frameworks and platforms** such as Next.js, SvelteKit, Nuxt, Cloudflare Workers, and Deno Deploy using the same [Agent Streaming Protocol](#).

## 팁

LangSmith offers multiple deployment options beyond Cloud, including [hybrid](#), [standalone servers](#), and [self-hosted with control plane](#). For more information, see the [LangSmith Deployment overview](#).

## LangSmith Cloud

---

This section walks through deploying your agent to LangSmith Cloud from a GitHub repository. LangSmith handles infrastructure, scaling, and operational concerns.

### Prerequisites

Before you begin, ensure you have the following:

- A [GitHub account](#)
- A [LangSmith account](#) (free to sign up)

### Deploy your agent

#### 1. Create a repository on GitHub

Your application's code must reside in a GitHub repository to be deployed on LangSmith. Both public and private repositories are supported. For this quickstart, first make sure your app is LangGraph-compatible by following the [local server setup guide](#). Then, push your code to the repository.

## python

js

# Event streaming

langchain/event-streaming.mdx

LangChain agents are built on LangGraph, so they support the same streaming stack with agent-focused projections for messages, tool calls, state, and custom updates.

For most application and frontend use cases, use **Event Streaming** through `stream_events(..., version="v3")`. Event Streaming returns a run object with typed projections, so each projection can be consumed independently instead of parsing stream-mode tuples.

## python

```
from langchain.agents import create_agent

def get_weather(city: str) -> str:
 """Get weather for a city."""
 return f"It's always sunny in {city}!"

agent = create_agent(
 model="gpt-5-nano",
 tools=[get_weather],
)

stream = agent.stream_events({
 "messages": [{"role": "user", "content": "What is the weather in SF?"}],
}, version="v3")

for message in stream.messages:
 for delta in message.text:
 print(delta, end="", flush=True)

final_state = stream.output
```

## js

```
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
 async ({ city }) => `It's always sunny in ${city}!`,
 {
 name: "get_weather",
 description: "Get weather for a city.",
 schema: z.object({ city: z.string() }),
 }
)
```

```

);

const agent = createAgent({
 model: "gpt-5-nano",
 tools: [getWeather],
});

const stream = await agent.streamEvents(
 { messages: [{ role: "user", content: "What is the weather in SF?" }] },
 { version: "v3" }
);

for await (const message of stream.messages) {
 for await (const delta of message.text) {
 process.stdout.write(delta);
 }
}

const finalState = await stream.output;

```

## What you can stream

### python

Projection	Use
for event in stream	Raw protocol events with full envelope and access to every channel.
stream.messages	Model message streams, one per LLM call.
message.text	Text deltas and final text for a message.
message.reasoning	Reasoning deltas for models that expose reasoning content.
message.tool_calls	Tool-call argument chunks and finalized tool calls.
message.output	Final message object after the model call completes.
stream.values	Agent state snapshots.
stream.output	Final agent state.
stream.subgraphs	Nested graph runs (sub-agents and plain subgraphs).
stream.extensions	Custom transformer projections.
stream.tool_calls	Tool execution lifecycle, inputs, output deltas, final output, and errors.

### js

Projection	Use
for event in stream	Raw protocol events with full envelope and access to every channel.

Projection	Use
<code>stream.messages</code>	Model message streams, one per LLM call.
<code>message.text</code>	Text deltas and final text for a message.
<code>message.reasoning</code>	Reasoning deltas for models that expose reasoning content.
<code>message.toolCalls</code>	Tool-call argument chunks and finalized tool calls.
<code>message.output</code>	Final message object after the model call completes.
<code>message.usage</code>	Token usage metadata when the provider returns it.
<code>stream.values</code>	Agent state snapshots.
<code>stream.output</code>	Final agent state.
<code>stream.subgraphs</code>	Nested graph runs (sub-agents and plain subgraphs).
<code>stream.extensions</code>	Custom transformer projections.
<code>stream.toolCalls</code>	Tool execution lifecycle, inputs, output deltas, final output, and errors.

## python

`stream.messages` yields `ChatModelStream` objects. Each message stream exposes `.text`, `.reasoning`, `.tool_calls`, and `.output`. Sync projections are iterable for live deltas and drainable for final values: use `str(message.text)` for final text and `message.tool_calls.get()` for finalized tool calls.

## js

`stream.messages` yields message streams. Each message stream exposes `.text`, `.reasoning`, `.toolCalls`, `.output`, and `.usage`. Async projections can be iterated for live deltas or awaited for final values.

## Agent messages

Use `stream.messages` when you want model output from each LLM call.

## python

```
stream = agent.stream_events(input, version="v3")

for message in stream.messages:
 print(f"[{message.node}] ", end="")
 for delta in message.text:
 print(delta, end="", flush=True)
```



```

full_message = message.output
usage = full_message.usage_metadata
if usage:
 print(usage)

```

js

```

const stream = await agent.streamEvents(input, { version: "v3" });

for await (const message of stream.messages) {
 process.stdout.write(`[${message.node}] `);
 for await (const delta of message.text) {
 process.stdout.write(delta);
 }

 const fullMessage = await message.output;
 console.log(fullMessage.content);

 const usage = await message.usage;
 if (usage) {
 console.log(usage);
 }
}

```

`message.output` gives you the finalized AI message, including provider-specific content blocks. In TypeScript, use `message.usage` when you only need token counts or other usage metadata; in Python, read usage from `message.output.usage_metadata`.

## Reasoning content

Reasoning content uses the same shape as text content, but it is available only when the selected model emits reasoning blocks.

python

```

stream = agent.stream_events(input, version="v3")

for message in stream.messages:
 for delta in message.reasoning:
 print(f"[thinking] {delta}", end="", flush=True)

 for delta in message.text:
 print(delta, end="", flush=True)

```

js

```
const stream = await agent.streamEvents(input, { version: "v3" });

for await (const message of stream.messages) {
 for await (const delta of message.reasoning) {
 process.stdout.write(`[thinking] ${delta}`);
 }

 for await (const delta of message.text) {
 process.stdout.write(delta);
 }
}
```

See the [reasoning guide](#) and your provider's integration page for model configuration details.

## Tool calls

There are two useful tool-call projections:

- `message.tool_calls` streams tool-call argument chunks while the model is producing the tool call.
- `stream.tool_calls` streams the lifecycle of tool execution after the tool call starts.

### python

```
stream = agent.stream_events(input, version="v3")

for message in stream.messages:
 for chunk in message.tool_calls:
 print(f"tool call chunk: {chunk}")

 finalized = message.tool_calls.get()
 if finalized:
 print(f"finalized tool calls: {finalized}")

 for call in stream.tool_calls:
 print(f"{call.tool_name}({call.input})")
 for delta in call.output_deltas:
 print(delta, end="", flush=True)
 print(call.output, call.error)
```

### js

```
const stream = await agent.streamEvents(input, { version: "v3" });

await Promise.all([
 (async () => {
```

```

 for await (const message of stream.messages) {
 for await (const chunk of message.toolCalls) {
 console.log("tool call chunk", chunk);
 }
 }
 })(),
 (async () => {
 for await (const call of stream.toolCalls) {
 console.log(call.name, call.input);
 console.log(await call.output, await call.error);
 }
 })(),
]);

```

## Streaming sub-agents

### python

When a `create_agent` call invokes another named `create_agent` (via a wrapping tool, typically), the inner agent's events flow at a nested namespace. The `name=` you pass to `create_agent` identifies that inner agent in the stream, so you can filter and label per agent.

Named sub-agents surface on the dedicated `stream.subagents` projection. Each handle exposes the inner agent's own `.messages`, `.values`, `.tool_calls`, and `.output`, plus `.name` (the `name=` you passed) and `.cause` (the tool call that dispatched the sub-agent). Because only named `create_agent` runs appear here, you don't need to filter plain subgraphs out.

```

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

def get_weather(city: str) -> str:
 """Get weather for a given city."""
 return f"It's always sunny in {city}!"

weather_agent = create_agent(
 model=init_chat_model("openai:gpt-5.5"),
 tools=[get_weather],
 name="weather_agent",
)

def call_weather(query: str) -> str:
 """Query the weather agent."""
 result = weather_agent.invoke({"messages": [{"role": "user", "content": query}]})
 return result["messages"][-1].text

supervisor = create_agent(
 model=init_chat_model("openai:gpt-5.5"),

```

```

 tools=[call_weather],
 name="supervisor",
)

stream = supervisor.stream_events(
 {"messages": [{"role": "user", "content": "What's the weather in Boston?"}]},
 version="v3",
)

for subagent in stream.subagents:
 print(f"{subagent.name}: ", end="")
 for message in subagent.messages:
 for token in message.text:
 print(token, end="", flush=True)
 print()

```

## js

When a `createAgent` call invokes another named `createAgent` (via a wrapping tool, typically), the inner agent's events flow at a nested namespace. The `name` you pass to `createAgent` identifies that inner agent in the stream, so you can filter and label per agent.

Named sub-agents surface on the dedicated `stream.subagents` projection. Each handle exposes the inner agent's own `.messages`, `.toolCalls`, and `.output`, plus `.name` (the `name=` you passed), `.cause` (the tool call that dispatched the sub-agent), and nested `.subagents`. Because only named `createAgent` runs appear here, you don't need to filter plain subgraphs out.

```

import { createAgent, tool } from "langchain";
import { z } from "zod";

const getWeather = tool(
 async ({ city }) => `It's always sunny in ${city}!`,
 { name: "get_weather", schema: z.object({ city: z.string() }) }
);

const weatherAgent = createAgent({
 model: "openai:gpt-5.5",
 tools: [getWeather],
 name: "weather_agent",
});

const callWeather = tool(
 async ({ query }) => {
 const result = await weatherAgent.invoke({
 messages: [{ role: "user", content: query }],
 });
 return result.messages.at(-1)?.text ?? "";
 }
);

```

```

 },
 { name: "call_weather", schema: z.object({ query: z.string() }) }
);

 const supervisor = createAgent({
 model: "openai:gpt-5.5",
 tools: [callWeather],
 name: "supervisor",
 });

 const stream = await supervisor.streamEvents(
 { messages: [{ role: "user", content: "What's the weather in Boston?" }] },
 { version: "v3" }
);

 for await (const subagent of stream.subagents) {
 process.stdout.write(`${subagent.name}: `);
 for await (const message of subagent.messages) {
 for await (const token of message.text) {
 process.stdout.write(token);
 }
 }
 process.stdout.write("\n");
 }
 //Output: "weather_agent: The weather in Boston is sunny!"

```

Plain `StateGraph` subgraphs invoked from a tool also surface on `stream.subgraphs` — set `name=` on `.compile(name=...)` to get a label in `subagent.graph_name`.

## python

`stream.subagents` is the focused view of named `create_agent` sub-agents, while `stream.subgraphs` covers every nested graph. Use whichever matches your UI.

## js

`stream.subagents` is the focused view of named `createAgent` sub-agents, while `stream.subgraphs` covers every nested graph. Use whichever matches your UI.

## State and final output

Use `stream.values` for state snapshots and `stream.output` for the final agent state.

## python

```

stream = agent.stream_events(input, version="v3")

for snapshot in stream.values:
 print(snapshot)

final_state = stream.output

```

**js**

```

const stream = await agent.streamEvents(input, { version: "v3" });

for await (const snapshot of stream.values) {
 console.log(snapshot);
}

const finalState = await stream.output;

```

## Multiple projections

**python**

For concurrent consumption in async code, use `astream_events` with `asyncio.gather`:

```

import asyncio

stream = await agent.astream_events(input, version="v3")

async def consume_messages():
 async for message in stream.messages:
 print(await message.text)

async def consume_tool_calls():
 async for call in stream.tool_calls:
 print(call.tool_name, call.input)

await asyncio.gather(consume_messages(), consume_tool_calls())

```

For synchronous code, use `stream.interleave(...)` instead:

```

stream = agent.stream_events(input, version="v3")

for name, item in stream.interleave("messages", "tool_calls", "values"):
 if name == "messages":
 print(item.text)
 elif name == "tool_calls":

```

```
 print(item.tool_name, item.input)
elif name == "values":
 print(item)
```

**js**

Use concurrent consumers when you want multiple projections in JavaScript:

```
const stream = await agent.streamEvents(input, { version: "v3" });

await Promise.all([
 (async () => {
 for await (const message of stream.messages) {
 console.log(await message.text);
 }
 })(),
 (async () => {
 for await (const call of stream.toolCalls) {
 console.log(call.name, call.input);
 }
 })(),
]);
```

To access channels that aren't exposed as typed projections, or to inspect the full event envelope, iterate raw protocol events:

**python**

```
for event in stream:
 print(event["method"], event["params"]["namespace"], event["params"]["data"])
```

**js**

```
for await (const event of stream) {
 console.log(event.method, event.params.namespace, event.params.data);
}
```

## Custom updates

Use custom stream transformers when your application needs a projection that is not built in, such as retrieval progress, artifacts, or domain-specific events.

**python**

```

stream = agent.stream_events(
 input,
 version="v3",
 transformers=[ToolActivityTransformer],
)

for activity in stream.extensions["tool_activity"]:
 print(activity)

```

js

```

const stream = await agent.streamEvents(input, {
 version: "v3",
 transformers: [toolActivityTransformer],
});

for await (const activity of stream.extensions.toolActivity) {
 console.log(activity);
}

```

## Register transformers on middleware

python

참고

Middleware-registered transformers require `langchain>=1.3.2`.

js

참고

Middleware-registered transformers require `langchain@1.4.3` or later.

Middleware can declare stream transformer factories alongside its hooks and tools. The factory shape differs between languages:

python

Set the `transformers` attribute on an `AgentMiddleware` subclass to a sequence of factories. Each factory has the shape `Callable[[tuple[str, ...]], StreamTransformer]` and is invoked



as `factory(scope)`, where `scope` is the mini-mux scope tuple `()` for the root mux, non-empty for subgraphs). Returning a fresh transformer per call keeps each subgraph isolated.

```
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware

class ToolActivityMiddleware(AgentMiddleware):
 transformers = (ToolActivityTransformer,)

agent = create_agent(
 model="gpt-5-nano",
 tools=[get_weather],
 middleware=[ToolActivityMiddleware()],
)
```

## js

Pass `streamTransformers` to `createMiddleware` as a tuple of factories. Each factory has the shape `() => StreamTransformer<any>` (zero arguments) and is invoked once per scope. Returning a fresh transformer per call keeps each subgraph isolated.

```
import { createAgent, createMiddleware } from "langchain";

const toolActivityMiddleware = createMiddleware({
 name: "ToolActivityMiddleware",
 streamTransformers: [toolActivityTransformer],
});

const agent = createAgent({
 model: "gpt-5-nano",
 tools: [getWeather],
 middleware: [toolActivityMiddleware],
});
```

## python

At compile time, `create_agent` merges middleware-registered factories with anything passed to its own `transformers=` argument. The final order on the compiled graph is:

1. The built-in `ToolCallTransformer`.
2. Middleware-registered factories, in middleware order.
3. Caller-supplied `transformers=` from `create_agent`.

This keeps the built-in tool-call projection in front of consumer transformers and gives caller-supplied entries the final word.

The built-in `PIIMiddleware` uses this hook to redact PII from streamed wire output. With `apply_to_output=True`, its registered transformer scrubs detected PII from text deltas, tool-call args, tool outputs, and state snapshots before they leave the run, closing the window where `after_model` state-level redaction would otherwise let raw PII through to live readers of `stream_events(version="v3")`.

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
 model="gpt-5-nano",
 tools=[],
 middleware=[
 PIIMiddleware("email", strategy="redact", apply_to_output=True),
],
)
```

See [PII detection](#) for the full configuration surface.

## js

At compile time, `createAgent` merges middleware-registered factories with anything passed to its own `streamTransformers` option. The final order on the compiled graph is:

1. The built-in `ToolCallTransformer`.
2. Middleware-registered factories, in middleware order.
3. Caller-supplied `streamTransformers` from `createAgent`.

This keeps the built-in tool-call projection in front of consumer transformers and gives caller-supplied entries the final word.

See [Build your own projection](#) for the transformer contract.

## Related

- [Streaming](#) covers low-level Pregel stream modes.
- [Build your own projection](#) covers writing application-specific projections.
- [Frontend streaming patterns](#) shows UI use cases built on streamed state.

# Get help

---

langchain/get-help.mdx

Connect with the LangChain community, access learning resources, and get the support you need to build with confidence.

## Learning resources

---

Start your journey or deepen your knowledge with our comprehensive learning materials.

- [Chat LangChain](#): Ask the docs anything about LangChain, powered by real-time docs
- [API Reference](#): Complete documentation for all LangChain packages

## Community support

---

Get help from fellow developers and the LangChain team through our active community channels.

- [Community Forum](#): Ask questions, share solutions, and discuss best practices
- [Community Slack](#): Connect with other builders and get quick help

## Professional support

---

For enterprise needs and critical applications, access dedicated support channels.

- [Support portal](#): Submit tickets and track support requests
- [LangSmith status](#): Real-time status of LangSmith services and APIs

## Contribute

---

Help us improve LangChain for everyone. Whether you're fixing bugs, adding features, or improving documentation, we welcome your contributions.

- [Contributing Guide](#): Everything you need to know about contributing to LangChain

## Stay connected

---

Follow us for the latest updates, announcements, and community highlights.

- [X \(Twitter\)](#): Daily updates and community spotlights
- [LinkedIn](#): Professional network and company updates

# Guardrails

---

langchain/guardrails.mdx

Guardrails help you build safe, compliant AI applications by validating and filtering content at key points in your agent's execution. They can detect sensitive information, enforce content policies, validate outputs, and prevent unsafe behaviors before they cause problems.

Common use cases include:

- Preventing PII leakage
- Detecting and blocking prompt injection attacks
- Blocking inappropriate or harmful content
- Enforcing business rules and compliance requirements
- Validating output quality and accuracy

You can implement guardrails using [middleware](#) to intercept execution at strategic points - before the agent starts, after it completes, or around model and tool calls.



Middleware flow diagram

Guardrails can be implemented using two complementary approaches:

## Deterministic guardrails

Use rule-based logic like regex patterns, keyword matching, or explicit checks. Fast, predictable, and cost-effective, but may miss nuanced violations.

## Model-based guardrails

Use LLMs or classifiers to evaluate content with semantic understanding. Catch subtle issues that rules miss, but are slower and more expensive.

LangChain provides both built-in guardrails (e.g., [PII detection](#), [human-in-the-loop](#)) and a flexible middleware system for building custom guardrails using either approach.

## Built-in guardrails

---

### PII detection

LangChain provides built-in middleware for detecting and handling Personally Identifiable Information (PII) in conversations. This middleware can detect common PII types like emails, credit cards, IP addresses, and more.

PII detection middleware is helpful for cases such as health care and financial applications with compliance requirements, customer service agents that need to sanitize logs, and generally any application handling sensitive user data.

The PII middleware supports multiple strategies for handling detected PII:

Strategy	Description	Example
redact	Replace with [REDACTED_{PII_TYPE}]	[REDACTED_EMAIL]
mask	Partially obscure (e.g., last 4 digits)	****_****_****-1234
hash	Replace with deterministic hash	a8f5f167...
block	Raise exception when detected	Error thrown

## python

### 참고

With `apply_to_output=True`, `PIIMiddleware` also redacts streamed wire output—text deltas, tool-call args, tool outputs, and state snapshots—via a registered stream transformer. Requires `langchain>=1.3.2`. See [Register transformers on middleware](#).

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[customer_service_tool, email_tool],
 middleware=[
 # Redact emails in user input before sending to model
 PIIMiddleware(
 "email",
 strategy="redact",
 apply_to_input=True,
),
 # Mask credit cards in user input
 PIIMiddleware(
 "credit_card",
 strategy="mask",
 apply_to_input=True,
),
 # Block API keys - raise error if detected
 PIIMiddleware(
 "api_key",
 detector=r"sk-[a-zA-Z0-9]{32}",
 strategy="block",
 apply_to_input=True,
),
],
)

When user provides PII, it will be handled according to the strategy
result = agent.invoke({
 "messages": [{"role": "user", "content": "My email is john.doe@example.com and card
```

```
is 5105-1051-0510-5100"}]
})
```

js

```
import { createAgent, piiMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [customerServiceTool, emailTool],
 middleware: [
 // Redact emails in user input before sending to model
 piiMiddleware("email", {
 strategy: "redact",
 applyToInput: true,
 }),
 // Mask credit cards in user input
 piiMiddleware("credit_card", {
 strategy: "mask",
 applyToInput: true,
 }),
 // Block API keys - raise error if detected
 piiMiddleware("api_key", {
 detector: /sk-[a-zA-Z0-9]{32}/,
 strategy: "block",
 applyToInput: true,
 }),
],
});

// When user provides PII, it will be handled according to the strategy
const result = await agent.invoke({
 messages: [{
 role: "user",
 content: "My email is john.doe@example.com and card is 5105-1051-0510-5100"
 }]
});
```

## Built-in PII types and configuration

**Built-in PII types:** - `email` - Email addresses - `credit_card` - Credit card numbers (Luhn validated) - `ip` - IP addresses - `mac_address` - MAC addresses - `url` - URLs

## Configuration options:

python

Parameter	Description	Default
pii_type	Type of PII to detect (built-in or custom)	Required
strategy	How to handle detected PII ( "block" , "redact" , "mask" , "hash" )	"redact"
detector	Custom detector function or regex pattern	None (uses built-in)
apply_to_input	Check user messages before model call	True
apply_to_output	Check AI messages after model call	False
apply_to_tool_results	Check tool result messages after execution	False

js

Parameter	Description	Default
piiType	Type of PII to detect (built-in or custom)	Required
strategy	How to handle detected PII ( "block" , "redact" , "mask" , "hash" )	"redact"
detector	Custom detector regex pattern	undefined (uses built-in)
applyToInput	Check user messages before model call	true
applyToOutput	Check AI messages after model call	false
applyToToolResults	Check tool result messages after execution	false

See the [middleware documentation](#) for complete details on PII detection capabilities.

## Human-in-the-loop

LangChain provides built-in middleware for requiring human approval before executing sensitive operations. This is one of the most effective guardrails for high-stakes decisions.

Human-in-the-loop middleware is helpful for cases such as financial transactions and transfers, deleting or modifying production data, sending communications to external parties, and any operation with significant business impact.

python

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
```

```

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, send_email_tool, delete_database_tool],
 middleware=[
 HumanInTheLoopMiddleware(
 interrupt_on={
 # Require approval for sensitive operations
 "send_email": True,
 "delete_database": True,
 # Auto-approve safe operations
 "search": False,
 }
),
],
 # Persist the state across interrupts
 checkpointer=InMemorySaver(),
)

Human-in-the-loop requires a thread ID for persistence
config = {"configurable": {"thread_id": "some_id"}}

Agent will pause and wait for approval before executing sensitive tools
result = agent.invoke(
 {"messages": [{"role": "user", "content": "Send an email to the team"}]},
 config=config
)

result = agent.invoke(
 Command(resume={"decisions": [{"type": "approve"}]}),
 config=config # Same thread ID to resume the paused conversation
)

```

js

```

import { createAgent, humanInTheLoopMiddleware } from "langchain";
import { MemorySaver, Command } from "@langchain/langgraph";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, sendEmailTool, deleteDatabaseTool],
 middleware: [
 humanInTheLoopMiddleware({
 interruptOn: {
 // Require approval for sensitive operations
 send_email: { allowAccept: true, allowEdit: true, allowRespond: true },
 delete_database: { allowAccept: true, allowEdit: true, allowRespond: true },
 // Auto-approve safe operations
 search: false,
 }
 })
]
});

```



```

 }
 })),
],
 checkpointer: new MemorySaver(),
});

// Human-in-the-loop requires a thread ID for persistence
const config = { configurable: { thread_id: "some_id" } };

// Agent will pause and wait for approval before executing sensitive tools
let result = await agent.invoke(
 { messages: [{ role: "user", content: "Send an email to the team" }] },
 config
);

result = await agent.invoke(
 new Command({ resume: { decisions: [{ type: "approve" }] } }),
 config // Same thread ID to resume the paused conversation
);

```

### 팁

See the [human-in-the-loop documentation](#) for complete details on implementing approval workflows.

## Custom guardrails

For more sophisticated guardrails, you can create custom middleware that runs before or after the agent executes. This gives you full control over validation logic, content filtering, and safety checks.

### Before agent guardrails

Use "before agent" hooks to validate requests once at the start of each invocation. This is useful for session-level checks like authentication, rate limiting, or blocking inappropriate requests before any processing begins.

#### python

```

` `` python title="Class syntax" from typing import Any

from langchain.agents.middleware import AgentMiddleware, AgentState, hook_config from
langgraph.runtime import Runtime

class ContentFilterMiddleware(AgentMiddleware): """Deterministic guardrail: Block
requests containing banned keywords."""

```

```
def init(self, banned_keywords: list[str]): super().init() self.banned_keywords = [kw.lower()
for kw in banned_keywords]

@hook_config(can_jump_to=["end"]) def before_agent(self, state: AgentState, runtime:
Runtime) -> dict[str, Any] | None: # Get the first user message if not state["messages"]:
return None

first_message = state["messages"][0] if first_message.type != "human": return None

content = first_message.content.lower()
```

## Check for banned keywords

---

```
for keyword in self.banned_keywords: if keyword in content: # Block execution before any
processing return { "messages": [{ "role": "assistant", "content": "I cannot process requests
containing inappropriate content. Please rephrase your request." }], "jump_to": "end" }

return None
```

## Use the custom guardrail

---

```
from langchain.agents import create_agent

agent = create_agent(model="gpt-5.5", tools=[search_tool, calculator_tool], middleware=[
ContentFilterMiddleware(banned_keywords=["hack", "exploit", "malware"]),],)
```

## This request will be blocked before any processing

---

```
result = agent.invoke({ "messages": [{"role": "user", "content": "How do I hack into a
database?"}] })
```

```
```python title="Decorator syntax"
from typing import Any

from langchain.agents.middleware import before_agent, AgentState, hook_config
from langgraph.runtime import Runtime

banned_keywords = ["hack", "exploit", "malware"]

@before_agent(can_jump_to=["end"])
def content_filter(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """Deterministic guardrail: Block requests containing banned keywords."""
```

```

# Get the first user message
if not state["messages"]:
    return None

first_message = state["messages"][0]
if first_message.type != "human":
    return None

content = first_message.content.lower()

# Check for banned keywords
for keyword in banned_keywords:
    if keyword in content:
        # Block execution before any processing
        return {
            "messages": [{
                "role": "assistant",
                "content": "I cannot process requests containing inappropriate content.
Please rephrase your request."
            }],
            "jump_to": "end"
        }

return None

# Use the custom guardrail
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-5.5",
    tools=[search_tool, calculator_tool],
    middleware=[content_filter],
)

# This request will be blocked before any processing
result = agent.invoke({
    "messages": [{"role": "user", "content": "How do I hack into a database?"}]
})

```

js

```

import { createMiddleware, AIMessage } from "langchain";

const contentFilterMiddleware = (bannedKeywords: string[]) => {
    const keywords = bannedKeywords.map(kw => kw.toLowerCase());

    return createMiddleware({
        name: "ContentFilterMiddleware",
        beforeAgent: {

```

```

hook: (state) => {
  // Get the first user message
  if (!state.messages || state.messages.length === 0) {
    return;
  }

  const firstMessage = state.messages[0];
  if (firstMessage._getType() !== "human") {
    return;
  }

  const content = firstMessage.content.toString().toLowerCase();

  // Check for banned keywords
  for (const keyword of keywords) {
    if (content.includes(keyword)) {
      // Block execution before any processing
      return {
        messages: [
          new AIMessage(
            "I cannot process requests containing inappropriate content. Please rephrase your request."
          ),
        ],
        jumpTo: "end",
      };
    }
  }

  return;
},
canJumpTo: ['end']
});

// Use the custom guardrail
import { createAgent } from "langchain";

const agent = createAgent({
  model: "gpt-5.5",
  tools: [searchTool, calculatorTool],
  middleware: [
    contentFilterMiddleware(["hack", "exploit", "malware"]),
  ],
});

// This request will be blocked before any processing
const result = await agent.invoke({

```

```
messages: [{ role: "user", content: "How do I hack into a database?" }]
});
```

After agent guardrails

Use "after agent" hooks to validate final outputs once before returning to the user. This is useful for model-based safety checks, quality validation, or final compliance scans on the complete agent response.

python

```
` `` python title="Class syntax" from langchain.agents.middleware import
AgentMiddleware, AgentState, hook_config from langgraph.runtime import Runtime from
langchain.messages import AIMessage from langchain.chat_models import
init_chat_model from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware): """Model-based guardrail: Use an LLM
to evaluate response safety."""

def init(self): super().init() self.safety_model = init_chat_model("gpt-5.4-mini")

@hook_config(can_jump_to=["end"]) def after_agent(self, state: AgentState, runtime:
Runtime) -> dict[str, Any] | None: # Get the final AI response if not state["messages"]: return
None

last_message = state["messages"][-1] if not isinstance(last_message, AIMessage): return
None
```

Use a model to evaluate safety

```
safety_prompt = f"""Evaluate if this response is safe and appropriate. Respond with only
'SAFE' or 'UNSAFE'.
```

```
Response: {last_message.content}"""
```

```
result = self.safety_model.invoke([{"role": "user", "content": safety_prompt}])
```

```
if "UNSAFE" in result.content: last_message.content = "I cannot provide that response.
Please rephrase your request."
```

```
return None
```

Use the safety guardrail

```
from langchain.agents import create_agent
```

```
agent = create_agent( model="gpt-5.5", tools=[search_tool, calculator_tool], middleware=[SafetyGuardrailMiddleware()], )
```

```
result = agent.invoke({ "messages": [{"role": "user", "content": "How do I make explosives?"}] })
```

```
```python title="Decorator syntax"
from langchain.agents.middleware import after_agent, AgentState, hook_config
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

safety_model = init_chat_model("gpt-5.4-mini")

@after_agent(can_jump_to=["end"])
def safety_guardrail(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 """Model-based guardrail: Use an LLM to evaluate response safety."""
 # Get the final AI response
 if not state["messages"]:
 return None

 last_message = state["messages"][-1]
 if not isinstance(last_message, AIMessage):
 return None

 # Use a model to evaluate safety
 safety_prompt = f"""Evaluate if this response is safe and appropriate.
Respond with only 'SAFE' or 'UNSAFE'.

Response: {last_message.content}"""

 result = safety_model.invoke([{"role": "user", "content": safety_prompt}])

 if "UNSAFE" in result.content:
 last_message.content = "I cannot provide that response. Please rephrase your request."

 return None

Use the safety guardrail
from langchain.agents import create_agent

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, calculator_tool],
 middleware=[safety_guardrail],
)
```

```
result = agent.invoke({
 "messages": [{ "role": "user", "content": "How do I make explosives?" }]
})
```

js

```
import { createMiddleware, AIMessage, initChatModel } from "langchain";

const safetyGuardrailMiddleware = () => {
 const safetyModel = initChatModel("gpt-5.4-mini");

 return createMiddleware({
 name: "SafetyGuardrailMiddleware",
 afterAgent: {
 hook: async (state) => {
 // Get the final AI response
 if (!state.messages || state.messages.length === 0) {
 return;
 }

 const lastMessage = state.messages[state.messages.length - 1];
 if (lastMessage._getType() !== "ai") {
 return;
 }

 // Use a model to evaluate safety
 const safetyPrompt = `Evaluate if this response is safe and appropriate.
Respond with only 'SAFE' or 'UNSAFE'.

Response: ${lastMessage.content.toString()}`;

 const result = await safetyModel.invoke([
 { role: "user", content: safetyPrompt }
]);

 if (result.content.toString().includes("UNSAFE")) {
 return {
 messages: [
 new AIMessage(
 "I cannot provide that response. Please rephrase your request."
)
],
 jumpTo: "end",
 };
 }

 return;
 },
 canJumpTo: ['end']
 }
 });
}
```

```

 }
 });
};

// Use the safety guardrail
import { createAgent } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, calculatorTool],
 middleware: [safetyGuardrailMiddleware()],
});

const result = await agent.invoke({
 messages: [{ role: "user", content: "How do I make explosives?" }]
});

```

## Combine multiple guardrails

You can stack multiple guardrails by adding them to the middleware array. They execute in order, allowing you to build layered protection:

### python

```

from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, send_email_tool],
 middleware=[
 # Layer 1: Deterministic input filter (before agent)
 ContentFilterMiddleware(banned_keywords=["hack", "exploit"]),

 # Layer 2: PII protection (before and after model)
 PIIMiddleware("email", strategy="redact", apply_to_input=True),
 PIIMiddleware("email", strategy="redact", apply_to_output=True),

 # Layer 3: Human approval for sensitive tools
 HumanInTheLoopMiddleware(interrupt_on={"send_email": True}),

 # Layer 4: Model-based safety check (after agent)
 SafetyGuardrailMiddleware(),
],
)

```

### js



```

import { createAgent, piiMiddleware, humanInTheLoopMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, sendEmailTool],
 middleware: [
 // Layer 1: Deterministic input filter (before agent)
 contentFilterMiddleware(["hack", "exploit"]),

 // Layer 2: PII protection (before and after model)
 piiMiddleware("email", {
 strategy: "redact",
 applyToInput: true,
 }),
 piiMiddleware("email", {
 strategy: "redact",
 applyToOutput: true,
 }),

 // Layer 3: Human approval for sensitive tools
 humanInTheLoopMiddleware({
 interruptOn: {
 send_email: { allowAccept: true, allowEdit: true, allowRespond: true },
 }
 }),

 // Layer 4: Model-based safety check (after agent)
 safetyGuardrailMiddleware(),
],
});

```

## Additional resources

- [Middleware documentation](#) - Complete guide to custom middleware
- [Middleware API reference](#) - Complete guide to custom middleware
- [Human-in-the-loop](#) - Add human review for sensitive operations
- [Testing agents](#) - Strategies for testing safety mechanisms

# Human-in-the-loop

---

langchain/human-in-the-loop.mdx

The Human-in-the-Loop (HITL) [middleware](#) lets you add human oversight to agent tool calls. When a model proposes an action that might require review—for example, writing to a file or executing SQL—the middleware can pause execution and wait for a decision.

It does this by checking each tool call against a configurable policy. If intervention is needed, the middleware issues an `@[interrupt]` that halts execution. The graph state is saved using LangGraph's [persistence layer](#), so execution can pause safely and resume later.

## python

A human decision then determines what happens next: the action can be approved as-is (`approve`), modified before running (`edit`), rejected with feedback (`reject`), or responded to directly (`respond`) for "ask user" style tools.

## js

A human decision then determines what happens next: the action can be approved as-is (`approve`), modified before running (`edit`), or rejected with feedback (`reject`).

## Interrupt decision types

---

### python

The [middleware](#) defines four built-in ways a human can respond to an interrupt:

### js

The [middleware](#) defines three built-in ways a human can respond to an interrupt:

The available decision types for each tool depend on the policy you configure in `interrupt_on`. When multiple tool calls are paused at the same time, each action requires a separate decision. Decisions must be provided in the same order as the actions appear in the interrupt request.

### python

Use `reject` when the human is denying the requested action. Use `respond` only when the human is acting as the tool, such as answering an `ask_user` prompt. Do not use `respond` to deny side-effecting tools, because its message is treated as a successful tool result.

## js

Use `reject` when the human is denying the requested action.

## 팁

When **editing** tool arguments, make changes conservatively. Significant modifications to the original arguments may cause the model to re-evaluate its approach and potentially execute the tool multiple times or take unexpected actions.

## Configuring interrupts

To use HITL, add the [middleware](#) to the agent's `middleware` list when creating the agent.

You configure it with a mapping of tool actions to the decision types that are allowed for each action. The middleware will interrupt execution when a tool call matches an action in the mapping.

## python

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware # [!code highlight]
from langgraph.checkpoint.memory import InMemorySaver # [!code highlight]

agent = create_agent(
 model="gpt-5.5",
 tools=[write_file, execute_sql, read_data],
 middleware=[
 HumanInTheLoopMiddleware(# [!code highlight]
 interrupt_on={
 "write_file": True, # All decisions (approve, edit, reject, respond) a
allowed
 "execute_sql": {"allowed_decisions": ["approve", "reject"]}, # No edit
ing allowed
 "read_data": False, # Safe operation, no approval needed
 },
 # Prefix for interrupt messages - combined with tool name and args to form
the full message
 # e.g., "Tool execution pending approval: execute_sql with query='DELETE FR
OM..."
 # Individual tools can override this by specifying a "description" in their
```

```

interrupt config
 description_prefix="Tool execution pending approval",
),
],
Human-in-the-loop requires checkpointing to handle interrupts.
In production, use a persistent checkpointer like AsyncPostgresSaver or MongoDBSaver.
checkpointer=InMemorySaver(), # [!code highlight]
)

```

js

```

import { createAgent, humanInTheLoopMiddleware } from "langchain"; // [!code highlight]
import { MemorySaver } from "@langchain/langgraph"; // [!code highlight]

const agent = createAgent({
 model: "gpt-5.5",
 tools: [writeFileTool, executeSQLTool, readDataTool],
 middleware: [
 humanInTheLoopMiddleware({
 interruptOn: {
 write_file: true, // All decisions (approve, edit, reject) allowed
 execute_sql: {
 allowedDecisions: ["approve", "reject"],
 // No editing allowed
 description: "🚨 SQL execution requires DBA approval",
 },
 // Safe operation, no approval needed
 read_data: false,
 },
 // Prefix for interrupt messages - combined with tool name and args to form
 // the full message
 // e.g., "Tool execution pending approval: execute_sql with query='DELETE FROM...'
 // Individual tools can override this by specifying a "description" in their interrupt config
 descriptionPrefix: "Tool execution pending approval",
 }),
],
 // Human-in-the-loop requires checkpointing to handle interrupts.
 // In production, use a persistent checkpointer like AsyncPostgresSaver or MongoDBSaver.
 checkpointer: new MemorySaver(), // [!code highlight]
});

```

## 정보

You must configure a checkpointer to persist the graph state across interrupts.

## python

In production, use a persistent checkpointer like `@[ AsyncPostgresSaver ]` or [MongoDBSaver](#) . For testing or prototyping, use `@[ InMemorySaver ]`.

## js

In production, use a persistent checkpointer like `@[ AsyncPostgresSaver ]` or `@[ MongoDBSaver ]`. For testing or prototyping, use `@[ InMemorySaver ]`.

When invoking the agent, pass a `config` that includes the **thread ID** to associate execution with a conversation thread. See the [LangGraph interrupts documentation](#) for details.

## Configuration options

### python

Mapping of tool names to approval configs. Values can be `True` (interrupt with default config), `False` (auto-approve), or an `InterruptOnConfig` object.

Prefix for action request descriptions

#### `InterruptOnConfig` options:

List of allowed decisions: `'approve'` , `'edit'` , `'reject'` , Or `'respond'`

Static string or callable function for custom description

Optional predicate that receives a `@[ ToolCallRequest ]` and returns `True` to interrupt or `False` to auto-approve. Use it to gate interrupts on a call's arguments. Requires `langchain>=1.3.3` .

### js

Mapping of tool names to approval configs. Values can be `true` (interrupt with default config), `false` (auto-approve), or an `InterruptOnConfig` object.

Prefix for action request descriptions

#### `InterruptOnConfig` options:

List of allowed decisions: `'approve'` , `'edit'` , Or `'reject'`

Static string or callable function for custom description

Optional predicate that receives a `@[ToolCallRequest]` and returns `true` to interrupt or `false` to auto-approve. Use it to gate interrupts on a call's arguments. Requires `langchain>=1.4.6`.

## Conditional interrupts

By default, every tool call listed in `interrupt_on` pauses for review. To pause only some calls, add a `when` predicate to a tool's `InterruptOnConfig`. The predicate receives a `ToolCallRequest` and returns `True` to interrupt or `False` to auto-approve, so you can gate on the tool's arguments.

### python

#### 참고

Conditional interrupts require `langchain>=1.3.3`.

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware, ToolCallRequest
from langgraph.checkpoint.memory import InMemorySaver

def writes_outside_workspace(request: ToolCallRequest) -> bool:
 """Pause writes to paths outside the workspace directory."""
 path = request.tool_call["args"].get("path", "")
 return not path.startswith("/workspace/")

def is_write_query(request: ToolCallRequest) -> bool:
 """Pause SQL that isn't a read-only SELECT."""
 query = request.tool_call["args"].get("query", "")
 return not query.lstrip().upper().startswith("SELECT")

agent = create_agent(
 model="gpt-5.5",
 tools=[write_file, execute_sql, read_data],
 middleware=[
 HumanInTheLoopMiddleware(
 interrupt_on={
 "write_file": {
 "allowed_decisions": ["approve", "edit", "reject"],
 "when": writes_outside_workspace,
 },
 "execute_sql": {
 "allowed_decisions": ["approve", "reject"],
 "when": is_write_query,
 },
 },
),
],
)
```

```

),
],
 checkpointer=InMemorySaver(),
)

```

When the `when` predicate returns `False`, the call runs without interrupting. When it returns `True`, or when you omit `when`, the call pauses as usual. Calls that evaluate to `False` are never added to the interrupt batch, so a reviewer only sees the actions that need a decision.

js

## 참고

Conditional interrupts require `langchain>=1.4.6`.

```

import { createAgent, humanInTheLoopMiddleware } from "langchain";
import type { ToolCallRequest } from "langchain";
import { MemorySaver } from "@langchain/langgraph";

// Pause writes to paths outside the workspace directory.
const writesOutsideWorkspace = (request: ToolCallRequest) =>
 !String(request.toolCall.args.path ?? "").startsWith("/workspace/");

// Pause SQL that isn't a read-only SELECT.
const isWriteQuery = (request: ToolCallRequest) =>
 !String(request.toolCall.args.query ?? "")
 .trimStart()
 .toUpperCase()
 .startsWith("SELECT");

const agent = createAgent({
 model: "gpt-5.5",
 tools: [writeFile, executeSql, readData],
 middleware: [
 humanInTheLoopMiddleware({
 interruptOn: {
 write_file: {
 allowedDecisions: ["approve", "edit", "reject"],
 when: writesOutsideWorkspace,
 },
 execute_sql: {
 allowedDecisions: ["approve", "reject"],
 when: isWriteQuery,
 },
 },
 },
]),
],

```

```
 checkpointer: new MemorySaver(),
 });
```

When the `when` predicate returns `false`, the call runs without interrupting. When it returns `true`, or when you omit `when`, the call pauses as usual. Calls that evaluate to `false` are never added to the interrupt batch, so a reviewer only sees the actions that need a decision.

## Responding to interrupts

When you invoke the agent, it runs until it either completes or an interrupt is raised. An interrupt is triggered when a tool call matches the policy you configured in `interrupt_on`. With `version="v2"`, the result is a `GraphOutput` with an `interrupts` attribute containing the actions that require review. You can then present those actions to a reviewer and resume execution once decisions are provided.

### python

```
from langgraph.types import Command

Human-in-the-loop leverages LangGraph's persistence layer.
You must provide a thread ID to associate the execution with a conversation thread,
so the conversation can be paused and resumed (as is needed for human review).
config = {"configurable": {"thread_id": "some_id"}} # [!code highlight]
Run the graph until the interrupt is hit.
result = agent.invoke(
 {
 "messages": [
 {
 "role": "user",
 "content": "Delete old records from the database",
 }
]
 },
 config=config, # [!code highlight]
 version="v2", # [!code highlight]
)

result is a GraphOutput with .value and .interrupts
print(result.interrupts) # [!code highlight]
> (
> Interrupt(
> value={
> 'action_requests': [
> {
> 'name': 'execute_sql',
> 'arguments': {'query': 'DELETE FROM records WHERE created_at < NOW()'}
```



```

- INTERVAL \'30 days\';'},
> 'description': 'Tool execution pending approval\n\nTool: execute_sql
\nArgs: {...}'
> }
>],
> 'review_configs': [
> {
> 'action_name': 'execute_sql',
> 'allowed_decisions': ['approve', 'reject']
> }
>]
> }
>),
>)

Resume with approval decision
agent.invoke(
 Command(# [!code highlight]
 resume={"decisions": [{"type": "approve"]}} # or "reject" [!code highlight]
), # [!code highlight]
 config=config, # Same thread ID to resume the paused conversation
 version="v2",
)

```

js

```

import { HumanMessage } from "@langchain/core/messages";
import { Command } from "@langchain/langgraph";

// You must provide a thread ID to associate the execution with a conversation thread,
// so the conversation can be paused and resumed (as is needed for human review).
const config = { configurable: { thread_id: "some_id" } }; // [!code highlight]

// Run the graph until the interrupt is hit.
const result = await agent.invoke(
 {
 messages: [new HumanMessage("Delete old records from the database")],
 },
 config // [!code highlight]
);

// The interrupt contains the full HITL request with action_requests and review_configs
console.log(result.__interrupt__);
// > [
// > Interrupt(
// > value: {
// > actionRequests: [
// > {
// > name: 'execute_sql',

```

```
// > arguments: { query: 'DELETE FROM records WHERE created_at < NOW() -
INTERVAL \'30 days\';' },
// > description: 'Tool execution pending approval\n\nTool: execute_sql
\nArgs: {...}'
// > }
// > },
// > reviewConfigs: [
// > {
// > actionName: 'execute_sql',
// > allowedDecisions: ['approve', 'reject']
// > }
// >]
// > }
// >)
// >]

// Resume with approval decision
await agent.invoke(
 new Command({ // [!code highlight]
 resume: { decisions: [{ type: "approve" }] }, // or "reject" [!code highlight]
 }), // [!code highlight]
 config // Same thread ID to resume the paused conversation
);
```

## Decision types

### ✅ approve

Use `approve` to approve the tool call as-is and execute it without changes.

### python

```
agent.invoke(
 Command(
 # Decisions are provided as a list, one per action under review.
 # The order of decisions must match the order of actions
 # in the interrupt request.
 resume={
 "decisions": [
 {
 "type": "approve",
 }
]
 },
 config=config, # Same thread ID to resume the paused conversation
),
```

```
version="v2",
)
```

js

```
await agent.invoke(
 new Command({
 // Decisions are provided as a list, one per action under review.
 // The order of decisions must match the order of actions
 // in the interrupt request.
 resume: {
 decisions: [
 {
 type: "approve",
 }
]
 }
 })),
 config // Same thread ID to resume the paused conversation
);
```

 **edit**

Use `edit` to modify the tool call before execution. Provide the edited action with the new tool name and arguments.

python

```
agent.invoke(
 Command(
 # Decisions are provided as a list, one per action under review.
 # The order of decisions must match the order of actions
 # in the interrupt request.
 resume={
 "decisions": [
 {
 "type": "edit",
 # Edited action with tool name and args
 "edited_action": {
 # Tool name to call.
 # Will usually be the same as the original action.
 "name": "new_tool_name",
 # Arguments to pass to the tool.
 "args": {"key1": "new_value", "key2": "original_value"},
 }
 }
]
 }
)
)
```

```

 }
),
 config=config, # Same thread ID to resume the paused conversation
 version="v2",
)

```

js

```

await agent.invoke(
 new Command({
 // Decisions are provided as a list, one per action under review.
 // The order of decisions must match the order of actions
 // in the interrupt request.
 resume: {
 decisions: [
 {
 type: "edit",
 // Edited action with tool name and args
 editedAction: {
 // Tool name to call.
 // Will usually be the same as the original action.
 name: "new_tool_name",
 // Arguments to pass to the tool.
 args: { key1: "new_value", key2: "original_value" },
 },
 },
],
 },
 })),
 config // Same thread ID to resume the paused conversation
);

```

### 팁

When **editing** tool arguments, make changes conservatively. Significant modifications to the original arguments may cause the model to re-evaluate its approach and potentially execute the tool multiple times or take unexpected actions.

### ✖ reject

Use `reject` to deny the tool call and provide feedback instead of execution. The tool is not executed.

python

```

agent.invoke(
 Command(
 # Decisions are provided as a list, one per action under review.
 # The order of decisions must match the order of actions
 # in the interrupt request.
 resume={
 "decisions": [
 {
 "type": "reject",
 # Optional: explain why the action was rejected
 # and whether the agent should retry a different approach.
 "message": "User rejected this action. Do not retry this tool cal
1.",
 }
]
 },
 config=config, # Same thread ID to resume the paused conversation
 version="v2",
)

```

js

```

await agent.invoke(
 new Command({
 // Decisions are provided as a list, one per action under review.
 // The order of decisions must match the order of actions
 // in the interrupt request.
 resume: {
 decisions: [
 {
 type: "reject",
 // Optional: explain why the action was rejected
 // and whether the agent should retry a different approach.
 message: "User rejected this action. Do not retry this tool call.",
 }
]
 }
 }),
 config // Same thread ID to resume the paused conversation
);

```

The `message` is added to the conversation as feedback to help the agent understand why the action was rejected and what it should do instead. When you omit `message`, the middleware uses a default rejection message that tells the model the tool was not executed and not to retry the same tool call unless the user asks. For side-effecting tools, provide a domain-specific

message that is explicit about whether the agent should abandon the action, ask a follow-up question, or try a safer alternative.

## python

### respond

Use `respond` for "ask user" style tools where the tool's real implementation is the human's reply. The `message` content is returned directly as the tool result; the tool itself is not executed.

```
agent.invoke(
 Command(
 # Decisions are provided as a list, one per action under review.
 # The order of decisions must match the order of actions
 # in the interrupt request.
 resume={
 "decisions": [
 {
 "type": "respond",
 # The human's reply, returned directly as the tool result
 "message": "Blue.",
 }
]
 },
 config=config, # Same thread ID to resume the paused conversation
 version="v2",
)
)
```

The `message` is returned to the agent as a successful `ToolMessage`. Use `respond` when the tool is intentionally a placeholder for human input, for example, an `ask_user` tool that prompts for clarification. Do not use `respond` to deny a proposed action, because it tells the model that the tool completed successfully.

---

## Multiple decisions

When multiple actions are under review, provide a decision for each action in the same order as they appear in the interrupt:

## python

```
{
 "decisions": [
 {"type": "approve"},
 {
```

```

 "type": "edit",
 "edited_action": {
 "name": "tool_name",
 "args": {"param": "new_value"}
 }
 },
 {
 "type": "reject",
 "message": "This action is not allowed"
 }
]
}

```

js

```

{
 decisions: [
 { type: "approve" },
 {
 type: "edit",
 editedAction: {
 name: "tool_name",
 args: { param: "new_value" }
 }
 },
 {
 type: "reject",
 message: "This action is not allowed"
 }
]
}

```

## Streaming with human-in-the-loop

You can stream real-time updates while the agent runs and handles interrupts using `stream_events()`. Use `stream.messages` to stream LLM tokens and `stream.values` to check agent state snapshots for interrupts.

python

```

from langgraph.types import Command

config = {"configurable": {"thread_id": "some_id"}}

Stream agent progress and LLM tokens until interrupt
stream = agent.stream_events(

```

```

 {"messages": [{"role": "user", "content": "Delete old records from the database"}]},
 config=config,
 version="v3", # [!code highlight]
)
for message in stream.messages: # [!code highlight]
 for token in message.text: # [!code highlight]
 print(token, end="", flush=True)

Check whether the run paused for human input
if stream.interrupted: # [!code highlight]
 print(f"\n\nInterrupt: {stream.interrupts}") # [!code highlight]

Resume with streaming after human decision
stream = agent.stream_events(
 Command(resume={"decisions": [{"type": "approve"}]}),
 config=config,
 version="v3", # [!code highlight]
)
for message in stream.messages: # [!code highlight]
 for token in message.text:
 print(token, end="", flush=True)

```

js

```

import { Command } from "@langchain/langgraph";

const config = { configurable: { thread_id: "some_id" } };

// Stream agent progress and LLM tokens until interrupt
const stream = await agent.streamEvents(
 { messages: [{ role: "user", content: "Delete old records from the database" }] },
 { ...config, version: "v3" } // [!code highlight]
);
for await (const message of stream.messages) { // [!code highlight]
 for await (const token of message.text) { // [!code highlight]
 process.stdout.write(token);
 }
}

// Check whether the run paused for human input
if (stream.interrupted) { // [!code highlight]
 console.log(`\n\nInterrupt: ${JSON.stringify(stream.interrupts)}`); // [!code highlight]
}

// Resume with streaming after human decision
const resumeStream = await agent.streamEvents(
 new Command({ resume: { decisions: [{ type: "approve" }] } }),

```



```

 { ...config, version: "v3" } // [!code highlight]
);
 for await (const message of resumeStream.messages) { // [!code highlight]
 for await (const token of message.text) {
 process.stdout.write(token);
 }
 }
}

```

See the [Streaming](#) guide for more details on stream modes.

## Execution lifecycle

The middleware defines an `after_model` hook that runs after the model generates a response but before any tool calls are executed:

1. The agent invokes the model to generate a response.
2. The middleware inspects the response for tool calls.
3. If any calls require human input, the middleware builds a `@[HITLRequest]` with `action_requests` and `review_configs` and calls `@[interrupt]`.
4. The agent waits for human decisions.

### python

5. Based on the `HITLResponse` decisions, the middleware executes approved or edited calls, synthesizes `@[ToolMessage]`'s for rejected calls, returns human replies directly as `@[ToolMessage]`'s for `respond` decisions, and resumes execution.

### js

5. Based on the `HITLResponse` decisions, the middleware executes approved or edited calls, synthesizes `@[ToolMessage]`'s for rejected calls, and resumes execution.

## Custom HITL logic

For more specialized workflows, you can build custom HITL logic directly using the `@[interrupt]` primitive and [middleware](#) abstraction.

Review the [execution lifecycle](#) above to understand how to integrate interrupts into the agent's operation.

# Install LangChain

---

langchain/install.mdx

To install the LangChain package:

## python

```
```bash pip pip install -U langchain
```

Requires Python 3.10+

```
```bash uv
uv add langchain
Requires Python 3.10+
```

## js

```
```bash npm npm install langchain @langchain/core
```

Requires Node.js 22+

```
```bash pnpm
pnpm add langchain @langchain/core
Requires Node.js 22+
```

```
```bash yarn yarn add langchain @langchain/core
```

Requires Node.js 22+

```
```bash bun
bun add langchain @langchain/core
Requires Bun v1.0.0+
```

LangChain provides integrations to hundreds of LLMs and thousands of other integrations. These live in independent provider packages.

**python**

```
```bash pip
```

Installing the OpenAI integration

```
pip install -U langchain-openai
```

Installing the Anthropic integration

```
pip install -U langchain-anthropic
```

```
```bash uv
Installing the OpenAI integration
uv add langchain-openai

Installing the Anthropic integration
uv add langchain-anthropic
```

**js**

```
```bash npm
```

Installing the OpenAI integration

```
npm install @langchain/openai
```

Installing the Anthropic integration

```
npm install @langchain/anthropic
```

```
```bash pnpm
Installing the OpenAI integration
pnpm install @langchain/openai
Installing the Anthropic integration
pnpm install @langchain/anthropic
```

```
```bash yarn
```

Installing the OpenAI integration

```
yarn add @langchain/openai
```

Installing the Anthropic integration

```
yarn add @langchain/anthropic
```

```
```bash bun
Installing the OpenAI integration
bun add @langchain/openai
Installing the Anthropic integration
bun add @langchain/anthropic
```

## 팁

See the [Integrations tab](#) for a full list of available integrations.

Now that you have LangChain installed, you can get started by following the [Quickstart guide](#).

## 팁

Set up [LangSmith](#) tracing to debug your first LangChain app. Follow the [tracing quickstart](#) to get started. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

# Build a semantic search engine with LangChain

langchain/knowledge-base.mdx

## Overview

Build a semantic search engine over a PDF with LangChain [embeddings](#) and [vector stores](#). Use it to retrieve passages similar to a query, then plug the retriever into [retrieval-augmented generation \(RAG\)](#) or other LLM workflows.

This tutorial covers:

1. Create `Document` objects from a PDF.
2. Generate embeddings.
3. Load and split a PDF.
4. Index chunks in a vector store and query by similarity.
5. Wrap the store as a retriever.

The guide also includes a minimal RAG implementation on top of the search engine.

## Concepts

This tutorial focuses on text retrieval and covers the following concepts:

- `@[ Document ]`
- [Text splitters](#)
- [Embeddings](#)
- [Vector stores](#) and [retrievers](#)

## Setup

### Install dependencies

#### python

This tutorial reads a PDF using the `pypdf` package:

```
```bash pip install pypdf
```

```
```bash conda
conda install pypdf -c conda-forge
```

```
```bash uv uv add pypdf
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

This tutorial reads a PDF using the `pdf-parse` package:

```
```bash npm
npm i pdf-parse
```

```
```bash yarn yarn add pdf-parse
```

```
```bash pnpm
pnpm add pdf-parse
```

For more details, see the [Installation guide](#).

## Configure LangSmith

Many of the applications you build with LangChain will contain multiple steps with multiple invocations of LLM calls. As these applications get more and more complex, it becomes crucial to be able to inspect what exactly is going on inside your chain or agent. The best way to do this is with [LangSmith](#).

After you sign up at the link above, make sure to set your environment variables to start logging traces:

```
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."
```

### python

In a notebook, you can set them with:

```
import getpass
import os

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

## Create documents

LangChain implements a `@[Document]` abstraction for a unit of text and associated metadata. It has three attributes:

### python

- `page_content`: a string representing the content.
- `metadata`: a dict containing arbitrary metadata.
- `id`: (optional) a string identifier for the document.

### js

- `pageContent`: a string representing the content.
- `metadata`: a dict containing arbitrary metadata.
- `id`: (optional) a string identifier for the document.

`metadata` can capture the source of the document, its relationship to other documents, and other information. An individual `@[Document]` often represents a chunk of a larger document.

The following code creates sample documents:

### python

```
from langchain_core.documents import Document

documents = [
 Document(
 page_content="Dogs are great companions, known for their loyalty and friendline ss.",
 metadata={"source": "mammal-pets-doc"},
),
 Document(
 page_content="Cats are independent pets that often enjoy their own space.",
 metadata={"source": "mammal-pets-doc"},
),
]
```

### js

```
import { Document } from "@langchain/core/documents";

const documents = [
 new Document({
 pageContent:
```

```

 "Dogs are great companions, known for their loyalty and friendliness.",
 metadata: { source: "mammal-pets-doc" },
 }},
 new Document({
 pageContent: "Cats are independent pets that often enjoy their own space.",
 metadata: { source: "mammal-pets-doc" },
 }},
];

```

## Generate embeddings

Vector search stores numeric vectors associated with text. Embed a query as a vector of the same dimension, then use similarity metrics (such as cosine similarity) to find related text.

LangChain supports embeddings from [many providers](#). Select a model to specify how text should be converted into a numeric vector:

### python

```

vector_1 = embeddings.embed_query(documents[0].page_content)
vector_2 = embeddings.embed_query(documents[1].page_content)

assert len(vector_1) == len(vector_2)
print(f"Generated vectors of length {len(vector_1)}\n")
print(vector_1[:10])

```

### js

```

const vector1 = await embeddings.embedQuery(documents[0].pageContent);
const vector2 = await embeddings.embedQuery(documents[1].pageContent);

assert vector1.length === vector2.length;
console.log(`Generated vectors of length ${vector1.length}\n`);
console.log(vector1.slice(0, 10));

```

```text wrap Generated vectors of length 1536

```

[-0.008586574345827103, -0.03341241180896759, -0.008936782367527485,
-0.0036674530711025, 0.010564599186182022, 0.009598285891115665, -0.028587326407432556,
-0.015824200585484505, 0.0030416189692914486, -0.012899317778646946]

```

Next, store embeddings in a vector store that supports efficient similarity search.

```

## Select a vector store

```


LangChain `@[`VectorStore`]` objects add text and `@[`Document`]` objects to a store and query them with similarity metrics. They are often initialized with `[embedding](/oss/integrations/embeddings)` models that translate text into numeric vectors.

LangChain includes `[integrations](/oss/integrations/vectorstores)` with many vector store technologies. Some are hosted and need credentials, some run in separate infrastructure (local or third-party), and others run in-memory for lightweight workloads. Select a vector store:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
</div>
```

```
## Load and split a PDF
```

Load content from a PDF, then split it into smaller chunks before indexing. This example uses `[a sample Nike 10-K filing from 2023](https://github.com/langchain-ai/langchain/blob/v0.3/docs/docs/example_data/nke-10k-2023.pdf)`.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

For dedicated PDF document loaders (including ``PyPDFLoader`` and partner packages), see `[PDF document loaders](/oss/integrations/document_loaders#pdfs)`.

```
```python
import pypdf
from langchain_core.documents import Document

Below is a minimal helper for demonstration purposes.
def load_pdf_pages(file_path: str) -> list[Document]:
 reader = pypdf.PdfReader(file_path)
 return [
 Document(
 page_content=page.extract_text() or "",
 metadata={"source": file_path, "page": i},
)
 for i, page in enumerate(reader.pages)
]

file_path = "../example_data/nke-10k-2023.pdf"
```

```
docs = load_pdf_pages(file_path)
print(len(docs))
```

## js

```
import { readFileSync } from "node:fs";
import { Document } from "@langchain/core/documents";
import { PDFParse } from "pdf-parse";

// Below is a minimal helper for demonstration purposes.
async function loadPdfPages(filePath: string): Promise<Document[]> {
 const parser = new PDFParse({
 data: new Uint8Array(readFileSync(filePath)),
 });
 try {
 const { pages } = await parser.getText();
 return pages.map(
 (page) =>
 new Document({
 pageContent: page.text,
 metadata: { source: filePath, page: page.num - 1 },
 })
);
 } finally {
 await parser.destroy();
 }
}

const filePath = "../../data/nke-10k-2023.pdf";
const docs = await loadPdfPages(filePath);
console.log(docs.length);
```

107

A page is often too coarse for retrieval. Split pages further so relevant passages are not diluted by surrounding text. `@[RecursiveCharacterTextSplitter]` recursively splits on common separators (such as newlines) until each chunk is the target size. This is the recommended text splitter for generic text use cases.

## python

Set `add_start_index=True` so each split keeps a `start_index` metadata field for its character offset in the original document.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(
 chunk_size=1000, chunk_overlap=200, add_start_index=True
)
all_splits = text_splitter.split_documents(docs)

print(len(all_splits))
```

js

```
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";

const textSplitter = new RecursiveCharacterTextSplitter({
 chunkSize: 1000,
 chunkOverlap: 200,
});

const allSplits = await textSplitter.splitDocuments(docs);

console.log(allSplits.length);
```

516

## Index documents

Index the chunks into the vector store:

python

```
ids = vector_store.add_documents(documents=all_splits)
```

js

```
await vectorStore.addDocuments(allSplits);
```

Most vector store integrations also support connecting to an existing store (for example with a client or index name). See the docs for a specific [integration](#) for details.

## Query the vector store

python

After you have added the documents to the `@[vectorStore]`, you can query it:

- Synchronously and asynchronously
- By string query and by vector
- With and without similarity scores
- By similarity and @[maximum marginal relevance]  
[VectorStore.max\_marginal\_relevance\_search] (to balance similarity with diversity)

js

After you have added the documents to the @[VectorStore], you can query it:

- Synchronously and asynchronously
- By string query and by vector
- With and without similarity scores
- By similarity and @[maximum marginal relevance]  
[VectorStore.maxMarginalRelevanceSearch] (to balance similarity with diversity)

These methods generally return a list of @[Document] objects.

## Search by string

Embeddings map text to dense vectors so similar meanings are geometrically close. This means you can Retrieve relevant passages by passing a natural-language question:

python

```
results = vector_store.similarity_search(
 "How many distribution centers does Nike have in the US?"
)

print(results[0])
```

```
```python wrap page_content='direct to consumer operations sell products through the
following number of retail stores in the United States: U.S. RETAIL STORES NUMBER NIKE Brand
factory stores 213 NIKE Brand in-line stores (including employee-only stores) 74 Converse stores
(including factory stores) 82 TOTAL 369 In the United States, NIKE has eight significant
distribution centers. Refer to Item 2. Properties for further information. 2023 FORM 10-K 2'
metadata={'page': 4, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 3125}
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>
```

```
```typescript
const results1 = await vectorStore.similaritySearch(
 "When was Nike incorporated?"
);

console.log(results1[0]);
```

```
Document {
 pageContent: 'direct to consumer operations sell products...',
 metadata: {'page': 4, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 3125}
}
```

## python

Async query:

```
results = await vector_store.asimilarity_search("When was Nike incorporated?")

print(results[0])
```

```
```python wrap page_content='Table of Contents PART I ITEM 1. BUSINESS GENERAL NIKE, Inc.
was incorporated in 1967 under the laws of the State of Oregon. As used in this Annual Report
on Form 10-K (this "Annual Report"), the terms "we," "us," "our," "NIKE" and the "Company"
refer to NIKE, Inc. and its predecessors, subsidiaries and affiliates, collectively, unless the
context indicates otherwise. Our principal business activity is the design, development and
worldwide marketing and selling of athletic footwear, apparel, equipment, accessories and
services. NIKE is the largest seller of athletic footwear and apparel in the world. We sell our
products through NIKE Direct operations, which are comprised of both NIKE-owned retail stores
and sales through our digital platforms (also referred to as "NIKE Brand Digital"), to retail
accounts and to a mix of independent distributors, licensees and sales' metadata={'page': 3,
'source': '../example_data/nke-10k-2023.pdf', 'start_index': 0}
```

```
</div>
```

```
### Return scores
```

You can return similarity scores with the documents. Score meaning varies by provider. In this case, the score is a distance metric that varies inversely with similarity:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```python
```

```
Note that providers implement different scores; the score here
```

```
is a distance metric that varies inversely with similarity.
```

```
results = vector_store.similarity_search_with_score("What was Nike's revenue in 2023?")
doc, score = results[0]
print(f"Score: {score}\n")
print(doc)
```

```
```python wrap Score: 0.23699893057346344
```

page_content='Table of Contents FISCAL 2023 NIKE BRAND REVENUE HIGHLIGHTS The following tables present NIKE Brand revenues disaggregated by reportable operating segment, distribution channel and major product line: FISCAL 2023 COMPARED TO FISCAL 2022 •NIKE, Inc. Revenues were \$51.2 billion in fiscal 2023, which increased 10% and 16% compared to fiscal 2022 on a reported and currency-neutral basis, respectively. The increase was due to higher revenues in North America, Europe, Middle East & Africa ("EMEA"), APLA and Greater China, which contributed approximately 7, 6, 2 and 1 percentage points to NIKE, Inc. Revenues, respectively. •NIKE Brand revenues, which represented over 90% of NIKE, Inc. Revenues, increased 10% and 16% on a reported and currency-neutral basis, respectively. This increase was primarily due to higher revenues in Men's, the Jordan Brand, Women's and Kids' which grew 17%, 35%,11% and 10%, respectively, on a wholesale equivalent basis.' metadata={'page': 35, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 0}

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```typescript
```

```
const results2 = await vectorStore.similaritySearchWithScore(
 "What was Nike's revenue in 2023?"
);
```

```
console.log(results2[0]);
```

```
Score: 0.23699893057346344
```

```
Document {
 pageContent: 'Table of Contents...',
 metadata: {'page': 35, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 0}
}
```

## Search by vector

Embed the query yourself, then search with the resulting vector:

## python

```
embedding = embeddings.embed_query("How were Nike's margins impacted in 2023?")

results = vector_store.similarity_search_by_vector(embedding)
print(results[0])
```

```
```python wrap page_content='Table of Contents GROSS MARGIN FISCAL 2023 COMPARED TO FISCAL 2022 For fiscal 2023, our consolidated gross profit increased 4% to $22,292 million compared to $21,479 million for fiscal 2022. Gross margin decreased 250 basis points to 43.5% for fiscal 2023 compared to 46.0% for fiscal 2022 due to the following: *Wholesale equivalent The decrease in gross margin for fiscal 2023 was primarily due to: •Higher NIKE Brand product costs, on a wholesale equivalent basis, primarily due to higher input costs and elevated inbound freight and logistics costs as well as product mix; •Lower margin in our NIKE Direct business, driven by higher promotional activity to liquidate inventory in the current period compared to lower promotional activity in the prior period resulting from lower available inventory supply; •Unfavorable changes in net foreign currency exchange rates, including hedges; and •Lower off-price margin, on a wholesale equivalent basis. This was partially offset by:' metadata={'page': 36, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 0}
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript
const embedding = await embeddings.embedQuery(
 "How were Nike's margins impacted in 2023?"
);

const results3 = await vectorStore.similaritySearchVectorWithScore(
 embedding,
 1
);

console.log(results3[0]);
```

```
Document {
 pageContent: 'FISCAL 2023 COMPARED TO FISCAL 2022...',
 metadata: {
 'page': 36,
 'source': '../example_data/nke-10k-2023.pdf',
 'start_index': 0
```

```
}
}
```

Learn more:

- [@\[API Reference\]\[VectorStore\]](#)
- [Integration-specific docs](#)

## Use retrievers

LangChain `@[VectorStore]` objects do not subclass `@[Runnable]`. `@[Retrievers]` are `Runnables`, so they support standard methods such as `sync` and `async invoke` and `batch`.

You can also build retrievers from vector stores, and retrievers can also wrap non-vector sources (such as external APIs).

### python

In this case, create a simple retriever without subclassing `Retriever` by wrapping `similarity_search`:

```
from typing import List

from langchain_core.documents import Document
from langchain_core.runnables import chain

@chain
def retriever(query: str) -> List[Document]:
 return vector_store.similarity_search(query, k=1)

retriever.batch(
 [
 "How many distribution centers does Nike have in the US?",
 "When was Nike incorporated?",
],
)
```

```
```text wrap [[Document(metadata={'page': 4, 'source': '../example_data/nke-10k-2023.pdf',  
'start_index': 3125}, page_content='direct to consumer operations sell products through the  
following number of retail stores in the United States:\nU.S. RETAIL STORES NUMBER\nNIKE  
Brand factory stores 213 \nNIKE Brand in-line stores (including employee-only stores) 74  
\nConverse stores (including factory stores) 82 \nTOTAL 369 \nIn the United States, NIKE has  
eight significant distribution centers. Refer to Item 2. Properties for further information.\n2023  
FORM 10-K 2')], [Document(metadata={'page': 3, 'source': '../example_data/nke-10k-2023.pdf',  
'start_index': 0}, page_content='Table of Contents\nPART I\nITEM 1.  
BUSINESS\nGENERAL\nNIKE, Inc. was incorporated in 1967 under the laws of the State of
```


Oregon. As used in this Annual Report on Form 10-K (this "Annual Report"), the terms "we," "us," "our," "NIKE" and the "Company" refer to NIKE, Inc. and its predecessors, subsidiaries and affiliates, collectively, unless the context indicates otherwise.

Our principal business activity is the design, development and worldwide marketing and selling of athletic footwear, apparel, equipment, accessories and services. NIKE is the largest seller of athletic footwear and apparel in the world. We sell our products through NIKE Direct operations, which are comprised of both NIKE-owned retail stores and sales through our digital platforms (also referred to as "NIKE Brand Digital"), to retail accounts and to a mix of independent distributors, licensees and sales]]

</div>

Vector stores implement an `as_retriever` method that returns a `[VectorStoreRetriever]` (<https://reference.langchain.com/python/langchain-core/vectorstores/base/VectorStoreRetriever>). These retrievers expose `search_type` and `search_kwargs` to select and parameterize the underlying store methods. Replicate the example above with:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

```
```python
retriever = vector_store.as_retriever(
 search_type="similarity",
 search_kwargs={"k": 1},
)

retriever.batch(
 [
 "How many distribution centers does Nike have in the US?",
 "When was Nike incorporated?",
],
)
```

```
```text wrap [[Document(metadata={'page': 4, 'source': '../example_data/nke-10k-2023.pdf',
'start_index': 3125}, page_content='direct to consumer operations sell products through the
following number of retail stores in the United States:\nU.S. RETAIL STORES NUMBER\nNIKE
Brand factory stores 213 \nNIKE Brand in-line stores (including employee-only stores) 74
\nConverse stores (including factory stores) 82 \nTOTAL 369 \n\nIn the United States, NIKE has
eight significant distribution centers. Refer to Item 2. Properties for further information.\n2023
FORM 10-K 2')], [Document(metadata={'page': 3, 'source': '../example_data/nke-10k-2023.pdf',
'start_index': 0}, page_content='Table of Contents\nPART I\nITEM 1.
BUSINESS\nGENERAL\nNIKE, Inc. was incorporated in 1967 under the laws of the State of
Oregon. As used in this Annual Report on Form 10-K (this "Annual Report"), the terms "we,"
"us," "our," "NIKE" and the "Company" refer to NIKE, Inc. and its predecessors, subsidiaries
and affiliates, collectively, unless the context indicates otherwise.\nOur principal business
```

activity is the design, development and worldwide marketing and selling of athletic footwear, apparel, equipment, accessories and services. NIKE is the largest seller of athletic footwear and apparel in the world. We sell our products through NIKE Direct operations, which are comprised of both NIKE-owned retail stores and sales through our digital platforms (also referred to as "NIKE Brand Digital"), to retail accounts and to a mix of independent distributors, licensees and sales']]]

```
`VectorStoreRetriever` supports search types of `"similarity"` (default), `"mmr"` (maximum marginal relevance), and `"similarity_score_threshold"`. Use the last option to filter documents by similarity score.
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```typescript
```

```
const retriever = vectorStore.asRetriever({
 searchType: "mmr",
 searchKwargs: {
 fetchK: 1,
 },
});
```

```
await retriever.batch([
 "When was Nike incorporated?",
 "What was Nike's revenue in 2023?",
]);
```

```
[
 [Document {
 metadata: {'page': 4, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 3125},
 pageContent: 'direct to consumer operations sell products...',
 }],
 [Document {
 metadata: {'page': 3, 'source': '../example_data/nke-10k-2023.pdf', 'start_index': 0},
 pageContent: 'Table of Contents...',
 }],
]
```

You can use retrievers in more complex apps such as [retrieval-augmented generation \(RAG\)](#), which combine a question with retrieved context in a prompt for an LLM. To learn more about building such an application, check out the [RAG tutorial](#) tutorial.

## Next steps

---

You've now seen how to build a semantic search engine over a PDF document.

For more information see:

- [Available embedding integrations](#)
- [Available vector store integrations](#)

For more on RAG:

- [Retrieval overview](#)
- [RAG with Deep Agents](#)
- [Evaluate a RAG application](#)

# Long-term memory

---

langchain/long-term-memory.mdx

Long-term memory lets your agent store and recall information across different conversations and sessions. Unlike [short-term memory](#), which is scoped to a single thread, long-term memory persists across threads and can be recalled at any time.

Long-term memory is built on [LangGraph stores](#), which save data as JSON documents organized by namespace and key.

## Usage

---

To add long-term memory to an agent, create a store and pass it to `@[ create_agent ]`:

### InMemoryStore

python

js

### PostgreSQL

python

```
```bash pip pip install -U langgraph-checkpoint-postgres "psycopg[binary]"
```

```
```bash uv
uv add langgraph-checkpoint-postgres "psycopg[binary]"
```

#### 참고

By default, `langgraph-checkpoint-postgres` installs `psycopg` (Psycopg 3) without extras. The install above adds `psycopg[binary]`, which is recommended for most users. For other options, see the [Psycopg installation docs](#).

js

```
npm install @langchain/langgraph-checkpoint-postgres
```

#### 참고

For other store backends, including Redis and MongoDB, see the [store integrations](#) list. For a MongoDB walkthrough, see [long-term memory with MongoDB](#).

Tools can then read from and write to the store using the `runtime.store` parameter. See [Read long-term memory in tools](#) and [Write long-term memory from tools](#) for examples.

#### 팁

For a deeper dive into memory types (semantic, episodic, procedural) and strategies for writing memories, see the [Memory conceptual guide](#).

## Memory storage

LangGraph stores long-term memories as JSON documents in a [store](#).

Each memory is organized under a custom `namespace` (similar to a folder) and a distinct `key` (like a file name). Namespaces often include user or org IDs or other labels that makes it easier to organize information.

This structure enables hierarchical organization of memories. Cross-namespace searching is then supported through content filters.

### InMemoryStore

```
python
```

```
js
```

### PostgreSQL

```
python
```

```
js
```

For more information about the memory store, see the [Persistence](#) guide.

## Read long-term memory in tools

### InMemoryStore

```
python
```

```
js
```

PostgreSQL

python

js

## Write long-term memory from tools

---

InMemoryStore

python

js

PostgreSQL

python

js

# Messages

---

langchain/messages.mdx

Messages are the fundamental unit of context for models in LangChain. They represent the input and output of models, carrying both the content and metadata needed to represent the state of a conversation when interacting with an LLM.

Messages are objects that contain:

- 

**Role** - Identifies the message type (e.g. `system`, `user`) \*

**Content** - Represents the actual content of the message (like text, images, audio, documents, etc.) \*

**Metadata** - Optional fields such as response information, message IDs, and token usage

LangChain provides a standard message type that works across all model providers, ensuring consistent behavior regardless of the model being called.

## Basic usage

---

The simplest way to use messages is to create message objects and pass them to a model when [invoking](#).

### python

```
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, AIMessage, SystemMessage

model = init_chat_model("gpt-5-nano")

system_msg = SystemMessage("You are a helpful assistant.")
human_msg = HumanMessage("Hello, how are you?")

Use with chat models
messages = [system_msg, human_msg]
response = model.invoke(messages) # Returns AIMessage
```

### js

```
import { initChatModel, HumanMessage, SystemMessage } from "langchain";

const model = await initChatModel("gpt-5-nano");
```

```
const systemMsg = new SystemMessage("You are a helpful assistant.");
const humanMsg = new HumanMessage("Hello, how are you?");

const messages = [systemMsg, humanMsg];
const response = await model.invoke(messages); // Returns AIMessage
```

### 팁

Multi-turn [agents](#) accumulate long message histories. [LangSmith](#) records each turn, tool result, and model response so you can inspect the full conversation. Follow the [tracing quickstart](#) to enable tracing.

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

## Text prompts

Text prompts are strings - ideal for straightforward generation tasks where you don't need to retain conversation history.

### python

```
response = model.invoke("Write a haiku about spring")
```

### js

```
const response = await model.invoke("Write a haiku about spring");
```

**Use text prompts when:** \* You have a single, standalone request \* You don't need conversation history \* You want minimal code complexity

## Message prompts

Alternatively, you can pass in a list of messages to the model by providing a list of message objects.

### python

```
from langchain.messages import SystemMessage, HumanMessage, AIMessage

messages = [
 SystemMessage("You are a poetry expert"),
 HumanMessage("Write a haiku about spring"),
 AIMessage("Cherry blossoms bloom...")
]
```



```
]
response = model.invoke(messages)
```

js

```
import { SystemMessage, HumanMessage, AIMessage } from "langchain";

const messages = [
 new SystemMessage("You are a poetry expert"),
 new HumanMessage("Write a haiku about spring"),
 new AIMessage("Cherry blossoms bloom..."),
];
const response = await model.invoke(messages);
```

**Use message prompts when:** \* Managing multi-turn conversations \* Working with multimodal content (images, audio, files) \* Including system instructions

## Dictionary format

You can also specify messages directly in OpenAI chat completions format.

python

```
messages = [
 {"role": "system", "content": "You are a poetry expert"},
 {"role": "user", "content": "Write a haiku about spring"},
 {"role": "assistant", "content": "Cherry blossoms bloom..."}
]
response = model.invoke(messages)
```

js

```
const messages = [
 { role: "system", content: "You are a poetry expert" },
 { role: "user", content: "Write a haiku about spring" },
 { role: "assistant", content: "Cherry blossoms bloom..." },
];
const response = await model.invoke(messages);
```

## Message types

-

**System message** - Tells the model how to behave and provide context for interactions

---

**Human message** - Represents user input and interactions with the model

---

**AI message** - Responses generated by the model, including text content, tool calls, and metadata

---

Tool message - Represents the outputs of tool calls

## System message

A `@[ SystemMessage ]` represent an initial set of instructions that primes the model's behavior. You can use a system message to set the tone, define the model's role, and establish guidelines for responses.

### python

```
```python Basic instructions system_msg = SystemMessage("You are a helpful coding assistant.")

messages = [ system_msg, HumanMessage("How do I create a REST API?") ] response = model.invoke(messages)
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript Basic instructions
import { SystemMessage, HumanMessage, AIMessage } from "langchain";

const systemMsg = new SystemMessage("You are a helpful coding assistant.");

const messages = [
 systemMsg,
 new HumanMessage("How do I create a REST API?"),
];
const response = await model.invoke(messages);
```

### python

```

python Detailed persona from langchain.messages import SystemMessage, HumanMessage

system_msg = SystemMessage(""" You are a senior Python developer with expertise in web
frameworks. Always provide code examples and explain your reasoning. Be concise but
thorough in your explanations. """)

messages = [system_msg, HumanMessage("How do I create a REST API?")] response =
model.invoke(messages)

```

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript Detailed persona
import { SystemMessage, HumanMessage } from "langchain";

const systemMsg = new SystemMessage(`
You are a senior TypeScript developer with expertise in web frameworks.
Always provide code examples and explain your reasoning.
Be concise but thorough in your explanations.
`);

const messages = [
  systemMsg,
  new HumanMessage("How do I create a REST API?"),
];
const response = await model.invoke(messages);

```

Human message

A `[ HumanMessage ]` represents user input and interactions. They can contain text, images, audio, files, and any other amount of multimodal [content](#).

Text content

python

```

python Message object response = model.invoke([ HumanMessage("What is machine
learning?") ])

```

```

python String shortcut

```

```
# Using a string is a shortcut for a single HumanMessage
response = model.invoke("What is machine learning?")
```

js

```
```typescript Message object const response = await model.invoke([ new
HumanMessage("What is machine learning?"),]);
```

```
```typescript String shortcut
const response = await model.invoke("What is machine learning?");
```

Message metadata

python

```
```python Add metadata human_msg = HumanMessage( content="Hello!", name="alice", #
Optional: identify different users id="msg_123", # Optional: unique identifier for tracing)
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript Add metadata
const humanMsg = new HumanMessage({
  content: "Hello!",
  name: "alice",
  id: "msg_123",
});
```

참고

The `name` field behavior varies by provider—some use it for user identification, others ignore it. To check, refer to the model provider's [reference](#).

AI message

An `@[AIMessage]` represents the output of a model invocation. They can include multimodal data, tool calls, and provider-specific metadata that you can later access.

python

```
response = model.invoke("Explain AI")
print(type(response)) # <class 'langchain.messages.AIMessage'>
```

js

```
const response = await model.invoke("Explain AI");
console.log(typeof response); // AIMessage
```

@[AIMessage] objects are returned by the model when calling it, which contains all of the associated metadata in the response.

Providers weigh/contextualize types of messages differently, which means it is sometimes helpful to manually create a new @[AIMessage] object and insert it into the message history as if it came from the model.

python

```
from langchain.messages import AIMessage, SystemMessage, HumanMessage

# Create an AI message manually (e.g., for conversation history)
ai_msg = AIMessage("I'd be happy to help you with that question!")

# Add to conversation history
messages = [
    SystemMessage("You are a helpful assistant"),
    HumanMessage("Can you help me?"),
    ai_msg, # Insert as if it came from the model
    HumanMessage("Great! What's 2+2?")
]

response = model.invoke(messages)
```

js

```
import { AIMessage, SystemMessage, HumanMessage } from "langchain";

const aiMsg = new AIMessage("I'd be happy to help you with that question!");

const messages = [
    new SystemMessage("You are a helpful assistant"),
    new HumanMessage("Can you help me?"),
    aiMsg, // Insert as if it came from the model
    new HumanMessage("Great! What's 2+2?")
]
```

```
]

const response = await model.invoke(messages);
```

Attributes

python

The text content of the message.

The raw content of the message.

The standardized [content blocks](#) of the message.

The tool calls made by the model.

Empty if no tools are called.

A unique identifier for the message (either automatically generated by LangChain or returned in the provider response)

The usage metadata of the message, which can contain token counts when available.

The response metadata of the message.

js

The text content of the message.

The raw content of the message.

The standardized content blocks of the message. (See [content](#))

The tool calls made by the model.

Empty if no tools are called.

A unique identifier for the message (either automatically generated by LangChain or returned in the provider response)

The usage metadata of the message, which can contain token counts when available. See `@[ UsageMetadata ]`.

The response metadata of the message.

Tool calls

When models make [tool calls](#), they're included in the `@[ AIMessage ]`:

python

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5-nano")

def get_weather(location: str) -> str:
    """Get the weather at a location."""
    ...

model_with_tools = model.bind_tools([get_weather])
response = model_with_tools.invoke("What's the weather in Paris?")

for tool_call in response.tool_calls:
    print(f"Tool: {tool_call['name']}")
    print(f"Args: {tool_call['args']}")
    print(f"ID: {tool_call['id']}")
```

js

```
const modelWithTools = model.bindTools([getWeather]);
const response = await modelWithTools.invoke("What's the weather in Paris?");

for (const toolCall of response.tool_calls) {
    console.log(`Tool: ${toolCall.name}`);
    console.log(`Args: ${toolCall.args}`);
    console.log(`ID: ${toolCall.id}`);
}
```

Other structured data, such as reasoning or citations, can also appear in message [content](#).

Token usage

An `@[AIMessage]` can hold token counts and other usage metadata in its `@[usage_metadata]` `[UsageMetadata]` field:

python

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5-nano")

response = model.invoke("Hello!")
response.usage_metadata
```

```
{'input_tokens': 8,
  'output_tokens': 304,
  'total_tokens': 312,
  'input_token_details': {'audio': 0, 'cache_read': 0},
  'output_token_details': {'audio': 0, 'reasoning': 256}}
```

js

```
import { initChatModel } from "langchain";

const model = await initChatModel("gpt-5-nano");

const response = await model.invoke("Hello!");
console.log(response.usage_metadata);
```

```
{
  "output_tokens": 304,
  "input_tokens": 8,
  "total_tokens": 312,
  "input_token_details": {
    "cache_read": 0
  },
  "output_token_details": {
    "reasoning": 256
  }
}
```

See `@[UsageMetadata]` for details.

Streaming and chunks

During streaming, you'll receive `@[AIMessageChunk]` objects that can be combined into a full message object:

python

```
chunks = []
full_message = None
for chunk in model.stream("Hi"):
    chunks.append(chunk)
    print(chunk.text)
    full_message = chunk if full_message is None else full_message + chunk
```

js


```
import { AIMessageChunk } from "langchain";

let finalChunk: AIMessageChunk | undefined;
for (const chunk of chunks) {
  finalChunk = finalChunk ? finalChunk.concat(chunk) : chunk;
}
```

참고

Learn more: - [Streaming tokens from chat models](#) - [Streaming tokens and/or steps from agents](#)

Tool message

For models that support [tool calling](#), AI messages can contain tool calls. Tool messages are used to pass the results of a single tool execution back to the model.

[Tools](#) can generate `@[ToolMessage]` objects directly. Below, we show a simple example. Read more in the [tools guide](#).

python

```
from langchain.messages import AIMessage
from langchain.messages import ToolMessage

# After a model makes a tool call
# (Here, we demonstrate manually creating the messages for brevity)
ai_message = AIMessage(
    content=[],
    tool_calls=[{
        "name": "get_weather",
        "args": {"location": "San Francisco"},
        "id": "call_123"
    }]
)

# Execute tool and create result message
weather_result = "Sunny, 72°F"
tool_message = ToolMessage(
    content=weather_result,
    tool_call_id="call_123" # Must match the call ID
)

# Continue conversation
messages = [
    HumanMessage("What's the weather in San Francisco?"),
    ai_message, # Model's tool call
    tool_message
```

```
    tool_message, # Tool execution result
  ]
  response = model.invoke(messages) # Model processes the result
```

js

```
import { AIMessage, ToolMessage } from "langchain";

const aiMessage = new AIMessage({
  content: [],
  tool_calls: [{
    name: "get_weather",
    args: { location: "San Francisco" },
    id: "call_123"
  }]
});

const toolMessage = new ToolMessage({
  content: "Sunny, 72°F",
  tool_call_id: "call_123"
});

const messages = [
  new HumanMessage("What's the weather in San Francisco?"),
  aiMessage, // Model's tool call
  toolMessage, // Tool execution result
];

const response = await model.invoke(messages); // Model processes the result
```

Attributes

The stringified output of the tool call.

The ID of the tool call that this message is responding to. Must match the ID of the tool call in the `@[AIMessage]`.

The name of the tool that was called.

Additional data not sent to the model but can be accessed programmatically.

참고

The `artifact` field stores supplementary data that won't be sent to the model but can be accessed programmatically. This is useful for storing raw results, debugging information, or data for downstream processing without cluttering the model's context.

Example: Using artifact for retrieval metadata

For example, a [retrieval](#) tool could retrieve a passage from a document for reference by a model. Where message `content` contains text that the model will reference, an `artifact` can contain document identifiers or other metadata that an application can use (e.g., to render a page). See example below:

python

```
from langchain.messages import ToolMessage

# Sent to model
message_content = "It was the best of times, it was the worst of times."

# Artifact available downstream
artifact = {"document_id": "doc_123", "page": 0}

tool_message = ToolMessage(
    content=message_content,
    tool_call_id="call_123",
    name="search_books",
    artifact=artifact,
)
```

js

```
import { ToolMessage } from "langchain";

// Artifact available downstream
const artifact = { document_id: "doc_123", page: 0 };

const toolMessage = new ToolMessage({
  content: "It was the best of times, it was the worst of times.",
  tool_call_id: "call_123",
  name: "search_books",
  artifact
});
```

See the [RAG tutorial](#) for an end-to-end example of building retrieval [agents](#) with LangChain.

Message content

You can think of a message's content as the payload of data that gets sent to the model. Messages have a `content` attribute that is loosely-typed, supporting strings and lists of untyped objects (e.g., dictionaries). This allows support for provider-native structures directly in LangChain chat models, such as [multimodal](#) content and other data.

Separately, LangChain provides dedicated content types for text, reasoning, citations, multi-modal data, server-side tool calls, and other message content. See [content blocks](#) below.

LangChain chat models accept message content in the `content` attribute.

This may contain either:

1. A string
2. A list of content blocks in a provider-native format
3. A list of [LangChain's standard content blocks](#)

See below for an example using [multimodal](#) inputs:

python

```
from langchain.messages import HumanMessage

# String content
human_message = HumanMessage("Hello, how are you?")

# Provider-native format (e.g., OpenAI)
human_message = HumanMessage(content=[
    {"type": "text", "text": "Hello, how are you?"},
    {"type": "image_url", "image_url": {"url": "https://example.com/image.jpg"}}
])

# List of standard content blocks
human_message = HumanMessage(content_blocks=[
    {"type": "text", "text": "Hello, how are you?"},
    {"type": "image", "url": "https://example.com/image.jpg"},
])
```

팁

Specifying `content_blocks` when initializing a message will still populate message `content`, but provides a type-safe interface for doing so.

js

```
import { HumanMessage } from "langchain";

// String content
const humanMessage = new HumanMessage("Hello, how are you?");

// Provider-native format (e.g., OpenAI)
const humanMessage = new HumanMessage({
  content: [
```

```

    { type: "text", text: "Hello, how are you?" },
    {
      type: "image_url",
      image_url: { url: "https://example.com/image.jpg" },
    },
  ],
});

// List of standard content blocks
const humanMessage = new HumanMessage({
  contentBlocks: [
    { type: "text", text: "Hello, how are you?" },
    { type: "image", url: "https://example.com/image.jpg" },
  ],
});

```

Standard content blocks

LangChain provides a standard representation for message content that works across providers.

python

Message objects implement a `content_blocks` property that will lazily parse the `content` attribute into a standard, type-safe representation. For example, messages generated from [ChatAnthropic](#) or [ChatOpenAI](#) will include `thinking` or `reasoning` blocks in the format of the respective provider, but can be lazily parsed into a consistent [ReasoningContentBlock](#) representation:

Anthropic

```

from langchain.messages import AIMessage

message = AIMessage(
    content=[
        {"type": "thinking", "thinking": "...", "signature": "WaUjzkyp..."},
        {"type": "text", "text": "..."},
    ],
    response_metadata={"model_provider": "anthropic"}
)
message.content_blocks

```

```

[{'type': 'reasoning',
  'reasoning': '...',
  'extras': {'signature': 'WaUjzkyp...'}},
 {'type': 'text', 'text': '...'}]

```

OpenAI

```

from langchain.messages import AIMessage

message = AIMessage(
    content=[
        {
            "type": "reasoning",
            "id": "rs_abc123",
            "summary": [
                {"type": "summary_text", "text": "summary 1"},
                {"type": "summary_text", "text": "summary 2"},
            ],
        },
        {"type": "text", "text": "...", "id": "msg_abc123"},
    ],
    response_metadata={"model_provider": "openai"}
)
message.content_blocks

```

```

[{'type': 'reasoning', 'id': 'rs_abc123', 'reasoning': 'summary 1'},
 {'type': 'reasoning', 'id': 'rs_abc123', 'reasoning': 'summary 2'},
 {'type': 'text', 'text': '...', 'id': 'msg_abc123'}]

```

js

Message objects implement a `contentBlocks` property that will lazily parse the `content` attribute into a standard, type-safe representation. For example, messages generated from [ChatAnthropic](#) or [ChatOpenAI](#) will include `thinking` or `reasoning` blocks in the format of the respective provider, but can be lazily parsed into a consistent [ReasoningContentBlock](#) representation:

Anthropic

```

import { AIMessage } from "@langchain/core/messages";

const message = new AIMessage({
  content: [
    {
      "type": "thinking",
      "thinking": "...",
      "signature": "WaUjzkyp...",
    },
    {
      "type": "text",
      "text": "...",
      "id": "msg_abc123",
    },
  ],
  response_metadata: { model_provider: "anthropic" },
});

```

```
});  
  
console.log(message.contentBlocks);
```

OpenAI

```
import { AIMessage } from "@langchain/core/messages";  
  
const message = new AIMessage({  
  content: [  
    {  
      "type": "reasoning",  
      "id": "rs_abc123",  
      "summary": [  
        {"type": "summary_text", "text": "summary 1"},  
        {"type": "summary_text", "text": "summary 2"},  
      ],  
    },  
    {"type": "text", "text": "..."},  
  ],  
  response_metadata: { model_provider: "openai" },  
});  
  
console.log(message.contentBlocks);
```

See the [integrations guides](#) to get started with the inference provider of your choice.

참고

Serializing standard content

If an application outside of LangChain needs access to the standard content block representation, you can opt-in to storing content blocks in message content.

python

To do this, you can set the `LC_OUTPUT_VERSION` environment variable to `v1`. Or, initialize any chat model with `output_version="v1"`:

```
from langchain.chat_models import init_chat_model  
  
model = init_chat_model("gpt-5-nano", output_version="v1")
```

js

To do this, you can set the `LC_OUTPUT_VERSION` environment variable to `v1`. Or, initialize any chat model with `outputVersion: "v1"`:

```
import { initChatModel } from "langchain";

const model = await initChatModel(
  "gpt-5-nano",
  { outputVersion: "v1" }
);
```

Multimodal

Multimodality refers to the ability to work with data that comes in different forms, such as text, audio, images, and video. LangChain includes standard types for these data that can be used across providers.

[Chat models](#) can accept multimodal data as input and generate it as output. Below we show short examples of input messages featuring multimodal data.

참고

Extra keys can be included top-level in the content block or nested in `"extras": { "key": value }`.

[OpenAI](#), for example, requires a filename for PDFs. See the [provider page](#) for your chosen model for specifics.

python

```python Image input

## From URL

```
message = { "role": "user", "content": [{"type": "text", "text": "Describe the content of this image."}, {"type": "image", "url": "https://example.com/path/to/image.jpg"},] }
```

## From base64 data

```
message = { "role": "user", "content": [{"type": "text", "text": "Describe the content of this image."}, {"type": "image", "base64": "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...", "mime_type": "image/jpeg"},], }
```



# From provider-managed File ID

---

```
message = { "role": "user", "content": [{"type": "text", "text": "Describe the content of this image."}, {"type": "image", "file_id": "file-abc123"},] }
```

```
```python PDF document input
# From URL
message = {
    "role": "user",
    "content": [
        {"type": "text", "text": "Describe the content of this document."},
        {"type": "file", "url": "https://example.com/path/to/document.pdf"},
    ]
}

# From base64 data
message = {
    "role": "user",
    "content": [
        {"type": "text", "text": "Describe the content of this document."},
        {
            "type": "file",
            "base64": "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...",
            "mime_type": "application/pdf",
        },
    ]
}

# From provider-managed File ID
message = {
    "role": "user",
    "content": [
        {"type": "text", "text": "Describe the content of this document."},
        {"type": "file", "file_id": "file-abc123"},
    ]
}
```

```
```python Audio input
```

## From base64 data

---

```
message = { "role": "user", "content": [{"type": "text", "text": "Describe the content of this audio."}, {"type": "audio", "base64": "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...", "mime_type": "audio/wav"},] }
```

# From provider-managed File ID

```
message = { "role": "user", "content": [{"type": "text", "text": "Describe the content of this audio."}, {"type": "audio", "file_id": "file-abc123"},] }
```

```
```python Video input
# From base64 data
message = {
    "role": "user",
    "content": [
        {"type": "text", "text": "Describe the content of this video."},
        {
            "type": "video",
            "base64": "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...",
            "mime_type": "video/mp4",
        },
    ],
}

# From provider-managed File ID
message = {
    "role": "user",
    "content": [
        {"type": "text", "text": "Describe the content of this video."},
        {"type": "video", "file_id": "file-abc123"},
    ],
}
```

js

```
```typescript Image input // From URL const message = new HumanMessage({ content: [ { type: "text", text: "Describe the content of this image." }, { type: "image", source_type: "url", url: "https://example.com/path/to/image.jpg" }, ], });
```

```
// From base64 data const message = new HumanMessage({ content: [{ type: "text", text: "Describe the content of this image." }, { type: "image", source_type: "base64", mime_type: "image/jpeg", data: "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...", },], });
```

```
// From provider-managed File ID const message = new HumanMessage({ content: [{ type: "text", text: "Describe the content of this image." }, { type: "image", source_type: "id", id: "file-abc123" },], });
```

```
```typescript PDF document input
// From URL
```

```

const message = new HumanMessage({
  content: [
    { type: "text", text: "Describe the content of this document." },
    { type: "file", source_type: "url", url: "https://example.com/path/to/document.pdf", mime_type: "application/pdf" },
  ],
});

// From base64 data
const message = new HumanMessage({
  content: [
    { type: "text", text: "Describe the content of this document." },
    {
      type: "file",
      source_type: "base64",
      data: "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...",
      mime_type: "application/pdf",
    },
  ],
});

// From provider-managed File ID
const message = new HumanMessage({
  content: [
    { type: "text", text: "Describe the content of this document." },
    { type: "file", source_type: "id", id: "file-abc123" },
  ],
});

```

```

```typescript Audio input // From base64 data
const message = new HumanMessage({ content:
[{ type: "text", text: "Describe the content of this audio." }, { type: "audio", source_type:
"base64", data: "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...", },], });

```

```

// From provider-managed File ID
const message = new HumanMessage({ content: [{ type:
"text", text: "Describe the content of this audio." }, { type: "audio", source_type: "id", id: "file-
abc123" },], });

```

```

```typescript Video input
// From base64 data
const message = new HumanMessage({
  content: [
    { type: "text", text: "Describe the content of this video." },
    {
      type: "video",
      source_type: "base64",
      data: "AAAAIGZ0eXBtcDQyAAAAAGlzb21tcDQyAAACAGlzb2...",
    },
  ],
});

```

```
});

// From provider-managed File ID
const message = new HumanMessage({
  content: [
    { type: "text", text: "Describe the content of this video." },
    { type: "video", source_type: "id", id: "file-abc123" },
  ],
});
```

주의

Not all models support all file types. Check the model provider's [reference](#) for supported formats and size limits.

Content block reference

python

Content blocks are represented (either when creating a message or accessing the `content_blocks` property) as a list of typed dictionaries. Each item in the list must adhere to one of the following block types:

Core

TextContentBlock

Purpose: Standard text output

Always `"text"`

The text content

List of annotations for the text

Additional provider-specific data

Example:

```
{
  "type": "text",
  "text": "Hello world",
  "annotations": []
}
```

ReasoningContentBlock

Purpose: Model reasoning steps

Always "reasoning"

The reasoning content

Additional provider-specific data

Example:

```
{
  "type": "reasoning",
  "reasoning": "The user is asking about...",
  "extras": {"signature": "abc123"},
}
```

Multimodal

ImageContentBlock

Purpose: Image data

Always "image"

URL pointing to the image location.

Base64-encoded image data.

Unique identifier for this content block (either generated by the provider or by LangChain).

Image [MIME type](#) (e.g., image/jpeg, image/png). Required for base64 data.

AudioContentBlock

Purpose: Audio data

Always "audio"

URL pointing to the audio location.

Base64-encoded audio data.

Unique identifier for this content block (either generated by the provider or by LangChain).

Audio [MIME type](#) (e.g., audio/mpeg, audio/wav). Required for base64 data.

VideoContentBlock

Purpose: Video data

Always "video"

URL pointing to the video location.

Base64-encoded video data.

Unique identifier for this content block (either generated by the provider or by LangChain).

Video [MIME type](#) (e.g., `video/mp4`, `video/webm`). Required for base64 data.

FileContentBlock

Purpose: Generic files (PDF, etc)

Always `"file"`

URL pointing to the file location.

Base64-encoded file data.

Unique identifier for this content block (either generated by the provider or by LangChain).

File [MIME type](#) (e.g., `application/pdf`). Required for base64 data.

PlainTextContentBlock

Purpose: Document text (`.txt`, `.md`)

Always `"text-plain"`

The text content

[MIME type](#) of the text (e.g., `text/plain`, `text/markdown`)

Tool Calling

ToolCall

Purpose: Function calls

Always `"tool_call"`

Name of the tool to call

Arguments to pass to the tool

Unique identifier for this tool call

Example:

```
{
  "type": "tool_call",
  "name": "search",
  "args": {"query": "weather"},
  "id": "call_123"
}
```

ToolCallChunk

Purpose: Streaming tool call fragments

Always `"tool_call_chunk"`

Name of the tool being called

Partial tool arguments (may be incomplete JSON)

Tool call identifier

Position of this chunk in the stream

InvalidToolCall

Purpose: Malformed calls, intended to catch JSON parsing errors.

Always `"invalid_tool_call"`

Name of the tool that failed to be called

Arguments to pass to the tool

Description of what went wrong

Server-Side Tool Execution

ServerToolCall

Purpose: Tool call that is executed server-side.

Always `"server_tool_call"`

An identifier associated with the tool call.

The name of the tool to be called.

Partial tool arguments (may be incomplete JSON)

ServerToolCallChunk

Purpose: Streaming server-side tool call fragments

Always `"server_tool_call_chunk"`

An identifier associated with the tool call.

Name of the tool being called

Partial tool arguments (may be incomplete JSON)

Position of this chunk in the stream

ServerToolResult

Purpose: Search results

Always `"server_tool_result"`

Identifier of the corresponding server tool call.

Identifier associated with the server tool result.

Execution status of the server-side tool. `"success"` or `"error"`.

Output of the executed tool.

Provider-Specific Blocks

NonStandardContentBlock

Purpose: Provider-specific escape hatch

Always `"non_standard"`

Provider-specific data structure

Usage: For experimental or provider-unique features

Additional provider-specific content types may be found within the [reference documentation](#) of each model provider.

js

Content blocks are represented (either when creating a message or accessing the `contentBlocks` field) as a list of typed objects. Each item in the list must adhere to one of the following block types:

Core

ContentBlock.Text

Purpose: Standard text output

Always `"text"`

The text content

List of annotations for the text

Example:

```
{  
  type: "text",
```



```
text: "Hello world",
annotations: []
}
```

ContentBlock.Reasoning

Purpose: Model reasoning steps

Always "reasoning"

The reasoning content

Example:

```
{
  type: "reasoning",
  reasoning: "The user is asking about..."
}
```

Multimodal

ContentBlock.Multimodal.Image

Purpose: Image data

Always "image"

URL pointing to the image location.

Base64-encoded image data.

Reference to the image in an external file storage system (e.g., OpenAI or Anthropic's Files API).

Image [MIME type](#) (e.g., `image/jpeg`, `image/png`). Required for base64 data.

ContentBlock.Multimodal.Audio

Purpose: Audio data

Always "audio"

URL pointing to the audio location.

Base64-encoded audio data.

Reference to the audio file in an external file storage system (e.g., OpenAI or Anthropic's Files API).

Audio [MIME type](#) (e.g., `audio/mpeg`, `audio/wav`). Required for base64 data.

ContentBlock.Multimodal.Video

Purpose: Video data

Always "video"

URL pointing to the video location.

Base64-encoded video data.

Reference to the video file in an external file storage system (e.g., OpenAI or Anthropic's Files API).

Video [MIME type](#) (e.g., video/mp4, video/webm). Required for base64 data.

ContentBlock.Multimodal.File

Purpose: Generic files (PDF, etc)

Always "file"

URL pointing to the file location.

Base64-encoded file data.

Reference to the file in an external file storage system (e.g., OpenAI or Anthropic's Files API).

File [MIME type](#) (e.g., application/pdf). Required for base64 data.

ContentBlock.Multimodal.PlainText

Purpose: Document text (.txt , .md)

Always "text-plain"

The text content

Title of the text content

[MIME type](#) of the text (e.g., text/plain, text/markdown)

Tool Calling

ContentBlock.Tools.ToolCall

Purpose: Function calls

Always "tool_call"

Name of the tool to call

Arguments to pass to the tool

Unique identifier for this tool call

Example:

```
{
  type: "tool_call",
  name: "search",
  args: { query: "weather" },
  id: "call_123"
}
```

ContentBlock.Tools.ToolCallChunk

Purpose: Streaming tool fragments

Always `"tool_call_chunk"`

Name of the tool being called

Partial tool arguments (may be incomplete JSON)

Tool call identifier

Position of this chunk in the stream

ContentBlock.Tools.InvalidToolCall

Purpose: Malformed calls

Always `"invalid_tool_call"`

Name of the tool that failed to be called

Raw arguments that failed to parse

Description of what went wrong

Common errors: Invalid JSON, missing required fields

Server-Side Tool Execution

ContentBlock.Tools.ServerToolCall

Purpose: Tool call that is executed server-side.

Always `"server_tool_call"`

An identifier associated with the tool call.

The name of the tool to be called.

Partial tool arguments (may be incomplete JSON)

ContentBlock.Tools.ServerToolCallChunk

Purpose: Streaming server-side tool call fragments

Always `"server_tool_call_chunk"`

An identifier associated with the tool call.

Name of the tool being called

Partial tool arguments (may be incomplete JSON)

Position of this chunk in the stream

ContentBlock.Tools.ServerToolResult

Purpose: Search results

Always `"server_tool_result"`

Identifier of the corresponding server tool call.

Identifier associated with the server tool result.

Execution status of the server-side tool. `"success"` or `"error"`.

Output of the executed tool.

Provider-Specific Blocks

ContentBlock.NonStandard

Purpose: Provider-specific escape hatch

Always `"non_standard"`

Provider-specific data structure

Usage: For experimental or provider-unique features

Additional provider-specific content types may be found within the [reference documentation](#) of each model provider.

Each of these content blocks mentioned above are individually addressable as types when importing the `@[ContentBlock]` type.

```
import { ContentBlock } from "langchain";

// Text block
const textBlock: ContentBlock.Text = {
  type: "text",
  text: "Hello world",
}
```

```
// Image block
const imageBlock: ContentBlock.Multimodal.Image = {
  type: "image",
  url: "https://example.com/image.png",
  mimeType: "image/png",
}
```

팁

View the canonical type definitions in the [@\[API reference\]\[langchain.messages\]](#).

정보

Content blocks were introduced as a new property on messages in LangChain v1 to standardize content formats across providers while maintaining backward compatibility with existing code.

Content blocks are not a replacement for the `@[ content ][BaseMessage(content)]` property, but rather a new property that can be used to access the content of a message in a standardized format.

Serialization

You can serialize messages to plain objects for storage and deserialize back to message instances. This is useful for persisting conversation history and resuming sessions.

python

js

python

주의

`load()` instantiates Python objects and can trigger side effects during deserialization. Never call `load()` on data from an untrusted or unauthenticated source.

js

주의

`load()` **deserializes data by instantiating classes and invoking constructors. Never call `load()` on untrusted or user-supplied input.** Only deserialize data that originates from a source you control, such as your own database.

Use with chat models

[Chat models](#) accept a sequence of message objects as input and return an `@[AIMessage]` as output. Interactions are often stateless, so that a simple conversational loop involves invoking a model with a growing list of messages.

Refer to the below guides to learn more:

- Built-in features for [persisting and managing conversation histories](#)
- Strategies for managing context windows, including [trimming and summarizing messages](#)

Models

langchain/models.mdx

[LLMs](#) are powerful AI tools that can interpret and generate text like humans. They're versatile enough to write content, translate languages, summarize, and answer questions without needing specialized training for each task.

In addition to text generation, many models support:

-

[Tool calling](#) - calling external tools (like databases queries or API calls) and use results in their responses. *

[Structured output](#) - where the model's response is constrained to follow a defined format. *

[Multimodality](#) - process and return data other than text, such as images, audio, and video. *

[Reasoning](#) - models perform multi-step reasoning to arrive at a conclusion.

Models are the reasoning engine of [agents](#). They drive the agent's decision-making process, determining which tools to call, how to interpret results, and when to provide a final answer.

The quality and capabilities of the model you choose directly impact your agent's baseline reliability and performance. Different models excel at different tasks - some are better at following complex instructions, others at structured reasoning, and some support larger context windows for handling more information.

LangChain's standard model interfaces give you access to many different provider integrations, which makes it easy to experiment with and switch between models to find the best fit for your use case.

For provider-specific integration information and capabilities, see the provider's [chat model page](#).

팁

[LangSmith](#) traces each model call so you can compare providers, inspect tool routing, and debug failures. Follow the [tracing quickstart](#) to get set up.

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Basic usage

Models can be utilized in two ways:

1. **With agents** - Models can be dynamically specified when creating an [agent](#).
2. **Standalone** - Models can be called directly (outside of the agent loop) for tasks like text generation, classification, or extraction without the need for an agent framework.

The same model interface works in both contexts, which gives you the flexibility to start simple and scale up to more complex agent-based workflows as needed.

Initialize a model

python

The easiest way to get started with a standalone model in LangChain is to use `@[ init_chat_model ]` to initialize one from a chat model provider of your choice (examples below):

```
response = model.invoke("Why do parrots talk?")
```

See `@[ init_chat_model ]` for more detail, including information on how to pass model [parameters](#).

js

The easiest way to get started with a standalone model in LangChain is to use `initChatModel` to initialize one from a [chat model provider](#) of your choice (examples below):

```
const response = await model.invoke("Why do parrots talk?");
```

See `@[ initChatModel ]` for more detail, including information on how to pass model [parameters](#).

Supported providers and models

LangChain supports all major model providers through dedicated integration packages. Each provider package implements the same standard interface, so you can swap providers without rewriting application logic. New model names work immediately — no LangChain update required — because provider packages pass model names directly to the provider's API.

Browse the [full list of supported providers](#), or see [Providers and models](#) for a conceptual overview of how providers, packages, and model names work together in LangChain.

Key methods

[Invoke](#)

The model takes messages as input and outputs messages after generating a complete response.

[Stream](#)

Invoke the model, but stream the output as it is generated in real-time.

[Batch](#)

Send multiple requests to a model in a batch for more efficient processing.

정보

In addition to chat models, LangChain provides support for other adjacent technologies, such as embedding models and vector stores. See the [integrations page](#) for details.

Parameters

A chat model takes parameters that can be used to configure its behavior. The full set of supported parameters varies by model and provider, but standard ones include:

The name or identifier of the specific model you want to use with a provider. You can also specify both the model and its provider in a single argument using the '{model_provider}:{model}' format, for example, 'openai:o1'.

python

The key required for authenticating with the model's provider. This is usually issued when you sign up for access to the model. Often accessed by setting an

environment variable

.

js

The key required for authenticating with the model's provider. This is usually issued when you sign up for access to the model. Often accessed by setting an

environment variable

.

Controls the randomness of the model's output. A higher number makes responses more creative; lower ones make them more deterministic.

python

Limits the total number of

tokens

in the response, effectively controlling how long the output can be.

js

Limits the total number of

tokens

in the response, effectively controlling how long the output can be.

The maximum time (in seconds) to wait for a response from the model before canceling the request.

python

The maximum number of attempts the system will make to resend a request if it fails due to issues like network timeouts or rate limits. Retries use exponential backoff with jitter. Network errors, rate limits (429), and server errors (5xx) are retried automatically. Client errors such as 401 (unauthorized) or 404 are not retried. For long-running [agent](#) tasks on unreliable networks, consider increasing this to 10–15.

js

The maximum number of attempts the system will make to resend a request if it fails due to issues like network timeouts or rate limits. Retries use exponential backoff with jitter. Network errors, rate limits (429), and server errors (5xx) are retried automatically. Client errors such as 401 (unauthorized) or 404 are not retried. For long-running [agent](#) tasks on unreliable networks, consider increasing this to 10–15.

On providers that support it, [model retry middleware](#) takes over retries for the calls it wraps, and this value does not apply to them.

python

Using `@[init_chat_model]`, pass these parameters as inline

```
**kwargs
```

```
:
```

```
```python Initialize using model parameters model = init_chat_model( "claude-sonnet-4-6", #
Kwargs passed to the model: temperature=0.7, timeout=30, max_tokens=1000, max_retries=6, #
Default; increase for unreliable networks)
```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

Using `initChatModel`, pass these parameters as inline parameters:

```
```typescript Initialize using model parameters
const model = await initChatModel(
  "claude-sonnet-4-6",
  { temperature: 0.7, timeout: 30, maxTokens: 1000, maxRetries: 6 }
)
```

Connection resilience

LangChain chat models automatically retry failed API requests with exponential backoff. By default, models retry up to **6 times** for network errors, rate limits (429), and server errors (5xx). Client errors like 401 (unauthorized) or 404 are not retried.

python

You can adjust `max_retries` and `timeout` when creating a model, then pass that instance to `create_agent`, `create_deep_agent`, or call it standalone:

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "google_genai:gemin-3.6-flash",
    max_retries=10, # Increase for unreliable networks (default: 6)
    timeout=120, # Seconds; increase for slow connections
)
```

js

You can adjust `maxRetries` and `timeout` when creating a model, then pass that instance to `createAgent`, `createDeepAgent`, or call it standalone:

```
import { initChatModel } from "langchain";

const model = await initChatModel("google:gemin-3.6-flash", {
```

```
maxRetries: 10, // Increase for unreliable networks (default: 6)
timeout: 120_000, // Milliseconds; increase for slow connections
});
```

팁

For long-running agent graphs on unreliable networks, consider higher `max_retries` (for example 10–15) and a [checkpointer](#) so that progress is preserved across failures.

정보

Each chat model integration may have additional params used to control provider-specific functionality.

For example, `@[ ChatOpenAI ]` has `use_responses_api` to dictate whether to use the OpenAI Responses or Completions API.

To find all the parameters supported by a given chat model, head to the [chat model integrations](#) page.

Invocation

A chat model must be invoked to generate an output. There are three primary invocation methods, each suited to different use cases.

Invoke

The most straightforward way to call a model is to use `@[ invoke() ]` `[BaseChatModel.invoke]` with a single message or a list of messages.

python

```
```python Single message response = model.invoke("Why do parrots have colorful feathers?")
print(response)
```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```
```typescript Single message
```

```
const response = await model.invoke("Why do parrots have colorful feathers?");
console.log(response);
```

A list of messages can be provided to a chat model to represent conversation history. Each message has a role that models use to indicate who sent the message in the conversation.

See the [messages](#) guide for more detail on roles, types, and content.

python

```
```python Dictionary format conversation = [{"role": "system", "content": "You are a helpful assistant that translates English to French."}, {"role": "user", "content": "Translate: I love programming."}, {"role": "assistant", "content": "J'adore la programmation."}, {"role": "user", "content": "Translate: I love building applications."}]
```

```
response = model.invoke(conversation) print(response) # AIMessage("J'adore créer des applications.")
```

```
```python Message objects
from langchain.messages import HumanMessage, AIMessage, SystemMessage

conversation = [
    SystemMessage("You are a helpful assistant that translates English to French."),
    HumanMessage("Translate: I love programming."),
    AIMessage("J'adore la programmation."),
    HumanMessage("Translate: I love building applications.")
]

response = model.invoke(conversation)
print(response) # AIMessage("J'adore créer des applications.")
```

js

```
```typescript Object format const conversation = [ { role: "system", content: "You are a helpful assistant that translates English to French." }, { role: "user", content: "Translate: I love programming." }, { role: "assistant", content: "J'adore la programmation." }, { role: "user", content: "Translate: I love building applications." }, ];
```

```
const response = await model.invoke(conversation); console.log(response); //
AIMessage("J'adore créer des applications.")
```

```
```typescript Message objects
import { HumanMessage, AIMessage, SystemMessage } from "langchain";

const conversation = [
    new SystemMessage("You are a helpful assistant that translates English to French."),
```

```

    new HumanMessage("Translate: I love programming."),
    new AIMessage("J'adore la programmation."),
    new HumanMessage("Translate: I love building applications."),
  ];

  const response = await model.invoke(conversation);
  console.log(response); // AIMessage("J'adore créer des applications.")

```

정보

If the return type of your invocation is a string, ensure that you are using a chat model as opposed to an LLM. Legacy, text-completion LLMs return strings directly. LangChain chat models are prefixed with "Chat", e.g., @ [ChatOpenAI](#) .

Stream

Most models can stream their output content while it is being generated. By displaying output progressively, streaming significantly improves user experience, particularly for longer responses.

Calling @`stream()` [`BaseChatModel.stream`] returns an

iterator

that yields output chunks as they are produced. You can use a loop to process each chunk in real-time:

python

```

```python Basic text streaming for chunk in model.stream("Why do parrots have colorful
feathers?"): print(chunk.text, end="|", flush=True)

```

```

```python Stream tool calls, reasoning, and other content
for chunk in model.stream("What color is the sky?"):
    for block in chunk.content_blocks:
        if block["type"] == "reasoning" and (reasoning := block.get("reasoning")):
            print(f"Reasoning: {reasoning}")
        elif block["type"] == "tool_call_chunk":
            print(f"Tool call chunk: {block}")
        elif block["type"] == "text":
            print(block["text"])
        else:
            ...

```

js

```
```typescript Basic text streaming const stream = await model.stream("Why do parrots have colorful feathers?"); for await (const chunk of stream) { console.log(chunk.text) }
```

```
```typescript Stream tool calls, reasoning, and other content
const stream = await model.stream("What color is the sky?");
for await (const chunk of stream) {
  for (const block of chunk.contentBlocks) {
    if (block.type === "reasoning") {
      console.log(`Reasoning: ${block.reasoning}`);
    } else if (block.type === "tool_call_chunk") {
      console.log(`Tool call chunk: ${block}`);
    } else if (block.type === "text") {
      console.log(block.text);
    } else {
      ...
    }
  }
}
}
```

As opposed to [invoke\(\)](#), which returns a single `@[AIMessage][AIMessage]` after the model has finished generating its full response, `stream()` returns multiple `@[AIMessageChunk][AIMessageChunk]` objects, each containing a portion of the output text. Importantly, each chunk in a stream is designed to be gathered into a full message via summation:

python

```
```python Construct an AIMessage full = None # None | AIMessageChunk for chunk in
model.stream("What color is the sky?"): full = chunk if full is None else full + chunk
print(full.text)
```

The

---

The sky

---

The sky is

---

The sky is typically

---

The sky is typically blue

---

...

---

print(full.content\_blocks)

[{"type": "text", "text": "The sky is typically blue..."}]

---

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript Construct AIMessage
let full: AIMessageChunk | null = null;
for await (const chunk of stream) {
  full = full ? full.concat(chunk) : chunk;
  console.log(full.text);
}

// The
// The sky
// The sky is
// The sky is typically
// The sky is typically blue
// ...
```



```
console.log(full.contentBlocks);  
// [{"type": "text", "text": "The sky is typically blue..."}]
```

The resulting message can be treated the same as a message that was generated with [invoke\(\)](#)—for example, it can be aggregated into a message history and passed back to the model as conversational context.

주의

Streaming only works if all steps in the program know how to process a stream of chunks. For instance, an application that isn't streaming-capable would be one that needs to store the entire output in memory before it can be processed.

Advanced streaming topics

Streaming events

python

LangChain chat models can also stream semantic events using `astream_events()`.

This simplifies filtering based on event types and other metadata, and will aggregate the full message in the background. See below for an example.

```
async for event in model.astream_events("Hello"):  
  
    if event["event"] == "on_chat_model_start":  
        print(f"Input: {event['data']['input']}")  
  
    elif event["event"] == "on_chat_model_stream":  
        print(f"Token: {event['data']['chunk'].text}")  
  
    elif event["event"] == "on_chat_model_end":  
        print(f"Full message: {event['data']['output'].text}")  
  
    else:  
        pass
```

```
Input: Hello  
Token: Hi  
Token:  there  
Token: !  
Token:  How  
Token:  can  
Token:  I
```

...
Full message: Hi there! How can I help today?

팁

See the `@[ astream_events() ]` `[BaseChatModel.astream_events]` reference for event types and other details.

js

LangChain chat models can also stream semantic events using `[ streamEvents() ]` `[BaseChatModel.streamEvents]`.

This simplifies filtering based on event types and other metadata, and will aggregate the full message in the background. See below for an example.

```
const stream = await model.streamEvents("Hello");
for await (const event of stream) {
  if (event.event === "on_chat_model_start") {
    console.log(`Input: ${event.data.input}`);
  }
  if (event.event === "on_chat_model_stream") {
    console.log(`Token: ${event.data.chunk.text}`);
  }
  if (event.event === "on_chat_model_end") {
    console.log(`Full message: ${event.data.output.text}`);
  }
}
```

```
Input: Hello
Token: Hi
Token:  there
Token: !
Token:  How
Token:  can
Token:  I
...
Full message: Hi there! How can I help today?
```

See the `@[ streamEvents() ]` `[BaseChatModel.streamEvents]` reference for event types and other details.

"Auto-streaming" chat models

LangChain simplifies streaming from chat models by automatically enabling streaming mode in certain cases, even when you're not explicitly calling the streaming methods. This is particularly

useful when you use the non-streaming `invoke` method but still want to stream the entire application, including intermediate results from the chat model.

In [LangGraph agents](#), for example, you can call `model.invoke()` within nodes, but LangChain will automatically delegate to streaming if running in a streaming mode.

How it works

When you `invoke()` a chat model, LangChain will automatically switch to an internal streaming mode if it detects that you are trying to stream the overall application. The result of the invocation will be the same as far as the code that was using `invoke` is concerned; however, while the chat model is being streamed, LangChain will take care of invoking `@[on_llm_new_token]` events in LangChain's callback system.

python

Callback events allow LangGraph `stream()` and `astream_events()` to surface the chat model's output in real-time.

js

Callback events allow LangGraph `stream()` and `streamEvents()` to surface the chat model's output in real-time.

Batch

Batching a collection of independent requests to a model can significantly improve performance and reduce costs, as the processing can be done in parallel:

python

```
python Batch responses = model.batch([ "Why do parrots have colorful feathers?", "How do airplanes fly?", "What is quantum computing?" ]) for response in responses: print(response)
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

This section describes a chat model method `@[`batch()`][BaseChatModel.batch]`, which parallelizes model calls client-side.

It is **distinct** from batch APIs supported by inference providers, such as [OpenAI](<https://platform.openai.com/docs/guides/batch>) or [Anthropic](<https://platform.claude.com/docs/en/build-with-claude/batch-processing#message-batches-api>).

</div>

By default, `@[`batch()`][BaseChatModel.batch]` will only return the final output for the entire batch. If you want to receive the output for each individual input as it finishes generating, you can stream results with `@[`batch_as_completed()`][BaseChatModel.batch_as_completed]`:

```
```python
Yield batch responses upon completion
for response in model.batch_as_completed([
 "Why do parrots have colorful feathers?",
 "How do airplanes fly?",
 "What is quantum computing?"
]):
 print(response)
```

## 참고

When using `@[`batch_as_completed()`][BaseChatModel.batch_as_completed]`, results may arrive out of order. Each includes the input index for matching to reconstruct the original order as needed.

## 팁

When processing a large number of inputs using `@[`batch()`][BaseChatModel.batch]` or `@[`batch_as_completed()`][BaseChatModel.batch_as_completed]`, you may want to control the maximum number of parallel calls. This can be done by setting the `@[`max_concurrency`][RunnableConfig(max_concurrency)]` attribute in the `@[`RunnableConfig`]` dictionary.

```
```python
Batch with max concurrency
model.batch(list_of_inputs, config={
    'max_concurrency': 5, # Limit to 5 parallel calls
})
```

See the `@[`RunnableConfig`]` reference for a full list of supported attributes.

</div>

For more details on batching, see the `@[reference][BaseChatModel.batch]`.

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```
```typescript
Batch
const responses = await model.batch([
 "Why do parrots have colorful feathers?",
 "How do airplanes fly?",
```

```

 "What is quantum computing?",
 "Why do parrots have colorful feathers?",
 "How do airplanes fly?",
 "What is quantum computing?",
]);
 for (const response of responses) {
 console.log(response);
 }
}

```

## 팁

When processing a large number of inputs using `batch()`, you may want to control the maximum number of parallel calls. This can be done by setting the `maxConcurrency` attribute in the `@RunnableConfig` dictionary.

```

```typescript
Batch with max concurrency model.batch( listOfInputs, {
  maxConcurrency: 5, // Limit to 5 parallel calls } )

```

See the `@RunnableConfig` reference for a full list of supported attributes.

</div>

For more details on batching, see the `@reference`[BaseChatModel.batch].

</div>

Tool calling

Models can request to call tools that perform tasks such as fetching data from a database, searching the web, or running code. Tools are pairings of:

1. A schema, including the name of the tool, a description, and/or argument definitions (often a JSON schema)
2. A function or

coroutine

to execute.

<div class="callout note" markdown="1">

<p class="callout-label">참고</p>

You may hear the term "function calling". We use this interchangeably with "tool calling".

</div>

Here's the basic tool calling flow between a user and a model:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

```
```mermaid
sequenceDiagram
 participant U as User
 participant M as Model
 participant T as Tools

 U->>M: "What's the weather in SF and NYC?"
 M->>M: Analyze request & decide tools needed

 par Parallel Tool Calls
 M->>T: get_weather("San Francisco")
 M->>T: get_weather("New York")
 end

 par Tool Execution
 T-->>M: SF weather data
 T-->>M: NYC weather data
 end

 M->>M: Process results & generate response
 M->>U: "SF: 72°F sunny, NYC: 68°F cloudy"
```

js

```
sequenceDiagram
 participant U as User
 participant M as Model
 participant T as Tools

 U->>M: "What's the weather in SF and NYC?"
 M->>M: Analyze request & decide tools needed

 par Parallel Tool Calls
 M->>T: getWeather("San Francisco")
 M->>T: getWeather("New York")
 end

 par Tool Execution
 T-->>M: SF weather data
 T-->>M: NYC weather data
 end

 end
```

```
M->>M: Process results & generate response
M->>U: "SF: 72°F sunny, NYC: 68°F cloudy"
```

## python

To make tools that you have defined available for use by a model, you must bind them using `@[bind_tools][BaseChatModel.bind_tools]`. In subsequent invocations, the model can choose to call any of the bound tools as needed.

## js

To make tools that you have defined available for use by a model, you must bind them using `@[bindTools][BaseChatModel.bindTools]`. In subsequent invocations, the model can choose to call any of the bound tools as needed.

Some model providers offer

built-in tools

that can be enabled via model or invocation parameters (e.g. [ChatOpenAI](#), [ChatAnthropic](#)). Check the respective [provider reference](#) for details.

## 팁

See the [tools guide](#) for details and other options for creating tools.

## python

```
` `` python Binding user tools from langchain.tools import tool

@tool def get_weather(location: str) -> str: """Get the weather at a location.""" return
f"It's sunny in {location}."

model_with_tools = model.bind_tools([get_weather]) # [!code highlight]

response = model_with_tools.invoke("What's the weather like in Boston?") for tool_call
in response.tool_calls: # View tool calls made by the model print(f"Tool:
{tool_call['name']}") print(f"Args: {tool_call['args']}")
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```

<p class="callout-label">js</p>

```typescript
import { tool } from "langchain";
import * as z from "zod";
import { ChatOpenAI } from "@langchain/openai";

const getWeather = tool(
  (input) => `It's sunny in ${input.location}.`,
  {
    name: "get_weather",
    description: "Get the weather at a location.",
    schema: z.object({
      location: z.string().describe("The location to get the weather for"),
    }),
  },
);

const model = new ChatOpenAI({ model: "gpt-5.5" });
const modelWithTools = model.bindTools([getWeather]); // [!code highlight]

const response = await modelWithTools.invoke("What's the weather like in Boston?");
const toolCalls = response.tool_calls || [];
for (const tool_call of toolCalls) {
  // View tool calls made by the model
  console.log(`Tool: ${tool_call.name}`);
  console.log(`Args: ${tool_call.args}`);
}

```

When binding user-defined tools, the model's response includes a **request** to execute a tool. When using a model separately from an [agent](#), it is up to you to execute the requested tool and return the result back to the model for use in subsequent reasoning. When using an [agent](#), the agent loop will handle the tool execution loop for you.

Below, we show some common ways you can use tool calling.

Tool execution loop

When a model returns tool calls, you need to execute the tools and pass the results back to the model. This creates a conversation loop where the model can use tool results to generate its final response. LangChain includes [agent](#) abstractions that handle this orchestration for you.

Here's a simple example of how to do this:

python

```

```python
Tool execution loop

```



# Bind (potentially multiple) tools to the model

---

```
tools = [get_weather] tools_by_name = {tool.name: tool for tool in tools}
model_with_tools = model.bind_tools(tools)
```

## Step 1: Model generates tool calls

---

```
messages = [{"role": "user", "content": "What's the weather in Boston?"}] ai_msg =
model_with_tools.invoke(messages) messages.append(ai_msg)
```

## Step 2: Execute tools and collect results

---

```
for tool_call in ai_msg.tool_calls: # Execute the tool with the generated arguments tool
= tools_by_name[tool_call["name"]] tool_result = tool.invoke(tool_call)
messages.append(tool_result)
```

## Step 3: Pass results back to model for final response

---

```
final_response = model_with_tools.invoke(messages) print(final_response.text)
```

**"The current weather in Boston is 72°F and sunny."**

---

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```typescript
Tool execution loop
// Bind (potentially multiple) tools to the model
const tools = [getWeather]
const toolsByName = Object.fromEntries(tools.map((tool) => [tool.name, tool]))
const modelWithTools = model.bindTools(tools)
```

```
// Step 1: Model generates tool calls
const messages = [{"role": "user", "content": "What's the weather in Boston?"}]
const ai_msg = await modelWithTools.invoke(messages)
messages.push(ai_msg)

// Step 2: Execute tools and collect results
for (const tool_call of ai_msg.tool_calls) {
  // Execute the tool with the generated arguments
  const tool = toolsByName[tool_call.name]
  const tool_result = await tool.invoke(tool_call)
  messages.push(tool_result)
}

// Step 3: Pass results back to model for final response
const final_response = await modelWithTools.invoke(messages)
console.log(final_response.text)
// "The current weather in Boston is 72°F and sunny."
```

Each `@[ToolMessage]` returned by the tool includes a `tool_call_id` that matches the original tool call, helping the model correlate results with requests.

Forcing tool calls

By default, the model has the freedom to choose which bound tool to use based on the user's input. However, you might want to force choosing a tool, ensuring the model uses either a particular tool or **any** tool from a given list:

python

```
```python Force use of any tool
model_with_tools = model.bind_tools([tool_1],
tool_choice="any")
```

```
```python Force use of specific tools
model_with_tools = model.bind_tools([tool_1], tool_choice="tool_1")
```

js

```
```typescript Force use of any tool
const modelWithTools = model.bindTools([tool_1],
{ toolChoice: "any" })
```

```
```typescript Force use of specific tools
const modelWithTools = model.bindTools([tool_1], { toolChoice: "tool_1" })
```

Parallel tool calls

Many models support calling multiple tools in parallel when appropriate. This allows the model to gather information from different sources simultaneously.

python

```
```python Parallel tool calls tools = [get_weather] tools_by_name = {tool.name: tool
for tool in tools} model_with_tools = model.bind_tools(tools)

response = model_with_tools.invoke("What's the weather in Boston and Tokyo?")
```

## The model may generate multiple tool calls

```
print(response.tool_calls)
```

```
[
```

```
{'name': 'get_weather', 'args': {'location':
'Boston'}, 'id': 'call_1'},
```

```
{'name': 'get_weather', 'args': {'location':
'Tokyo'}, 'id': 'call_2'},
```

```
]
```

## Execute all tools (can be done in parallel with async)

```
results = [] for tool_call in response.tool_calls: tool =
tools_by_name[tool_call["name"]] result = tool.invoke(tool_call) results.append(result)
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```

```typescript
Parallel tool calls
const tools = [getWeather]
const toolsByName = Object.fromEntries(tools.map((tool) => [tool.name, tool]))
const modelWithTools = model.bindTools(tools)

const response = await modelWithTools.invoke(
  "What's the weather in Boston and Tokyo?"
)

// The model may generate multiple tool calls
console.log(response.tool_calls)
// [
//   { name: 'get_weather', args: { location: 'Boston' }, id: 'call_1' },
//   { name: 'get_weather', args: { location: 'Tokyo' }, id: 'call_2' }
// ]

// Execute all tools (can be done in parallel with async)
const results = []
for (const tool_call of response.tool_calls || []) {
  const tool = toolsByName[tool_call.name]
  const result = await tool.invoke(tool_call)
  results.push(result)
}

```

The model intelligently determines when parallel execution is appropriate based on the independence of the requested operations.

팁

Most models supporting tool calling enable parallel tool calls by default. Some (including [OpenAI](#) and [Anthropic](#)) allow you to disable this feature. To do this, set `parallel_tool_calls=False`:

```
model.bind_tools([get_weather], parallel_tool_calls=False)
```

Streaming tool calls

When streaming responses, tool calls are progressively built through `@[ ToolCallChunk ]`. This allows you to see tool calls as they're being generated rather than waiting for the complete response.

python

```

```python
Streaming tool calls for chunk in model_with_tools.stream("What's the
weather in Boston and Tokyo?"): # Tool call chunks arrive progressively for tool_chunk
in chunk.tool_call_chunks: if name := tool_chunk.get("name"): print(f"Tool: {name}") if

```

```
id_ := tool_chunk.get("id"): print(f"ID: {id_}") if args := tool_chunk.get("args"):
print(f"Args: {args}")
```

**Output:**

---

**Tool: get\_weather**

---

**ID: call\_SvMLU1TVIZugrFLckFE2ceRE**

---

**Args: {"lo**

---

**Args: catio**

---

**Args: n": "B**

---

**Args: osto**

---

**Args: n"}  
}**

---

**Tool: get\_weather**

---

**ID: call\_QMZdy6qInx13oWKE7KhuhOLR**

---

**Args: {"lo**

---

**Args: catio**

---

**Args: n": "T**

---

**Args: okyo**

---

# Args: "}

You can accumulate chunks to build complete tool calls:

```
```python Accumulate tool calls
gathered = None
for chunk in model_with_tools.stream("What's the weather in Boston?"):
    gathered = chunk if gathered is None else gathered + chunk
print(gathered.tool_calls)
```

js

```
```typescript Streaming tool calls
const stream = await modelWithTools.stream("What's the weather in Boston and Tokyo?")
for await (const chunk of stream) { // Tool call chunks arrive progressively
 if (chunk.tool_call_chunks) {
 for (const tool_chunk of chunk.tool_call_chunks) {
 console.log(Tool: ${tool_chunk.get('name', '')})
 console.log(Args: ${tool_chunk.get('args', '')}`)
 }
 }
}
```

// Output: // Tool: get\_weather // Args: // Tool: // Args: {"loc": "Boston"} // Tool: get\_time // Args: // Tool: // Args: {"timezone": "Tokyo"}

You can accumulate chunks to build complete tool calls:

```
```typescript Accumulate tool calls
let full: AIMessageChunk | null = null
const stream = await modelWithTools.stream("What's the weather in Boston?")
for await (const chunk of stream) {
    full = full ? full.concat(chunk) : chunk
    console.log(full.contentBlocks)
}
```

Structured output

Models can be requested to provide their response in a format matching a given schema. This is useful for ensuring the output can be easily parsed and used in subsequent processing. LangChain supports multiple schema types and methods for enforcing structured output.

팁

To learn about structured output, see [Structured output](#).

python

Pydantic

[Pydantic models](#) provide the richest feature set with field validation, descriptions, and nested structures.

```
from pydantic import BaseModel, Field

class Movie(BaseModel):
    """A movie with details."""
    title: str = Field(description="The title of the movie")
    year: int = Field(description="The year the movie was released")
    director: str = Field(description="The director of the movie")
    rating: float = Field(description="The movie's rating out of 10")

model_with_structure = model.with_structured_output(Movie)
response = model_with_structure.invoke("Provide details about the movie Inception")
print(response) # Movie(title="Inception", year=2010, director="Christopher Nolan", rating=8.8)
```

TypedDict

Python's `TypedDict` provides a simpler alternative to Pydantic models, ideal when you don't need runtime validation.

```
from typing_extensions import TypedDict, Annotated

class MovieDict(TypedDict):
    """A movie with details."""
    title: Annotated[str, ..., "The title of the movie"]
    year: Annotated[int, ..., "The year the movie was released"]
    director: Annotated[str, ..., "The director of the movie"]
    rating: Annotated[float, ..., "The movie's rating out of 10"]

model_with_structure = model.with_structured_output(MovieDict)
response = model_with_structure.invoke("Provide details about the movie Inception")
print(response) # {'title': 'Inception', 'year': 2010, 'director': 'Christopher Nolan', 'rating': 8.8}
```

JSON Schema

Provide a [JSON Schema](#) for maximum control and interoperability.

```
import json
```



```

json_schema = {
    "title": "Movie",
    "description": "A movie with details",
    "type": "object",
    "properties": {
        "title": {
            "type": "string",
            "description": "The title of the movie"
        },
        "year": {
            "type": "integer",
            "description": "The year the movie was released"
        },
        "director": {
            "type": "string",
            "description": "The director of the movie"
        },
        "rating": {
            "type": "number",
            "description": "The movie's rating out of 10"
        }
    },
    "required": ["title", "year", "director", "rating"]
}

model_with_structure = model.with_structured_output(
    json_schema,
    method="json_schema",
)
response = model_with_structure.invoke("Provide details about the movie Inception")
print(response) # {'title': 'Inception', 'year': 2010, ...}

```

js

Zod

A [zod schema](#) is the preferred method of defining an output schema. Note that when a zod schema is provided, the model output will also be validated against the schema using zod's parse methods.

```

import * as z from "zod";

const Movie = z.object({
    title: z.string().describe("The title of the movie"),
    year: z.number().describe("The year the movie was released"),
    director: z.string().describe("The director of the movie"),
    rating: z.number().describe("The movie's rating out of 10"),
});

```

```
const modelWithStructure = model.withStructuredOutput(Movie);

const response = await modelWithStructure.invoke("Provide details about the movie Inception");
console.log(response);
// {
//   title: "Inception",
//   year: 2010,
//   director: "Christopher Nolan",
//   rating: 8.8,
// }
```

JSON Schema

For maximum control or interoperability, you can provide a raw JSON Schema.

```
const jsonSchema = {
  "title": "Movie",
  "description": "A movie with details",
  "type": "object",
  "properties": {
    "title": {
      "type": "string",
      "description": "The title of the movie",
    },
    "year": {
      "type": "integer",
      "description": "The year the movie was released",
    },
    "director": {
      "type": "string",
      "description": "The director of the movie",
    },
    "rating": {
      "type": "number",
      "description": "The movie's rating out of 10",
    },
  },
  "required": ["title", "year", "director", "rating"],
}

const modelWithStructure = model.withStructuredOutput(
  jsonSchema,
  { method: "jsonSchema" },
)

const response = await modelWithStructure.invoke("Provide details about the movie Inception")
console.log(response) // {'title': 'Inception', 'year': 2010, ...}
```

Standard Schema

Any schema from a library implementing the [Standard Schema](#) specification is also supported. Standard Schema objects are validated at runtime via the schema's `~standard.validate()` method.

```
import * as v from "valibot";
import { toStandardJsonSchema } from "@valibot/to-json-schema";

const Movie = toStandardJsonSchema(
  v.object({
    title: v.pipe(v.string(), v.description("The title of the movie")),
    year: v.pipe(v.number(), v.description("The year the movie was released")),
    director: v.pipe(v.string(), v.description("The director of the movie")),
    rating: v.pipe(v.number(), v.description("The movie's rating out of 10")),
  })
);

const modelWithStructure = model.withStructuredOutput(Movie);

const response = await modelWithStructure.invoke("Provide details about the movie Inception");
console.log(response);
// {
//   title: "Inception",
//   year: 2010,
//   director: "Christopher Nolan",
//   rating: 8.8,
// }
```

python

참고

Key considerations for structured output

- **Method parameter:** Some providers support different methods for structured output:
 - `'json_schema'` : Uses dedicated structured output features offered by the provider.
 - `'function_calling'` : Derives structured output by forcing a [tool call](#) that follows the given schema.
 - `'json_mode'` : A precursor to `'json_schema'` offered by some providers. Generates valid JSON, but the schema must be described in the prompt.
- **Include raw:** Set `include_raw=True` to get both the parsed output and the raw AI message.

- **Validation:** Pydantic models provide automatic validation. `TypedDict` and JSON Schema require manual validation.

See your [provider's integration page](#) for supported methods and configuration options.

js

참고

Key considerations for structured output:

- **Method parameter:** Some providers support different methods (`'jsonSchema'`, `'functionCalling'`, `'jsonMode'`)
- **Include raw:** Use `@[includeRaw: true]` `[BaseChatModel.with_structured_output(include_raw)]` to get both the parsed output and the raw `@[AIMessage]`
- **Validation:** Zod and Standard Schema objects provide automatic validation, while JSON Schema requires manual validation
- **Standard Schema:** Any schema library implementing the [Standard Schema](#) spec is supported and validated at runtime

See your [provider's integration page](#) for supported methods and configuration options.

Example: Message output alongside parsed structure

It can be useful to return the raw `@[AIMessage]` object alongside the parsed representation to access response metadata such as [token counts](#). To do this, set `@[include_raw=True]` `[BaseChatModel.with_structured_output(include_raw)]` when calling `@[with_structured_output]` `[BaseChatModel.with_structured_output]`:

python

```
from pydantic import BaseModel, Field

class Movie(BaseModel):
    """A movie with details."""
    title: str = Field(description="The title of the movie")
    year: int = Field(description="The year the movie was released")
    director: str = Field(description="The director of the movie")
    rating: float = Field(description="The movie's rating out of 10")

model_with_structure = model.with_structured_output(Movie, include_raw=True) # [!c
```

```
ode highlight]
response = model_with_structure.invoke("Provide details about the movie Inception")
response
# {
#     "raw": AIMessage(...),
#     "parsed": Movie(title=..., year=..., ...),
#     "parsing_error": None,
# }
```

js

```
import * as z from "zod";

const Movie = z.object({
  title: z.string().describe("The title of the movie"),
  year: z.number().describe("The year the movie was released"),
  director: z.string().describe("The director of the movie"),
  rating: z.number().describe("The movie's rating out of 10"),
  title: z.string().describe("The title of the movie"),
  year: z.number().describe("The year the movie was released"),
  director: z.string().describe("The director of the movie"), // [!code highlight]
  rating: z.number().describe("The movie's rating out of 10"),
});

const modelWithStructure = model.withStructuredOutput(Movie, { includeRaw: true });

const response = await modelWithStructure.invoke("Provide details about the movie I
nception");
console.log(response);
// {
//   raw: AIMessage { ... },
//   parsed: { title: "Inception", ... }
// }
```

Example: Nested structures

Schemas can be nested:

python

```
```python
from pydantic BaseModel from pydantic import BaseModel, Field

class Actor(BaseModel):
 name: str
 role: str

class MovieDetails(BaseModel):
 title: str
 year: int
 cast: list[Actor]
 genres: list[str]
 budget: float | None = Field(None, description="Budget in millions USD")

model_with_structure = model.with_structured_output(MovieDetails)
```

```

python TypedDict
from typing_extensions import Annotated, TypedDict

class Actor(TypedDict):
 name: str
 role: str

class MovieDetails(TypedDict):
 title: str
 year: int
 cast: list[Actor]
 genres: list[str]
 budget: Annotated[float | None, ..., "Budget in millions USD"]

model_with_structure = model.with_structured_output(MovieDetails)

```

js

```

import * as z from "zod";

const Actor = z.object({
 name: z.string(),
 role: z.string(),
});

const MovieDetails = z.object({
 title: z.string(),
 year: z.number(),
 cast: z.array(Actor),
 genres: z.array(z.string()),
 budget: z.number().nullable().describe("Budget in millions USD"),
});

const modelWithStructure = model.withStructuredOutput(MovieDetails);

```

## Advanced topics

### Model profiles

#### 정보

Model profiles require `langchain>=1.1`.

## python

LangChain chat models can expose a dictionary of supported features and capabilities through a `profile` attribute:

```
model.profile
{
"max_input_tokens": 400000,
"image_inputs": True,
"reasoning_output": True,
"tool_calling": True,
...
}
```

Refer to the full set of fields in the [API reference](#).

Much of the model profile data is powered by the [models.dev](#) project, an open source initiative that provides model capability data. These data are augmented with additional fields for purposes of use with LangChain. These augmentations are kept aligned with the upstream project as it evolves.

Model profile data allow applications to work around model capabilities dynamically. For example:

1. [Summarization middleware](#) can trigger summarization based on a model's context window size.
2. [Structured output](#) strategies in `create_agent` can be inferred automatically (e.g., by checking support for native structured output features).
3. Model inputs can be gated based on supported [modalities](#) and maximum input tokens.
4. [Deep Agents Code](#) filters the [interactive model switcher](#) to models whose profiles report `tool_calling` support and text I/O, and displays context window sizes and capability flags in the selector detail view.

### Updating or overwriting profile data

Model profile data can be changed if it is missing, stale, or incorrect.

#### Option 1 (quick fix)

You can instantiate a chat model with any valid profile:

```
custom_profile = {
 "max_input_tokens": 100_000,
 "tool_calling": True,
 "structured_output": True,
 # ...
}
```

```
}
model = init_chat_model("...", profile=custom_profile)
```

The `profile` is also a regular `dict` and can be updated in place. If the model instance is shared, consider using `model_copy` to avoid mutating shared state.

```
new_profile = model.profile | {"key": "value"}
model.model_copy(update={"profile": new_profile})
```

## Option 2 (fix data upstream)

The primary source for the data is the [models.dev](https://github.com/langchain-ai/models.dev) project. This data is merged with additional fields and overrides in LangChain [integration packages](#) and are shipped with those packages.

Model profile data can be updated through the following process:

1. (If needed) update the source data at [models.dev](https://github.com/langchain-ai/models.dev) through a pull request to its [repository on GitHub](#).
2. (If needed) update additional fields and overrides in `langchain_<package>/data/profile_augmentations.toml` through a pull request to the LangChain [integration package](#).
3. Use the [langchain-model-profiles](#) CLI tool to pull the latest data from [models.dev](https://github.com/langchain-ai/models.dev), merge in the augmentations and update the profile data:

```
```bash pip
pip install -U langchain-model-profiles
```
```

```
```bash uv uv add langchain-model-profiles
```

```
```bash
langchain-profiles refresh --provider <provider> --data-dir <data_dir>
```

This command: - Downloads the latest data for `<provider>` from `models.dev` - Merges augmentations from `profile_augmentations.toml` in `<data_dir>` - Writes merged profiles to `profiles.py` in `<data_dir>`

For example: from `[`libs/partners/anthropic`](https://github.com/langchain-ai/langchain/tree/master/libs/partners/anthropic)` in the `[LangChain monorepo](https://github.com/langchain-ai/langchain)`:

```
uv run --with langchain-model-profiles --provider anthropic --data-dir langchain_anthropic/data
```



js

LangChain chat models can expose a dictionary of supported features and capabilities through a `profile` property:

```
model.profile;
// {
// maxInputTokens: 400000,
// imageInputs: true,
// reasoningOutput: true,
// toolCalling: true,
// ...
// }
```

Refer to the full set of fields in the [API reference](#).

Much of the model profile data is powered by the [models.dev](#) project, an open source initiative that provides model capability data. This data is augmented with additional fields for purposes of use with LangChain. These augmentations are kept aligned with the upstream project as it evolves.

Model profile data allow applications to work around model capabilities dynamically. For example:

1. [Summarization middleware](#) can trigger summarization based on a model's context window size.
2. [Structured output](#) strategies in `createAgent` can be inferred automatically (e.g., by checking support for native structured output features).
3. Model inputs can be gated based on supported [modalities](#) and maximum input tokens.
4. [Deep Agents Code](#) filters the [interactive model switcher](#) to models whose profiles report `tool_calling` support and text I/O, and displays context window sizes and capability flags in the selector detail view.

## Modify profile data

Model profile data can be changed if it is missing, stale, or incorrect.

### Option 1 (quick fix)

You can instantiate a chat model with any valid profile:

```
const customProfile = {
 maxInputTokens: 100_000,
 toolCalling: true,
 structuredOutput: true,
 // ...
}
```

```
};
const model = initChatModel("...", { profile: customProfile });
```

## Option 2 (fix data upstream)

The primary source for the data is the [models.dev](#) project. These data are merged with additional fields and overrides in LangChain [integration packages](#) and are shipped with those packages.

Model profile data can be updated through the following process:

1. (If needed) update the source data at [models.dev](#) through a pull request to its [repository on GitHub](#).
2. (If needed) update additional fields and overrides in `langchain-<package>/profiles.toml` through a pull request to the LangChain [integration package](#).

## 주의

Model profiles are a beta feature. The format of a profile is subject to change.

## Multimodal

Certain models can process and return non-textual data such as images, audio, and video. You can pass non-textual data to a model by providing [content blocks](#).

## 팁

All LangChain chat models with underlying multimodal capabilities support:

1. Data in the cross-provider standard format (see [our messages guide](#))
2. OpenAI [chat completions](#) format
3. Any format that is native to that specific provider (e.g., Anthropic models accept Anthropic native format)

See the [multimodal section](#) of the messages guide for details.

Some models

can return multimodal data as part of their response. If invoked to do so, the resulting `@[ AIMessage ]` will have content blocks with multimodal types.

## python

```
```python Multimodal output response = model.invoke("Create a picture of a cat")
print(response.content_blocks)
```

[

{"type": "text", "text": "Here's a picture of a cat"},

**{"type": "image", "base64": "...",
"mime_type": "image/jpeg"},**

]

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```typescript Multimodal output
```

```
const response = await model.invoke("Create a picture of a cat");
```

```
console.log(response.contentBlocks);
```

```
// [
```

```
// { type: "text", text: "Here's a picture of a cat" },
```

```
// { type: "image", data: "...", mimeType: "image/jpeg" },
```

```
//]
```

See the [integrations page](#) for details on specific providers.

## Reasoning

Many models are capable of performing multi-step reasoning to arrive at a conclusion. This involves breaking down complex problems into smaller, more manageable steps.

**If supported by the underlying model**, you can surface this reasoning process to better understand how the model arrived at its final answer.

python

```
```python Stream reasoning output for chunk in model.stream("Why do parrots have colorful feathers?"): reasoning_steps = [r for r in chunk.content_blocks if r["type"] == "reasoning"] print(reasoning_steps if reasoning_steps else chunk.text)
```

```
```python Complete reasoning output
response = model.invoke("Why do parrots have colorful feathers?")
reasoning_steps = [b for b in response.content_blocks if b["type"] == "reasoning"]
print(" ".join(step["reasoning"] for step in reasoning_steps))
```

**js**

```
```typescript Stream reasoning output const stream = model.stream("Why do parrots have colorful feathers?"); for await (const chunk of stream) { const reasoningSteps = chunk.contentBlocks.filter(b => b.type === "reasoning"); console.log(reasoningSteps.length > 0 ? reasoningSteps : chunk.text); }
```

```
```typescript Complete reasoning output
const response = await model.invoke("Why do parrots have colorful feathers?");
const reasoningSteps = response.contentBlocks.filter(b => b.type === "reasoning");
console.log(reasoningSteps.map(step => step.reasoning).join(" "));
```

Depending on the model, you can sometimes specify the level of effort it should put into reasoning. Similarly, you can request that the model turn off reasoning entirely. This may take the form of categorical "tiers" of reasoning (e.g., 'low' or 'high' ) or integer token budgets.

**python**

**참고**

`reasoning_effort` as a standard parameter requires `langchain-core`  $\geq 1.5.2$ , plus the corresponding partner package version: `langchain-anthropic`  $\geq 1.5.3$ , `langchain-openai`  $\geq 1.4.1$ , `langchain-fireworks`  $\geq 1.5.2$ , `langchain-xai`  $\geq 1.3.0$ , `langchain-google-genai`  $\geq 4.3.1$ , Or `langchain-aws`  $\geq 1.7.0$ .

@[ ChatOpenAI ], @[ ChatAnthropic ], @[ ChatFireworks ], @[ ChatXAI ], @[ ChatGoogleGenerativeAI ], and [ChatBedrockConverse](#) support a standard `reasoning_effort` parameter. Like `temperature`, it can be set at model construction or per invocation, and each provider translates it into its own API format:

```
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(model="claude-sonnet-4-6")
response = model.invoke(
 "Why do parrots have colorful feathers?",
 reasoning_effort="high",
)
```

Supported effort levels and the provider's documented default vary by model. Check a model's [profile](#) for the levels it supports and its default:

```
model.profile["reasoning_effort_levels"] # e.g. ['low', 'medium', 'high']
model.profile["reasoning_effort_default"] # e.g. 'high'
```

Some providers also accept a native alias for `reasoning_effort` (for example, `ChatAnthropic` accepts `effort` and `ChatGoogleGenerativeAI` accepts `thinking_level`). See the [chat model integrations](#) page for provider-specific detail.

For details, see the [integrations page](#) or [reference](#) for your respective chat model.

## Local models

LangChain supports running models locally on your own hardware. This is useful for scenarios where either data privacy is critical, you want to invoke a custom model, or when you want to avoid the costs incurred when using a cloud-based model.

[Ollama](#) is one of the easiest ways to run chat and embedding models locally.

## Prompt caching

Many providers offer prompt caching features to reduce latency and cost on repeat processing of the same tokens. You can engage caching at three levels:

- **Implicit provider caching:** providers automatically pass on cost savings if a request hits a cache, with no configuration required. Examples: [OpenAI](#) and [Gemini](#).
- **Provider-level explicit controls:** providers let you manually indicate cache points for greater control or to guarantee cost savings. These mirror the underlying provider/API behavior. Examples:
  - `@[ ChatOpenAI ]` (via `prompt_cache_key`)
  - Anthropic content-block [cache\\_control](#)
  - [Gemini](#).

## python

```
- AWS Bedrock [`cachePoint`] (/oss/integrations/chat/bedrock#prompt-caching) blocks
```

- **LangChain middleware:** for agents, middleware lets LangChain optimize caching of stable system prompt and tool content. Examples:
  - Anthropic's [AnthropicPromptCachingMiddleware](#)
  - AWS Bedrock's [BedrockPromptCachingMiddleware](#)

## 주의

Prompt caching is often only engaged above a minimum input token threshold. See [provider pages](#) for details.

Cache usage will be reflected in the [usage metadata](#) of the model response.

## Server-side tool use

Some providers support server-side [tool-calling](#) loops: models can interact with web search, code interpreters, and other tools and analyze the results in a single conversational turn.

If a model invokes a tool server-side, the content of the response message will include content representing the invocation and result of the tool. Accessing the [content blocks](#) of the response will return the server-side tool calls and results in a provider-agnostic format:

### python

```
```python Invoke with server-side tool use from langchain.chat_models import
init_chat_model

model = init_chat_model("gpt-5.4-mini")

tool = {"type": "web_search"} model_with_tools = model.bind_tools([tool])

response = model_with_tools.invoke("What was a positive news story from today?")
print(response.content_blocks)
```

```
```python Result expandable
[
 {
 "type": "server_tool_call",
 "name": "web_search",
 "args": {
 "query": "positive news stories today",
 "type": "search"
 },
 "id": "ws_abc123"
 },
 {
 "type": "server_tool_result",
```

```

 "tool_call_id": "ws_abc123",
 "status": "success"
 },
 {
 "type": "text",
 "text": "Here are some positive news stories from today...",
 "annotations": [
 {
 "end_index": 410,
 "start_index": 337,
 "title": "article title",
 "type": "citation",
 "url": "..."
 }
]
 }
]

```

js

```

import { initChatModel } from "langchain";

const model = await initChatModel("gpt-5.4-mini");
const modelWithTools = model.bindTools([{ type: "web_search" }])

const message = await modelWithTools.invoke("What was a positive news story from to day?");
console.log(message.contentBlocks);

```

This represents a single conversational turn; there are no associated [ToolMessage](#) objects that need to be passed in as in client-side [tool-calling](#).

See the [integration page](#) for your given provider for available tools and usage details.

python

## Model exceptions

Major integration packages raise standard exception types from `langchain_core.exceptions` for common model failures such as authentication errors, rate limits, and timeouts. These exceptions inherit from both the LangChain base type and the provider SDK's own exception type, so you can catch either one:

```

from langchain.chat_models import init_chat_model
from langchain_core.exceptions import ModelTimeoutError

```

```

model = init_chat_model("openai:gpt-5.6-luna", timeout=0.0001)

try:
 response = model.invoke("Hello")
except ModelTimeoutError:
 print("caught")

```

Each type carries an `is_retryable` attribute that [retry middleware](#) respects by default.

## Exception types

- `@[ ModelAuthenticationError ]` — missing, invalid, or expired API key (not retryable)
- `@[ ModelPermissionDeniedError ]` — credentials lack permission (not retryable)
- `@[ ModelInvalidRequestError ]` — provider rejects the request (not retryable)
- `@[ ModelNotFoundError ]` — requested model not found (not retryable)
- `@[ ModelRateLimitError ]` — provider rate limit exceeded (retryable)
- `@[ ModelAPIError ]` — provider server failure (retryable)
- `@[ ModelConnectionError ]` — provider cannot be reached (retryable)
- `@[ ModelTimeoutError ]` — request times out (retryable)
- `@[ ContextOverflowError ]` — input exceeds the model context limit (not retryable)

## python

### Rate limiting

Many chat model providers impose a limit on the number of invocations that can be made in a given time period. If you hit a rate limit, you will typically receive a rate limit error response from the provider, and will need to wait before making more requests.

To help manage rate limits, chat model integrations accept a `rate_limiter` parameter that can be provided during initialization to control the rate at which requests are made.

#### Initialize and use a rate limiter

LangChain comes with (an optional) built-in `@[ InMemoryRateLimiter ]`. This limiter is thread safe and can be shared by multiple threads in the same process.

```

```python
Define a rate limiter from langchain.rate_limiters import
InMemoryRateLimiter

```

```

rate_limiter = InMemoryRateLimiter( requests_per_second=0.1, # 1 request every 10s
check_every_n_seconds=0.1, # Check every 100ms whether allowed to make a request
max_bucket_size=10, # Controls the maximum burst size. )

```



```
model = init_chat_model( model="gpt-5.5", model_provider="openai",
rate_limiter=rate_limiter # [!code highlight] )
```

```
<div class="callout warning" markdown="1">
```

```
<p class="callout-label">주의</p>
```

The provided rate limiter can only limit the number of requests per unit time. It will not help if you need to also limit based on the size of the requests.

```
</div>
```

```
</div>
```

Base URL and proxy settings

You can configure a custom base URL for providers that implement the OpenAI Chat Completions API.

```
<div class="callout warning" markdown="1">
```

```
<p class="callout-label">주의</p>
```

``model_provider="openai"`` (or direct ``ChatOpenAI`` usage) targets the official OpenAI API specification. Provider-specific fields from routers and proxies may not be extracted or preserved.

For OpenRouter and LiteLLM, prefer the dedicated integrations:

- [OpenRouter via ``ChatOpenRouter``](/oss/integrations/chat/openrouter) (``langchain-openrouter``)
- [LiteLLM via ``ChatLiteLLM`` / ``ChatLiteLLMRouter``](/oss/integrations/chat) (``langchain-litellm``)

```
</div>
```

****Custom base URL****

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

Many model providers offer OpenAI-compatible APIs (e.g., [Together AI](https://www.together.ai/), [vLLM](https://github.com/vllm-project/vllm)). You can use `@`init_chat_model`` with these providers by specifying the appropriate ``base_url`` parameter:

```
```python
model = init_chat_model(
 model="MODEL_NAME",
```

```
model_provider="openai",
base_url="BASE_URL",
api_key="YOUR_API_KEY",
)
```

## js

Many model providers offer OpenAI-compatible APIs (e.g., [Together AI](#), [vLLM](#)). You can use `initChatModel` with these providers by specifying the appropriate `base_url` parameter:

```
model = initChatModel(
 "MODEL_NAME",
 {
 modelProvider: "openai",
 baseUrl: "BASE_URL",
 apiKey: "YOUR_API_KEY",
 }
)
```

## 참고

When using direct chat model class instantiation, the parameter name may vary by provider. Check the respective [reference](#) for details.

## python

### HTTP proxy configuration

For deployments requiring HTTP proxies, some model integrations support proxy configuration:

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
 model="gpt-5.5",
 openai_proxy="http://proxy.example.com:8080"
)
```

## 참고

Proxy support varies by integration. Check the specific model provider's [reference](#) for proxy configuration options.

## Log probabilities

Certain models can be configured to return token-level log probabilities representing the likelihood of a given token by setting the `logprobs` parameter when initializing the model:

### python

```
model = init_chat_model(
 model="gpt-5.5",
 model_provider="openai"
).bind(logprobs=True)

response = model.invoke("Why do parrots talk?")
print(response.response_metadata["logprobs"])
```

### js

```
const model = new ChatOpenAI({
 model: "gpt-5.5",
 logprobs: true,
});

const responseMessage = await model.invoke("Why do parrots talk?");

responseMessage.response_metadata.logprobs.content.slice(0, 5);
```

## Token usage

A number of model providers return token usage information as part of the invocation response. When available, this information will be included on the `@[AIMessage]` objects produced by the corresponding model. For more details, see the [messages](#) guide.

### python

#### 참고

Some provider APIs, notably OpenAI and Azure OpenAI chat completions, require users opt-in to receiving token usage data in streaming contexts. See the [streaming usage metadata](#) section of the integration guide for details.

### python

You can track aggregate token counts across models in an application using either a callback or context manager, as shown below:

## Callback handler

```
from langchain.chat_models import init_chat_model
from langchain_core.callbacks import UsageMetadataCallbackHandler

model_1 = init_chat_model(model="gpt-5.4-mini")
model_2 = init_chat_model(model="claude-haiku-4-5-20251001")

callback = UsageMetadataCallbackHandler()
result_1 = model_1.invoke("Hello", config={"callbacks": [callback]})
result_2 = model_2.invoke("Hello", config={"callbacks": [callback]})
print(callback.usage_metadata)
```

```
{
 'gpt-5.4-mini': {
 'input_tokens': 8,
 'output_tokens': 10,
 'total_tokens': 18,
 'input_token_details': {'audio': 0, 'cache_read': 0},
 'output_token_details': {'audio': 0, 'reasoning': 0}
 },
 'claude-haiku-4-5-20251001': {
 'input_tokens': 8,
 'output_tokens': 21,
 'total_tokens': 29,
 'input_token_details': {'cache_read': 0, 'cache_creation': 0}
 }
}
```

## Context manager

```
from langchain.chat_models import init_chat_model
from langchain_core.callbacks import get_usage_metadata_callback

model_1 = init_chat_model(model="gpt-5.4-mini")
model_2 = init_chat_model(model="claude-haiku-4-5-20251001")

with get_usage_metadata_callback() as cb:
 model_1.invoke("Hello")
 model_2.invoke("Hello")
 print(cb.usage_metadata)
```

```
{
 'gpt-5.4-mini': {
 'input_tokens': 8,
 'output_tokens': 10,
 'total_tokens': 18,
 'input_token_details': {'audio': 0, 'cache_read': 0},
 'output_token_details': {'audio': 0, 'reasoning': 0}
 },
 'claude-haiku-4-5-20251001': {
 'input_tokens': 8,
 'output_tokens': 21,
 'total_tokens': 29,
 'input_token_details': {'cache_read': 0, 'cache_creation': 0}
 }
}
```

```

 },
 'claude-haiku-4-5-20251001': {
 'input_tokens': 8,
 'output_tokens': 21,
 'total_tokens': 29,
 'input_token_details': {'cache_read': 0, 'cache_creation': 0}
 }
}

```

## Invocation config

### python

When invoking a model, you can pass additional configuration through the `config` parameter using a `@[RunnableConfig]` dictionary. This provides run-time control over execution behavior, callbacks, and metadata tracking.

### js

When invoking a model, you can pass additional configuration through the `config` parameter using a `@[RunnableConfig]` object. This provides run-time control over execution behavior, callbacks, and metadata tracking.

Common configuration options include:

### python

```

```python
Invocation with config response = model.invoke( "Tell me a joke", config={
    "run_name": "joke_generation", # Custom name for this run
    "tags": ["humor", "demo"], # Tags for categorization
    "metadata": {"user_id": "123"}, # Custom metadata
    "callbacks": [my_callback_handler], # Callback handlers } )

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```

```typescript
Invocation with config
const response = await model.invoke(
 "Tell me a joke",
 {

```

```

 runName: "joke_generation", // Custom name for this run
 tags: ["humor", "demo"], // Tags for categorization
 }
)

```

```

 metadata: {"user_id": "123"}, // Custom metadata
 callbacks: [my_callback_handler], // Callback handlers
 }
)

```

These configuration values are particularly useful when:

- Debugging with [LangSmith](#) tracing
- Implementing custom logging or monitoring
- Controlling resource usage in production
- Tracking invocations across complex pipelines

## python

### Key configuration attributes

Identifies this specific invocation in logs and traces. Not inherited by sub-calls.

Labels inherited by all sub-calls for filtering and organization in debugging tools.

Custom key-value pairs for tracking additional context, inherited by all sub-calls.

Controls the maximum number of parallel calls when using `@[ batch() ]`

`[BaseChatModel.batch]` or `@[ batch_as_completed() ]`

`[BaseChatModel.batch_as_completed]`.

Handlers for monitoring and responding to events during execution.

Maximum recursion depth for chains to prevent infinite loops in complex pipelines.

## js

### Key configuration attributes

Identifies this specific invocation in logs and traces. Not inherited by sub-calls.

Labels inherited by all sub-calls for filtering and organization in debugging tools.

Custom key-value pairs for tracking additional context, inherited by all sub-calls.

Controls the maximum number of parallel calls when using `batch()`.

Handlers for monitoring and responding to events during execution.

Maximum recursion depth for chains to prevent infinite loops in complex pipelines.

## 팁

See full `@[ RunnableConfig ]` reference for all supported attributes.

## python

### Configurable models

You can also create a runtime-configurable model by specifying `@[configurable_fields][BaseChatModel.configurable_fields]`. If you don't specify a model value, then `'model'` and `'model_provider'` will be configurable by default.

```
from langchain.chat_models import init_chat_model

configurable_model = init_chat_model(temperature=0)

configurable_model.invoke(
 "what's your name",
 config={"configurable": {"model": "gpt-5-nano"}}, # Run with GPT-5-Nano
)

configurable_model.invoke(
 "what's your name",
 config={"configurable": {"model": "claude-sonnet-4-6"}}, # Run with Claude
)
```

### Configurable model with default values

We can create a configurable model with default model values, specify which parameters are configurable, and add prefixes to configurable params:

```
first_model = init_chat_model(
 model="gpt-5.4-mini",
 temperature=0,
 configurable_fields=("model", "model_provider", "temperature", "max_tokens"),
 config_prefix="first", # Useful when you have a chain with multiple models
)

first_model.invoke("what's your name")
```

```
first_model.invoke(
 "what's your name",
 config={
 "configurable": {
 "first_model": "claude-sonnet-4-6",
 "first_temperature": 0.5,
 "first_max_tokens": 100,
 }
 },
)
```

See the `@[init_chat_model]` reference for more details on `configurable_fields` and `config_prefix`.

## Using a configurable model declaratively

We can call declarative operations like `bind_tools`, `with_structured_output`, `with_configurable`, etc. on a configurable model and chain a configurable model in the same way that we would a regularly instantiated chat model object.

```
from pydantic import BaseModel, Field

class GetWeather(BaseModel):
 """Get the current weather in a given location"""

 location: str = Field(description="The city and state, e.g. San Francisco, CA")

class GetPopulation(BaseModel):
 """Get the current population in a given location"""

 location: str = Field(description="The city and state, e.g. San Francisco, CA")

model = init_chat_model(temperature=0)
model_with_tools = model.bind_tools([GetWeather, GetPopulation])

model_with_tools.invoke(
 "what's bigger in 2024 LA or NYC", config={"configurable": {"model": "gpt-5.4-m
ini"}}
).tool_calls
```

```
[
 {
 'name': 'GetPopulation',
 'args': {'location': 'Los Angeles, CA'},
 'id': 'call_Ga9m8FAArIyEjItHmztPYA22',
 'type': 'tool_call'
 },
 {
 'name': 'GetPopulation',
 'args': {'location': 'New York, NY'},
 'id': 'call_jh2dEvBaAHRaw5JUDth0s7rt',
 'type': 'tool_call'
 }
]
```

```
model_with_tools.invoke(
 "what's bigger in 2024 LA or NYC",
 config={"configurable": {"model": "claude-sonnet-4-6"}},
).tool_calls
```



```
[
 {
 'name': 'GetPopulation',
 'args': {'location': 'Los Angeles, CA'},
 'id': 'toolu_01JMufPf4F4t2zLj7miFeqXp',
 'type': 'tool_call'
 },
 {
 'name': 'GetPopulation',
 'args': {'location': 'New York City, NY'},
 'id': 'toolu_01RQBHcE8kEEbYTuuS8WqY1u',
 'type': 'tool_call'
 }
]
```

## Dynamic model selection

Dynamic models are selected at

runtime

based on the current

state

and context. This enables sophisticated routing logic and cost optimization.

### python

To use a dynamic model, create middleware using the `@wrap_model_call` decorator that modifies the model in the request:

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

basic_model = ChatOpenAI(model="gpt-5.4-mini")
advanced_model = ChatOpenAI(model="gpt-5.5")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
 """Choose model based on conversation complexity."""
 message_count = len(request.state["messages"])

 if message_count > 10:
 # Use an advanced model for longer conversations
 model = advanced_model
```

```

else:
 model = basic_model

return handler(request.override(model=model))

agent = create_agent(
 model=basic_model, # Default model
 tools=[],
 middleware=[dynamic_model_selection]
)

```

## 주의

Pre-bound models (models with `@bind_tools` [`BaseChatModel.bind_tools`] already called) are not supported when using structured output. If you need dynamic model selection with structured output, ensure the models passed to the middleware are not pre-bound.

## js

To use a dynamic model, create middleware with `wrapModelCall` that modifies the model in the request:

```

import { ChatOpenAI } from "@langchain/openai";
import { createAgent, createMiddleware } from "langchain";

const basicModel = new ChatOpenAI({ model: "gpt-5.4-mini" });
const advancedModel = new ChatOpenAI({ model: "gpt-5.5" });

const dynamicModelSelection = createMiddleware({
 name: "DynamicModelSelection",
 wrapModelCall: (request, handler) => {
 // Choose model based on conversation complexity
 const messageCount = request.messages.length;

 return handler({
 ...request,
 model: messageCount > 10 ? advancedModel : basicModel,
 });
 },
});

const agent = createAgent({
 model: "gpt-5.4-mini", // Base model (used when messageCount ≤ 10)
 tools: [],
 middleware: [dynamicModelSelection],
});

```

For more details on middleware and advanced patterns, see the [middleware documentation](#).

#### 팁

For model configuration details, see [Models](#). For dynamic model selection patterns, see [Dynamic model in middleware](#).

# LangSmith Observability

langchain/observability.mdx

## python

As you build and run agents with LangChain, you need visibility into how they behave: which [tools](#) they call, what prompts they generate, and how they make decisions. LangChain agents built with `@[ create_agent ]` automatically support tracing through [LangSmith](#), a platform for capturing, debugging, evaluating, and monitoring LLM application behavior.

## js

As you build and run agents with LangChain, you need visibility into how they behave: which [tools](#) they call, what prompts they generate, and how they make decisions. LangChain agents built with `@[ createAgent ]` automatically support tracing through [LangSmith](#), a platform for capturing, debugging, evaluating, and monitoring LLM application behavior.

[Traces](#) record every step of your agent's execution, from the initial user input to the final response, including all tool calls, model interactions, and decision points. This execution data helps you debug issues, evaluate performance across different inputs, and monitor usage patterns in production.

This guide shows you how to enable tracing for your LangChain agents and use LangSmith to analyze their execution.

## Prerequisites

Before you begin, ensure you have the following:

- **A LangSmith account:** Sign up (for free) or log in at [smith.langchain.com](https://smith.langchain.com).
- **A LangSmith API key:** Follow the [Create an API key](#) guide.

## Enable tracing

All LangChain agents automatically support LangSmith tracing. To enable it, set the following environment variables:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
```

# Quickstart

No extra code is needed to log a trace to LangSmith. Just run your agent code as you normally would:

## python

```
from langchain.agents import create_agent

def send_email(to: str, subject: str, body: str):
 """Send an email to a recipient."""
 # ... email sending logic
 return f"Email sent to {to}"

def search_web(query: str):
 """Search the web for information."""
 # ... web search logic
 return f"Search results for: {query}"

agent = create_agent(
 model="gpt-5.5",
 tools=[send_email, search_web],
 system_prompt="You are a helpful assistant that can send emails and search the web."
)

Run the agent - all steps will be traced automatically
response = agent.invoke({
 "messages": [{"role": "user", "content": "Search for the latest AI news and email a summary to john@example.com"}]
})
```

## js

```
import { createAgent } from "@langchain/agents";

function sendEmail(to: string, subject: string, body: string): string {
 // ... email sending logic
 return `Email sent to ${to}`;
}

function searchWeb(query: string): string {
 // ... web search logic
 return `Search results for: ${query}`;
}

const agent = createAgent({
 model: "gpt-5.5",
```

```
tools: [sendEmail, searchWeb],
systemPrompt: "You are a helpful assistant that can send emails and search the
web."
});

// Run the agent - all steps will be traced automatically
const response = await agent.invoke({
 messages: [{ role: "user", content: "Search for the latest AI news and email a
summary to john@example.com" }]
});
```

By default, the trace will be logged to the project with the name `default`. To configure a custom project name, see [Log to a project](#).

**python**

**js**

# LangChain overview

---

langchain/overview.mdx

**Agent = Model + Harness.** LangChain provides `create_agent`: a minimal, highly configurable harness. The harness is everything around the model loop: the prompt, the tools, and any middleware that shapes behavior. Start with the primitives and compose exactly what your use case needs. Supports [OpenAI](#), [Anthropic](#), [Google](#), and [more](#).

## 팁

### LangChain vs. LangGraph vs. Deep Agents

Start with [Deep Agents](#) for a "batteries-included" agent with features like automatic context compression, a virtual filesystem, and subagent-spawning. Deep Agents are built on LangChain [agents](#) which you can also use directly.

Use [LangChain](#) (`create_agent`) for a highly customizable harness, easily tailored to your use case and data.

Use [LangGraph](#), our low-level orchestration framework, for advanced needs combining deterministic and agentic workflows.

Use [LangSmith](#) to trace, debug, and evaluate agents built with any of these frameworks. Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

---

Create an agent

This example demonstrates how to create a simple LangChain agent with a custom tool:

**python**

```
```python OpenAI
```

```
pip install -qU langchain  
"langchain[openai]"
```

```
from langchain.agents import create_agent
```

```
def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always  
sunny in {city}!"
```

```
agent = create_agent( model="openai:gpt-5.5", tools=[get_weather],
system_prompt="You are a helpful assistant", )
```

```
result = agent.invoke( {"messages": [{"role": "user", "content": "What's the weather in
San Francisco?"}]} ) print(result["messages"][-1].content_blocks)
```

```
```python Google Gemini
pip install -qU langchain "langchain[google-genai]"
from langchain.agents import create_agent

def get_weather(city: str) -> str:
 """Get weather for a given city."""
 return f"It's always sunny in {city}!"

agent = create_agent(
 model="google_genai:gemini-2.5-flash-lite",
 tools=[get_weather],
 system_prompt="You are a helpful assistant",
)

result = agent.invoke(
 {"messages": [{"role": "user", "content": "What's the weather in San Francisc
o?"}]}
)
print(result["messages"][-1].content_blocks)
```

```
```python Claude (Anthropic)
```

pip install -qU langchain "langchain[anthropic]"

```
from langchain.agents import create_agent
```

```
def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always
sunny in {city}!"
```

```
agent = create_agent( model="claude-sonnet-4-6", tools=[get_weather],
system_prompt="You are a helpful assistant", )
```

```
result = agent.invoke( {"messages": [{"role": "user", "content": "What's the weather in
San Francisco?"}]} ) print(result["messages"][-1].content_blocks)
```

```
```python OpenRouter
pip install -qU langchain langchain-openrouter
from langchain.agents import create_agent
```



```
def get_weather(city: str) -> str:
 """Get weather for a given city."""
 return f"It's always sunny in {city}!"

agent = create_agent(
 model="openrouter:anthropic/claude-sonnet-4-6",
 tools=[get_weather],
 system_prompt="You are a helpful assistant",
)

result = agent.invoke(
 {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)
```

```python Fireworks

pip install -qU langchain langchain-fireworks

from langchain.agents import create_agent

def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"

agent = create_agent(model="fireworks:accounts/fireworks/models/qwen3p5-397b-a17b", tools=[get_weather], system_prompt="You are a helpful assistant",)

result = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}) print(result["messages"][-1].content_blocks)

```
```python Baseten
pip install -qU langchain langchain-baseten
from langchain.agents import create_agent

def get_weather(city: str) -> str:
 """Get weather for a given city."""
 return f"It's always sunny in {city}!"

agent = create_agent(
 model="baseten:zai-org/GLM-5.2",
 tools=[get_weather],
 system_prompt="You are a helpful assistant",
)

result = agent.invoke(
```

```

 {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)

```

```python Ollama

pip install -qU langchain langchain-ollama

```

from langchain.agents import create_agent

```

```

def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"

```

```

agent = create_agent( model="ollama:devstral-2", tools=[get_weather],
system_prompt="You are a helpful assistant", )

```

```

result = agent.invoke( {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]} ) print(result["messages"][-1].content_blocks)

```

```

```python Azure
pip install -qU langchain "langchain[openai]"
import os
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

def get_weather(city: str) -> str:
 """Get weather for a given city."""
 return f"It's always sunny in {city}!"

model = init_chat_model(
 "azure_openai:gpt-5.5",
 azure_deployment=os.environ["AZURE_OPENAI_DEPLOYMENT_NAME"],
)
agent = create_agent(
 model=model,
 tools=[get_weather],
 system_prompt="You are a helpful assistant",
)

result = agent.invoke(
 {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)

```

```python AWS Bedrock

`pip install -qU langchain langchain-aws`

```
from langchain.agents import create_agent
```

```
def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"
```

US cross-region inference profile; use `global.anthropic.claude-sonnet-4-6` for worldwide routing.

```
agent = create_agent( model="bedrock_converse:us.anthropic.claude-sonnet-4-6",  
tools=[get_weather], system_prompt="You are a helpful assistant", )
```

```
result = agent.invoke( {"messages": [{"role": "user", "content": "What's the weather in  
San Francisco?"}]} ) print(result["messages"][-1].content_blocks)
```

```
```python HuggingFace  
pip install -qU langchain "langchain[huggingface]"
from langchain.agents import create_agent

def get_weather(city: str) -> str:
 """Get weather for a given city."""
 return f"It's always sunny in {city}!"

agent = create_agent(
 model="huggingface:microsoft/Phi-3-mini-4k-instruct",
 tools=[get_weather],
 system_prompt="You are a helpful assistant",
)

result = agent.invoke(
 {"messages": [{"role": "user", "content": "What's the weather in San Francisc
o?"}]}
)
print(result["messages"][-1].content_blocks)
```

js

```
```ts OpenAI // First install: npm install langchain zod @langchain/openai import {  
createAgent, tool } from "langchain"; import * as z from "zod";
```

```
const getWeather = tool( (input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), } );
```

```
const agent = createAgent({ model: "gpt-5.5", tools: [getWeather], });
```

```
console.log( await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }) );
```

```
```ts Google Gemini
// First install: npm install langchain zod @langchain/google
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
 (input) => `It's always sunny in ${input.city}!`,
 {
 name: "get_weather",
 description: "Get the weather for a given city",
 schema: z.object({
 city: z.string().describe("The city to get the weather for"),
 }),
 }
);

const agent = createAgent({
 model: "google:gemin-2.5-flash-lite",
 tools: [getWeather],
});

console.log(
 await agent.invoke({
 messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
 })
);
```

```
```ts Claude (Anthropic) // First install: npm install langchain zod
@langchain/anthropic import { createAgent, tool } from "langchain"; import * as z from
"zod";
```

```
const getWeather = tool( (input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), } );
```

```
const agent = createAgent({ model: "claude-sonnet-4-6", tools: [getWeather], });
```

```
console.log( await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }) );
```

```

```ts OpenRouter
// First install: npm install langchain zod @langchain/openrouter
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
 (input) => `It's always sunny in ${input.city}!`,
 {
 name: "get_weather",
 description: "Get the weather for a given city",
 schema: z.object({
 city: z.string().describe("The city to get the weather for"),
 }),
 }
);

const agent = createAgent({
 model: "openrouter:anthropic/claude-sonnet-4-6",
 tools: [getWeather],
});

console.log(
 await agent.invoke({
 messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
 })
);

```

```

```ts Fireworks // First install: npm install langchain zod
import { createAgent, tool } from "langchain"; import * as z from "zod";

const getWeather = tool( (input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), } );

const agent = createAgent({ model: "fireworks:accounts/fireworks/models/qwen3p5-
397b-a17b", tools: [getWeather], });

console.log( await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }) );

```

```

```ts Baseten
// First install: npm install langchain zod
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
 (input) => `It's always sunny in ${input.city}!`,
 {
 name: "get_weather",

```

```

 description: "Get the weather for a given city",
 schema: z.object({
 city: z.string().describe("The city to get the weather for"),
 }),
 },
}
);

const agent = createAgent({
 model: "baseten:zai-org/GLM-5.2",
 tools: [getWeather],
});

console.log(
 await agent.invoke({
 messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
 })
);

```

```ts Ollama // First install: npm install langchain zod @langchain/ollama import { createAgent, tool } from "langchain"; import \* as z from "zod";

```

const getWeather = tool( (input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), } );

```

```

const agent = createAgent({ model: "ollama:devstral-2", tools: [getWeather], });

```

```

console.log( await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }) );

```

```

```ts Azure
// First install: npm install langchain zod @langchain/openai
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
 (input) => `It's always sunny in ${input.city}!`,
 {
 name: "get_weather",
 description: "Get the weather for a given city",
 schema: z.object({
 city: z.string().describe("The city to get the weather for"),
 }),
 }
);

const agent = createAgent({
 model: "azure_openai:gpt-5.5",
 tools: [getWeather],
});

```

```

console.log(
 await agent.invoke({
 messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
 })
);

```

```

` ``ts AWS Bedrock // First install: npm install langchain zod @langchain/aws import {
createAgent, tool } from "langchain"; import * as z from "zod";

```

```

const getWeather = tool((input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), });

```

```

const agent = createAgent({ model: "bedrock:gpt-5.5", tools: [getWeather], });

```

```

console.log(await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }));

```

```

</div>

```

See the [Installation instructions](/oss/langchain/install) and [Quickstart guide](/oss/langchain/quickstart) to get started building your own agents and applications with LangChain.

```

<div class="callout tip" markdown="1">

```

```

<p class="callout-label">팁</p>

```

Use [LangSmith](/langsmith/observability) to trace requests, debug agent behavior, and evaluate outputs. Set `LANGSMITH\_TRACING=true` and your API key to get started.

```

</div>

```

```

##

```

Core benefits

```

[Standard model interface](/oss/langchain/models)

```

Use one interface for chat models, embeddings, and more across providers. Switch models with minimal code changes and keep your application portable as requirements evolve.

```

[Highly configurable harness](/oss/langchain/agents)

```

Start with `create\_agent` as a minimal harness and add capabilities incrementally through middleware. Compose only what your use case needs, from guardrails and retrievers to routing and custom tool policies.

**[Built on top of LangGraph](/oss/langgraph/overview)**

LangChain's agents are built on top of LangGraph. This allows us to take advantage of LangGraph's durable execution, human-in-the-loop support, persistence, and more.

**[Debug with LangSmith](/langsmith/observability)**

Inspect traces, tool calls, state transitions, and latency in one place. Find failure modes, evaluate quality, and improve agent behavior with execution data.

<div class="page-break"></div>

## # Philosophy

<p class="source">langchain/philosophy.md</p>

LangChain is driven by a few core beliefs:

- Large Language Models (LLMs) are great, powerful new technology.
- LLMs are even better when you combine them with external sources of data.
- LLMs will transform what the applications of the future look like. Specifically, the applications of the future will look more and more agentic.
- It is still very early on in that transformation.
- While it's easy to build a prototype of those agentic applications, it's still really hard to build agents that are reliable enough to put into production.

Today developers can choose how they build agents: use [LangChain](/oss/langchain/overview) for maximum flexibility and control, or [Deep Agents](/oss/langchain/overview) which allows for similar flexibility and control but comes with opinionated built-in planning, filesystem tools, subagents, and context management. Both are built on [LangGraph](/oss/langgraph/overview).

With LangChain, we have two core focuses:

**We want to enable developers to build with the best models.**

Different providers expose different APIs, with different model parameters and different message formats.

Standardizing these model inputs and outputs is a core focus, making it easy for developer to easily change to the most recent state-of-the-art model, avoiding lock-in.

**We want to make it easy to use models to orchestrate more complex flows that interact with other data and computation.**



Models should be used for more than just *\*text generation\** - they should also be used to orchestrate more complex flows that interact with other data. LangChain makes it easy to define [tools](/oss/langchain/tools) that LLMs can use dynamically, as well as help with parsing of and access to unstructured data.

## ## History

Given the constant rate of change in the field, LangChain has also evolved over time. Below is a brief timeline of how LangChain has changed over the years, evolving alongside what it means to build with LLMs:

**\*\*2022-10-24\*\***

A month before ChatGPT, **\*\*LangChain was launched as a Python package\*\***. It consisted of two main components:

- LLM abstractions
  - "Chains", or predetermined steps of computation to run, for common use cases. For example - RAG: run a retrieval step, then run a generation step.

The name LangChain comes from "Language" (like Language models) and "Chains".

**\*\*2022-12\*\***

The first general purpose agents were added to LangChain.

These general purpose agents were based on the [ReAct paper](https://arxiv.org/abs/2210.03629) (ReAct standing for Reasoning and Acting). They used LLMs to generate JSON that represented tool calls, and then parsed that JSON to determine what tools to call.

**\*\*2023-01\*\***

OpenAI releases a 'Chat Completion' API.

Previously, models took in strings and returned a string. In the ChatCompletions API, they evolved to take in a list of messages and return a message. Other model providers followed suit, and LangChain updated to work with lists of messages.

**\*\*2023-01\*\***

LangChain releases a JavaScript version.

LLMs and agents will change how applications are built and JavaScript is the language of application developers.

**\*\*2023-02\*\***

**\*\*LangChain Inc. was formed as a company\*\*** around the open source LangChain project

t.

The main goal was to "make intelligent agents ubiquitous". The team recognized that while LangChain was a key part (LangChain made it simple to get started with LLMs), there was also a need for other components.

**\*\*2023-03\*\***

OpenAI releases 'function calling' in their API.

This allowed the API to explicitly generate payloads that represented tool calls. Other model providers followed suit, and LangChain was updated to use this as the preferred method for tool calling (rather than parsing JSON).

**\*\*2023-06\*\***

**\*\*LangSmith is released\*\*** as closed source platform by LangChain Inc., providing observability and evals.

The main issue with building agents is getting them to be reliable, and LangSmith, which provides observability and evals, was built to solve that need. LangChain was updated to integrate seamlessly with LangSmith.

**\*\*2024-01\*\***

**\*\*LangChain releases 0.1.0\*\***, its first non-0.0.x.

The industry matured from prototypes to production, and as such, LangChain increased its focus on stability.

**\*\*2024-02\*\***

**\*\*LangGraph is released\*\*** as an open-source library.

The original LangChain had two focuses: LLM abstractions, and high-level interfaces for getting started with common applications; however, it was missing a low-level orchestration layer that allowed developers to control the exact flow of their agent. Enter: LangGraph.

When building LangGraph, we learned from lessons when building LangChain and added functionality we discovered was needed: streaming, durable execution, short-term memory, human-in-the-loop, and more.

**\*\*2024-06\*\***

**\*\*LangChain has over 700 integrations.\*\***

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

Integrations were split out of the core LangChain package, and either moved into their own standalone packages (for the core integrations) or `langchain-community`.

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

Integrations were split out of the core LangChain package, and either moved into their own standalone packages (for the core integrations) or `@langchain/community`.

</div>

**\*\*2024-10\*\***

LangGraph becomes the preferred way to build any AI application that is more than a single LLM call.

As developers tried to improve the reliability of their applications, they needed more control than the high-level interfaces provided. LangGraph provided that low-level flexibility. Most chains and agents were marked as deprecated in LangChain with guides on how to migrate them to LangGraph. There is still one high-level abstraction created in LangGraph: an agent abstraction. It is built on top of low-level LangGraph and has the same interface as the ReAct agents from LangChain.

**\*\*2025-04\*\***

Model APIs become more multimodal.

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

Models started to accept files, images, videos, and more. We updated the `langchain-core` message format accordingly to allow developers to specify these multimodal inputs in a standard way.

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

Models started to accept files, images, videos, and more. We updated the `@langchain/core` message format accordingly to allow developers to specify these multimodal inputs in a standard way.

</div>

**\*\*2025-10-20\*\***

**\*\*LangChain releases 1.0\*\*** with two major changes:

1. Complete revamp of all chains and agents in `langchain`. All chains and agents are now replaced with only one high level abstraction: an agent abstraction built on top of LangGraph. This was the high-level abstraction that was originally created in LangGraph, but just moved to LangChain.

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

For users still using old LangChain chains/agents who do NOT want to upgrade (note: we recommend you do), you can continue using old LangChain by installing the `langchain-classic` package.

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

For users still using old LangChain chains/agents who do NOT want to upgrade (note: we recommend you do), you can continue using old LangChain by installing the `@langchain/classic` package.

</div>

2. A standard message content format: Model APIs evolved from returning messages with a simple content string to more complex output types - reasoning blocks, citations, server-side tool calls, etc. LangChain evolved its message formats to standardize these across providers.

**\*\*2026-03-15\*\***

**\*\*Deep Agents is released\*\*** as an open-source agent harness built on LangGraph.

While LangChain provides flexible building blocks for custom agent architectures, [Deep Agents](/oss/langchain/overview) offers a batteries-included option for complex, long-running tasks like research and coding. It adds built-in planning tools, a virtual filesystem with pluggable backends (in-memory, disk, LangGraph store, sandboxes), and subagent spawning for context isolation. Use Deep Agents for more autonomous agents with predefined tools; use LangChain for full control over your agent architecture.

<div class="page-break"></div>

# Quickstart

<p class="source">langchain/quickstart.mdx</p>

This quickstart shows you how to create a fully functional AI agent in just a few minutes.

Build a basic LangChain agent in this working directory by following the LangChain quickstart.

### ## Step 1: Read the guide

Detect whether this project uses Python or TypeScript/JavaScript. Fetch and follow the matching page; treat it as the source of truth for package names, model strings, and code:

- Python: <https://docs.langchain.com/oss/python/langchain/quickstart.md>
- TypeScript: <https://docs.langchain.com/oss/javascript/langchain/quickstart.md>

### ## Step 2: Install dependencies

Install the packages from the guide with the package manager already used in this project (`uv`, `pip`, `npm`, `pnpm`, `yarn`, or `bun`). Prefer pinning current stable versions over a floating `latest` tag when the project already pins versions.

### ## Step 3: Configure model credentials

Check whether a supported provider API key is already set (for example `OPENAI\_API\_KEY`, `GOOGLE\_API\_KEY`, or `ANTHROPIC\_API\_KEY`). If none is set, ask the user which provider to use, then stop and wait while they create a key and set it in the shell or a `.env` file. Do not invent, hardcode, or commit API keys.

### ## Step 4: Implement the basic agent

Create the "Build a basic agent" example from the guide: `create\_agent`, a simple tool such as `get\_weather`, a short system prompt, and an `invoke` call. Use a model string for the provider the user chose. Print the final message content (Python: `.content\_blocks`) so the user can verify the run.

### ## Step 5: Optional LangSmith tracing

Ask the user if they want tracing. If yes, ask them to set `LANGSMITH\_TRACING=true` and `LANGSMITH\_API\_KEY` themselves, then re-run. Do not invent a LangSmith key.

### ## Rules

- Stay scoped to this quickstart. Do not add unrelated frameworks, evals, or production deployment.
- Prefer `create\_agent` from `langchain.agents` / `langchain` as shown in the guide.
- Ask rather than guess when a secret, provider choice, or project convention is unclear.

<div class="callout tip" markdown="1">

<p class="callout-label">팁</p>

**\*\*Using an AI coding assistant?\*\***

- Install the [LangChain Docs MCP servers](/use-these-docs) to give your agent access to up-to-date LangChain documentation and examples.

- Install [LangChain Skills](https://github.com/langchain-ai/langchain-skills) to improve your agent's performance on LangChain ecosystem tasks.

</div>

## Install dependencies

Install the following packages to follow along:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

```
```bash uv
uv python pin 3.11
uv init
uv add langchain
uv sync
```

```
```bash pip
```

## Install Python 3.11+ separately if needed.

---

pip install -U langchain

```
```bash venv
# Install Python 3.11+ separately if needed.
python3.11 -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -U langchain
```

js

```
```bash npm npm install langchain @langchain/core
```

## Requires Node.js 22+

---

```
```bash pnpm
pnpm add langchain @langchain/core
# Requires Node.js 22+
```

```
```bash yarn yarn add langchain @langchain/core
```

## Requires Node.js 22+

---

```
```bash bun
bun add langchain @langchain/core
# Requires Bun v1.0.0+
```

Set up API keys

Get an API key from [any supported model provider](#) (for example, Google Gemini or OpenAI).

Set the API keys in your shell or in a `.env` file, for example:

OpenAI

```
```bash Shell export OPENAI_API_KEY="your-api-key"
```

```
```bash .env
OPENAI_API_KEY=your-api-key
```

Google Gemini

```
```bash Shell export GOOGLE_API_KEY="your-api-key"
```

```
```bash .env
GOOGLE_API_KEY=your-api-key
```

Claude (Anthropic)

```
```bash Shell export ANTHROPIC_API_KEY="your-api-key"
```

```
```bash .env
ANTHROPIC_API_KEY=your-api-key
```

OpenRouter

```
```bash Shell export OPENROUTER_API_KEY="your-api-key"
```

```
```bash .env
OPENROUTER_API_KEY=your-api-key
```

Fireworks

```
```bash Shell export FIREWORKS_API_KEY="your-api-key"
```

```
```bash .env
FIREWORKS_API_KEY=your-api-key
```

Baseten

```
```bash Shell export BASETEN_API_KEY="your-api-key"
```

```
```bash .env
BASETEN_API_KEY=your-api-key
```

Ollama

```
```bash Shell
```

# Local: Ollama must be running (<https://ollama.com>)

---

# Cloud: Set your Ollama API key for hosted inference

---

```
export OLLAMA_API_KEY="your-api-key"
```

```
```bash .env
# Local: Ollama must be running (https://ollama.com)
```



```
# Cloud: Set your Ollama API key for hosted inference
OLLAMA_API_KEY=your-api-key
```

Azure

```
```bash Shell export AZURE_OPENAI_API_KEY="your-api-key" export
AZURE_OPENAI_ENDPOINT="https://your-resource.openai.azure.com" export
AZURE_OPENAI_DEPLOYMENT_NAME="your-deployment"
```

```
```bash .env
AZURE_OPENAI_API_KEY=your-api-key
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com
AZURE_OPENAI_DEPLOYMENT_NAME=your-deployment
```

AWS Bedrock

```
```bash Shell export AWS_ACCESS_KEY_ID="your-access-key" export
AWS_SECRET_ACCESS_KEY="your-secret-key" export AWS_REGION="us-east-1"
```

```
```bash .env
AWS_ACCESS_KEY_ID=your-access-key
AWS_SECRET_ACCESS_KEY=your-secret-key
AWS_REGION=us-east-1
```

HuggingFace

```
```bash Shell export HUGGINGFACEHUB_API_TOKEN="hf_..."
```

```
```bash .env
HUGGINGFACEHUB_API_TOKEN=hf_...
```

Other

See the full list of supported [chat model integrations](#).

python

To load a `.env` file, use `python-dotenv` (`load_dotenv()`).

js

To load a `.env` file, use `dotenv` (`import "dotenv/config"`).

팁

Using LangSmith Gateway

The [LangSmith Gateway](#) routes most major providers through LangSmith. You can [bring your own provider keys](#), or use [Gateway Credits](#) to access models without a provider key.

Build a basic agent

Start by creating a simple agent that can answer questions and call tools. The agent in this example uses the chosen language model, a basic weather function as a tool, and a simple prompt to guide its behavior:

python

```
```python
from langchain.agents import create_agent

def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"

agent = create_agent(model="openai:gpt-5.5", tools=[get_weather],
system_prompt="You are a helpful assistant",)

result = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]})
print(result["messages"][-1].content_blocks)
```

```
```python
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="google_genai:gemin-2.5-flash-lite",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)
```

```
```python
from langchain.agents import create_agent
```

```
def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"
```

```
agent = create_agent(model="claude-sonnet-4-6", tools=[get_weather],
system_prompt="You are a helpful assistant",)
```

```
result = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in
San Francisco?"}]}) print(result["messages"][-1].content_blocks)
```

```
```python OpenRouter  
from langchain.agents import create_agent  
  
def get_weather(city: str) -> str:  
    """Get weather for a given city."""  
    return f"It's always sunny in {city}!"  
  
agent = create_agent(  
    model="openrouter:anthropic/claude-sonnet-4-6",  
    tools=[get_weather],  
    system_prompt="You are a helpful assistant",  
)  
  
result = agent.invoke(  
    {"messages": [{"role": "user", "content": "What's the weather in San Francisc  
o?"}]}  
)  
print(result["messages"][-1].content_blocks)
```

```
```python Fireworks from langchain.agents import create_agent
```

```
def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"
```

```
agent = create_agent(model="fireworks:accounts/fireworks/models/qwen3p5-397b-
a17b", tools=[get_weather], system_prompt="You are a helpful assistant",)
```

```
result = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in
San Francisco?"}]}) print(result["messages"][-1].content_blocks)
```

```
```python Baseten  
from langchain.agents import create_agent  
  
def get_weather(city: str) -> str:  
    """Get weather for a given city."""  
    return f"It's always sunny in {city}!"  
  
agent = create_agent(  
    model="baseten:zai-org/GLM-5.2",  
    tools=[get_weather],
```

```

    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)

```

```

```python Ollama from langchain.agents import create_agent

```

```

def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"

```

```

agent = create_agent(model="ollama:devstral-2", tools=[get_weather],
system_prompt="You are a helpful assistant",)

```

```

result = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}) print(result["messages"][-1].content_blocks)

```

```

```python Azure
import os
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

model = init_chat_model(
    "azure_openai:gpt-5.5",
    azure_deployment=os.environ["AZURE_OPENAI_DEPLOYMENT_NAME"],
)
agent = create_agent(
    model=model,
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)

```

```

```python AWS Bedrock from langchain.agents import create_agent

```

```

def get_weather(city: str) -> str: """Get weather for a given city.""" return f"It's always sunny in {city}!"

```

```
agent = create_agent(model="bedrock_converse:us.anthropic.claude-sonnet-4-6",
tools=[get_weather], system_prompt="You are a helpful assistant",)
```

```
result = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in
San Francisco?"}]}) print(result["messages"][-1].content_blocks)
```

```
```python HuggingFace
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="huggingface:microsoft/Phi-3-mini-4k-instruct",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisc
o?"}]}
)
print(result["messages"][-1].content_blocks)
```

js

```
```ts OpenAI import { createAgent, tool } from "langchain"; import * as z from "zod";
```

```
const getWeather = tool((input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), });
```

```
const agent = createAgent({ model: "gpt-5.5", tools: [getWeather], });
```

```
console.log(await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }));
```

```
```ts Google Gemini
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
  (input) => `It's always sunny in ${input.city}!`,
  {
    name: "get_weather",
    description: "Get the weather for a given city",
    schema: z.object({
```

```

        city: z.string().describe("The city to get the weather for"),
      }},
    }
  );

const agent = createAgent({
  model: "google:gemini-2.5-flash-lite",
  tools: [getWeather],
});

console.log(
  await agent.invoke({
    messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
  })
);

```

```

```ts Claude (Anthropic) import { createAgent, tool } from "langchain"; import * as z
from "zod";

```

```

const getWeather = tool((input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), });

```

```

const agent = createAgent({ model: "claude-sonnet-4-6", tools: [getWeather], });

```

```

console.log(await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }));

```

```

```ts OpenRouter
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
  (input) => `It's always sunny in ${input.city}!`,
  {
    name: "get_weather",
    description: "Get the weather for a given city",
    schema: z.object({
      city: z.string().describe("The city to get the weather for"),
    }),
  }
);

const agent = createAgent({
  model: "openrouter:anthropic/claude-sonnet-4-6",
  tools: [getWeather],
});

console.log(
  await agent.invoke({

```

```

    messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
  })
);

```

```

```ts
import { createAgent, tool } from "langchain"; import * as z from "zod";

```

```

const getWeather = tool((input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), });

```

```

const agent = createAgent({ model: "fireworks:accounts/fireworks/models/qwen3p5-
397b-a17b", tools: [getWeather], });

```

```

console.log(await agent.invoke({ messages: [{ role: "user", content: "What's the
weather in San Francisco?" }], }));

```

```

```ts
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
  (input) => `It's always sunny in ${input.city}!`,
  {
    name: "get_weather",
    description: "Get the weather for a given city",
    schema: z.object({
      city: z.string().describe("The city to get the weather for"),
    }),
  },
);

const agent = createAgent({
  model: "baseten:zai-org/GLM-5.2",
  tools: [getWeather],
});

console.log(
  await agent.invoke({
    messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
  })
);

```

```

```ts
import { createAgent, tool } from "langchain"; import * as z from "zod";

```

```

const getWeather = tool((input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), });

```

```
const agent = createAgent({ model: "ollama:devstral-2", tools: [getWeather], });

console.log(await agent.invoke({ messages: [{ role: "user", content: "What's the weather in San Francisco?" }], }));
```

```
```ts Azure
import { createAgent, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
  (input) => `It's always sunny in ${input.city}!`,
  {
    name: "get_weather",
    description: "Get the weather for a given city",
    schema: z.object({
      city: z.string().describe("The city to get the weather for"),
    }),
  }
);

const agent = createAgent({
  model: "azure_openai:gpt-5.5",
  tools: [getWeather],
});

console.log(
  await agent.invoke({
    messages: [{ role: "user", content: "What's the weather in San Francisco?" }],
  })
);
```

```
```ts AWS Bedrock import { createAgent, tool } from "langchain"; import * as z from "zod";
```

```
const getWeather = tool((input) => It's always sunny in ${input.city}!, { name:
"get_weather", description: "Get the weather for a given city", schema: z.object({ city:
z.string().describe("The city to get the weather for"), }), });
```

```
const agent = createAgent({ model: "bedrock:gpt-5.5", tools: [getWeather], });
```

```
console.log(await agent.invoke({ messages: [{ role: "user", content: "What's the weather in San Francisco?" }], }));
```

```
</div>
```

When you run the code and prompt the agent to tell you about the weather in San Francisco, the agent uses that input and its available context. The agent understands that you are asking about the weather for the city San Francisco.



sco and therefore calls the weather tool with the provided city name.

```
<div class="callout tip" markdown="1">
```

```
<p class="callout-label">팁</p>
```

You can use [any supported model](/oss/integrations/providers/overview) by changing the model name and setting up the appropriate API key. Trace what is happening inside your agent with [LangSmith](https://smith.langchain.com). Follow the [tracing quickstart](/langsmith/trace-with-langchain) to get set up.

We recommend you also set up [LangSmith Engine](/langsmith/engine) which monitors your traces, detects issues, and proposes fixes.

```
</div>
```

## ## Build a real-world agent

In the following example you will build a research agent that can answer questions about text files.

Along the way you will explore the following concepts:

1. **Detailed system prompts** for better agent behavior
1. **Create tools** that integrate with external data
1. **Model configuration** for consistent responses
1. **Conversational memory** for chat-like interactions
1. **Deep Agents** for built-in features
1. **Testing** your agent

**Define the system prompt**

The system prompt defines your agent's role and behavior. Keep it specific and actionable:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```python wrap
```

```
SYSTEM_PROMPT = """You are a literary data assistant.
```

```
## Capabilities
```

```
- `fetch_text_from_url`: loads document text from a URL into the conversation.
```

```
Do not guess line counts or positions—ground them in tool results from the saved file."""
```

js

```
```ts wrap const SYSTEM_PROMPT = You are a literary data assistant.
```

## Capabilities

- `fetch_text_from_url`: loads document text from a URL into the conversation. Do not guess line counts or positions—ground them in tool results from the saved file.

```
</div>
```

```
Create tools
```

[Tools](/oss/langchain/tools) let a model interact with external systems by calling functions you define.

Tools can depend on [runtime context](/oss/langchain/runtime) and also interact with [agent memory](/oss/langchain/short-term-memory).

This example uses a tool to load a document from a given URL:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```python
import urllib.error
import urllib.request

from langchain.tools import tool

@tool
def fetch_text_from_url(url: str) -> str:
    """Fetch the document from a URL.
    """
    req = urllib.request.Request(
        url,
        headers={"User-Agent": "Mozilla/5.0 (compatible; quickstart-research/1.0)"},
    )
    try:
        with urllib.request.urlopen(req, timeout=120) as resp:
            raw = resp.read()
    except urllib.error.URLError as e:
        return f"Fetch failed: {e}"
    text = raw.decode("utf-8", errors="replace")
    return text
```

팁

Tools should be well-documented: their name, description, and argument names become part of the model's prompt. LangChain's `@tool` decorator adds metadata and enables runtime injection with the `ToolRuntime` parameter. Learn more in the [tools guide](#).

js

```
import { tool } from "@langchain/core/tools";
import { createAgent, initChatModel } from "langchain";
import { z } from "zod";

const fetchTextFromUrl = tool(
  async ({ url }: { url: string }): Promise<string> => {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 120_000);
    try {
      const resp = await fetch(url, {
        headers: {
          "User-Agent": "Mozilla/5.0 (compatible; quickstart-research/1.0)",
        },
        signal: controller.signal,
      });
      if (!resp.ok) {
        return `Fetch failed: HTTP ${resp.status} ${resp.statusText}`;
      }
      return await resp.text();
    } catch (e) {
      const msg = e instanceof Error ? e.message : String(e);
      return `Fetch failed: ${msg}`;
    } finally {
      clearTimeout(timeoutId);
    }
  },
  {
    name: "fetch_text_from_url",
    description: "Fetch the document from a URL.",
    schema: z.object({ url: z.string().url() }),
  },
);
```

참고

[Zod](#) is a library for validating and parsing pre-defined schemas. You can use it to define the input schema for your tools to make sure the agent only calls the tool with the correct arguments.

Alternatively, you can define the `schema` property as a [JSON schema](#) object. Keep in mind that JSON schemas **won't** be validated at runtime.

Example: Using JSON schema for tool input

```
import { tool } from "@langchain/core/tools";

const fetchTextFromUrl = tool(
  async ({ url }: { url: string }): Promise<string> => {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 120_000);
    try {
      const resp = await fetch(url, {
        headers: {
          "User-Agent": "Mozilla/5.0 (compatible; quickstart-research/1.0)",
        },
        signal: controller.signal,
      });
      if (!resp.ok) {
        return `Fetch failed: HTTP ${resp.status} ${resp.statusText}`;
      }
      return await resp.text();
    } catch (e) {
      const msg = e instanceof Error ? e.message : String(e);
      return `Fetch failed: ${msg}`;
    } finally {
      clearTimeout(timeoutId);
    }
  },
  {
    name: "fetch_text_from_url",
    description: "Fetch the document from a URL.",
    schema: {
      type: "object",
      properties: {
        url: {
          type: "string",
          description: "The URL of the document to fetch.",
          format: "uri",
        },
      },
      required: ["url"],
    },
  },
);
```

Configure your model

Set up your [language model](#) with the right parameters for your use case. For example:

python

```
```python OpenAI from langchain.chat_models import init_chat_model

model = init_chat_model("openai:gpt-5.5", temperature=0.5, timeout=300,
max_tokens=25000,)
```

```
```python Google Gemini
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "gemini-3.1-pro-preview",
    model_provider="google-genai",
    temperature=0.5,
    timeout=600,
    max_tokens=25000,
    streaming=True,
)
```

```
```python Claude (Anthropic) from langchain.chat_models import init_chat_model

model = init_chat_model("claude-sonnet-4-6", temperature=0.5, timeout=600,
max_tokens=25000, streaming=True,)
```

```
```python OpenRouter
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "openrouter:anthropic/claude-sonnet-4-6",
    temperature=0.5,
    timeout=300,
    max_tokens=25000,
)
```

```
```python Fireworks from langchain.chat_models import init_chat_model

model = init_chat_model("fireworks:accounts/fireworks/models/qwen3p5-397b-a17b",
temperature=0.5, timeout=300, max_tokens=25000,)
```

```
```python Baseten
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "baseten:zai-org/GLM-5.2",
    temperature=0.5,
    timeout=300,
```

```
max_tokens=25000,  
)
```

```
```python Ollama from langchain.chat_models import init_chat_model
```

```
model = init_chat_model("ollama:devstral-2", temperature=0.5, timeout=300,
max_tokens=25000,)
```

```
```python Azure  
import os  
from langchain.chat_models import init_chat_model  
  
model = init_chat_model(  
    "azure_openai:gpt-5.5",  
    temperature=0.5,  
    timeout=300,  
    max_tokens=25000,  
    azure_deployment=os.environ["AZURE_OPENAI_DEPLOYMENT_NAME"],  
)
```

```
```python AWS Bedrock from langchain.chat_models import init_chat_model
```

```
model = init_chat_model("us.anthropic.claude-sonnet-4-6",
model_provider="bedrock_converse", temperature=0.5, timeout=300,
max_tokens=25000,)
```

```
```python HuggingFace  
from langchain.chat_models import init_chat_model  
  
model = init_chat_model(  
    "microsoft/Phi-3-mini-4k-instruct",  
    model_provider="huggingface",  
    temperature=0.5,  
    timeout=300,  
    max_tokens=25000,  
)
```

js

```
```ts OpenAI import { initChatModel } from "langchain";
```

```
const model = await initChatModel("gpt-5.5", { temperature: 0.5, timeout: 300,
maxTokens: 25000, });
```

```
```ts Google Gemini  
import { initChatModel } from "langchain";  
  
const model = await initChatModel("gemini-3.1-pro-preview", {
```

```
modelProvider: "google",
temperature: 0.5,
timeout: 600_000,
maxTokens: 25000,
});
```

```
```ts Claude (Anthropic) import { initChatModel } from "langchain";
```

```
const model = await initChatModel("claude-sonnet-4-6", { temperature: 0.5, timeout:
300, maxTokens: 25000, });
```

```
```ts OpenRouter
import { initChatModel } from "langchain";

const model = await initChatModel("openrouter:anthropic/claude-sonnet-4-6", {
  temperature: 0.5,
  timeout: 300,
  maxTokens: 25000,
});
```

```
```ts Fireworks import { initChatModel } from "langchain";
```

```
const model = await initChatModel("fireworks:accounts/fireworks/models/qwen3p5-
397b-a17b", { temperature: 0.5, timeout: 300, maxTokens: 25000 });
```

```
```ts Baseten
import { initChatModel } from "langchain";

const model = await initChatModel("baseten:zai-org/GLM-5.2", {
  temperature: 0.5,
  timeout: 300,
  maxTokens: 25000,
});
```

```
```ts Ollama import { initChatModel } from "langchain";
```

```
const model = await initChatModel("ollama:devstral-2", { temperature: 0.5, timeout:
300, maxTokens: 25000, });
```

```
```ts Azure
import { initChatModel } from "langchain";

const model = await initChatModel("azure_openai:gpt-5.5", {
  temperature: 0.5,
  timeout: 300,
  maxTokens: 25000,
});
```

```
```ts AWS Bedrock import { initChatModel } from "langchain";

const model = await initChatModel("bedrock:gpt-5.5", { temperature: 0.5, timeout: 300,
maxTokens: 25000, });
```

</div>

Depending on the model and provider chosen, initialization parameters may vary; refer to their reference pages for details.

**\*\*Add memory\*\***

Add [memory](/oss/langchain/short-term-memory) to your agent to maintain state across interactions. This allows the agent to remember previous conversations and context.

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

```
```python
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
```

js

```
import { MemorySaver } from "@langchain/langgraph";

const checkpointer = new MemorySaver();
```

정보

In production, use a persistent checkpointer that saves message history to a database. See [Add and manage memory](#) for more details.

Create and run the agent

Now assemble your agent with all the components and run it.

There are two different frameworks for creating agents: LangChain agents and deep agents. Both LangChain and deep agents provide you with fine-grained control over tools, memory, and more. The main difference between both is that deep agents come with a range of commonly useful capabilities already built in, such as planning, file system tools, and subagents.

Use deep agents when you want maximum capability with minimal setup; choose LangChain agents when you need fine-grained control.

To compare both in this step, install the `deepagents` package:

python

```
```bash uv uv add deepagents
```

```
```bash pip  
pip install -U deepagents
```

js

```
```bash npm npm install deepagents
```

```
```bash pnpm  
pnpm add deepagents
```

```
```bash yarn yarn add deepagents
```

```
```bash bun  
bun add deepagents
```

주의

This example sends the entire text of *The Great Gatsby* to the model and may take a minute or two to respond. You can view example output in the next step.

Let's try both:

python

```
```python wrap from langchain.agents import create_agent from deepagents import  
create_deep_agent

agent = create_agent(model=model, tools=[fetch_text_from_url],
system_prompt=SYSTEM_PROMPT, checkpointer=checkerpointer,)

deep_agent = create_deep_agent(model=model, tools=[fetch_text_from_url],
system_prompt=SYSTEM_PROMPT, checkpointer=checkerpointer,)
```

```
content = f"""Project Gutenberg hosts a full plain-text copy of F. Scott Fitzgerald's The Great Gatsby. URL: https://www.gutenberg.org/files/64317/64317-0.txt
```

Answer as much as you can:

1) How many lines in the complete Gutenberg file contain the substring `Gatsby` (count lines, not occurrences within a line, each line ends with a line break). 2) The 1-based line number of the first line in the file that contains `Daisy`. 3) A two-sentence neutral synopsis.

```
Do your best on (1) and (2). If at any point you realize you cannot verify
an exact answer with
 your available tools and reasoning, do not fabricate numbers: use `null` for th
at field and spell out
 the limitation in `how_you_computed_counts`. If you encounter any errors please
report what the error was and what the error message was."""
```

```
print("Running create_agent...", flush=True) agent_result = agent.invoke({"messages":
[{"role": "user", "content": content}], config={"configurable": {"thread_id": "great-
gatsby-lc"}},) print("Running create_deep_agent...", flush=True) deep_agent_result =
deep_agent.invoke({"messages": [{"role": "user", "content": content}], config=
{"configurable": {"thread_id": "great-gatsby-da"}},) print("\ncreate_agent:")
print(agent_result["messages"][-1].content_blocks) print("\ncreate_deep_agent:")
print(deep_agent_result["messages"][-1].content_blocks)
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```ts
async function main() {
  const agent = createAgent({
    model,
    tools: [fetchTextFromUrl],
    systemPrompt: SYSTEM_PROMPT,
    checkpoint,
  });

  const deepAgent = createDeepAgent({
    model,
    tools: [fetchTextFromUrl],
    systemPrompt: SYSTEM_PROMPT,
    checkpoint,
  });
```

```
const content = `Project Gutenberg hosts a full plain-text copy of F. Scott Fitzgerald's The Great Gatsby.
```

```
URL: https://www.gutenberg.org/files/64317/64317-0.txt
```

Answer as much as you can:

- 1) How many lines in the complete Gutenberg file contain the substring ``Gatsby`` (count lines, not occurrences within a line, each line ends with a line break).
- 2) The 1-based line number of the first line in the file that contains ``Daisy``.
- 3) A two-sentence neutral synopsis.

Do your best on (1) and (2). If at any point you realize you cannot **verify** an exact answer with

your available tools and reasoning, do not fabricate numbers: use ``null`` for that field and spell out

the limitation in ``how_you_computed_counts``. If you encounter any errors please report what the error was and what the error message was.`;

```
console.log("Running createAgent...");
const agentResult = await agent.invoke(
  { messages: [{ role: "user", content }] },
  { configurable: { thread_id: "great-gatsby-lc" } },
);
console.log("Running createDeepAgent...");
const deepAgentResult = await deepAgent.invoke(
  { messages: [{ role: "user", content }] },
  { configurable: { thread_id: "great-gatsby-da" } },
);

const agentMessages = agentResult.messages;
const deepMessages = deepAgentResult.messages;
console.log("\ncreateAgent:");
console.log(agentMessages[agentMessages.length - 1]!.contentBlocks);
console.log("\ncreateDeepAgent:");
console.log(deepMessages[deepMessages.length - 1]!.contentBlocks);
}

main().catch((err) => {
  console.error(err);
  process.exitCode = 1;
});
```

The following expander has everything together in a runnable script:

Full example code

python

```
```python wrap import urllib.error import urllib.request
```

```
from langchain.agents import create_agent from deepagents import
create_deep_agent from langchain.chat_models import init_chat_model from
langchain.tools import tool from langgraph.checkpoint.memory import
InMemorySaver
```

```
SYSTEM_PROMPT = """You are a literary data assistant.
```

## Capabilities

- `fetch_text_from_url`: loads document text from a URL into the conversation. Do not guess line counts or positions—ground them in tool results from the saved file."""

```
@tool def fetch_text_from_url(url: str) -> str: """Fetch the document from a URL. """
req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0 (compatible;
quickstart-research/1.0)"},) try: with urllib.request.urlopen(req, timeout=120) as resp:
raw = resp.read() except urllib.error.URLError as e: return f"Fetch failed: {e}" text =
raw.decode("utf-8", errors="replace") return text
```

```
model = init_chat_model("gemini-3.1-pro-preview", model_provider="google-genai",
temperature=0.5, timeout=600, max_tokens=25000, streaming=True,)
```

```
checkpointer = InMemorySaver()
```

```
agent = create_agent(model=model, tools=[fetch_text_from_url],
system_prompt=SYSTEM_PROMPT, checkpointer=checkpointer,)
```

```
deep_agent = create_deep_agent(model=model, tools=[fetch_text_from_url],
system_prompt=SYSTEM_PROMPT, checkpointer=checkpointer,)
```

```
content = f"""Project Gutenberg hosts a full plain-text copy of F. Scott Fitzgerald's The
Great Gatsby. URL: https://www.gutenberg.org/files/64317/64317-0.txt
```

Answer as much as you can:

1) How many lines in the complete Gutenberg file contain the substring `Gatsby` (count lines, not occurrences within a line, each line ends with a line break). 2) The 1-based line number of the first line in the file that contains `Daisy`. 3) A two-sentence neutral synopsis.

```
Do your best on (1) and (2). If at any point you realize you cannot verify
an exact answer with
 your available tools and reasoning, do not fabricate numbers: use `null` for th
at field and spell out
 the limitation in `how_you_computed_counts`. If you encounter any errors please
report what the error was and what the error message was."""
```

```

print("Running create_agent...", flush=True) agent_result = agent.invoke({"messages":
[{"role": "user", "content": content}]}], config={"configurable": {"thread_id": "great-
gatsby-lc"}},) print("Running create_deep_agent...", flush=True) deep_agent_result =
deep_agent.invoke({"messages": [{"role": "user", "content": content}]}], config=
{"configurable": {"thread_id": "great-gatsby-da"}},) print("\ncreate_agent:")
print(agent_result["messages"][-1].content_blocks) print("\ncreate_deep_agent:")
print(deep_agent_result["messages"][-1].content_blocks)

```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```ts wrap
```

```

import { MemorySaver } from "@langchain/langgraph";
import { createDeepAgent } from "deepagents";
import { tool } from "@langchain/core/tools";
import { createAgent, initChatModel } from "langchain";
import { z } from "zod";
const SYSTEM_PROMPT = `You are a literary data assistant.

```

```
## Capabilities
```

```

- `fetch_text_from_url`: loads document text from a URL into the conversation.
Do not guess line counts or positions—ground them in tool results from the saved fi
le.`;

```

```

const fetchTextFromUrl = tool(
  async ({ url }: { url: string }): Promise<string> => {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 120_000);
    try {
      const resp = await fetch(url, {
        headers: {
          "User-Agent": "Mozilla/5.0 (compatible; quickstart-research/1.
0)",
        },
        signal: controller.signal,
      });
      if (!resp.ok) {
        return `Fetch failed: HTTP ${resp.status} ${resp.statusText}`;
      }
      return await resp.text();
    } catch (e) {
      const msg = e instanceof Error ? e.message : String(e);
      return `Fetch failed: ${msg}`;
    } finally {

```

```

        clearTimeout(timeoutId);
    }
},
{
    name: "fetch_text_from_url",
    description: "Fetch the document from a URL.",
    schema: z.object({ url: z.string().url() }),
},
);

const model = await initChatModel("gemini-3.1-pro-preview", {
    modelProvider: "google",
    temperature: 0.5,
    timeout: 600_000,
    maxTokens: 25000,
    streaming: true,
});

const checkpointer = new MemorySaver();

async function main() {
    const agent = createAgent({
        model,
        tools: [fetchTextFromUrl],
        systemPrompt: SYSTEM_PROMPT,
        checkpointer,
    });

    const deepAgent = createDeepAgent({
        model,
        tools: [fetchTextFromUrl],
        systemPrompt: SYSTEM_PROMPT,
        checkpointer,
    });

    const content = `Project Gutenberg hosts a full plain-text copy of F. Scott Fitzgerald's The Great Gatsby.
URL: https://www.gutenberg.org/files/64317/64317-0.txt

Answer as much as you can:

1) How many lines in the complete Gutenberg file contain the substring ``Gatsby`` (count lines, not occurrences within a line, each line ends with a line break).
2) The 1-based line number of the first line in the file that contains ``Daisy``.
3) A two-sentence neutral synopsis.

Do your best on (1) and (2). If at any point you realize you cannot **verify** an exact answer with
your available tools and reasoning, do not fabricate numbers: use ``null`` for that field and spell out
the limitation in ``how_you_computed_counts``. If you encounter any err

```

```

ors please report what the error was and what the error message was.`;

console.log("Running createAgent...");
const agentResult = await agent.invoke(
  { messages: [{ role: "user", content }] },
  { configurable: { thread_id: "great-gatsby-lc" } },
);
console.log("Running createDeepAgent...");
const deepAgentResult = await deepAgent.invoke(
  { messages: [{ role: "user", content }] },
  { configurable: { thread_id: "great-gatsby-da" } },
);

const agentMessages = agentResult.messages;
const deepMessages = deepAgentResult.messages;
console.log("\ncreateAgent:");
console.log(agentMessages[agentMessages.length - 1]!.contentBlocks);
console.log("\ncreateDeepAgent:");
console.log(deepMessages[deepMessages.length - 1]!.contentBlocks);
}

main().catch((err) => {
  console.error(err);
  process.exitCode = 1;
});

```

Review the results

The results will differ based on the model and the execution.

LangChain agents

```
```txt wrap expandable **1) Number of lines containing Gatsby:** null`
```

### 2) First line containing Daisy: null

**3) Synopsis:** The Great Gatsby follows the mysterious millionaire Jay Gatsby and his obsession with reuniting with his former lover, Daisy Buchanan, as narrated by his neighbor Nick Carraway. Set against the backdrop of the Roaring Twenties on Long Island, the novel explores themes of wealth, class, and the elusive nature of the American Dream.

**how\_you\_computed\_counts:** I successfully fetched the full text of the eBook using the `fetch_text_from_url` tool. However, because I do not have access to a code execution environment (like Python) or text-processing tools (like `grep`), I cannot deterministically split the text by line breaks, iterate through the thousands of lines, and verify the exact line numbers or match counts. LLMs cannot reliably perform exact line-counting or indexing over massive texts within their context window without external computational tools. As

instructed, rather than fabricating or guessing a number, I have output `null` for the exact counts and positions.

```
Deep agents

```txt wrap expandable
Based on the text fetched directly from the Gutenberg URL and analyzed using filesystem
search tools, here are the answers to your questions:

**1) Lines containing the substring `Gatsby`**
**258** lines contain the exact substring `Gatsby`.

**2) First line containing `Daisy`**
Line **181** is the first line in the file that contains the exact substring `Daisy`.
*(For context, the line reads: "Buchanans. Daisy was my second cousin once removed, and
I'd known Tom")*

**3) Two-sentence neutral synopsis**
*The Great Gatsby* follows the mysterious millionaire Jay Gatsby and his obsessive purs
uit to reunite with his former lover, Daisy Buchanan, in 1920s Long Island. The story i
s narrated by Nick Carraway, who observes the tragic consequences of Gatsby's relentles
s ambition and the shallow materialism of the era's wealthy elite.

***

**How counts were computed:**
When fetching the document from the URL, the file was too large for the standard output
and was automatically saved to the local filesystem by the system (`/large_tool_result
s/x246ax2x`). I then used the `grep` tool to search the saved file for the exact litera
l substrings `Gatsby` and `Daisy`. The `grep` tool returned every matching line along w
ith its 1-based line number. I manually counted the exact number of lines returned for
`Gatsby` (which totaled 258) and identified the first line number returned for `Daisy`
(which was 181). I also verified there were no uppercase variations (`GATSBY` or `DAISY
`) that would have been missed. No errors were encountered during this process.
```

If you look at the output on both tabs, you notice that the LangChain agent provided answers but they are estimates. The agent lacks the tools to answer this question. You may also get errors that the prompt is too long.

The deep agent, on the other hand can:

1. **Plans its approach** using the built-in [write_todos](#) tool to break down the research task.
2. **Loads the file** by calling the `fetch_text_from_url` tool to gather information.
3. **Manages context** by using file system tools ([grep](#) and [read_file](#)).
4. **Spawns subagents** as needed to delegate complex subtasks to specialized subagents.

For LangChain agents, you must implement more capabilities to get a similar level of service and can customize them along the way as needed.

Trace agent calls

Most interesting applications you build with LangChain make many calls to LLMs. As these applications get more complex, it becomes important to be able to inspect what exactly is going on inside your agent. The best way to do this is with [LangSmith](#).

Sign up for a [LangSmith](#) account and set these to start logging traces:

```
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."
```

Once set, run your script again and then inspect what happened during your agent calls on [LangSmith](#).

팁

To learn more about tracing your agent with LangSmith, see the [LangSmith documentation](#).

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Next steps

You now have agents that can:

- **Understand context** and remember conversations
- **Use tools** intelligently
- **Provide structured responses** in a consistent format
- **Handle user-specific information** through context
- **Maintain conversation state** across interactions
- **Plan, research, and synthesize** (deep agents only)

Continue with:

- **LangChain agents:** [Add and manage memory](#), [deploy to production](#)
- **Deep Agents:** [Customization options](#), [persistent memory](#), [deploy to production](#)

Retrieval

langchain/retrieval.mdx

Runtime

langchain/runtime.mdx

Overview

python

LangChain's `@[ create_agent ]` runs on LangGraph's runtime under the hood.

js

LangChain's `createAgent` runs on LangGraph's runtime under the hood.

LangGraph exposes a `@[ Runtime ]` object with the following information:

1. **Context:** static information like user id, db connections, or other dependencies for an agent invocation
2. **Store:** a `@[BaseStore]` instance used for [long-term memory](#).
3. **Stream writer:** an object used for streaming information via the "custom" stream mode
4. **Execution info:** identity and retry information for the current execution (thread ID, run ID, attempt number)
5. **Server info:** server-specific metadata when running on LangGraph Server (assistant ID, graph ID, authenticated user)

python

팁

Runtime context provides **dependency injection** for your tools and middleware. Instead of hardcoding values or using global state, you can inject runtime dependencies (like database connections, user IDs, or configuration) when invoking your agent. This makes your tools more testable, reusable, and flexible.

js

팁

The runtime context is how you thread data through your agent. Rather than storing things in global state, you can attach values—like a database connection,

user session, or configuration—to the context and access them inside tools and middleware. This keeps things stateless, testable, and reusable.

You can access the runtime information within [tools](#) and [middleware](#).

Access

python

When creating an agent with `@[ create_agent ]`, you can specify a `context_schema` to define the structure of the `context` stored in the agent `@[ Runtime ]`.

js

When creating an agent with `createAgent`, you can specify a `contextSchema` to define the structure of the `context` stored in the agent `@[ Runtime ]`.

When invoking the agent, pass the `context` argument with the relevant configuration for the run:

python

```
from dataclasses import dataclass

from langchain.agents import create_agent

@dataclass
class Context:
    user_name: str

agent = create_agent(
    model="gpt-5-nano",
    tools=[...],
    context_schema=Context # [!code highlight]
)

agent.invoke(
    {"messages": [{"role": "user", "content": "What's my name?"}]},
    context=Context(user_name="John Smith") # [!code highlight]
)
```

js

```

import * as z from "zod";
import { createAgent } from "langchain";

const contextSchema = z.object({ // [!code highlight]
  userName: z.string(), // [!code highlight]
}); // [!code highlight]

const agent = createAgent({
  model: "gpt-5.5",
  tools: [
    /* ... */
  ],
  contextSchema, // [!code highlight]
});

const result = await agent.invoke(
  { messages: [{ role: "user", content: "What's my name?" }] },
  { context: { userName: "John Smith" } } // [!code highlight]
);

```

Inside tools

You can access the runtime information inside tools to:

- Access the context
- Read or write long-term memory
- Write to the [custom stream](#) (ex, tool progress / updates)

python

Use the `ToolRuntime` parameter to access the `@[Runtime]` object inside a tool.

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime # [!code highlight]

@dataclass
class Context:
    user_id: str

@tool
def fetch_user_email_preferences(runtime: ToolRuntime[Context]) -> str: # [!code highlight]
    """Fetch the user's email preferences from the store."""
    user_id = runtime.context.user_id # [!code highlight]

    preferences: str = "The user prefers you to write a brief and polite email."
    if runtime.store: # [!code highlight]
        if memory := runtime.store.get(("users",), user_id): # [!code highlight]

```

```
preferences = memory.value["preferences"]

return preferences
```

js

Use the `runtime` parameter to access the `@[ Runtime ]` object inside a tool.

```
import * as z from "zod";
import { tool } from "langchain";
import { type ToolRuntime } from "@langchain/core/tools"; // [!code highlight]

const contextSchema = z.object({
  userName: z.string(),
});

const fetchUserEmailPreferences = tool(
  async (_, runtime: ToolRuntime<any, typeof contextSchema>) => { // [!code highlight]
    const userName = runtime.context?.userName; // [!code highlight]
    if (!userName) {
      throw new Error("userName is required");
    }

    let preferences = "The user prefers you to write a brief and polite email.";
    if (runtime.store) { // [!code highlight]
      const memory = await runtime.store?.get(["users"], userName); // [!code highlight]
      if (memory) {
        preferences = memory.value.preferences;
      }
    }
    return preferences;
  },
  {
    name: "fetch_user_email_preferences",
    description: "Fetch the user's email preferences.",
    schema: z.object({}),
  }
);
```

Execution info and server info inside tools

python

Access execution identity (thread ID, run ID) via `runtime.execution_info`, and server-specific metadata (assistant ID, authenticated user) via `runtime.server_info` when

running on LangGraph Server:

```
from langchain.tools import tool, ToolRuntime

@tool
def context_aware_tool(runtime: ToolRuntime) -> str:
    """A tool that uses execution and server info."""
    # Access thread and run IDs
    info = runtime.execution_info
    print(f"Thread: {info.thread_id}, Run: {info.run_id}") # [!code highlight]

    # Access server info (only available on LangGraph Server)
    server = runtime.server_info
    if server is not None:
        print(f"Assistant: {server.assistant_id}") # [!code highlight]
        if server.user is not None:
            print(f"User: {server.user.identity}") # [!code highlight]

    return "done"
```

`server_info` is `None` when not running on LangGraph Server (e.g., during local development).

js

Access execution identity (thread ID, run ID) via `runtime.executionInfo`, and server-specific metadata (assistant ID, authenticated user) via `runtime.serverInfo` when running on LangGraph Server:

```
import { tool } from "langchain";
import * as z from "zod";

const contextAwareTool = tool(
  async (_input, runtime) => {
    // Access thread and run IDs
    const info = runtime.executionInfo;
    console.log(`Thread: ${info.threadId}, Run: ${info.runId}`); // [!code highlight]

    // Access server info (only available on LangGraph Server)
    const server = runtime.serverInfo;
    if (server != null) {
      console.log(`Assistant: ${server.assistantId}`); // [!code highlight]
      if (server.user != null) {
        console.log(`User: ${server.user.identity}`); // [!code highlight]
      }
    }
  }
);
```

```

    return "done";
  },
  {
    name: "context_aware_tool",
    description: "A tool that uses execution and server info.",
    schema: z.object({}),
  }
);

```

`serverInfo` is `null` when not running on LangGraph Server (e.g., during local development).

python

참고

Requires `deepagents>=0.5.0` (or `langgraph>=1.1.5`) for `runtime.execution_info` and `runtime.server_info`.

js

참고

Requires `deepagents>=1.9.0` (or `@langchain/langgraph>=1.2.8`) for `runtime.executionInfo` and `runtime.serverInfo`.

Inside middleware

You can access runtime information in middleware to create dynamic prompts, modify messages, or control agent behavior based on user context.

python

Use the `Runtime` parameter to access the `@[Runtime]` object inside [node-style hooks](#). For [wrap-style hooks](#), the `Runtime` object is available inside the `@[ModelRequest]` parameter.

```

from dataclasses import dataclass

from langchain.messages import AnyMessage
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import dynamic_prompt, ModelRequest, before_model,
after_model
from langgraph.runtime import Runtime

```



```

@dataclass
class Context:
    user_name: str

# Dynamic prompts
@dynamic_prompt
def dynamic_system_prompt(request: ModelRequest) -> str:
    user_name = request.runtime.context.user_name # [!code highlight]
    system_prompt = f"You are a helpful assistant. Address the user as {user_name}."
    return system_prompt

# Before model hook
@before_model
def log_before_model(state: AgentState, runtime: Runtime[Context]) -> dict | None:
    # [!code highlight]
    print(f"Processing request for user: {runtime.context.user_name}") # [!code highlight]
    return None

# After model hook
@after_model
def log_after_model(state: AgentState, runtime: Runtime[Context]) -> dict | None:
    # [!code highlight]
    print(f"Completed request for user: {runtime.context.user_name}") # [!code highlight]
    return None

agent = create_agent(
    model="gpt-5-nano",
    tools=[...],
    middleware=[dynamic_system_prompt, log_before_model, log_after_model], # [!code highlight]
    context_schema=Context
)

agent.invoke(
    {"messages": [{"role": "user", "content": "What's my name?"}]},
    context=Context(user_name="John Smith")
)

```

js

Use the `runtime` parameter to access the `@[Runtime]` object inside middleware.

```

import * as z from "zod";
import { createAgent, createMiddleware, SystemMessage } from "langchain";

```

```

const contextSchema = z.object({
  userName: z.string(),
});

// Dynamic prompt middleware
const dynamicPromptMiddleware = createMiddleware({
  name: "DynamicPrompt",
  contextSchema,
  beforeModel: (state, runtime) => { // [!code highlight]
    const userName = runtime.context?.userName; // [!code highlight]
    if (!userName) {
      throw new Error("userName is required");
    }

    const systemMsg = `You are a helpful assistant. Address the user as ${userNam
e}`;
    return {
      messages: [new SystemMessage(systemMsg), ...state.messages],
    };
  },
});

// Logging middleware
const loggingMiddleware = createMiddleware({
  name: "Logging",
  contextSchema,
  beforeModel: (state, runtime) => { // [!code highlight]
    console.log(`Processing request for user: ${runtime.context?.userName}`); //
[!code highlight]
    return;
  },
  afterModel: (state, runtime) => { // [!code highlight]
    console.log(`Completed request for user: ${runtime.context?.userName}`); // [!
code highlight]
    return;
  },
});

const agent = createAgent({
  model: "gpt-5.5",
  tools: [
    /* ... */
  ],
  middleware: [dynamicPromptMiddleware, loggingMiddleware], // [!code highlight]
  contextSchema,
});

const result = await agent.invoke(
  { messages: [{ role: "user", content: "What's my name?" }] },
  { context: { userName: "John Smith" } }
);

```

```
);
```

Execution info and server info inside middleware

python

Middleware hooks can also access `runtime.execution_info` and `runtime.server_info`:

```
from langchain.agents import AgentState
from langchain.agents.middleware import before_model
from langgraph.runtime import Runtime

@before_model
def auth_gate(state: AgentState, runtime: Runtime) -> dict | None:
    """Block unauthenticated users when running on LangGraph Server."""
    server = runtime.server_info
    if server is not None and server.user is None: # [!code highlight]
        raise ValueError("Authentication required")
    print(f"Thread: {runtime.execution_info.thread_id}") # [!code highlight]
    return None
```

js

Middleware hooks can also access `runtime.executionInfo` and `runtime.serverInfo`:

```
import { createMiddleware } from "langchain";

const authGate = createMiddleware({
  name: "AuthGate",
  beforeModel: (state, runtime) => {
    const server = runtime.serverInfo;
    if (server !== null && server.user === null) { // [!code highlight]
      throw new Error("Authentication required");
    }
    console.log(`Thread: ${runtime.executionInfo.threadId}`); // [!code highlight]
    return;
  },
});
```

python

참고

Requires `deepagents>=0.5.0` (or `langgraph>=1.1.5`).

js

참고

Requires `deepagents>=1.9.0` (or `@langchain/langgraph>=1.2.8`).

Short-term memory

langchain/short-term-memory.mdx

Overview

Memory is a system that remembers information about previous interactions. For AI agents, memory is crucial because it lets them remember previous interactions, learn from feedback, and adapt to user preferences. As agents tackle more complex tasks with numerous user interactions, this capability becomes essential for both efficiency and user satisfaction.

Short term memory lets your application remember previous interactions within a single thread or conversation.

참고

A thread organizes multiple interactions in a session, similar to the way email groups messages in a single conversation.

Conversation history is the most common form of short-term memory. Long conversations pose a challenge to today's LLMs; a full history may not fit inside an LLM's context window, resulting in a context loss or errors.

Even if your model supports the full context length, most LLMs still perform poorly over long contexts. They get "distracted" by stale or off-topic content, all while suffering from slower response times and higher costs.

Chat models accept context using [messages](#), which include instructions (a system message) and inputs (human messages). In chat applications, messages alternate between human inputs and model responses, resulting in a list of messages that grows longer over time. Because context windows are limited, many applications can benefit from using techniques to remove or "forget" stale information.

팁

Need to remember information **across** conversations? Use [long-term memory](#) to store and recall user-specific or application-level data across different threads and sessions.

Usage

To add short-term memory (thread-level persistence) to an agent, you need to specify a `checkpointers` when creating an agent.

정보

LangChain's agent manages short-term memory as a part of your agent's state.

By storing these in the graph's state, the agent can access the full context for a given conversation while maintaining separation between different threads.

State is persisted to a database (or memory) using a checkpointer so the thread can be resumed at any time.

Short-term memory updates when the agent is invoked or a step (like a tool call) is completed, and the state is read at the start of each step.

python

js

In production

In production, use a checkpointer backed by a database:

python

```
```bash pip install -U langgraph-checkpoint-postgres "psycpg[binary]"
```

```
```bash uv
uv add langgraph-checkpoint-postgres "psycpg[binary]"
```

참고

By default, `langgraph-checkpoint-postgres` installs `psycpg` (Psycpg 3) without extras. The install above adds `psycpg[binary]`, which is recommended for most users. For other options, see the [Psycpg installation docs](#).

```
from langchain.agents import create_agent
from langgraph.checkpoint.postgres import PostgresSaver # [!code highlight]

def get_user_info() -> str:
    """Look up information about the current user."""
    return "No user profile on file."

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # auto create tables in PostgreSQL
    agent = create_agent(
```

```

    "gpt-5.5",
    tools=[get_user_info],
    checkpointer=checkpointer, # [!code highlight]
)

```

js

```

import { PostgresSaver } from "@langchain/langgraph-checkpoint-postgres";

const DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable";
const checkpointer = PostgresSaver.fromConnString(DB_URI);

```

참고

For more checkpointer options including SQLite, Postgres, and Azure Cosmos DB, see the [list of checkpointer libraries](#) in the Persistence documentation.

Customizing agent memory

python

By default, agents use `@[AgentState]` to manage short term memory, specifically the conversation history via a `messages` key.

You can extend `@[AgentState]` to add additional fields. Custom state schemas are passed to `@[create_agent]` using the `@[state_schema]` parameter.

```

from langchain.agents import create_agent, AgentState
from langgraph.checkpoint.memory import InMemorySaver

class CustomAgentState(AgentState): # [!code highlight]
    user_id: str # [!code highlight]
    preferences: dict # [!code highlight]

agent = create_agent(
    "gpt-5.5",
    tools=[get_user_info],
    state_schema=CustomAgentState, # [!code highlight]
    checkpointer=InMemorySaver(),
)

# Custom state can be passed in invoke
result = agent.invoke(
    {

```

```

    "messages": [{ "role": "user", "content": "Hello" }],
    "user_id": "user_123", # [!code highlight]
    "preferences": { "theme": "dark" } # [!code highlight]
  },
  {"configurable": {"thread_id": "1"}})

```

js

You can extend the agent state by creating custom middleware with a state schema. Custom state schemas can be passed using the `stateSchema` parameter in middleware. Use the `StateSchema` class for state definitions preferably (plain Zod objects are also supported).

```

import { createAgent, createMiddleware } from "langchain";
import { StateSchema, MemorySaver } from "@langchain/langgraph";
import * as z from "zod";

const CustomState = new StateSchema({ // [!code highlight]
  userId: z.string(), // [!code highlight]
  preferences: z.record(z.string(), z.any()), // [!code highlight]
}); // [!code highlight]

const stateExtensionMiddleware = createMiddleware({
  name: "StateExtension",
  stateSchema: CustomState, // [!code highlight]
});

const checkpointer = new MemorySaver();
const agent = createAgent({
  model: "gpt-5.5",
  tools: [],
  middleware: [stateExtensionMiddleware], // [!code highlight]
  checkpointer,
});

// Custom state can be passed in invoke
const result = await agent.invoke({
  messages: [{ role: "user", content: "Hello" }],
  userId: "user_123", // [!code highlight]
  preferences: { theme: "dark" }, // [!code highlight]
});

```

Common patterns

With [short-term memory](#) enabled, long conversations can exceed the LLM's context window. Common solutions are:

[Trim messages](#)

Remove first or last N messages (before calling LLM)

[Delete messages](#)

Delete messages from LangGraph state permanently

[Summarize messages](#)

Summarize earlier messages in the history and replace them with a summary

Custom strategies

Custom strategies (e.g., message filtering, etc.)

This allows the agent to keep track of the conversation without exceeding the LLM's context window.

Trim messages

Most LLMs have a maximum supported context window (denominated in tokens).

python

One way to decide when to truncate messages is to count the tokens in the message history and truncate whenever it approaches that limit. If you're using LangChain, you can use the trim messages utility and specify the number of tokens to keep from the list, as well as the `strategy` (e.g., keep the last `max_tokens`) to use for handling the boundary.

js

One way to decide when to truncate messages is to count the tokens in the message history and truncate whenever it approaches that limit. If you're using LangChain, you can use the trim messages utility and specify the number of tokens to keep from the list, as well as the `strategy` (e.g., keep the last `maxTokens`) to use for handling the boundary.

python

To trim message history in an agent, use the `@before_model` middleware decorator:

```
from langchain.messages import RemoveMessage
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent, AgentState
```

```

from langchain.agents.middleware import before_model
from langgraph.runtime import Runtime
from langchain_core.runnables import RunnableConfig
from typing import Any

@before_model
def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """Keep only the last few messages to fit context window."""
    messages = state["messages"]

    if len(messages) <= 3:
        return None # No changes needed

    first_msg = messages[0]
    recent_messages = messages[-3:] if len(messages) % 2 == 0 else messages[-4:]
    new_messages = [first_msg] + recent_messages

    return {
        "messages": [
            RemoveMessage(id=REMOVE_ALL_MESSAGES),
            *new_messages
        ]
    }

agent = create_agent(
    "gpt-5.5",
    tools=[...],
    middleware=[trim_messages],
    checkpointeer=InMemorySaver(),
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

agent.invoke({"messages": "hi, my name is bob"}, config)
agent.invoke({"messages": "write a short poem about cats"}, config)
agent.invoke({"messages": "now do the same but for dogs"}, config)
final_response = agent.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()
"""
===== Ai Message =====

Your name is Bob. You told me that earlier.
If you'd like me to call you a nickname or use a different name, just say the word.
"""

```

js

To trim message history in an agent, use `@[createMiddleware]` with a `beforeModel` hook:

```

import { RemoveMessage } from "@langchain/core/messages";
import { createAgent, createMiddleware } from "langchain";
import { MemorySaver, REMOVE_ALL_MESSAGES } from "@langchain/langgraph";

const trimMessages = createMiddleware({
  name: "TrimMessages",
  beforeModel: (state) => {
    const messages = state.messages;

    if (messages.length <= 3) {
      return; // No changes needed
    }

    const firstMsg = messages[0];
    const recentMessages =
      messages.length % 2 === 0 ? messages.slice(-3) : messages.slice(-4);
    const newMessages = [firstMsg, ...recentMessages];

    return {
      messages: [
        new RemoveMessage({ id: REMOVE_ALL_MESSAGES }),
        ...newMessages,
      ],
    };
  },
});

const checkpointer = new MemorySaver();
const agent = createAgent({
  model: "gpt-5.5",
  tools: [...],
  middleware: [trimMessages],
  checkpointer,
});

```

Delete messages

You can delete messages from the graph state to manage the message history.

This is useful when you want to remove specific messages or clear the entire message history.

python

To delete messages from the graph state, you can use the `RemoveMessage` .

For `RemoveMessage` to work, you need to use a state key with `@[ add_messages ]` [reducer](#).

The default `@[AgentState]` provides this.

To remove specific messages:

```
from langchain.messages import RemoveMessage # [!code highlight]

def delete_messages(state):
    messages = state["messages"]
    if len(messages) > 2:
        # remove the earliest two messages
        return {"messages": [RemoveMessage(id=m.id) for m in messages[:2]]} # [!code highlight]
    de highlight]
```

To remove **all** messages:

```
from langgraph.graph.message import REMOVE_ALL_MESSAGES # [!code highlight]

def delete_messages(state):
    return {"messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES)]} # [!code highlight]
t]
```

js

To delete messages from the graph state, you can use the `RemoveMessage`. For `RemoveMessage` to work, you need to use a state key with `@[messagesStateReducer]` `[messagesStateReducer]` [reducer](#), like `MessagesValue`.

To remove specific messages:

```
import { RemoveMessage } from "@langchain/core/messages";

const deleteMessages = (state) => {
  const messages = state.messages;
  if (messages.length > 2) {
    // remove the earliest two messages
    return {
      messages: messages
        .slice(0, 2)
        .map((m) => new RemoveMessage({ id: m.id })),
    };
  }
};
```

주의

When deleting messages, **make sure** that the resulting message history is valid. Check the limitations of the LLM provider you're using. For example:

- Some providers expect message history to start with a `user` message
- Most providers require `assistant` messages with tool calls to be followed by corresponding `tool` result messages.

python

```
from langchain.messages import RemoveMessage
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import after_model
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.runtime import Runtime
from langchain_core.runnables import RunnableConfig

@after_model
def delete_old_messages(state: AgentState, runtime: Runtime) -> dict | None:
    """Remove old messages to keep conversation manageable."""
    messages = state["messages"]
    if len(messages) > 2:
        # remove the earliest two messages
        return {"messages": [RemoveMessage(id=m.id) for m in messages[:2]]}
    return None

agent = create_agent(
    "gpt-5-nano",
    tools=[...],
    system_prompt="Please be concise and to the point.",
    middleware=[delete_old_messages],
    checkpointer=InMemorySaver(),
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

stream = agent.stream_events(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config,
    version="v3",
)

for snapshot in stream.values:
    print([(message.type, message.content) for message in snapshot["messages"]])

stream = agent.stream_events(
    {"messages": [{"role": "user", "content": "write a short poem about cats"}]},
    config,
    version="v3",
)

for snapshot in stream.values:
```

```

    print([(message.type, message.content) for message in snapshot["messages"]])

stream = agent.stream_events(
    {"messages": [{"role": "user", "content": "what's my name?"}]},
    config,
    version="v3",
)
for snapshot in stream.values:
    print([(message.type, message.content) for message in snapshot["messages"]])

```

```

[('human', "hi! I'm bob")]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! Nice to meet you. How can I help you today? I can answer questions, brainstorm ideas, draft text, explain things, or help with code.')]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! Nice to meet you. How can I help you today? I can answer questions, brainstorm ideas, draft text, explain things, or help with code.'), ('human', "write a short poem about cats")]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! Nice to meet you. How can I help you today? I can answer questions, brainstorm ideas, draft text, explain things, or help with code.'), ('human', "write a short poem about cats"), ('ai', 'There once was a cat on a wall, Who barely moved at all...')]
[('human', 'write a short poem about cats'), ('ai', 'There once was a cat on a wall, Who barely moved at all...')]
[('human', 'write a short poem about cats'), ('ai', 'There once was a cat on a wall, Who barely moved at all...'), ('human', "what's my name?")]
[('human', 'write a short poem about cats'), ('ai', 'There once was a cat on a wall, Who barely moved at all...'), ('human', "what's my name?"), ('ai', "I don't know your name - you haven't told me!")]
[('human', "what's my name?"), ('ai', "I don't know your name - you haven't told me!")]

```

js

```

import { RemoveMessage } from "@langchain/core/messages";
import { createAgent, createMiddleware } from "langchain";
import { MemorySaver } from "@langchain/langgraph";

const deleteOldMessages = createMiddleware({
  name: "DeleteOldMessages",
  afterModel: (state) => {
    const messages = state.messages;
    if (messages.length > 2) {
      // remove the earliest two messages
      return {
        messages: messages
          .slice(0, 2)
          .map((m) => new RemoveMessage({ id: m.id! })),
      };
    }
  }
});

```

```

        return;
    },
});

const agent = createAgent({
    model: "gpt-5.5",
    tools: [],
    systemPrompt: "Please be concise and to the point.",
    middleware: [deleteOldMessages],
    checkpointer: new MemorySaver(),
});

const config = { configurable: { thread_id: "1" } };

const streamA = await agent.streamEvents(
    { messages: [{ role: "user", content: "hi! I'm bob" }] },
    { ...config, version: "v3" }
);
for await (const snapshot of streamA.values) {
    const messageDetails = snapshot.messages.map((message) => [
        message.getType(),
        message.content,
    ]);
    console.log(messageDetails);
}

const streamB = await agent.streamEvents(
    { messages: [{ role: "user", content: "write a short poem about cats" }] },
    { ...config, version: "v3" }
);
for await (const snapshot of streamB.values) {
    const messageDetails = snapshot.messages.map((message) => [
        message.getType(),
        message.content,
    ]);
    console.log(messageDetails);
}

const streamC = await agent.streamEvents(
    { messages: [{ role: "user", content: "what's my name?" }] },
    { ...config, version: "v3" }
);
for await (const snapshot of streamC.values) {
    const messageDetails = snapshot.messages.map((message) => [
        message.getType(),
        message.content,
    ]);
    console.log(messageDetails);
}

```

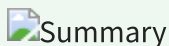
```

[["human", "hi! I'm bob"]]
[["human", "hi! I'm bob"], ["ai", "Hi Bob! Nice to meet you. How can I help you today? I can answer questions, brainstorm ideas, draft text, explain things, or help with code."]]
[["human", "hi! I'm bob"], ["ai", "Hi Bob! Nice to meet you. How can I help you today? I can answer questions, brainstorm ideas, draft text, explain things, or help with code."], ["human", "write a short poem about cats"]]
[["human", "hi! I'm bob"], ["ai", "Hi Bob! Nice to meet you. How can I help you today? I can answer questions, brainstorm ideas, draft text, explain things, or help with code."], ["human", "write a short poem about cats"], ["ai", "There once was a cat on a wall, Who barely moved at all..."]]
[["human", "write a short poem about cats"], ["ai", "There once was a cat on a wall, Who barely moved at all..."]]
[["human", "write a short poem about cats"], ["ai", "There once was a cat on a wall, Who barely moved at all..."], ["human", "what's my name?"]]
[["human", "write a short poem about cats"], ["ai", "There once was a cat on a wall, Who barely moved at all..."], ["human", "what's my name?"], ["ai", "I don't know your name - you haven't told me!"]]
[["human", "what's my name?"], ["ai", "I don't know your name - you haven't told me!"]]

```

Summarize messages

The problem with trimming or removing messages, as shown above, is that you may lose information from culling of the message queue. Because of this, some applications benefit from a more sophisticated approach of summarizing the message history using a chat model.



python

To summarize message history in an agent, use the built-in [SummarizationMiddleware](#):

```

from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.runnables import RunnableConfig

checkpointer = InMemorySaver()

agent = create_agent(
    model="gpt-5.5",
    tools=[...],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5.4-mini",
            trigger=("tokens", 4000),

```



```

        keep=("messages", 20)
    )
],
    checkpointer=checkpointer,
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}
agent.invoke({"messages": "hi, my name is bob"}, config)
agent.invoke({"messages": "write a short poem about cats"}, config)
agent.invoke({"messages": "now do the same but for dogs"}, config)
final_response = agent.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()
"""
===== Ai Message =====

Your name is Bob!
"""

```

See [SummarizationMiddleware](#) for more configuration options.

js

To summarize message history in an agent, use the built-in [summarizationMiddleware](#):

```

import { createAgent, summarizationMiddleware } from "langchain";
import { MemorySaver } from "@langchain/langgraph";

const checkpointer = new MemorySaver();

const agent = createAgent({
  model: "gpt-5.5",
  tools: [],
  middleware: [
    summarizationMiddleware({
      model: "gpt-5.4-mini",
      trigger: { tokens: 4000 },
      keep: { messages: 20 },
    }),
  ],
  checkpointer,
});

const config = { configurable: { thread_id: "1" } };
await agent.invoke({ messages: "hi, my name is bob" }, config);
await agent.invoke({ messages: "write a short poem about cats" }, config);
await agent.invoke({ messages: "now do the same but for dogs" }, config);
const finalResponse = await agent.invoke({ messages: "what's my name?" }, config);

```

```
console.log(finalResponse.messages.at(-1)?.content);  
// Your name is Bob!
```

See [summarizationMiddleware](#) for more configuration options.

Access memory

You can access and modify the short-term memory (state) of an agent in several ways:

Tools

Read short-term memory in a tool

Access short term memory (state) in a tool using the `runtime` parameter (typed as `ToolRuntime`).

The `runtime` parameter is hidden from the tool signature (so the model doesn't see it), but the tool can access the state through it.

python

```
from langchain.agents import create_agent, AgentState  
from langchain.tools import tool, ToolRuntime  
  
class CustomState(AgentState):  
    user_id: str  
  
@tool  
def get_user_info(  
    runtime: ToolRuntime  
) -> str:  
    """Look up user info."""  
    user_id = runtime.state["user_id"]  
    return "User is John Smith" if user_id == "user_123" else "Unknown user"  
  
agent = create_agent(  
    model="gpt-5-nano",  
    tools=[get_user_info],  
    state_schema=CustomState,  
)  
  
result = agent.invoke({  
    "messages": "look up user information",  
    "user_id": "user_123"  
})
```

```
print(result["messages"][-1].content)
# > User is John Smith.
```

js

```
import { createAgent, tool, type ToolRuntime } from "langchain";
import { StateSchema } from "@langchain/langgraph";
import * as z from "zod";

const CustomState = new StateSchema({
  userId: z.string(),
});

const getUserInfo = tool(
  async (_, config: ToolRuntime<typeof CustomState.State>) => {
    const userId = config.state.userId;
    return userId === "user_123" ? "John Doe" : "Unknown User";
  },
  {
    name: "get_user_info",
    description: "Get user info",
    schema: z.object({}),
  }
);

const agent = createAgent({
  model: "gpt-5-nano",
  tools: [getUserInfo],
  stateSchema: CustomState,
});

const result = await agent.invoke(
  {
    messages: [{ role: "user", content: "what's my name?" }],
    userId: "user_123",
  },
  {
    context: {},
  }
);

console.log(result.messages.at(-1)?.content);
// Outputs: "Your name is John Doe."
```

Write short-term memory from tools

To modify the agent's short-term memory (state) during execution, you can return state updates directly from the tools.

This is useful for persisting intermediate results or making information accessible to subsequent tools or prompts.

python

```
from langchain.tools import tool, ToolRuntime
from langchain_core.runnables import RunnableConfig
from langchain.messages import ToolMessage
from langchain.agents import create_agent, AgentState
from langgraph.types import Command
from pydantic import BaseModel

class CustomState(AgentState): # [!code highlight]
    user_name: str

class CustomContext(BaseModel):
    user_id: str

@tool
def update_user_info(
    runtime: ToolRuntime[CustomContext, CustomState],
) -> Command:
    """Look up and update user info."""
    user_id = runtime.context.user_id
    name = "John Smith" if user_id == "user_123" else "Unknown user"
    return Command(update={ # [!code highlight]
        "user_name": name,
        # update the message history
        "messages": [
            ToolMessage(
                "Successfully looked up user information",
                tool_call_id=runtime.tool_call_id
            )
        ]
    })

@tool
def greet(
    runtime: ToolRuntime[CustomContext, CustomState]
) -> str | Command:
    """Use this to greet the user once you found their info."""
    user_name = runtime.state.get("user_name", None)
    if user_name is None:
        return Command(update={
            "messages": [
```

```

        ToolMessage(
            "Please call the 'update_user_info' tool it will get and update
the user's name.",
            tool_call_id=runtime.tool_call_id
        )
    ]
})
return f"Hello {user_name}!"

agent = create_agent(
    model="gpt-5-nano",
    tools=[update_user_info, greet],
    state_schema=CustomState, # [!code highlight]
    context_schema=CustomContext,
)

agent.invoke(
    {"messages": [{"role": "user", "content": "greet the user"}]},
    context=CustomContext(user_id="user_123"),
)

```

js

```

import { tool, createAgent, ToolMessage, type ToolRuntime } from "langchain";
import { Command, StateSchema } from "@langchain/langgraph";
import * as z from "zod";

const CustomState = new StateSchema({
  userId: z.string().optional(),
  userName: z.string().optional(),
});

const updateUserInfo = tool(
  async (_, config: ToolRuntime<typeof CustomState.State>) => {
    const userId = config.state.userId;
    const name = userId === "user_123" ? "John Smith" : "Unknown user";
    return new Command({
      update: {
        userName: name,
        // update the message history
        messages: [
          new ToolMessage({
            content: "Successfully looked up user information",
            tool_call_id: config.toolCall?.id ?? "",
          }),
        ],
      },
    });
  },
);

```

```

    {
      name: "update_user_info",
      description: "Look up and update user info.",
      schema: z.object({}),
    }
  );

  const greet = tool(
    async (_, config: ToolRuntime<typeof CustomState.State>) => {
      const userName = config.state.userName;
      return `Hello ${userName}!`;
    },
    {
      name: "greet",
      description: "Use this to greet the user once you found their info.",
      schema: z.object({}),
    }
  );

  const agent = createAgent({
    model: "openai:gpt-5-mini",
    tools: [updateUserInfo, greet],
    stateSchema: CustomState,
  });

  const result = await agent.invoke({
    messages: [{ role: "user", content: "greet the user" }],
    userId: "user_123",
  });

  console.log(result.messages.at(-1)?.content);
  // Output: "Hello John Smith! It's great to meet you. How can I help you today?"

```

Prompt

Access short term memory (state) in middleware to create dynamic prompts based on conversation history or custom state fields.

python

```

from langchain.agents import create_agent
from typing import TypedDict
from langchain.agents.middleware import dynamic_prompt, ModelRequest

class CustomContext(TypedDict):
    user_name: str

def get_weather(city: str) -> str:

```

```

    """Get the weather in a city."""
    return f"The weather in {city} is always sunny!"

@dynamic_prompt
def dynamic_system_prompt(request: ModelRequest) -> str:
    user_name = request.runtime.context["user_name"]
    system_prompt = f"You are a helpful assistant. Address the user as {user_name}."
    return system_prompt

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_weather],
    middleware=[dynamic_system_prompt],
    context_schema=CustomContext,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What is the weather in SF?"}]},
    context=CustomContext(user_name="John Smith"),
)
for msg in result["messages"]:
    msg.pretty_print()

```

```

```shell title="Output" ===== Human Message
=====

```

```

What is the weather in SF? ===== Ai Message
===== Tool Calls: get_weather
(call_WFQlOGn4b2yoJrv7cih342FG) Call ID: call_WFQlOGn4b2yoJrv7cih342FG Args:
city: San Francisco ===== Tool Message
===== Name: get_weather

```

```

The weather in San Francisco is always sunny! =====
Ai Message =====

```

Hi John Smith, the weather in San Francisco is always sunny!

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript
import * as z from "zod";
import { createAgent, tool, dynamicSystemPromptMiddleware } from "langchain";

```

```

const contextSchema = z.object({
  userName: z.string(),
});
type ContextSchema = z.infer<typeof contextSchema>;

const getWeather = tool(
  async ({ city }) => {
    return `The weather in ${city} is always sunny!`;
  },
  {
    name: "get_weather",
    description: "Get user info",
    schema: z.object({
      city: z.string(),
    }),
  }
);

const agent = createAgent({
  model: "gpt-5-nano",
  tools: [getWeather],
  contextSchema,
  middleware: [
    dynamicSystemPromptMiddleware<ContextSchema>((_, config) => {
      return `You are a helpful assistant. Address the user as ${config.context?.userName}.`;
    }),
  ],
});

const result = await agent.invoke(
  {
    messages: [{ role: "user", content: "What is the weather in SF?" }],
  },
  {
    context: {
      userName: "John Smith",
    },
  }
);

for (const message of result.messages) {
  console.log(message);
}
/**
 * HumanMessage {
 *   "content": "What is the weather in SF?",
 *   // ...
 * }
 * AIMessage {

```



```

* // ...
* "tool_calls": [
*   {
*     "name": "get_weather",
*     "args": {
*       "city": "San Francisco"
*     },
*     "type": "tool_call",
*     "id": "call_tCidbv0apTpQpEWb302zQ4Yx"
*   }
* ],
* // ...
* }
* ToolMessage {
*   "content": "The weather in San Francisco is always sunny!",
*   "tool_call_id": "call_tCidbv0apTpQpEWb302zQ4Yx"
*   // ...
* }
* AIMessage {
*   "content": "John Smith, here's the latest: The weather in San Francisco is alw
ays sunny!\n\nIf you'd like more details (temperature, wind, humidity) or a forecas
t for the next few days, I can pull that up. What would you like?",
*   // ...
* }
*/

```

Before model

python

Access short term memory (state) in `@before_model` middleware to process messages before model calls.

```

%%{
  init: {
    "fontFamily": "monospace",
    "flowchart": {
      "curve": "basis"
    }
  }
}%%
graph TD
  S(["`\\_\\_start\\_\\_`"])
  PRE(before_model)
  MODEL(model)
  TOOLS(tools)
  END(["`\\_\\_end\\_\\_`"])

```

```

S --> PRE
PRE --> MODEL
MODEL --> TOOLS
MODEL --> END
TOOLS --> PRE
classDef blueHighlight fill:#E5F4FF,stroke:#006DDD,color:#030710;
classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68;
class S blueHighlight;
class END blueHighlight;
class PRE,MODEL,TOOLS neutral;

```

python

```

from langchain.messages import RemoveMessage
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import before_model
from langchain_core.runnables import RunnableConfig
from langgraph.runtime import Runtime
from typing import Any

@before_model
def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """Keep only the last few messages to fit context window."""
    messages = state["messages"]

    if len(messages) <= 3:
        return None # No changes needed

    first_msg = messages[0]
    recent_messages = messages[-3:] if len(messages) % 2 == 0 else messages[-4:]
    new_messages = [first_msg] + recent_messages

    return {
        "messages": [
            RemoveMessage(id=REMOVE_ALL_MESSAGES),
            *new_messages
        ]
    }

agent = create_agent(
    "gpt-5-nano",
    tools=[],
    middleware=[trim_messages],
    checkpointer=InMemorySaver()
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

```

```

agent.invoke({"messages": "hi, my name is bob"}, config)
agent.invoke({"messages": "write a short poem about cats"}, config)
agent.invoke({"messages": "now do the same but for dogs"}, config)
final_response = agent.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()
"""
===== Ai Message =====

Your name is Bob. You told me that earlier.
If you'd like me to call you a nickname or use a different name, just say the word.
"""

```

js

```

import { RemoveMessage } from "@langchain/core/messages";
import { createAgent, createMiddleware, trimMessages } from "langchain";
import { MemorySaver } from "@langchain/langgraph";
import { REMOVE_ALL_MESSAGES } from "@langchain/langgraph";

const trimMessageHistory = createMiddleware({
  name: "TrimMessages",
  beforeModel: async (state) => {
    const trimmed = await trimMessages(state.messages, {
      maxTokens: 384,
      strategy: "last",
      startOn: "human",
      endOn: ["human", "tool"],
      tokenCounter: (msgs) => msgs.length,
    });
    return {
      messages: [new RemoveMessage({ id: REMOVE_ALL_MESSAGES }), ...trimmed],
    };
  },
});

const checkpointer = new MemorySaver();
const agent = createAgent({
  model: "gpt-5-nano",
  tools: [],
  middleware: [trimMessageHistory],
  checkpointer,
});

```

After model

python

Access short term memory (state) in @[after_model] middleware to process messages after model calls.

```
%%{
  init: {
    "fontFamily": "monospace",
    "flowchart": {
      "curve": "basis"
    }
  }
}%%
graph TD
  S(["\\_\\_start\\_\\_"])
  MODEL(model)
  POST(after_model)
  TOOLS(tools)
  END(["\\_\\_end\\_\\_"])
  S --> MODEL
  MODEL --> POST
  POST -.-> END
  POST -.-> TOOLS
  TOOLS --> MODEL
  classDef blueHighlight fill:#E5F4FF,stroke:#006DDD,color:#030710;
  classDef greenHighlight fill:#F6FFDB,stroke:#6E8900,color:#2E3900;
  classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68;
  class S blueHighlight;
  class END blueHighlight;
  class POST greenHighlight;
  class MODEL,TOOLS neutral;
```

python

```
from langchain.messages import RemoveMessage
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import after_model
from langgraph.runtime import Runtime

@after_model
def validate_response(state: AgentState, runtime: Runtime) -> dict | None:
    """Remove messages containing sensitive words."""
    STOP_WORDS = ["password", "secret"]
    last_message = state["messages"][-1]
    if any(word in last_message.content for word in STOP_WORDS):
        return {"messages": [RemoveMessage(id=last_message.id)]}
    return None

agent = create_agent(
```

```

    model="gpt-5-nano",
    tools=[],
    middleware=[validate_response],
    checkpointer=InMemorySaver(),
)

```

js

```

import { RemoveMessage } from "@langchain/core/messages";
import { createAgent, createMiddleware } from "langchain";
import { REMOVE_ALL_MESSAGES } from "@langchain/langgraph";

const validateResponse = createMiddleware({
  name: "ValidateResponse",
  afterModel: (state) => {
    const lastMessage = state.messages.at(-1)?.content;
    if (
      typeof lastMessage === "string" &&
      lastMessage.toLowerCase().includes("confidential")
    ) {
      return {
        messages: [
          new RemoveMessage({ id: REMOVE_ALL_MESSAGES }),
        ],
      };
    }
    return;
  },
});

const agent = createAgent({
  model: "gpt-5-nano",
  tools: [],
  middleware: [validateResponse],
});

```

Build a SQL agent

langchain/sql-agent.mdx

Overview

In this tutorial, you will learn how to build an agent that can answer questions about a SQL database using LangChain [agents](#).

At a high level, the agent will:

1. Fetch the available tables and schemas from the database
2. Decide which tables are relevant to the question
3. Fetch the schemas for the relevant tables
4. Generate a query based on the question and information from the schemas
5. Double-check the query for common mistakes using an LLM
6. Execute the query and return the results
7. Correct mistakes surfaced by the database engine until the query is successful
8. Formulate a response based on the results

주의

Building Q&A systems of SQL databases requires executing model-generated SQL queries. There are inherent risks in doing this. Make sure that your database connection permissions are always scoped as narrowly as possible for your agent's needs. This will mitigate, though not eliminate, the risks of building a model-driven system.

Concepts

The following tutorial covers the following concepts:

- [Tools](#) for reading from SQL databases
- LangChain [agents](#)
- [Human-in-the-loop](#) processes

Setup

Install dependencies

python

```
```bash pip install langchain langgraph
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```bash npm
npm i langchain @langchain/core sqlite3 zod
```

```
```bash yarn yarn add langchain @langchain/core sqlite3 zod
```

```
```bash pnpm
pnpm add langchain @langchain/core sqlite3 zod
```

Set up LangSmith

Set up [LangSmith](#) to inspect what is happening inside your chain or agent. Then set the following environment variables:

```
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."
```

Build your SQL agent

Select an LLM

python

Select a model that supports [tool-calling](#):

js

Select a model that supports [tool-calling](#):

The output shown in the examples below used OpenAI.

Configure the database

You will be creating a [SQLite database](#) for this tutorial. SQLite is a lightweight database that is easy to set up and use. We will be loading the `chinook` database, which is a sample database that represents a digital media store.

For convenience, we have hosted the database (`chinook.db`) on a public GCS bucket.

python

js

python

We will use Python's built-in `sqlite3` module to interact with the database:

```
Dialect: sqlite
Available tables: ['Album', 'Artist', 'Customer', 'Employee', 'Genre', 'Invoice',
'InvoiceLine', 'MediaType', 'Playlist', 'PlaylistTrack', 'Track']
Sample output: [(1, 'AC/DC'), (2, 'Accept'), (3, 'Aerosmith'), (4, 'Alanis Morissette'), (5, 'Alice In Chains')]
```

Add tools for database interactions

주의

The following database tools are minimal wrappers for demonstration purposes only. They are not intended to be secure or used in production. Use narrowly scoped database permissions and add application-specific validation before executing model-generated SQL.

python

We can implement database [tools](#) as thin wrappers using the `@tool` decorator from `langchain.tools`:

```
sql_db_query: Input to this tool is a detailed and correct SQL query, output is a result from the database.
```

```
    If the query is not correct, an error message will be returned.
```

```
    If an error is returned, rewrite the query, check the query, and try again.
```

```
    If you encounter an issue with Unknown column 'xxxx' in 'field list', use sql_db_schema to query the correct table fields.
```

```
sql_db_schema: Input to this tool is a comma-separated list of tables, output is the schema and sample rows for those tables.
```

```
    Be sure that the tables actually exist by calling sql_db_list_tables first!
```

```
    Example Input: table1, table2, table3
```

```
sql_db_list_tables: Input is an empty string, output is a comma-separated list of tables in the database.
```

```
sql_db_query_checker: Use this tool to double check if your query is correct before
```


executing it.

Always use this tool before executing a query with `sql_db_query!`

js

We will use the `sqlite3` library to query the database and fetch schemas:

Create the agent

python

Use `@[ create_agent ]` to build a [ReAct agent](#) with minimal code. The agent will interpret the request and generate a SQL command, which the tools will execute. If the command has an error, the error message is returned to the model. The model can then examine the original request and the new error message and generate a new command. This can continue until the LLM generates the command successfully or reaches an end count. This pattern of providing a model with feedback - error messages in this case - is very powerful.

Initialize the agent with a descriptive system prompt to customize its behavior:

Now, create an agent with the model, tools, and prompt:

js

Before running the command, do a check to check the LLM generated command in `_safe_sql`:

Then, execute commands with an `execute_sql` tool:

Use `createAgent` to build a [ReAct agent](#) with minimal code. The agent will interpret the request and generate a SQL command. The tools will check the command for safety and then try to execute the command. If the command has an error, the error message is returned to the model. The model can then examine the original request and the new error message and generate a new command. This can continue until the LLM generates the command successfully or reaches an end count. This pattern of providing a model with feedback - error messages in this case - is very powerful.

Initialize the agent with a descriptive system prompt to customize its behavior:

Now, create an agent with the model, tools, and prompt:

Run the agent

python

Run the agent on a sample query and observe its behavior:

```
===== Human Message =====

Which genre on average has the longest tracks?
===== Ai Message =====

Tool Calls:
  sql_db_list_tables (call_BQswg8P65apHc8BTJ1NPDvnM)
Call ID: call_BQswg8P65apHc8BTJ1NPDvnM
Args:
===== Tool Message =====
Name: sql_db_list_tables

Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLine, MediaType, Playlist, PlaylistTrack, Track
===== Ai Message =====

Tool Calls:
  sql_db_schema (call_i89tjKECFSeERbuACym4w0cU)
Call ID: call_i89tjKECFSeERbuACym4w0cU
Args:
  table_names: Track, Genre
===== Tool Message =====
Name: sql_db_schema

CREATE TABLE "Genre" (
  "GenreId" INTEGER NOT NULL,
  "Name" NVARCHAR(120),
  PRIMARY KEY ("GenreId")
)

/*
3 rows from Genre table:
GenreId Name
1    Rock
2    Jazz
3    Metal
*/

CREATE TABLE "Track" (
  "TrackId" INTEGER NOT NULL,
  "Name" NVARCHAR(200) NOT NULL,
  "AlbumId" INTEGER,
  "MediaTypeId" INTEGER NOT NULL,
  "GenreId" INTEGER,
  "Composer" NVARCHAR(220),
  "Milliseconds" INTEGER NOT NULL,
  "Bytes" INTEGER,
  "UnitPrice" NUMERIC(10, 2) NOT NULL,
```

```

PRIMARY KEY ("TrackId"),
FOREIGN KEY("MediaTypeId") REFERENCES "MediaType" ("MediaTypeId"),
FOREIGN KEY("GenreId") REFERENCES "Genre" ("GenreId"),
FOREIGN KEY("AlbumId") REFERENCES "Album" ("AlbumId")
)

/*
3 rows from Track table:
TrackId Name      AlbumId MediaTypeId GenreId Composer      Milliseconds      Bytes      UnitPrice
1  For Those About To Rock (We Salute You) 1  1  1  Angus Young, Malcolm Young, Brian Johnson  343719  11170334  0.99
2  Balls to the Wall 2  2  1  U. Dirkschneider, W. Hoffmann, H. Frank, P. Baltes, S. Kaufmann, G. Hoffmann  342562  5510424  0.99
3  Fast As a Shark 3  2  1  F. Baltes, S. Kaufman, U. Dirkschneider & W. Hoffman  230619  3990994  0.99
*/

===== Ai Message =====
Tool Calls:
  sql_db_query_checker (call_G64yYm6R6UauivPCXJZMA49b)
Call ID: call_G64yYm6R6UauivPCXJZMA49b
Args:
  query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AverageLength FROM Track INNER JOIN Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AverageLength DESC LIMIT 5;
===== Tool Message =====
Name: sql_db_query_checker

SELECT Genre.Name, AVG(Track.Milliseconds) AS AverageLength FROM Track INNER JOIN Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AverageLength DESC LIMIT 5;
===== Ai Message =====
Tool Calls:
  sql_db_query (call_An03SrhD00DJBxh6dHMwvHwZ)
Call ID: call_An03SrhD00DJBxh6dHMwvHwZ
Args:
  query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AverageLength FROM Track INNER JOIN Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AverageLength DESC LIMIT 5;
===== Tool Message =====
Name: sql_db_query

[('Sci Fi & Fantasy', 2911783.0384615385), ('Science Fiction', 2625549.076923077), ('Drama', 2575283.78125), ('TV Shows', 2145041.0215053763), ('Comedy', 1585263.705882353)]
===== Ai Message =====

On average, the genre with the longest tracks is "Sci Fi & Fantasy" with an average track length of approximately 2,911,783 milliseconds. This is followed by "Science Fiction," "Drama," "TV Shows," and "Comedy."

```

The agent correctly wrote a query, checked the query, and ran it to inform its final response.

참고

You can inspect all aspects of the above run, including steps taken, tools invoked, what prompts were seen by the LLM, and more in the [LangSmith trace](#).

js

Run the agent on a sample query and observe its behavior:

```
human: Which genre, on average, has the longest tracks?  
ai:  
tool: [{"Genre": "Sci Fi & Fantasy", "AvgMilliseconds": 2911783.0384615385}]  
ai: Sci Fi & Fantasy – average track length ≈ 48.5 minutes (about 2,911,783 ms).
```

The agent correctly wrote a query, checked the query, and ran it to inform its final response.

참고

You can inspect all aspects of the above run, including steps taken, tools invoked, what prompts were seen by the LLM, and more in the [LangSmith trace](#).

(Optional) Use Studio

python

[Studio](#) provides a "client side" loop as well as memory so you can run this as a chat interface and query the database. You can ask questions like "Tell me the scheme of the database" or "Show me the invoices for the 5 top customers". You will see the SQL command that is generated and the resulting output. The details of how to get that started are below.

Run your agent in Studio

In addition to the previously mentioned packages, you will need to:

```
pip install -U langgraph-cli[inmem]>=0.4.0
```

In directory you will run in, you will need a `langgraph.json` file with the following contents:

```
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./sql_agent.py:agent",
    "graph": "./sql_agent_langgraph.py:graph"
  },
  "env": ".env"
}
```

Create a file `sql_agent.py` and insert this:

js

[Studio](#) provides a "client side" loop as well as memory so you can run this as a chat interface and query the database. You can ask questions like "Tell me the scheme of the database" or "Show me the invoices for the 5 top customers". You will see the SQL command that is generated and the resulting output. The details of how to get that started are below.

Run your agent in Studio

In addition to the previously mentioned packages, you will need to:

```
npm i -g @langchain/langgraph-cli@latest
```

In directory you will run in, you will need a `langgraph.json` file with the following contents:

```
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./sqlAgent.ts:agent",
    "graph": "./sqlAgentLanggraph.ts:graph"
  },
  "env": ".env"
}
```

Create a file `sqlAgent.ts` and insert this:

Implement human-in-the-loop review

python

It can be prudent to check the agent's SQL queries before they are executed for any unintended actions or inefficiencies.

LangChain agents feature support for built-in [human-in-the-loop middleware](#) to add oversight to agent tool calls. Let's configure the agent to pause for human review on calling the `sql_db_query` tool:

참고

We've added a [checkpoint](#) to our agent to allow execution to be paused and resumed. See the [human-in-the-loop guide](#) for details on this as well as available middleware configurations.

On running the agent, it will now pause for review before executing the `sql_db_query` tool:

```
...

INTERRUPTED:
Tool execution pending approval

Tool: sql_db_query
Args: {'query': 'SELECT g.Name AS Genre, AVG(t.Milliseconds) AS AvgTrackLength FROM
Track t JOIN Genre g ON t.GenreId = g.GenreId GROUP BY g.Name ORDER BY AvgTrackLength
DESC LIMIT 1;'}

```

We can resume execution, in this case accepting the query, using [Command](#):

```
===== Ai Message =====
Tool Calls:
  sql_db_query (call_7oz86Epg7lYRqi9rQHbZPS1U)
Call ID: call_7oz86Epg7lYRqi9rQHbZPS1U
Args:
  query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AvgDuration FROM Track JOIN
Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AvgDuration D
ESC LIMIT 5;
===== Tool Message =====
Name: sql_db_query

[('Sci Fi & Fantasy', 2911783.0384615385), ('Science Fiction', 2625549.076923077),
('Drama', 2575283.78125), ('TV Shows', 2145041.0215053763), ('Comedy', 1585263.7058
82353)]
===== Ai Message =====

The genre with the longest average track length is "Sci Fi & Fantasy" with an avera
ge duration of about 2,911,783 milliseconds, followed by "Science Fiction" and "Dra
ma."
```

Refer to the [human-in-the-loop guide](#) for details.

js

It can be prudent to check the agent's SQL queries before they are executed for any unintended actions or inefficiencies.

LangChain agents feature support for built-in [human-in-the-loop middleware](#) to add oversight to agent tool calls. Let's configure the agent to pause for human review on calling the `execute_sql` tool:

참고

We've added a [checkpoint](#) to our agent to allow execution to be paused and resumed. See the [human-in-the-loop guide](#) for details on this as well as available middleware configurations.

On running the agent, it will now pause for review before executing the `execute_sql` tool:

```
...

INTERRUPTED:
Tool execution pending approval

Tool: execute_sql
Args: {'query': 'SELECT g.Name AS Genre, AVG(t.Milliseconds) AS AvgTrackLength FROM
Track t JOIN Genre g ON t.GenreId = g.GenreId GROUP BY g.Name ORDER BY AvgTrackLeng
th DESC LIMIT 1;'}

```

We can resume execution, in this case accepting the query, using [Command](#):

```
===== Ai Message =====
Tool Calls:
  execute_sql (call_7oz86Epg7lYRqi9rQHbZPS1U)
  Call ID: call_7oz86Epg7lYRqi9rQHbZPS1U
  Args:
    query: SELECT Genre.Name, AVG(Track.Milliseconds) AS AvgDuration FROM Track JOI
N Genre ON Track.GenreId = Genre.GenreId GROUP BY Genre.Name ORDER BY AvgDuration D
ESC LIMIT 5;
===== Tool Message =====
Name: execute_sql

[('Sci Fi & Fantasy', 2911783.0384615385), ('Science Fiction', 2625549.076923077),
('Drama', 2575283.78125), ('TV Shows', 2145041.0215053763), ('Comedy', 1585263.7058
82353)]
===== Ai Message =====

The genre with the longest average track length is "Sci Fi & Fantasy" with an avera

```

ge duration of about 2,911,783 milliseconds, followed by "Science Fiction" and "Drama."

Refer to the [human-in-the-loop guide](#) for details.

Next steps

For deeper customization, check out [this tutorial](#) for implementing a SQL agent directly using LangGraph primitives.

Streaming

langchain/streaming.mdx

팁

For new applications, we recommend [event streaming](#)—the typed-projection API introduced in LangChain v1.3. Event streaming gives you separate iterators per projection (messages, values, tool calls, subgraphs) so you can consume them independently instead of branching on `stream_mode` chunks.

LangChain implements a streaming system to surface real-time updates.

Streaming is crucial for enhancing the responsiveness of applications built on LLMs. By displaying output progressively, even before a complete response is ready, streaming significantly improves user experience (UX), particularly when dealing with the latency of LLMs.

Overview

LangChain's streaming system lets you surface live feedback from agent runs to your application.

What's possible with LangChain streaming:

-

[Stream agent progress](#)—get state updates after each agent step. *

[Stream LLM tokens](#)—stream language model tokens as they're generated. *

[Stream thinking / reasoning tokens](#)—surface model reasoning as it's generated. *

[Stream custom updates](#)—emit user-defined signals (e.g., "Fetched 10/100 records"). *

[Stream multiple modes](#)—choose from `updates` (agent progress), `messages` (LLM tokens + metadata), or `custom` (arbitrary user data).

See the [common patterns](#) section below for additional end-to-end examples.

Supported stream modes

python

Pass one or more of the following stream modes as a list to the `@[ stream ]` `[CompiledStateGraph.stream]` or `@[ astream ]` `[CompiledStateGraph.astream]` methods:

js

Pass one or more of the following stream modes as a list to the `@[ stream ]` `[CompiledStateGraph.stream]` method:

Mode	Description
updates	Streams state updates after each agent step. If multiple updates are made in the same step (e.g., multiple nodes are run), those updates are streamed separately.
messages	Streams tuples of <code>(token, metadata)</code> from any graph nodes where an LLM is invoked.
custom	Streams custom data from inside your graph nodes using the stream writer.

Agent progress

python

To stream agent progress, use the `@[ stream ]` `[CompiledStateGraph.stream]` or `@[ astream ]` `[CompiledStateGraph.astream]` methods with `stream_mode="updates"`. This emits an event after every agent step.

js

To stream agent progress, use the `@[ stream ]` `[CompiledStateGraph.stream]` method with `streamMode: "updates"`. This emits an event after every agent step.

For example, if you have an agent that calls a tool once, you should see the following updates:

- **LLM node:** `@[ AIMessage ]` with tool call requests
- **Tool node:** `@[ ToolMessage ]` with execution result
- **LLM node:** Final AI response

python

Pass a `thread_id` via `config` so the conversation is checkpointed and follow-up turns can resume the same history. `thread_id` is independent of `stream_mode`; you can also pass `context` alongside it for per-run data your tools read from `runtime.context`.

```
`` `shell title="Output" step: model content: [{ 'type': 'tool_call', 'name': 'get_weather',  
'args': { 'city': 'San Francisco'}, 'id': 'call_9lBtsDbmmobzyA8xc4l4Ctne' }] step: tools  
content: [{ 'type': 'text', 'text': "It's always sunny in San Francisco!" }] step: model  
content: [{ 'type': 'text', 'text': "San Francisco weather: It's always sunny in San  
Francisco!\n\nIf you'd like the exact current conditions (temperature, humidity, wind)  
and a short forecast, I can fetch that next. Would you like me to pull live details for San  
Francisco?" }]
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

```
Persisting conversation history with `thread_id` requires the agent to be configure  
d with a [checkpointer](/oss/langchain/long-term-memory). On [LangSmith deployment  
s](/langsmith/deployment) a checkpointer is provisioned automatically. Locally, pas  
s one explicitly, for example `create_agent(..., checkpointer=InMemorySaver())`. Th  
e remaining snippets on this page omit `thread_id` for brevity, but you should pass  
it in production.
```

```
</div>
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
Pass a `thread_id` via `configurable` so the conversation is checkpointed and follo  
w-up turns can resume the same history. `thread_id` is independent of `streamMode`;  
you can also pass `context` alongside it for per-run data your tools read from `run  
time.context`.
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

```
Persisting conversation history with `thread_id` requires the agent to be configure  
d with a [checkpointer](/oss/langchain/long-term-memory). On [LangSmith deployment  
s](/langsmith/deployment) a checkpointer is provisioned automatically. Locally, pas  
s one explicitly, for example `createAgent({ ..., checkpointer: new MemorySaver()  
})`. The remaining snippets on this page omit `thread_id` for brevity, but you shou  
ld pass it in production.
```

```
</div>
```

```
</div>
```

```
## LLM tokens
```

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

To stream tokens as they are produced by the LLM, use `stream\_mode="messages"`. Below you can see the output of the agent streaming tool calls and the final response.

```
```python title="Streaming LLM tokens"
from langchain.agents import create_agent

def get_weather(city: str) -> str:
 """Get weather for a given city."""

 return f"It's always sunny in {city}!"

agent = create_agent(
 model="gpt-5-nano",
 tools=[get_weather],
)
for chunk in agent.stream(# [!code highlight]
 {"messages": [{"role": "user", "content": "What is the weather in SF?"}]},
 stream_mode="messages",
 version="v2", # [!code highlight]
):
 if chunk["type"] == "messages": # [!code highlight]
 token, metadata = chunk["data"] # [!code highlight]
 print(f"node: {metadata['langgraph_node']}")
 print(f"content: {token.content_blocks}")
 print("\n")
```

```
```shell title="Output"
expandable node: model content: [{'type': 'tool_call_chunk',
'id': 'call_vbCyBcP8VuneUzyYISBZZsVa', 'name': 'get_weather', 'args': '', 'index': 0}]
```

```
node: model content: [{'type': 'tool_call_chunk', 'id': None, 'name': None, 'args': '{}',
'index': 0}]
```

```
node: model content: [{'type': 'tool_call_chunk', 'id': None, 'name': None, 'args': 'city',
'index': 0}]
```

```
node: model content: [{'type': 'tool_call_chunk', 'id': None, 'name': None, 'args': '""',
'index': 0}]
```

```
node: model content: [{'type': 'tool_call_chunk', 'id': None, 'name': None, 'args': 'San',
'index': 0}]
```

```
node: model content: [{'type': 'tool_call_chunk', 'id': None, 'name': None, 'args': '
Francisco', 'index': 0}]
```

node: model content: [{'type': 'tool_call_chunk', 'id': None, 'name': None, 'args': '{}', 'index': 0}]

node: model content: []

node: tools content: [{'type': 'text', 'text': "It's always sunny in San Francisco!"}]

node: model content: []

node: model content: [{'type': 'text', 'text': 'Here'}]

node: model content: [{'type': 'text', 'text': 's'}]

node: model content: [{'type': 'text', 'text': ' what'}]

node: model content: [{'type': 'text', 'text': ' I'}]

node: model content: [{'type': 'text', 'text': ' got'}]

node: model content: [{'type': 'text', 'text': ':'}]

node: model content: [{'type': 'text', 'text': ' '}]

node: model content: [{'type': 'text', 'text': "It's"}]

node: model content: [{'type': 'text', 'text': ' always'}]

node: model content: [{'type': 'text', 'text': ' sunny'}]

node: model content: [{'type': 'text', 'text': ' in'}]

node: model content: [{'type': 'text', 'text': ' San'}]

node: model content: [{'type': 'text', 'text': ' Francisco'}]

node: model content: [{'type': 'text', 'text': '!"\n\n'}]

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

To stream tokens as they are produced by the LLM, use `streamMode: "messages"`:

```
```typescript
import z from "zod";
import { createAgent, tool } from "langchain";

const getWeather = tool(
 async ({ city }) => {
```

```

 return `The weather in ${city} is always sunny!`;
 },
 {
 name: "get_weather",
 description: "Get weather for a given city.",
 schema: z.object({
 city: z.string(),
 }),
 }
});

const agent = createAgent({
 model: "gpt-5.4-mini",
 tools: [getWeather],
});

for await (const [token, metadata] of await agent.stream(
 { messages: [{ role: "user", content: "what is the weather in sf" }] },
 { streamMode: "messages" }
)) {
 console.log(`node: ${metadata.langgraph_node}`);
 console.log(`content: ${JSON.stringify(token.contentBlocks, null, 2)}`);
}

```

## python

### 참고

**Wrapping an agent as a node in a parent `StateGraph`?** `@[ create_agent ]` returns a compiled graph, so using it as a node makes it a subgraph. `stream_mode="messages"` on the parent graph will not emit token chunks from the inner agent's LLM calls unless you pass `subgraphs=True`. See [Subgraph outputs](#).

## js

### 참고

**Wrapping an agent as a node in a parent `StateGraph`?** `@[ createAgent ]` returns a `ReactAgent` wrapper; pass `agent.graph` when adding it as a node. Use `subgraphs: true` so message chunks include the subgraph namespace. See [Subgraph outputs](#).

## Custom updates

## python

To stream updates from tools as they are executed, you can use `@[get_stream_writer]`.

```
```python title="Streaming custom updates" from langchain.agents import
create_agent from langgraph.config import get_stream_writer # [!code highlight]

def get_weather(city: str) -> str: """Get weather for a given city.""" writer =
get_stream_writer() # [!code highlight] # stream any arbitrary data writer(f"Looking up
data for city: {city}") writer(f"Acquired data for city: {city}") return f"It's always sunny in
{city}!"

agent = create_agent( model="claude-sonnet-4-6", tools=[get_weather], )

for chunk in agent.stream( {"messages": [{"role": "user", "content": "What is the
weather in SF?"}]}, stream_mode="custom", # [!code highlight] version="v2", # [!code
highlight] ): if chunk["type"] == "custom": # [!code highlight] print(chunk["data"]) #
[!code highlight]
```

```
```shell title="Output"
Looking up data for city: San Francisco
Acquired data for city: San Francisco
```

## 참고

If you add `@[get_stream_writer]` inside your tool, you won't be able to invoke the tool outside of a LangGraph execution context.

## js

To stream updates from tools as they are executed, you can use the `writer` parameter from the configuration.

```
import z from "zod";
import { tool, createAgent } from "langchain";
import { LangGraphRunnableConfig } from "@langchain/langgraph";

const getWeather = tool(
 async (input, config: LangGraphRunnableConfig) => {
 // Stream any arbitrary data
 config.writer?.(`Looking up data for city: ${input.city}`);
 // ... fetch city data
 config.writer?.(`Acquired data for city: ${input.city}`);
 return `It's always sunny in ${input.city}!`;
 },
 {
 name: "get_weather",
 description: "Get weather for a given city.",
 }
);
```

```

 schema: z.object({
 city: z.string().describe("The city to get weather for."),
 }),
 },
);

const agent = createAgent({
 model: "gpt-5.4-mini",
 tools: [getWeather],
});

for await (const chunk of await agent.stream(
 { messages: [{ role: "user", content: "what is the weather in sf" }] },
 { streamMode: "custom" }
)) {
 console.log(chunk);
}

```

```shell title="Output" Looking up data for city: San Francisco Acquired data for city: San Francisco

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

If you add the `writer` parameter to your tool, you won't be able to invoke the tool outside of a LangGraph execution context without providing a writer function.

```
</div>
```

```
</div>
```

Stream multiple modes

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

You can specify multiple streaming modes by passing stream mode as a list: `stream\_mode=["updates", "custom"]`.

Each streamed chunk is a `StreamPart` dict with `type`, `ns`, and `data` keys. Use `chunk["type"]` to determine the stream mode and `chunk["data"]` to access the payload.

```

```python title="Streaming multiple modes"
from langchain.agents import create_agent
from langgraph.config import get_stream_writer

```



```
def get_weather(city: str) -> str:
 """Get weather for a given city."""
 writer = get_stream_writer()
 writer(f"Looking up data for city: {city}")
 writer(f"Acquired data for city: {city}")
 return f"It's always sunny in {city}!"

agent = create_agent(
 model="gpt-5-nano",
 tools=[get_weather],
)

for chunk in agent.stream(# [!code highlight]
 {"messages": [{"role": "user", "content": "What is the weather in SF?"}]},
 stream_mode=["updates", "custom"],
 version="v2", # [!code highlight]
):
 print(f"stream_mode: {chunk['type']}") # [!code highlight]
 print(f"content: {chunk['data']}") # [!code highlight]
 print("\n")
```

```
```shell title="Output" stream_mode: updates content: {'model': {'messages':
[AIMessage(content='', response_metadata={'token_usage': {'completion_tokens': 280,
'prompt_tokens': 132, 'total_tokens': 412, 'completion_tokens_details':
{'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 256,
'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0,
'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-nano-2025-08-
07', 'system_fingerprint': None, 'id': 'chatcmpl-C9tlgBzGEbedGYxZ0rTCz5F7OXpL7',
'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run-
480c07cb-e405-4411-aa7f-0520fddeed66-0', tool_calls=[{'name': 'get_weather', 'args':
{'city': 'San Francisco'}, 'id': 'call_KTNQIfMrl9vgNwEfAJMVu7r', 'type': 'tool_call'}],
usage_metadata={'input_tokens': 132, 'output_tokens': 280, 'total_tokens': 412,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0,
'reasoning': 256}}]}}}
```

stream_mode: custom content: Looking up data for city: San Francisco

stream_mode: custom content: Acquired data for city: San Francisco

stream_mode: updates content: {'tools': {'messages': [ToolMessage(content="It's
always sunny in San Francisco!", name='get_weather',
tool_call_id='call_KTNQIfMrl9vgNwEfAJMVu7r')]]}}

stream_mode: updates content: {'model': {'messages': [AIMessage(content='San
Francisco weather: It's always sunny in San Francisco!\n\n', response_metadata=
{'token_usage': {'completion_tokens': 764, 'prompt_tokens': 168, 'total_tokens': 932,
'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0,
'reasoning_tokens': 704, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details':

```
{'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name':  
'gpt-5-nano-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-  
C9tljDFVki1e1haCyikBptAuXuHYG', 'service_tier': 'default', 'finish_reason': 'stop',  
'logprobs': None}, id='lc_run--acbc740a-18fe-4a14-8619-da92a0d0ee90-0',  
usage_metadata={'input_tokens': 168, 'output_tokens': 764, 'total_tokens': 932,  
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0,  
'reasoning': 704}}}}}]}
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

You can specify multiple streaming modes by passing `streamMode` as an array: ``streamMode: ["updates", "messages", "custom"]``.

The streamed outputs will be tuples of ``[mode, chunk]`` where ``mode`` is the name of the stream mode and ``chunk`` is the data streamed by that mode.

```
```typescript
```

```
import z from "zod";
import { tool, createAgent } from "langchain";
import { LangGraphRunnableConfig } from "@langchain/langgraph";

const getWeather = tool(
 async (input, config: LangGraphRunnableConfig) => {
 // Stream any arbitrary data
 config.writer?.(`Looking up data for city: ${input.city}`);
 // ... fetch city data
 config.writer?.(`Acquired data for city: ${input.city}`);
 return `It's always sunny in ${input.city}!`;
 },
 {
 name: "get_weather",
 description: "Get weather for a given city.",
 schema: z.object({
 city: z.string().describe("The city to get weather for."),
 }),
 }
);

const agent = createAgent({
 model: "gpt-5.4-mini",
 tools: [getWeather],
});

for await (const [streamMode, chunk] of await agent.stream(

```

```
{ messages: [{ role: "user", content: "what is the weather in sf" }] },
{ streamMode: ["updates", "messages", "custom"] }
}) {
 console.log(`${streamMode}: ${JSON.stringify(chunk, null, 2)}`);
}
```

## Common patterns

Below are examples showing common use cases for streaming.

### Streaming thinking / reasoning tokens

Some models perform internal reasoning before producing a final answer. You can stream these thinking / reasoning tokens as they're generated by filtering [standard content blocks](#) for the type "reasoning".

#### 참고

Reasoning output must be enabled on the model.

See the [reasoning section](#) and your [provider's integration page](#) for configuration details.

To quickly check a model's reasoning support, see [models.dev](#).

#### python

To stream thinking tokens from an agent, use `stream_mode="messages"` and filter for reasoning content blocks:

```
```shell title="Output" [thinking] The user is asking about the weather in San
Francisco. I have a tool [thinking] available to get this information. Let me call the
get_weather tool [thinking] with "San Francisco" as the city parameter. The weather in
San Francisco is: It's always sunny in San Francisco!
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

To stream thinking tokens from an agent, use `streamMode: "messages"` and filter for reasoning content blocks. Use a model instance (e.g. `ChatAnthropic`) with extended thinking enabled when the model supports it:

```
```shell title="Output"
[thinking] The user is asking about the weather in San Francisco. I have a tool
[thinking] available to get this information. Let me call the get_weather tool
[thinking] with "San Francisco" as the city parameter.
The weather in San Francisco is: It's always sunny in San Francisco!
```

This works the same way regardless of the model provider—LangChain normalizes provider-specific formats (Anthropic thinking blocks, OpenAI reasoning summaries, etc.) into a standard "reasoning" content block type via the [content\\_blocks](#) property.

To stream reasoning tokens directly from a chat model (without an agent), see [streaming with chat models](#).

## python

### Streaming tool calls

You may want to stream both:

1. Partial JSON as [tool calls](#) are generated
2. The completed, parsed tool calls that are executed

Specifying `stream_mode="messages"` will stream incremental [message chunks](#) generated by all LLM calls in the agent. To access the completed messages with parsed tool calls:

1. If those messages are tracked in the [state](#) (as in the model node of [create\\_agent](#)), use `stream_mode=["messages", "updates"]` to access completed messages through [state updates](#) (demonstrated below).
2. If those messages are not tracked in the state, use [custom updates](#) or aggregate the chunks during the streaming loop ([next section](#)).

## 참고

Refer to the section below on [streaming from sub-agents](#) if your agent includes multiple LLMs.

```
from typing import Any

from langchain.agents import create_agent
from langchain.messages import AIMessage, AIMessageChunk, AnyMessage, ToolMessage

def get_weather(city: str) -> str:
 """Get weather for a given city."""

 return f"It's always sunny in {city}!"
```

```

agent = create_agent("openai:gpt-5.5", tools=[get_weather])

def _render_message_chunk(token: AIMessageChunk) -> None:
 if token.text:
 print(token.text, end="|")
 if token.tool_call_chunks:
 print(token.tool_call_chunks)
 # N.B. all content is available through token.content_blocks

def _render_completed_message(message: AnyMessage) -> None:
 if isinstance(message, AIMessage) and message.tool_calls:
 print(f"Tool calls: {message.tool_calls}")
 if isinstance(message, ToolMessage):
 print(f"Tool response: {message.content_blocks}")

input_message = {"role": "user", "content": "What is the weather in Boston?"}
for chunk in agent.stream(
 {"messages": [input_message]},
 stream_mode=["messages", "updates"], # [!code highlight]
 version="v2", # [!code highlight]
):
 if chunk["type"] == "messages": # [!code highlight]
 token, metadata = chunk["data"] # [!code highlight]
 if isinstance(token, AIMessageChunk):
 _render_message_chunk(token) # [!code highlight]
 elif chunk["type"] == "updates": # [!code highlight]
 for source, update in chunk["data"].items(): # [!code highlight]
 if source in ("model", "tools"): # `source` captures node name
 _render_completed_message(update["messages"][-1]) # [!code highlight]
ht]

```

```

```shell title="Output" expandable [{ 'name': 'get_weather', 'args': '', 'id':
'call_D3Orjr89KgsLTZ9hTzYv7HpF', 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': None,
'args': '{}', 'id': None, 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args': 'city',
'id': None, 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args': 'Boston', 'id': None, 'index':
0, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args': 'Boston', 'id': None, 'index':
0, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args': 'Boston', 'id': None, 'index':
0, 'type': 'tool_call_chunk' }] Tool calls: [{ 'name': 'get_weather', 'args': { 'city': 'Boston' }, 'id':
'call_D3Orjr89KgsLTZ9hTzYv7HpF', 'type': 'tool_call' }] Tool response: [{ 'type': 'text',
'text': "It's always sunny in Boston!" }] The| weather| in| Boston| is| sun|ny||

```

Accessing completed messages

<div class="callout note" markdown="1">

<p class="callout-label">참고</p>

If completed messages are tracked in an agent's [state](/oss/langchain/short-term-m

emory), you can use `stream\_mode=["messages", "updates"]` as demonstrated in the [Streaming tool calls](#streaming-tool-calls) section to access completed messages during streaming.

</div>

In some cases, completed messages are not reflected in [state updates](#agent-progress). If you have access to the agent internals, you can use [custom updates](#custom-updates) to access these messages during streaming. Otherwise, you can aggregate message chunks in the streaming loop (see below).

Consider the below example, where we incorporate a [stream writer](#custom-updates) into a simplified [guardrail middleware](/oss/langchain/guardrails#after-agent-guardrails). This middleware demonstrates tool calling to generate a structured "safe / unsafe" evaluation (one could also use [structured outputs](/oss/langchain/models#structured-output) for this):

```
```python
from typing import Any, Literal

from langchain.agents.middleware import after_agent, AgentState
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from langgraph.config import get_stream_writer # [!code highlight]
from pydantic import BaseModel

class ResponseSafety(BaseModel):
 """Evaluate a response as safe or unsafe."""
 evaluation: Literal["safe", "unsafe"]

safety_model = init_chat_model("openai:gpt-5.5")

@after_agent(can_jump_to=["end"])
def safety_guardrail(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 """Model-based guardrail: Use an LLM to evaluate response safety."""
 stream_writer = get_stream_writer() # [!code highlight]
 # Get the model response
 if not state["messages"]:
 return None

 last_message = state["messages"][-1]
 if not isinstance(last_message, AIMessage):
 return None

 # Use another model to evaluate safety
 model_with_tools = safety_model.bind_tools([ResponseSafety], tool_choice="any")
 result = model_with_tools.invoke(
 [
 {
 "role": "system",
 "content": "Evaluate this AI response as generally safe or unsafe."
 }
]
)
```
```

```

        },
        {
            "role": "user",
            "content": f"AI response: {last_message.text}"
        }
    ]
)
stream_writer(result) # [!code highlight]

tool_call = result.tool_calls[0]
if tool_call["args"]["evaluation"] == "unsafe":
    last_message.content = "I cannot provide that response. Please rephrase your request."

return None

```

We can then incorporate this middleware into our agent and include its custom stream events:

```

from typing import Any

from langchain.agents import create_agent
from langchain.messages import AIMessageChunk, AIMessage, AnyMessage

def get_weather(city: str) -> str:
    """Get weather for a given city."""

    return f"It's always sunny in {city}!"

agent = create_agent(
    model="openai:gpt-5.5",
    tools=[get_weather],
    middleware=[safety_guardrail], # [!code highlight]
)

def _render_message_chunk(token: AIMessageChunk) -> None:
    if token.text:
        print(token.text, end="|")
    if token.tool_call_chunks:
        print(token.tool_call_chunks)

def _render_completed_message(message: AnyMessage) -> None:
    if isinstance(message, AIMessage) and message.tool_calls:
        print(f"Tool calls: {message.tool_calls}")
    if isinstance(message, ToolMessage):
        print(f"Tool response: {message.content_blocks}")

input_message = {"role": "user", "content": "What is the weather in Boston?"}
for chunk in agent.stream(
    {"messages": [input_message]},

```

```

stream_mode=["messages", "updates", "custom"], # [!code highlight]
version="v2", # [!code highlight]
):
    if chunk["type"] == "messages": # [!code highlight]
        token, metadata = chunk["data"] # [!code highlight]
        if isinstance(token, AIMessageChunk):
            _render_message_chunk(token)
    elif chunk["type"] == "updates": # [!code highlight]
        for source, update in chunk["data"].items(): # [!code highlight]
            if source in ("model", "tools"):
                _render_completed_message(update["messages"][-1])
    elif chunk["type"] == "custom": # [!code highlight]
        # access completed message in stream
        print(f"Tool calls: {chunk['data'].tool_calls}") # [!code highlight]

```

```

```shell title="Output" expandable [{"name": 'get_weather', 'args': '', 'id':
'call_je6LWgxYzuZ84mmoDalTYMJJC', 'index': 0, 'type': 'tool_call_chunk'}] [{"name":
None, 'args': '{}', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{"name": None, 'args':
'city', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{"name": None, 'args': '""', 'id':
None, 'index': 0, 'type': 'tool_call_chunk'}] [{"name": None, 'args': 'Boston', 'id': None,
'index': 0, 'type': 'tool_call_chunk'}] [{"name": None, 'args': ''}, 'id': None, 'index': 0,
'type': 'tool_call_chunk'}] Tool calls: [{"name": 'get_weather', 'args': {'city': 'Boston'},
'id': 'call_je6LWgxYzuZ84mmoDalTYMJJC', 'type': 'tool_call'}] Tool response: [{'type':
'text', 'text': 'It's always sunny in Boston!'}] The| weather| in| Boston| is| sun|ny||.|
[{"name": 'ResponseSafety', 'args': '', 'id': 'call_O8VJIbOG4Q9nQF0T8ltVi58O', 'index': 0,
'type': 'tool_call_chunk'}] [{"name": None, 'args': '{}', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] [{"name": None, 'args': 'evaluation', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] [{"name": None, 'args': '""', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] [{"name": None, 'args': 'safe', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] [{"name": None, 'args': ''}, 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] Tool calls: [{"name": 'ResponseSafety', 'args': {'evaluation': 'safe'},
'id': 'call_O8VJIbOG4Q9nQF0T8ltVi58O', 'type': 'tool_call'}]

```

Alternatively, if you aren't able to add custom events to the stream, you can aggregate message chunks within the streaming loop:

```

```python
input_message = {"role": "user", "content": "What is the weather in Boston?"}
full_message = None # [!code highlight]
for chunk in agent.stream(
    {"messages": [input_message]},
    stream_mode=["messages", "updates"],
    version="v2", # [!code highlight]
):
    if chunk["type"] == "messages": # [!code highlight]
        token, metadata = chunk["data"] # [!code highlight]
        if isinstance(token, AIMessageChunk):

```



```

        _render_message_chunk(token)
        full_message = token if full_message is None else full_message + token
# [!code highlight]
        if token.chunk_position == "last": # [!code highlight]
            if full_message.tool_calls: # [!code highlight]
                print(f"Tool calls: {full_message.tool_calls}") # [!code highlight]
            full_message = None # [!code highlight]
        elif chunk["type"] == "updates": # [!code highlight]
            for source, update in chunk["data"].items(): # [!code highlight]
                if source == "tools":
                    _render_completed_message(update["messages"][-1])

```

Streaming with human-in-the-loop

To handle human-in-the-loop [interrupts](#), we build on the [above example](#):

1. We configure the agent with [human-in-the-loop middleware and a checkpointer](#)
2. We collect interrupts generated during the "updates" stream mode
3. We respond to those interrupts with a [command](#)

```

from typing import Any

from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langchain.messages import AIMessage, AIMessageChunk, AnyMessage, ToolMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command, Interrupt

def get_weather(city: str) -> str:
    """Get weather for a given city."""

    return f"It's always sunny in {city}!"

checkpointer = InMemorySaver()

agent = create_agent(
    "openai:gpt-5.5",
    tools=[get_weather],
    middleware=[ # [!code highlight]
        HumanInTheLoopMiddleware(interrupt_on={"get_weather": True}), # [!code highlight]
    ], # [!code highlight]
    checkpointer=checkpointer, # [!code highlight]
)

def _render_message_chunk(token: AIMessageChunk) -> None:
    if token.text:
        print(token.text, end="|")

```

```

    if token.tool_call_chunks:
        print(token.tool_call_chunks)

def _render_completed_message(message: AnyMessage) -> None:
    if isinstance(message, AIMessage) and message.tool_calls:
        print(f"Tool calls: {message.tool_calls}")
    if isinstance(message, ToolMessage):
        print(f"Tool response: {message.content_blocks}")

def _render_interrupt(interrupt: Interrupt) -> None: # [!code highlight]
    interrupts = interrupt.value # [!code highlight]
    for request in interrupts["action_requests"]: # [!code highlight]
        print(request["description"]) # [!code highlight]

input_message = {
    "role": "user",
    "content": (
        "Can you look up the weather in Boston and San Francisco?"
    ),
}

config = {"configurable": {"thread_id": "some_id"}} # [!code highlight]
interrupts = [] # [!code highlight]
for chunk in agent.stream(
    {"messages": [input_message]},
    config=config, # [!code highlight]
    stream_mode=["messages", "updates"],
    version="v2", # [!code highlight]
):
    if chunk["type"] == "messages": # [!code highlight]
        token, metadata = chunk["data"] # [!code highlight]
        if isinstance(token, AIMessageChunk):
            _render_message_chunk(token)
    elif chunk["type"] == "updates": # [!code highlight]
        for source, update in chunk["data"].items(): # [!code highlight]
            if source in ("model", "tools"):
                _render_completed_message(update["messages"][-1])
            if source == "__interrupt__": # [!code highlight]
                interrupts.extend(update) # [!code highlight]
                _render_interrupt(update[0]) # [!code highlight]

```

```

```shell title="Output" expandable [{ 'name': 'get_weather', 'args': '', 'id':
'call_GOWNaQHeqMixay2qy80padfE', 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name':
None, 'args': '{"ci', 'id': None, 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': None,
'args': 'ty": ', 'id': None, 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args':
'"Bosto', 'id': None, 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args': 'n"',
'id': None, 'index': 0, 'type': 'tool_call_chunk' }] [{ 'name': 'get_weather', 'args': '', 'id':
'call_Ndb4jvWm2uMA0JDQXu37wDH6', 'index': 1, 'type': 'tool_call_chunk' }] [{ 'name':
None, 'args': '{"ci', 'id': None, 'index': 1, 'type': 'tool_call_chunk' }] [{ 'name': None,
'args': 'ty": ', 'id': None, 'index': 1, 'type': 'tool_call_chunk' }] [{ 'name': None, 'args':

```

```
""San F', 'id': None, 'index': 1, 'type': 'tool_call_chunk']] [{'name': None, 'args': 'ranc',
'id': None, 'index': 1, 'type': 'tool_call_chunk']] [{'name': None, 'args': 'isco'", 'id': None,
'index': 1, 'type': 'tool_call_chunk']] [{'name': None, 'args': '}', 'id': None, 'index': 1,
'type': 'tool_call_chunk']] Tool calls: [{'name': 'get_weather', 'args': {'city': 'Boston'},
'id': 'call_GOWNaQHeqMixay2qy80padfE', 'type': 'tool_call'}, {'name': 'get_weather',
'args': {'city': 'San Francisco'}, 'id': 'call_Ndb4jvWm2uMA0JDQXu37wDH6', 'type':
'tool_call'}] Tool execution requires approval
```

Tool: get\_weather Args: {'city': 'Boston'} Tool execution requires approval

Tool: get\_weather Args: {'city': 'San Francisco'}

We next collect a [decision](/oss/langchain/human-in-the-loop#interrupt-decision-types) for each interrupt. Importantly, the order of decisions must match the order of actions we collected.

To illustrate, we will edit one tool call and accept the other:

```
```python
def _get_interrupt_decisions(interrupt: Interrupt) -> list[dict]:
    return [
        {
            "type": "edit",
            "edited_action": {
                "name": "get_weather",
                "args": {"city": "Boston, U.K."},
            },
        }
        if "boston" in request["description"].lower()
        else {"type": "approve"}
        for request in interrupt.value["action_requests"]
    ]

decisions = {}
for interrupt in interrupts:
    decisions[interrupt.id] = {
        "decisions": _get_interrupt_decisions(interrupt)
    }

decisions
```

```
```shell title="Output" { 'a96c40474e429d661b5b32a8d86f0f3e': { 'decisions': [ { 'type':
'edit', 'edited_action': { 'name': 'get_weather', 'args': {'city': 'Boston, U.K.'} } }, { 'type':
'approve'},] }
```

We can then resume by passing a [command](/oss/langchain/human-in-the-loop#responding-to-interrupts) into the same streaming loop:

```
```python
```

```

interrupts = []
for chunk in agent.stream(
    Command(resume=decisions), # [!code highlight]
    config=config,
    stream_mode=["messages", "updates"],
    version="v2", # [!code highlight]
):
    # Streaming loop is unchanged
    if chunk["type"] == "messages": # [!code highlight]
        token, metadata = chunk["data"] # [!code highlight]
        if isinstance(token, AIMessageChunk):
            _render_message_chunk(token)
    elif chunk["type"] == "updates": # [!code highlight]
        for source, update in chunk["data"].items(): # [!code highlight]
            if source in ("model", "tools"):
                _render_completed_message(update["messages"][-1])
            if source == "__interrupt__":
                interrupts.extend(update)
                _render_interrupt(update[0])

```

``shell title="Output" Tool response: [{'type': 'text', 'text': "It's always sunny in Boston, U.K.!"}] Tool response: [{'type': 'text', 'text': "It's always sunny in San Francisco!"}] -| **Boston**||: It's| always| sunny| in| Boston|,| U.K.| -| **San Francisco**||: It's| always| sunny| in| San| Francisco|!

Streaming from sub-agents

When there are multiple LLMs at any point in an agent, it's often necessary to disambiguate the source of messages as they are generated.

To do this, pass a `[`name`](https://reference.langchain.com/python/langchain/agents/create_agent)` to each agent when creating it. This name is then available in metadata via the ``lc_agent_name`` key when streaming in ``"messages"`` mode.

Below, we update the `[streaming tool calls](#streaming-tool-calls)` example:

1. We replace our tool with a ``call_weather_agent`` tool that invokes an agent internally
2. We add a ``name`` to each agent
3. We specify `[`subgraphs=True`](/oss/langgraph/use-subgraphs#stream-subgraph-outputs)` when creating the stream
4. Our stream processing is identical to before, but we add logic to keep track of what agent is active using ``create_agent``'s ``name`` parameter

<div class="callout tip" markdown="1">

<p class="callout-label">팁</p>

When you set a ``name`` on an agent, that name is also attached to any ``AIMessage``'s `g`

enerated by that agent.

</div>

First we construct the agent:

```
```python
from typing import Any

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage, AnyMessage

def get_weather(city: str) -> str:
 """Get weather for a given city."""

 return f"It's always sunny in {city}!"

weather_model = init_chat_model("openai:gpt-5.5")
weather_agent = create_agent(
 model=weather_model,
 tools=[get_weather],
 name="weather_agent", # [!code highlight]
)

def call_weather_agent(query: str) -> str:
 """Query the weather agent."""
 result = weather_agent.invoke({
 "messages": [{"role": "user", "content": query}]
 })
 return result["messages"][-1].text

supervisor_model = init_chat_model("openai:gpt-5.5")
agent = create_agent(
 model=supervisor_model,
 tools=[call_weather_agent],
 name="supervisor", # [!code highlight]
)
```

Next, we add logic to the streaming loop to report which agent is emitting tokens:

```
def _render_message_chunk(token: AIMessageChunk) -> None:
 if token.text:
 print(token.text, end="|")
 if token.tool_call_chunks:
 print(token.tool_call_chunks)

def _render_completed_message(message: AnyMessage) -> None:
 if isinstance(message, AIMessage) and message.tool_calls:
 print(f"Tool calls: {message.tool_calls}")
 if isinstance(message, ToolMessage):
```

```

print(f"Tool response: {message.content_blocks}")

input_message = {"role": "user", "content": "What is the weather in Boston?"}
current_agent = None # [!code highlight]
for chunk in agent.stream(
 {"messages": [input_message]},
 stream_mode=["messages", "updates"],
 subgraphs=True, # [!code highlight]
 version="v2", # [!code highlight]
):
 if chunk["type"] == "messages": # [!code highlight]
 token, metadata = chunk["data"] # [!code highlight]
 if agent_name := metadata.get("lc_agent_name"): # [!code highlight]
 if agent_name != current_agent: # [!code highlight]
 print(f"🤖 {agent_name}: ") # [!code highlight]
 current_agent = agent_name # [!code highlight]
 if isinstance(token, AIMessageChunk):
 _render_message_chunk(token)
 elif chunk["type"] == "updates": # [!code highlight]
 for source, update in chunk["data"].items(): # [!code highlight]
 if source in ("model", "tools"):
 _render_completed_message(update["messages"][-1])

```

```

`` `shell title="Output" expandable 🤖 supervisor: [{'name': 'call_weather_agent',
'args': '', 'id': 'call_asorzUf0mB6sb7MiKfgojp7I', 'index': 0, 'type': 'tool_call_chunk'}]
[{'name': None, 'args': '', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name':
None, 'args': 'query', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None,
'args': 'Boston', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args':
'weather', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': 'right',
'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': 'now', 'id': None,
'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': 'and', 'id': None, 'index': 0,
'type': 'tool_call_chunk'}] [{'name': None, 'args': "today's", 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] [{'name': None, 'args': 'forecast', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] [{'name': None, 'args': '', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] Tool calls: [{'name': 'call_weather_agent', 'args': {'query': "Boston
weather right now and today's forecast"}, 'id': 'call_asorzUf0mB6sb7MiKfgojp7I', 'type':
'tool_call'}] 🤖 weather_agent: [{'name': 'get_weather', 'args': '', 'id':
'call_LZ89lT8fW6w8vqck5pZeaDlx', 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None,
'args': '', 'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': 'city',
'id': None, 'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': '', 'id': None,
'index': 0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': 'Boston', 'id': None, 'index':
0, 'type': 'tool_call_chunk'}] [{'name': None, 'args': '', 'id': None, 'index': 0, 'type':
'tool_call_chunk'}] Tool calls: [{'name': 'get_weather', 'args': {'city': 'Boston'}, 'id':
'call_LZ89lT8fW6w8vqck5pZeaDlx', 'type': 'tool_call'}] Tool response: [{'type': 'text',
'text': "It's always sunny in Boston!"}] Boston| weather| right| now| | Sunny||

```

|Today|'s| forecast| for| Boston|:| |**Sunny| all| day**||.|Tool response: [{ 'type': 'text', 'text': 'Boston weather right now: **Sunny**.\n\nToday's forecast for Boston: **Sunny all day**.'}]  
🗣️ supervisor: Boston| weather| right| now|:| |**Sunny**||.

|Today|'s| forecast| for| Boston|:| |**Sunny| all| day**||.|

</div>

### ## Disable streaming

In some applications you might need to disable streaming of individual tokens for a given model. This is useful when:

- Working with [multi-agent](/oss/langchain/multi-agent) systems to control which agents stream their output
- Mixing models that support streaming with those that do not
- Deploying to [LangSmith](/langsmith/observability) and wanting to prevent certain model outputs from being streamed to the client

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

Set `streaming=False` when initializing the model.

```
```python
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.5",
    streaming=False # [!code highlight]
)
```

js

Set `streaming: false` when initializing the model.

```
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
  model: "gpt-5.5",
  streaming: false, // [!code highlight]
});
```

팁

When deploying to LangSmith, set `streaming=False` on any models whose output you don't want streamed to the client. This is configured in your graph code before deployment.

python

참고

Not all chat model integrations support the `streaming` parameter. If your model doesn't support it, use `disable_streaming=True` instead. This parameter is available on all chat models via the base class.

js

참고

Not all chat model integrations support the `streaming` parameter. If your model doesn't support it, use `disableStreaming: true` instead. This parameter is available on all chat models via the base class.

See the [LangGraph streaming guide](#) for more details.

python

v2 streaming format

참고

Requires LangGraph 1.1 or later.

Pass `version="v2"` to `stream()` or `astream()` to get a unified output format. Every chunk is a `StreamPart` dict with `type`, `ns`, and `data` keys — the same shape regardless of stream mode or number of modes:

```
```python v2 (new)
```

## Unified format — no more tuple unpacking

---

```
for chunk in agent.stream({"messages": [{"role": "user", "content": "What is the weather in SF?"}]}, stream_mode=["updates", "custom"], version="v2",):
```



```
print(chunk["type"]) # "updates" or "custom" print(chunk["data"]) # payload
```

```
```python v1 (current default)
# Must unpack (mode, data) tuples
for mode, chunk in agent.stream(
    {"messages": [{"role": "user", "content": "What is the weather in SF?"}]},
    stream_mode=["updates", "custom"],
):
    print(mode)    # "updates" or "custom"
    print(chunk)  # payload
```

The v2 format also improves `invoke()` — it returns a `GraphOutput` object with `.value` and `.interrupts` attributes, cleanly separating state from interrupt metadata:

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Hello"}]},
    version="v2",
)
print(result.value)      # state (dict, Pydantic model, or dataclass)
print(result.interrupts) # tuple of Interrupt objects (empty if none)
```

See the [LangGraph streaming docs](#) for more details on the v2 format, including type narrowing, Pydantic/dataclass coercion, and subgraph streaming.

Related

- [Frontend streaming](#)—Build React UIs with `@[useStream]` for real-time agent interactions
- [Streaming with chat models](#)—Stream tokens directly from a chat model without using an agent or graph
- [Reasoning with chat models](#)—Configure and access reasoning output from chat models
- [Standard content blocks](#)—Understand the normalized content block format used for reasoning, text, and other content types
- [Streaming with human-in-the-loop](#)—Stream agent progress while handling interrupts for human review
- [LangGraph streaming](#)—Advanced streaming options including `values`, `debug` modes, and subgraph streaming

Structured output

langchain/structured-output.mdx

python

Structured output allows agents to return data in a specific, predictable format. Instead of parsing natural language responses, you get structured data in the form of JSON objects, [Pydantic models](#), or dataclasses that your application can use directly.

팁

This page covers structured output with agents using `create_agent`. To use structured output directly on a model (outside of agents), see [Models - Structured output](#).

LangChain's `@[ create_agent ]` handles structured output automatically. The user sets their desired structured output schema, and when the model generates the structured data, it's captured, validated, and returned in the `'structured_response'` key of the agent's state.

```
def create_agent(
    ...
    response_format: Union[
        ToolStrategy[StructuredResponseT],
        ProviderStrategy[StructuredResponseT],
        type[StructuredResponseT],
        None,
    ]
)
```

Response format

Use `response_format` to control how the agent returns structured data:

- `ToolStrategy[StructuredResponseT]` : Uses tool calling for structured output
- `ProviderStrategy[StructuredResponseT]` : Uses provider-native structured output
- `type[StructuredResponseT]` : Schema type - automatically selects best strategy based on model capabilities
- `None` : Structured output not explicitly requested

When a schema type is provided directly, LangChain automatically chooses:

- `ProviderStrategy` if the model and provider chosen supports native structured output (e.g. [OpenAI](#), [Anthropic \(Claude\)](#), or [xAI \(Grok\)](#)).
- `ToolStrategy` for all other models.

주의

JSON Schema dictionaries must be wrapped in an explicit strategy (`ProviderStrategy` or `ToolStrategy`). They are not automatically detected when passed directly to `response_format`.

참고

Support for native structured output features is read dynamically from the model's [profile data](#) if using `langchain>=1.1`. If data are not available, use another condition or specify manually:

```
custom_profile = {
    "structured_output": True,
    # ...
}
model = init_chat_model("...", profile=custom_profile)
```

If tools are specified, the model must support simultaneous use of tools and structured output.

The structured response is returned in the `structured_response` key of the agent's final state.

js

Structured output allows agents to return data in a specific, predictable format. Instead of parsing natural language responses, you get typed structured data.

팁

This page covers structured output with agents using `createAgent`. To use structured output directly on a model (outside of agents), see [Models - Structured output](#).

LangChain's prebuilt ReAct agent `createAgent` handles structured output automatically. The user sets their desired structured output schema, and when the model generates the structured data, it's captured, validated, and returned in the `structuredResponse` key of the agent's state.

```

type ResponseFormat = (
  | ZodSchema<StructuredResponseT> // a Zod schema
  | StandardSchema<StructuredResponseT> // any Standard Schema library
  | Record<string, unknown> // a JSON Schema
)

const agent = createAgent({
  // ...
  responseFormat: ResponseFormat | ResponseFormat[]
})

```

Response format

Controls how the agent returns structured data. You can provide a Zod schema, any [Standard Schema](#)-compatible schema, or a JSON Schema object. By default, the agent uses a tool calling strategy, in which the output is created by an additional tool call. Certain models support native structured output, in which case the agent will use that strategy instead.

You can control the behavior by wrapping `ResponseFormat` in a `toolStrategy` or `providerStrategy` function call:

```

import { toolStrategy, providerStrategy } from "langchain";

const agent = createAgent({
  // use a provider strategy if supported by the model
  responseFormat: providerStrategy(z.object({ ... })),
  // or enforce a tool strategy
  responseFormat: toolStrategy(z.object({ ... })),
})

```

The structured response is returned in the `structuredResponse` key of the agent's final state.

팁

Support for native structured output features is read dynamically from the model's [profile data](#) if using `langchain>=1.1`. If data are not available, use another condition or specify manually:

```

const customProfile: ModelProfile = {
  structuredOutput: true,
  // ...
}

const model = await initChatModel("...", { profile: customProfile });

```

If tools are specified, the model must support simultaneous use of tools and structured output.

Provider strategy

Some model providers support structured output natively through their APIs (e.g. OpenAI, xAI (Grok), Gemini, Anthropic (Claude)). This is the most reliable method when available.

To use this strategy, configure a `ProviderStrategy`:

python

```
class ProviderStrategy(Generic[SchemaT]):
    schema: type[SchemaT]
    strict: bool | None = None
```

정보

The `strict` param requires `langchain>=1.2`.

The schema defining the structured output format. Supports: - **Pydantic models:**

`BaseModel` subclasses with field validation. Returns validated Pydantic instance. -

Dataclasses: Python dataclasses with type annotations. Returns dict. - **TypedDict:**

Typed dictionary classes. Returns dict. - **JSON Schema:** Dictionary with JSON schema specification. Must include top-level `title` and `description` keys. Returns dict.

Optional boolean parameter to enable strict schema adherence. Supported by some providers (e.g., `[OpenAI](/oss/integrations/chat/openai)` and `[xAI](/oss/integrations/chat/xai)`). Defaults to ``None`` (disabled).

LangChain automatically uses `ProviderStrategy` when you pass a schema type directly to `@[create_agent.response_format][create_agent(response_format)]` and the model supports native structured output:

```
```python
from pydantic import BaseModel, Field
from langchain.agents import create_agent
```

```
class ContactInfo(BaseModel):
 """Contact information for a person."""
 name: str = Field(description="The name of the person")
 email: str = Field(description="The email address of the person")
 phone: str = Field(description="The phone number of the person")
```

```
agent = create_agent(
 model="gpt-5.5",
 response_format=ContactInfo, # Auto-selects ProviderStrategy
)
```

```
result = agent.invoke({ "messages": [{"role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"}] })

print(result["structured_response"])
```

## ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')

---

```
```python
from dataclasses import dataclass
from langchain.agents import create_agent

@dataclass
class ContactInfo:
    """Contact information for a person."""
    name: str # The name of the person
    email: str # The email address of the person
    phone: str # The phone number of the person

agent = create_agent(
    model="gpt-5.5",
    tools=[],
    response_format=ContactInfo # Auto-selects ProviderStrategy
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"}]
})

result["structured_response"]
# {'name': 'John Doe', 'email': 'john@example.com', 'phone': '(555) 123-4567'}
```

```
```python
from typing_extensions import TypedDict
from langchain.agents import create_agent
```

```
class ContactInfo(TypedDict):
 """Contact information for a person."""
 name: str # The name of the person
 email: str # The email address of the person
 phone: str # The phone number of the person
```

```
agent = create_agent(model="gpt-5.5", tools=[], response_format=ContactInfo # Auto-selects ProviderStrategy)
```

```
result = agent.invoke({ "messages": [{"role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"}] })
```

```
result["structured_response"]
```

**`{'name': 'John Doe', 'email': 'john@example.com', 'phone': '(555) 123-4567'}`**

---

```
```python JSON Schema
from langchain.agents import create_agent
from langchain.agents.structured_output import ProviderStrategy

contact_info_schema = {
    "title": "ContactInfo",
    "type": "object",
    "description": "Contact information for a person.",
    "properties": {
        "name": {"type": "string", "description": "The name of the person"},
        "email": {"type": "string", "description": "The email address of the person"},
        "phone": {"type": "string", "description": "The phone number of the person"}
    },
    "required": ["name", "email", "phone"]
}

agent = create_agent(
    model="gpt-5.5",
    tools=[],
    response_format=ProviderStrategy(contact_info_schema)
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"}]
})

result["structured_response"]
# {'name': 'John Doe', 'email': 'john@example.com', 'phone': '(555) 123-4567'}
```

js

```
function providerStrategy<StructuredResponseT>(
  schema: ZodSchema<StructuredResponseT> | SerializableSchema | JsonSchemaFormat
): ProviderStrategy<StructuredResponseT>
```

The schema defining the structured output format. Supports: - **Zod Schema**: A zod schema - **Standard Schema**: Any schema implementing the [Standard Schema](#) spec - **JSON Schema**: A JSON schema object

LangChain automatically uses `ProviderStrategy` when you pass a schema type directly to `createAgent.responseFormat` and the model supports native structured output:

```
```ts
import * as z from "zod";
import { createAgent, providerStrategy } from "langchain";
```

```
const ContactInfo = z.object({
 name: z.string().describe("The name of the person"),
 email: z.string().describe("The email address of the person"),
 phone: z.string().describe("The phone number of the person"),
});
```

```
const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 responseFormat: providerStrategy(ContactInfo)
});
```

```
const result = await agent.invoke({
 messages: [{ role: "user", content: "Extract contact info from: John Doe, john@example.com, (555) 123-4567" }]
});
```

```
console.log(result.structuredResponse); // { name: "John Doe", email: "john@example.com", phone: "(555) 123-4567" }
```

```
```ts
import * as v from "valibot";
import { toStandardJsonSchema } from "@valibot/to-json-schema";
import { createAgent, providerStrategy } from "langchain";

const ContactInfo = toStandardJsonSchema(
  v.object({
    name: v.pipe(v.string(), v.description("The name of the person")),
    email: v.pipe(v.string(), v.description("The email address of the person")),
    phone: v.pipe(v.string(), v.description("The phone number of the person")),
  })
);

const agent = createAgent({
  model: "gpt-5.5",
  tools: [],
  responseFormat: providerStrategy(ContactInfo)
});

const result = await agent.invoke({
```



```

    messages: [{ "role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567" } ]
  });

  console.log(result.structuredResponse);
  // { name: "John Doe", email: "john@example.com", phone: "(555) 123-4567" }

```

```

` ``ts JSON Schema import { createAgent, providerStrategy } from "langchain";

```

```

const contactInfoSchema = { "type": "object", "description": "Contact information for a person.", "properties": { "name": { "type": "string", "description": "The name of the person"}, "email": { "type": "string", "description": "The email address of the person"}, "phone": { "type": "string", "description": "The phone number of the person" } }, "required": ["name", "email", "phone"] }

```

```

const agent = createAgent({ model: "gpt-5.5", tools: [], responseFormat: providerStrategy(contactInfoSchema) });

```

```

const result = await agent.invoke({ messages: [{ "role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567" } ] });

```

```

console.log(result.structuredResponse); // { name: "John Doe", email: "john@example.com", phone: "(555) 123-4567" }

```

```

</div>

```

Provider-native structured output provides high reliability and strict validation because the model provider enforces the schema. Use it when available.

```

<div class="callout python" markdown="1">

```

```

<p class="callout-label">python</p>

```

```

<div class="callout note" markdown="1">

```

```

<p class="callout-label">참고</p>

```

If the provider natively supports structured output for your model choice, it is functionally equivalent to write `response_format=ProductReview` instead of `response_format=ProviderStrategy(ProductReview)`.

In either case, if structured output is not supported, the agent will fall back to a tool calling strategy.

```

</div>

```

```

</div>

```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
<div class="callout note" markdown="1">
```

```
<p class="callout-label">참고</p>
```

If the provider natively supports structured output for your model choice, it is functionally equivalent to write `responseFormat: contactInfoSchema` instead of `responseFormat: providerStrategy(contactInfoSchema)`.

In either case, if structured output is not supported, the agent will fall back to a tool calling strategy.

```
</div>
```

```
</div>
```

Tool calling strategy

For models that don't support native structured output, LangChain uses tool calling to achieve the same result. This works with all models that support tool calling (most modern models).

To use this strategy, configure a `ToolStrategy`:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```python
class ToolStrategy(Generic[SchemaT]):
 schema: type[SchemaT]
 tool_message_content: str | None
 handle_errors: Union[
 bool,
 str,
 type[Exception],
 tuple[type[Exception], ...],
 Callable[[Exception], str],
]
```

The schema defining the structured output format. Supports:

- **Pydantic models:**

`BaseModel` subclasses with field validation. Returns validated Pydantic instance.

- **Dataclasses:** Python dataclasses with type annotations. Returns dict.

- **TypedDict:** Typed dictionary classes. Returns dict.
- **JSON Schema:** Dictionary with JSON schema specification. Must include top-level `title` and `description` keys. Returns dict.
- **Union**

**types:** Multiple schema options. The model will choose the most appropriate schema based on the context.

Custom content for the tool message returned when structured output is generated. If not provided, defaults to a message showing the structured response data.

Error handling strategy for structured output validation failures. Defaults to `True`.

- `True`: Catch all errors with default error template
- `str`: Catch all errors with this custom message
- `type[Exception]`: Only catch this exception type with default message
- `tuple[type[Exception], ...]`: Only catch these exception types with default message
- `Callable[[Exception], str]`: Custom function that returns error message
- `False`: No retry, let exceptions propagate

```
```python
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.strstructured_output import ToolStrategy
```

```
class ProductReview(BaseModel):
    """Analysis of a product review."""
    rating: int | None = Field(
        description="The rating of the product",
        ge=1,
        le=5
    )
    sentiment: Literal["positive", "negative"] = Field(
        description="The sentiment of the review"
    )
    key_points: list[str] = Field(
        description="The key points of the review. Lowercase, 1-3 words each."
    )
```

```
agent = create_agent(
    model="gpt-5.5",
    tools=[],
    response_format=ToolStrategy(ProductReview)
)
```

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'"
            }
        ]
    }
)
result["structured_response"]
```

ProductReview(rating=5, sentiment='positive', key_points=['fast shipping', 'expensive'])

```
```python
from dataclasses import dataclass
from typing import Literal
from langchain.agents import create_agent
```

```

from langchain.agents.structured_output import ToolStrategy

@dataclass
class ProductReview:
 """Analysis of a product review."""
 rating: int | None # The rating of the product (1-5)
 sentiment: Literal["positive", "negative"] # The sentiment of the review
 key_points: list[str] # The key points of the review

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 response_format=ToolStrategy(ProductReview)
)

result = agent.invoke({
 "messages": [{"role": "user", "content": "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'"}]
})
result["structured_response"]
{'rating': 5, 'sentiment': 'positive', 'key_points': ['fast shipping', 'expensive']}

```

```

```python
from typing import TypedDict, Literal
from typing_extensions import TypedDict
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ProductReview(TypedDict): """Analysis of a product review."""
    rating: int | None # The rating of the product (1-5)
    sentiment: Literal["positive", "negative"] # The sentiment of the review
    key_points: list[str] # The key points of the review

agent = create_agent( model="gpt-5.5", tools=[],
    response_format=ToolStrategy(ProductReview) )

result = agent.invoke({ "messages": [{"role": "user", "content": "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'"}] })
result["structured_response"]

```

**{'rating': 5, 'sentiment': 'positive',
'key_points': ['fast shipping', 'expensive']}**

```

```python
JSON Schema
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

```

```

product_review_schema = {
 "title": "ProductReview",
 "type": "object",
 "description": "Analysis of a product review.",
 "properties": {
 "rating": {
 "type": ["integer", "null"],
 "description": "The rating of the product (1-5)",
 "minimum": 1,
 "maximum": 5
 },
 "sentiment": {
 "type": "string",
 "enum": ["positive", "negative"],
 "description": "The sentiment of the review"
 },
 "key_points": {
 "type": "array",
 "items": {"type": "string"},
 "description": "The key points of the review"
 }
 },
 "required": ["sentiment", "key_points"]
}

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 response_format=ToolStrategy(product_review_schema)
)

result = agent.invoke({
 "messages": [{"role": "user", "content": "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'"}]
})
result["structured_response"]
{'rating': 5, 'sentiment': 'positive', 'key_points': ['fast shipping', 'expensive']}

```

```

```python
from pydantic import BaseModel, Field
from typing import Literal, Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

```

```

class ProductReview(BaseModel):
    """Analysis of a product review."""
    rating: int | None = Field(
        description="The rating of the product",
        ge=1,
        le=5
    )
    sentiment: Literal["positive", "negative"] = Field(
        description="The sentiment of the review"
    )
    key_points: list[str] = Field(
        description="The key points of the review. Lowercase, 1-3 words each."
    )

```

```
class CustomerComplaint(BaseModel): """A customer complaint about a product or
service.""" issue_type: Literal["product", "service", "shipping", "billing"] =
Field(description="The type of issue") severity: Literal["low", "medium", "high"] =
Field(description="The severity of the complaint") description: str =
Field(description="Brief description of the complaint")
```

```
agent = create_agent( model="gpt-5.5", tools=[],
response_format=ToolStrategy(Union[ProductReview, CustomerComplaint]) )
```

```
result = agent.invoke({ "messages": [{"role": "user", "content": "Analyze this review:
'Great product: 5 out of 5 stars. Fast shipping, but expensive'"}] })
result["structured_response"]
```

ProductReview(rating=5, sentiment='positive', key_points=['fast shipping', 'expensive'])

Custom tool message content

The `tool_message_content` parameter allows you to customize the message that appears in the conversation history when structured output is generated:

```
```python
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class MeetingAction(BaseModel):
 """Action items extracted from a meeting transcript."""
 task: str = Field(description="The specific task to be completed")
 assignee: str = Field(description="Person responsible for the task")
 priority: Literal["low", "medium", "high"] = Field(description="Priority level")

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 response_format=ToolStrategy(
 schema=MeetingAction,
 tool_message_content="Action item captured and added to meeting notes!"
)
)
```

```
agent.invoke({
 "messages": [{ "role": "user", "content": "From our meeting: Sarah needs to update the project timeline as soon as possible" }]
})
```

```
===== Human Message =====

From our meeting: Sarah needs to update the project timeline as soon as possible
===== Ai Message =====

Tool Calls:
 MeetingAction (call_1)
 Call ID: call_1
 Args:
 task: Update the project timeline
 assignee: Sarah
 priority: high
===== Tool Message =====
Name: MeetingAction

Action item captured and added to meeting notes!
```

Without `tool_message_content`, our final `@[ ToolMessage ]` would be:

```
===== Tool Message =====
Name: MeetingAction

Returning structured response: { 'task': 'update the project timeline', 'assignee': 'Sarah', 'priority': 'high' }
```

js

```
function toolStrategy<StructuredResponseT>(
 responseFormat:
 | JsonSchemaFormat
 | ZodSchema<StructuredResponseT>
 | SerializableSchema
 | (ZodSchema<StructuredResponseT> | SerializableSchema | JsonSchemaFormat)
 []
 options?: ToolStrategyOptions
): ToolStrategy<StructuredResponseT>
```

The schema defining the structured output format. Supports: - **Zod Schema**: A zod schema - **Standard Schema**: Any schema implementing the [Standard Schema](#) spec - **JSON Schema**: A JSON schema object

Custom content for the tool message returned when structured output is generated. If not provided, defaults to a message showing the structured response data.

Options parameter containing an optional `handleError` parameter for customizing the error handling strategy.

- `true`: Catch all errors with default error template (default)
- `False`: No retry, let exceptions propagate
- `(error: ToolStrategyError) => string | Promise<string>`: retry with the provided message or throw the error

```
```ts
import * as z from "zod";
import { createAgent, toolStrategy } from "langchain";
```

```
const ProductReview = z.object({
  rating: z.number().min(1).max(5).optional(),
  sentiment: z.enum(["positive", "negative"]),
  keyPoints: z.array(z.string()).describe("The key points of the review. Lowercase, 1-3 words each."),
});
```

```
const agent = createAgent({
  model: "gpt-5.5",
  tools: [],
  responseFormat: toolStrategy(ProductReview)
});
```

```
const result = await agent.invoke({
  messages: [{ role: "user", content: "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'" }]
});
```

```
console.log(result.structuredResponse); // { rating: 5, sentiment: "positive",
  keyPoints: ["fast shipping", "expensive"] }
```

```
```ts
import * as v from "valibot";
import { toStandardJsonSchema } from "@valibot/to-json-schema";
import { createAgent, toolStrategy } from "langchain";

const ProductReview = toStandardJsonSchema(
 v.object({
 rating: v.optional(v.pipe(v.number(), v.minValue(1), v.maxValue(5))),
 sentiment: v.picklist(["positive", "negative"]),
 keyPoints: v.pipe(v.array(v.string()), v.description("The key points of the review. Lowercase, 1-3 words each.")),
 })
);

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 responseFormat: toolStrategy(ProductReview)
});

const result = await agent.invoke({
 messages: [{ role: "user", content: "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'" }]
});
```



```

}))

console.log(result.structuredResponse);
// { "rating": 5, "sentiment": "positive", "keyPoints": ["fast shipping", "expensive"] }

```

```

```ts
import { createAgent, toolStrategy } from "langchain";

const productReviewSchema = { "type": "object", "description": "Analysis of a product review.", "properties": { "rating": { "type": ["integer", "null"], "description": "The rating of the product (1-5)", "minimum": 1, "maximum": 5 }, "sentiment": { "type": "string", "enum": ["positive", "negative"], "description": "The sentiment of the review" }, "key_points": { "type": "array", "items": { "type": "string" }, "description": "The key points of the review" } }, "required": ["sentiment", "key_points"] }

const agent = createAgent({ model: "gpt-5.5", tools: [], responseFormat: toolStrategy(productReviewSchema) });

const result = await agent.invoke({ messages: [{"role": "user", "content": "Analyze this review: 'Great product: 5 out of 5 stars. Fast shipping, but expensive'"}] })

console.log(result.structuredResponse); // { "rating": 5, "sentiment": "positive", "keyPoints": ["fast shipping", "expensive"] }

```

```

```ts
// Union Types
import * as z from "zod";
import { createAgent, toolStrategy } from "langchain";

const ProductReview = z.object({
 rating: z.number().min(1).max(5).optional(),
 sentiment: z.enum(["positive", "negative"]),
 keyPoints: z.array(z.string()).describe("The key points of the review. Lowercase, 1-3 words each."),
});

const CustomerComplaint = z.object({
 issueType: z.enum(["product", "service", "shipping", "billing"]),
 severity: z.enum(["low", "medium", "high"]),
 description: z.string().describe("Brief description of the complaint"),
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 responseFormat: toolStrategy([ProductReview, CustomerComplaint])
});

const result = await agent.invoke({

```

```

 messages: [{"role": "user", "content": "Analyze this review: 'Great product: 5
out of 5 stars. Fast shipping, but expensive'"}]
 })

 console.log(result.structuredResponse);
 // { "rating": 5, "sentiment": "positive", "keyPoints": ["fast shipping", "expensiv
e"] }

```

## Custom tool message content

The `toolMessageContent` parameter allows you to customize the message that appears in the conversation history when structured output is generated:

```

import * as z from "zod";
import { createAgent, toolStrategy } from "langchain";

const MeetingAction = z.object({
 task: z.string().describe("The specific task to be completed"),
 assignee: z.string().describe("Person responsible for the task"),
 priority: z.enum(["low", "medium", "high"]).describe("Priority level"),
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 responseFormat: toolStrategy(MeetingAction, {
 toolMessageContent: "Action item captured and added to meeting notes!"
 })
});

const result = await agent.invoke({
 messages: [{"role": "user", "content": "From our meeting: Sarah needs to update
the project timeline as soon as possible"}]
});

console.log(result);
/**
 * {
 * messages: [
 * { role: "user", content: "From our meeting: Sarah needs to update the projec
t timeline as soon as possible" },
 * { role: "assistant", content: "Action item captured and added to meeting not
es!", tool_calls: [{ name: "MeetingAction", args: { task: "update the project time
line", assignee: "Sarah", priority: "high" }, id: "call_456" }] },
 * { role: "tool", content: "Action item captured and added to meeting notes!",
tool_call_id: "call_456", name: "MeetingAction" }
 *],
 * structuredResponse: { task: "update the project timeline", assignee: "Sarah",
priority: "high" }

```

```
* }
*/
```

Without `toolMessageContent`, we'd see:

```
console.log(result);
/**
 * {
 * messages: [
 * ...
 * { role: "tool", content: "Returning structured response: {'task': 'update the project timeline', 'assignee': 'Sarah', 'priority': 'high'}", tool_call_id: "call_456", name: "MeetingAction" }
 *],
 * structuredResponse: { task: "update the project timeline", assignee: "Sarah", priority: "high" }
 * }
*/
```

## Error handling

Models can make mistakes when generating structured output via tool calling. LangChain provides intelligent retry mechanisms to handle these errors automatically.

### Multiple structured outputs error

When a model incorrectly calls multiple structured output tools, the agent provides error feedback in a `@[ ToolMessage ]` and prompts the model to retry:

#### python

```
from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ContactInfo(BaseModel):
 name: str = Field(description="Person's name")
 email: str = Field(description="Email address")

class EventDetails(BaseModel):
 event_name: str = Field(description="Name of the event")
 date: str = Field(description="Event date")

agent = create_agent(
 model="gpt-5.5",
 tools=[],
```

```

 response_format=ToolStrategy(Union[ContactInfo, EventDetails]) # Default: handle_errors=True
)

 agent.invoke({
 "messages": [{"role": "user", "content": "Extract info: John Doe (john@email.com) is organizing Tech Conference on March 15th"}]
 })

```

```

===== Human Message =====

Extract info: John Doe (john@email.com) is organizing Tech Conference on March 15th
None

===== Ai Message =====

Tool Calls:
 ContactInfo (call_1)
 Call ID: call_1
 Args:
 name: John Doe
 email: john@email.com
 EventDetails (call_2)
 Call ID: call_2
 Args:
 event_name: Tech Conference
 date: March 15th

===== Tool Message =====
Name: ContactInfo

Error: Model incorrectly returned multiple structured responses (ContactInfo, Event
Details) when only one is expected.
Please fix your mistakes.

===== Tool Message =====
Name: EventDetails

Error: Model incorrectly returned multiple structured responses (ContactInfo, Event
Details) when only one is expected.
Please fix your mistakes.

===== Ai Message =====

Tool Calls:
 ContactInfo (call_3)
 Call ID: call_3
 Args:
 name: John Doe
 email: john@email.com

===== Tool Message =====
Name: ContactInfo

Returning structured response: {'name': 'John Doe', 'email': 'john@email.com'}

```

js

```
import * as z from "zod";
import { createAgent, toolStrategy } from "langchain";

const ContactInfo = z.object({
 name: z.string().describe("Person's name"),
 email: z.string().describe("Email address"),
});

const EventDetails = z.object({
 event_name: z.string().describe("Name of the event"),
 date: z.string().describe("Event date"),
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 responseFormat: toolStrategy([ContactInfo, EventDetails]),
});

const result = await agent.invoke({
 messages: [
 {
 role: "user",
 content:
 "Extract info: John Doe (john@email.com) is organizing Tech Conference on March 15th",
 },
],
});

console.log(result);

/**
 * {
 * messages: [
 * { role: "user", content: "Extract info: John Doe (john@email.com) is organizing Tech Conference on March 15th" },
 * { role: "assistant", content: "", tool_calls: [{ name: "ContactInfo", args: { name: "John Doe", email: "john@email.com" }, id: "call_1" }, { name: "EventDetails", args: { event_name: "Tech Conference", date: "March 15th" }, id: "call_2" }] },
 * { role: "tool", content: "Error: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected.\n Please fix your mistakes.", tool_call_id: "call_1", name: "ContactInfo" },
 * { role: "tool", content: "Error: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected.\n Please fix your mistakes.", tool_call_id: "call_2", name: "EventDetails" },
 * { role: "assistant", content: "", tool_calls: [{ name: "ContactInfo", args:
```

```
{ name: "John Doe", email: "john@email.com" }, id: "call_3" }] },
* { role: "tool", content: "Returning structured response: {'name': 'John Do
* e', 'email': 'john@email.com'}", tool_call_id: "call_3", name: "ContactInfo" }
*],
* structuredResponse: { name: "John Doe", email: "john@email.com" }
* }
*/
```

## Schema validation error

When structured output doesn't match the expected schema, the agent provides specific error feedback:

### python

```
from pydantic import BaseModel, Field
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ProductRating(BaseModel):
 rating: int | None = Field(description="Rating from 1-5", ge=1, le=5)
 comment: str = Field(description="Review comment")

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 response_format=ToolStrategy(ProductRating), # Default: handle_errors=True
 system_prompt="You are a helpful assistant that parses product reviews. Do not
make any field or value up."
)

agent.invoke({
 "messages": [{"role": "user", "content": "Parse this: Amazing product, 10/1
0!"}]
})
```

```
===== Human Message =====

Parse this: Amazing product, 10/10!

===== Ai Message =====

Tool Calls:
 ProductRating (call_1)
 Call ID: call_1
 Args:
 rating: 10
 comment: Amazing product

===== Tool Message =====
Name: ProductRating
```

```
Error: Failed to parse structured output for tool 'ProductRating': 1 validation error
or for ProductRating.rating
 Input should be less than or equal to 5 [type=less_than_equal, input_value=10, input_type=int].
 Please fix your mistakes.
===== Ai Message =====
Tool Calls:
 ProductRating (call_2)
Call ID: call_2
Args:
 rating: 5
 comment: Amazing product
===== Tool Message =====
Name: ProductRating

Returning structured response: {'rating': 5, 'comment': 'Amazing product'}
```

## Error handling strategies

You can customize how errors are handled using the `handle_errors` parameter:

### Custom error message:

```
ToolStrategy(
 schema=ProductRating,
 handle_errors="Please provide a valid rating between 1-5 and include a comment."
)
```

If `handle_errors` is a string, the agent will *always* prompt the model to re-try with a fixed tool message:

```
===== Tool Message =====
Name: ProductRating

Please provide a valid rating between 1-5 and include a comment.
```

### Handle specific exceptions only:

```
ToolStrategy(
 schema=ProductRating,
 handle_errors=ValueError # Only retry on ValueError, raise others
)
```

If `handle_errors` is an exception type, the agent will only retry (using the default error message) if the exception raised is the specified type. In all other cases, the exception will be raised.

## Handle multiple exception types:

```
ToolStrategy(
 schema=ProductRating,
 handle_errors=(ValueError, TypeError) # Retry on ValueError and TypeError
)
```

If `handle_errors` is a tuple of exceptions, the agent will only retry (using the default error message) if the exception raised is one of the specified types. In all other cases, the exception will be raised.

## Custom error handler function:

```
from langchain.agents.structured_output import StructuredOutputValidationError
from langchain.agents.structured_output import MultipleStructuredOutputsError

def custom_error_handler(error: Exception) -> str:
 if isinstance(error, StructuredOutputValidationError):
 return "There was an issue with the format. Try again."
 elif isinstance(error, MultipleStructuredOutputsError):
 return "Multiple structured outputs were returned. Pick the most relevant one."
 else:
 return f"Error: {str(error)}"

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 response_format=ToolStrategy(
 schema=Union[ContactInfo, EventDetails],
 handle_errors=custom_error_handler
) # Default: handle_errors=True
)

result = agent.invoke({
 "messages": [{"role": "user", "content": "Extract info: John Doe (john@email.com) is organizing Tech Conference on March 15th"}]
})

for msg in result['messages']:
 # If message is actually a ToolMessage object (not a dict), check its class name
 if type(msg).__name__ == "ToolMessage":
 print(msg.content)
 # If message is a dictionary or you want a fallback
 elif isinstance(msg, dict) and msg.get('tool_call_id'):
 print(msg['content'])
```



#### On StructuredOutputValidationError:

```
===== Tool Message =====
Name: ToolStrategy

There was an issue with the format. Try again.
```

#### On MultipleStructuredOutputsError:

```
===== Tool Message =====
Name: ToolStrategy

Multiple structured outputs were returned. Pick the most relevant one.
```

#### On other errors:

```
===== Tool Message =====
Name: ToolStrategy

Error: <error message>
```

#### No error handling:

```
response_format = ToolStrategy(
 schema=ProductRating,
 handle_errors=False # All errors raised
)
```

#### js

```
import * as z from "zod";
import { createAgent, toolStrategy } from "langchain";

const ProductRating = z.object({
 rating: z.number().min(1).max(5).describe("Rating from 1-5"),
 comment: z.string().describe("Review comment"),
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 responseFormat: toolStrategy(ProductRating),
});

const result = await agent.invoke({
 messages: [
 {
```

```

 role: "user",
 content: "Parse this: Amazing product, 10/10!",
 },
],
 });

console.log(result);

/**
 * {
 * messages: [
 * { role: "user", content: "Parse this: Amazing product, 10/10!" },
 * { role: "assistant", content: "", tool_calls: [{ name: "ProductRating", args: { rating: 10, comment: "Amazing product" }, id: "call_1" }] },
 * { role: "tool", content: "Error: Failed to parse structured output for tool 'ProductRating': 1 validation error for ProductRating\nrating\n Input should be less than or equal to 5 [type=less_than_equal, input_value=10, input_type=int].\n Please fix your mistakes.", tool_call_id: "call_1", name: "ProductRating" },
 * { role: "assistant", content: "", tool_calls: [{ name: "ProductRating", args: { rating: 5, comment: "Amazing product" }, id: "call_2" }] },
 * { role: "tool", content: "Returning structured response: {'rating': 5, 'comment': 'Amazing product'}", tool_call_id: "call_2", name: "ProductRating" }
 *],
 * structuredResponse: { rating: 5, comment: "Amazing product" }
 * }
 */

```

## Error handling strategies

You can customize how errors are handled using the `handleErrors` parameter:

### Custom error message:

```

const responseFormat = toolStrategy(ProductRating, {
 handleError: "Please provide a valid rating between 1-5 and include a comment."
})

// Error message becomes:
// { role: "tool", content: "Please provide a valid rating between 1-5 and include a comment." }

```

### Handle specific exceptions only:

```

import { ToolInputParsingException } from "@langchain/core/tools";

const responseFormat = toolStrategy(ProductRating, {
 handleError: (error: ToolStrategyError) => {
 if (error instanceof ToolInputParsingException) {
 return "Please provide a valid rating between 1-5 and include a comment.";
 }
 }
})

```

```

 }
 return error.message;
 }
)

// Only validation errors get retried with default message:
// { role: "tool", content: "Error: Failed to parse structured output for tool 'ProductRating': ...\n Please fix your mistakes." }

```

### Handle multiple exception types:

```

const responseFormat = toolStrategy(ProductRating, {
 handleError: (error: ToolStrategyError) => {
 if (error instanceof ToolInputParsingException) {
 return "Please provide a valid rating between 1-5 and include a comment.";
 }
 if (error instanceof CustomUserError) {
 return "This is a custom user error.";
 }
 return error.message;
 }
})

```

### No error handling:

```

const responseFormat = toolStrategy(ProductRating, {
 handleError: false // All errors raised
})

```

# LangSmith Studio

---

langchain/studio.mdx

**python**

**js**

**팁**

For more information about deployed agents, see [Deployment](#).

# Tools

langchain/tools.mdx

Tools extend what [agents](#) can do—letting them fetch real-time data, execute code, query external databases, and take actions in the world.

Under the hood, tools are callable functions with well-defined inputs and outputs that get passed to a [chat model](#). The model decides when to invoke a tool based on the conversation context, and what input arguments to provide.

## 팁

For details on how models handle tool calls, see [Tool calling](#). Trace tool calls and debug errors with [LangSmith](#). Follow the [tracing quickstart](#) to get set up.

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

## Create tools

### Basic tool definition

#### python

The simplest way to create a tool is with the `@tool` decorator. By default, the function's docstring becomes the tool's description that helps the model understand when to use it:

```
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
 """Search the customer database for records matching the query.

 Args:
 query: Search terms to look for
 limit: Maximum number of results to return
 """
 return f"Found {limit} results for '{query}'"
```

Type hints are **required** as they define the tool's input schema. The docstring should be informative and concise to help the model understand the tool's purpose.

## js

The simplest way to create a tool is by importing the `tool` function from the `langchain` package. You can use [zod](#) to define the tool's input schema:

```
import * as z from "zod"
import { tool } from "langchain"

const searchDatabase = tool(
 ({ query, limit }) => `Found ${limit} results for '${query}'`,
 {
 name: "search_database",
 description: "Search the customer database for records matching the query.",
 schema: z.object({
 query: z.string().describe("Search terms to look for"),
 limit: z.number().describe("Maximum number of results to return"),
 }),
 }
);
```

## 참고

**Server-side tool use:** Some chat models feature built-in tools (web search, code interpreters) that are executed server-side. See [Server-side tool use](#) for details.

## 주의

Prefer `snake_case` for tool names (e.g., `web_search` instead of `web Search`). Some model providers have issues with or reject names containing spaces or special characters with errors. Sticking to alphanumeric characters, underscores, and hyphens helps to improve compatibility across providers.

## python

### Customize tool properties

#### Custom tool name

By default, the tool name comes from the function name. Override it when you need something more descriptive:

```
@tool("web_search") # Custom name
def search(query: str) -> str:
 """Search the web for information."""
 return f"Results for: {query}"
```

```
print(search.name) # web_search
```

## Custom tool description

Override the auto-generated tool description for clearer model guidance:

```
@tool("calculator", description="Performs arithmetic calculations. Use this for any
math problems.")
def calc(expression: str) -> str:
 """Evaluate mathematical expressions."""
 return str(eval(expression))
```

## Advanced schema definition

Define complex inputs with Pydantic models or JSON schemas:

```
```python Pydantic model from pydantic import BaseModel, Field from typing import  
Literal
```

```
class WeatherInput(BaseModel): """Input for weather queries.""" location: str =  
Field(description="City name or coordinates") units: Literal["celsius", "fahrenheit"] =  
Field( default="celsius", description="Temperature unit preference" ) include_forecast:  
bool = Field( default=False, description="Include 5-day forecast" )
```

```
@tool(args_schema=WeatherInput) def get_weather(location: str, units: str = "celsius",  
include_forecast: bool = False) -> str: """Get current weather and optional forecast."""  
temp = 22 if units == "celsius" else 72 result = f"Current weather in {location}: {temp}  
degrees {units[0].upper()}" if include_forecast: result += "\nNext 5 days: Sunny" return  
result
```

```
```python JSON Schema  
weather_schema = {
 "type": "object",
 "properties": {
 "location": {"type": "string"},
 "units": {"type": "string"},
 "include_forecast": {"type": "boolean"}
 },
 "required": ["location", "units", "include_forecast"]
}

@tool(args_schema=weather_schema)
def get_weather(location: str, units: str = "celsius", include_forecast: bool = Fal
se) -> str:
 """Get current weather and optional forecast."""
 temp = 22 if units == "celsius" else 72
```

```
result = f"Current weather in {location}: {temp} degrees {units[0].upper()}"
if include_forecast:
 result += "\nNext 5 days: Sunny"
return result
```

## Reserved argument names

The following parameter names are reserved and cannot be used as tool arguments. Using these names will cause runtime errors.

Parameter name	Purpose
config	Reserved for passing <code>RunnableConfig</code> to tools internally
runtime	Reserved for <code>ToolRuntime</code> parameter (accessing state, context, store)

To access runtime information, use the `@[ ToolRuntime ]` parameter instead of naming your own arguments `config` or `runtime`.

If you use `InjectedState`, `InjectedStore`, `get_runtime()`, or `InjectedToolCallId`, see [Migrate from older injection patterns](#).

## Access context

Tools are most powerful when they can access runtime information like conversation history, user data, and persistent memory. This section covers how to access and update this information from within your tools.

### python

Tools can access runtime information through the `@[ ToolRuntime ]` parameter, which provides:

Component	Description	Use case
State	Short-term memory - mutable data that exists for the current conversation (messages, counters, custom fields)	Access conversation history, track tool call counts
Context	Immutable configuration passed at invocation time (user IDs, session info)	Personalize responses based on user identity
Store	Long-term memory - persistent data that survives across conversations	Save user preferences, maintain knowledge base
Stream Writer	Emit real-time updates during tool execution	Show progress for long-running operations



Component	Description	Use case
<b>Execution Info</b>	Identity and retry information for the current execution (thread ID, run ID, attempt number)	Access thread/run IDs, adjust behavior based on retry state
<b>Server Info</b>	Server-specific metadata when running on LangGraph Server (assistant ID, graph ID, authenticated user)	Access assistant ID, graph ID, or authenticated user info
<b>Config</b>	@[ RunnableConfig ] for the execution	Access callbacks, tags, and metadata
<b>Tool Call ID</b>	Unique identifier for the current tool invocation	Correlate tool calls for logs and model invocations

```

graph LR
 %% Runtime Context
 subgraph "🔧 Tool Runtime Context"
 A[Tool Call] --> B[ToolRuntime]
 B --> C[State Access]
 B --> D[Context Access]
 B --> E[Store Access]
 B --> F[Stream Writer]
 end

 %% Available Resources
 subgraph "📦 Available Resources"
 C --> G[Messages]
 C --> H[Custom State]
 D --> I[User ID]
 D --> J[Session Info]
 E --> K[Long-term Memory]
 E --> L[User Preferences]
 end

 %% Tool Capabilities
 subgraph "⚡ Enhanced Tool Capabilities"
 M[Context-Aware Tools]
 N[Stateful Tools]
 O[Memory-Enabled Tools]
 P[Streaming Tools]
 end

 %% Connections
 G --> M
 H --> N
 I --> M
 J --> M
 K --> O

```

```
L --> O
F --> P
```

```
classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef output fill:#EBD0F0,stroke:#885270,stroke-width:2px,color:#441E33
classDef neutral fill:#F2FAFF,stroke:#40668D,stroke-width:2px,color:#2F4B68
```

```
class A trigger
class B,C,D,E,F process
class G,H,I,J,K,L neutral
class M,N,O,P output
```

## Short-term memory (State)

State represents short-term memory that exists for the duration of a conversation. It includes the message history and any custom fields you define in your [graph state](#).

### Access state

Add `runtime: ToolRuntime` to your tool signature to access state. At call time, `@[ ToolNode ]` injects the value automatically; the parameter is not included in the tool schema sent to the model. Use `runtime.state` to read the current conversation state:

```
from langchain.tools import tool, ToolRuntime
from langchain.messages import HumanMessage

@tool
def get_last_user_message(runtime: ToolRuntime) -> str:
 """Get the most recent message from the user."""
 messages = runtime.state["messages"]

 # Find the last human message
 for message in reversed(messages):
 if isinstance(message, HumanMessage):
 return message.content

 return "No user messages found"

Access custom state fields
@tool
def get_user_preference(
 pref_name: str,
 runtime: ToolRuntime
) -> str:
 """Get a user preference value."""
 preferences = runtime.state.get("user_preferences", {})
 return preferences.get(pref_name, "Not set")
```

## 주의

The `runtime` parameter is hidden from the model. For the example above, the model only sees `pref_name` in the tool schema.

## Update state

Use `@[Command]` to update the agent's state. This is useful for tools that need to update custom state fields. Include a `ToolMessage` in the update so the model can see the result of the tool call:

## 팁

When tools update state variables, consider defining a [reducer](#) for those fields. Since LLMs can call multiple tools in parallel, a reducer determines how to resolve conflicts when the same state field is updated by concurrent tool calls.

## Context

Context provides immutable configuration data that is passed at invocation time. Use it for user IDs, session details, or application-specific settings that shouldn't change during a conversation.

## 참고

While `thread_id` (passed via `config={"configurable": {"thread_id": ...}}`) scopes the *conversation*: message history and checkpoints, `context` carries *per-run* data your tools and middleware read at invocation time. Persistence requires a [checkpointer](#). In production you typically pass both a stable `thread_id` per conversation and a `context` object on every invoke.

## python

Access context through `runtime.context`. Configure a [checkpointer](#) and pass `context` with a stable `thread_id` so the conversation persists across invocations:

## js

Tools can access an agent's runtime context through the `config` parameter. Configure a [checkpointer](#) and pass `context` with a stable `thread_id` so the conversation persists across invocations:

## Long-term memory (Store)

The `@BaseStore` provides persistent storage that survives across conversations. Unlike state (short-term memory), data saved to the store remains available in future sessions.

### python

Access the store through `runtime.store`. The store uses a namespace/key pattern to organize data:

#### 팁

For production deployments, use a persistent store implementation like `@PostgresStore`, `MongoDBStore`, or `RedisStore` instead of `InMemoryStore`. See the [memory documentation](#) for setup details.

```
```python
from langgraph.store.memory import InMemoryStore
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime
from langchain_openai import ChatOpenAI
```

Access memory

```
@tool
def get_user_info(user_id: str, runtime: ToolRuntime) -> str:
    """Look up user info."""
    store = runtime.store
    user_info = store.get(("users",), user_id)
    return str(user_info.value) if user_info else "Unknown user"
```

Update memory

```
@tool
def save_user_info(user_id: str, name: str, age: int, email: str, runtime: ToolRuntime) -> str:
    """Save user info."""
    store = runtime.store
    store.put(("users",), user_id, {"name": name, "age": age, "email": email})
    return "Successfully saved user info."
```

```
model = ChatOpenAI(model="gpt-5.5")
```

```
store = InMemoryStore()
agent = create_agent(model, tools=[get_user_info, save_user_info], store=store)
```

First session: save user info

```
agent.invoke({ "messages": [{"role": "user", "content": "Save the following user: userid: abc123, name: Foo, age: 25, email: foo@langchain.dev"}] })
```

Second session: get user info

```
agent.invoke({ "messages": [{ "role": "user", "content": "Get user info for user with id 'abc123'" }] })
```

Here is the user info for user with ID "abc123":

- Name: Foo

- Age: 25

- Email: foo@langchain.dev

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

Access the store through `config.store`. The store uses a namespace/key pattern to organize data:

```
<div class="callout tip" markdown="1">
```

```
<p class="callout-label">팁</p>
```

For production deployments, use a persistent store implementation like `@`PostgresStore``, ``MongoDBStore``, or ``RedisStore`` instead of ``InMemoryStore``. See the [memory documentation](/oss/langgraph/add-memory) for setup details.

```
</div>
```

```
```ts expandable
```

```
import * as z from "zod";
```

```
import { createAgent, tool } from "langchain";
```

```
import { InMemoryStore } from "@langchain/langgraph";
```

```
import { ChatOpenAI } from "@langchain/openai";
```

```
const store = new InMemoryStore();
```

```

// Access memory
const getUserInfo = tool(
 async ({ user_id }) => {
 const value = await store.get(["users"], user_id);
 console.log("get_user_info", user_id, value);
 return value;
 },
 {
 name: "get_user_info",
 description: "Look up user info.",
 schema: z.object({
 user_id: z.string(),
 }),
 }
);

// Update memory
const saveUserInfo = tool(
 async ({ user_id, name, age, email }) => {
 console.log("save_user_info", user_id, name, age, email);
 await store.put(["users"], user_id, { name, age, email });
 return "Successfully saved user info.";
 },
 {
 name: "save_user_info",
 description: "Save user info.",
 schema: z.object({
 user_id: z.string(),
 name: z.string(),
 age: z.number(),
 email: z.string(),
 }),
 }
);

const agent = createAgent({
 model: new ChatOpenAI({ model: "gpt-5.5" }),
 tools: [getUserInfo, saveUserInfo],
 store,
});

// First session: save user info
await agent.invoke({
 messages: [
 {
 role: "user",
 content: "Save the following user: userid: abc123, name: Foo, age: 25, email: foo@langchain.dev",
 },
],
});

```

```
});

// Second session: get user info
const result = await agent.invoke({
 messages: [
 { role: "user", content: "Get user info for user with id 'abc123'" },
],
});

console.log(result);
// Here is the user info for user with ID "abc123":
// - Name: Foo
// - Age: 25
// - Email: foo@langchain.dev
```

## Stream writer

Stream real-time updates from tools during execution. This is useful for providing progress feedback to users during long-running operations.

### python

Use `runtime.stream_writer` to emit custom updates:

```
from langchain.tools import tool, ToolRuntime

@tool
def get_weather(city: str, runtime: ToolRuntime) -> str:
 """Get weather for a given city."""
 writer = runtime.stream_writer

 # Stream custom updates as the tool executes
 writer(f"Looking up data for city: {city}")
 writer(f"Acquired data for city: {city}")

 return f"It's always sunny in {city}!"
```

### 참고

If you use `runtime.stream_writer` inside your tool, the tool must be invoked within a LangGraph execution context. See [Streaming](#) for more details.

### js

Use `config.writer` to emit custom updates:

```

import * as z from "zod";
import { tool, ToolRuntime } from "langchain";

const getWeather = tool(
 ({ city }, config: ToolRuntime) => {
 const writer = config.writer;

 // Stream custom updates as the tool executes
 if (writer) {
 writer(`Looking up data for city: ${city}`);
 writer(`Acquired data for city: ${city}`);
 }

 return `It's always sunny in ${city}!`;
 },
 {
 name: "get_weather",
 description: "Get weather for a given city.",
 schema: z.object({
 city: z.string(),
 }),
 }
);

```

## Execution info

Access thread ID, run ID, and retry state from within a tool via `runtime.execution_info`:

### python

```

from langchain.tools import tool, ToolRuntime

@tool
def log_execution_context(runtime: ToolRuntime) -> str:
 """Log execution identity information."""
 info = runtime.execution_info
 print(f"Thread: {info.thread_id}, Run: {info.run_id}") # [!code highlight]
 print(f"Attempt: {info.node_attempt}")
 return "done"

```

### js

```

import { tool } from "langchain";
import * as z from "zod";

const logExecutionContext = tool(
 async (_input, runtime) => {

```



```

const info = runtime.executionInfo;
console.log(`Thread: ${info.threadId}, Run: ${info.runId}`); // [!code highlight]
console.log(`Attempt: ${info.nodeAttempt}`);
return "done";
},
{
 name: "log_execution_context",
 description: "Log execution identity information.",
 schema: z.object({}),
}
);

```

## python

### 참고

Requires `deepagents>=0.5.0` (or `langgraph>=1.1.5`).

## js

### 참고

Requires `deepagents>=1.9.0` (or `@langchain/langgraph>=1.2.8`).

## Server info

When your tool runs on LangGraph Server, access the assistant ID, graph ID, and authenticated user via `runtime.server_info`:

## python

```

from langchain.tools import tool, ToolRuntime

@tool
def get_assistant_scoped_data(runtime: ToolRuntime) -> str:
 """Fetch data scoped to the current assistant."""
 server = runtime.server_info
 if server is not None:
 print(f"Assistant: {server.assistant_id}, Graph: {server.graph_id}") # [!code highlight]
 if server.user is not None:
 print(f"User: {server.user.identity}") # [!code highlight]
 return "done"

```

`server_info` is `None` when the tool is not running on LangGraph Server (e.g., during local development or testing).

**js**

```
import { tool } from "langchain";
import * as z from "zod";

const getAssistantScopedData = tool(
 async (_input, runtime) => {
 const server = runtime.serverInfo;
 if (server != null) {
 console.log(`Assistant: ${server.assistantId}, Graph: ${server.graphId}`);
 // [!code highlight]
 if (server.user != null) {
 console.log(`User: ${server.user.identity}`); // [!code highlight]
 }
 }
 return "done";
 },
 {
 name: "get_assistant_scoped_data",
 description: "Fetch data scoped to the current assistant.",
 schema: z.object({}),
 }
);
```

`serverInfo` is `null` when the tool is not running on LangGraph Server.

**python**

**참고**

Requires `deepagents`  $\geq 0.5.0$  (or `langgraph`  $\geq 1.1.5$ ).

**js**

**참고**

Requires `deepagents`  $\geq 1.9.0$  (or `@langchain/langgraph`  $\geq 1.2.8$ ).

**python**

**Migrate from older injection patterns**

Older examples used `InjectedState`, `InjectedStore`, `get_runtime()`, or `InjectedToolCallId`. Use `@[ToolRuntime]` instead for one explicit interface to state, context, store, and execution metadata.

### Previous pattern

```
from langchain.tools import tool, InjectedState

@tool
def summarize(state: InjectedState) -> str:
 """Summarize the conversation."""
 messages = state["messages"]
 return f"Conversation length: {len(messages)} messages."
```

### Recommended pattern

```
from langchain.tools import tool, ToolRuntime

@tool
def summarize(runtime: ToolRuntime) -> str:
 """Summarize the conversation."""
 messages = runtime.state["messages"]
 return f"Conversation length: {len(messages)} messages."
```

For agent-level migrations (for example `create_react_agent` and custom state), see the [LangChain v1 migration guide](#).

## Tool execution

In LangChain, tools are used by agents (for example via `@[create_agent]`) and tool error handling is configured through [middleware](#).

For LangGraph workflows, tool execution is handled by `@[ToolNode]`. See [ToolNode](#) for Graph API usage, including how tools can access the current graph state and run-scoped context.

### Tool return values

You can choose different return values for your tools:

- Return a `string` for human-readable results.
- Return an `object` for structured results the model should parse.
- Return a `Command` with optional message when you need to write to state.

## Return a string

Return a string when the tool should provide plain text for the model to read and use in its next response.

```
python
```

```
js
```

Behavior:

- The return value is converted to a `ToolMessage`.
- The model sees that text and decides what to do next.
- No agent state fields are changed unless the model or another tool does so later.

Use this when the result is naturally human-readable text.

## Return an object

Return an object (for example, a `dict`) when your tool produces structured data that the model should inspect.

```
python
```

```
js
```

Behavior:

- The object is serialized and sent back as tool output.
- The model can read specific fields and reason over them.
- Like string returns, this does not directly update graph state.

Use this when downstream reasoning benefits from explicit fields instead of free-form text.

## Return multimodal content

Tools are not limited to plain text. When the model supports multimodal tool results, the tool can return [standard content blocks](#) so the model receives text, images, and other media in one tool result.

```
python
```

```
from langchain.tools import tool

@tool
def capture_screenshot() -> list[dict]:
 """Capture a screenshot of the current page."""
```

```

return [
 {"type": "text", "text": "Screenshot of the current page:"},
 {"type": "image", "url": "https://example.com/page.png"},
]

```

**js**

```

import { tool } from "langchain";
import { z } from "zod";

const captureScreenshot = tool(
 async () => [
 { type: "text", text: "Screenshot of the current page:" },
 { type: "image", url: "https://example.com/page.png" },
],
 {
 name: "capture_screenshot",
 description: "Capture a screenshot of the current page.",
 schema: z.object({}),
 }
);

```

Behavior:

- The return value is converted to a `ToolMessage` with multimodal content.
- Use `message.content_blocks` to read the normalized block list after the tool runs.
- The model must support the modalities you return. Check your [model's capabilities](#) before returning images, audio, or video.

For block types and provider-specific requirements, see [Multimodal messages](#).

**python**

MCP tools that return images or mixed content are converted the same way. See [Multimodal content](#).

## Return a Command

Return a `@[Command]` when the tool needs to update graph state (for example, setting user preferences or app state). When the `Command` targets the current graph, include a `ToolMessage` in the update whose tool call ID matches the current tool call. Every tool call in the message history must have a corresponding `ToolMessage`.

**python**

Use `runtime.tool_call_id` for the `tool_call_id` parameter. `ToolNode` enforces this requirement: if the update has no `ToolMessage` matching the tool call, it raises a `ValueError`.

**js**

Use `runtime.toolCallId` for the `tool_call_id` parameter.

**python**

**js**

Behavior:

- The command updates state using `update`.
- Updated state is available to subsequent steps in the same run.
- Use reducers for fields that may be updated by parallel tool calls.

Use this when the tool is not just returning data, but also mutating agent state.

### Return directly from a tool

Set `return_direct` on a tool to short-circuit the agent loop: the agent returns the tool's output to the caller immediately, without sending it back through the model for further processing.

**python**

**js**

Behavior:

- The tool executes normally and its output is wrapped in a `ToolMessage`.
- The agent stops looping and returns the tool's output as the final response, bypassing any additional model call.
- **Multiple parallel tool calls:** When the model calls several tools in one step, all of them execute first. After all tools finish, the agent routes to `END` only if **every** tool in that batch has `return_direct=True`. The final response includes the `ToolMessage` outputs of every tool that was called in that step.

Use this when:

- The tool's output is the complete, user-ready answer (for example, a lookup that returns a ready-to-display result).

- You want to avoid an extra model call when no additional reasoning is needed.
- You need deterministic, unmodified output: the model cannot rephrase, summarize, or act on the tool result.

### 주의

Because the model does not process the tool's output, `return_direct=True` is not suitable for tools whose results require further reasoning, summarization, or chaining with other tool calls.

### 주의

**Mixed parallel calls:** If the model calls a `return_direct=True` tool alongside tools that do not have `return_direct=True`, the agent does **not** exit after that step. It routes back to the model with every `ToolMessage` from the batch, so the model can reason over all results. `return_direct` only short-circuits the loop when every tool call in the step has `return_direct=True`.

## python

### Return a Command with `return_direct`

A tool with `return_direct=True` can also return a `@[Command]` to update graph state before the agent exits. Unlike a plain return value, a `Command` is not automatically converted into a `ToolMessage`. When the `Command` targets the current graph (`graph` is not set or is `None`), include a `ToolMessage` in `Command.update` matching the tool call's `tool_call_id`. Omitting it causes `ToolNode` to raise a `ValueError`, because every `AIMessage` tool call must have a corresponding `ToolMessage` in the message history.

To write to a parent graph instead, set `graph=Command.PARENT`. In that case the `ToolMessage` requirement is lifted because execution leaves the current graph entirely.

## Error handling

Handle tool errors using LangChain agent [middleware](#) to retry failed tool calls or return custom error messages:

### python

### js

## State injection

Tools access graph state through `@[ToolRuntime]`. See [Access context](#) for state, context, store, and streaming APIs.

### python

```
from langchain.tools import tool, ToolRuntime

@tool
def get_message_count(runtime: ToolRuntime) -> str:
 """Get the number of messages in the conversation."""
 messages = runtime.state["messages"]
 return f"There are {len(messages)} messages."
```

For more details on accessing state, context, and long-term memory from tools, see [Access context](#).

## Dynamic tool selection

With dynamic tools, the set of tools available to the agent is modified at runtime rather than defined all upfront. Not every tool is appropriate for every situation. Too many tools may overwhelm the model (overload context) and increase errors; too few limit capabilities. Dynamic tool selection enables adapting the available toolset based on authentication state, user permissions, feature flags, or conversation stage.

There are two approaches depending on whether tools are known ahead of time:

### Filtering pre-registered tools

When all possible tools are known at agent creation time, you can pre-register them and dynamically filter which ones are exposed to the model based on state, permissions, or context.

### State

Enable advanced tools only after certain conversation milestones:

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def state_based_tools(
```



```

 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Filter tools based on conversation State."""
 # Read from State: check if user has authenticated
 state = request.state
 is_authenticated = state.get("authenticated", False)
 message_count = len(state["messages"])

 # Only enable sensitive tools after authentication
 if not is_authenticated:
 tools = [t for t in request.tools if t.name.startswith("public_")]
 request = request.override(tools=tools)
 elif message_count < 5:
 # Limit tools early in conversation
 tools = [t for t in request.tools if t.name != "advanced_search"]
 request = request.override(tools=tools)

 return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[public_search, private_search, advanced_search],
 middleware=[state_based_tools]
)

```

js

```

import { createMiddleware, tool } from "langchain";
import { createDeepAgent } from "deepagents";

const stateBasedTools = createMiddleware({
 name: "StateBasedTools",
 wrapModelCall: (request, handler) => {
 // Read from State: check authentication and conversation length
 const state = request.state as typeof request.state & {
 authenticated?: boolean;
 };
 const isAuthenticated = state.authenticated ?? false;
 const messageCount = state.messages.length;

 let filteredTools = request.tools;

 // Only enable sensitive tools after authentication
 if (!isAuthenticated) {
 filteredTools = request.tools.filter(
 (t: any) => typeof t.name === "string" && t.name.startsWith("public_"),
);
 } else if (messageCount < 5) {

```

```

 filteredTools = request.tools.filter(
 (t: any) => typeof t.name === "string" && t.name !== "advanced_search",
);
 }

 return handler({ ...request, tools: filteredTools });
},
});

const agent = await createDeepAgent({
 model: "claude-sonnet-4-6",
 tools: tools,
 middleware: [stateBasedTools] as any,
});

```

## Store

Filter tools based on user preferences or feature flags in Store:

### python

```

from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
 user_id: str

@wrap_model_call
def store_based_tools(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Filter tools based on Store preferences."""
 user_id = request.runtime.context.user_id

 # Read from Store: get user's enabled features
 store = request.runtime.store
 feature_flags = store.get(("features",), user_id)

 if feature_flags:
 enabled_features = feature_flags.value.get("enabled_tools", [])
 # Only include tools that are enabled for this user
 tools = [t for t in request.tools if t.name in enabled_features]
 request = request.override(tools=tools)

```

```

return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, analysis_tool, export_tool],
 middleware=[store_based_tools],
 context_schema=Context,
 store=InMemoryStore()
)

```

js

```

import { createMiddleware } from "langchain";
import { createDeepAgent, StoreBackend } from "deepagents";
import * as z from "zod";
import { InMemoryStore } from "@langchain/langgraph";

const contextSchema = z.object({
 userId: z.string(),
});

const storeBasedTools = createMiddleware({
 name: "StoreBasedTools",
 contextSchema,
 wrapModelCall: async (request, handler) => {
 const userId =
 (request.runtime?.context as { userId?: string } | undefined)?.userId ??
 "user-123";

 // Read from Store: get user's enabled features
 const runtimeStore = request.runtime?.store as InMemoryStore | undefined;
 const rawFlags = (await runtimeStore?.get(
 ["features"],
 userId as string,
)) as unknown;
 const featureFlags = rawFlags as FeatureFlags | undefined;

 let filteredTools = request.tools;

 if (featureFlags) {
 const enabledFeatures = featureFlags.enabledTools || [];
 filteredTools = request.tools.filter((t) =>
 enabledFeatures.includes(t.name as string)
);
 }

 return handler({ ...request, tools: filteredTools });
 },

```

```
});

const agent = await createDeepAgent({
 model: "claude-sonnet-4-6",
 backend: new StoreBackend(),
 store,
 checkpointier,
 tools,
 middleware: [storeBasedTools] as any,
});
```

## Runtime Context

Filter tools based on user permissions from Runtime Context:

### python

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@dataclass
class Context:
 user_role: str

@wrap_model_call
def context_based_tools(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Filter tools based on Runtime Context permissions."""
 # Read from Runtime Context: get user role
 if request.runtime is None or request.runtime.context is None:
 # If no context provided, default to viewer (most restrictive)
 user_role = "viewer"
 else:
 user_role = request.runtime.context.user_role

 if user_role == "admin":
 # Admins get all tools
 pass
 elif user_role == "editor":
 # Editors can't delete
 tools = [t for t in request.tools if t.name != "delete_data"]
 request = request.override(tools=tools)
 else:
 # Viewers get read-only tools
```

```

 tools = [t for t in request.tools if t.name.startswith("read_")]
 request = request.override(tools=tools)

return handler(request)

agent = create_agent(
 model="gpt-5.5",
 tools=[read_data, write_data, delete_data],
 middleware=[context_based_tools],
 context_schema=Context
)

```

js

```

import * as z from "zod";
import { createMiddleware } from "langchain";
import { createDeepAgent } from "deepagents";

const contextSchema = z.object({
 userRole: z.string(),
});

const contextBasedTools = createMiddleware({
 name: "ContextBasedTools",
 contextSchema,
 wrapModelCall: (request, handler) => {
 // Read from Runtime Context: get user role
 const userRole = request.runtime.context.userRole;

 let filteredTools = request.tools;

 if (userRole === "admin") {
 // Admins get all tools
 } else if (userRole === "editor") {
 filteredTools = request.tools.filter((t) => t.name !== "delete_data");
 } else {
 filteredTools = request.tools.filter(
 (t) => (t.name as string).startsWith("read_"),
);
 }

 return handler({ ...request, tools: filteredTools });
 },
});

const agent = await createDeepAgent({
 model: "claude-sonnet-4-6",
 store,
 checkpoint,
});

```

```
tools,
middleware: [contextBasedTools] as any,
});
```

This approach is best when: - All possible tools are known at compile/startup time - You want to filter based on permissions, feature flags, or conversation state - Tools are static but their availability is dynamic

See [Dynamically selecting tools](/oss/langchain/middleware/custom#dynamically-selecting-tools) for more examples.

## Runtime tool registration

When tools are discovered or created at runtime (e.g., loaded from an MCP server, generated based on user data, or fetched from a remote registry), you need to both register the tools and handle their execution dynamically.

This requires two middleware hooks: 1. `wrap_model_call` - Add the dynamic tools to the request 2. `wrap_tool_call` - Handle execution of the dynamically added tools

### python

```
from langchain.tools import tool
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ToolCallRequest

A tool that will be added dynamically at runtime
@tool
def calculate_tip(bill_amount: float, tip_percentage: float = 20.0) -> str:
 """Calculate the tip amount for a bill."""
 tip = bill_amount * (tip_percentage / 100)
 return f"Tip: ${tip:.2f}, Total: ${bill_amount + tip:.2f}"

class DynamicToolMiddleware(AgentMiddleware):
 """Middleware that registers and handles dynamic tools."""

 def wrap_model_call(self, request: ModelRequest, handler):
 # Add dynamic tool to the request
 # This could be loaded from an MCP server, database, etc.
 updated = request.override(tools=[*request.tools, calculate_tip])
 return handler(updated)

 def wrap_tool_call(self, request: ToolCallRequest, handler):
 # Handle execution of the dynamic tool
 if request.tool_call["name"] == "calculate_tip":
 return handler(request.override(tool=calculate_tip))
 return handler(request)
```

```

agent = create_agent(
 model="gpt-5.5",
 tools=[get_weather], # Only static tools registered here
 middleware=[DynamicToolMiddleware()],
)

The agent can now use both get_weather AND calculate_tip
result = agent.invoke({
 "messages": [{"role": "user", "content": "Calculate a 20% tip on $85"}]
})

```

js

```

import { createAgent, createMiddleware, tool } from "langchain";
import * as z from "zod";

// A tool that will be added dynamically at runtime
const calculateTip = tool(
 ({ billAmount, tipPercentage = 20 }) => {
 const tip = billAmount * (tipPercentage / 100);
 return `Tip: ${tip.toFixed(2)}, Total: ${((billAmount + tip).toFixed(2))}`;
 },
 {
 name: "calculate_tip",
 description: "Calculate the tip amount for a bill",
 schema: z.object({
 billAmount: z.number().describe("The bill amount"),
 tipPercentage: z.number().default(20).describe("Tip percentage"),
 }),
 }
);

const dynamicToolMiddleware = createMiddleware({
 name: "DynamicToolMiddleware",
 wrapModelCall: (request, handler) => {
 // Add dynamic tool to the request
 // This could be loaded from an MCP server, database, etc.
 return handler({
 ...request,
 tools: [...request.tools, calculateTip],
 });
 },
 wrapToolCall: (request, handler) => {
 // Handle execution of the dynamic tool
 if (request.toolCall.name === "calculate_tip") {
 return handler({ ...request, tool: calculateTip });
 }
 return handler(request);
 }
});

```

```

 },
 });

 const agent = createAgent({
 model: "gpt-5.5",
 tools: [getWeather], // Only static tools registered here
 middleware: [dynamicToolMiddleware],
 });

 // The agent can now use both getWeather AND calculateTip
 const result = await agent.invoke({
 messages: [{ role: "user", content: "Calculate a 20% tip on $85" }],
 });

```

This approach is best when: - Tools are discovered at runtime (e.g., from an MCP server) - Tools are generated dynamically based on user data or configuration - You're integrating with external tool registries

## 참고

The `wrap_tool_call` hook is required for runtime-registered tools because the agent needs to know how to execute tools that weren't in the original tool list. Without it, the agent won't know how to invoke the dynamically added tool.

## Headless tools

Some tools should run **where your user's app runs** (typically the browser), not inside the process. **Headless tools** are tool definitions, which include the name, description, and argument schema, that you register on the **server** with your agent. The **implementation** is registered only on the **client** and executed after a short interrupt/resume handshake.

This is different from ordinary tools whose function body runs on the server, and from [server-side tool use](#) where the model provider executes built-in tools remotely.

## When to use headless tools

Use them when the work depends on the **environment, device, or UI** that only exists on the client. For example:

- **Browser APIs:** Geolocation, IndexedDB, Clipboard, Canvas 2D, file pickers, Battery API, etc.
- **Privacy and locality:** Data stays on the device (for example, local “memory” in IndexedDB).
- **Latency:** No extra server round trip for purely local operations.



- **Structured, safe effects:** Prefer many small, typed tools (for example one tool per canvas primitive) instead of sending arbitrary code to `eval`.

## How the pattern works

In both runtimes, the model sees a normal tool it can call, but the actual execution happens outside the server process.

### python

1. **Define** a headless tool with `tool(name=..., description=..., args_schema=...)` from `langchain.tools`. A headless tool is schema-only, with no in-process implementation.
2. **Register** that tool with `create_agent` or your `LangGraph` graph so the model can call it normally.
3. **Handle** the interrupt payload when the tool is invoked. Instead of running locally, the graph pauses with a payload shaped like `{"type": "tool", "tool_call": {"id", "name", "args"}}`.
4. **Resume** the graph after your app, another service, or a human step performs the action. For browser-based flows, you can mirror the schema in the frontend and attach `.implement(...)` there.

### 정보

If you call `tool(...)` in Python with only `name`, `description`, and `args_schema`, `LangChain` returns a `HeadlessTool`. There is no `.implement()` API on the Python side.

### js

1. **Define** the tool with `tool({ name, description, schema })` from `langchain`, metadata and validation only, no server-side runner.
2. **Attach** the real behavior with `.implement(async (args) => { ... })`, which returns a **headless tool implementation** (definition + `execute` function).
3. **Register** the definition from step 1 with `createAgent` or your graph so the model sees the tool in its usual tool-calling loop.
4. **Pass** the implementation from step 2 to your streaming hook's `tools` option.

### 정보

Put **tool definitions** (`tool({ name, description, schema })`) and **implementations** (`.implement(...)`) in **separate modules**. Import the shared definition file from your

server agent and from your frontend so names and schemas stay aligned; keep client-only execute logic in implementation modules the server never loads.

When the model issues a tool call for one of these tools, the run **interrupts** instead of executing the tool locally. Your app can inspect the payload, perform the action in the right environment (for example a browser, another service, or a human review step), then **resume** the graph with the tool result. When you use the supported JS SDK hooks, they can detect headless-tool interrupts, run the matching client implementation, and submit the resume command for you.

Use the optional `onTool` callback to observe lifecycle events ( `start` , `success` , `error` ) for UI feedback such as spinners or toasts.

### [Headless tools frontend pattern](#)

See an end-to-end example of schema-only tools executed in the client with `useStream`.

## Prebuilt tools

LangChain provides a large collection of prebuilt tools and toolkits for common tasks like web search, code interpretation, database access, and more. These ready-to-use tools can be directly integrated into your agents without writing custom code.

See the [tools and toolkits](#) integration page for a complete list of available tools organized by category.

## Tools from MCP servers

[Model Context Protocol \(MCP\)](#) is an open protocol that standardizes how applications expose tools to language models. Rather than hand-authoring a tool, you connect to an MCP server and adapt the tools it advertises into LangChain tools, ready to pass to an agent like any other tool.

### python

`@[ MCPAdapter ]` discovers a server's tools and converts them into LangChain tools. Open the adapter, call `list_tools()`, and hand the result to `@[ create_agent ]`:

```
from langchain.agents import create_agent
from langchain.mcp import MCPAdapter

async with MCPAdapter("https://example.com/mcp") as adapter:
 tools = await adapter.list_tools()
 agent = create_agent("claude-sonnet-4-6", tools)
```

## 참고

The `langchain.mcp` namespace requires `langchain[mcp]>=1.4.0` and is in beta. The API may change.

For transports, authentication, multiple servers, and handling tool results, see [Model Context Protocol \(MCP\)](#).

## js

LangChain agents call tools defined on MCP servers through the [@langchain/mcp-adapters](#) library, which discovers a server's tools and adapts them into LangChain tools.

For setup, transports, and authentication, see [Model Context Protocol \(MCP\)](#).

## Server-side tool use

---

Some chat models feature built-in tools that are executed server-side by the model provider. These include capabilities like web search and code interpreters that don't require you to define or host the tool logic.

Refer to the individual [chat model integration pages](#) and the [tool calling documentation](#) for details on enabling and using these built-in tools.

# Agent Chat UI

---

langchain/ui.mdx

## Connect to your agent

Agent Chat UI can connect to both [local](#) and [deployed agents](#).

After starting Agent Chat UI, you'll need to configure it to connect to your agent:

1. **Graph ID:** Enter your graph name (find this under `graphs` in your `langgraph.json` file)
2. **Deployment URL:** Your Agent server's endpoint (e.g., `http://localhost:2024` for local development, or your deployed agent's URL)
3. **LangSmith API key (optional):** Add your LangSmith API key (not required if you're using a local Agent server)

Once configured, Agent Chat UI will automatically fetch and display any interrupted threads from your agent.

### 팁

Agent Chat UI has out-of-the-box support for rendering tool calls and tool result messages. To customize what messages are shown, see [Hiding Messages in the Chat](#).

# Build a voice agent with LangChain

langchain/voice-agent.mdx

## Overview

Chat interfaces have dominated how we interact with AI, but recent breakthroughs in multimodal AI are opening up exciting new possibilities. High-quality generative models and expressive text-to-speech (TTS) systems now make it possible to build agents that feel less like tools and more like conversational partners.

Voice agents are one example of this. Instead of relying on a keyboard and mouse to type inputs into an agent, you can use spoken words to interact with it. This can be a more natural and engaging way to interact with AI, and can be especially useful for certain contexts.

## What are voice agents?

Voice agents are [agents](#) that can engage in natural spoken conversations with users. These agents combine speech recognition, natural language processing, generative AI, and text-to-speech technologies to create seamless, natural conversations.

They're suited for a variety of use cases, including:

- Customer support
- Personal assistants
- Hands-free interfaces
- Coaching and training

## How do voice agents work?

At a high level, every voice agent needs to handle three tasks:

1. **Listen** - capture audio and transcribe it
2. **Think** - interpret intent, reason, plan
3. **Speak** - generate audio and stream it back to the user

The difference lies in how these steps are sequenced and coupled. In practice, production agents follow one of two main architectures:

### 1. STT > Agent > TTS architecture (The "Sandwich")

The Sandwich architecture composes three distinct components: speech-to-text (STT), a text-based LangChain agent, and text-to-speech (TTS).

```

flowchart LR
 A[User Audio] --> B[Speech-to-Text]
 B --> C[LangChain Agent]
 C --> D[Text-to-Speech]
 D --> E[Audio Output]

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,E trigger
class B,C,D process

```

**Pros:** - Full control over each component (swap STT/TTS providers as needed) - Access to latest capabilities from modern text-modality models - Transparent behavior with clear boundaries between components

**Cons:** - Requires orchestrating multiple services - Additional complexity in managing the pipeline - Conversion from speech to text loses information (e.g., tone, emotion)

## 2. Speech-to-Speech architecture (S2S)

Speech-to-speech uses a multimodal model that processes audio input and generates audio output natively.

```

flowchart LR
 A[User Audio] --> B[Multimodal Model]
 B --> C[Audio Output]

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,C trigger
class B process

```

**Pros:** - Simpler architecture with fewer moving parts - Typically lower latency for simple interactions - Direct audio processing captures tone and other nuances of speech

**Cons:** - Limited model options, greater risk of provider lock-in - Features may lag behind text-modality models - Less transparency in how audio is processed - Reduced controllability and customization options

This guide demonstrates the **sandwich architecture** to balance performance, controllability, and access to modern model capabilities. The sandwich can achieve sub-700ms latency with some STT and TTS providers while maintaining control over modular components.

## Demo Application overview

We'll walk through building a voice-based agent using the sandwich architecture. The agent will manage orders for a sandwich shop. The application will demonstrate all three components of the sandwich architecture, using [AssemblyAI](#) for STT and [Cartesia](#) for TTS (although adapters can be built for most providers).

An end-to-end reference application is available in the [voice-sandwich-demo](#) repository. We will walk through that application here.

The demo uses WebSockets for real-time bidirectional communication between the browser and server. The same architecture can be adapted for other transports like telephony systems (Twilio, Vonage) or WebRTC connections.

## Architecture

The demo implements a streaming pipeline where each stage processes data asynchronously:

**Client (Browser)** - Captures microphone audio and encodes it as PCM - Establishes WebSocket connection to the backend server - Streams audio chunks to the server in real-time - Receives and plays back synthesized speech audio

**python**

**Server (Python)**

**js**

**Server (Node.js)**

- Accepts WebSocket connections from clients
- Orchestrates the three-step pipeline:
- [Speech-to-text \(STT\)](#): Forwards audio to the STT provider (e.g., AssemblyAI), receives transcript events
- [Agent](#): Processes transcripts with LangChain agent, streams response tokens
- [Text-to-speech \(TTS\)](#): Sends agent responses to the TTS provider (e.g., Cartesia), receives audio chunks
- Returns synthesized audio to the client for playback

**python**

The pipeline uses async generators to enable streaming at each stage. This allows downstream components to begin processing before upstream stages complete,

minimizing end-to-end latency.

js

The pipeline uses async iterators to enable streaming at each stage. This allows downstream components to begin processing before upstream stages complete, minimizing end-to-end latency.

## Setup

For detailed installation instructions and setup, see the [repository README](#).

## 1. Speech-to-text

The STT stage transforms an incoming audio stream into text transcripts. The implementation uses a producer-consumer pattern to handle audio streaming and transcript reception concurrently.

### Key concepts

**Producer-Consumer Pattern:** Audio chunks are sent to the STT service concurrently with receiving transcript events. This allows transcription to begin before all audio has arrived.

**Event Types:** - `stt_chunk` : Partial transcripts provided as the STT service processes audio - `stt_output` : Final, formatted transcripts that trigger agent processing

**WebSocket Connection:** Maintains a persistent connection to AssemblyAI's real-time STT API, configured for 16kHz PCM audio with automatic turn formatting.

### Implementation

python

```
from typing import AsyncIterator
import asyncio
from assemblyai_stt import AssemblyAISTT
from events import VoiceAgentEvent

async def stt_stream(
 audio_stream: AsyncIterator[bytes],
) -> AsyncIterator[VoiceAgentEvent]:
 """
 Transform stream: Audio (Bytes) → Voice Events (VoiceAgentEvent)

 Uses a producer-consumer pattern where:
```



```

- Producer: Reads audio chunks and sends them to AssemblyAI
- Consumer: Receives transcription events from AssemblyAI
"""

stt = AssemblyAISTT(sample_rate=16000)

async def send_audio():
 """Background task that pumps audio chunks to AssemblyAI."""
 try:
 async for audio_chunk in audio_stream:
 await stt.send_audio(audio_chunk)
 finally:
 # Signal completion when audio stream ends
 await stt.close()

Launch audio sending in background
send_task = asyncio.create_task(send_audio())

try:
 # Receive and yield transcription events as they arrive
 async for event in stt.receive_events():
 yield event
finally:
 # Cleanup
 with contextlib.suppress(asyncio.CancelledError):
 send_task.cancel()
 await send_task
 await stt.close()

```

js

```

import { AssemblyAISTT } from "../assemblyai";
import type { VoiceAgentEvent } from "../types";

async function* sttStream(
 audioStream: AsyncIterable<Uint8Array>
): AsyncGenerator<VoiceAgentEvent> {
 const stt = new AssemblyAISTT({ sampleRate: 16000 });
 const passthrough = writableIterator<VoiceAgentEvent>();

 // Producer: pump audio chunks to AssemblyAI
 const producer = (async () => {
 try {
 for await (const audioChunk of audioStream) {
 await stt.sendAudio(audioChunk);
 }
 } finally {
 await stt.close();
 }
 })();

```

```
// Consumer: receive transcription events
const consumer = (async () => {
 for await (const event of stt.receiveEvents()) {
 passthrough.push(event);
 }
})();

try {
 // Yield events as they arrive
 yield* passthrough;
} finally {
 // Wait for producer and consumer to complete
 await Promise.all([producer, consumer]);
}
}
```

The application implements an AssemblyAI client to manage the WebSocket connection and message parsing. See below for implementations; similar adapters can be constructed for other STT providers.

## AssemblyAI Client

### python

```
class AssemblyAISTT:
 def __init__(self, api_key: str | None = None, sample_rate: int = 16000):
 self.api_key = api_key or os.getenv("ASSEMBLYAI_API_KEY")
 self.sample_rate = sample_rate
 self._ws: WebSocketClientProtocol | None = None

 async def send_audio(self, audio_chunk: bytes) -> None:
 """Send PCM audio bytes to AssemblyAI."""
 ws = await self._ensure_connection()
 await ws.send(audio_chunk)

 async def receive_events(self) -> AsyncIterator[STTEvent]:
 """Yield STT events as they arrive from AssemblyAI."""
 async for raw_message in self._ws:
 message = json.loads(raw_message)

 if message["type"] == "Turn":
 # Final formatted transcript
 if message.get("turn_is_formatted"):
 yield STTOutputEvent.create(message["transcript"])
 # Partial transcript
 else:
 yield STTChunkEvent.create(message["transcript"])
```

```

 async def _ensure_connection(self) -> WebSocketClientProtocol:
 """Establish WebSocket connection if not already connected."""
 if self._ws is None:
 url = f"wss://streaming.assemblyai.com/v3/ws?sample_rate={self.sample_rate}&format_turns=true"
 self._ws = await websockets.connect(
 url,
 additional_headers={"Authorization": self.api_key}
)
 return self._ws

```

js

```

export class AssemblyAISTT {
 protected _bufferIterator = writableIterator<VoiceAgentEvent.STTEvent>();
 protected _connectionPromise: Promise<WebSocket> | null = null;

 async sendAudio(buffer: Uint8Array): Promise<void> {
 const conn = await this._connection;
 conn.send(buffer);
 }

 async *receiveEvents(): AsyncGenerator<VoiceAgentEvent.STTEvent> {
 yield* this._bufferIterator;
 }

 protected get _connection(): Promise<WebSocket> {
 if (this._connectionPromise) return this._connectionPromise;

 this._connectionPromise = new Promise((resolve, reject) => {
 const params = new URLSearchParams({
 sample_rate: this.sampleRate.toString(),
 format_turns: "true",
 });
 const url = `wss://streaming.assemblyai.com/v3/ws?${params}`;
 const ws = new WebSocket(url, {
 headers: { Authorization: this.apiKey },
 });

 ws.on("open", () => resolve(ws));

 ws.on("message", (data) => {
 const message = JSON.parse(data.toString());
 if (message.type === "Turn") {
 if (message.turn_is_formatted) {
 this._bufferIterator.push({
 type: "stt_output",
 transcript: message.transcript,
 ts: Date.now()
 });
 }
 }
 });
 });
 }
}

```

```

 });
 } else {
 this._bufferIterator.push({
 type: "stt_chunk",
 transcript: message.transcript,
 ts: Date.now()
 });
 }
}
});
});

return this._connectionPromise;
}
}

```

## 2. LangChain agent

The agent stage processes text transcripts through a LangChain [agent](#) and streams the response tokens. In this case, we stream all [text content blocks](#) generated by the agent.

### Key concepts

**Streaming Responses:** The agent uses [stream\\_events\(version="v3"\)](#) with `stream.messages` to emit response tokens as they are generated, rather than waiting for the complete response. This enables the TTS stage to begin synthesis immediately.

**Conversation Memory:** A [checkpointer](#) maintains conversation state across turns using a unique thread ID. This allows the agent to reference previous exchanges in the conversation.

### Implementation

#### python

```

from langchain_core.utils.uuid import uuid7
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langgraph.checkpoint.memory import InMemorySaver

Define agent tools
def add_to_order(item: str, quantity: int) -> str:
 """Add an item to the customer's sandwich order."""
 return f"Added {quantity} x {item} to the order."

def confirm_order(order_summary: str) -> str:
 """Confirm the final order with the customer."""

```

```

 return f"Order confirmed: {order_summary}. Sending to kitchen."

Create agent with tools and memory
agent = create_agent(
 model="google_genai:gemin-3.6-flash", # Select your model
 tools=[add_to_order, confirm_order],
 system_prompt="""You are a helpful sandwich shop assistant.
 Your goal is to take the user's order. Be concise and friendly.
 Do NOT use emojis, special characters, or markdown.
 Your responses will be read by a text-to-speech engine.""",
 checkpointer=InMemorySaver(),
)

async def agent_stream(
 event_stream: AsyncIterator[VoiceAgentEvent],
) -> AsyncIterator[VoiceAgentEvent]:
 """
 Transform stream: Voice Events → Voice Events (with Agent Responses)

 Passes through all upstream events and adds agent_chunk events
 when processing STT transcripts.
 """
 # Generate unique thread ID for conversation memory
 thread_id = str(uuid7())

 async for event in event_stream:
 # Pass through all upstream events
 yield event

 # Process final transcripts through the agent
 if event.type == "stt_output":
 # Stream agent response with conversation context
 stream = await agent.astream_events(
 {"messages": [HumanMessage(content=event.transcript)]},
 {"configurable": {"thread_id": thread_id}},
 version="v3",
)

 # Yield agent response chunks as they arrive
 async for message in stream.messages:
 async for token in message.text:
 yield AgentChunkEvent.create(token)

```

js

```

import { createAgent } from "langchain";
import { HumanMessage } from "@langchain/core/messages";
import { MemorySaver } from "@langchain/langgraph";
import { tool } from "@langchain/core/tools";

```

```

import { z } from "zod";

// Define agent tools
const addToOrder = tool(
 async ({ item, quantity }) => {
 return `Added ${quantity} x ${item} to the order.`;
 },
 {
 name: "add_to_order",
 description: "Add an item to the customer's sandwich order.",
 schema: z.object({
 item: z.string(),
 quantity: z.number(),
 }),
 }
);

const confirmOrder = tool(
 async ({ orderSummary }) => {
 return `Order confirmed: ${orderSummary}. Sending to kitchen.`;
 },
 {
 name: "confirm_order",
 description: "Confirm the final order with the customer.",
 schema: z.object({
 orderSummary: z.string().describe("Summary of the order"),
 }),
 }
);

// Create agent with tools and memory
const agent = createAgent({
 model: "claude-haiku-4-5",
 tools: [addToOrder, confirmOrder],
 checkpointer: new MemorySaver(),
 systemPrompt: `You are a helpful sandwich shop assistant.
Your goal is to take the user's order. Be concise and friendly.
Do NOT use emojis, special characters, or markdown.
Your responses will be read by a text-to-speech engine.`,
});

async function* agentStream(
 eventStream: AsyncIterable<VoiceAgentEvent>
): AsyncGenerator<VoiceAgentEvent> {
 // Generate unique thread ID for conversation memory
 const threadId = crypto.randomUUID();

 for await (const event of eventStream) {
 // Pass through all upstream events
 yield event;
 }
}

```

```
// Process final transcripts through the agent
if (event.type === "stt_output") {
 const stream = await agent.streamEvents(
 { messages: [new HumanMessage(event.transcript)] },
 {
 configurable: { thread_id: threadId },
 version: "v3",
 }
);

 // Yield agent response chunks as they arrive
 for await (const message of stream.messages) {
 for await (const token of message.text) {
 yield { type: "agent_chunk", text: token, ts: Date.now() };
 }
 }
}
}
```

### 3. Text-to-speech

The TTS stage synthesizes agent response text into audio and streams it back to the client. Like the STT stage, it uses a producer-consumer pattern to handle concurrent text sending and audio reception.

#### Key concepts

**Concurrent Processing:** The implementation merges two async streams: - **Upstream processing:** Passes through all events and sends agent text chunks to the TTS provider - **Audio reception:** Receives synthesized audio chunks from the TTS provider

**Streaming TTS:** Some providers (such as [Cartesia](#)) begin synthesizing audio as soon as it receives text, enabling audio playback to start before the agent finishes generating its complete response.

**Event Passthrough:** All upstream events flow through unchanged, allowing the client or other observers to track the full pipeline state.

#### Implementation

##### python

```
from cartesia_tts import CartesiaTTS
from utils import merge_async_iters
```

```

async def tts_stream(
 event_stream: AsyncIterator[VoiceAgentEvent],
) -> AsyncIterator[VoiceAgentEvent]:
 """
 Transform stream: Voice Events → Voice Events (with Audio)

 Merges two concurrent streams:
 1. process_upstream(): passes through events and sends text to Cartesia
 2. tts.receive_events(): yields audio chunks from Cartesia
 """
 tts = CartesiaTTS()

 async def process_upstream() -> AsyncIterator[VoiceAgentEvent]:
 """Process upstream events and send agent text to Cartesia."""
 async for event in event_stream:
 # Pass through all events
 yield event
 # Send agent text to Cartesia for synthesis
 if event.type == "agent_chunk":
 await tts.send_text(event.text)

 try:
 # Merge upstream events with TTS audio events
 # Both streams run concurrently
 async for event in merge_async_iters(
 process_upstream(),
 tts.receive_events()
):
 yield event
 finally:
 await tts.close()

```

js

```

import { CartesiaTTS } from "../cartesia";

async function* ttsStream(
 eventStream: AsyncIterable<VoiceAgentEvent>
): AsyncGenerator<VoiceAgentEvent> {
 const tts = new CartesiaTTS();
 const passthrough = writableIterator<VoiceAgentEvent>();

 // Producer: read upstream events and send text to Cartesia
 const producer = (async () => {
 try {
 for await (const event of eventStream) {
 passthrough.push(event);
 if (event.type === "agent_chunk") {
 await tts.sendText(event.text);
 }
 }
 } catch {
 // ...
 }
 })();
}

```



```

 }
 } finally {
 await tts.close();
 }
})();

// Consumer: receive audio from Cartesia
const consumer = (async () => {
 for await (const event of tts.receiveEvents()) {
 passthrough.push(event);
 }
})();

try {
 // Yield events from both producer and consumer
 yield* passthrough;
} finally {
 await Promise.all([producer, consumer]);
}
}

```

The application implements a Cartesia client to manage the WebSocket connection and audio streaming. See below for implementations; similar adapters can be constructed for other TTS providers.

## Cartesia Client

### python

```

import base64
import json
import websockets

class CartesiaTTS:
 def __init__(
 self,
 api_key: Optional[str] = None,
 voice_id: str = "f6ff7c0c-e396-40a9-a70b-f7607edb6937",
 model_id: str = "sonic-3",
 sample_rate: int = 24000,
 encoding: str = "pcm_s16le",
):
 self.api_key = api_key or os.getenv("CARTESIA_API_KEY")
 self.voice_id = voice_id
 self.model_id = model_id
 self.sample_rate = sample_rate
 self.encoding = encoding
 self._ws: WebSocketClientProtocol | None = None

```

```

def _generate_context_id(self) -> str:
 """Generate a valid context_id for Cartesia."""
 timestamp = int(time.time() * 1000)
 counter = self._context_counter
 self._context_counter += 1
 return f"ctx_{timestamp}_{counter}"

async def send_text(self, text: str | None) -> None:
 """Send text to Cartesia for synthesis."""
 if not text or not text.strip():
 return

ws = await self._ensure_connection()
payload = {
 "model_id": self.model_id,
 "transcript": text,
 "voice": {
 "mode": "id",
 "id": self.voice_id,
 },
 "output_format": {
 "container": "raw",
 "encoding": self.encoding,
 "sample_rate": self.sample_rate,
 },
 "language": self.language,
 "context_id": self._generate_context_id(),
}
await ws.send(json.dumps(payload))

async def receive_events(self) -> AsyncIterator[TTSChunkEvent]:
 """Yield audio chunks as they arrive from Cartesia."""
 async for raw_message in self._ws:
 message = json.loads(raw_message)

Decode and yield audio chunks
if "data" in message and message["data"]:
 audio_chunk = base64.b64decode(message["data"])
 if audio_chunk:
 yield TTSChunkEvent.create(audio_chunk)

async def _ensure_connection(self) -> WebSocketClientProtocol:
 """Establish WebSocket connection if not already connected."""
 if self._ws is None:
 url = (
 f"wss://api.cartesia.ai/tts/websocket"
 f"?api_key={self.api_key}&cartesia_version={self.cartesia_version}"
)
 self._ws = await websockets.connect(url)

```

```
return self._ws
```

js

```
export class CartesiaTTS {
 protected _bufferIterator = writableIterator<VoiceAgentEvent.TTSEvent>();
 protected _connectionPromise: Promise<WebSocket> | null = null;

 async sendText(text: string | null): Promise<void> {
 if (!text || !text.trim()) return;

 const conn = await this._connection;
 const payload = { text, try_trigger_generation: false };
 conn.send(JSON.stringify(payload));
 }

 async *receiveEvents(): AsyncGenerator<VoiceAgentEvent.TTSEvent> {
 yield* this._bufferIterator;
 }

 protected _generateContextId(): string {
 const timestamp = Date.now();
 const counter = this._contextCounter++;
 return `ctx_${timestamp}_${counter}`;
 }

 protected get _connection(): Promise<WebSocket> {
 if (this._connectionPromise) return this._connectionPromise;

 this._connectionPromise = new Promise((resolve, reject) => {
 const params = new URLSearchParams({
 api_key: this.apiKey,
 cartesia_version: this.cartesiaVersion,
 });
 const url = `wss://api.cartesia.ai/tts/websocket?${params.toString()}`;
 const ws = new WebSocket(url);

 ws.on("open", () => {
 resolve(ws);
 });

 ws.on("message", (data: WebSocket.RawData) => {
 const message: CartesiaTTSResponse = JSON.parse(data.toString());
 if (message.data) {
 this._bufferIterator.push({
 type: "tts_chunk",
 audio: message.data,
 ts: Date.now(),
 });
 }
 });
 });
 }
}
```

```

 });
 } else if (message.error) {
 throw new Error(`Cartesia error: ${message.error}`);
 }
 });
});

 return this._connectionPromise;
}
}

```

## LangSmith

Many of the applications you build with LangChain will contain multiple steps with multiple invocations of LLM calls. As these applications get more complex, it becomes crucial to be able to inspect what exactly is going on inside your chain or agent. The best way to do this is with [LangSmith](#).

After you sign up at the link above, make sure to set your environment variables to start logging traces:

```

export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."

```

### python

Or, set them in Python:

```

import getpass
import os

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()

```

## Putting it all together

The complete pipeline chains the three stages together:

### python

```

from langchain_core.runnables import RunnableGenerator

pipeline = (
 RunnableGenerator(stt_stream) # Audio → STT events
 | RunnableGenerator(agent_stream) # STT events → Agent events

```

```

 | RunnableGenerator(tts_stream) # Agent events → TTS audio
)

Use in WebSocket endpoint
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
 await websocket.accept()

 async def websocket_audio_stream():
 """Yield audio bytes from WebSocket."""
 while True:
 data = await websocket.receive_bytes()
 yield data

 # Transform audio through pipeline
 output_stream = pipeline.atransform(websocket_audio_stream())

 # Send TTS audio back to client
 async for event in output_stream:
 if event.type == "tts_chunk":
 await websocket.send_bytes(event.audio)

```

We use [RunnableGenerators](#) to compose each step of the pipeline. This is an abstraction LangChain uses internally to manage [streaming across components](#).

js

```

// using https://hono.dev/
app.get("/ws", upgradeWebSocket(async () => {
 const inputStream = writableIterator<Uint8Array>();

 // Chain the three stages
 const transcriptEventStream = sttStream(inputStream);
 const agentEventStream = agentStream(transcriptEventStream);
 const outputEventStream = ttsStream(agentEventStream);

 // Process pipeline and send TTS audio to client
 const flushPromise = (async () => {
 for await (const event of outputEventStream) {
 if (event.type === "tts_chunk") {
 currentSocket?.send(event.audio);
 }
 }
 })();

 return {
 onMessage(event) {
 // Push incoming audio into pipeline
 const data = event.data;

```

```
 if (Buffer.isBuffer(data)) {
 inputStream.push(new Uint8Array(data));
 }
 },
 async onClose() {
 inputStream.cancel();
 await flushPromise;
 },
};
}));
```

Each stage processes events independently and concurrently: audio transcription begins as soon as audio arrives, the agent starts reasoning as soon as a transcript is available, and speech synthesis begins as soon as agent text is generated. This architecture can achieve sub-700ms latency to support natural conversation.

For more on building agents with LangChain, see the [Agents guide](#).

## langchain / errors

# INVALID\_PROMPT\_INPUT

langchain/errors/INVALID\_PROMPT\_INPUT.mdx

Occurs when a [prompt template](#) received missing or invalid input variables.

**js**

One unexpected way this can occur is if you add a JSON object directly into a prompt template:

```
import { PromptTemplate } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";

const prompt = PromptTemplate.fromTemplate(`You are a helpful assistant.

Here is an example of how you should respond:

{
 "firstName": "John",
 "lastName": "Doe",
 "age": 21
}

Now, answer the following question:

{question}`);
```

You might think that the above prompt template should require a single input key named question, but the JSON object will be interpreted as an additional variable because the curly braces ( { } ) are not escaped, and should be preceded by a second brace instead, like this:

```
import { PromptTemplate } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";

const prompt = PromptTemplate.fromTemplate(`You are a helpful assistant.

Here is an example of how you should respond:

{{
 "firstName": "John",
 "lastName": "Doe",
 "age": 21
}}

Now, answer the following question:
```



```
{question}`);
```

## Troubleshooting

---

To resolve this error, you can:

1. Review your prompt template for correctness. When using f-string formats, ensure proper escaping of curly braces: \* Use `{{` for single braces in f-strings \* Use `{{{{` for double braces in f-strings
2. When using `MessagesPlaceholder` components, confirm you're passing message arrays or message-like objects. If using shorthand tuples, wrap variable names in curly braces like `["placeholder", "{messages}"]`
3. Debug by examining actual inputs to your prompt template by using [LangSmith](#) or logging to verify they match expectations
4. If sourcing prompts from LangChain [Prompt Hub](#), isolate and test the prompt with sample inputs to ensure it functions as intended

# INVALID\_TOOL\_RESULTS

langchain/errors/INVALID\_TOOL\_RESULTS.mdx

## 참고

Currently only used in `langchainjs` (JavaScript/TypeScript).

This error occurs when passing mismatched, insufficient, or excessive `@[ ToolMessage ]` objects to a model during tool calling operations.

The error stems from a fundamental requirement: an assistant message with `tool_calls` must be followed by tool messages responding to each `tool_call_id`.

When a model returns an `@[ AIMessage ]` with tool calls, you must provide exactly one corresponding `@[ ToolMessage ]` for each tool call, with matching `tool_call_id` values.

## Common causes

- **Insufficient responses:** If a model requests two tool executions but you only provide one response message, the model rejects the incomplete message chain
- **Duplicate responses:** Providing multiple `@[ ToolMessage ]` objects for the same tool call ID results in rejection, as does having unmatched IDs
- **Orphaned tool messages:** Sending a `@[ ToolMessage ]` without a preceding `@[ AIMessage ]` containing tool calls violates protocol requirements

## python

Here's an example of a problematic pattern:

```
Model requests two tool calls
response_message.tool_calls # Returns 2 calls

But only one ToolMessage provided
chat_history.append(ToolMessage(
 content=str(tool_response),
 tool_call_id=tool_call.get("id")
))

model_with_tools.invoke(chat_history)
```

## js

Here's an example of a problematic pattern:

```
// Model requests two tool calls
responseMessage.tool_calls // Returns 2 calls

// But only one ToolMessage provided
chatHistory.push({
 role: "tool",
 content: toolResponse,
 tool_call_id: responseMessage.tool_calls[0].id
});

await modelWithTools.invoke(chatHistory); // Fails with INVALID_TOOL_RESULTS
```

## Troubleshooting

---

To resolve this error:

- **Count matching pairs:** Ensure one `@[ ToolMessage ]` exists per tool call in the preceding `@[ AIMessage ]`
- **Verify IDs:** Confirm each `ToolMessage.tool_call_id` matches an actual tool call identifier

# MESSAGE\_COERCION\_FAILURE

langchain/errors/MESSAGE\_COERCION\_FAILURE.mdx

This error occurs when message objects don't conform to the expected format.

python

## Accepted message formats

LangChain modules accept `MessageLikeRepresentation`, which is defined as:

```
from typing import Union

from langchain_core.prompts.chat import (
 BaseChatPromptTemplate,
 BaseMessage,
 BaseMessagePromptTemplate,
)

MessageLikeRepresentation = Union[
 Union[BaseMessagePromptTemplate, BaseMessage, BaseChatPromptTemplate],
 tuple[
 Union[str, type],
 Union[str, list[dict], list[object]],
],
 str,
]
```

These include OpenAI style message objects (`{ role: "user", content: "Hello world!" }`), tuples, and plain strings (which are converted to `@[HumanMessage]` objects).

If a module receives a value outside of one of these formats, you will receive an error:

```
from langchain_anthropic import ChatAnthropic

uncoercible_message = {"role": "HumanMessage", "random_field": "random value"}

model = ChatAnthropic(model="claude-sonnet-4-6")

model.invoke([uncoercible_message])
```

```
ValueError: Message dict must contain 'role' and 'content' keys, got {'role': 'HumanMessage', 'random_field': 'random value'}
```

js

TODO: Add JS example

## Troubleshooting

---

To resolve this error:

1. **Ensure proper format:** All inputs to chat models must be an array of LangChain message classes or a supported message-like format
2. Verify no unintended stringification or transformation occurs to your messages
3. Examine the error's stack trace and add logging statements to inspect message objects before they're passed to the model

# MODEL\_AUTHENTICATION

langchain/errors/MODEL\_AUTHENTICATION.mdx

## 참고

Currently only used in `langchainjs` (JavaScript/TypeScript).

Your model provider is denying you access to their service.

This error typically occurs when there's an issue with your authentication credentials or API keys.

## Troubleshooting

- Confirm that your API key or authentication credentials are accurate and valid.
- If using environment-based authentication, verify:
  - The variable name is spelled correctly
  - The variable contains an assigned value
  - Third-party packages like `dotenv` haven't interfered with loading
- If using a proxy or non-standard endpoint, make sure that your custom provider does not expect an alternative authentication scheme.
- Bypass environment variable issues by passing credentials explicitly:

### python

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(api_key="YOUR_KEY_HERE")
```

### js

```
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
 apiKey: "YOUR_KEY_HERE",
});
```

# MODEL\_NOT\_FOUND

---

langchain/errors/MODEL\_NOT\_FOUND.mdx

## 참고

Currently only used in `langchainjs` (JavaScript/TypeScript).

The model name you have specified is not acknowledged by your provider.

## Troubleshooting

---

To resolve this error:

1. **Verify the model identifier:** Double check the model string you are passing in.  
Ensure the spelling and format are correct
2. **Check proxy/wrapper configurations:** If you are using a proxy or other alternative host with a model wrapper, confirm that the permitted model names are not restricted or altered

The error typically stems from either a typo in the model name string itself or restrictions imposed by a proxy service or model wrapper between your code and the provider's API.

# MODEL\_RATE\_LIMIT

---

langchain/errors/MODEL\_RATE\_LIMIT.mdx

## 참고

Currently only used in `langchainjs` (JavaScript/TypeScript).

You have hit the maximum number of requests that a model provider allows over a given time period and are being temporarily blocked.

This error occurs when you exceed the maximum number of requests permitted by your model provider within a specific timeframe, resulting in temporary blocking. The restriction is generally temporary and lifts after the limit resets.

## Troubleshooting

---

To resolve this error, you can:

1. **Implement Rate Limiting:** Deploy a rate limiter to regulate the frequency of requests sent to the model.

## python

See [rate limiting](#) docs.

2. **Implement Response Caching:** Use model response caching to reduce redundant requests when incoming queries are repetitive.
3. **Use Multiple Providers:** Distribute requests across multiple providers if your application architecture supports this approach
4. **Contact Your Provider:** Reach out to your model provider requesting an increase to your rate limits



# OUTPUT\_PARSING\_FAILURE

---

langchain/errors/OUTPUT\_PARSING\_FAILURE.mdx

An [output parser](#) was unable to handle model output as expected.

## 참고

Some prebuilt constructs like legacy LangChain agents and chains may use output parsers internally, so you may see this error even if you're not visibly instantiating and using an output parser.

## Troubleshooting

---

- Consider using tool calling or other structured output techniques if possible without an output parser to reliably output parseable values.
- Add more precise formatting instructions to your prompt.
- If you are using a smaller or less capable model, try using a more capable one.

**langchain / mcp**

# Authentication

langchain/mcp/auth.mdx

## python

Most remote MCP servers require authentication. `@[MCPAdapter]` delegates auth to FastMCP, so any credential a `fastmcp.Client` accepts works: a static [bearer token](#), a full [OAuth 2.1](#) flow, or any [httpx.Auth](#). Pass the credential on a prebuilt client and hand that client to the adapter. When one agent talks to several servers, use [per-server authentication](#); in a deployment, use [per-user authentication](#) so each run reaches the server as the caller.

## 참고

The `langchain.mcp` namespace requires `langchain[mcp]>=1.4.0` and is in beta. The API may change.

## Bearer token

The simplest case: the server verifies a token you provisioned and adds it in the `Authorization: Bearer <token>` header. There is no discovery, browser, or refresh. Pass the token as the client's `auth`:

The `auth` argument accepts a bearer-token string, the literal `"oauth"`, or any `httpx.Auth`. In an `MCPConfig` fleet, each server takes the same key.

## OAuth authentication

For a server that issues its own credentials, pass the literal string `"oauth"`. FastMCP runs the full OAuth 2.1 flow: discovery, [dynamic client registration](#), the browser redirect, and the token exchange. Dynamic client registration means the client registers itself at runtime instead of you pre-provisioning a client ID:

By default tokens are held in memory, so each run repeats the browser step. Pass a prebuilt `OAuth` provider with a token store to persist them across runs:

```
from fastmcp.client import Client
from fastmcp.client.auth import OAuth

oauth = OAuth(mcp_url="https://example.com/mcp", token_storage=...)
client = Client("https://example.com/mcp", auth=oauth)
```

FastMCP ships providers for common identity providers (Auth0, WorkOS, Okta, and more); the flow the client runs is identical across them. See [OAuth authentication](#) in the FastMCP documentation.

## Per-server authentication

---

When one agent talks to several servers, each server may need its own credential. Give each its own connection with a [ClientGroup](#), and set `auth` per client, so every server authenticates independently:

## Per-user authentication

---

In a deployment, each run should reach the MCP server as the user who initiated it, not with one shared credential. The pattern has two halves:

1. **Authenticate the caller at the LangGraph server.** A [custom auth handler](#) resolves the incoming request to a user identity, which each run reads off its runtime.
2. **Mint or exchange a credential for that user.** Inside the [graph factory](#), read the user's identity and build the MCP client with a per-user token, so the connection carries that user's authorization.

In production, `token_for` stands in for whatever the deployment already has: an OAuth gateway that exchanges the session for a per-user token, or a `fastmcp.client.auth` provider that runs the authorization-code flow per identity. Isolate any [cached responses](#) per user, keyed off the verified identity, so one user never sees another's cached tool list.

## See also

---

- [FastMCP OAuth authentication](#)
- [FastMCP bearer token authentication](#)
- [FastMCP machine-to-machine authentication](#)
- [FastMCP client groups](#) — independent connections for per-server auth
- [FastMCP server authentication providers](#)
- [MCP authorization specification](#)
- [Custom authentication for a LangGraph server](#)

js

Most remote MCP servers require authentication. Each HTTP or SSE server under `servers` takes its own `authProvider` and `headers`, and `MCPAdapter` hands the provider object to the MCP SDK transport as-is and sends the headers with each request. Stdio servers take neither. Use a [header or token provider](#) for credentials your app manages,

or an [OAuth client provider](#) for OAuth 2.1. When each run should reach the server as its caller, use [per-user authentication](#).

## 참고

This page requires `@langchain/mcp-adapters` 2.0 or later. If you are upgrading from 1.x, see the [migration guide](#).

## Bearer token

For a fixed API key, send a static `Authorization` header. For a token your app issues or rotates, pass an `AuthProvider` (`{ token, onUnauthorized? }`) as `authProvider`.

The example uses an application-provided `tokenStore`. Implement `current()` to read the token and `refresh()` to save a newer token and report whether it succeeded.

```
import { MCPAdapter, UnauthorizedError } from "@langchain/mcp-adapters";

const adapter = new MCPAdapter({
 servers: {
 // A fixed API key, sent as a header.
 search: {
 url: "https://search.example.com/mcp",
 headers: { Authorization: `Bearer ${process.env.SEARCH_API_KEY}` },
 },
 // A token your app issues and rotates.
 crm: {
 url: "https://crm.example.com/mcp",
 authProvider: {
 token: async () => tokenStore.current(),
 onUnauthorized: async () => {
 // Throw when no newer token exists: over SSE, the SDK calls this
 // again every time a reconnect is rejected.
 if (!(await tokenStore.refresh())) {
 throw new UnauthorizedError("No newer token for the CRM server");
 }
 },
 },
 },
 },
});
```

- `token()` : Runs before every request and returns the current token.
- `onUnauthorized()` : Optional. Runs on a 401, before the request is retried once. Over SSE, the SDK calls it again every time a reconnect is rejected, with no cap. Throw `UnauthorizedError` when you have no newer token.

Without `onUnauthorized`, a 401 fails the connection with the SDK's `UnauthorizedError`. See [Handle authentication errors](#).

Each named server opens its own connection, so servers in one adapter authenticate independently, as the two servers in this example do.

The adapter re-exports the `AuthProvider` and `OAuthClientProvider` types, so you can type a provider without importing the SDK.

## OAuth authentication

OAuth lets users authorize your application to access an MCP server. Pass an `OAuthClientProvider` as `authProvider`. The SDK handles discovery, client registration, the code exchange, and token refresh. Your application supplies the provider's storage, browser redirect, and callback handler.

Add `@modelcontextprotocol/client` 2.2.0 or later as a direct dependency. The adapter depends on the same range, so your app and the adapter share one SDK copy. The callback transport, `SdkHttpError`, and `SseError` come from it. Implement your provider using the SDK's [OAuth client guide](#). Configure its `redirectUrl` for your callback route, and store client information, tokens, the PKCE verifier, and discovery state. Include `invalidateCredentials()` so rejected refresh credentials can trigger a new login.

Keep each provider and its storage separate for each user and MCP server. The example below also binds each pending login to an authenticated application session. Read the user and session IDs from your application's session middleware, never from callback query parameters.

The `createMcpSession()` helper below connects these steps:

1. Call `listTools()` to discover tools. If login is required, redirect the user's browser to the returned `authorizationUrl`.
2. In your callback route, call `completeLogin()` on the same session. It checks the caller and consumes the pending `state` before exchanging the code.
3. Use the returned tools. After the exchange, the helper retries discovery with the saved tokens.

### Session helper

This example keeps pending login state in one Node.js process. For multiple processes or restarts, use shared storage with an expiry. Check the state, user, session, and server, and consume the record in one atomic operation. The SDK does not validate `state`.

```
import { randomUUID } from "node:crypto";
import {
```

```

MCPAdapter,
MCPCliientError,
UnauthorizedError,
type OAuthClientProvider,
} from "@langchain/mcp-adapters";
import { StreamableHTTPClientTransport } from "@modelcontextprotocol/client";

function createMcpSession(
 serverUrl: string,
 userId: string,
 sessionId: string,
 authProvider: OAuthClientProvider,
) {
 let pendingState: string | undefined;
 let authorizationUrl: URL | undefined;

 authProvider.state = () => (pendingState = randomUUID());
 authProvider.redirectToAuthorization = (url) => {
 authorizationUrl = url;
 };

 const adapter = new MCPAdapter({
 servers: { crm: { url: serverUrl, authProvider } },
 });

 function causedByUnauthorized(error: unknown): boolean {
 // Discovery wraps UnauthorizedError in MCPCliientError, and wraps it
 // again when an HTTP connection falls back to SSE. Walk a few hops.
 let current: unknown = error;
 for (let depth = 0; depth <= 3; depth += 1) {
 if (UnauthorizedError.isInstance(current)) return true;
 if (!(current instanceof Error)) return false;
 current = current.cause;
 }
 return false;
 }

 async function listTools() {
 // Reuse the pending login instead of starting another redirect.
 if (authorizationUrl) return { authorizationUrl };
 try {
 return { tools: await adapter.listTools() };
 } catch (error) {
 if (
 MCPCliientError.isInstance(error) &&
 causedByUnauthorized(error) &&
 authorizationUrl
) {
 return { authorizationUrl };
 }
 throw error;
 }
 }
}

```

```

 }
}

async function completeLogin(
 callbackUrl: URL,
 callbackUserId: string,
 callbackSessionId: string,
) {
 const params = callbackUrl.searchParams;
 if (
 callbackUserId !== userId ||
 callbackSessionId !== sessionId ||
 !pendingState ||
 params.get("state") !== pendingState
) {
 throw new Error("Invalid OAuth callback");
 }
 // No await between checking and consuming state: only one callback wins.
 pendingState = undefined;

 const transport = new StreamableHTTPClientTransport(new URL(serverUrl), {
 authProvider,
 });
 try {
 await transport.finishAuth(params);
 } catch {
 // SDK errors can contain text supplied by the authorization server.
 throw new Error("OAuth login failed");
 } finally {
 authorizationUrl = undefined;
 await transport.close();
 }
 return listTools();
}

return { listTools, completeLogin, close: () => adapter.close() };
}

```

To start discovery, pass your provider and the authenticated caller's identity:

```

const crm = createMcpSession(
 "https://crm.example.com/mcp",
 userId,
 sessionId,
 authProvider,
);
const result = await crm.listTools();
// If result.authorizationUrl is present, redirect the browser there.
// Otherwise, pass result.tools to your agent.

```



Keep `crm` available to that application's session until the callback completes. Do not share its provider with another session.

## Complete the redirect

After the user signs in, the authorization server redirects to your provider's `redirectUrl`. In that route, retrieve the same `crm` instance and the caller's verified user and session IDs, then complete login:

```
const result = await crm.completeLogin(new URL(req.url, "http://localhost"), userI
d, sessionId);
// The helper exchanges the code and retrieves discovery using saved tokens.
// Pass result.tools to your agent, or redirect if another login is required.
```

The helper passes the full query to `finishAuth()` so the SDK can validate the authorization server's `iss` parameter. It rejects missing, mismatched, or already consumed state before the exchange. Keep the adapter open while using its tools, then call `await crm.close()` when the application session ends.

For a server configured with `transport: "sse"`, use `SSEClientTransport` in the callback helper. For machine-to-machine access without a user, use the SDK's `ClientCredentialsProvider` instead of a browser login.

## Handle authentication errors

A connection rejected for credentials throws an `MCPClientError`. The SDK error sits one level down its `cause` chain, or two when an HTTP connection falls back to SSE. This example walks a few levels and handles HTTP authentication failures with either of these causes:

- **UnauthorizedError**: The SDK cannot recover on its own. An OAuth login is needed, or a token provider has no `onUnauthorized`.
- **An HTTP 401** (an `SdkHttpError` with `status: 401`, or an `SseError` with `code: 401` over SSE): The server rejects the credentials, or none were provided.

```
import { MCPClientError, UnauthorizedError } from "@langchain/mcp-adapters";
import { SdkHttpError, SseError } from "@modelcontextprotocol/client";

function authFailure(error: unknown): "login" | "rejected" | undefined {
 if (!MCPClientError.isInstance(error)) return undefined;

 let current: unknown = error;
 for (let depth = 0; depth <= 3; depth += 1) {
 if (UnauthorizedError.isInstance(current)) return "login";
 if (SdkHttpError.isInstance(current) && current.status === 401)
 return "rejected";
 }
```

```

 if (SseError.isInstance(current) && current.code === 401)
 return "rejected";
 if (!(current instanceof Error)) return undefined;
 current = current.cause;
}
return undefined;
}

```

The adapter re-exports `UnauthorizedError`. Import `SdkHttpError` and `SseError` from `@modelcontextprotocol/client`. An `MCPClientError` message includes the SDK's error text, so log it rather than show it to users.

When a server rejects the credentials during a tool call, the tool fails with a `ToolException` instead. Under `createAgent`, the model sees it as an error message and the run continues.

Discovery retries failures caused by `UnauthorizedError` or HTTP 401 on the next call, even when `onConnectionError` skips failed servers. A later login can recover without recreating the adapter. Other errors from `token()` or `onUnauthorized()`, such as a failed token-store request, do not count as authentication failures. When `onConnectionError` skips a connection after one of these errors, that connection stays blocked until `close()` clears it.

## Header precedence

Once a provider has a token, it replaces a configured `Authorization` header. Until then, the header is sent, so a static API key can fall back to OAuth.

The SDK also forwards a static `Authorization` header to the authorization server's discovery, registration, and token endpoints, not only to the MCP server. Do not pair a secret API key with an OAuth provider whose authorization server lives on another origin.

## Per-user authentication

In a deployment, each run should reach the MCP server as the user who started it, not with one shared credential. Pass `authProvider` or `headers` in the options of `listTools()`, `listToolsets()`, `getClient()`, or the resource methods (`listResources()`, `listResourceTemplates()`, and `readResource()`):

```

import { MCPAdapter, type AuthProvider } from "@langchain/mcp-adapters";

// A single-server adapter, because each user brings a different credential.
const crm = new MCPAdapter({
 servers: { crm: { url: "https://crm.example.com/mcp" } },
});

```

```
// Reuse one provider object per user: each object opens its own connection.
const providers = new Map<string, AuthProvider>();

function providerFor(userId: string): AuthProvider {
 let provider = providers.get(userId);
 if (!provider) {
 provider = { token: async () => tokenFor(userId) };
 providers.set(userId, provider);
 }
 return provider;
}

const tools = await crm.listTools(["crm"], {
 authProvider: providerFor(userId),
});
```

In this example, implement `tokenFor(userId)` to retrieve a token for the authenticated user.

The returned tools call the server through that user's connection. These rules apply to method-level authentication:

- **Adapter-wide override:** An `authProvider` or `headers` in a method's options applies to every HTTP and SSE server the adapter holds, not only the one you name. Use a single-server adapter when different users need different credentials.
- **Header precedence:** Server-configured headers take precedence over same-named method-level headers, regardless of capitalization. When passing a per-user `Authorization` header, omit that header from the server configuration. A provider token still takes precedence over either header.
- **One connection per provider object:** Each distinct provider object gets its own connection, kept until `close()`. Reuse one provider object per user rather than creating one per call.
- **Isolated catalogs:** Catalogs are keyed by provider object and headers, so one user never receives another user's cached tool list. Recreate the adapter when the account behind the same provider object changes.

Resolve the user from the incoming request, for example with a [custom auth handler](#) on a LangGraph server.

## See also

- [Migrate to @langchain/mcp-adapters 2.0](#)
- [MCP authorization specification](#)
- [MCP TypeScript SDK OAuth client guide](#)

- [Custom authentication for a LangGraph server](#)

# Connections

langchain/mcp/connections.mdx

## python

### 참고

The `langchain.mcp` namespace requires `langchain[mcp]>=1.4.0` and is in beta. The API may change.

`@[MCPAdapter]` opens an MCP connection, discovers tools, and returns LangChain tools your agent can call. How you pass servers in, and how long you keep the adapter open, depends on the shape of your app. The connection itself is FastMCP's; this page covers the LangChain patterns and links out to FastMCP for transport and client details.

Prefer the [default lifecycle](#) unless you are connecting to several servers or deploying many concurrent runs. For what a single target can be (URL, script path, in-process server), see [Transports](#).

## js

`@[MCPAdapter]` discovers MCP tools and keeps their connections open so your agent can reuse them across calls. Discover tools with `listTools()` or `listToolsets()`, and close the adapter when your application finishes using them.

Choose how long to keep the adapter open:

Situation	Pattern	Go to
Script or agent invocation	Create the adapter, run the agent, then close in <code>finally</code>	<a href="#">Connection lifecycle</a>
Long-lived worker	Reuse one adapter and close it during shutdown	<a href="#">Scale a deployment</a>

See [Transports](#) for each server's connection options.

## python

### Choose a pattern

Pick one row from each table. Server shape and connection lifetime are independent; any shape works with any lifetime.

## Server shape

If you need...	Use	Go to
One server	A URL, Path , or in-process target	<a href="#">Transport s</a>
Several servers behind one connection	An MCPConfig dict	<a href="#">MCPConfi g</a>
Several servers with separate auth, protocol eras, or pools	A <a href="#">ClientGroup</a>	<a href="#">ClientGro up</a>

## Connection lifetime

Situation	Pattern	Go to
Script, notebook, or most agents	Discover inside <code>async with</code> , then exit	<a href="#">Connection lifecycle</a>
Hold one session across several tool calls in a run	Keep the adapter open around the agent call	<a href="#">One session per invocation</a>
Many concurrent runs in a deployment	Discover per run; reuse a <a href="#">shared pool</a> and <a href="#">cache</a>	<a href="#">Scale a deployment</a>

# Connection lifecycle

## python

`@[ MCPAdapter ]` is an `async` context manager. Entering it connects the underlying client; exiting it releases the connection. Discovery happens inside the context, but the tools it returns hold the client, so they stay callable after the context exits.

To discover tools and build an agent (the default pattern):

You do not need to keep the adapter open for the life of the agent. Prefer this pattern unless you are [holding a session open](#) or [scaling a deployment](#).

## One session per invocation

The tools `@[ MCPAdapter ]` returns are reentrant: each time a tool is invoked, it opens the client, runs the MCP call, and releases it, whether or not a connection is already held elsewhere. A single agent run therefore opens one session per tool call and closes it when the call returns, rather than holding a session open across the whole run. This keeps a long-running agent from pinning an idle connection between tool calls, and it is why the tools stay callable after the discovery context exits.

If you want a session held open across several calls, keep the adapter's context open when you call the agent. The reentrant client reuses the existing connection rather

than opening a second one.

## js

Constructing an adapter validates its configuration without opening connections.

`listTools()`, `listToolsets()`, `getClient()`, and the resource methods connect when they run discovery. Keep the adapter open while your agent uses those tools.

To discover tools, run an agent, and release the connections, pass your MCP server's HTTP URL to `runAgent`:

`close()` aborts in-flight adapter work, closes its connections, and clears its caches. You can call `listTools()` again to open fresh connections, but use the newly returned tools. Previously returned tools retain their closed clients.

## Multiple servers

### python

To give one agent tools from several servers, choose `MCPCConfig` when a single aggregate connection is enough, or `ClientGroup` when each server needs its own connection (different [protocol eras](#), [per-server authentication](#), or a [shared pool](#) configured per client).

### One aggregate connection with `MCPCConfig`

Give the adapter an `MCPCConfig` dict to connect to several servers behind one aggregate endpoint. FastMCP prefixes every tool with its config key, so two servers exposing the same tool name stay distinguishable in the list handed to a model:

Each backend is addressed independently, so a fleet can mix transports: one server over stdio, another over HTTP. An `MCPCConfig` fleet shares a single negotiated [protocol era](#) across every backend, though: add a legacy-only server and the whole fleet drops to the legacy era.

### Independent connections with `ClientGroup`

To keep each server on its own connection, pass a [ClientGroup](#). Each member keeps its own negotiated protocol era, authentication, and handlers, and the group routes each call back to the client that advertised the tool. This is what lets a legacy and a modern server run side by side, and it namespaces tools the same way so identical tool names across servers never collide:

js

Each named entry in `servers` gets its own connection, authentication, and [protocol mode](#). One adapter can connect to both HTTP and stdio servers.

To discover tools from both transports, pass a calendar server's HTTP URL and a files server's script path to `listServerTools`:

`listToolsets()` groups tools by server name. `listTools()` returns one array; pass a server name or an array of names to select which tools it returns. Selection filters the result, but discovery still contacts every configured server. A failure on an unselected server can fail the call.

The adapter prefixes tool names with their server name by default, such as `calendar_search` and `files_search`. Set `prefixToolNameWithServerName: false` to keep raw names. `listTools()` throws if its selected tools contain duplicate names.

OpenAI and Anthropic accept only letters, digits, `_`, and `-` in tool names (`^[a-zA-Z0-9_-]+$`), up to 64 characters for OpenAI and 128 for Anthropic. The adapter does not rename a prefixed name that breaks these limits, so keep server names short and free of dots and spaces.

Set `defaultToolTimeout` (in milliseconds) at the top level to apply it to every server's tools; it wins over a server's own `defaultToolTimeout`.

## Scale a deployment

python

A deployment that serves many runs should discover per run but reuse its connections underneath, rather than reconnecting on every request. Build the agent inside a [langgraph dev](#) graph factory so each run picks up the current tool catalog, and let a [shared connection pool](#) and [response cache](#) absorb the cost:

### 참고

In a `langgraph dev` graph factory, the annotated parameter types and the return type must be importable at runtime, not only under `TYPE_CHECKING`. `langgraph-api` classifies the factory with `typing.get_type_hints()`; if an annotation cannot resolve, it injects a config dict instead of the runtime.

For a fully worked deployment example, including per-user authentication that mints a token for each caller, see [Authentication](#).



## Shared connection pool

By default each FastMCP client manages its own HTTP connections. Across a fleet of servers, or many concurrent runs, that means many independent pools. To share one pool, pass an `httpx_client_factory` that draws from a single transport, and lend it out without letting any one client close it:

Because every client borrows from `_POOL`, the deployment opens one set of HTTP connections for the whole fleet rather than one per server.

### js

Create the adapter at module scope, outside the graph factory. Each factory call can discover the current tool catalog and build an agent using the shared connections:

Keep the adapter open across runs. Close it during application shutdown, after active runs finish. For runs with different user credentials, see [Per-user authentication](#).

Discovery throws on connection failures by default. Set `onConnectionError: "ignore"`, or provide a callback that returns normally, to continue with the remaining servers. Non-authentication failures keep that connection skipped on later discoveries, even with `cacheMode: "refresh"`. Close the adapter to clear skipped connections before discovering again. Authentication failures are retried on the next discovery.

Stdio servers support `restart` settings, and HTTP and SSE servers support `reconnect` settings when configured with `mode: "legacy"`. After a restart, call `listTools()` again and use the returned tools; tools returned before the restart keep the closed connection.

Modern MCP servers keep no session, so their replicas can run behind any load balancer. A sessionful legacy server still needs sticky routing to one replica. A modern server that runs several replicas must share one `requestState` signing key across them, or an elicitation answer that reaches a different replica is rejected. See [Protect requestState with the codec](#) in the MCP TypeScript SDK documentation.

## Caching

### python

FastMCP can cache the result of `list_tools` so repeated discovery avoids a network round trip. Caching is opt-in and honors the server's own cache hints, so it only takes effect against modern-era servers that advertise them.

`list_tools()` accepts a `cache_mode` that selects how discovery reads a configured cache:

- **use** (the default): serve a cached tool list when one is present and still within the server's TTL hint, otherwise fetch and store.
- **refresh**: fetch a fresh list from the server and repopulate the cache.
- **bypass**: skip the cache entirely.

```
tools = await adapter.list_tools(cache_mode="refresh")
```

The cache and its per-principal isolation are configured on the client itself, with `Client(cache=...)`. For a shared store across a fleet of replicas, or partitioning cached responses per user, see [Response caching](#) in the FastMCP documentation.

## js

The MCP SDK provides an in-memory cache for tool lists. Repeated discovery avoids a network request while the server's `ttlMs` hint remains valid. Without a positive `ttlMs`, discovery fetches the list again.

Use `cacheMode` to control how `listTools()` and `listToolsets()` read the response cache:

- **"use"** (the default): Serve a valid cached tool list when one is available, otherwise fetch and store it.
- **"refresh"**: Fetch a fresh tool list and update the cache.
- **"bypass"**: Fetch a fresh tool list without reading or updating the response cache.

To refresh tools from all configured servers, pass an empty server-selection array and the discovery options:

```
const tools = await adapter.listTools([], { cacheMode: "refresh" });
```

The adapter also reuses converted LangChain tool wrappers when the server descriptors have not changed. Rediscovery returns the current tools; it does not update an existing agent's tool list. Build the next agent with the returned tools.

## Protocol eras

### python

MCP changed how a client and server agree on what each supports. The **legacy** era begins every connection with an `initialize` handshake; the **modern** era (protocol

version `2026-07-28` and later) discovers support by probing the server's `server/discover` endpoint. FastMCP negotiates the era per connection, so nothing on the LangChain side has to know which a given server speaks.

To hold tools from servers on different eras in one agent, give each its own connection so it keeps the best era its server supports, either through a [ClientGroup](#) or one adapter per server. A prebuilt `fastmcp.Client` selects the era with its `mode` parameter:

Passing both servers as a single `MCPConfig` fleet instead would negotiate one era for everything the fleet holds, dropping every server to the oldest era any member requires.

For the full negotiation rules, see [Protocol negotiation](#) in the FastMCP documentation.

## js

The adapter negotiates the MCP protocol separately for each server. The default `mode`: `"auto"` supports modern and legacy servers in the same adapter.

The legacy era starts with an `initialize` handshake. The modern era uses `server/discover` to discover server capabilities. Override `mode` when you need to require a specific era:

Each server's `mode` controls its negotiation independently:

- `"auto"` (the default): Try the current protocol and fall back for a legacy-only server.
- `"modern"`: Require protocol version `2026-07-28` and fail instead of using legacy MCP.
- `"legacy"`: Skip modern probing and use the legacy client interface.

In `"auto"` mode, a URL server that answers the Streamable HTTP request with 404 or 405 is retried over SSE, at the same URL and then with a trailing `/mcp` replaced by `/sse`. In `"legacy"` mode, any 4xx falls back unless `automaticSSEFallback: false`.

`setLogLevel()` is legacy-only and throws when a connected server negotiated the modern protocol; set `logLevel` on each modern server instead.

## See also

- [Authentication](#): Bearer, OAuth, and per-user credentials.

## python

- [Deploy a LangGraph server](#): Graph factories for long-lived deployments.

- [FastMCP connection lifecycle](#)
- [MCP configuration format](#)

js

- [Human-in-the-loop](#): Pause and resume tool calls.
- [MCP specification](#)

# Model Context Protocol (MCP)

langchain/mcp/index.mdx

## python

[Model Context Protocol \(MCP\)](#) is an open protocol that standardizes how applications provide tools and context to language models. LangChain agents call tools defined on MCP servers through `@[MCPAdapter]`, which discovers a server's tools and adapts them into LangChain tools you can pass straight to `@[create_agent]`.

`@[MCPAdapter]` is built on [FastMCP](#), which handles transport inference, protocol negotiation, connection management, and authentication. This section covers the LangChain-specific layer and links out to the FastMCP client documentation for the connection details underneath.

## 참고

The `langchain.mcp` namespace requires `langchain[mcp]>=1.4.0` and is in beta. Importing from it raises a `LangChainBetaWarning` once per process. The API may change.

If you used MCP before v1.4.0, see [Migrate from langchain-mcp-adapters](#).

## js

[Model Context Protocol \(MCP\)](#) is an open protocol that standardizes how applications provide tools and context to language models. LangChain agents call tools defined on MCP servers through `@[MCPAdapter]`, which discovers a server's tools and adapts them into LangChain tools you can pass straight to `@[createAgent]`.

## 참고

This guide requires `@langchain/mcp-adapters 2.0` or later. If you use 1.x, see the [1.x docs](#) or the [migration guide](#).

## Install

## python

Install LangChain with the `mcp` extra, which pulls in FastMCP:

```
```bash pip pip install "langchain[mcp]"
```

```
```bash uv  
uv add "langchain[mcp]"
```

## js

Install the `@langchain/mcp-adapters` package and its peer dependencies:

```
```bash npm npm install @langchain/mcp-adapters@^2.0.0 @langchain/core  
@langchain/langgraph
```

```
```bash pnpm  
pnpm add @langchain/mcp-adapters@^2.0.0 @langchain/core @langchain/langgraph
```

```
```bash yarn yarn add @langchain/mcp-adapters@^2.0.0 @langchain/core  
@langchain/langgraph
```

```
```bash bun  
bun add @langchain/mcp-adapters@^2.0.0 @langchain/core @langchain/langgraph
```

Add `@modelcontextprotocol/client` too if you import SDK types or transports.

## Quickstart

### python

Open an `@[MCPAdapter]`, discover the server's tools with `list_tools()`, and build the agent inside the context. The tools hold the client, so the agent stays usable after the context exits:

### js

Connect an agent to the [LangChain docs MCP server](#) to search the documentation. This example requires `langchain`, `@langchain/anthropic`, and an Anthropic API key. See the [LangChain quickstart](#) for setup.

Keep the adapter open while the agent runs, then close it in `finally`:

### LangChain docs MCP server

The [LangChain docs MCP server](https://docs.langchain.com/mcp) is a public HTTP endpoint at `https://docs.langchain.com/mcp`.

Connect an agent to it to search and read documentation without writing custom tools:

python

js

참고

The docs MCP server is public and does not require an API key. For IDE and coding-agent setup (Claude Code, Cursor, and others), see [Use docs programmatically](#).

The server exposes these tools:

Tool	Description
<code>search_docs_by_lang_chain</code>	Search docs for relevant guides, how-tos, and examples.
<code>query_docs_filesystem_docs_by_lang_chain</code>	Read or search docs through a virtual filesystem ( <code>rg</code> , <code>head</code> , <code>cat</code> , and related commands).
<code>submit_feedback</code>	Report a problem with a documentation page.

js

`MCPAdapter` exposes these tools with the server-name prefix, for example `docs_search_docs_by_lang_chain`.

## Transports

python

`@[MCPAdapter]` infers the transport from the target you provide. The target is the only difference between an in-process server, a local script over stdio, and a remote URL:

A target can be any of the following:

- An `http / https URL ( str )`: reached over Streamable HTTP.
- A `script path ( Path )`: launched as a subprocess over stdio.
- A `transport object ( StreamableTransport )`: a preconfigured transport object. See [Client Transports](#).
- An `in-process FastMCP server`: connected in memory, without a subprocess or socket.

- An `MCPConfig dict` ( `{"mcpServers": {...}}` ): several servers behind one adapter. See [Connections](#).
- A **prebuilt** `fastmcp.Client` : for full control over transport, [caching](#), and [protocol negotiation](#).

### 주의

A `str` `target` must be an `http` or `https` URL. FastMCP resolves a string by testing it as a filesystem path before testing it as a URL, so a string naming an existing `.py` or `.js` file would launch that file as a subprocess. Because strings are the form a target most often arrives in from configuration or from a model, `@[MCPAdapter]` rejects strings that do not match the shape of a URL.

### js

`@[MCPAdapter]` accepts a map of named server definitions. It selects the transport from each definition, so one adapter can connect to local stdio servers and remote HTTP servers at the same time:

Each server definition can use one of these transports:

- **stdio**: Provide `command` and `args` . The adapter starts the command as a child process and communicates over standard input and output.
- **Streamable HTTP**: Provide `url` . This is the default transport for URL-based servers, so `transport: "http"` is optional.
- **SSE**: Provide `url` and set `transport: "sse"` . Use this transport only for legacy servers that expose an SSE endpoint.

Transport selection and protocol negotiation are separate. The optional `mode` setting controls which MCP protocol era the client accepts. See [Protocol eras](#).

### 참고

For new remote servers, omit both `transport` and `mode` . The adapter uses Streamable HTTP with automatic protocol negotiation. Specify these settings only when connecting to a server with known legacy requirements.

## Next steps

### [Tools](#)

Load MCP tools into agents, control their execution, and handle their outputs.



## Connections

Connection lifecycle, multiple servers, protocol eras, and caching.

## Authentication

Bearer tokens, OAuth 2.1, and per-user server auth.

# Tools

langchain/mcp/tools.mdx

## python

### 참고

The `langchain.mcp` namespace requires `langchain[mcp]>=1.4.0` and is in beta. The API may change.

`@[MCPAdapter]` bridges MCP servers and LangChain agents: it discovers the tools a server advertises and adapts them into standard LangChain tools. Pass the tools from `list_tools()` to `@[create_agent]` as you would any other LangChain tool.

## js

`@[MCPAdapter]` bridges MCP servers and LangChain agents: it discovers the tools a server advertises and adapts them into standard LangChain tools. Pass the tools from `listTools()` to `@[createAgent]` as you would any other LangChain tool.

This page covers what is specific to that bridge: identifying MCP tools, controlling their execution, handling their outputs, and responding when a server needs input during a call. For the runnable discovery-and-agent example, see the [MCP quickstart](#).

## Use MCP tools in an agent

## python

Discover the server's catalog with `@[MCPAdapter.list_tools]`, then give the returned tools to `@[create_agent]`. From the agent's perspective, they behave like LangChain tools: the model chooses a tool, LangChain invokes it, and the resulting `@[ToolMessage]` returns to the model.

## js

Discover the server's catalog with `@[MCPAdapter.listTools]`, then give the returned tools to `@[createAgent]`. From the agent's perspective, they behave like LangChain tools: the model chooses a tool, LangChain invokes it, and the resulting `@[ToolMessage]` returns to the model.

Keep the adapter open while the agent can call its tools, including when resuming an interrupted run. Call `await adapter.close()` when you finish using the agent.

For general guidance on defining, binding, and using LangChain tools, see [Tools](#). For several MCP servers and their namespaced tool catalogs, see [Connections](#).

## Handle tool outputs

MCP tool results become `@[ ToolMessage ]` objects with content the model can read, an artifact for application data, and a status indicating success or failure.

### Multimodal content

The adapter converts model-visible MCP content into LangChain [content blocks](#). For example, a tool that returns an MCP image block with a screenshot reaches the model as an `image` block.

**python**

**js**

### Structured content

When a tool returns structured content, the adapter attaches it to the `@[ ToolMessage ]` as an artifact rather than folding it into the model-visible text. Run the agent, then read the `artifact` from the `@[ ToolMessage ]` instances in the result:

**python**

The artifact is an `MCPToolArtifact`, whose `structured_content` field holds the tool result's `structuredContent`. A tool that returns no structured content leaves `artifact` as `None`.

**js**

The `artifact` is an array of MCP result entries. Structured content appears in the entry with `type: "mcp_structured_content"`, and its `data` field holds the tool result's `structuredContent`.

### Errors

**python**

An MCP tool result carries an `isError` flag. When a server reports `isError=True`, the adapter converts it into a `@[ToolMessage]` with `status="error"` carrying the server's own message, so the agent can read it and correct itself:

A server-reported error reaches the model as a failed tool message, but a transport or session failure raises instead.

## js

When a server returns `isError: true` during an agent tool call, the adapter returns a `@[ToolMessage]` with `status: "error"` and the server's message. The model can read the error and try again.

The adapter throws transport failures. When a tool runs inside `createAgent`, the agent's default error handling turns those failures into error tool messages too. The message's `status` alone does not distinguish the cause:

If you invoke an adapted tool directly with plain arguments, a server result with `isError: true` throws a `ToolException` instead, with the MCP result in `error.result`.

## Tool metadata

### python

Each adapted tool may carry its MCP provenance under an `mcp` namespace on the LangChain tool's metadata:

```
tool.metadata
{
"mcp": {
"tool": {
"annotations": {
"destructive_hint": True,
"read_only_hint": False,
},
"_meta": {"origin": "crm"},
},
"server": {
"name": "crm",
"version": "2.1.0",
},
},
}
```

js

The adapter stores MCP tool annotations in `tool.metadata.annotations`:

```
tool.metadata;
// {
// annotations: {
// destructiveHint: true,
// readOnlyHint: false,
// },
// }
```

python

Every nested field is optional: a server may provide tool annotations, `_meta`, server identity, any combination of those, or none. `annotations` contains MCP hints such as `read_only_hint` and `destructive_hint`; `_meta` is opaque metadata supplied by the server; and `server` identifies the MCP implementation that advertised the tool.

js

`annotations` holds the server's MCP tool annotations, such as `readOnlyHint` and `destructiveHint`. A server may omit them, so the field can be `undefined`.

Read optional metadata defensively, so a missing field returns a default rather than failing:

python

js

This example and the [human-in-the-loop example](#) import types from `@modelcontextprotocol/client`; add it as a direct dependency.

## Human-in-the-loop

Reading annotations lets you gate a tool based on what the server declares about it, rather than hardcoding tool names. The MCP annotation classifies the tool and LangChain's human-in-the-loop middleware enforces the approval policy.

Read the destructive hint from metadata once during tool discovery. Then give the human-in-the-loop configuration a `when` predicate that receives each pending tool call and returns whether the call needs approval:

## python

## js

### 주의

`interruptOn` keys must be the adapter's tool names, which include the server prefix (`crm_delete_file`). An unprefixed key never matches, so the tool runs without approval.

When the agent calls a tool that the predicate gates, the run pauses. Approve the call to run it, or reject it to skip the tool and tell the model:

## python

```
from langgraph.types import Command

Approve the pending destructive call and resume.
resumed = await agent.ainvoke(
 Command(resume={"decisions": [{"type": "approve"}]}), config
)
```

## js

```
import { Command } from "@langchain/langgraph";

// Approve the pending destructive call and resume.
const resumed = await agent.invoke(
 new Command({ resume: { decisions: [{ type: "approve" }] } }),
 config
);
```

The predicate can also inspect the call's arguments. This lets a tool run freely for safe inputs and pause only for risky ones, such as a `delete_file` call targeting a protected path.

## python

Access the arguments through `request.tool_call["args"]`.

## js

Access the arguments through `request.toolCall.args`.

Combine the metadata classification and argument check to gate a tool only when its type and inputs warrant approval. For the full approval workflow, see [Human-in-the-loop](#).

## Server requests during tool execution

Most tools finish without asking the client for anything mid-call. When a server needs input, `@[ MCPAdapter ]` surfaces [elicitation](#) as a `LangGraph @[ interrupt ]`.

### Elicitation

[Elicitation](#) lets an MCP server request input during a tool call. The adapter pauses the run with a `LangGraph interrupt`. Your application presents the request to the user and resumes the run with their answer.

Attach a [checkpoint](#) to the agent and use the same `thread_id` when invoking and resuming it.

This example assumes the booking tool asks for one date and pauses once. Replace the sample date with the user's answer.

#### python

Elicitation is on by default. The adapter advertises the capability and drives the interrupt loop. A prebuilt client that already carries its own elicitation handler is honored instead of overridden.

Resume with `Command(resume={"responses": {key: answer}})`. The interrupt payload and answer types live in `langchain.mcp.elicitation`.

#### js

Interrupt-driven elicitation requires a modern MCP server and is enabled by default. Set `elicitation: false` on the server to opt out. For a legacy server, configure an `onElicitation` handler with `mode: "legacy"` instead. Without a checkpoint, a tool that asks for input fails with an error `ToolMessage` instead of pausing.

`createMCP ElicitationResume` addresses the answer to the interrupt that requested it. The response uses the server's request key. Build every resume value with `createMCP ElicitationResume` rather than a bare `{ responses }` object, which does not say which interrupt it answers.

Each interrupt's value is an `MCP ElicitationInterrupt`: `type: "mcp_elicitation"`, `server`, `tool` (the server's own tool name, without the prefix), `arguments` (the effective tool arguments), and `requests`. Check `type` when the same agent can also raise human-in-the-loop interrupts.

When several calls pause at once, answer every interrupt. Merge the objects that `createMCPElicitationResume` returns into one `Command({ resume })`, for example `Object.assign({}, ...paused.__interrupt__!.map((q) => createMCPElicitationResume(q, answers)))`, or resume until `__interrupt__` is empty. A resume that answers only the first interrupt leaves the others paused.

## 참고

Resuming reruns the tool from the beginning. Any work performed before the server asks for input can repeat. Make that work safe to repeat without duplicating side effects.

Each answer uses one of these actions:

- **accept** : Provide form `content` matching the request's schema, or confirm completion of a URL interaction without `content`.
- **decline** : Refuse to provide the requested information.
- **cancel** : Indicate that the user canceled the interaction.

The adapter forwards the answer to the server, which decides how the tool call finishes.

## python

Only elicitation is answered this way. A server that instead asks for [sampling](#) or [roots](#) raises an error, because the modern, sessionless protocol has no live back-channel for those requests. See [Sampling and roots](#).

## js

Only elicitation is answered this way. The adapter declares only the elicitation capability on each call, so a tool that needs [sampling](#) or [roots](#) fails with a `ToolException`, which `createAgent` returns to the model as an error `ToolMessage`. See [Sampling and roots](#).

## 참고

Interrupt-driven elicitation answers a server that returns its request as an `InputRequiredResult`. A server that only pushes elicitation over a legacy handshake session cannot be answered this way.



## See also

---

- [Content blocks](#)
- [Tools](#)
- [Human-in-the-loop](#)

### python

- [FastMCP calling tools](#)
  - [FastMCP client elicitation](#)
  - [FastMCP server elicitation](#)
- 
- [MCP elicitation specification](#)
  - [MCP tool annotations](#)

**langchain / middleware**

# Prebuilt middleware

langchain/middleware/built-in.mdx

LangChain and [Deep Agents](#) provide prebuilt middleware for common use cases. Each middleware is production-ready and configurable for your specific needs.

## Provider-agnostic middleware

The following middleware work with any LLM provider:

### python

Middleware	Description
<a href="#">Tool error</a>	Catch tool execution exceptions and convert them to error messages for the model.
<a href="#">Tool retry</a>	Automatically retry failed tool calls with exponential backoff.
<a href="#">Model retry</a>	Automatically retry failed model calls with exponential backoff.
<a href="#">Model fallback</a>	Automatically fallback to alternative models when primary fails.
<a href="#">Summarization</a>	Automatically summarize conversation history when approaching token limits.
<a href="#">Human-in-the-loop</a>	Pause execution for human approval of tool calls.
<a href="#">Model call limit</a>	Limit the number of model calls to prevent excessive costs.
<a href="#">Tool call limit</a>	Control tool execution by limiting call counts.
<a href="#">PII detection</a>	Detect and handle Personally Identifiable Information (PII).
<a href="#">To-do list</a>	Equip agents with task planning and tracking capabilities.
<a href="#">LLM tool selector</a>	Use an LLM to select relevant tools before calling main model.
<a href="#">Provider tool search</a>	Defer tools behind providers' server-side tool search, surfacing them on demand.
<a href="#">Shell tool</a>	Expose a persistent shell session to agents for command execution.
<a href="#">Filesystem</a>	Provide agents with a filesystem for storing context and long-term memories.
<a href="#">Subagent</a>	Add the ability to spawn subagents.
<a href="#">Rubric grading (Beta)</a>	Apply LLM-as-a-judge grading so agents self-evaluate and iterate until a rubric is satisfied.
<a href="#">File search</a>	Provide Glob and Grep search tools over filesystem files.
<a href="#">Context editing</a>	Manage conversation context by trimming or clearing tool uses.

Middleware	Description
<a href="#">LLM tool emulator</a>	Emulate tool execution using an LLM for testing purposes.

## js

Middleware	Description
<a href="#">Tool retry</a>	Automatically retry failed tool calls with exponential backoff.
<a href="#">Model retry</a>	Automatically retry failed model calls with exponential backoff.
<a href="#">Model fallback</a>	Automatically fallback to alternative models when primary fails.
<a href="#">Summarization</a>	Automatically summarize conversation history when approaching token limits.
<a href="#">Human-in-the-loop</a>	Pause execution for human approval of tool calls.
<a href="#">Model call limit</a>	Limit the number of model calls to prevent excessive costs.
<a href="#">Tool call limit</a>	Control tool execution by limiting call counts.
<a href="#">PII detection</a>	Detect and handle Personally Identifiable Information (PII).
<a href="#">To-do list</a>	Equip agents with task planning and tracking capabilities.
<a href="#">LLM tool selector</a>	Use an LLM to select relevant tools before calling main model.
<a href="#">Provider tool search</a>	Defer tools behind providers' server-side tool search, surfacing them on demand.
<a href="#">Filesystem</a>	Provide agents with a filesystem for storing context and long-term memories.
<a href="#">Subagent middleware</a>	Add the ability to spawn subagents.
<a href="#">Context editing</a>	Manage conversation context by trimming or clearing tool uses.
<a href="#">LLM tool emulator</a>	Emulate tool execution using an LLM for testing purposes.

## Tool error

### python

Catch exceptions raised during tool execution and convert them into error `ToolMessage`s that the model can see and recover from, instead of halting the agent run. Tool error is useful for the following:

- Letting the model retry a failed tool call with corrected arguments.
- Surfacing controlled, sanitized error messages instead of raw exception details.
- Preventing unexpected tool exceptions from crashing the agent.

### 참고

Tool error middleware does not automatically retry failed calls. For retries, compose with [Tool retry](#) middleware placed *inner* (later in the `middleware` list) and configured with `on_failure="error"` so that exceptions reach the tool error middleware. See the [full example](#) below.

**API reference:** `@[ ToolErrorMiddleware ]`

## 참고

`ToolErrorMiddleware` requires `langchain>=1.3.14`.

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolErrorMiddleware

def on_error(exc: Exception, request: ToolCallRequest) -> str | None:
 if isinstance(exc, ValueError):
 return f"`{request.tool_call['name']}` failed with {type(exc).__name__}."
 # propagate everything else

agent = create_agent(
 model="gpt-5.5",
 tools=[your_tools],
 middleware=[ToolErrorMiddleware(on_error)],
)
```

## Configuration options

Sync handler called for each exception raised by tool execution. Return content (a `str` or list of content blocks) to convert the exception into a `ToolMessage(status="error")`. Return `None` or omit a return statement to let the exception propagate. Used on the sync path and, unless `on_error` is given, on the async path.

Optional async handler, used on the async execution path. Falls back to `on_error` when not provided.

Optional list of tools or tool names to apply error handling to. If `None`, applies to all tools.

## Tool error full example

The `on_error` handler receives the exception and the `ToolCallRequest` (which includes the tool call dict with name, args, and call ID). Return `None` for exceptions you do not want to handle, and they will propagate normally.

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolErrorMiddleware, ToolRetryMiddleware
```

```

def on_error(exc: Exception, request: ToolCallRequest) -> str | None:
 # Surface ValueError to the model so it can correct the input
 if isinstance(exc, ValueError):
 return f"`{request.tool_call['name']}` failed: {type(exc).__name__}. Fix the input and retry."
 # Let all other exceptions propagate (halts the run)
 return None

Async-only usage
async def aon_error(exc: Exception, request: ToolCallRequest) -> str | None:
 if isinstance(exc, ConnectionError):
 return f"Tool `{request.tool_call['name']}` encountered a connection error."
 return None

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, database_tool],
 middleware=[
 # Place retry inner so exceptions reach ToolErrorMiddleware after retries are exhausted
 ToolErrorMiddleware(on_error=on_error, tools=["search_tool"]),
 ToolRetryMiddleware(max_retries=3, on_failure="error"),
],
)

Async-only: pass aon_error alone (do not pass on_error)
async_agent = create_agent(
 model="gpt-5.5",
 tools=[api_tool],
 middleware=[ToolErrorMiddleware(aon_error=aon_error)],
)

```

## 참고

Prefer returning content that names the exception type over the raw exception message, which may carry sensitive or internal detail. The `on_error` handler controls disclosure: the raw exception message is never sent to the model unless you choose to include it.

## Tool retry

Automatically retry failed tool calls with configurable exponential backoff. Tool retry is useful for the following:

- Handling transient failures in external API calls.

- Improving reliability of network-dependent tools.
- Building resilient agents that gracefully handle temporary errors.

## python

**API reference:** @[ ToolRetryMiddleware ]

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolRetryMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, database_tool],
 middleware=[
 ToolRetryMiddleware(
 max_retries=3,
 backoff_factor=2.0,
 initial_delay=1.0,
),
],
)
```

## js

**API reference:** @[ toolRetryMiddleware ]

```
import { createAgent, toolRetryMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, databaseTool],
 middleware: [
 toolRetryMiddleware({
 maxRetries: 3,
 backoffFactor: 2.0,
 initialDelayMs: 1000,
 }),
],
});
```

## Configuration options

### python

Maximum number of retry attempts after the initial call (3 total attempts with default)

Optional list of tools or tool names to apply retry logic to. If `None`, applies to all tools.

Either a tuple of exception types to retry on, or a callable that takes an exception and returns `True` if it should be retried. With `langchain>=1.3.16`, the default retries retryable [model errors](#) and all unclassified exceptions, and no longer retries model errors marked non-retryable.

Behavior when all retries are exhausted. Options: - `'continue'` (default) - Return a `ToolMessage` with error details, allowing the LLM to handle the failure - `'error'` - Re-raise the exception, stopping agent execution - Custom callable - Function that takes the exception and returns a string for the `ToolMessage` content

```
Deprecated values: 'return_message' (use 'continue' instead) and 'raise' (use 'error' instead).
```

Multiplier for exponential backoff. Each retry waits `initial_delay * (backoff_factor ** retry_number)` seconds. Set to `0.0` for constant delay.

Initial delay in seconds before first retry

Maximum delay in seconds between retries (caps exponential backoff growth)

Whether to add random jitter ( $\pm 25\%$ ) to delay to avoid thundering herd

## js

Maximum number of retry attempts after the initial call (3 total attempts with default). Must be  $\geq 0$ .

Optional array of tools or tool names to apply retry logic to. Can be a list of `BaseTool` instances or tool name strings. If `undefined`, applies to all tools.

boolean) | (new (...args: any[]) => Error)[]" default="(error) => getRetryable(error) ?? true"> Either an array of error constructors to retry on, or a function that takes an error and returns `true` if it should be retried. With `langchain>=1.5.9`, the default no longer retries errors that are marked non-retryable. Every other error is retried.

string)" default="continue"> Behavior when all retries are exhausted. Options: - `'continue'` (default) - Return a `ToolMessage` with error details, allowing the LLM to handle the failure and potentially recover - `'error'` - Re-raise the exception, stopping agent execution - Custom function - Function that takes the exception and returns a string for the `ToolMessage` content, allowing custom error formatting

```
Deprecated values: 'raise' (use 'error' instead) and 'return_message' (use 'continue' instead). These deprecated values still work but will show a warning.
```



Multiplier for exponential backoff. Each retry waits `initialDelayMs * (backoffFactor ** retryNumber)` milliseconds. Set to `0.0` for constant delay. Must be `>= 0`.

Initial delay in milliseconds before first retry. Must be `>= 0`.

Maximum delay in milliseconds between retries (caps exponential backoff growth). Must be `>= 0`.

Whether to add random jitter ( $\pm 25\%$ ) to delay to avoid thundering herd

## Full example

The middleware automatically retries failed tool calls with exponential backoff.

### python

**Key configuration:** - `max_retries` - Number of retry attempts (default: 2) - `backoff_factor` - Multiplier for exponential backoff (default: 2.0) - `initial_delay` - Starting delay in seconds (default: 1.0) - `max_delay` - Cap on delay growth (default: 60.0) - `jitter` - Add random variation (default: True)

**Failure handling:** - `on_failure='continue'` (default) - Return error message - `on_failure='error'` - Re-raise exception - Custom function - Function returning error message

### js

**Key configuration:** - `maxRetries` - Number of retry attempts (default: 2) - `backoffFactor` - Multiplier for exponential backoff (default: 2.0) - `initialDelayMs` - Starting delay in milliseconds (default: 1000ms) - `maxDelayMs` - Cap on delay growth (default: 60000ms) - `jitter` - Add random variation (default: true)

**Failure handling:** - `onFailure: "continue"` (default) - Return error message - `onFailure: "error"` - Re-raise exception - Custom function - Function returning error message

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolRetryMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, database_tool, api_tool],
 middleware=[
 ToolRetryMiddleware(
```

```

 max_retries=3,
 backoff_factor=2.0,
 initial_delay=1.0,
 max_delay=60.0,
 jitter=True,
 tools=["api_tool"],
 retry_on=(ConnectionError, TimeoutError),
 on_failure="continue",
),
],
)

```

js

```

import { createAgent, toolRetryMiddleware } from "langchain";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

// Basic usage with default settings (2 retries, exponential backoff)
const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, databaseTool],
 middleware: [toolRetryMiddleware()],
});

// Retry specific exceptions only
const retry = toolRetryMiddleware({
 maxRetries: 4,
 retryOn: [TimeoutError, NetworkError],
 backoffFactor: 1.5,
});

// Custom exception filtering
function shouldRetry(error: Error): boolean {
 // Only retry on 5xx errors
 if (error.name === "HTTPError" && "statusCode" in error) {
 const statusCode = (error as any).statusCode;
 return 500 <= statusCode && statusCode < 600;
 }
 return false;
}

const retryWithFilter = toolRetryMiddleware({
 maxRetries: 3,
 retryOn: shouldRetry,
});

// Apply to specific tools with custom error handling
const formatError = (error: Error) =>

```

```

 "Database temporarily unavailable. Please try again later.";

const retrySpecificTools = toolRetryMiddleware({
 maxRetries: 4,
 tools: ["search_database"],
 onFailure: formatError,
});

// Apply to specific tools using BaseTool instances
const searchDatabase = tool(
 async ({ query }) => {
 // Search implementation
 return results;
 },
 {
 name: "search_database",
 description: "Search the database",
 schema: z.object({ query: z.string() }),
 }
);

const retryWithToolInstance = toolRetryMiddleware({
 maxRetries: 4,
 tools: [searchDatabase], // Pass BaseTool instance
});

// Constant backoff (no exponential growth)
const constantBackoff = toolRetryMiddleware({
 maxRetries: 5,
 backoffFactor: 0.0, // No exponential growth
 initialDelayMs: 2000, // Always wait 2 seconds
});

// Raise exception on failure
const strictRetry = toolRetryMiddleware({
 maxRetries: 2,
 onFailure: "error", // Re-raise exception instead of returning message
});

```

js

## Mark an error as retryable

### 참고

Requires `@langchain/core`>=1.2.8 and `langchain`>=1.5.9.

Mark an error before throwing it and the retry middleware honors that verdict. This keeps retries from being spent on a failure that repeats identically, such as invalid input or a rejected credential.

Use `stampRetryable` from `@langchain/core/errors`:

```
import { stampRetryable } from "@langchain/core/errors";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const lookupRecord = tool(
 async ({ id }) => {
 const response = await fetch(`https://api.example.com/records/${id}`);

 if (response.status === 422) {
 // Retrying the same identifier fails the same way
 throw stampRetryable(new Error("Malformed record identifier"), false);
 }
 if (response.status === 503) {
 throw stampRetryable(new Error("Record service unavailable"), true);
 }

 return response.json();
 },
 {
 name: "lookup_record",
 description: "Look up a record by identifier",
 schema: z.object({ id: z.string() }),
 }
);
```

Pass `false` for a failure that cannot succeed on another attempt, and `true` for a transient one. Errors that you do not mark are retried, so existing tools keep their current behavior.

Marking an error does not change its type or shape, so `instanceof` checks against it continue to work. To read a mark back, call `getRetryable(error)`, which returns `true`, `false`, or `undefined` when the error was never marked.

Chat model integrations already mark the failures they recognize, so you do not need to mark these yourself. The following fail on the first attempt rather than consuming retries:

- **Authentication failures:** A missing, invalid, or expired API key.
- **Permission denied:** Valid credentials without access to the requested resource.
- **Unknown model:** A model identifier the provider does not recognize.
- **Context window overflow:** Input longer than the model accepts.
- **Invalid tool results:** A malformed tool result block.

- **Payload too large:** A request body the provider rejects on size.
- **Exhausted quota:** A billing or usage limit, as opposed to a rate limit.
- **Cancelled calls:** A request stopped through an abort signal.

Rate limits, timeouts, and server errors stay retryable. Anything a provider does not recognize is left unmarked and retries as before.

## Model retry

Automatically retry failed model calls with configurable exponential backoff. Model retry is useful for the following:

- Handling transient failures in model API calls.
- Improving reliability of network-dependent model requests.
- Building resilient agents that gracefully handle temporary model errors.

### python

**API reference:** `@[ModelRetryMiddleware]`

```
from langchain.agents import create_agent
from langchain.agents.middleware import ModelRetryMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, database_tool],
 middleware=[
 ModelRetryMiddleware(
 max_retries=3,
 backoff_factor=2.0,
 initial_delay=1.0,
),
],
)
```

### js

**API reference:** `@[modelRetryMiddleware]`

```
import { createAgent, modelRetryMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, databaseTool],
 middleware: [
 modelRetryMiddleware({
```

```

 maxRetries: 3,
 backoffFactor: 2.0,
 initialDelayMs: 1000,
 }},
],
});

```

## Configuration options

### python

Maximum number of retry attempts after the initial call (3 total attempts with default)

Either a tuple of exception types to retry on, or a callable that takes an exception and returns `True` if it should be retried. With `langchain>=1.3.16`, the default retries retryable [model errors](#) and all unclassified exceptions, and no longer retries model errors marked non-retryable.

Behavior when all retries are exhausted. Options: - `'continue'` (default) - Return an `AIMessage` with error details, allowing the agent to potentially handle the failure gracefully - `'error'` - Re-raise the exception (stops agent execution) - Custom callable - Function that takes the exception and returns a string for the `AIMessage` content

Multiplier for exponential backoff. Each retry waits ``initial_delay * (backoff_factor ** retry_number)`` seconds. Set to ``0.0`` for constant delay.

Initial delay in seconds before first retry

Maximum delay in seconds between retries (caps exponential backoff growth)

Whether to add random jitter ( $\pm 25\%$ ) to delay to avoid thundering herd

### js

Maximum number of retry attempts after the initial call (3 total attempts with default). Must be  $\geq 0$ .

`boolean) | (new (...args: any[]) => Error)[]"` default="(error) => getRetryable(error) ?? true"> Either an array of error constructors to retry on, or a function that takes an error and returns `true` if it should be retried. With `langchain>=1.5.9`, the default no longer retries errors that are marked non-retryable. Every other error is retried.

`string)"` default="continue"> Behavior when all retries are exhausted. Options: - `'continue'` (default) - Return an `AIMessage` with error details, allowing the agent to potentially handle the failure gracefully - `'error'` - Re-raise the exception, stopping

agent execution - Custom function - Function that takes the exception and returns a string for the AIMessage content, allowing custom error formatting

```
Multiplier for exponential backoff. Each retry waits `initialDelayMs * (backoffFact
or ** retryNumber)` milliseconds. Set to `0.0` for constant delay. Must be >= 0.
```

Initial delay in milliseconds before first retry. Must be >= 0.

Maximum delay in milliseconds between retries (caps exponential backoff growth).  
Must be >= 0.

Whether to add random jitter ( $\pm 25\%$ ) to delay to avoid thundering herd

## Full example

The middleware automatically retries failed model calls with exponential backoff.

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import ModelRetryMiddleware

Basic usage with default settings (2 retries, exponential backoff)
agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool],
 middleware=[ModelRetryMiddleware()],
)

Custom exception filtering
class TimeoutError(Exception):
 """Custom exception for timeout errors."""
 pass

class ConnectionError(Exception):
 """Custom exception for connection errors."""
 pass

Retry specific exceptions only
retry = ModelRetryMiddleware(
 max_retries=4,
 retry_on=(TimeoutError, ConnectionError),
 backoff_factor=1.5,
)

def should_retry(error: Exception) -> bool:
 # Only retry on rate limit errors
 if isinstance(error, TimeoutError):
```

```

 return True
 # Or check for specific HTTP status codes
 if hasattr(error, "status_code"):
 return error.status_code in (429, 503)
 return False

retry_with_filter = ModelRetryMiddleware(
 max_retries=3,
 retry_on=should_retry,
)

Return error message instead of raising
retry_continue = ModelRetryMiddleware(
 max_retries=4,
 on_failure="continue", # Return AIMessage with error instead of raising
)

Custom error message formatting
def format_error(error: Exception) -> str:
 return f"Model call failed: {error}. Please try again later."

retry_with_formatter = ModelRetryMiddleware(
 max_retries=4,
 on_failure=format_error,
)

Constant backoff (no exponential growth)
constant_backoff = ModelRetryMiddleware(
 max_retries=5,
 backoff_factor=0.0, # No exponential growth
 initial_delay=2.0, # Always wait 2 seconds
)

Raise exception on failure
strict_retry = ModelRetryMiddleware(
 max_retries=2,
 on_failure="error", # Re-raise exception instead of returning message
)

```

js

```

import { createAgent, modelRetryMiddleware } from "langchain";

// Basic usage with default settings (2 retries, exponential backoff)
const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool],
 middleware: [modelRetryMiddleware()],
});

```



```

class TimeoutError extends Error {
 // ...
}
class NetworkError extends Error {
 // ...
}

// Retry specific exceptions only
const retry = modelRetryMiddleware({
 maxRetries: 4,
 retryOn: [TimeoutError, NetworkError],
 backoffFactor: 1.5,
});

// Custom exception filtering
function shouldRetry(error: Error): boolean {
 // Only retry on rate limit errors
 if (error.name === "RateLimitError") {
 return true;
 }
 // Or check for specific HTTP status codes
 if (error.name === "HTTPError" && "statusCode" in error) {
 const statusCode = (error as any).statusCode;
 return statusCode === 429 || statusCode === 503;
 }
 return false;
}

const retryWithFilter = modelRetryMiddleware({
 maxRetries: 3,
 retryOn: shouldRetry,
});

// Return error message instead of raising
const retryContinue = modelRetryMiddleware({
 maxRetries: 4,
 onFailure: "continue", // Return AIMessage with error instead of throwing
});

// Custom error message formatting
const formatError = (error: Error) =>
 `Model call failed: ${error.message}. Please try again later.`;

const retryWithFormatter = modelRetryMiddleware({
 maxRetries: 4,
 onFailure: formatError,
});

// Constant backoff (no exponential growth)
const constantBackoff = modelRetryMiddleware({

```

```

maxRetries: 5,
backoffFactor: 0.0, // No exponential growth
initialDelayMs: 2000, // Always wait 2 seconds
});

// Raise exception on failure
const strictRetry = modelRetryMiddleware({
 maxRetries: 2,
 onFailure: "error", // Re-raise exception instead of returning message
});

```

## Model fallback

Automatically fallback to alternative models when the primary model fails. Model fallback is useful for the following:

- Building resilient agents that handle model outages.
- Cost optimization by falling back to cheaper models.
- Provider redundancy across OpenAI, Anthropic, etc.

### python

**API reference:** `@[ModelFallbackMiddleware]`

```

from langchain.agents import create_agent
from langchain.agents.middleware import ModelFallbackMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 ModelFallbackMiddleware(
 "gpt-5.4-mini",
 "claude-3-5-sonnet-20241022",
),
],
)

```

### js

```

import { createAgent, modelFallbackMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 middleware: [
 modelFallbackMiddleware(

```

```
"gpt-5.4-mini",
"claude-3-5-sonnet-20241022"
),
],
});
```

## python

### 참고

Watch this [video guide](#) demonstrating Model Fallback middleware behavior.

## Configuration options

### python

First fallback model to try when the primary model fails. Can be a model identifier string (e.g., 'openai:gpt-5.4-mini' ) or a `BaseChatModel` instance.

Additional fallback models to try in order if previous models fail

### js

The middleware accepts a variable number of string arguments representing fallback models in order:

One or more fallback model strings to try in order when the primary model fails

```
modelFallbackMiddleware(
 "first-fallback-model",
 "second-fallback-model",
 // ... more models
)
```

## Summarization

Automatically summarize conversation history when approaching token limits, preserving recent messages while compressing older context. Summarization is useful for the following:

- Long-running conversations that exceed context windows.
- Multi-turn dialogues with extensive history.
- Applications where preserving full conversation context matters.

### 참고

Summarization is text-oriented context compression. It does not resize, downsample, or otherwise compress image/audio/video payloads. Recent messages retained by `keep` still include their original multimodal blocks, while older multimodal messages that are summarized are represented only by the generated text summary. For image-heavy applications, store media in a filesystem or object store and pass URLs or file references through message history.

## python

**API reference:** `@[SummarizationMiddleware]`

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[your_weather_tool, your_calculator_tool],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger=("tokens", 4000),
 keep=("messages", 20),
),
],
)
```

## js

```
import { createAgent, summarizationMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [weatherTool, calculatorTool],
 middleware: [
 summarizationMiddleware({
 model: "gpt-5.4-mini",
 trigger: { tokens: 4000 },
 keep: { messages: 20 },
 }),
],
});
```

## Configuration options

### python

## 팁

The `fraction` conditions for `trigger` and `keep` (shown below) rely on a chat model's [profile data](#) if using `langchain>=1.1`. If data are not available, use another condition or specify manually:

```
from langchain.chat_models import init_chat_model

custom_profile = {
 "max_input_tokens": 100_000,
 # ...
}

model = init_chat_model("gpt-5.5", profile=custom_profile)
```

Model for generating summaries. Can be a model identifier string (e.g., `'openai:gpt-5.4-mini'`) or a `BaseChatModel` instance. See `@[init_chat_model]` `[init_chat_model(model)]` for more information.

Condition(s) for triggering summarization. Can be:

- A single `@[ContextSize]` tuple (the specified threshold must be met)
- A single `@[TriggerClause]` dict (all specified thresholds must be met - AND logic)
- A list mixing either form (any item must be met - OR logic)

Supported thresholds are:

- `fraction` (float): Fraction of model's context size (0-1)
- `tokens` (int): Absolute token count
- `messages` (int): Message count

A `@[ContextSize]` tuple expresses exactly one threshold. A `@[TriggerClause]` dict can include one or more thresholds, e.g. `{"tokens": 4000, "messages": 10}`, and all thresholds in the dict must be met (AND).

Each `@[TriggerClause]` dict must specify at least one threshold. If `trigger` is not provided, summarization will not trigger automatically.

See the API reference for `@[ContextSize]` and `@[TriggerClause]` for more information.

How much context to preserve after summarization. Specify exactly one of:

- `fraction` (float): Fraction of model's context size to keep (0-1)
- `tokens` (int): Absolute token count to keep
- `messages` (int): Number of recent messages to keep

See the API reference for `@[ ContextSize ]` for more information.

Custom token counting function. Defaults to character-based counting.

Custom prompt template for summarization. Uses built-in template if not specified. The template should include `{messages}` placeholder where conversation history will be inserted.

Maximum number of tokens to include when generating the summary. Messages will be trimmed to fit this limit before summarization.

**Deprecated:** Use `summary_prompt` to provide the full prompt instead.

**Deprecated:** Use `trigger: ("tokens", value)` instead. Token threshold for triggering summarization.

**Deprecated:** Use `keep: ("messages", value)` instead. Recent messages to preserve.

js

#### 팁

The `fraction` conditions for `trigger` and `keep` (shown below) rely on a chat model's [profile data](#) if using `langchain@1.1.0`. If data are not available, use another condition or specify manually:

```
const customProfile: ModelProfile = {
 maxInputTokens: 100_000,
 // ...
}
model = await initChatModel("...", {
 profile: customProfile,
});
```

Model for generating summaries. Can be a model identifier string (e.g., `'openai:gpt-5.4-mini'`) or a `BaseChatModel` instance.

Conditions for triggering summarization. Can be:

- **Single condition (AND logic):** a single object where all properties must be met simultaneously.

```
```typescript
// Trigger when tokens >= 4000 AND messages >= 10
trigger: { tokens: 4000, messages: 10 }
```
```

- **Multiple conditions (OR logic):** an array of condition objects. Summarization triggers when any object's conditions are met.

```
```typescript
// Trigger when tokens >= 3000 OR messages >= 6
trigger: [{ tokens: 3000 }, { messages: 6 }]
```
```

Each condition can include:

- `fraction` (number): Fraction of model's context size (0-1)
- `tokens` (number): Absolute token count
- `messages` (number): Message count

At least one property must be specified per condition. If not provided, summarization will not trigger automatically.

How much context to preserve after summarization. Specify exactly one of:

- `fraction` (number): Fraction of model's context size to keep (0-1)
- `tokens` (number): Absolute token count to keep
- `messages` (number): Number of recent messages to keep

Custom token counting function. Defaults to character-based counting.

Custom prompt template for summarization. Uses built-in template if not specified. The template should include `{messages}` placeholder where conversation history will be inserted.

Maximum number of tokens to include when generating the summary. Messages will be trimmed to fit this limit before summarization.

Prefix to add to the summary message. If not provided, a default prefix is used.

**Deprecated:** Use `trigger: { tokens: value }` instead. Token threshold for triggering summarization.

**Deprecated:** Use `keep: { messages: value }` instead. Recent messages to preserve.

## Full example

The summarization middleware monitors message token counts and automatically summarizes older messages when thresholds are reached.

**Trigger conditions** control when summarization runs:

- A single threshold triggers when that threshold is met
- A trigger clause with multiple thresholds triggers only when all thresholds are met (AND logic)
- A list of trigger conditions triggers when any item is met (OR logic)
- Each threshold can use `fraction` (of model's context size), `tokens` (absolute count), or `messages` (message count)

**Keep condition** control how much context to preserve (specify exactly one): - fraction - Fraction of model's context size to keep - tokens - Absolute token count to keep - messages - Number of recent messages to keep

## python

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

Single condition: trigger if tokens >= 4000
agent = create_agent(
 model="gpt-5.5",
 tools=[your_weather_tool, your_calculator_tool],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger=("tokens", 4000),
 keep=("messages", 20),
),
],
)

Multiple conditions: trigger if number of tokens >= 3000 OR messages >= 6
agent2 = create_agent(
 model="gpt-5.5",
 tools=[your_weather_tool, your_calculator_tool],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger=[
 ("tokens", 3000),
 ("messages", 6),
],
 keep=("messages", 20),
),
],
)

AND logic: trigger only when tokens >= 4000 AND messages >= 10
agent3 = create_agent(
 model="gpt-5.5",
 tools=[your_weather_tool, your_calculator_tool],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger={"tokens": 4000, "messages": 10},
 keep=("messages", 20),
),
],
)
```



```

Combine AND and OR: trigger if (tokens >= 5000 AND messages >= 3)
OR (tokens >= 3000 AND messages >= 6)
agent4 = create_agent(
 model="gpt-5.5",
 tools=[your_weather_tool, your_calculator_tool],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger=[
 {"tokens": 5000, "messages": 3},
 {"tokens": 3000, "messages": 6},
],
 keep=("messages", 20),
),
],
)

Using fractional limits
agent5 = create_agent(
 model="gpt-5.5",
 tools=[your_weather_tool, your_calculator_tool],
 middleware=[
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger=("fraction", 0.8),
 keep=("fraction", 0.3),
),
],
)

```

js

```

import { createAgent, summarizationMiddleware } from "langchain";

// Single condition (AND logic): trigger if tokens >= 4000 AND messages >= 10
const agent = createAgent({
 model: "gpt-5.5",
 tools: [weatherTool, calculatorTool],
 middleware: [
 summarizationMiddleware({
 model: "gpt-5.4-mini",
 trigger: { tokens: 4000, messages: 10 },
 keep: { messages: 20 },
 }),
],
});

// Multiple conditions (OR logic): trigger if tokens >= 3000 OR messages >= 6

```

```

const agent2 = createAgent({
 model: "gpt-5.5",
 tools: [weatherTool, calculatorTool],
 middleware: [
 summarizationMiddleware({
 model: "gpt-5.4-mini",
 trigger: [
 { tokens: 3000 },
 { messages: 6 },
],
 keep: { messages: 20 },
 }),
],
});

// Using fractional limits
const agent3 = createAgent({
 model: "gpt-5.5",
 tools: [weatherTool, calculatorTool],
 middleware: [
 summarizationMiddleware({
 model: "gpt-5.4-mini",
 trigger: { fraction: 0.8 },
 keep: { fraction: 0.3 },
 }),
],
});

```

## Human-in-the-loop

Pause agent execution for human approval, editing, or rejection of tool calls before they execute. [Human-in-the-loop](#) is useful for the following:

- High-stakes operations requiring human approval (e.g. database writes, financial transactions).
- Compliance workflows where human oversight is mandatory.
- Long-running conversations where human feedback guides the agent.

### python

**API reference:** `@[ HumanInTheLoopMiddleware ]`

### 주의

Human-in-the-loop middleware requires a [checkpointer](#) to maintain state across interruptions.

## python

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

def your_read_email_tool(email_id: str) -> str:
 """Mock function to read an email by its ID."""
 return f"Email content for ID: {email_id}"

def your_send_email_tool(recipient: str, subject: str, body: str) -> str:
 """Mock function to send an email."""
 return f"Email sent to {recipient} with subject '{subject}'"

agent = create_agent(
 model="gpt-5.5",
 tools=[your_read_email_tool, your_send_email_tool],
 checkpoint=InMemorySaver(),
 middleware=[
 HumanInTheLoopMiddleware(
 interrupt_on={
 "your_send_email_tool": {
 "allowed_decisions": ["approve", "edit", "reject"],
 },
 "your_read_email_tool": False,
 }
),
],
)
```

## js

```
import { createAgent, humanInTheLoopMiddleware } from "langchain";

function readEmailTool(emailId: string): string {
 /** Mock function to read an email by its ID. */
 return `Email content for ID: ${emailId}`;
}

function sendEmailTool(recipient: string, subject: string, body: string): string {
 /** Mock function to send an email. */
 return `Email sent to ${recipient} with subject '${subject}'`;
}

const agent = createAgent({
 model: "gpt-5.5",
 tools: [readEmailTool, sendEmailTool],
 middleware: [
```

```

humanInTheLoopMiddleware({
 interruptOn: {
 sendEmailTool: {
 allowedDecisions: ["approve", "edit", "reject"],
 },
 readEmailTool: false,
 }
})
]
});

```

## 팁

For complete examples, configuration options, and integration patterns, see the [Human-in-the-loop documentation](#).

## python

### 참고

Watch this [video guide](#) demonstrating Human-in-the-loop middleware behavior.

## js

### 참고

Watch this [video guide](#) demonstrating Human-in-the-loop middleware behavior.

## Model call limit

Limit the number of model calls to prevent infinite loops or excessive costs. Model call limit is useful for the following:

- Preventing runaway agents from making too many API calls.
- Enforcing cost controls on production deployments.
- Testing agent behavior within specific call budgets.

## python

**API reference:** @[ModelCallLimitMiddleware]

```

from langchain.agents import create_agent
from langchain.agents.middleware import ModelCallLimitMiddleware

```

```

from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
 model="gpt-5.5",
 checkpointer=InMemorySaver(), # Required for thread limiting
 tools=[],
 middleware=[
 ModelCallLimitMiddleware(
 thread_limit=10,
 run_limit=5,
 exit_behavior="end",
),
],
)

```

js

```

import { createAgent, modelCallLimitMiddleware } from "langchain";
import { MemorySaver } from "@langchain/langgraph";

const agent = createAgent({
 model: "gpt-5.5",
 checkpointer: new MemorySaver(), // Required for thread limiting
 tools: [],
 middleware: [
 modelCallLimitMiddleware({
 threadLimit: 10,
 runLimit: 5,
 exitBehavior: "end",
 }),
],
});

```

python

참고

Watch this [video guide](#) demonstrating Model Call Limit middleware behavior.

js

참고

Watch this [video guide](#) demonstrating Model Call Limit middleware behavior.

## Configuration options

### python

Maximum model calls across all runs in a thread. Defaults to no limit.

Maximum model calls per single invocation. Defaults to no limit.

Behavior when limit is reached. Options: 'end' (graceful termination) or 'error' (raise exception)

### js

Maximum model calls across all runs in a thread. Defaults to no limit.

Maximum model calls per single invocation. Defaults to no limit.

Behavior when limit is reached. Options: 'end' (graceful termination) or 'error' (throw exception)

## Tool call limit

Control agent execution by limiting the number of tool calls, either globally across all tools or for specific tools. Tool call limits are useful for the following:

- Preventing excessive calls to expensive external APIs.
- Limiting web searches or database queries.
- Enforcing rate limits on specific tool usage.
- Protecting against runaway agent loops.

### python

**API reference:** @[ ToolCallLimitMiddleware ]

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolCallLimitMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, database_tool],
 middleware=[
 # Global limit
 ToolCallLimitMiddleware(thread_limit=20, run_limit=10),
 # Tool-specific limit
 ToolCallLimitMiddleware(
 tool_name="search",
 thread_limit=5,
```

```

 run_limit=3,
),
],
)

```

js

```

import { createAgent, toolCallLimitMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, databaseTool],
 middleware: [
 toolCallLimitMiddleware({ threadLimit: 20, runLimit: 10 }),
 toolCallLimitMiddleware({
 toolName: "search",
 threadLimit: 5,
 runLimit: 3,
 }),
],
});

```

python

참고

Watch this [video guide](#) demonstrating Tool Call Limit middleware behavior.

js

참고

Watch this [video guide](#) demonstrating Tool Call Limit middleware behavior.

## Configuration options

python

Name of specific tool to limit. If not provided, limits apply to **all tools globally**.

Maximum tool calls across all runs in a thread (conversation). Persists across multiple invocations with the same thread ID. Requires a checkpointer to maintain state. `None` means no thread limit.

Maximum tool calls per single invocation (one user message → response cycle). Resets with each new user message. `None` means no run limit.

**Note:** At least one of `thread_limit` or `run_limit` must be specified.

Behavior when limit is reached:

- `'continue'` (default) - Block exceeded tool calls with error messages, let other tools and the model continue. The model decides when to end based on the error messages.
- `'error'` - Raise a `ToolCallLimitExceededError` exception, stopping execution immediately
- `'end'` - Stop execution immediately with a `ToolMessage` and AI message for the exceeded tool call. Only works when limiting a single tool; raises `NotImplementedError` if other tools have pending calls.

## js

Name of specific tool to limit. If not provided, limits apply to **all tools globally**.

Maximum tool calls across all runs in a thread (conversation). Persists across multiple invocations with the same thread ID. Requires a checkpointer to maintain state. `undefined` means no thread limit.

Maximum tool calls per single invocation (one user message → response cycle). Resets with each new user message. `undefined` means no run limit.

**Note:** At least one of `threadLimit` or `runLimit` must be specified.

Behavior when limit is reached:

- `'continue'` (default) - Block exceeded tool calls with error messages, let other tools and the model continue. The model decides when to end based on the error messages.
- `'error'` - Throw a `ToolCallLimitExceededError` exception, stopping execution immediately
- `'end'` - Stop execution immediately with a `ToolMessage` and AI message for the exceeded tool call. Only works when limiting a single tool; throws error if other tools have pending calls.

## Full example

Specify limits with: - **Thread limit** - Max calls across all runs in a conversation (requires checkpointer) - **Run limit** - Max calls per single invocation (resets each turn)



Exit behaviors: - 'continue' (default) - Block exceeded calls with error messages, agent continues - 'error' - Raise exception immediately - 'end' - Stop with ToolMessage + AI message (single-tool scenarios only)

## python

```
from langchain.agents import create_agent
from langchain.agents.middleware import ToolCallLimitMiddleware

global_limiter = ToolCallLimitMiddleware(thread_limit=20, run_limit=10)
search_limiter = ToolCallLimitMiddleware(tool_name="search", thread_limit=5, run_limit=3)
database_limiter = ToolCallLimitMiddleware(tool_name="query_database", thread_limit=10)
strict_limiter = ToolCallLimitMiddleware(tool_name="scrape_webpage", run_limit=2, exit_behavior="error")

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, database_tool, scraper_tool],
 middleware=[global_limiter, search_limiter, database_limiter, strict_limiter],
)
```

## js

```
import { createAgent, toolCallLimitMiddleware } from "langchain";

const globalLimiter = toolCallLimitMiddleware({ threadLimit: 20, runLimit: 10 });
const searchLimiter = toolCallLimitMiddleware({ toolName: "search", threadLimit: 5, runLimit: 3 });
const databaseLimiter = toolCallLimitMiddleware({ toolName: "query_database", threadLimit: 10 });
const strictLimiter = toolCallLimitMiddleware({ toolName: "scrape_webpage", runLimit: 2, exitBehavior: "error" });

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, databaseTool, scraperTool],
 middleware: [globalLimiter, searchLimiter, databaseLimiter, strictLimiter],
});
```

## PII detection

Detect and handle Personally Identifiable Information (PII) in conversations using configurable strategies. PII detection is useful for the following:

- Healthcare and financial applications with compliance requirements.
- Customer service agents that need to sanitize logs.
- Any application handling sensitive user data.

## python

### 참고

With `apply_to_output=True`, `PIIMiddleware` also redacts streamed wire output—text deltas, tool-call args, tool outputs, and state snapshots—via a registered stream transformer. Requires `langchain>=1.3.2`. See [Register transformers on middleware](#).

### API reference: @[ `PIIMiddleware` ]

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 PIIMiddleware("email", strategy="redact", apply_to_input=True),
 PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
],
)
```

## js

```
import { createAgent, piiMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 middleware: [
 piiMiddleware("email", { strategy: "redact", applyToInput: true }),
 piiMiddleware("credit_card", { strategy: "mask", applyToInput: true }),
],
});
```

## Custom PII types

You can create custom PII types by providing a `detector` parameter. This allows you to detect patterns specific to your use case beyond the built-in types.

### Three ways to create custom detectors:

## 1. Regexp pattern string - Simple pattern matching

js

### 1. RegExp object - More control over regex flags

## 1. Custom function - Complex detection logic with validation

python

```
import re

from langchain.agents import create_agent
from langchain.agents.middleware import PIIMatch, PIIMiddleware

Method 1: Regex pattern string
agent1 = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 PIIMiddleware(
 "api_key",
 detector=r"sk-[a-zA-Z0-9]{32}",
 strategy="block",
),
],
)

Method 2: Compiled regex pattern
agent2 = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 PIIMiddleware(
 "phone_number",
 detector=re.compile(r"\+?\d{1,3}[\s.-]? \d{3,4}[\s.-]? \d{4}"),
 strategy="mask",
),
],
)

Method 3: Custom detector function
def detect_ssn(content: str) -> list[PIIMatch]:
 """Detect SSNs with validation."""
 matches: list[PIIMatch] = []
 pattern = r"\d{3}-\d{2}-\d{4}"
 for match in re.finditer(pattern, content):
 ssn = match.group(0)
 # Validate: first 3 digits shouldn't be 000, 666, or 900-999
```

```

 first_three = int(ssn[:3])
 if first_three not in [0, 666] and not (900 <= first_three <= 999):
 matches.append({
 "type": "ssn",
 "value": ssn,
 "start": match.start(),
 "end": match.end(),
 })
 return matches

agent3 = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 PIIMiddleware(
 "ssn",
 detector=detect_ssn,
 strategy="hash",
),
],
)

```

js

```

import { createAgent, piiMiddleware, type PIIMatch } from "langchain";

// Method 1: Regex pattern string
const agent1 = createAgent({
 model: "gpt-5.5",
 tools: [],
 middleware: [
 piiMiddleware("api_key", {
 detector: "sk-[a-zA-Z0-9]{32}",
 strategy: "block",
 }),
],
});

// Method 2: RegExp object
const agent2 = createAgent({
 model: "gpt-5.5",
 tools: [],
 middleware: [
 piiMiddleware("phone_number", {
 detector: /\+?\d{1,3}[\s.-]?\d{3,4}[\s.-]?\d{4}/,
 strategy: "mask",
 }),
],
});

```

```
// Method 3: Custom detector function
function detectSSN(content: string): PIIMatch[] {
 const matches: PIIMatch[] = [];
 const pattern = /\d{3}-\d{2}-\d{4}/g;
 let match: RegExpExecArray | null;

 while ((match = pattern.exec(content)) !== null) {
 const ssn = match[0];
 // Validate: first 3 digits shouldn't be 000, 666, or 900-999
 const firstThree = parseInt(ssn.substring(0, 3), 10);
 if (firstThree !== 0 && firstThree !== 666 && !(firstThree >= 900 && firstThree
<= 999)) {
 matches.push({
 text: ssn,
 start: match.index ?? 0,
 end: (match.index ?? 0) + ssn.length,
 });
 }
 }
 return matches;
}

const agent3 = createAgent({
 model: "gpt-5.5",
 tools: [],
 middleware: [
 piiMiddleware("ssn", {
 detector: detectSSN,
 strategy: "hash",
 }),
],
});
```

### Custom detector function signature:

The detector function must accept a string (content) and return matches:

#### python

Returns a list of `PIIMatch` objects:

```
from langchain.agents.middleware import PIIMatch

def detector(content: str) -> list[PIIMatch]:
 return [
 {
 "type": "custom_type",
 "value": "matched_text",
```

```

 "start": 0,
 "end": 12,
 },
 # ... more matches
]

```

## js

Returns an array of `PIIMatch` objects:

```

interface PIIMatch {
 text: string; // The matched text
 start: number; // Start index in content
 end: number; // End index in content
}

function detector(content: string): PIIMatch[] {
 return [
 { text: "matched_text", start: 0, end: 12 },
 // ... more matches
];
}

```

## 팁

For custom detectors:

- Use regex strings for simple patterns
- Use `RegExp` objects when you need flags (e.g., case-insensitive matching)
- Use custom functions when you need validation logic beyond pattern matching
- Custom functions give you full control over detection logic and can implement complex validation rules

## Configuration options

### python

Type of PII to detect. Can be a built-in type (`email`, `credit_card`, `ip`, `mac_address`, `url`) or a custom type name.

How to handle detected PII. Options:

- `'block'` - Raise exception when detected
- `'redact'` - Replace with `[REDACTED_{PII_TYPE}]`
- `'mask'` - Partially mask (e.g., `****-****-****-1234`)

- 'hash' - Replace with deterministic hash

Custom detector function or regex pattern. If not provided, uses built-in detector for the PII type.

Check user messages before model call

Check AI messages after model call. With `langchain>=1.3.2`, also redacts streamed wire output (text deltas, tool-call args, tool outputs, state snapshots) via a registered stream transformer. See [event streaming](#).

Check tool result messages after execution

## js

Type of PII to detect. Can be a built-in type (`email`, `credit_card`, `ip`, `mac_address`, `url`) or a custom type name.

How to handle detected PII. Options:

- 'block' - Throw error when detected
- 'redact' - Replace with `[REDACTED_TYPE]`
- 'mask' - Partially mask (e.g., `****-****-****-1234`)
- 'hash' - Replace with deterministic hash (e.g., `<email_hash:a1b2c3d4>`)

Custom detector. Can be:

- `RegExp` - Regex pattern for matching
- `string` - Regex pattern string (e.g., `"sk-[a-zA-Z0-9]{32}"`)
- `function` - Custom detector function (`content: string`) => `PIIMatch[]`

If not provided, uses built-in detector for the PII type.

Check user messages before model call

Check AI messages after model call

Check tool result messages after execution

## To-do list

Equip agents with task planning and tracking capabilities for complex multi-step tasks. To-do lists are useful for the following:

- Complex multi-step tasks requiring coordination across multiple tools.

- Long-running operations where progress visibility is important.

## 참고

This middleware automatically provides agents with a `write_todos` tool and system prompts to guide effective task planning.

## python

**API reference:** `@[TodoListMiddleware]`

```
from langchain.agents import create_agent
from langchain.agents.middleware import TodoListMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[read_file, write_file, run_tests],
 middleware=[TodoListMiddleware()],
)
```

## js

```
import { createAgent, todoListMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [readFile, writeFile, runTests],
 middleware: [todoListMiddleware()],
});
```

## python

### 참고

Watch this [video guide](#) demonstrating To-do List middleware behavior.

## js

### 참고

Watch this [video guide](#) demonstrating To-do List middleware behavior.

## Configuration options



## python

Custom system prompt for guiding todo usage. Uses built-in prompt if not specified.

Custom description for the `write_todos` tool. Uses built-in description if not specified.

## js

No configuration options available (uses defaults).

## LLM tool selector

Use an LLM to intelligently select relevant tools before calling the main model. LLM tool selectors are useful for the following:

- Agents with many tools (10+) where most aren't relevant per query.
- Reducing token usage by filtering irrelevant tools.
- Improving model focus and accuracy.

This middleware uses structured output to ask an LLM which tools are most relevant for the current query. The structured output schema defines the available tool names and descriptions. Model providers often add this structured output information to the system prompt behind the scenes.

## python

**API reference:** `@[ LLMToolSelectorMiddleware ]`

```
from langchain.agents import create_agent
from langchain.agents.middleware import LLMToolSelectorMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[tool1, tool2, tool3, tool4, tool5, ...],
 middleware=[
 LLMToolSelectorMiddleware(
 model="gpt-5.4-mini",
 max_tools=3,
 always_include=["search"],
),
],
)
```

## js

```
import { createAgent, llmToolSelectorMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [tool1, tool2, tool3, tool4, tool5, ...],
 middleware: [
 llmToolSelectorMiddleware({
 model: "gpt-5.4-mini",
 maxTools: 3,
 alwaysInclude: ["search"],
 }),
],
});
```

## Configuration options

### python

Model for tool selection. Can be a model identifier string (e.g., 'openai:gpt-5.4-mini' ) or a `BaseChatModel` instance. See `@[init_chat_model ][init_chat_model(model)]` for more information.

Defaults to the agent's main model.

Instructions for the selection model. Uses built-in prompt if not specified.

Maximum number of tools to select. If the model selects more, only the first `max_tools` will be used. No limit if not specified.

Tool names to always include regardless of selection. These do not count against the `max_tools` limit.

### js

Model for tool selection. Can be a model identifier string (e.g., 'openai:gpt-5.4-mini' ) or a `BaseChatModel` instance. Defaults to the agent's main model.

Instructions for the selection model. Uses built-in prompt if not specified.

Maximum number of tools to select. If the model selects more, only the first `maxTools` will be used. No limit if not specified.

Tool names to always include regardless of selection. These do not count against the `maxTools` limit.

### python

## Provider tool search

Defer selected tools behind model providers' server-side tool search, so the model discovers them on demand instead of receiving every tool schema up front. Provider tool search is useful for:

- Reducing context bloat when using many tools.
- Improving tool selection accuracy by surfacing only relevant tools.

### 참고

Requires a model with server-side tool search support: Anthropic (Claude Sonnet 4+/Opus 4+/Haiku 4.5+) or OpenAI (gpt-5.5+). Other providers raise a `ValueError`.

**API reference:** `@[ ProviderToolSearchMiddleware ]`

```
from langchain.agents import create_agent
from langchain.agents.middleware import ProviderToolSearchMiddleware

agent = create_agent(
 model="anthropic:claude-opus-4-8",
 tools=[get_weather, lookup_order],
 middleware=[
 ProviderToolSearchMiddleware(searchable_tools=["lookup_order"]),
],
)
```

## Configuration options

Tools to defer behind the provider's tool search, given by name or instance. Deferred tools are withheld from the model until its search surfaces them. Tools constructed with `extras={"defer_loading": True}` are deferred regardless of this option; if `searchable_tools` is omitted, only those pre-marked tools are deferred.

## Full example

The middleware opts-in all tools included in `searchable_tools` for deferral and search. A tool can also opt into deferral at construction time by setting `extras={"defer_loading": True}`.

```
from langchain.agents import create_agent
from langchain.agents.middleware import ProviderToolSearchMiddleware
from langchain.tools import tool

Marked `defer_loading` at construction, so it's deferred on its own -
no need to list it in `searchable_tools`.
@tool(extras={"defer_loading": True})
```

```
def send_email(to: str) -> str:
 """Send an email."""
 return "sent"

agent = create_agent(
 model="anthropic:claude-opus-4-8",
 tools=[send_email],
 middleware=[ProviderToolSearchMiddleware()],
)
```

js

## Provider tool search

Defer selected tools behind model providers' server-side tool search, so the model discovers them on demand instead of receiving every tool schema up front. Provider tool search is useful for:

- Reducing context bloat when using many tools.
- Improving tool selection accuracy by surfacing only relevant tools.

### 참고

Requires a model with server-side tool search support: Anthropic (Claude Sonnet 4+/Opus 4+/Haiku 4.5+) or OpenAI (gpt-5.5+). Other providers throw an error.

**API reference:** `@[providerToolSearchMiddleware]`

```
import { createAgent, providerToolSearchMiddleware } from "langchain";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const getWeather = tool(async () => "Sunny, 22C", {
 name: "get_weather",
 description: "Get the current weather for a city",
 schema: z.object({ city: z.string() }),
});

const lookupOrderStatus = tool(async () => "OUT_FOR_DELIVERY", {
 name: "lookup_order_status",
 description: "Look up the current delivery status of a customer order by ID",
 schema: z.object({ orderId: z.string() }),
});

const nicheTools = [lookupOrderStatus];

const agent = createAgent({
```

```

model: "anthropic:claude-opus-4-8",
tools: [getWeather, ...nicheTools],
middleware: [
 providerToolSearchMiddleware({ searchableTools: nicheTools }),
],
});

```

## Configuration options

Tools to defer behind the provider's tool search, given by name or instance. Deferred tools are withheld from the model until its search surfaces them. Tools constructed with `extras.defer_loading: true` are deferred regardless of this option; if `searchableTools` is omitted, only those pre-marked tools are deferred.

## Full example

The middleware opts-in all tools included in the `searchableTools` for deferral and search. A tool can also opt into deferral at construction time by setting `extras.defer_loading: true`

```

import { createAgent, providerToolSearchMiddleware } from "langchain";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

// Marked `defer_loading` at construction, so it's deferred on its own –
// no need to list it in `searchableTools`.
const sendEmail = tool(async () => "sent", {
 name: "send_email",
 description: "Send an email",
 schema: z.object({ to: z.string() }),
 extras: { defer_loading: true },
});

const agent = createAgent({
 model: "anthropic:claude-opus-4-8",
 tools: [sendEmail],
 middleware: [providerToolSearchMiddleware()],
});

```

## python

### Shell tool

Expose a persistent shell session to agents for command execution. Shell tool middleware is useful for the following:

- Agents that need to execute system commands

- Development and deployment automation tasks
- Testing and validation workflows
- File system operations and script execution

## 주의

**Security consideration:** Use appropriate execution policies ( `HostExecutionPolicy` , `DockerExecutionPolicy` , OR `CodexSandboxExecutionPolicy` ) to match your deployment's security requirements.

## 참고

**Limitation:** Persistent shell sessions do not currently work with interrupts (human-in-the-loop). We anticipate adding support for this in the future.

**API reference:** `@[ ShellToolMiddleware ]`

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
 ShellToolMiddleware,
 HostExecutionPolicy,
)

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool],
 middleware=[
 ShellToolMiddleware(
 workspace_root="/workspace",
 execution_policy=HostExecutionPolicy(),
),
],
)
```

## Configuration options

Base directory for the shell session. If omitted, a temporary directory is created when the agent starts and removed when it ends.

Optional commands executed sequentially after the session starts

Optional commands executed before the session shuts down

Execution policy controlling timeouts, output limits, and resource configuration.  
Options:

- `HostExecutionPolicy` - Full host access (default); best for trusted environments where the agent already runs inside a container or VM
- `DockerExecutionPolicy` - Launches a separate Docker container for each agent run, providing harder isolation
- `CodexSandboxExecutionPolicy` - Reuses the Codex CLI sandbox for additional syscall/filesystem restrictions

Optional redaction rules to sanitize command output before returning it to the model.

## 주의

Redaction rules are applied post execution and do not prevent exfiltration of secrets or sensitive data when using `HostExecutionPolicy`.

Optional override for the registered shell tool description

Optional shell executable (string) or argument sequence used to launch the persistent session. Defaults to `/bin/bash`.

Optional environment variables to supply to the shell session. Values are coerced to strings before command execution.

## Full example

The middleware provides a single persistent shell session that agents can use to execute commands sequentially.

**Execution policies:** - `HostExecutionPolicy` (default) - Native execution with full host access - `DockerExecutionPolicy` - Isolated Docker container execution - `CodexSandboxExecutionPolicy` - Sandboxed execution via Codex CLI

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
 ShellToolMiddleware,
 HostExecutionPolicy,
 DockerExecutionPolicy,
 RedactionRule,
)

Basic shell tool with host execution
agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool],
 middleware=[
 ShellToolMiddleware(
 workspace_root="/workspace",
```

```

 execution_policy=HostExecutionPolicy(),
),
],
)

Docker isolation with startup commands
agent_docker = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 ShellToolMiddleware(
 workspace_root="/workspace",
 startup_commands=["pip install requests", "export PYTHONPATH=/workspace"],
 execution_policy=DockerExecutionPolicy(
 image="python:3.11-slim",
 command_timeout=60.0,
),
),
],
)

With output redaction (applied post execution)
agent_redacted = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 ShellToolMiddleware(
 workspace_root="/workspace",
 redaction_rules=[
 RedactionRule(pii_type="api_key", detector=r"sk-[a-zA-Z0-9]{32}"),
],
),
],
)

```

## Filesystem middleware

Context engineering is a main challenge in building effective agents. This is particularly difficult when using tools that return variable-length results (for example, `web_search` and `RAG`), as long tool results can quickly fill your context window.

`FilesystemMiddleware` from [Deep Agents](#) provides four tools for interacting with both short-term and long-term memory:

- `ls` : List the files in the filesystem
- `read_file` : Read an entire file or a certain number of lines from a file
- `write_file` : Write a new file to the filesystem



- `edit_file`: Edit an existing file in the filesystem

## python

```
from langchain.agents import create_agent
from deepagents.middleware.filesystem import FilesystemMiddleware

FilesystemMiddleware is included by default in create_deep_agent
You can customize it if building a custom agent
agent = create_agent(
 model="claude-sonnet-4-6",
 middleware=[
 FilesystemMiddleware(
 backend=None, # Optional: custom backend (defaults to StateBackend)
 system_prompt="Write to the filesystem when...", # Optional custom addition to the system prompt
 custom_tool_descriptions={
 "ls": "Use the ls tool when...",
 "read_file": "Use the read_file tool to..."
 }, # Optional: Custom descriptions for filesystem tools
 tools=["read_file", "ls", "glob", "grep"], # Optional: Allowlist restricting which filesystem tools are exposed
),
],
)
```

## js

```
import { createAgent } from "langchain";
import { createFilesystemMiddleware } from "deepagents";

// FilesystemMiddleware is included by default in createDeepAgent
// You can customize it if building a custom agent
const agent = createAgent({
 model: "claude-sonnet-4-6",
 middleware: [
 createFilesystemMiddleware({
 backend: undefined, // Optional: custom backend (defaults to StateBackend)
 systemPrompt: "Write to the filesystem when...", // Optional custom system prompt override
 customToolDescriptions: {
 ls: "Use the ls tool when...",
 read_file: "Use the read_file tool to...",
 }, // Optional: Custom descriptions for filesystem tools
 }),
],
});
```

## Short-term vs. long-term filesystem

By default, these tools write to a local "filesystem" in your graph state. To enable persistent storage across threads, configure a `CompositeBackend` that routes specific paths (like `/memories/`) to a `StoreBackend`.

### python

```
from langchain.agents import create_agent
from deepagents.middleware import FilesystemMiddleware
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

agent = create_agent(
 model="claude-sonnet-4-6",
 store=store,
 middleware=[
 FilesystemMiddleware(
 backend=CompositeBackend(
 default=StateBackend(),
 routes={"/memories/": StoreBackend()}
),
 custom_tool_descriptions={
 "ls": "Use the ls tool when...",
 "read_file": "Use the read_file tool to..."
 } # Optional: Custom descriptions for filesystem tools
),
],
)
```

### js

```
import { createAgent } from "langchain";
import { createFilesystemMiddleware, CompositeBackend, StateBackend, StoreBackend }
from "deepagents";
import { InMemoryStore } from "@langchain/langgraph-checkpoint";

const store = new InMemoryStore();

const agent = createAgent({
 model: "claude-sonnet-4-6",
 store,
 middleware: [
 createFilesystemMiddleware({
 backend: new CompositeBackend(
 new StateBackend(),
```

```

 { "/memories/": new StoreBackend() }
),
 systemPrompt: "Write to the filesystem when...", // Optional custom system prompt override
 customToolDescriptions: {
 ls: "Use the ls tool when...",
 read_file: "Use the read_file tool to...",
 }, // Optional: Custom descriptions for filesystem tools
 }),
],
});

```

When you configure a `CompositeBackend` with a `StoreBackend` for `/memories/`, any files prefixed with **/memories/** are saved to persistent storage and survive across different threads. Files without this prefix remain in ephemeral state storage.

## Subagent

Handing off tasks to subagents isolates context, keeping the main (supervisor) agent's context window clean while still going deep on a task.

The subagents middleware from [Deep Agents](#) allows you to supply subagents through a task tool.

### python

```

from langchain.tools import tool
from langchain.agents import create_agent
from deepagents.middleware.subagents import SubAgentMiddleware

@tool
def get_weather(city: str) -> str:
 """Get the weather in a city."""
 return f"The weather in {city} is sunny."

agent = create_agent(
 model="claude-sonnet-4-6",
 middleware=[
 SubAgentMiddleware(
 default_model="claude-sonnet-4-6",
 default_tools=[],
 subagents=[
 {
 "name": "weather",
 "description": "This subagent can get weather in cities.",
 "system_prompt": "Use the get_weather tool to get the weather in a city.",
 "tools": [get_weather],

```

```

 "model": "gpt-5.5",
 "middleware": [],
 }
],
)
],
)

```

js

```

import { tool } from "langchain";
import { createAgent } from "langchain";
import { createSubAgentMiddleware } from "deepagents";
import { z } from "zod";

const getWeather = tool(
 async ({ city }: { city: string }) => {
 return `The weather in ${city} is sunny.`;
 },
 {
 name: "get_weather",
 description: "Get the weather in a city.",
 schema: z.object({
 city: z.string(),
 }),
 },
);

const agent = createAgent({
 model: "claude-sonnet-4-6",
 middleware: [
 createSubAgentMiddleware({
 defaultModel: "claude-sonnet-4-6",
 defaultTools: [],
 subagents: [
 {
 name: "weather",
 description: "This subagent can get weather in cities.",
 systemPrompt: "Use the get_weather tool to get the weather in a city.",
 tools: [getWeather],
 model: "gpt-5.5",
 middleware: [],
 },
],
 }),
],
});

```

A subagent is defined with a **name**, **description**, **system prompt**, and **tools**. You can also provide a subagent with a custom **model**, or with additional **middleware**. This can be particularly useful when you want to give the subagent an additional state key to share with the main agent.

For more complex use cases, you can also provide your own prebuilt LangGraph graph as a subagent.

### python

```
from langchain.agents import create_agent
from deepagents.middleware.subagents import SubAgentMiddleware
from deepagents import CompiledSubAgent
from langgraph.graph import StateGraph

Create a custom LangGraph graph
def create_weather_graph():
 workflow = StateGraph(...)
 # Build your custom graph
 return workflow.compile()

weather_graph = create_weather_graph()

Wrap it in a CompiledSubAgent
weather_subagent = CompiledSubAgent(
 name="weather",
 description="This subagent can get weather in cities.",
 runnable=weather_graph
)

agent = create_agent(
 model="claude-sonnet-4-6",
 middleware=[
 SubAgentMiddleware(
 default_model="claude-sonnet-4-6",
 default_tools=[],
 subagents=[weather_subagent],
)
],
)
```

### js

```
import { tool, createAgent } from "langchain";
import { createSubAgentMiddleware, type SubAgent } from "deepagents";
import { z } from "zod";

const getWeather = tool(
```

```

async ({ city }: { city: string }) => {
 return `The weather in ${city} is sunny.`;
},
{
 name: "get_weather",
 description: "Get the weather in a city.",
 schema: z.object({
 city: z.string(),
 }),
},
);

const weatherSubagent: SubAgent = {
 name: "weather",
 description: "This subagent can get weather in cities.",
 systemPrompt: "Use the get_weather tool to get the weather in a city.",
 tools: [getWeather],
 model: "gpt-5.5",
 middleware: [],
};

const agent = createAgent({
 model: "claude-sonnet-4-6",
 middleware: [
 createSubAgentMiddleware({
 defaultModel: "claude-sonnet-4-6",
 defaultTools: [],
 subagents: [weatherSubagent],
 }),
],
});

```

In addition to any user-defined subagents, the main agent has access to a `general-purpose` subagent at all times. This subagent has the same instructions as the main agent and all the tools it has access to. The primary purpose of the `general-purpose` subagent is context isolation—the main agent can delegate a complex task to this subagent and get a concise answer back without bloat from intermediate tool calls.

## python

### Rubric grading

#### 참고

`RubricMiddleware` requires `deepagents>=0.6.5`. It is in [beta](#); the API may change in the future.

Some tasks have a clear definition of "done" that an agent cannot reliably hit on the first try. `RubricMiddleware` lets you declare *what done looks like* as a rubric and have the agent self-evaluate and iterate until the rubric is satisfied or a maximum iteration cap is hit.

**API reference:** @[ `RubricMiddleware` ]

For full configuration options, streaming events, and a complete code generation example, see [Grading rubrics](#).

## python

### File search

Provide Glob and Grep search tools over a filesystem. File search middleware is useful for the following:

- Code exploration and analysis
- Finding files by name patterns
- Searching code content with regex
- Large codebases where file discovery is needed

**API reference:** @[ `FilesystemFileSearchMiddleware` ]

```
from langchain.agents import create_agent
from langchain.agents.middleware import FilesystemFileSearchMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 FilesystemFileSearchMiddleware(
 root_path="/workspace",
 use_ripgrep=True,
),
],
)
```

### Configuration options

Root directory to search. All file operations are relative to this path.

Whether to use ripgrep for search. Falls back to Python regex if ripgrep is unavailable.

Maximum file size to search in MB. Files larger than this are skipped.

### Full example

The middleware adds two search tools to agents:

**Glob tool** - Fast file pattern matching: - Supports patterns like `**/*.py`, `src/**/*.ts` - Returns matching file paths sorted by modification time

**Grep tool** - Content search with regex: - Full regex syntax support - Filter by file patterns with `include` parameter - Three output modes: `files_with_matches`, `content`, `count`

```
from langchain.agents import create_agent
from langchain.agents.middleware import FilesystemFileSearchMiddleware
from langchain.messages import HumanMessage

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 FilesystemFileSearchMiddleware(
 root_path="/workspace",
 use_ripgrep=True,
 max_file_size_mb=10,
),
],
)

Agent can now use glob_search and grep_search tools
result = agent.invoke({
 "messages": [HumanMessage("Find all Python files containing 'async def'")]
})

The agent will use:
1. glob_search(pattern="**/*.py") to find Python files
2. grep_search(pattern="async def", include="*.py") to find async functions
```

## Context editing

Manage conversation context by clearing older tool call outputs when token limits are reached, while preserving recent results. This helps keep context windows manageable in long conversations with many tool calls. Context editing is useful for the following:

- Long conversations with many tool calls that exceed token limits
- Reducing token costs by removing older tool outputs that are no longer relevant
- Maintaining only the most recent N tool results in context

### python

**API reference:** `@[ ContextEditingMiddleware ]`, `@[ ClearToolUsesEdit ]`



```

from langchain.agents import create_agent
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

agent = create_agent(
 model="gpt-5.5",
 tools=[],
 middleware=[
 ContextEditingMiddleware(
 edits=[
 ClearToolUsesEdit(
 trigger=100000,
 keep=3,
),
],
),
],
)

```

js

```

import { createAgent, contextEditingMiddleware, ClearToolUsesEdit } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [],
 middleware: [
 contextEditingMiddleware({
 edits: [
 new ClearToolUsesEdit({
 triggerTokens: 100000,
 keep: 3,
 }),
],
 }),
],
});

```

## Configuration options

python

List of `@[ContextEdit]` strategies to apply

Token counting method. Options: 'approximate' or 'model'

`@[ClearToolUsesEdit]` options:

Token count that triggers the edit. When the conversation exceeds this token count, older tool outputs will be cleared.

Minimum number of tokens to reclaim when the edit runs. If set to 0, clears as much as needed.

Number of most recent tool results that must be preserved. These will never be cleared.

Whether to clear the originating tool call parameters on the AI message. When `True`, tool call arguments are replaced with empty objects.

List of tool names to exclude from clearing. These tools will never have their outputs cleared.

Placeholder text inserted for cleared tool outputs. This replaces the original tool message content.

**js**

Array of `@[ ContextEdit ]` strategies to apply

`@[ ClearToolUsesEdit ]` **options:**

Token count that triggers the edit. When the conversation exceeds this token count, older tool outputs will be cleared.

Minimum number of tokens to reclaim when the edit runs. If set to 0, clears as much as needed.

Number of most recent tool results that must be preserved. These will never be cleared.

Whether to clear the originating tool call parameters on the AI message. When `true`, tool call arguments are replaced with empty objects.

List of tool names to exclude from clearing. These tools will never have their outputs cleared.

Placeholder text inserted for cleared tool outputs. This replaces the original tool message content.

### Full example

The middleware applies context editing strategies when token limits are reached. The most common strategy is `ClearToolUsesEdit`, which clears older tool results while preserving recent ones.

**How it works:** 1. Monitor token count in conversation 2. When threshold is reached, clear older tool outputs 3. Keep most recent N tool results 4. Optionally preserve tool call arguments for context

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

agent = create_agent(
 model="gpt-5.5",
 tools=[search_tool, your_calculator_tool, database_tool],
 middleware=[
 ContextEditingMiddleware(
 edits=[
 ClearToolUsesEdit(
 trigger=2000,
 keep=3,
 clear_tool_inputs=False,
 exclude_tools=[],
 placeholder="[cleared]",
),
],
),
],
)
```

### js

```
import { createAgent, contextEditingMiddleware, ClearToolUsesEdit } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [searchTool, calculatorTool, databaseTool],
 middleware: [
 contextEditingMiddleware({
 edits: [
 new ClearToolUsesEdit({
 triggerTokens: 2000,
 keep: 3,
 clearToolInputs: false,
 excludeTools: [],
 placeholder: "[cleared]",
 }),
],
 }),
],
});
```

```
],
});
```

## LLM tool emulator

Emulate tool execution using an LLM for testing purposes, replacing actual tool calls with AI-generated responses. LLM tool emulators are useful for the following:

- Testing agent behavior without executing real tools.
- Developing agents when external tools are unavailable or expensive.
- Prototyping agent workflows before implementing actual tools.

### python

**API reference:** @[ [LLMToolEmulator](#) ]

```
from langchain.agents import create_agent
from langchain.agents.middleware import LLMToolEmulator

agent = create_agent(
 model="gpt-5.5",
 tools=[get_weather, search_database, send_email],
 middleware=[
 LLMToolEmulator(), # Emulate all tools
],
)
```

### js

```
import { createAgent, toolEmulatorMiddleware } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [getWeather, searchDatabase, sendEmail],
 middleware: [
 toolEmulatorMiddleware(), // Emulate all tools
],
});
```

## Configuration options

### python

List of tool names (str) or BaseTool instances to emulate. If `None` (default), ALL tools will be emulated. If empty list `[]`, no tools will be emulated. If array with tool

names/instances, only those tools will be emulated.

Model to use for generating emulated tool responses. Can be a model identifier string (e.g., 'google\_genai:gemini-3.6-flash') or a `BaseChatModel` instance. Defaults to the agent's model if not specified. See `@[init_chat_model][init_chat_model(model)]` for more information.

### js

List of tool names (string) or tool instances to emulate. If `undefined` (default), ALL tools will be emulated. If empty array `[]`, no tools will be emulated. If array with tool names/instances, only those tools will be emulated.

Model to use for generating emulated tool responses. Can be a model identifier string (e.g., 'google:gemini-3.6-flash') or a `BaseChatModel` instance. Defaults to the agent's model if not specified.

## Full example

The middleware uses an LLM to generate plausible responses for tool calls instead of executing the actual tools.

### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import LLMToolEmulator
from langchain.tools import tool

@tool
def get_weather(location: str) -> str:
 """Get the current weather for a location."""
 return f"Weather in {location}"

@tool
def send_email(to: str, subject: str, body: str) -> str:
 """Send an email."""
 return "Email sent"

Emulate all tools (default behavior)
agent = create_agent(
 model="gpt-5.5",
 tools=[get_weather, send_email],
 middleware=[LLMToolEmulator()],
)

Emulate specific tools only
agent2 = create_agent(
```

```

 model="gpt-5.5",
 tools=[get_weather, send_email],
 middleware=[LLMToolEmulator(tools=["get_weather"])],
)

Use custom model for emulation
agent4 = create_agent(
 model="gpt-5.5",
 tools=[get_weather, send_email],
 middleware=[LLMToolEmulator(model="claude-sonnet-4-6")],
)

```

js

```

import { createAgent, toolEmulatorMiddleware, tool } from "langchain";
import * as z from "zod";

const getWeather = tool(
 async ({ location }) => `Weather in ${location}`,
 {
 name: "get_weather",
 description: "Get the current weather for a location",
 schema: z.object({ location: z.string() }),
 }
);

const sendEmail = tool(
 async ({ to, subject, body }) => "Email sent",
 {
 name: "send_email",
 description: "Send an email",
 schema: z.object({
 to: z.string(),
 subject: z.string(),
 body: z.string(),
 }),
 }
);

// Emulate all tools (default behavior)
const agent = createAgent({
 model: "gpt-5.5",
 tools: [getWeather, sendEmail],
 middleware: [toolEmulatorMiddleware()],
});

// Emulate specific tools by name
const agent2 = createAgent({
 model: "gpt-5.5",

```

```

tools: [getWeather, sendEmail],
middleware: [
 toolEmulatorMiddleware({
 tools: ["get_weather"],
 }),
],
});

// Emulate specific tools by passing tool instances
const agent3 = createAgent({
 model: "gpt-5.5",
 tools: [getWeather, sendEmail],
 middleware: [
 toolEmulatorMiddleware({
 tools: [getWeather],
 }),
],
});

// Use custom model for emulation
const agent5 = createAgent({
 model: "gpt-5.5",
 tools: [getWeather, sendEmail],
 middleware: [
 toolEmulatorMiddleware({
 model: "claude-sonnet-4-6",
 }),
],
});

```

## Provider-specific middleware

These middleware are optimized for specific LLM providers. See each provider's documentation for full details and examples.

### python

#### [Anthropic](#)

Prompt caching, bash tool, text editor, memory, and file search middleware for Claude models.

#### [AWS](#)

Prompt caching middleware for Amazon Bedrock models.

#### [OpenAI](#)

Content moderation middleware for OpenAI models.

js

### [Anthropic](#)

Prompt caching, bash tool, text editor, memory, and file search middleware for Claude models.

### [AWS](#)

Prompt caching middleware for Amazon Bedrock models.



# Custom middleware

langchain/middleware/custom.mdx

Build custom middleware by implementing hooks that run at specific points in the agent execution flow.

## Hooks

Middleware provides two styles of hooks to intercept agent execution:

### [Node-style hooks](#)

Run sequentially at specific execution points.

### [Wrap-style hooks](#)

Run around each model or tool call.

## Node-style hooks

Run sequentially at specific execution points. Use for logging, validation, and state updates.

Choose the hooks your middleware needs. You can choose between node-style hooks and wrap-style hooks.

**Node-style hooks** run at specific execution points:

### python

| Hook         | When it runs                                |
|--------------|---------------------------------------------|
| before_agent | Before agent starts (once per invocation)   |
| before_model | Before each model call                      |
| after_model  | After each model response                   |
| after_agent  | After agent completes (once per invocation) |

### js

| Hook        | When it runs                                |
|-------------|---------------------------------------------|
| beforeAgent | Before agent starts (once per invocation)   |
| beforeModel | Before each model call                      |
| afterModel  | After each model response                   |
| afterAgent  | After agent completes (once per invocation) |

**Wrap-style hooks** run around each call, giving you control over execution:

### python

| Hook            | When it runs           |
|-----------------|------------------------|
| wrap_model_call | Around each model call |
| wrap_tool_call  | Around each tool call  |

### js

| Hook          | When it runs           |
|---------------|------------------------|
| wrapModelCall | Around each model call |
| wrapToolCall  | Around each tool call  |

### Example:

### python

#### Decorator

```
from langchain.agents.middleware import before_model, after_model, AgentState
from langchain.messages import AIMessage
from langgraph.runtime import Runtime
from typing import Any

@before_model(can_jump_to=["end"])
def check_message_limit(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 if len(state["messages"]) >= 50:
 return {
 "messages": [AIMessage("Conversation limit reached.")],
 "jump_to": "end"
 }
 return None

@after_model
def log_response(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 print(f"Model returned: {state['messages'][-1].content}")
 return None
```

#### Class

```
from langchain.agents.middleware import AgentMiddleware, AgentState, hook_config
from langchain.messages import AIMessage
from langgraph.runtime import Runtime
from typing import Any
```

```

class MessageLimitMiddleware(AgentMiddleware):
 def __init__(self, max_messages: int = 50):
 super().__init__()
 self.max_messages = max_messages

@hook_config(can_jump_to=["end"])
def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 if len(state["messages"]) >= self.max_messages:
 return {
 "messages": [AIMessage("Conversation limit reached.")],
 "jump_to": "end"
 }
 return None

def after_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 print(f"Model returned: {state['messages'][-1].content}")
 return None

```

js

```

import { createMiddleware, AIMessage } from "langchain";

const createMessageLimitMiddleware = (maxMessages: number = 50) => {
 return createMiddleware({
 name: "MessageLimitMiddleware",
 beforeModel: {
 canJumpTo: ["end"],
 hook: (state) => {
 if (state.messages.length === maxMessages) {
 return {
 messages: [new AIMessage("Conversation limit reached.")],
 jumpTo: "end",
 };
 }
 return;
 }
 },
 afterModel: (state) => {
 const lastMessage = state.messages[state.messages.length - 1];
 console.log(`Model returned: ${lastMessage.content}`);
 return;
 },
 });
};

```

## Wrap-style hooks

Intercept execution and control when the handler is called. Use for retries, caching, and transformation.

You decide if the handler is called zero times (short-circuit), once (normal flow), or multiple times (retry logic).

### Available hooks:

#### python

- `wrap_model_call` - Around each model call
- `wrap_tool_call` - Around each tool call

#### js

- `wrapModelCall` - Around each model call
- `wrapToolCall` - Around each tool call

### Example:

#### python

##### Decorator

```
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def retry_model(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 for attempt in range(3):
 try:
 return handler(request)
 except Exception as e:
 if attempt == 2:
 raise
 print(f"Retry {attempt + 1}/3 after error: {e}")
```

##### Class

```
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ModelResponse
```

```

from typing import Callable

class RetryMiddleware(AgentMiddleware):
 def __init__(self, max_retries: int = 3):
 super().__init__()
 self.max_retries = max_retries

 def wrap_model_call(
 self,
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 for attempt in range(self.max_retries):
 try:
 return handler(request)
 except Exception as e:
 if attempt == self.max_retries - 1:
 raise
 print(f"Retry {attempt + 1}/{self.max_retries} after error: {e}")

```

js

```

import { createMiddleware } from "langchain";

const createRetryMiddleware = (maxRetries: number = 3) => {
 return createMiddleware({
 name: "RetryMiddleware",
 wrapModelCall: (request, handler) => {
 for (let attempt = 0; attempt < maxRetries; attempt++) {
 try {
 return handler(request);
 } catch (e) {
 if (attempt === maxRetries - 1) {
 throw e;
 }
 console.log(`Retry ${attempt + 1}/${maxRetries} after error: ${e}`);
 }
 }
 throw new Error("Unreachable");
 },
 });
};

```

## State updates

Both node-style and wrap-style hooks can update agent state. The mechanism differs:

## python

- **Node-style hooks** ( `before_agent` , `before_model` , `after_model` , `after_agent` ): Return a dict directly. The dict is applied to the agent state using the graph's reducers.
- **Wrap-style hooks** ( `wrap_model_call` , `wrap_tool_call` ): For model calls, return `@[ ExtendedModelResponse ]` with a `@[ Command ]` to inject state updates alongside the model response. For tool calls, return a `@[ Command ]` directly. Use these when you need to track or update state based on logic that runs during the model or tool call, such as summarization trigger points, usage metadata, or custom fields calculated from the request or response.

## js

- **Node-style hooks** ( `beforeAgent` , `beforeModel` , `afterModel` , `afterAgent` ): Return a dict directly. The dict is applied to the agent state using the graph's reducers.
- **Wrap-style hooks** ( `wrapModelCall` , `wrapToolCall` ): For model calls, return a `@[ Command ]` directly to inject state updates alongside the model response. For tool calls, return a `@[ Command ]` directly. Use these when you need to track or update state based on logic that runs during the model or tool call, such as summarization trigger points, usage metadata, or custom fields calculated from the request or response.

## Node-style hooks

Return a dict from a node-style hook to merge updates into agent state. The dict keys map to state fields.

## python

```
from langchain.agents.middleware import after_model, AgentState
from langgraph.runtime import Runtime
from typing import Any
from typing_extensions import NotRequired

class TrackingState(AgentState):
 model_call_count: NotRequired[int]

@after_model(state_schema=TrackingState)
def increment_after_model(state: TrackingState, runtime: Runtime) -> dict[str, Any] | None:
 return {"model_call_count": state.get("model_call_count", 0) + 1}
```

js

```
import { createMiddleware } from "langchain";
import * as z from "zod";

const trackingStateSchema = z.object({
 modelCallCount: z.number().default(0),
});

const incrementAfterModel = createMiddleware({
 name: "incrementAfterModel",
 stateSchema: trackingStateSchema,
 afterModel: (state) => {
 return { modelCallCount: state.modelCallCount + 1 };
 },
});
```

## Wrap-style hooks

python

Return a `@[ExtendedModelResponse]` with a `@[Command]` from `wrap_model_call` to inject state updates from the model call layer:

```
from typing import Callable
from langchain.agents.middleware import (
 wrap_model_call,
 ModelRequest,
 ModelResponse,
 AgentState,
 ExtendedModelResponse
)
from langgraph.types import Command
from typing_extensions import NotRequired

class UsageTrackingState(AgentState):
 """Agent state with token usage tracking."""

 last_model_call_tokens: NotRequired[int]

@wrap_model_call(state_schema=UsageTrackingState)
def track_usage(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ExtendedModelResponse:
 response = handler(request)
 return ExtendedModelResponse(
 model_response=response,
```

```
 command=Command(update={"last_model_call_tokens": 150}),
)
```

## js

Return a `@[Command]` directly from `wrapModelCall` to inject state updates from the model call layer:

```
import * as z from "zod";
import { createMiddleware } from "langchain";
import { Command } from "@langchain/langgraph";

const usageTrackingStateSchema = z.object({
 lastModelCallTokens: z.number().optional(),
});

const trackUsage = createMiddleware({
 name: "trackUsage",
 stateSchema: usageTrackingStateSchema,
 wrapModelCall: async (request, handler) => {
 const response = await handler(request);
 return new Command({ update: { lastModelCallTokens: 150 } });
 },
});
```

The `@[Command]` flows through the graph's reducers, so updates are applied correctly and messages are additive rather than replacing existing state.

## Composition with multiple middleware

### python

When multiple middleware layers return `ExtendedModelResponse`, their commands compose:

- **Commands are applied through reducers:** Each `Command` becomes a separate state update. For messages, this means they are additive.
- **Outer wins on conflicts:** For non-reducer state fields, commands are applied inner-first, then outer. The outermost middleware's value takes precedence on conflicting keys.
- **Retry-safe:** If the outer middleware implements logic that can result in multiple calls to `handler()` again (for example, retry logic), commands from earlier calls are discarded.



js

When multiple middleware layers return responses, the framework passes on the last `AIMessage` s produced:

- **AIMessage flows through:** Each middleware's `handler()` receives the `AIMessage` from the previous layer. When a middleware returns an `AIMessage`, that becomes the input to the next middleware's handler.
- **Command without message updates is pass-through:** If a middleware returns a `Command` whose state update does not touch `messages`, the framework treats it as a no-op for message flow. The next middleware's handler receives the `AIMessage` from the middleware *before* the one that returned the `Command`.
- **Reducer behavior and retry-safety:** Commands still apply through reducers (messages additive, outer wins on conflicts). Retry logic discards commands from earlier calls.

python

```
from typing import Annotated, Callable

from langchain.agents.middleware import (
 AgentMiddleware,
 AgentState,
 ExtendedModelResponse,
 ModelRequest,
 ModelResponse,
)
from langchain.messages import SystemMessage
from langgraph.types import Command
from typing_extensions import NotRequired

def _last_wins(_a: str, b: str) -> str:
 """Reducer: last writer wins (outer overwrites inner)."""
 return b

class CustomMiddlewareState(AgentState):
 """Agent state: trace_layer uses last-wins (outer wins), messages use additive
 reducer."""

 # Non-reducer field with last-wins: both middleware write; outermost value wins
 trace_layer: NotRequired[Annotated[str, _last_wins]]

class OuterMiddleware(AgentMiddleware):
 def wrap_model_call(
 self,
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
):
```

```

) -> ExtendedModelResponse:
 response = handler(request)
 return ExtendedModelResponse(
 model_response=response,
 command=Command(update={
 "trace_layer": "outer",
 "messages": [SystemMessage(content="[Outer ran]")]},
)),
)

class InnerMiddleware(AgentMiddleware):
 """Adds trace_layer and message. Outer adds to same keys; trace_layer: outer wi
ns, messages: additive."""

 def wrap_model_call(
 self,
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
):
 response = handler(request)
 return ExtendedModelResponse(
 model_response=response,
 command=Command(update={
 "trace_layer": "inner",
 "messages": [SystemMessage(content="[Inner ran]")]},
)),
)

```

js

```

import * as z from "zod";
import { createMiddleware } from "langchain";
import { Command, StateSchema, ReducedValue } from "@langchain/langgraph";
import { AIMessage, SystemMessage } from "@langchain/core/messages";

/** Last-wins reducer: when both middleware write, outer overwrites inner. */
const customMiddlewareStateSchema = new StateSchema({
 traceLayer: new ReducedValue(
 z.string().optional(),
 { reducer: (a, b) => b },
),
});

const outerMiddleware = createMiddleware({
 name: "OuterMiddleware",
 stateSchema: customMiddlewareStateSchema,
 wrapModelCall: async (_request, handler) => {
 await handler(_request);
 return new Command({

```

```

 update: {
 traceLayer: "outer",
 messages: [new SystemMessage({ content: "[Outer ran]" })],
 },
 });
},
});

const innerMiddleware = createMiddleware({
 name: "InnerMiddleware",
 stateSchema: customMiddlewareStateSchema,
 wrapModelCall: async (_request, handler) => {
 await handler(_request);
 return new Command({
 update: {
 traceLayer: "inner",
 messages: [new SystemMessage({ content: "[Inner ran]" })],
 },
 });
 },
});

```

## Create middleware

### python

You can create middleware in two ways:

#### [Decorator-based middleware](#)

Quick and simple for single-hook middleware. Use decorators to wrap individual functions.

#### [Class-based middleware](#)

More powerful for complex middleware with multiple hooks or configuration.

### Decorator-based middleware

Quick and simple for single-hook middleware. Use decorators to wrap individual functions.

#### Available decorators:

**Node-style:** - `@before_agent` - Runs before agent starts (once per invocation) -  
`@before_model` - Runs before each model call - `@after_model` - Runs after each  
model response - `@after_agent` - Runs after agent completes (once per invocation)

**Wrap-style:** - `@wrap_model_call` - Wraps each model call with custom logic -

`@wrap_tool_call` - Wraps each tool call with custom logic

**Convenience:** - `@dynamic_prompt` - Generates dynamic system prompts

### Example:

```
from langchain.agents.middleware import (
 before_model,
 wrap_model_call,
 AgentState,
 ModelRequest,
 ModelResponse,
)
from langchain.agents import create_agent
from langgraph.runtime import Runtime
from typing import Any, Callable

@before_model
def log_before_model(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 print(f>About to call model with {len(state['messages'])} messages")
 return None

@wrap_model_call
def retry_model(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 for attempt in range(3):
 try:
 return handler(request)
 except Exception as e:
 if attempt == 2:
 raise
 print(f"Retry {attempt + 1}/3 after error: {e}")

agent = create_agent(
 model="gpt-5.5",
 middleware=[log_before_model, retry_model],
 tools=[...],
)
```

**When to use decorators:** - Single hook needed - No complex configuration - Quick prototyping

## Class-based middleware

More powerful for complex middleware with multiple hooks or configuration. Use classes when you need to define both sync and async implementations for the same

hook, or when you want to combine multiple hooks in a single middleware.

## python

An `AgentMiddleware` subclass can declare three class attributes that the agent factory picks up at compile time:

- `state_schema` — extend the agent state with custom fields. See [Custom state schema](#).
- `tools` — register additional tools that ship with the middleware (e.g., `write_todos` on the to-do list middleware).
- `transformers` — register scope-aware stream transformer factories. See [Custom stream transformers](#).

## js

`createMiddleware` accepts three configuration fields that the agent factory picks up at compile time:

- `stateSchema` — extend the agent state with custom fields. See [Custom state schema](#).
- `tools` — register additional tools that ship with the middleware.
- `streamTransformers` — register scope-aware stream transformer factories. See [Custom stream transformers](#).

## Example:

```
from langchain.agents.middleware import (
 AgentMiddleware,
 AgentState,
 ModelRequest,
 ModelResponse,
)
from langgraph.runtime import Runtime
from typing import Any, Callable

class LoggingMiddleware(AgentMiddleware):
 def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 print(f"About to call model with {len(state['messages'])} messages")
 return None

 def after_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 print(f"Model returned: {state['messages'][-1].content}")
 return None
```

```

async def abefore_model(
 self, state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
 # Async version of before_model
 return None

async def aafter_model(
 self, state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
 # Async version of after_model
 print(f"Model returned: {state['messages'][-1].content}")
 return None

agent = create_agent(
 model="gpt-5.5",
 middleware=[LoggingMiddleware()],
 tools=[...],
)

```

**When to use classes:** - Defining both sync and async implementations for the same hook - Multiple hooks needed in a single middleware - Complex configuration required (e.g., configurable thresholds, custom models) - Reuse across projects with init-time configuration

js

Use the `createMiddleware` function to define custom middleware:

```

import { createMiddleware } from "langchain";

const loggingMiddleware = createMiddleware({
 name: "LoggingMiddleware",
 beforeModel: (state) => {
 console.log(`About to call model with ${state.messages.length} messages`);
 return;
 },
 afterModel: (state) => {
 const lastMessage = state.messages[state.messages.length - 1];
 console.log(`Model returned: ${lastMessage.content}`);
 return;
 },
});

```

## Custom state schema

If your middleware needs to track state across hooks, middleware can extend the agent's state with custom properties. This enables middleware to:

- **Track state across execution:** Maintain counters, flags, or other values that persist throughout the agent's execution lifecycle

### python

- **Share data between hooks:** Pass information from `before_model` to `after_model` or between different middleware instances

### js

- **Share data between hooks:** Pass information from `beforeModel` to `afterModel` or between different middleware instances

- **Implement cross-cutting concerns:** Add functionality like rate limiting, usage tracking, user context, or audit logging without modifying the core agent logic
- **Make conditional decisions:** Use accumulated state to determine whether to continue execution, jump to different nodes, or modify behavior dynamically

### python

#### Decorator

```
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.agents.middleware import AgentState, before_model, after_model
from typing_extensions import NotRequired
from typing import Any
from langgraph.runtime import Runtime

class CustomState(AgentState):
 model_call_count: NotRequired[int]
 user_id: NotRequired[str]

@before_model(state_schema=CustomState, can_jump_to=["end"])
def check_call_limit(state: CustomState, runtime: Runtime) -> dict[str, Any] | None:
 count = state.get("model_call_count", 0)
 if count > 10:
 return {"jump_to": "end"}
 return None
```

```

@after_model(state_schema=CustomState)
def increment_counter(state: CustomState, runtime: Runtime) -> dict[str, Any] | None:
 return {"model_call_count": state.get("model_call_count", 0) + 1}

agent = create_agent(
 model="gpt-5.5",
 middleware=[check_call_limit, increment_counter],
 tools=[],
)

Invoke with custom state
result = agent.invoke({
 "messages": [HumanMessage("Hello")],
 "model_call_count": 0,
 "user_id": "user-123",
})

```

## Class

```

from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.agents.middleware import AgentState, AgentMiddleware
from typing_extensions import NotRequired
from typing import Any

class CustomState(AgentState):
 model_call_count: NotRequired[int]
 user_id: NotRequired[str]

class CallCounterMiddleware(AgentMiddleware[CustomState]):
 state_schema = CustomState

 def before_model(self, state: CustomState, runtime) -> dict[str, Any] | None:
 count = state.get("model_call_count", 0)
 if count > 10:
 return {"jump_to": "end"}
 return None

 def after_model(self, state: CustomState, runtime) -> dict[str, Any] | None:
 return {"model_call_count": state.get("model_call_count", 0) + 1}

agent = create_agent(
 model="gpt-5.5",
 middleware=[CallCounterMiddleware()],
 tools=[],
)

Invoke with custom state
result = agent.invoke({

```



```
"messages": [HumanMessage("Hello")],
"model_call_count": 0,
"user_id": "user-123",
})
```

js

```
import { createMiddleware, createAgent, HumanMessage } from "langchain";
import { StateSchema } from "@langchain/langgraph";
import * as z from "zod";

const CustomState = new StateSchema({
 modelCallCount: z.number().default(0),
 userId: z.string().optional(),
});

const callCounterMiddleware = createMiddleware({
 name: "CallCounterMiddleware",
 stateSchema: CustomState,
 beforeModel: {
 canJumpTo: ["end"],
 hook: (state) => {
 if (state.modelCallCount > 10) {
 return { jumpTo: "end" };
 }

 return;
 },
 },
 afterModel: (state) => {
 return { modelCallCount: state.modelCallCount + 1 };
 },
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 middleware: [callCounterMiddleware],
});

const result = await agent.invoke({
 messages: [new HumanMessage("Hello")],
 modelCallCount: 0,
 userId: "user-123",
});
```

State fields can be either public or private. Fields that start with an underscore ( `_` ) are considered private and will not be included in the agent's result. Only public fields

(those without a leading underscore) are returned.

This is useful for storing internal middleware state that shouldn't be exposed to the caller, such as temporary tracking variables or internal flags:

```
import { StateSchema } from "@langchain/langgraph";
import * as z from "zod";

const PrivateState = new StateSchema({
 // Public field - included in invoke result
 publicCounter: z.number().default(0),
 // Private field - excluded from invoke result
 _internalFlag: z.boolean().default(false),
});

const middleware = createMiddleware({
 name: "ExampleMiddleware",
 stateSchema: PrivateState,
 afterModel: (state) => {
 // Both fields are accessible during execution
 if (state._internalFlag) {
 return { publicCounter: state.publicCounter + 1 };
 }
 return { _internalFlag: true };
 },
});

const result = await agent.invoke({
 messages: [new HumanMessage("Hello")],
 publicCounter: 0
});

// result only contains publicCounter, not _internalFlag
console.log(result.publicCounter); // 1
console.log(result._internalFlag); // undefined
```

## Custom stream transformers

### python

#### 참고

Middleware-registered transformers require `langchain>=1.3.2`.

### js

## 참고

Middleware-registered transformers require `langchain@1.4.3` or later.

Middleware can register stream transformer factories that project events from the live agent stream onto typed extension channels. This is useful for surfacing counters, side-channel artifacts, partial outputs, or wire-level redaction without coupling to the framework's built-in projections.

At compile time, middleware-registered factories merge with anything the caller passes directly to the agent factory. The [final ordering rules](#) keep the built-in `ToolCallTransformer` in front and let caller-supplied entries land last.

### python

Set the `transformers` class attribute to a tuple of factory callables. Each factory has the shape `Callable[[tuple[str, ...]], StreamTransformer]` and is invoked as `factory(scope)`, where `scope` is the mini-mux scope tuple (`()` for the root, non-empty for subgraphs); returning a fresh transformer per call keeps each subgraph isolated.

```
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware

class ToolActivityMiddleware(AgentMiddleware):
 transformers = (ToolActivityTransformer,)

agent = create_agent(
 model="gpt-5-nano",
 tools=[...],
 middleware=[ToolActivityMiddleware()],
)
```

### js

Pass `streamTransformers` to `createMiddleware` as a tuple of factories. Each factory has the shape `() => StreamTransformer<any>` (zero arguments) and is invoked once per scope; returning a fresh transformer per call keeps each subgraph isolated.

```
import { createAgent, createMiddleware } from "langchain";

const toolActivityMiddleware = createMiddleware({
 name: "ToolActivityMiddleware",
 streamTransformers: [toolActivityTransformer],
});
```

```
const agent = createAgent({
 model: "gpt-5-nano",
 tools: [...],
 middleware: [toolActivityMiddleware],
});
```

See [Register transformers on middleware](#) for the full ordering rules and the PII redaction example.

js

## Custom context

Middleware can define a custom context schema to access per-invocation metadata. Unlike state, context is read-only and not persisted between invocations. This makes it ideal for:

- **User information:** Pass user ID, roles, or preferences that don't change during execution
- **Configuration overrides:** Provide per-invocation settings like rate limits or feature flags
- **Tenant/workspace context:** Include organization-specific data for multi-tenant applications
- **Request metadata:** Pass request IDs, API keys, or other metadata needed by middleware

Define a context schema using Zod and access it via `runtime.context` in middleware hooks. Required fields in the context schema will be enforced at the TypeScript level, ensuring you must provide them when calling `agent.invoke()`.

```
import { createAgent, createMiddleware, HumanMessage } from "langchain";
import * as z from "zod";

const contextSchema = z.object({
 userId: z.string(),
 tenantId: z.string(),
 apiKey: z.string().optional(),
});

const userContextMiddleware = createMiddleware({
 name: "UserContextMiddleware",
 contextSchema,
 wrapModelCall: (request, handler) => {
 // Access context from runtime
 const { userId, tenantId } = request.runtime.context;
```

```
// Add user context to system message
const contextText = `User ID: ${userId}, Tenant: ${tenantId}`;
const newSystemMessage = request.systemMessage.concat(contextText);

 return handler({
 ...request,
 systemMessage: newSystemMessage,
 });
 },
});

const agent = createAgent({
 model: "gpt-5.5",
 middleware: [userContextMiddleware],
 tools: [],
 contextSchema,
});

const result = await agent.invoke(
 { messages: [new HumanMessage("Hello")] },
 // Required fields (userId, tenantId) must be provided
 {
 context: {
 userId: "user-123",
 tenantId: "acme-corp",
 },
 }
);
```

**Required context fields:** When you define required fields in your `contextSchema` (fields without `.optional()` or `.default()`), TypeScript will enforce that these fields must be provided during `agent.invoke()` calls. This ensures type safety and prevents runtime errors from missing required context.

```
// This will cause a TypeScript error if userId or tenantId are missing
const result = await agent.invoke(
 { messages: [new HumanMessage("Hello")] },
 { context: { userId: "user-123" } } // Error: tenantId is required
);
```

## Execution order

When using multiple middleware, understand how they execute:

**python**

```

agent = create_agent(
 model="gpt-5.5",
 middleware=[middleware1, middleware2, middleware3],
 tools=[...],
)

```

js

```

const agent = createAgent({
 model: "gpt-5.5",
 middleware: [middleware1, middleware2, middleware3],
 tools: [...],
});

```

## Execution flow

**Before hooks run in order:** 1. middleware1.before\_agent() 2. middleware2.before\_agent() 3. middleware3.before\_agent()

## Agent loop starts

4. middleware1.before\_model()
5. middleware2.before\_model()
6. middleware3.before\_model()

## Wrap hooks nest like function calls:

7. middleware1.wrap\_model\_call() → middleware2.wrap\_model\_call() →  
middleware3.wrap\_model\_call() → model

## After hooks run in reverse order:

8. middleware3.after\_model()
9. middleware2.after\_model()
10. middleware1.after\_model()

## Agent loop ends

11. middleware3.after\_agent()
12. middleware2.after\_agent()
13. middleware1.after\_agent()

**Key rules:** - before\_\* hooks: First to last - after\_\* hooks: Last to first (reverse) - wrap\_\* hooks: Nested (first middleware wraps all others)

# Agent jumps

To exit early from middleware, return a dictionary with `jump_to`:

**Available jump targets:** - `'end'`: Jump to the end of the agent execution (or the first `after_agent` hook) - `'tools'`: Jump to the tools node - `'model'`: Jump to the model node (or the first `before_model` hook)

## python

### Decorator

```
from langchain.agents.middleware import after_model, hook_config, AgentState
from langchain.messages import AIMessage
from langgraph.runtime import Runtime
from typing import Any

@after_model
@hook_config(can_jump_to=["end"])
def check_for_blocked(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 last_message = state["messages"][-1]
 if "BLOCKED" in last_message.content:
 return {
 "messages": [AIMessage("I cannot respond to that request.")],
 "jump_to": "end"
 }
 return None
```

### Class

```
from langchain.agents.middleware import AgentMiddleware, hook_config, AgentState
from langchain.messages import AIMessage
from langgraph.runtime import Runtime
from typing import Any

class BlockedContentMiddleware(AgentMiddleware):
 @hook_config(can_jump_to=["end"])
 def after_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
 last_message = state["messages"][-1]
 if "BLOCKED" in last_message.content:
 return {
 "messages": [AIMessage("I cannot respond to that request.")],
 "jump_to": "end"
 }
 return None
```

js

```
import { createAgent, createMiddleware, AIMessage } from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 middleware: [
 createMiddleware({
 name: "BlockedContentMiddleware",
 beforeModel: {
 canJumpTo: ["end"],
 hook: (state) => {
 if (state.messages.at(-1)?.content.includes("BLOCKED")) {
 return {
 messages: [new AIMessage("I cannot respond to that request.")],
 jumpTo: "end" as const,
 };
 }
 return;
 },
 },
 }),
],
});

const result = await agent.invoke({
 messages: "Hello, world! BLOCKED"
});

/**
 * Expected output:
 * I cannot respond to that request.
 */
console.log(result.messages.at(-1)?.content);
```

python

## Configure tracing

### 참고

Requires `langchain>=1.3.15`.

Middleware hook spans trace their inputs and outputs by default. Set a `trace_policy` to shape what they record. `TracePolicy` accepts callables such as `process_inputs` and `process_outputs` that transform the traced value; `omit_payload` drops it entirely. This



can be useful as an optimization when the full message history is uninformative to the function of the middleware.

To omit the input payload from a middleware's trace:

```
from langchain.agents.middleware import AgentMiddleware, TracePolicy, omit_payload

class MyMiddleware(AgentMiddleware):
 trace_policy = TracePolicy(process_inputs=omit_payload)
```

To apply a policy to all middleware, configure a global default:

```
from langchain.agents.middleware import configure_trace_policy, TracePolicy, omit_payload

configure_trace_policy(TracePolicy(process_inputs=omit_payload)) # pass None to clear
```

A middleware's own `trace_policy` overrides the global default.

## Best practices

1. Keep middleware focused - each should do one thing well
2. Handle errors gracefully - don't let middleware errors crash the agent
3. **Use appropriate hook types:**
  - Node-style for sequential logic (logging, validation)
  - Wrap-style for control flow (retry, fallback, caching)
4. Clearly document any custom state properties
5. Unit test middleware independently before integrating
6. Consider execution order - place critical middleware first in the list
7. Use built-in middleware when possible

## Examples

### Dynamic prompt

Dynamically modify the system prompt at runtime to inject context, user-specific instructions, or other information before each model call. This is one of the most common middleware use cases.

#### python

Use the `system_message` field on `ModelRequest` to read and modify the system prompt. It contains a `@[ SystemMessage ]` object (even if the agent was created with a string

`system_prompt` ).

## Decorator

## Class

### 참고

- `ModelRequest.system_message` is always a `@[ SystemMessage ]` object, even if the agent was created with `system_prompt="string"`
- Use `SystemMessage.content_blocks` to access content as a list of blocks, regardless of whether the original content was a string or list
- When modifying system messages, use `content_blocks` and append new blocks to preserve existing structure
- You can pass `@[ SystemMessage ]` objects directly to `create_agent`'s `system_prompt` parameter for advanced use cases like cache control

### js

Use the `systemMessage` field in `ModelRequest` to read and modify the system prompt. It contains a `@[ SystemMessage ]` object (even if the agent was created with a string `@[ systemPrompt ]`).

Use `@[ SystemMessage.concat ]` to preserve cache control metadata or structured content blocks created by other middleware.

## Dynamic model selection

### python

## Decorator

## Class

### js

## Dynamically selecting tools

Select relevant tools at runtime to improve performance and accuracy. This section covers filtering pre-registered tools. For registering tools that are discovered at runtime (e.g., from MCP servers), see [Runtime tool registration](#).

**Benefits:** - **Shorter prompts** - Reduce complexity by exposing only relevant tools - **Better accuracy** - Models choose correctly from fewer options - **Permission control** - Dynamically filter tools based on user access

## python

### Decorator

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def select_tools(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 """Middleware to select relevant tools based on state/context."""
 # Select a small, relevant subset of tools based on state/context
 relevant_tools = select_relevant_tools(request.state, request.runtime)
 return handler(request.override(tools=relevant_tools))

agent = create_agent(
 model="gpt-5.5",
 tools=all_tools, # All available tools need to be registered upfront
 middleware=[select_tools],
)
```

### Class

```
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ModelResponse
from typing import Callable

class ToolSelectorMiddleware(AgentMiddleware):
 def wrap_model_call(
 self,
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 """Middleware to select relevant tools based on state/context."""
 # Select a small, relevant subset of tools based on state/context
 relevant_tools = select_relevant_tools(request.state, request.runtime)
 return handler(request.override(tools=relevant_tools))

agent = create_agent(
 model="gpt-5.5",
```

```
tools=all_tools, # All available tools need to be registered upfront
middleware=[ToolSelectorMiddleware()],
)
```

js

```
import { createAgent, createMiddleware } from "langchain";

const toolSelectorMiddleware = createMiddleware({
 name: "ToolSelector",
 wrapModelCall: (request, handler) => {
 // Select a small, relevant subset of tools based on state/context
 const relevantTools = selectRelevantTools(request.state, request.runtime);
 const modifiedRequest = { ...request, tools: relevantTools };
 return handler(modifiedRequest);
 },
});

const agent = createAgent({
 model: "gpt-5.5",
 tools: allTools,
 middleware: [toolSelectorMiddleware],
});
```

## Tool call monitoring

python

Decorator

Class

js

## Prompt caching (Anthropic)

When working with Anthropic models, use structured content blocks with cache control directives to cache large system prompts:

Decorator

```
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.messages import SystemMessage
from typing import Callable
```

```

@wrap_model_call
def add_cached_context(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 # Always work with content blocks
 new_content = list(request.system_message.content_blocks) + [
 {
 "type": "text",
 "text": "Here is a large document to analyze:\n\n<document>...</document>",
 # content up until this point is cached
 "cache_control": {"type": "ephemeral"}
 }
]

 new_system_message = SystemMessage(content=new_content)
 return handler(request.override(system_message=new_system_message))

```

## Class

```

from langchain.agents.middleware import AgentMiddleware, ModelRequest, ModelResponse
from langchain.messages import SystemMessage
from typing import Callable

class CachedContextMiddleware(AgentMiddleware):
 def wrap_model_call(
 self,
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 # Always work with content blocks
 new_content = list(request.system_message.content_blocks) + [
 {
 "type": "text",
 "text": "Here is a large document to analyze:\n\n<document>...</document>",
 "cache_control": {"type": "ephemeral"} # This content will be cached
 }
]

 new_system_message = SystemMessage(content=new_content)
 return handler(request.override(system_message=new_system_message))

```

**Notes:** - `ModelRequest.system_message` is always a `@[ SystemMessage ]` object, even if the agent was created with `system_prompt="string"` - Use `SystemMessage.content_blocks` to access content as a list of blocks, regardless of whether the original content was a string or list - When modifying system messages, use `content_blocks` and append new blocks to preserve

existing structure - You can pass `@[ SystemMessage ]` objects directly to `create_agent`'s `system_prompt` parameter for advanced use cases like cache control

js

Modify system messages in middleware using the `systemMessage` field in `ModelRequest`. It contains a `@[ SystemMessage ]` object (even if the agent was created with a string `@[ systemPrompt ]`).

**Example: Chaining middleware** - Different middleware can use different approaches:

```
import { createMiddleware, SystemMessage, createAgent } from "langchain";

// Middleware 1: Uses systemMessage with simple concatenation
const myMiddleware = createMiddleware({
 name: "MyMiddleware",
 wrapModelCall: async (request, handler) => {
 return handler({
 ...request,
 systemMessage: request.systemMessage.concat(`Additional context.`),
 });
 },
});

// Middleware 2: Uses systemMessage with structured content (preserves structure)
const myOtherMiddleware = createMiddleware({
 name: "MyOtherMiddleware",
 wrapModelCall: async (request, handler) => {
 return handler({
 ...request,
 systemMessage: request.systemMessage.concat(
 new SystemMessage({
 content: [
 {
 type: "text",
 text: " More additional context. This will be cached.",
 cache_control: { type: "ephemeral", ttl: "5m" },
 },
],
 })
),
 });
 },
});

const agent = createAgent({
 model: "google:gemin-3.6-flash",
 systemPrompt: "You are a helpful assistant.",
});
```

```
middleware: [myMiddleware, myOtherMiddleware],
});
```

The resulting system message will be:

```
new SystemMessage({
 content: [
 { type: "text", text: "You are a helpful assistant." },
 { type: "text", text: "Additional context." },
 {
 type: "text",
 text: " More additional context. This will be cached.",
 cache_control: { type: "ephemeral", ttl: "5m" },
 },
],
});
```

Use `@[ SystemMessage.concat ]` to preserve cache control metadata or structured content blocks created by other middleware.

## Additional resources

---

- [Middleware API reference](#)
- [Built-in middleware](#)
- [Testing agents](#)

# Middleware overview

langchain/middleware/overview.mdx

Middleware provides a way to more tightly control what happens inside the agent. Middleware is useful for the following:

- Tracking agent behavior with logging, analytics, and debugging.
- Transforming prompts, [tool selection](#), and output formatting.
- Adding [retries](#), [fallbacks](#), and early termination logic.
- Applying [rate limits](#), guardrails, and [PII detection](#).

## python

Add middleware by passing them to `@[ create_agent ]`:

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
 model="gpt-5.5",
 tools=[...],
 middleware=[
 SummarizationMiddleware(...),
 HumanInTheLoopMiddleware(...)
],
)
```

## js

Add middleware by passing them to `createAgent`:

```
import {
 createAgent,
 summarizationMiddleware,
 humanInTheLoopMiddleware,
} from "langchain";

const agent = createAgent({
 model: "gpt-5.5",
 tools: [...],
 middleware: [summarizationMiddleware, humanInTheLoopMiddleware],
});
```



## The agent loop

The core agent loop involves calling a model, letting it choose tools to execute, and then finishing when it calls no more tools:

 Core agent loop diagram

Middleware exposes hooks before and after each of those steps:

 Middleware flow diagram

## Use middleware inside a LangGraph workflow

Middleware is not a separate runtime: hooks run inside the compiled [LangGraph](#) that `@[ create_agent ]` returns. You can drop the whole agent (middleware and all) into a larger `@[StateGraph]` as a node or subgraph, and every middleware hook continues to run.

Reach for this pattern when the surrounding topology is more than a standard "loop until done": classifying input before routing to one of several agents, fanning out work in parallel, or stitching agent calls together with deterministic steps.

`HumanInTheLoopMiddleware` matches against each tool's `.name`.

### python

`@tool`-decorated functions take their name from the function, so the key below is `"send_email"`.

```
from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.graph import START, StateGraph

Assumes read_email, send_email, classify_node, and route are defined elsewhere.
email_agent = create_agent(
 model="claude-sonnet-4-6",
 tools=[read_email, send_email],
 middleware=[HumanInTheLoopMiddleware(interrupt_on={"send_email": True})],
)

graph = (
 StateGraph(AgentState)
 .add_node("classify", classify_node)
 .add_node("email_agent", email_agent)
 .add_edge(START, "classify")
 .add_conditional_edges("classify", route)
 .compile()
)
```

js

The key matches the `name` you pass to `tool({...}, { name })`.

```
import { AgentState, createAgent, humanInTheLoopMiddleware } from "langchain";
import { StateGraph, START } from "@langchain/langgraph";

// Assumes readEmail, sendEmail, classifyNode, and route are defined elsewhere.
// readEmail / sendEmail are registered with name: "read_email" / "send_email".
const emailAgent = createAgent({
 model: "claude-sonnet-4-6",
 tools: [readEmail, sendEmail],
 middleware: [humanInTheLoopMiddleware({ interruptOn: { send_email: true } })],
});

const graph = new StateGraph(AgentState)
 .addNode("classify", classifyNode)
 .addNode("emailAgent", emailAgent)
 .addEdge(START, "classify")
 .addConditionalEdges("classify", route)
 .compile();
```

The HITL interrupt, summarization, PII redaction, retries, and any custom hooks all travel with the agent node. See [Use subgraphs](#) for the full set of composition patterns, including subgraph checkpointing scoping (per-invocation versus per-thread).

## Additional resources

---

### [Built-in middleware](#)

Explore built-in middleware for common use cases.

### [Custom middleware](#)

Build your own middleware with hooks and decorators.

### [Middleware API reference](#)

Complete API reference for middleware.

### [Middleware integrations](#)

Provider-specific middleware for Anthropic, AWS, OpenAI, and more.

### [Testing agents](#)

Test your agents with LangSmith.

**langchain / multi-agent**

# Custom workflow

langchain/multi-agent/custom-workflow.mdx

In the **custom workflow** architecture, you define your own bespoke execution flow using [LangGraph](#). You have complete control over the graph structure—including sequential steps, conditional branches, loops, and parallel execution.

```
graph LR
 A([Input]) --> B{{Conditional}}
 B -->|path_a| C[Deterministic step]
 B -->|path_b| D((Agentic step))
 C --> G([Output])
 D --> G([Output])

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef decision fill:#FDF3FF,stroke:#7E65AE,stroke-width:2px,color:#504B5F

class A,G trigger
class C,D process
class B decision
```

## Key characteristics

- Complete control over graph structure
- Mix deterministic logic with agentic behavior
- Support for sequential steps, conditional branches, loops, and parallel execution
- Embed other patterns as nodes in your workflow

## When to use

Use custom workflows when standard patterns (subagents, skills, etc.) don't fit your requirements, you need to mix deterministic logic with agentic behavior, or your use case requires complex routing or multi-stage processing.

Each node in your workflow can be a simple function, an LLM call, or an entire [agent](#) with [tools](#). You can also compose other architectures within a custom workflow—for example, embedding a multi-agent system as a single node.

For a complete example of a custom workflow, see the tutorial below.

[Tutorial: Build a multi-source knowledge base with routing](#)

The [router pattern](/oss/langchain/multi-agent/router) is an example of a custom workflow. This tutorial walks through building a router that queries GitHub, Notion, and Slack in parallel, then synthesizes results.

## Basic implementation

The core insight is that you can call a LangChain agent directly inside any LangGraph node, combining the flexibility of custom workflows with the convenience of prebuilt agents:

### python

```
from langchain.agents import create_agent
from langgraph.graph import StateGraph, START, END

agent = create_agent(model="openai:gpt-5.5", tools=[...])

def agent_node(state: State) -> dict:
 """A LangGraph node that invokes a LangChain agent."""
 result = agent.invoke({
 "messages": [{"role": "user", "content": state["query"]}]}
 })
 return {"answer": result["messages"][-1].content}

Build a simple workflow
workflow = (
 StateGraph(State)
 .add_node("agent", agent_node)
 .add_edge(START, "agent")
 .add_edge("agent", END)
 .compile()
)
```

### js

```
import { z } from "zod";
import { createAgent } from "langchain";
import { StateGraph, START, END, StateSchema, MessagesValue } from "@langchain/langgraph";

const agent = createAgent({ model: "openai:gpt-5.5", tools: [...] });

const AgentState = new StateSchema({
 messages: MessagesValue,
 query: z.string(),
});
```

```
const agentNode: GraphNode<typeof AgentState> = (state) => {
 // A LangGraph node that invokes a LangChain agent
 const result = await agent.invoke({
 messages: [{ role: "user", content: state.query }]
 });
 return { answer: result.messages.at(-1)?.content };
}

// Build a simple workflow
const workflow = new StateGraph(State)
 .addNode("agent", agentNode)
 .addEdge(START, "agent")
 .addEdge("agent", END)
 .compile();
```

## Example: RAG pipeline

A common use case is combining [retrieval](#) with an agent. This example builds a WNBA stats assistant that retrieves from a knowledge base and can fetch live news.

### Custom RAG workflow

The workflow demonstrates three types of nodes:

- **Model node** (Rewrite): Rewrites the user query for better retrieval using [structured output](#).
- **Deterministic node** (Retrieve): Performs vector similarity search — no LLM involved.
- **Agent node** (Agent): Reasons over retrieved context and can fetch additional information via tools.

```
graph LR
 A([Query]) --> B{{Rewrite}}
 B --> C[(Retrieve)]
 C --> D((Agent))
 D --> E([Response])

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,E trigger
class B,C,D process
```

팁

You can use LangGraph state to pass information between workflow steps. This allows each part of your workflow to read and update structured fields, making it easy to share data and context across nodes.

## python

```
from typing import TypedDict
from pydantic import BaseModel
from langgraph.graph import StateGraph, START, END
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

class State(TypedDict):
 question: str
 rewritten_query: str
 documents: list[str]
 answer: str

WNBA knowledge base with rosters, game results, and player stats
embeddings = OpenAIEmbeddings()
vector_store = InMemoryVectorStore(embeddings)
vector_store.add_texts([
 # Rosters
 "New York Liberty 2024 roster: Breanna Stewart, Sabrina Ionescu, Jonquel Jones, Courtney Vandersloot.",
 "Las Vegas Aces 2024 roster: A'ja Wilson, Kelsey Plum, Jackie Young, Chelsea Gray.",
 "Indiana Fever 2024 roster: Caitlin Clark, Aliyah Boston, Kelsey Mitchell, NaLyssa Smith.",
 # Game results
 "2024 WNBA Finals: New York Liberty defeated Minnesota Lynx 3-2 to win the championship.",
 "June 15, 2024: Indiana Fever 85, Chicago Sky 79. Caitlin Clark had 23 points and 8 assists.",
 "August 20, 2024: Las Vegas Aces 92, Phoenix Mercury 84. A'ja Wilson scored 35 points.",
 # Player stats
 "A'ja Wilson 2024 season stats: 26.9 PPG, 11.9 RPG, 2.6 BPG. Won MVP award.",
 "Caitlin Clark 2024 rookie stats: 19.2 PPG, 8.4 APG, 5.7 RPG. Won Rookie of the Year.",
 "Breanna Stewart 2024 stats: 20.4 PPG, 8.5 RPG, 3.5 APG.",
])
retriever = vector_store.as_retriever(search_kwargs={"k": 5})

@tool
def get_latest_news(query: str) -> str:
 """Get the latest WNBA news and updates."""
```

```

Your news API here
return "Latest: The WNBA announced expanded playoff format for 2025..."

agent = create_agent(
 model="openai:gpt-5.5",
 tools=[get_latest_news],
)

model = ChatOpenAI(model="gpt-5.5")

class RewrittenQuery(BaseModel):
 query: str

def rewrite_query(state: State) -> dict:
 """Rewrite the user query for better retrieval."""
 system_prompt = """Rewrite this query to retrieve relevant WNBA information.
The knowledge base contains: team rosters, game results with scores, and player statistics (PPG, RPG, APG).
Focus on specific player names, team names, or stat categories mentioned."""
 response = model.with_structured_output(RewrittenQuery).invoke([
 {"role": "system", "content": system_prompt},
 {"role": "user", "content": state["question"]}
])
 return {"rewritten_query": response.query}

def retrieve(state: State) -> dict:
 """Retrieve documents based on the rewritten query."""
 docs = retriever.invoke(state["rewritten_query"])
 return {"documents": [doc.page_content for doc in docs]}

def call_agent(state: State) -> dict:
 """Generate answer using retrieved context."""
 context = "\n\n".join(state["documents"])
 prompt = f"Context:\n{context}\n\nQuestion: {state['question']}"
 response = agent.invoke({"messages": [{"role": "user", "content": prompt}]})
 return {"answer": response["messages"][-1].content_blocks}

workflow = (
 StateGraph(State)
 .add_node("rewrite", rewrite_query)
 .add_node("retrieve", retrieve)
 .add_node("agent", call_agent)
 .add_edge(START, "rewrite")
 .add_edge("rewrite", "retrieve")
 .add_edge("retrieve", "agent")
 .add_edge("agent", END)
 .compile()
)

```



```
result = workflow.invoke({"question": "Who won the 2024 WNBA Championship?"})
print(result["answer"])
```

## 정보

In production, use a persistent vector store such as [Valkey](#), [Databricks Vector Search](#), or [MongoDB Atlas](#) instead of `InMemoryVectorStore`. See [all vector stores](#).

js

```
import { StateGraph, Annotation, START, END } from "@langchain/langgraph";
import { createAgent, tool } from "langchain";
import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { MemoryVectorStore } from "@langchain/classic/vectorstores/memory";
import * as z from "zod";

const State = Annotation.Root({
 question: Annotation<string>(),
 rewrittenQuery: Annotation<string>(),
 documents: Annotation<string[]>(),
 answer: Annotation<string>(),
});

// WNBA knowledge base with rosters, game results, and player stats
const embeddings = new OpenAIEmbeddings();
const vectorStore = await MemoryVectorStore.fromTexts(
 [
 // Rosters
 "New York Liberty 2024 roster: Breanna Stewart, Sabrina Ionescu, Jonquel Jones, Courtney Vandersloot.",
 "Las Vegas Aces 2024 roster: A'ja Wilson, Kelsey Plum, Jackie Young, Chelsea Gray.",
 "Indiana Fever 2024 roster: Caitlin Clark, Aliyah Boston, Kelsey Mitchell, NaLyssa Smith.",
 // Game results
 "2024 WNBA Finals: New York Liberty defeated Minnesota Lynx 3-2 to win the championship.",
 "June 15, 2024: Indiana Fever 85, Chicago Sky 79. Caitlin Clark had 23 points and 8 assists.",
 "August 20, 2024: Las Vegas Aces 92, Phoenix Mercury 84. A'ja Wilson scored 35 points.",
 // Player stats
 "A'ja Wilson 2024 season stats: 26.9 PPG, 11.9 RPG, 2.6 BPG. Won MVP award.",
 "Caitlin Clark 2024 rookie stats: 19.2 PPG, 8.4 APG, 5.7 RPG. Won Rookie of the Year.",
 "Breanna Stewart 2024 stats: 20.4 PPG, 8.5 RPG, 3.5 APG.",
],
 [],
 [{}, {}, {}, {}, {}, {}, {}, {}, {}],
```

```

 embeddings
);
 const retriever = vectorStore.asRetriever({ k: 5 });

 const getLatestNews = tool(
 async ({ query }) => {
 // Your news API here
 return "Latest: The WNBA announced expanded playoff format for 2025...";
 },
 {
 name: "get_latest_news",
 description: "Get the latest WNBA news and updates",
 schema: z.object({ query: z.string() }),
 }
);

 const agent = createAgent({
 model: "openai:gpt-5.5",
 tools: [getLatestNews],
 });

 const model = new ChatOpenAI({ model: "gpt-5.5" });

 const RewrittenQuery = z.object({ query: z.string() });

 async function rewriteQuery(state: typeof State.State) {
 const systemPrompt = `Rewrite this query to retrieve relevant WNBA information.
 The knowledge base contains: team rosters, game results with scores, and player statistics (PPG, RPG, APG).
 Focus on specific player names, team names, or stat categories mentioned.`;
 const response = await model.withStructuredOutput(RewrittenQuery).invoke([
 { role: "system", content: systemPrompt },
 { role: "user", content: state.question },
]);
 return { rewrittenQuery: response.query };
 }

 async function retrieve(state: typeof State.State) {
 const docs = await retriever.invoke(state.rewrittenQuery);
 return { documents: docs.map((doc) => doc.pageContent) };
 }

 async function callAgent(state: typeof State.State) {
 const context = state.documents.join("\n\n");
 const prompt = `Context:\n${context}\n\nQuestion: ${state.question}`;
 const response = await agent.invoke({
 messages: [{ role: "user", content: prompt }],
 });
 return { answer: response.messages.at(-1)?.contentBlocks };
 }

```

```
const workflow = new StateGraph(State)
 .addNode("rewrite", rewriteQuery)
 .addNode("retrieve", retrieve)
 .addNode("agent", callAgent)
 .addEdge(START, "rewrite")
 .addEdge("rewrite", "retrieve")
 .addEdge("retrieve", "agent")
 .addEdge("agent", END)
 .compile();

const result = await workflow.invoke({
 question: "Who won the 2024 WNBA Championship?",
});
console.log(result.answer);
```

## 정보

In production, use a persistent vector store such as [Weaviate](#), [Pinecone](#), or [MongoDB Atlas](#) instead of `MemoryVectorStore`. See [all vector stores](#).

# Build customer support with handoffs

langchain/multi-agent/handoffs-customer-support.mdx

The [state machine pattern](#) describes workflows where an agent's behavior changes as it moves through different states of a task. This tutorial shows how to implement a state machine by using tool calls to dynamically change a single agent's configuration—updating its available tools and instructions based on the current state. The state can be determined from multiple sources: the agent's past actions (tool calls), external state (such as API call results), or even initial user input (for example, by running a classifier to determine user intent).

In this tutorial, you'll build a customer support agent that does the following:

- Collects warranty information before proceeding.
- Classifies issues as hardware or software.
- Provides solutions or escalates to human support.
- Maintains conversation state across multiple turns.

Unlike the [subagents pattern](#) where sub-agents are called as tools, the **state machine pattern** uses a single agent whose configuration changes based on workflow progress. Each "step" is just a different configuration (system prompt + tools) of the same underlying agent, selected dynamically based on state.

Here's the workflow we'll build:

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#E5F4FF','primaryTextColor':'#030710','primaryBorderColor':'#006DDD','lineColor':'#40668D','secondaryColor':'#F6FFDB','tertiaryColor':'#FDF3FF'}}}%
flowchart TD
 %% Start
 Start([👤 Customer reports
an issue]) --> Warranty{Is the device
under warranty?}

 %% Warranty check
 Warranty -->|✅ Yes| IssueType{What type
of issue?}
 Warranty -->|❌ No| OutOfWarranty{What type
of issue?}

 %% In-Warranty branch
 IssueType -->|🔧 Hardware| Repair[Provide warranty
repair instructions]
 IssueType -->|💻 Software| Troubleshoot[Provide troubleshooting
steps]

 %% Out-of-Warranty branch
 OutOfWarranty -->|🔧 Hardware| Escalate[Escalate to human
for paid repair options]
 OutOfWarranty -->|💻 Software| Troubleshoot

 %% Troubleshooting follow-up
 Troubleshoot --> Close([✅ Issue Resolved])
 Repair --> Close
```

```
Escalate --> Close
```

```
%% === Styling ===
```

```
classDef startEnd fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef decisionNode fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef actionNode fill:#FDF3FF,stroke:#7E65AE,stroke-width:2px,color:#504B5F
classDef escalateNode fill:#F8E8E6,stroke:#B27D75,stroke-width:2px,color:#634643

class Start,Close startEnd
class Warranty,IssueType,OutOfWarranty decisionNode
class Repair,Troubleshoot actionNode
class Escalate escalateNode
```

## Setup

### Installation

This tutorial requires the `langchain` package:

#### python

```
```bash pip pip install langchain
```

```
```bash uv
uv add langchain
```

```
```bash conda conda install langchain -c conda-forge
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```bash npm
npm install langchain
```

```
```bash yarn yarn add langchain
```

```
```bash pnpm
pnpm add langchain
```

For more details, see our [Installation guide](#).

## LangSmith

Set up [LangSmith](#) to inspect what is happening inside your agent. Then set the following environment variables:

### python

```
```bash Shell export LANGSMITH_TRACING="true" export LANGSMITH_API_KEY="..."
```

```
```python Python
import getpass
import os

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

### js

```
```bash Shell export LANGSMITH_TRACING="true" export LANGSMITH_API_KEY="..."
```

```
```typescript TypeScript
process.env.LANGSMITH_TRACING = "true";
process.env.LANGSMITH_API_KEY = "...";
```

## Select an LLM

Select a chat model from LangChain's suite of integrations:

### python

### js

## 1. Define custom state

First, define a custom state schema that tracks which step is currently active:

### python

```
from langchain.agents import AgentState
from typing_extensions import NotRequired
from typing import Literal

Define the possible workflow steps
SupportStep = Literal["warranty_collector", "issue_classifier", "resolution_specialis
```

```
t"] # [!code highlight]

class SupportState(AgentState): # [!code highlight]
 """State for customer support workflow."""
 current_step: NotRequired[SupportStep] # [!code highlight]
 warranty_status: NotRequired[Literal["in_warranty", "out_of_warranty"]]
 issue_type: NotRequired[Literal["hardware", "software"]]
```

js

```
import { StateSchema } from "@langchain/langgraph";
import { z } from "zod";

// Define the possible workflow steps
const SupportStepSchema = z.enum(["warranty_collector", "issue_classifier", "resolution_specialist"]); // [!code highlight]
const WarrantyStatusSchema = z.enum(["in_warranty", "out_of_warranty"]);
const IssueTypeSchema = z.enum(["hardware", "software"]);

// State for customer support workflow
const SupportState = new StateSchema({ // [!code highlight]
 currentStep: SupportStepSchema.optional(), // [!code highlight]
 warrantyStatus: WarrantyStatusSchema.optional(),
 issueType: IssueTypeSchema.optional(),
});
```

The `current_step` field is the core of the state machine pattern - it determines which configuration (prompt + tools) is loaded on each turn.

## 2. Create tools that manage workflow state

Create tools that update the workflow state. These tools allow the agent to record information and transition to the next step.

The key is using `Command` to update state, including the `current_step` field:

python

```
from langchain.tools import tool, ToolRuntime
from langchain.messages import ToolMessage
from langgraph.types import Command

@tool
def record_warranty_status(
 status: Literal["in_warranty", "out_of_warranty"],
 runtime: ToolRuntime[None, SupportState],
```

```

) -> Command: # [!code highlight]
 """Record the customer's warranty status and transition to issue classification."""
 return Command(# [!code highlight]
 update={ # [!code highlight]
 "messages": [
 ToolMessage(
 content=f"Warranty status recorded as: {status}",
 tool_call_id=runtime.tool_call_id,
)
],
 "warranty_status": status,
 "current_step": "issue_classifier", # [!code highlight]
 }
)

@tool
def record_issue_type(
 issue_type: Literal["hardware", "software"],
 runtime: ToolRuntime[None, SupportState],
) -> Command: # [!code highlight]
 """Record the type of issue and transition to resolution specialist."""
 return Command(# [!code highlight]
 update={ # [!code highlight]
 "messages": [
 ToolMessage(
 content=f"Issue type recorded as: {issue_type}",
 tool_call_id=runtime.tool_call_id,
)
],
 "issue_type": issue_type,
 "current_step": "resolution_specialist", # [!code highlight]
 }
)

@tool
def escalate_to_human(reason: str) -> str:
 """Escalate the case to a human support specialist."""
 # In a real system, this would create a ticket, notify staff, etc.
 return f"Escalating to human support. Reason: {reason}"

@tool
def provide_solution(solution: str) -> str:
 """Provide a solution to the customer's issue."""
 return f"Solution provided: {solution}"

```

js

```

import { z } from "zod";
import { tool, ToolMessage, type ToolRuntime } from "langchain";

```



```

import { Command } from "@langchain/langgraph";

const recordWarrantyStatus = tool(
 async (input, config: ToolRuntime<typeof SupportState.State>) => {
 return new Command({ // [!code highlight]
 update: { // [!code highlight]
 messages: [
 new ToolMessage({
 content: `Warranty status recorded as: ${input.status}`,
 tool_call_id: config.toolCallId,
 }),
],
 warrantyStatus: input.status,
 currentStep: "issue_classifier", // [!code highlight]
 },
 });
 },
);

{
 name: "record_warranty_status",
 description:
 "Record the customer's warranty status and transition to issue classification.",
 schema: z.object({
 status: WarrantyStatusSchema,
 }),
}

);

const recordIssueType = tool(
 async (input, config: ToolRuntime<typeof SupportState.State>) => {
 return new Command({ // [!code highlight]
 update: { // [!code highlight]
 messages: [
 new ToolMessage({
 content: `Issue type recorded as: ${input.issueType}`,
 tool_call_id: config.toolCallId,
 }),
],
 issueType: input.issueType,
 currentStep: "resolution_specialist", // [!code highlight]
 },
 });
 },
);

{
 name: "record_issue_type",
 description:
 "Record the type of issue and transition to resolution specialist.",
 schema: z.object({
 issueType: IssueTypeSchema,
 }),
}

);

```

```

const escalateToHuman = tool(
 async (input) => {
 // In a real system, this would create a ticket, notify staff, etc.
 return `Escalating to human support. Reason: ${input.reason}`;
 },
 {
 name: "escalate_to_human",
 description: "Escalate the case to a human support specialist.",
 schema: z.object({
 reason: z.string(),
 }),
 }
);

const provideSolution = tool(
 async (input) => {
 return `Solution provided: ${input.solution}`;
 },
 {
 name: "provide_solution",
 description: "Provide a solution to the customer's issue.",
 schema: z.object({
 solution: z.string(),
 }),
 }
);

```

Notice how `record_warranty_status` and `record_issue_type` return `Command` objects that update both the data (`warranty_status`, `issue_type`) AND the `current_step`. This is how the state machine works - tools control workflow progression.

### 3. Define step configurations

Define prompts and tools for each step. First, define the prompts for each step:

**View complete prompt definitions**

#### python

```

Define prompts as constants for easy reference
WARRANTY_COLLECTOR_PROMPT = """You are a customer support agent helping with device issues.

CURRENT STAGE: Warranty verification

At this step, you need to:
1. Greet the customer warmly

```

```

2. Ask if their device is under warranty
3. Use record_warranty_status to record their response and move to the next step

Be conversational and friendly. Don't ask multiple questions at once."""

ISSUE_CLASSIFIER_PROMPT = """You are a customer support agent helping with device issues.

CURRENT STAGE: Issue classification
CUSTOMER INFO: Warranty status is {warranty_status}

At this step, you need to:
1. Ask the customer to describe their issue
2. Determine if it's a hardware issue (physical damage, broken parts) or software issue (app crashes, performance)
3. Use record_issue_type to record the classification and move to the next step

If unclear, ask clarifying questions before classifying."""

RESOLUTION_SPECIALIST_PROMPT = """You are a customer support agent helping with device issues.

CURRENT STAGE: Resolution
CUSTOMER INFO: Warranty status is {warranty_status}, issue type is {issue_type}

At this step, you need to:
1. For SOFTWARE issues: provide troubleshooting steps using provide_solution
2. For HARDWARE issues:
 - If IN WARRANTY: explain warranty repair process using provide_solution
 - If OUT OF WARRANTY: escalate_to_human for paid repair options

Be specific and helpful in your solutions."""

```

js

```

// Define prompts as constants for easy reference
const WARRANTY_COLLECTOR_PROMPT = `You are a customer support agent helping with device issues.

CURRENT STAGE: Warranty verification

At this step, you need to:
1. Greet the customer warmly
2. Ask if their device is under warranty
3. Use record_warranty_status to record their response and move to the next step

Be conversational and friendly. Don't ask multiple questions at once.`;

const ISSUE_CLASSIFIER_PROMPT = `You are a customer support agent helping with device i

```

issues.

CURRENT STAGE: Issue classification

CUSTOMER INFO: Warranty status is {warranty\_status}

At this step, you need to:

1. Ask the customer to describe their issue
2. Determine if it's a hardware issue (physical damage, broken parts) or software issue (app crashes, performance)
3. Use record\_issue\_type to record the classification and move to the next step

If unclear, ask clarifying questions before classifying.`;

const RESOLUTION\_SPECIALIST\_PROMPT = `You are a customer support agent helping with device issues.

CURRENT STAGE: Resolution

CUSTOMER INFO: Warranty status is {warranty\_status}, issue type is {issue\_type}

At this step, you need to:

1. For SOFTWARE issues: provide troubleshooting steps using provide\_solution
2. For HARDWARE issues:
  - If IN WARRANTY: explain warranty repair process using provide\_solution
  - If OUT OF WARRANTY: escalate\_to\_human for paid repair options

Be specific and helpful in your solutions.`;

Then map step names to their configurations using a dictionary:

## python

```
Step configuration: maps step name to (prompt, tools, required_state)
STEP_CONFIG = {
 "warranty_collector": {
 "prompt": WARRANTY_COLLECTOR_PROMPT,
 "tools": [record_warranty_status],
 "requires": [],
 },
 "issue_classifier": {
 "prompt": ISSUE_CLASSIFIER_PROMPT,
 "tools": [record_issue_type],
 "requires": ["warranty_status"],
 },
 "resolution_specialist": {
 "prompt": RESOLUTION_SPECIALIST_PROMPT,
 "tools": [provide_solution, escalate_to_human],
 "requires": ["warranty_status", "issue_type"],
 },
}
```

```
 },
 }
}
```

js

```
// Step configuration: maps step name to (prompt, tools, required_state)
const STEP_CONFIG = {
 warranty_collector: {
 prompt: WARRANTY_COLLECTOR_PROMPT,
 tools: [recordWarrantyStatus],
 requires: [],
 },
 issue_classifier: {
 prompt: ISSUE_CLASSIFIER_PROMPT,
 tools: [recordIssueType],
 requires: ["warrantyStatus"],
 },
 resolution_specialist: {
 prompt: RESOLUTION_SPECIALIST_PROMPT,
 tools: [provideSolution, escalateToHuman],
 requires: ["warrantyStatus", "issueType"],
 },
} as const;
```

This dictionary-based configuration makes it easy to: - See all steps at a glance - Add new steps (just add another entry) - Understand the workflow dependencies ( `requires` field) - Use prompt templates with state variables (e.g., `{warranty_status}`)

## 4. Create step-based middleware

Create middleware that reads `current_step` from state and applies the appropriate configuration. We'll use the `@wrap_model_call` decorator for a clean implementation:

python

```
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call # [!code highlight]
def apply_step_config(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 """Configure agent behavior based on the current step."""
 # Get current step (defaults to warranty_collector for first interaction)
 current_step = request.state.get("current_step", "warranty_collector") # [!code hi
```

```

ghlight]

Look up step configuration
stage_config = STEP_CONFIG[current_step] # [!code highlight]

Validate required state exists
for key in stage_config["requires"]:
 if request.state.get(key) is None:
 raise ValueError(f"{key} must be set before reaching {current_step}")

Format prompt with state values (supports {warranty_status}, {issue_type}, etc.)
system_prompt = stage_config["prompt"].format(**request.state)

Inject system prompt and step-specific tools
request = request.override(# [!code highlight]
 system_prompt=system_prompt, # [!code highlight]
 tools=stage_config["tools"], # [!code highlight]
)

return handler(request)

```

js

```

import { createMiddleware } from "langchain";

const applyStepMiddleware = createMiddleware({
 name: "applyStep",
 stateSchema: SupportState,
 wrapModelCall: async (request, handler) => {
 // Get current step (defaults to warranty_collector for first interaction)
 const currentStep = request.state.currentStep ?? "warranty_collector"; // [!code hi
ghlight]

 // Look up step configuration
 const stepConfig = STEP_CONFIG[currentStep]; // [!code highlight]

 // Validate required state exists
 for (const key of stepConfig.requires) {
 if (request.state[key] === undefined) {
 throw new Error(`${key} must be set before reaching ${currentStep}`);
 }
 }

 // Format prompt with state values (supports {warrantyStatus}, {issueType}, etc.)
 let systemPrompt: string = stepConfig.prompt;
 for (const [key, value] of Object.entries(request.state)) {
 systemPrompt = systemPrompt.replace(`${key}`, String(value ?? ""));
 }
 }
});

```

```
// Inject system prompt and step-specific tools
return handler({
 ...request, // [!code highlight]
 systemPrompt, // [!code highlight]
 tools: [...stepConfig.tools], // [!code highlight]
});
},
});
```

This middleware:

1. **Reads current step:** Gets `current_step` from state (defaults to "warranty\_collector").
2. **Looks up configuration:** Finds the matching entry in `STEP_CONFIG`.
3. **Validates dependencies:** Ensures required state fields exist.
4. **Formats prompt:** Injects state values into the prompt template.
5. **Applies configuration:** Overrides the system prompt and available tools.

The `request.override()` method is key - it allows us to dynamically change the agent's behavior based on state without creating separate agent instances.

## 5. Create the agent

Now create the agent with the step-based middleware and a checkpointer for state persistence:

**python**

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

Collect all tools from all step configurations
all_tools = [
 record_warranty_status,
 record_issue_type,
 provide_solution,
 escalate_to_human,
]

Create the agent with step-based configuration
agent = create_agent(
 model,
 tools=all_tools,
 state_schema=SupportState, # [!code highlight]
 middleware=[apply_step_config], # [!code highlight]
 checkpointer=InMemorySaver(), # [!code highlight]
)
```

js

```
import { createAgent } from "langchain";
import { MemorySaver } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

// Collect all tools from all step configurations
const allTools = [
 recordWarrantyStatus,
 recordIssueType,
 provideSolution,
 escalateToHuman,
];

// Initialize the model
const model = new ChatOpenAI({
 model: "gpt-5.4-mini",
 temperature: 0.7,
});

// Create the agent with step-based configuration
const agent = createAgent({
 model,
 tools: allTools,
 stateSchema: SupportState, // [!code highlight]
 middleware: [applyStepMiddleware], // [!code highlight]
 checkpointers: new MemorySaver(), // [!code highlight]
});
```

## 참고

**Why a checkpointer?** The checkpointer maintains state across conversation turns. Without it, the `current_step` state would be lost between user messages, breaking the workflow.

## 6. Test the workflow

Test the complete workflow:

python

```
from langchain.messages import HumanMessage
from langchain_core.utils.uuid import uuid7

Configuration for this conversation thread
thread_id = str(uuid7())
config = {"configurable": {"thread_id": thread_id}}
```



```

Turn 1: Initial message - starts with warranty_collector step
print("=== Turn 1: Warranty Collection ===")
result = agent.invoke(
 {"messages": [HumanMessage("Hi, my phone screen is cracked")]},
 config
)
for msg in result['messages']:
 msg.pretty_print()

Turn 2: User responds about warranty
print("\n=== Turn 2: Warranty Response ===")
result = agent.invoke(
 {"messages": [HumanMessage("Yes, it's still under warranty")]},
 config
)
for msg in result['messages']:
 msg.pretty_print()
print(f"Current step: {result.get('current_step')}")

Turn 3: User describes the issue
print("\n=== Turn 3: Issue Description ===")
result = agent.invoke(
 {"messages": [HumanMessage("The screen is physically cracked from dropping it")]},
 config
)
for msg in result['messages']:
 msg.pretty_print()
print(f"Current step: {result.get('current_step')}")

Turn 4: Resolution
print("\n=== Turn 4: Resolution ===")
result = agent.invoke(
 {"messages": [HumanMessage("What should I do?")]},
 config
)
for msg in result['messages']:
 msg.pretty_print()

```

js

```

import { HumanMessage } from "@langchain/core/messages";
import { v7 as uuid7 } from "uuid";

// Configuration for this conversation thread
const threadId = uuid7();
const config = { configurable: { thread_id: threadId } };

// Turn 1: Initial message - starts with warranty_collector step

```

```

console.log("=== Turn 1: Warranty Collection ===");
let result = await agent.invoke(
 { messages: [new HumanMessage("Hi, my phone screen is cracked")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}

// Turn 2: User responds about warranty
console.log("\n=== Turn 2: Warranty Response ===");
result = await agent.invoke(
 { messages: [new HumanMessage("Yes, it's still under warranty")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}
console.log(`Current step: ${result.currentStep}`);

// Turn 3: User describes the issue
console.log("\n=== Turn 3: Issue Description ===");
result = await agent.invoke(
 { messages: [new HumanMessage("The screen is physically cracked from dropping it")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}
console.log(`Current step: ${result.currentStep}`);

// Turn 4: Resolution
console.log("\n=== Turn 4: Resolution ===");
result = await agent.invoke(
 { messages: [new HumanMessage("What should I do?")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}

```

Expected flow: 1. **Warranty verification step:** Asks about warranty status 2. **Issue classification step:** Asks about the problem, determines it's hardware 3. **Resolution step:** Provides warranty repair instructions

## 7. Understanding state transitions

Let's trace what happens at each turn:

### Turn 1: Initial message

python

```
{
 "messages": [HumanMessage("Hi, my phone screen is cracked")],
 "current_step": "warranty_collector" # Default value
}
```

js

```
{
 messages: [new HumanMessage("Hi, my phone screen is cracked")],
 currentStep: "warranty_collector" // Default value
}
```

Middleware applies: - System prompt: WARRANTY\_COLLECTOR\_PROMPT - Tools:  
[record\_warranty\_status]

### Turn 2: After warranty recorded

python

Tool call: `record_warranty_status("in_warranty")` returns:

```
Command(update={
 "warranty_status": "in_warranty",
 "current_step": "issue_classifier" # State transition!
})
```

js

Tool call: `recordWarrantyStatus("in_warranty")` returns:

```
new Command({
 update: {
 warrantyStatus: "in_warranty",
 currentStep: "issue_classifier" // State transition!
 }
})
```

```
}
})
```

Next turn, middleware applies: - System prompt: `ISSUE_CLASSIFIER_PROMPT` (formatted with `warranty_status="in_warranty"`) - Tools: `[record_issue_type]`

### Turn 3: After issue classified

#### python

Tool call: `record_issue_type("hardware")` returns:

```
Command(update={
 "issue_type": "hardware",
 "current_step": "resolution_specialist" # State transition!
})
```

#### js

Tool call: `recordIssueType("hardware")` returns:

```
new Command({
 update: {
 issueType: "hardware",
 currentStep: "resolution_specialist" // State transition!
 }
})
```

Next turn, middleware applies: - System prompt: `RESOLUTION_SPECIALIST_PROMPT` (formatted with `warranty_status` and `issue_type`) - Tools: `[provide_solution, escalate_to_human]`

The key insight: **Tools drive the workflow** by updating `current_step`, and **middleware responds** by applying the appropriate configuration on the next turn.

## 8. Manage message history

As the agent progresses through steps, message history grows. Use [summarization middleware](#) to compress earlier messages while preserving conversational context:

#### python

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware # [!code highlight]
from langgraph.checkpoint.memory import InMemorySaver
```

```

agent = create_agent(
 model,
 tools=all_tools,
 state_schema=SupportState,
 middleware=[
 apply_step_config,
 SummarizationMiddleware(# [!code highlight]
 model="gpt-5.4-mini",
 trigger=("tokens", 4000),
 keep=("messages", 10)
)
],
 checkpointer=InMemorySaver(),
)

```

js

```

import { createAgent, SummarizationMiddleware } from "langchain"; // [!code highlight]
import { MemorySaver } from "@langchain/langgraph";

const agent = createAgent({
 model,
 tools: allTools,
 stateSchema: SupportState,
 middleware: [
 applyStepMiddleware,
 new SummarizationMiddleware({ // [!code highlight]
 model: "gpt-5.4-mini",
 trigger: { tokens: 4000 },
 keep: { messages: 10 },
 }),
],
 checkpointer: new MemorySaver(),
});

```

See the [short-term memory guide](#) for other memory management techniques.

## 9. Add flexibility: Go back

Some workflows need to allow users to return to previous steps to correct information (e.g., changing warranty status or issue classification). However, not all transitions make sense—for example, you typically can't go back once a refund has been processed. For this support workflow, we'll add tools to return to the warranty verification and issue classification steps.

팁

If your workflow requires arbitrary transitions between most steps, consider whether you need a structured workflow at all. This pattern works best when steps follow a clear sequential progression with occasional backwards transitions for corrections.

Add "go back" tools to the resolution step:

## python

```
@tool
def go_back_to_warranty() -> Command: # [!code highlight]
 """Go back to warranty verification step."""
 return Command(update={"current_step": "warranty_collector"}) # [!code highlight]

@tool
def go_back_to_classification() -> Command: # [!code highlight]
 """Go back to issue classification step."""
 return Command(update={"current_step": "issue_classifier"}) # [!code highlight]

Update the resolution_specialist configuration to include these tools
STEP_CONFIG["resolution_specialist"]["tools"].extend([
 go_back_to_warranty,
 go_back_to_classification
])
```

## js

```
import { tool } from "langchain";
import { Command } from "@langchain/langgraph";
import { z } from "zod";

const goBackToWarranty = tool(// [!code highlight]
 async () => {
 return new Command({ update: { currentStep: "warranty_collector" } }); // [!code highlight]
 },
 {
 name: "go_back_to_warranty",
 description: "Go back to warranty verification step.",
 schema: z.object({}),
 }
);

const goBackToClassification = tool(// [!code highlight]
 async () => {
 return new Command({ update: { currentStep: "issue_classifier" } }); // [!code highlight]
 },
 {
 name: "go_back_to_classification",
 description: "Go back to issue classification step.",
 schema: z.object({}),
 }
);
```

```

{
 name: "go_back_to_classification",
 description: "Go back to issue classification step.",
 schema: z.object({}),
}
);

// Update the resolution_specialist configuration to include these tools
STEP_CONFIG.resolution_specialist.tools.push(
 goBackToWarranty,
 goBackToClassification
);

```

Update the resolution specialist's prompt to mention these tools:

### python

```

RESOLUTION_SPECIALIST_PROMPT = """You are a customer support agent helping with device
issues.

CURRENT STAGE: Resolution
CUSTOMER INFO: Warranty status is {warranty_status}, issue type is {issue_type}

At this step, you need to:
1. For SOFTWARE issues: provide troubleshooting steps using provide_solution
2. For HARDWARE issues:
 - If IN WARRANTY: explain warranty repair process using provide_solution
 - If OUT OF WARRANTY: escalate_to_human for paid repair options

If the customer indicates any information was wrong, use:
- go_back_to_warranty to correct warranty status
- go_back_to_classification to correct issue type

Be specific and helpful in your solutions."""

```

### js

```

const RESOLUTION_SPECIALIST_PROMPT = `You are a customer support agent helping with dev
ice issues.

CURRENT STAGE: Resolution
CUSTOMER INFO: Warranty status is {warrantyStatus}, issue type is {issueType}

At this step, you need to:
1. For SOFTWARE issues: provide troubleshooting steps using provide_solution
2. For HARDWARE issues:
 - If IN WARRANTY: explain warranty repair process using provide_solution

```

- If OUT OF WARRANTY: `escalate_to_human` for paid repair options

If the customer indicates any information was wrong, use:

- `go_back_to_warranty` to correct warranty status
- `go_back_to_classification` to correct issue type

Be specific and helpful in your solutions.`;

Now the agent can handle corrections:

### python

```
result = agent.invoke(
 {"messages": [HumanMessage("Actually, I made a mistake - my device is out of warranty")]}],
 config
)
Agent will call go_back_to_warranty and restart the warranty verification step
```

### js

```
const result = await agent.invoke(
 { messages: [new HumanMessage("Actually, I made a mistake - my device is out of warranty")] },
 config
);
// Agent will call go_back_to_warranty and restart the warranty verification step
```

## Complete example

Here's everything together in a runnable script:

### Complete code

#### python

```
"""
Customer Support State Machine Example

This example demonstrates the state machine pattern.
A single agent dynamically changes its behavior based on the current_step state,
creating a state machine for sequential information collection.
"""

from langchain_core.utils.uuid import uuid7
```



```

from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from typing import Callable, Literal
from typing_extensions import NotRequired

from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse, SummarizationMiddleware
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, ToolMessage
from langchain.tools import tool, ToolRuntime

model = init_chat_model("google_genai:gemini-3.6-flash")

Define the possible workflow steps
SupportStep = Literal["warranty_collector", "issue_classifier", "resolution_specialist"]

class SupportState(AgentState):
 """State for customer support workflow."""

 current_step: NotRequired[SupportStep]
 warranty_status: NotRequired[Literal["in_warranty", "out_of_warranty"]]
 issue_type: NotRequired[Literal["hardware", "software"]]

 @tool
 def record_warranty_status(
 status: Literal["in_warranty", "out_of_warranty"],
 runtime: ToolRuntime[None, SupportState],
) -> Command:
 """Record the customer's warranty status and transition to issue classification."""
 return Command(
 update={
 "messages": [
 ToolMessage(
 content=f"Warranty status recorded as: {status}",
 tool_call_id=runtime.tool_call_id,
)
],
 "warranty_status": status,
 "current_step": "issue_classifier",
 }
)

 @tool
 def record_issue_type(
 issue_type: Literal["hardware", "software"],
 runtime: ToolRuntime[None, SupportState],
) -> Command:
 """Record the type of issue and transition to resolution specialist."""

```

```

 return Command(
 update={
 "messages": [
 ToolMessage(
 content=f"Issue type recorded as: {issue_type}",
 tool_call_id=runtime.tool_call_id,
)
],
 "issue_type": issue_type,
 "current_step": "resolution_specialist",
 }
)

@tool
def escalate_to_human(reason: str) -> str:
 """Escalate the case to a human support specialist."""
 # In a real system, this would create a ticket, notify staff, etc.
 return f"Escalating to human support. Reason: {reason}"

@tool
def provide_solution(solution: str) -> str:
 """Provide a solution to the customer's issue."""
 return f"Solution provided: {solution}"

Define prompts as constants
WARRANTY_COLLECTOR_PROMPT = """You are a customer support agent helping with device issues.

CURRENT STEP: Warranty verification

At this step, you need to:
1. Greet the customer warmly
2. Ask if their device is under warranty
3. Use record_warranty_status to record their response and move to the next step

Be conversational and friendly. Don't ask multiple questions at once."""

ISSUE_CLASSIFIER_PROMPT = """You are a customer support agent helping with device issues.

CURRENT STEP: Issue classification
CUSTOMER INFO: Warranty status is {warranty_status}

At this step, you need to:
1. Ask the customer to describe their issue
2. Determine if it's a hardware issue (physical damage, broken parts) or software issue (app crashes, performance)
3. Use record_issue_type to record the classification and move to the next step

If unclear, ask clarifying questions before classifying."""

```

```

RESOLUTION_SPECIALIST_PROMPT = """You are a customer support agent helping with device
issues.

CURRENT STEP: Resolution
CUSTOMER INFO: Warranty status is {warranty_status}, issue type is {issue_type}

At this step, you need to:
1. For SOFTWARE issues: provide troubleshooting steps using provide_solution
2. For HARDWARE issues:
 - If IN WARRANTY: explain warranty repair process using provide_solution
 - If OUT OF WARRANTY: escalate_to_human for paid repair options

Be specific and helpful in your solutions."""

Step configuration: maps step name to (prompt, tools, required_state)
STEP_CONFIG = {
 "warranty_collector": {
 "prompt": WARRANTY_COLLECTOR_PROMPT,
 "tools": [record_warranty_status],
 "requires": [],
 },
 "issue_classifier": {
 "prompt": ISSUE_CLASSIFIER_PROMPT,
 "tools": [record_issue_type],
 "requires": ["warranty_status"],
 },
 "resolution_specialist": {
 "prompt": RESOLUTION_SPECIALIST_PROMPT,
 "tools": [provide_solution, escalate_to_human],
 "requires": ["warranty_status", "issue_type"],
 },
}

@wrap_model_call
def apply_step_config(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
 """Configure agent behavior based on the current step."""
 # Get current step (defaults to warranty_collector for first interaction)
 current_step = request.state.get("current_step", "warranty_collector")

 # Look up step configuration
 step_config = STEP_CONFIG[current_step]

 # Validate required state exists
 for key in step_config["requires"]:
 if request.state.get(key) is None:
 raise ValueError(f"{key} must be set before reaching {current_step}")

 # Format prompt with state values

```

```

system_prompt = step_config["prompt"].format(**request.state)

Inject system prompt and step-specific tools
request = request.override(
 system_prompt=system_prompt,
 tools=step_config["tools"],
)

return handler(request)

Collect all tools from all step configurations
all_tools = [
 record_warranty_status,
 record_issue_type,
 provide_solution,
 escalate_to_human,
]

Create the agent with step-based configuration and summarization
agent = create_agent(
 model,
 tools=all_tools,
 state_schema=SupportState,
 middleware=[
 apply_step_config,
 SummarizationMiddleware(
 model="gpt-5.4-mini",
 trigger=("tokens", 4000),
 keep=("messages", 10)
)
],
 checkpointer=InMemorySaver(),
)

=====
Test the workflow
=====

if __name__ == "__main__":
 thread_id = str(uuid7())
 config = {"configurable": {"thread_id": thread_id}}

 result = agent.invoke(
 {"messages": [HumanMessage("Hi, my phone screen is cracked")]},
 config
)

 result = agent.invoke(
 {"messages": [HumanMessage("Yes, it's still under warranty")]},
 config
)

```

```

result = agent.invoke(
 {"messages": [HumanMessage("The screen is physically cracked from dropping it")]},
 config
)

result = agent.invoke(
 {"messages": [HumanMessage("What should I do?")]},
 config
)
for msg in result['messages']:
 msg.pretty_print()

```

js

```

import { createMiddleware, createAgent } from "langchain";

import { z } from "zod";
import { tool, ToolMessage, type ToolRuntime, HumanMessage } from "langchain";
import { Command, MemorySaver, StateSchema } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

// Define the possible workflow steps
const SupportStepSchema = z.enum([
 "warranty_collector",
 "issue_classifier",
 "resolution_specialist",
]);

const WarrantyStatusSchema = z.enum(["in_warranty", "out_of_warranty"]);
const IssueTypeSchema = z.enum(["hardware", "software"]);

// State for customer support workflow
const SupportState = new StateSchema({
 currentStep: SupportStepSchema.optional(),
 warrantyStatus: WarrantyStatusSchema.optional(),
 issueType: IssueTypeSchema.optional(),
});

const recordWarrantyStatus = tool(
 async (input, config: ToolRuntime<typeof SupportState.State>) => {
 return new Command({

update: {

 messages: [
 new ToolMessage({
 content: `Warranty status recorded as: ${input.status}`,
 tool_call_id: config.toolCallId,
 }),

```

```

],
 warrantyStatus: input.status,
 currentStep: "issue_classifier",
 },
});
},
{
 name: "record_warranty_status",
 description:
 "Record the customer's warranty status and transition to issue classification.",
 schema: z.object({
 status: WarrantyStatusSchema,
 }),
}
);

const recordIssueType = tool(
 async (input, config: ToolRuntime<typeof SupportState.State>) => {
 return new Command({

update: {

 messages: [
 new ToolMessage({
 content: `Issue type recorded as: ${input.issueType}`,
 tool_call_id: config.toolCallId,
 }),
],
 issueType: input.issueType,
 currentStep: "resolution_specialist",
 },
 });
},
{
 name: "record_issue_type",
 description:
 "Record the type of issue and transition to resolution specialist.",
 schema: z.object({
 issueType: IssueTypeSchema,
 }),
}
);

const escalateToHuman = tool(
 async (input) => {
 // In a real system, this would create a ticket, notify staff, etc.
 return `Escalating to human support. Reason: ${input.reason}`;
 },
 {
 name: "escalate_to_human",
 description: "Escalate the case to a human support specialist.",
 }
);

```

```

 schema: z.object({
 reason: z.string(),
 }),
 },
}
);

const provideSolution = tool(
 async (input) => {
 return `Solution provided: ${input.solution}`;
 },
 {
 name: "provide_solution",
 description: "Provide a solution to the customer's issue.",
 schema: z.object({
 solution: z.string(),
 }),
 }
);

// Define prompts as constants for easy reference
const WARRANTY_COLLECTOR_PROMPT = `You are a customer support agent helping with device issues.

CURRENT STAGE: Warranty verification

At this step, you need to:
1. Greet the customer warmly
2. Ask if their device is under warranty
3. Use record_warranty_status to record their response and move to the next step

Be conversational and friendly. Don't ask multiple questions at once.`;

const ISSUE_CLASSIFIER_PROMPT = `You are a customer support agent helping with device issues.

CURRENT STAGE: Issue classification
CUSTOMER INFO: Warranty status is {warranty_status}

At this step, you need to:
1. Ask the customer to describe their issue
2. Determine if it's a hardware issue (physical damage, broken parts) or software issue (app crashes, performance)
3. Use record_issue_type to record the classification and move to the next step

If unclear, ask clarifying questions before classifying.`;

const RESOLUTION_SPECIALIST_PROMPT = `You are a customer support agent helping with device issues.

CURRENT STAGE: Resolution
CUSTOMER INFO: Warranty status is {warranty_status}, issue type is {issue_type}

```

At this step, you need to:

1. For SOFTWARE issues: provide troubleshooting steps using `provide_solution`
2. For HARDWARE issues:
  - If IN WARRANTY: explain warranty repair process using `provide_solution`
  - If OUT OF WARRANTY: `escalate_to_human` for paid repair options

Be specific and helpful in your solutions.`;

```
// Step configuration: maps step name to (prompt, tools, required_state)
const STEP_CONFIG = {
 warranty_collector: {
 prompt: WARRANTY_COLLECTOR_PROMPT,
 tools: [recordWarrantyStatus],
 requires: [],
 },
 issue_classifier: {
 prompt: ISSUE_CLASSIFIER_PROMPT,
 tools: [recordIssueType],
 requires: ["warrantyStatus"],
 },
 resolution_specialist: {
 prompt: RESOLUTION_SPECIALIST_PROMPT,
 tools: [provideSolution, escalateToHuman],
 requires: ["warrantyStatus", "issueType"],
 },
} as const;

const applyStepMiddleware = createMiddleware({
 name: "applyStep",
 stateSchema: SupportState,
 wrapModelCall: async (request, handler) => {
 // Get current step (defaults to warranty_collector for first interaction)
 const currentStep = request.state.currentStep ?? "warranty_collector";

 // Look up step configuration
 const stepConfig = STEP_CONFIG[currentStep];

 // Validate required state exists
 for (const key of stepConfig.requires) {
 if (request.state[key] === undefined) {
 throw new Error(`${key} must be set before reaching ${currentStep}`);
 }
 }

 // Format prompt with state values (supports {warrantyStatus}, {issueType}, etc.)
 let systemPrompt: string = stepConfig.prompt;
 for (const [key, value] of Object.entries(request.state)) {
 systemPrompt = systemPrompt.replace(`${key}`, String(value ?? ""));
 }
 }
});
```



```

 // Inject system prompt and step-specific tools
 return handler({
 ...request,
 systemPrompt,
 tools: [...stepConfig.tools],
 });
 },
});

// Collect all tools from all step configurations
const allTools = [
 recordWarrantyStatus,
 recordIssueType,
 provideSolution,
 escalateToHuman,
];

const model = new ChatOpenAI({
 model: "gpt-5.4-mini",
});

// Create the agent with step-based configuration
const agent = createAgent({
 model,
 tools: allTools,
 middleware: [applyStepMiddleware],
 checkpointer: new MemorySaver(),
});

// Configuration for this conversation thread
const threadId = crypto.randomUUID();
const config = { configurable: { thread_id: threadId } };

// Turn 1: Initial message - starts with warranty_collector step
console.log("=== Turn 1: Warranty Collection ===");
let result = await agent.invoke(
 { messages: [new HumanMessage("Hi, my phone screen is cracked")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}

// Turn 2: User responds about warranty
console.log("\n=== Turn 2: Warranty Response ===");
result = await agent.invoke(
 { messages: [new HumanMessage("Yes, it's still under warranty")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}

```

```

}
console.log(`Current step: ${result.currentStep}`);

// Turn 3: User describes the issue
console.log("\n=== Turn 3: Issue Description ===");
result = await agent.invoke(
 {
 messages: [
 new HumanMessage("The screen is physically cracked from dropping it"),
],
 },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}
console.log(`Current step: ${result.currentStep}`);

// Turn 4: Resolution
console.log("\n=== Turn 4: Resolution ===");
result = await agent.invoke(
 { messages: [new HumanMessage("What should I do?")] },
 config
);
for (const msg of result.messages) {
 console.log(msg.content);
}

```

## Next steps

- Learn about the [subagents pattern](#) for centralized orchestration
- Explore [middleware](#) for more dynamic behaviors
- Read the [multi-agent overview](#) to compare patterns
- Use [LangSmith](#) to debug and monitor your multi-agent system

# Handoffs

langchain/multi-agent/handoffs.mdx

In the **handoffs** architecture, behavior changes dynamically based on state. The core mechanism: [tools](#) update a state variable (e.g., `current_step` or `active_agent`) that persists across turns, and the system reads this variable to adjust behavior—either applying different configuration (system prompt, tools) or routing to a different [agent](#). This pattern supports both handoffs between distinct agents and dynamic configuration changes within a single agent.

## 팁

The term **handoffs** was coined by [OpenAI](#) for using tool calls (e.g., `transfer_to_sales_agent`) to transfer control between agents or states.

```
sequenceDiagram
 participant User
 participant Agent
 participant Workflow State

 User->>Agent: "My phone is broken"
 Note over Agent,Workflow State: Step: Get warranty status
Tools: record_warranty_status
 Agent-->>User: "Is your device under warranty?"

 User->>Agent: "Yes, it's still under warranty"
 Agent->>Workflow State: record_warranty_status("in_warranty")
 Note over Agent,Workflow State: Step: Classify issue
Tools: record_issue_type
 Agent-->>User: "Can you describe the issue?"

 User->>Agent: "The screen is cracked"
 Agent->>Workflow State: record_issue_type("hardware")
 Note over Agent,Workflow State: Step: Provide resolution
Tools: provide_solution, escalate_to_human
 Agent-->>User: "Here's the warranty repair process..."
```

## Key characteristics

- State-driven behavior: Behavior changes based on a state variable (e.g., `current_step` or `active_agent`)
- Tool-based transitions: Tools update the state variable to move between states
- Direct user interaction: Each state's configuration handles user messages directly
- Persistent state: State survives across conversation turns

## When to use

Use the handoffs pattern when you need to enforce sequential constraints (unlock capabilities only after preconditions are met), the agent needs to converse directly with the user across different states, or you're building multi-stage conversational flows. This pattern is particularly valuable for customer support scenarios where you need to collect information in a specific sequence—for example, collecting a warranty ID before processing a refund.

## Basic implementation

The core mechanism is a [tool](#) that returns a [Command](#) to update state, triggering a transition to a new step or agent:

### python

```
from langchain.tools import tool
from langchain.messages import ToolMessage
from langgraph.types import Command

@tool
def transfer_to_specialist(runtime) -> Command:
 """Transfer to the specialist agent."""
 return Command(
 update={
 "messages": [
 ToolMessage(# [!code highlight]
 content="Transferred to specialist",
 tool_call_id=runtime.tool_call_id # [!code highlight]
)
],
 "current_step": "specialist" # Triggers behavior change
 }
)
```

### js

```
import { tool, ToolMessage, type ToolRuntime } from "langchain";
import { Command } from "@langchain/langgraph";
import { z } from "zod";

const transferToSpecialist = tool(
 async (_, config: ToolRuntime<typeof StateSchema>) => {
 return new Command({
 update: {
 messages: [
 new ToolMessage({ // [!code highlight]
```

```

 content: "Transferred to specialist",
 tool_call_id: config.toolCallId // [!code highlight]
 })
],
currentStep: "specialist" // Triggers behavior change
}
});
},
{
 name: "transfer_to_specialist",
 description: "Transfer to the specialist agent.",
 schema: z.object({})
}
);

```

## 참고

**Why include a `ToolMessage`?** When an LLM calls a tool, it expects a response. The `ToolMessage` with matching `tool_call_id` completes this request-response cycle—without it, the conversation history becomes malformed. This is required whenever your handoff tool updates messages.

For a complete implementation, see the tutorial below.

### [Tutorial: Build customer support with handoffs](#)

Learn how to build a customer support agent using the handoffs pattern, where a single agent transitions between different configurations.

## Implementation approaches

There are two ways to implement handoffs: [single agent with middleware](#) (one agent with dynamic configuration) or [multiple agent subgraphs](#) (distinct agents as graph nodes).

### Single agent with middleware

A single agent changes its behavior based on state. Middleware intercepts each model call and dynamically adjusts the system prompt and available tools. Tools update the state variable to trigger transitions:

#### python

```

from langchain.tools import ToolRuntime, tool
from langchain.messages import ToolMessage
from langgraph.types import Command

```

```

@tool
def record_warranty_status(
 status: str,
 runtime: ToolRuntime[None, SupportState]
) -> Command:
 """Record warranty status and transition to next step."""
 return Command(
 update={
 "messages": [
 ToolMessage(
 content=f"Warranty status recorded: {status}",
 tool_call_id=runtime.tool_call_id
)
],
 "warranty_status": status,
 "current_step": "specialist" # Update state to trigger transition
 }
)

```

js

```

import { tool, ToolMessage, type ToolRuntime } from "langchain";
import { Command } from "@langchain/langgraph";
import { z } from "zod";

const recordWarrantyStatus = tool(
 async ({ status }, config: ToolRuntime<typeof StateSchema>) => {
 return new Command({
 update: {
 messages: [
 new ToolMessage({
 content: `Warranty status recorded: ${status}`,
 tool_call_id: config.toolCallId,
 }),
],
 warrantyStatus: status,
 currentStep: "specialist", // Update state to trigger transition
 },
 });
 },
 {
 name: "record_warranty_status",
 description: "Record warranty status and transition to next step.",
 schema: z.object({
 status: z.string(),
 }),
 }
);

```

## Complete example: Customer support with middleware

### python

```
from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.tools import tool, ToolRuntime
from langchain.messages import ToolMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from typing import Callable

1. Define state with current_step tracker
class SupportState(AgentState): # [!code highlight]
 """Track which step is currently active."""
 current_step: str = "triage" # [!code highlight]
 warranty_status: str | None = None

2. Tools update current_step via Command
@tool
def record_warranty_status(
 status: str,
 runtime: ToolRuntime[None, SupportState]
) -> Command: # [!code highlight]
 """Record warranty status and transition to next step."""
 return Command(update={ # [!code highlight]
 "messages": [# [!code highlight]
 ToolMessage(
 content=f"Warranty status recorded: {status}",
 tool_call_id=runtime.tool_call_id
)
],
 "warranty_status": status,
 # Transition to next step
 "current_step": "specialist" # [!code highlight]
 })

3. Middleware applies dynamic configuration based on current_step
@wrap_model_call # [!code highlight]
def apply_step_config(
 request: ModelRequest,
 handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
 """Configure agent behavior based on current_step."""
 step = request.state.get("current_step", "triage") # [!code highlight]

Map steps to their configurations
configs = {
 "triage": {
 "prompt": "Collect warranty information...",

```

```

 "tools": [record_warranty_status]
 },
 "specialist": {
 "prompt": "Provide solutions based on warranty: {warranty_status}",
 "tools": [provide_solution, escalate]
 }
}

config = configs[step]
request = request.override(# [!code highlight]
 system_prompt=config["prompt"].format(**request.state), # [!code highlight]
 tools=config["tools"] # [!code highlight]
)
return handler(request)

4. Create agent with middleware
agent = create_agent(
 model,
 tools=[record_warranty_status, provide_solution, escalate],
 state_schema=SupportState,
 middleware=[apply_step_config], # [!code highlight]
 checkpointer=InMemorySaver() # Persist state across turns # [!code highlight]
)

```

js

```

import {
 createAgent,
 createMiddleware,
 tool,
 ToolMessage,
 type ToolRuntime,
} from "langchain";
import { Command, MemorySaver, StateSchema } from "@langchain/langgraph";
import { z } from "zod";

// 1. Define state with current_step tracker
const SupportState = new StateSchema({ // [!code highlight]
 currentStep: z.string().default("triage"), // [!code highlight]
 warrantyStatus: z.string().optional(),
});

// 2. Tools update currentStep via Command
const recordWarrantyStatus = tool(
 async ({ status }, config: ToolRuntime<typeof SupportState.State>) => {
 return new Command({ // [!code highlight]
 update: { // [!code highlight]
 messages: [// [!code highlight]
 new ToolMessage({

```



```

 content: `Warranty status recorded: ${status}`,
 tool_call_id: config.toolCallId,
 })),
],
warrantyStatus: status,
// Transition to next step
currentStep: "specialist", // [!code highlight]
},
));
},
{
 name: "record_warranty_status",
 description: "Record warranty status and transition",
 schema: z.object({ status: z.string() }),
}
);

// 3. Middleware applies dynamic configuration based on currentStep
const applyStepConfig = createMiddleware({
 name: "applyStepConfig",
 stateSchema: SupportState, // [!code highlight]
 wrapModelCall: async (request, handler) => {
 const step = request.state.currentStep || "triage"; // [!code highlight]

// Map steps to their configurations
const configs = {
 triage: {
 prompt: "Collect warranty information...",
 tools: [recordWarrantyStatus],
 },
 specialist: {
 prompt: `Provide solutions based on warranty: ${request.state.warrantyStatus}`,
 tools: [provideSolution, escalate],
 },
};

 const config = configs[step as keyof typeof configs];
 return handler({
 ...request,
 systemPrompt: config.prompt,
 tools: config.tools,
 });
},
});

// 4. Create agent with middleware
const agent = createAgent({
 model,
 tools: [recordWarrantyStatus, provideSolution, escalate],
 middleware: [applyStepConfig], // [!code highlight]
});

```

```
checkpointer: new MemorySaver(), // Persist state across turns // [!code highlight]
});
```

## Multiple agent subgraphs

Multiple distinct agents exist as separate nodes in a graph. Handoff tools navigate between agent nodes using `Command.PARENT` to specify which node to execute next.

### 주의

Subgraph handoffs require careful [context engineering](#). Unlike single-agent middleware (where message history flows naturally), you must explicitly decide what messages pass between agents. Get this wrong and agents receive malformed conversation history or bloated context. See [Context engineering](#) below.

### python

```
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool, ToolRuntime
from langgraph.types import Command

@tool
def transfer_to_sales(
 runtime: ToolRuntime,
) -> Command:
 """Transfer to the sales agent."""
 last_ai_message = next(# [!code highlight]
 msg for msg in reversed(runtime.state["messages"]) if isinstance(msg, AIMessag
e) # [!code highlight]
) # [!code highlight]
 transfer_message = ToolMessage(# [!code highlight]
 content="Transferred to sales agent", # [!code highlight]
 tool_call_id=runtime.tool_call_id, # [!code highlight]
) # [!code highlight]
 return Command(
 goto="sales_agent",
 update={
 "active_agent": "sales_agent",
 "messages": [last_ai_message, transfer_message], # [!code highlight]
 },
 graph=Command.PARENT
)
```

### js

```

import {
 tool,
 ToolMessage,
 AIMessage,
 type ToolRuntime,
} from "langchain";
import { Command, StateSchema, MessagesValue } from "@langchain/langgraph";

const CustomState = new StateSchema({
 messages: MessagesValue,
});

const transferToSales = tool(
 async (_, runtime: ToolRuntime<typeof CustomState.State>) => {
 const lastAiMessage = runtime.state.messages // [!code highlight]
 .reverse() // [!code highlight]
 .find(AIMessage.isInstance); // [!code highlight]

 const transferMessage = new ToolMessage({ // [!code highlight]
 content: "Transferred to sales agent", // [!code highlight]
 tool_call_id: runtime.toolCallId, // [!code highlight]
 }); // [!code highlight]
 return new Command({
 goto: "sales_agent",
 update: {
 activeAgent: "sales_agent",
 messages: [lastAiMessage, transferMessage].filter(Boolean), // [!code highlight]
 },
 graph: Command.PARENT,
 });
 },
 {
 name: "transfer_to_sales",
 description: "Transfer to the sales agent.",
 schema: z.object({}),
 }
);

```

## Complete example: Sales and support with handoffs

This example shows a multi-agent system with separate sales and support agents. Each agent is a separate graph node, and handoff tools allow agents to transfer conversations to each other.

**python**

```

from typing import Literal

from langchain.agents import AgentState, create_agent
from langchain.messages import AIMessage, ToolMessage
from langchain.tools import tool, ToolRuntime
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command
from typing_extensions import NotRequired

1. Define state with active_agent tracker
class MultiAgentState(AgentState):
 active_agent: NotRequired[str]

2. Create handoff tools
@tool
def transfer_to_sales(
 runtime: ToolRuntime,
) -> Command:
 """Transfer to the sales agent."""
 last_ai_message = next(# [!code highlight]
 msg for msg in reversed(runtime.state["messages"]) if isinstance(msg, AIMessag
e) # [!code highlight]
) # [!code highlight]
 transfer_message = ToolMessage(# [!code highlight]
 content="Transferred to sales agent from support agent", # [!code highlight]
 tool_call_id=runtime.tool_call_id, # [!code highlight]
) # [!code highlight]
 return Command(
 goto="sales_agent",
 update={
 "active_agent": "sales_agent",
 "messages": [last_ai_message, transfer_message], # [!code highlight]
 },
 graph=Command.PARENT,
)

@tool
def transfer_to_support(
 runtime: ToolRuntime,
) -> Command:
 """Transfer to the support agent."""
 last_ai_message = next(# [!code highlight]
 msg for msg in reversed(runtime.state["messages"]) if isinstance(msg, AIMessag
e) # [!code highlight]
) # [!code highlight]
 transfer_message = ToolMessage(# [!code highlight]
 content="Transferred to support agent from sales agent", # [!code highlight]
 tool_call_id=runtime.tool_call_id, # [!code highlight]
) # [!code highlight]
 return Command(

```

```

 goto="support_agent",
 update={
 "active_agent": "support_agent",
 "messages": [last_ai_message, transfer_message], # [!code highlight]
 },
 graph=Command.PARENT,
)

3. Create agents with handoff tools
sales_agent = create_agent(
 model="google_genai:gemini-3.6-flash",
 tools=[transfer_to_support],
 system_prompt="You are a sales agent. Help with sales inquiries. If asked about technical issues or support, transfer to the support agent.",
)

support_agent = create_agent(
 model="google_genai:gemini-3.6-flash",
 tools=[transfer_to_sales],
 system_prompt="You are a support agent. Help with technical issues. If asked about pricing or purchasing, transfer to the sales agent.",
)

4. Create agent nodes that invoke the agents
def call_sales_agent(state: MultiAgentState) -> Command:
 """Node that calls the sales agent."""
 response = sales_agent.invoke(state)
 return response

def call_support_agent(state: MultiAgentState) -> Command:
 """Node that calls the support agent."""
 response = support_agent.invoke(state)
 return response

5. Create router that checks if we should end or continue
def route_after_agent(
 state: MultiAgentState,
) -> Literal["sales_agent", "support_agent", "__end__"]:
 """Route based on active_agent, or END if the agent finished without handoff."""
 messages = state.get("messages", [])

Check the last message - if it's an AIMessage without tool calls, we're done
if messages:
 last_msg = messages[-1]
 if isinstance(last_msg, AIMessage) and not last_msg.tool_calls: # [!code highlight]
 return "__end__" # [!code highlight]

Otherwise route to the active agent
active = state.get("active_agent", "sales_agent")
return active if active else "sales_agent"

```

```

def route_initial(
 state: MultiAgentState,
) -> Literal["sales_agent", "support_agent"]:
 """Route to the active agent based on state, default to sales agent."""
 return state.get("active_agent") or "sales_agent"

6. Build the graph
builder = StateGraph(MultiAgentState)
builder.add_node("sales_agent", call_sales_agent)
builder.add_node("support_agent", call_support_agent)

Start with conditional routing based on initial active_agent
builder.add_conditional_edges(START, route_initial, ["sales_agent", "support_agent"])

After each agent, check if we should end or route to another agent
builder.add_conditional_edges(
 "sales_agent", route_after_agent, ["sales_agent", "support_agent", END]
)
builder.add_conditional_edges(
 "support_agent", route_after_agent, ["sales_agent", "support_agent", END]
)

graph = builder.compile()
result = graph.invoke(
 {
 "messages": [
 {
 "role": "user",
 "content": "Hi, I'm having trouble with my account login. Can you help?",
 }
]
 }
)

for msg in result["messages"]:
 msg.pretty_print()

```

js

```

import {
 StateGraph,
 START,
 END,
 StateSchema,
 MessagesValue,
 Command,
 ConditionalEdgeRouter,
} from 'langgraph'

```

```

 GraphNode,
 } from "@langchain/langgraph";
import { createAgent, AIMessage, ToolMessage } from "langchain";
import { tool, ToolRuntime } from "@langchain/core/tools";
import { z } from "zod/v4";

// 1. Define state with active_agent tracker
const MultiAgentState = new StateSchema({
 messages: MessagesValue,
 activeAgent: z.string().optional(),
});

// 2. Create handoff tools
const transferToSales = tool(
 async (_, runtime: ToolRuntime<typeof MultiAgentState.State>) => {
 const lastAiMessage = [...runtime.state.messages] // [!code highlight]
 .reverse() // [!code highlight]
 .find(AIMessage.isInstance); // [!code highlight]
 const transferMessage = new ToolMessage({ // [!code highlight]
 content: "Transferred to sales agent from support agent", // [!code highlight]
 tool_call_id: runtime.toolCallId, // [!code highlight]
 }); // [!code highlight]
 return new Command({
 goto: "sales_agent",
 update: {
 activeAgent: "sales_agent",
 messages: [lastAiMessage, transferMessage].filter(Boolean), // [!code highlight]
 },
 graph: Command.PARENT,
 });
 },
 {
 name: "transfer_to_sales",
 description: "Transfer to the sales agent.",
 schema: z.object({}),
 }
);

const transferToSupport = tool(
 async (_, runtime: ToolRuntime<typeof MultiAgentState.State>) => {
 const lastAiMessage = [...runtime.state.messages] // [!code highlight]
 .reverse() // [!code highlight]
 .find(AIMessage.isInstance); // [!code highlight]
 const transferMessage = new ToolMessage({ // [!code highlight]
 content: "Transferred to support agent from sales agent", // [!code highlight]
 tool_call_id: runtime.toolCallId, // [!code highlight]
 }); // [!code highlight]
 return new Command({
 goto: "support_agent",
 update: {

```

```

 activeAgent: "support_agent",
 messages: [lastAiMessage, transferMessage].filter(Boolean), // [!code highlight
t]
 },
 graph: Command.PARENT,
 });
},
{
 name: "transfer_to_support",
 description: "Transfer to the support agent.",
 schema: z.object({}),
}
);

// 3. Create agents with handoff tools
const salesAgent = createAgent({
 model: "google:gemin-3.6-flash",
 tools: [transferToSupport],
 systemPrompt:
 "You are a sales agent. Help with sales inquiries. If asked about technical issues
or support, transfer to the support agent.",
});

const supportAgent = createAgent({
 model: "google:gemin-3.6-flash",
 tools: [transferToSales],
 systemPrompt:
 "You are a support agent. Help with technical issues. If asked about pricing or pur
chasing, transfer to the sales agent.",
});

// 4. Create agent nodes that invoke the agents
const callSalesAgent: GraphNode<typeof MultiAgentState.State> = async (state) => {
 const response = await salesAgent.invoke(state);
 return response;
};

const callSupportAgent: GraphNode<typeof MultiAgentState.State> = async (state) => {
 const response = await supportAgent.invoke(state);
 return response;
};

// 5. Create router that checks if we should end or continue
const routeAfterAgent: ConditionalEdgeRouter<{ InputSchema: typeof MultiAgentState.Stat
e; Nodes: "sales_agent" | "support_agent" }> = (state) => {
 const messages = state.messages ?? [];

 // Check the last message - if it's an AIMessage without tool calls, we're done
 if (messages.length > 0) {
 const lastMsg = messages[messages.length - 1];
 if (lastMsg instanceof AIMessage && !lastMsg.tool_calls?.length) { // [!code highlight

```



```

t]
 return END; // [!code highlight]
} // [!code highlight]
}

// Otherwise route to the active agent
const active = state.activeAgent ?? "sales_agent";
return active as "sales_agent" | "support_agent";
};

const routeInitial: ConditionalEdgeRouter<{ InputSchema: typeof MultiAgentState.State;
Nodes: "sales_agent" | "support_agent" }> = (state) => {
 // Route to the active agent based on state, default to sales agent
 return (state.activeAgent ?? "sales_agent") as
 | "sales_agent"
 | "support_agent";
};

// 6. Build the graph
const builder = new StateGraph(MultiAgentState)
 .addNode("sales_agent", callSalesAgent)
 .addNode("support_agent", callSupportAgent);
// Start with conditional routing based on initial activeAgent
.addConditionalEdges(START, routeInitial, [
 "sales_agent",
 "support_agent",
])
// After each agent, check if we should end or route to another agent
.addConditionalEdges("sales_agent", routeAfterAgent, [
 "sales_agent",
 "support_agent",
 END,
]);
builder.addConditionalEdges("support_agent", routeAfterAgent, [
 "sales_agent",
 "support_agent",
 END,
]);

const graph = builder.compile();
const result = await graph.invoke({
 messages: [
 {
 role: "user",
 content: "Hi, I'm having trouble with my account login. Can you help?",
 },
],
});

for (const msg of result.messages) {

```

```
console.log(msg.content);
}
```

## 팁

Use **single agent with middleware** for most handoffs use cases—it's simpler. Only use **multiple agent subgraphs** when you need bespoke agent implementations (e.g., a node that's itself a complex graph with reflection or retrieval steps).

## Context engineering

With subgraph handoffs, you control exactly what messages flow between agents. This precision is essential for maintaining valid conversation history and avoiding context bloat that could confuse downstream agents. For more on this topic, see [context engineering](#).

### Handling context during handoffs

When handing off between agents, you need to ensure the conversation history remains valid. LLMs expect tool calls to be paired with their responses, so when using `Command.PARENT` to hand off to another agent, you must include both:

1. **The `AIMessage` containing the tool call** (the message that triggered the handoff)
2. **A `ToolMessage` acknowledging the handoff** (the artificial response to that tool call)

Without this pairing, the receiving agent will see an incomplete conversation and may produce errors or unexpected behavior.

The example below assumes only the handoff tool was called (no parallel tool calls):

## python

```
@tool
def transfer_to_sales(runtime: ToolRuntime) -> Command:
 # Get the AI message that triggered this handoff
 last_ai_message = runtime.state["messages"][-1]

 # Create an artificial tool response to complete the pair
 transfer_message = ToolMessage(
 content="Transferred to sales agent",
 tool_call_id=runtime.tool_call_id,
)

 return Command(
 goto="sales_agent",
 update={
 "active_agent": "sales_agent",
 # Pass only these two messages, not the full subagent history
```

```

 "messages": [last_ai_message, transfer_message],
 },
 graph=Command.PARENT,
)

```

js

```

const transferToSales = tool(
 async (_, runtime: ToolRuntime<typeof MultiAgentState.State>) => {
 // Get the AI message that triggered this handoff
 const lastAiMessage = runtime.state.messages.at(-1);

 // Create an artificial tool response to complete the pair
 const transferMessage = new ToolMessage({
 content: "Transferred to sales agent",
 tool_call_id: runtime.toolCallId,
 });

 return new Command({
 goto: "sales_agent",
 update: {
 activeAgent: "sales_agent",
 // Pass only these two messages, not the full subagent history
 messages: [lastAiMessage, transferMessage],
 },
 graph: Command.PARENT,
 });
 },
 {
 name: "transfer_to_sales",
 description: "Transfer to the sales agent.",
 schema: z.object({}),
 }
);

```

## 참고

**Why not pass all subagent messages?** While you could include the full subagent conversation in the handoff, this often creates problems. The receiving agent may become confused by irrelevant internal reasoning, and token costs increase unnecessarily. By passing only the handoff pair, you keep the parent graph's context focused on high-level coordination. If the receiving agent needs additional context, consider summarizing the subagent's work in the ToolMessage content instead of passing raw message history.

## Returning control to the user

When returning control to the user (ending the agent's turn), ensure the final message is an `AIMessage`. This maintains valid conversation history and signals to the user interface that the agent has finished its work.

## Implementation considerations

---

As you design your multi-agent system, consider:

- **Context filtering strategy:** Will each agent receive full conversation history, filtered portions, or summaries? Different agents may need different context depending on their role.
- **Tool semantics:** Clarify whether handoff tools only update routing state or also perform side effects. For example, should `transfer_to_sales()` also create a support ticket, or should that be a separate action?
- **Token efficiency:** Balance context completeness against token costs. Summarization and selective context passing become more important as conversations grow longer.

# Multi-agent

---

langchain/multi-agent/index.mdx

Multi-agent systems coordinate specialized components to tackle complex workflows. However, not every complex task requires this approach—a single agent with the right (sometimes dynamic) tools and prompt can often achieve similar results.

## 팁

For built-in multi-agent support, use [Deep Agents](#): a higher-level harness built on LangChain that ships with [subagents](#), [skills](#), planning, a virtual filesystem, and context management.

## Why multi-agent?

---

When developers say they need "multi-agent," they're usually looking for one or more of these capabilities:

- 

**Context management:** Provide specialized knowledge without overwhelming the model's context window. If context were infinite and latency zero, you could dump all knowledge into a single prompt—but since it's not, you need patterns to selectively surface relevant information.

---

**Distributed development:** Allow different teams to develop and maintain capabilities independently, composing them into a larger system with clear boundaries.

---

**Parallelization:** Spawn specialized workers for subtasks and execute them concurrently for faster results.

Multi-agent patterns are particularly valuable when a single agent has too many [tools](#) and makes poor decisions about which to use, when tasks require specialized knowledge with extensive context (long prompts and domain-specific tools), or when you need to enforce sequential constraints that unlock capabilities only after certain conditions are met.

## 팁

At the center of multi-agent design is [context engineering](#)—deciding what information each agent sees. The quality of your system depends on ensuring each agent has access to the right data for its task.

## Patterns

Here are the main patterns for building multi-agent systems, each suited to different use cases:

Pattern	How it works
<a href="#">Subagents</a>	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
<a href="#">Handoffs</a>	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
<a href="#">Skills</a>	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
<a href="#">Router</a>	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
<a href="#">Custom workflow</a>	Build bespoke execution flows with <a href="#">LangGraph</a> , mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

## Choosing a pattern

Use this table to match your requirements to the right pattern:

Pattern	Distributed development	Parallelization	Multi-hop	Direct user interaction
<a href="#">Subagents</a>	★★★★★	★★★★★	★★★★★	★
<a href="#">Handoffs</a>	★★★★★	★★★★★	★★★★★	★★★★★
<a href="#">Skills</a>	★★★★★	★★★★★	★★★★★	★★★★★
<a href="#">Router</a>	★★★★★	★★★★★	★★★★★	★★★★★

- **Distributed development:** Can different teams maintain components independently?
- **Parallelization:** Can multiple agents execute concurrently?
- **Multi-hop:** Does the pattern support calling multiple subagents in series?
- **Direct user interaction:** Can subagents converse directly with the user?

### 팁

You can mix patterns! For example, a **subagents** architecture can invoke tools that invoke custom workflows or router agents. Subagents can even use the **skills** pattern to load

context on-demand. The possibilities are endless!

## Visual overview

### Subagents

A main agent coordinates subagents as tools. All routing passes through the main agent.

 Subagents pattern: main agent coordinates subagents as tools

### Handoffs

Agents transfer control to each other via tool calls. Each agent can hand off to others or respond directly to the user.

 Handoffs pattern: agents transfer control via tool calls

### Skills

A single agent loads specialized prompts and knowledge on-demand while staying in control.

 Skills pattern: single agent loads specialized context on-demand

### Router

A routing step classifies input and directs it to specialized agents. Results are synthesized.

 Router pattern: routing step classifies input to specialized agents

#### 팁

Trace the full coordination flow across agents with [LangSmith](#). Follow the [tracing quickstart](#) to get set up.

We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

## Performance comparison

Different patterns have different performance characteristics. Understanding these tradeoffs helps you choose the right pattern for your latency and cost requirements.

**Key metrics:** - **Model calls:** Number of LLM invocations. More calls = higher latency (especially if sequential) and higher per-request API costs. - **Tokens processed:** Total [context window](#) usage across all calls. More tokens = higher processing costs and potential context limits.

## One-shot request

User: "Buy coffee"

A specialized coffee agent/skill can call a `buy_coffee` tool.

Pattern	Model calls	Best fit
<a href="#">Subagents</a>	4	
<a href="#">Handoffs</a>	3	✓
<a href="#">Skills</a>	3	✓
<a href="#">Router</a>	3	✓

### Subagents

#### 4 model calls:

 Subagents one-shot: 4 model calls for buy coffee request

### Handoffs

#### 3 model calls:

 Handoffs one-shot: 3 model calls for buy coffee request

### Skills

#### 3 model calls:

 Skills one-shot: 3 model calls for buy coffee request

### Router

#### 3 model calls:

 Router one-shot: 3 model calls for buy coffee request

**Key insight:** Handoffs, Skills, and Router are most efficient for single tasks (3 calls each). Subagents adds one extra call because results flow back through the main agent—this overhead provides centralized control.

## Repeat request

Turn 1: "Buy coffee" Turn 2: "Buy coffee again"

The user repeats the same request in the same conversation.



| Pattern | Turn 2 calls | Total (both turns) | Best fit | |-----|:-----:|:-----:|:-----:| |  
[\*\*Subagents\*\*](/oss/langchain/multi-agent/subagents) | 4 | 8 | | | [\*\*Handoffs\*\*]  
(/oss/langchain/multi-agent/handoffs) | 2 | 5 | ☒ | | [\*\*Skills\*\*](/oss/langchain/multi-  
agent/skills) | 2 | 5 | ☒ | | [\*\*Router\*\*](/oss/langchain/multi-agent/router) | 3 | 6 | |

## Subagents

### 4 calls again → 8 total

- Subagents are **stateless by design**—each invocation follows the same flow
- The main agent maintains conversation context, but subagents start fresh each time
- This provides strong context isolation but repeats the full flow

## Handoffs

### 2 calls → 5 total

- The coffee agent is **still active** from turn 1 (state persists)
- No handoff needed—agent directly calls `buy_coffee` tool (call 1)
- Agent responds to user (call 2)
- **Saves 1 call by skipping the handoff**

## Skills

### 2 calls → 5 total

- The skill context is **already loaded** in conversation history
- No need to reload—agent directly calls `buy_coffee` tool (call 1)
- Agent responds to user (call 2)
- **Saves 1 call by reusing loaded skill**

## Router

### 3 calls again → 6 total

- Routers are **stateless**—each request requires an LLM routing call
- Turn 2: Router LLM call (1) → Milk agent calls `buy_coffee` (2) → Milk agent responds (3)
- Can be optimized by wrapping as a tool in a stateful agent

**Key insight:** Stateful patterns (Handoffs, Skills) save 40-50% of calls on repeat requests. Subagents maintain consistent cost per request—this stateless design provides strong context isolation but at the cost of repeated model calls.

## Multi-domain

**User:** "Compare Python, JavaScript, and Rust for web development"

Each language agent/skill contains ~2000 tokens of documentation. All patterns can make parallel tool calls.

Pattern	Model calls	Total tokens	Best fit
<a href="#">Subagents</a>	5	~9K	✓
<a href="#">Handoffs</a>	7+	~14K+	
<a href="#">Skills</a>	3	~15K	
<a href="#">Router</a>	5	~9K	✓

## Subagents

### 5 calls, ~9K tokens

 Subagents multi-domain: 5 calls with parallel execution

Each subagent works in **isolation** with only its relevant context. Total: **9K tokens**.

## Handoffs

### 7+ calls, ~14K+ tokens

 Handoffs multi-domain: 7+ sequential calls

Handoffs executes **sequentially**—can't research all three languages in parallel. Growing conversation history adds overhead. Total: **~14K+ tokens**.

## Skills

### 3 calls, ~15K tokens

 Skills multi-domain: 3 calls with accumulated context

After loading, **every subsequent call processes all 6K tokens of skill documentation**. Subagents processes 67% fewer tokens overall due to context isolation. Total: **15K tokens**.

## Router

### 5 calls, ~9K tokens

 Router multi-domain: 5 calls with parallel execution

Router uses an **LLM for routing**, then invokes agents in parallel. Similar to Subagents but with explicit routing step. Total: **9K tokens**.

**Key insight:** For multi-domain tasks, patterns with parallel execution (Subagents, Router) are most efficient. Skills has fewer calls but high token usage due to context accumulation. Handoffs is inefficient here—it must execute sequentially and can't leverage parallel tool calling for consulting multiple domains simultaneously.

## Summary

Here's how patterns compare across all three scenarios:

Pattern	One-shot	Repeat request	Multi-domain
[**Subagents**](/oss/langchain/multi-agent/subagents)	4 calls	8 calls (4+4)	5 calls, 9K tokens
[**Handoffs**](/oss/langchain/multi-agent/handoffs)	3 calls	5 calls (3+2)	7+ calls, 14K+ tokens
[**Skills**](/oss/langchain/multi-agent/skills)	3 calls	5 calls (3+2)	3 calls, 15K tokens
[**Router**](/oss/langchain/multi-agent/router)	3 calls	6 calls (3+3)	5 calls, 9K tokens

### Choosing a pattern:

Optimize for	[Subagents](/oss/langchain/multi-agent/subagents)	[Handoffs](/oss/langchain/multi-agent/handoffs)	[Skills](/oss/langchain/multi-agent/skills)	[Router](/oss/langchain/multi-agent/router)	Single requests
Repeat requests	✓	✓	✓	✓	✓
Parallel execution	✓	✓	✓	✓	✓
Large-context domains	✓	✓	✓	✓	✓
Simple, focused tasks	✓	✓	✓	✓	✓

# Build a multi-source knowledge base with routing

langchain/multi-agent/router-knowledge-base.mdx

## Overview

The **router pattern** is a [multi-agent](#) architecture where a routing step classifies input and directs it to specialized agents, with results synthesized into a combined response. This pattern excels when your organization's knowledge lives across distinct **verticals** (separate knowledge domains that each require their own agent with specialized tools and prompts).

In this tutorial, you'll build a multi-source knowledge base router that demonstrates these benefits through a realistic enterprise scenario. The system will coordinate three specialists:

- A **GitHub agent** that searches code, issues, and pull requests.
- A **Notion agent** that searches internal documentation and wikis.
- A **Slack agent** that searches relevant threads and discussions.

When a user asks "How do I authenticate API requests?", the router decomposes the query into source-specific sub-questions, routes them to the relevant agents in parallel, and synthesizes results into a coherent answer.

```
graph LR
 A([Query]) --> B[Classify]
 B --> C[GitHub agent]
 B --> D[Notion agent]
 B --> E[Slack agent]
 C --> F[Synthesize]
 D --> F
 E --> F
 F --> G([Combined answer])

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,G trigger
class B,C,D,E,F process
```

## Why use a router?

The router pattern provides several advantages:

- **Parallel execution:** Query multiple sources simultaneously, reducing latency compared to sequential approaches.

- **Specialized agents:** Each vertical has focused tools and prompts optimized for its domain.
- **Selective routing:** Not every query needs every source—the router intelligently selects relevant verticals.
- **Targeted sub-questions:** Each agent receives a question tailored to its domain, improving result quality.
- **Clean synthesis:** Results from multiple sources are combined into a single, coherent response.

## Concepts

We will cover the following concepts:

- [Multi-agent systems](#)
- [StateGraph](#) for workflow orchestration
- [Send API](#) for parallel execution

### 팁

**Router vs. Subagents:** The [subagents pattern](#) can also route to multiple agents. Use the router pattern when you need specialized preprocessing, custom routing logic, or want explicit control over parallel execution. Use the subagents pattern when you want the LLM to decide which agents to call dynamically.

## Setup

### Installation

This tutorial requires the `langchain` and `langgraph` packages:

#### python

```
```bash pip pip install langchain langgraph
```

```
```bash uv
uv add langchain langgraph
```

```
```bash conda conda install langchain langgraph -c conda-forge
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```bash npm
npm install langchain @langchain/langgraph
```

```
```bash yarn yarn add langchain @langchain/langgraph
```

```
```bash pnpm
pnpm add langchain @langchain/langgraph
```

For more details, see our [Installation guide](#).

## LangSmith

Set up [LangSmith](#) to inspect what is happening inside your agent. Then set the following environment variables:

### python

```
```bash Shell export LANGSMITH_TRACING="true" export LANGSMITH_API_KEY="..."
```

```
```python Python
import getpass
import os

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

### js

```
```bash Shell export LANGSMITH_TRACING="true" export LANGSMITH_API_KEY="..."
```

```
```typescript TypeScript
process.env.LANGSMITH_TRACING = "true";
process.env.LANGSMITH_API_KEY = "...";
```

## Select an LLM

Select a chat model from LangChain's suite of integrations:

### python

### js

# 1. Define state

First, define the state schemas. We use three types:

- **AgentInput** : Simple state passed to each subagent (just a query)
- **AgentOutput** : Result returned by each subagent (source name + result)
- **RouterState** : Main workflow state tracking the query, classifications, results, and final answer

## python

```
from typing import Annotated, Literal, TypedDict
import operator

class AgentInput(TypedDict):
 """Simple input state for each subagent."""
 query: str

class AgentOutput(TypedDict):
 """Output from each subagent."""
 source: str
 result: str

class Classification(TypedDict):
 """A single routing decision: which agent to call with what query."""
 source: Literal["github", "notion", "slack"]
 query: str

class RouterState(TypedDict):
 query: str
 classifications: list[Classification]
 results: Annotated[list[AgentOutput], operator.add] # Reducer collects parallel results
 final_answer: str
```

## js

```
import { StateSchema, ReducedValue } from "@langchain/langgraph";
import { z } from "zod/v4";

const AgentOutput = z.object({
 source: z.string(),
 result: z.string(),
});

const RouterState = new StateSchema({
 query: z.string(),
```

```

classifications: z.array(
 z.object({
 source: z.enum(["github", "notion", "slack"]),
 query: z.string(),
 })
),
results: new ReducedValue(
 z.array(AgentOutput).default(() => []),
 { reducer: (current, update) => current.concat(update) },
),
finalAnswer: z.string(),
});

```

The `results` field uses a **reducer** (`operator.add` in Python, a `concat` function in JS) to collect outputs from parallel agent executions into a single list.

## 2. Define tools for each vertical

Create tools for each knowledge domain. In a production system, these would call actual APIs. For this tutorial, we use stub implementations that return mock data. We define 7 tools across 3 verticals: GitHub (search code, issues, PRs), Notion (search docs, get page), and Slack (search messages, get thread).

### python

```

` `` python expandable from langchain.tools import tool

```

```

@tool def search_code(query: str, repo: str = "main") -> str: """Search code in GitHub repositories.""" return f"Found code matching '{query}' in {repo}: authentication middleware in src/auth.py"

```

```

@tool def search_issues(query: str) -> str: """Search GitHub issues and pull requests.""" return f"Found 3 issues matching '{query}': #142 (API auth docs), #89 (OAuth flow), #203 (token refresh)"

```

```

@tool def search_prs(query: str) -> str: """Search pull requests for implementation details.""" return f"PR #156 added JWT authentication, PR #178 updated OAuth scopes"

```

```

@tool def search_notion(query: str) -> str: """Search Notion workspace for documentation.""" return f"Found documentation: 'API Authentication Guide' - covers OAuth2 flow, API keys, and JWT tokens"

```

```

@tool def get_page(page_id: str) -> str: """Get a specific Notion page by ID.""" return f"Page content: Step-by-step authentication setup instructions"

```

```

@tool def search_slack(query: str) -> str: """Search Slack messages and threads.""" return f"Found discussion in #engineering: 'Use Bearer tokens for API auth, see docs for refresh"

```



flow"

@tool def get\_thread(thread\_id: str) -> str: """Get a specific Slack thread.""" return f"Thread discusses best practices for API key rotation"

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript expandable

import { tool } from "langchain";

import { z } from "zod";

const searchCode = tool(

async ({ query, repo }) => {

return `Found code matching '\${query}' in \${repo || "main"}: authentication middlew
are in src/auth.py`;

},

{

name: "search_code",

description: "Search code in GitHub repositories.",

schema: z.object({

query: z.string(),

repo: z.string().optional().default("main"),

}),

}

);

const searchIssues = tool(

async ({ query }) => {

return `Found 3 issues matching '\${query}': #142 (API auth docs), #89 (OAuth flow),
#203 (token refresh)`;

},

{

name: "search_issues",

description: "Search GitHub issues and pull requests.",

schema: z.object({

query: z.string(),

}),

}

);

const searchPrs = tool(

async ({ query }) => {

return `PR #156 added JWT authentication, PR #178 updated OAuth scopes`;

},

{

```

    name: "search_prs",
    description: "Search pull requests for implementation details.",
    schema: z.object({
      query: z.string(),
    }),
  },
);

const searchNotion = tool(
  async ({ query }) => {
    return `Found documentation: 'API Authentication Guide' - covers OAuth2 flow, API keys, and JWT tokens`;
  },
  {
    name: "search_notion",
    description: "Search Notion workspace for documentation.",
    schema: z.object({
      query: z.string(),
    }),
  }
);

const getPage = tool(
  async ({ pageId }) => {
    return `Page content: Step-by-step authentication setup instructions`;
  },
  {
    name: "get_page",
    description: "Get a specific Notion page by ID.",
    schema: z.object({
      pageId: z.string(),
    }),
  }
);

const searchSlack = tool(
  async ({ query }) => {
    return `Found discussion in #engineering: 'Use Bearer tokens for API auth, see docs for refresh flow'`;
  },
  {
    name: "search_slack",
    description: "Search Slack messages and threads.",
    schema: z.object({
      query: z.string(),
    }),
  }
);

const getThread = tool(
  async ({ threadId }) => {

```

```

    return `Thread discusses best practices for API key rotation`;
  },
  {
    name: "get_thread",
    description: "Get a specific Slack thread.",
    schema: z.object({
      threadId: z.string(),
    }),
  },
}
);

```

3. Create specialized agents

Create an agent for each vertical. Each agent has domain-specific tools and a prompt optimized for its knowledge source. All three follow the same pattern—only the tools and system prompt differ.

python

```

```python
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

model = init_chat_model("openai:gpt-5.5")

github_agent = create_agent(
 model,
 tools=[search_code, search_issues, search_prs],
 system_prompt=(
 "You are a GitHub expert. Answer questions about code, "
 "API references, and implementation details by searching "
 "repositories, issues, and pull requests."
),
)

notion_agent = create_agent(
 model,
 tools=[search_notion, get_page],
 system_prompt=(
 "You are a Notion expert. Answer questions about internal "
 "processes, policies, and team documentation by searching "
 "the organization's Notion workspace."
),
)

slack_agent = create_agent(
 model,
 tools=[search_slack, get_thread],
 system_prompt=(
 "You are a Slack expert. Answer questions by searching "
 "relevant threads and discussions where team members have "
 "shared knowledge and solutions."
),
)

```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```

```typescript
import { createAgent } from "langchain";
import { ChatOpenAI } from "@langchain/openai";

```

```
const llm = new ChatOpenAI({ model: "gpt-5.5" });

const githubAgent = createAgent({
  model: llm,
  tools: [searchCode, searchIssues, searchPrs],
  systemPrompt: `
You are a GitHub expert. Answer questions about code,
API references, and implementation details by searching
repositories, issues, and pull requests.
`.trim(),
});

const notionAgent = createAgent({
  model: llm,
  tools: [searchNotion, getPage],
  systemPrompt: `
You are a Notion expert. Answer questions about internal
processes, policies, and team documentation by searching
the organization's Notion workspace.
`.trim(),
});

const slackAgent = createAgent({
  model: llm,
  tools: [searchSlack, getThread],
  systemPrompt: `
You are a Slack expert. Answer questions by searching
relevant threads and discussions where team members have
shared knowledge and solutions.
`.trim(),
});
```

4. Build the router workflow

Now build the router workflow using a StateGraph. The workflow has four main steps:

1. **Classify:** Analyze the query and determine which agents to invoke with what sub-questions
2. **Route:** Fan out to selected agents in parallel using `Send`
3. **Query agents:** Each agent receives a simple `AgentInput` and returns an `AgentOutput`
4. **Synthesize:** Combine collected results into a coherent response

python

```
from pydantic import BaseModel, Field
from langgraph.graph import StateGraph, START, END
```

```

from langgraph.types import Send

router_llm = init_chat_model("openai:gpt-5.4-mini")

# Define structured output schema for the classifier
class ClassificationResult(BaseModel): # [!code highlight]
    """Result of classifying a user query into agent-specific sub-questions."""
    classifications: list[Classification] = Field(
        description="List of agents to invoke with their targeted sub-questions"
    )

def classify_query(state: RouterState) -> dict:
    """Classify query and determine which agents to invoke."""
    structured_llm = router_llm.with_structured_output(ClassificationResult) # [!code highlight]

    result = structured_llm.invoke([
        {
            "role": "system",
            "content": """Analyze this query and determine which knowledge bases to consult.
For each relevant source, generate a targeted sub-question optimized for that source.

Available sources:
- github: Code, API references, implementation details, issues, pull requests
- notion: Internal documentation, processes, policies, team wikis
- slack: Team discussions, informal knowledge sharing, recent conversations

Return ONLY the sources that are relevant to the query. Each source should have
a targeted sub-question optimized for that specific knowledge domain.

Example for "How do I authenticate API requests?":
- github: "What authentication code exists? Search for auth middleware, JWT handling"
- notion: "What authentication documentation exists? Look for API auth guides"
(slack omitted because it's not relevant for this technical question)"""
        },
        {"role": "user", "content": state["query"]}
    ])

    return {"classifications": result.classifications}

def route_to_agents(state: RouterState) -> list[Send]:
    """Fan out to agents based on classifications."""
    return [
        Send(c["source"], {"query": c["query"]}) # [!code highlight]
        for c in state["classifications"]
    ]

def query_github(state: AgentInput) -> dict:
    """Query the GitHub agent."""
    result = github_agent.invoke({

```

```

        "messages": [{"role": "user", "content": state["query"]}]] # [!code highlight]
    })
    return {"results": [{"source": "github", "result": result["messages"][-1].content}]}

def query_notion(state: AgentInput) -> dict:
    """Query the Notion agent."""
    result = notion_agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}]] # [!code highlight]
    })
    return {"results": [{"source": "notion", "result": result["messages"][-1].content}]}

def query_slack(state: AgentInput) -> dict:
    """Query the Slack agent."""
    result = slack_agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}]] # [!code highlight]
    })
    return {"results": [{"source": "slack", "result": result["messages"][-1].content}]}

def synthesize_results(state: RouterState) -> dict:
    """Combine results from all agents into a coherent answer."""
    if not state["results"]:
        return {"final_answer": "No results found from any knowledge source."}

    # Format results for synthesis
    formatted = [
        f"""From {r['source'].title():*}\n{r['result']}"""
        for r in state["results"]
    ]

    synthesis_response = router_llm.invoke([
        {
            "role": "system",
            "content": f"""Synthesize these search results to answer the original question:
            "{state['query']}"

            - Combine information from multiple sources without redundancy
            - Highlight the most relevant and actionable information
            - Note any discrepancies between sources
            - Keep the response concise and well-organized"""
        },
        {"role": "user", "content": "\n\n".join(formatted)}
    ])

    return {"final_answer": synthesis_response.content}

```

js

```

import { StateGraph, START, END, Send } from "@langchain/langgraph";
import { z } from "zod";

const routerLlm = new ChatOpenAI({ model: "gpt-5.4-mini" });

// Define structured output schema for the classifier
const ClassificationResultSchema = z.object({ // [!code highlight]
  classifications: z.array(z.object({
    source: z.enum(["github", "notion", "slack"]),
    query: z.string(),
  })).describe("List of agents to invoke with their targeted sub-questions"),
});

async function classifyQuery(state: typeof RouterState.State) {
  const structuredLlm = routerLlm.withStructuredOutput(ClassificationResultSchema); // [!code highlight]

  const result = await structuredLlm.invoke([
    {
      role: "system",
      content: `Analyze this query and determine which knowledge bases to consult.
For each relevant source, generate a targeted sub-question optimized for that source.

Available sources:
- github: Code, API references, implementation details, issues, pull requests
- notion: Internal documentation, processes, policies, team wikis
- slack: Team discussions, informal knowledge sharing, recent conversations

Return ONLY the sources that are relevant to the query. Each source should have
a targeted sub-question optimized for that specific knowledge domain.

Example for "How do I authenticate API requests?":
- github: "What authentication code exists? Search for auth middleware, JWT handling"
- notion: "What authentication documentation exists? Look for API auth guides"
(slack omitted because it's not relevant for this technical question)`
    },
    { role: "user", content: state.query }
  ]);

  return { classifications: result.classifications };
}

function routeToAgents(state: typeof RouterState.State): Send[] {
  return state.classifications.map(
    (c) => new Send(c.source, { query: c.query }) // [!code highlight]
  );
}

async function queryGithub(state: AgentInput) {
  const result = await githubAgent.invoke({

```

```

    messages: [{ role: "user", content: state.query }] // [!code highlight]
  });
  return { results: [{ source: "github", result: result.messages.at(-1)?.content }] };
}

async function queryNotion(state: AgentInput) {
  const result = await notionAgent.invoke({
    messages: [{ role: "user", content: state.query }] // [!code highlight]
  });
  return { results: [{ source: "notion", result: result.messages.at(-1)?.content }] };
}

async function querySlack(state: AgentInput) {
  const result = await slackAgent.invoke({
    messages: [{ role: "user", content: state.query }] // [!code highlight]
  });
  return { results: [{ source: "slack", result: result.messages.at(-1)?.content }] };
}

async function synthesizeResults(state: typeof RouterState.State) {
  if (state.results.length === 0) {
    return { finalAnswer: "No results found from any knowledge source." };
  }

  // Format results for synthesis
  const formatted = state.results.map(
    (r) => `**From ${r.source.charAt(0).toUpperCase() + r.source.slice(1)}:**\n${r.result}\n`
  );

  const synthesisResponse = await routerLlm.invoke([
    {
      role: "system",
      content: `Synthesize these search results to answer the original question: "${state.query}"

      - Combine information from multiple sources without redundancy
      - Highlight the most relevant and actionable information
      - Note any discrepancies between sources
      - Keep the response concise and well-organized`
    },
    { role: "user", content: formatted.join("\n\n") }
  ]);

  return { finalAnswer: synthesisResponse.content };
}

```


5. Compile the workflow

Now assemble the workflow by connecting nodes with edges. The key is using `add_conditional_edges` with the routing function to enable parallel execution:

python

```
workflow = (  
    StateGraph(RouterState)  
    .add_node("classify", classify_query)  
    .add_node("github", query_github)  
    .add_node("notion", query_notion)  
    .add_node("slack", query_slack)  
    .add_node("synthesize", synthesize_results)  
    .add_edge(START, "classify")  
    .add_conditional_edges("classify", route_to_agents, ["github", "notion", "slack"])  
    .add_edge("github", "synthesize")  
    .add_edge("notion", "synthesize")  
    .add_edge("slack", "synthesize")  
    .add_edge("synthesize", END)  
    .compile()  
)
```

js

```
const workflow = new StateGraph(RouterState)  
    .addNode("classify", classifyQuery)  
    .addNode("github", queryGithub)  
    .addNode("notion", queryNotion)  
    .addNode("slack", querySlack)  
    .addNode("synthesize", synthesizeResults)  
    .addEdge(START, "classify")  
    .addConditionalEdges("classify", routeToAgents, ["github", "notion", "slack"])  
    .addEdge("github", "synthesize")  
    .addEdge("notion", "synthesize")  
    .addEdge("slack", "synthesize")  
    .addEdge("synthesize", END)  
    .compile();
```

The `add_conditional_edges` call connects the `classify` node to the agent nodes through the `route_to_agents` function. When `route_to_agents` returns multiple `Send` objects, those nodes execute in parallel.

6. Use the router

Test your router with queries that span multiple knowledge domains:

python

```
result = workflow.invoke({
    "query": "How do I authenticate API requests?"
})

print("Original query:", result["query"])
print("\nClassifications:")
for c in result["classifications"]:
    print(f"  {c['source']}: {c['query']}")
print("\n" + "=" * 60 + "\n")
print("Final Answer:")
print(result["final_answer"])
```

js

```
const result = await workflow.invoke({
  query: "How do I authenticate API requests?"
});

console.log("Original query:", result.query);
console.log("\nClassifications:");
for (const c of result.classifications) {
  console.log(`  ${c.source}: ${c.query}`);
}
console.log("\n" + "=".repeat(60) + "\n");
console.log("Final Answer:");
console.log(result.finalAnswer);
```

Expected output:

```
Original query: How do I authenticate API requests?

Classifications:
  github: What authentication code exists? Search for auth middleware, JWT handling
  notion: What authentication documentation exists? Look for API auth guides

=====

Final Answer:
To authenticate API requests, you have several options:
```

1. ****JWT Tokens****: The recommended approach for most use cases. Implementation details are in ``src/auth.py`` (PR #156).
2. ****OAuth2 Flow****: For third-party integrations, follow the OAuth2 flow documented in Notion's 'API Authentication Guide'.
3. ****API Keys****: For server-to-server communication, use Bearer tokens in the Authorization header.

For token refresh handling, see issue #203 and PR #178 for the latest OAuth scope updates.

The router analyzed the query, classified it to determine which agents to invoke (GitHub and Notion, but not Slack for this technical question), queried both agents in parallel, and synthesized the results into a coherent answer.

7. Understanding the architecture

The router workflow follows a clear pattern:

Classification phase

The `classify_query` function uses **structured output** to analyze the user's query and determine which agents to invoke. This is where the routing intelligence lives:

- Uses a Pydantic model (Python) or Zod schema (JS) to ensure valid output
- Returns a list of `Classification` objects, each with a `source` and targeted `query`
- Only includes relevant sources—irrelevant ones are simply omitted

This structured approach is more reliable than free-form JSON parsing and makes the routing logic explicit.

Parallel execution with send

The `route_to_agents` function maps classifications to `Send` objects. Each `Send` specifies the target node and the state to pass:

python

```
# Classifications: [{"source": "github", "query": "..."}, {"source": "notion", "query": "..."}]
# Becomes:
[Send("github", {"query": "..."}), Send("notion", {"query": "..."})]
# Both agents execute simultaneously, each receiving only the query it needs
```

js

```
// Classifications: [{ source: "github", query: "..."}, { source: "notion", query: "..."}]
// Becomes:
[new Send("github", { query: "..."}), new Send("notion", { query: "..."})]
// Both agents execute simultaneously, each receiving only the query it needs
```

Each agent node receives a simple `AgentInput` with just a `query` field—not the full router state. This keeps the interface clean and explicit.

Result collection with reducers

Agent results flow back to the main state via a **reducer**. Each agent returns:

python

```
{"results": [{"source": "github", "result": "..."}]}
```

js

```
{ results: [{ source: "github", result: "..."}] }
```

The reducer (`operator.add` in Python) concatenates these lists, collecting all parallel results into `state["results"]`.

Synthesis phase

After all agents complete, the `synthesize_results` function iterates over the collected results:

- Waits for all parallel branches to complete (LangGraph handles this automatically)
- References the original query to ensure the answer addresses what the user asked
- Combines information from all sources without redundancy

참고

Partial results: In this tutorial, all selected agents must complete before synthesis.

8. Complete working example

Here's everything together in a runnable script:

View complete code

python

```
"""
Multi-Source Knowledge Router Example

This example demonstrates the router pattern for multi-agent systems.
A router classifies queries, routes them to specialized agents in parallel,
and synthesizes results into a combined response.
"""

import operator
from typing import Annotated, Literal, TypedDict

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
from pydantic import BaseModel, Field

# State definitions
class AgentInput(TypedDict):
    """Simple input state for each subagent."""
    query: str

class AgentOutput(TypedDict):
    """Output from each subagent."""
    source: str
    result: str

class Classification(TypedDict):
    """A single routing decision: which agent to call with what query."""
    source: Literal["github", "notion", "slack"]
    query: str

class RouterState(TypedDict):
    query: str
    classifications: list[Classification]
    results: Annotated[list[AgentOutput], operator.add]
    final_answer: str

# Structured output schema for classifier
class ClassificationResult(BaseModel):
    """Result of classifying a user query into agent-specific sub-questions."""
    classifications: list[Classification] = Field(
        description="List of agents to invoke with their targeted sub-questions"
    )

# Tools
@tool
```

```

def search_code(query: str, repo: str = "main") -> str:
    """Search code in GitHub repositories."""
    return f"Found code matching '{query}' in {repo}: authentication middleware in src/auth.py"

@tool
def search_issues(query: str) -> str:
    """Search GitHub issues and pull requests."""
    return f"Found 3 issues matching '{query}': #142 (API auth docs), #89 (OAuth flow), #203 (token refresh)"

@tool
def search_prs(query: str) -> str:
    """Search pull requests for implementation details."""
    return f"PR #156 added JWT authentication, PR #178 updated OAuth scopes"

@tool
def search_notion(query: str) -> str:
    """Search Notion workspace for documentation."""
    return f"Found documentation: 'API Authentication Guide' - covers OAuth2 flow, API keys, and JWT tokens"

@tool
def get_page(page_id: str) -> str:
    """Get a specific Notion page by ID."""
    return f"Page content: Step-by-step authentication setup instructions"

@tool
def search_slack(query: str) -> str:
    """Search Slack messages and threads."""
    return f"Found discussion in #engineering: 'Use Bearer tokens for API auth, see docs for refresh flow'"

@tool
def get_thread(thread_id: str) -> str:
    """Get a specific Slack thread."""
    return f"Thread discusses best practices for API key rotation"

# Models and agents
model = init_chat_model("openai:gpt-5.5")
router_llm = init_chat_model("openai:gpt-5.4-mini")

github_agent = create_agent(
    model,
    tools=[search_code, search_issues, search_prs],
    system_prompt=(
        "You are a GitHub expert. Answer questions about code, "
        "API references, and implementation details by searching "
        "repositories, issues, and pull requests."
    ),
)

```

```

notion_agent = create_agent(
    model,
    tools=[search_notion, get_page],
    system_prompt=(
        "You are a Notion expert. Answer questions about internal "
        "processes, policies, and team documentation by searching "
        "the organization's Notion workspace."
    ),
)

slack_agent = create_agent(
    model,
    tools=[search_slack, get_thread],
    system_prompt=(
        "You are a Slack expert. Answer questions by searching "
        "relevant threads and discussions where team members have "
        "shared knowledge and solutions."
    ),
)

# Workflow nodes
def classify_query(state: RouterState) -> dict:
    """Classify query and determine which agents to invoke."""
    structured_llm = router_llm.with_structured_output(ClassificationResult)

    result = structured_llm.invoke([
        {
            "role": "system",
            "content": """Analyze this query and determine which knowledge bases to consult.
For each relevant source, generate a targeted sub-question optimized for that source.

Available sources:
- github: Code, API references, implementation details, issues, pull requests
- notion: Internal documentation, processes, policies, team wikis
- slack: Team discussions, informal knowledge sharing, recent conversations

Return ONLY the sources that are relevant to the query."""
        },
        {"role": "user", "content": state["query"]}
    ])

    return {"classifications": result.classifications}

def route_to_agents(state: RouterState) -> list[Send]:
    """Fan out to agents based on classifications."""
    return [
        Send(c["source"], {"query": c["query"]})
        for c in state["classifications"]
    ]

```

```

def query_github(state: AgentInput) -> dict:
    """Query the GitHub agent."""
    result = github_agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}]}
    })
    return {"results": [{"source": "github", "result": result["messages"][-1].content}]}

def query_notion(state: AgentInput) -> dict:
    """Query the Notion agent."""
    result = notion_agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}]}
    })
    return {"results": [{"source": "notion", "result": result["messages"][-1].content}]}

def query_slack(state: AgentInput) -> dict:
    """Query the Slack agent."""
    result = slack_agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}]}
    })
    return {"results": [{"source": "slack", "result": result["messages"][-1].content}]}

def synthesize_results(state: RouterState) -> dict:
    """Combine results from all agents into a coherent answer."""
    if not state["results"]:
        return {"final_answer": "No results found from any knowledge source."}

    formatted = [
        f"""From {r['source'].title()}:**\n{r['result']}"""
        for r in state["results"]
    ]

    synthesis_response = router_llm.invoke([
        {
            "role": "system",
            "content": f"""Synthesize these search results to answer the original question:
"{state['query']}"

- Combine information from multiple sources without redundancy
- Highlight the most relevant and actionable information
- Note any discrepancies between sources
- Keep the response concise and well-organized"""
        },
        {"role": "user", "content": "\n\n".join(formatted)}
    ])

    return {"final_answer": synthesis_response.content}

# Build workflow

```



```

workflow = (
    StateGraph(RouterState)
    .add_node("classify", classify_query)
    .add_node("github", query_github)
    .add_node("notion", query_notion)
    .add_node("slack", query_slack)
    .add_node("synthesize", synthesize_results)
    .add_edge(START, "classify")
    .add_conditional_edges("classify", route_to_agents, ["github", "notion", "slack"])
    .add_edge("github", "synthesize")
    .add_edge("notion", "synthesize")
    .add_edge("slack", "synthesize")
    .add_edge("synthesize", END)
    .compile()
)

if __name__ == "__main__":
    result = workflow.invoke({
        "query": "How do I authenticate API requests?"
    })

    print("Original query:", result["query"])
    print("\nClassifications:")
    for c in result["classifications"]:
        print(f"  {c['source']}: {c['query']}")
    print("\n" + "=" * 60 + "\n")
    print("Final Answer:")
    print(result["final_answer"])

```

js

```

/**
 * Multi-Source Knowledge Router Example
 *
 * This example demonstrates the router pattern for multi-agent systems.
 * A router classifies queries, routes them to specialized agents in parallel,
 * and synthesizes results into a combined response.
 */
import { z } from "zod/v4";
import { tool } from "langchain";
import { StateGraph, START, END, Send, StateSchema, ReducedValue } from "@langchain/langgraph";

const AgentOutput = z.object({
  source: z.string(),
  result: z.string(),
});

const RouterState = new StateSchema({

```

```

query: z.string(),
classifications: z.array(
  z.object({
    source: z.enum(["github", "notion", "slack"]),
    query: z.string(),
  })
),
results: new ReducedValue(
  z.array(AgentOutput).default(() => []),
  { reducer: (current, update) => current.concat(update) }
),
finalAnswer: z.string(),
});

const searchCode = tool(
  async ({ query, repo }) => {
    return `Found code matching '${query}' in ${repo || "main"}: authentication middlew
are in src/auth.py`;
  },
  {
    name: "search_code",
    description: "Search code in GitHub repositories.",
    schema: z.object({
      query: z.string(),
      repo: z.string().optional().default("main"),
    }),
  }
);

const searchIssues = tool(
  async ({ query }) => {
    return `Found 3 issues matching '${query}': #142 (API auth docs), #89 (OAuth flow),
#203 (token refresh)`;
  },
  {
    name: "search_issues",
    description: "Search GitHub issues and pull requests.",
    schema: z.object({
      query: z.string(),
    }),
  }
);

const searchPrs = tool(
  async ({ query }) => {
    return `PR #156 added JWT authentication, PR #178 updated OAuth scopes`;
  },
  {
    name: "search_prs",
    description: "Search pull requests for implementation details.",
    schema: z.object({

```

```

        query: z.string(),
      }),
    },
  );

const searchNotion = tool(
  async ({ query }) => {
    return `Found documentation: 'API Authentication Guide' - covers OAuth2 flow, API keys, and JWT tokens`;
  },
  {
    name: "search_notion",
    description: "Search Notion workspace for documentation.",
    schema: z.object({
      query: z.string(),
    }),
  }
);

const getPage = tool(
  async ({ pageId }) => {
    return `Page content: Step-by-step authentication setup instructions`;
  },
  {
    name: "get_page",
    description: "Get a specific Notion page by ID.",
    schema: z.object({
      pageId: z.string(),
    }),
  }
);

const searchSlack = tool(
  async ({ query }) => {
    return `Found discussion in #engineering: 'Use Bearer tokens for API auth, see docs for refresh flow'`;
  },
  {
    name: "search_slack",
    description: "Search Slack messages and threads.",
    schema: z.object({
      query: z.string(),
    }),
  }
);

const getThread = tool(
  async ({ threadId }) => {
    return `Thread discusses best practices for API key rotation`;
  },
  {

```

```

        name: "get_thread",
        description: "Get a specific Slack thread.",
        schema: z.object({
            threadId: z.string(),
        }),
    },
);

import { createAgent } from "langchain";
import { ChatOpenAI } from "@langchain/openai";

const llm = new ChatOpenAI({ model: "gpt-5.5" });

const githubAgent = createAgent({
    model: llm,
    tools: [searchCode, searchIssues, searchPrs],
    systemPrompt: `
You are a GitHub expert. Answer questions about code,
API references, and implementation details by searching
repositories, issues, and pull requests.
`.trim(),
});

const notionAgent = createAgent({
    model: llm,
    tools: [searchNotion, getPage],
    systemPrompt: `
You are a Notion expert. Answer questions about internal
processes, policies, and team documentation by searching
the organization's Notion workspace.
`.trim(),
});

const slackAgent = createAgent({
    model: llm,
    tools: [searchSlack, getThread],
    systemPrompt: `
You are a Slack expert. Answer questions by searching
relevant threads and discussions where team members have
shared knowledge and solutions.
`.trim(),
});

const routerLlm = new ChatOpenAI({ model: "gpt-5.4-mini" });

// Define structured output schema for the classifier
const ClassificationResultSchema = z.object({
    // [!code highlight]
    classifications: z
        .array(
            z.object({

```

```

        source: z.enum(["github", "notion", "slack"]),
        query: z.string(),
    })
)
.describe("List of agents to invoke with their targeted sub-questions"),
});

async function classifyQuery(state: typeof RouterState.State) {
    const structuredLlm = routerLlm.withStructuredOutput(
        ClassificationResultSchema
    ); // [!code highlight]

    const result = await structuredLlm.invoke([
        {
            role: "system",
            content: `Analyze this query and determine which knowledge bases to consult.
For each relevant source, generate a targeted sub-question optimized for that source.

Available sources:
- github: Code, API references, implementation details, issues, pull requests
- notion: Internal documentation, processes, policies, team wikis
- slack: Team discussions, informal knowledge sharing, recent conversations

Return ONLY the sources that are relevant to the query. Each source should have
a targeted sub-question optimized for that specific knowledge domain.

Example for "How do I authenticate API requests?":
- github: "What authentication code exists? Search for auth middleware, JWT handling"
- notion: "What authentication documentation exists? Look for API auth guides"
(slack omitted because it's not relevant for this technical question)`,
        },
        { role: "user", content: state.query },
    ]);

    return { classifications: result.classifications };
}

function routeToAgents(state: typeof RouterState.State): Send[] {
    return state.classifications.map(
        (c) => new Send(c.source, { query: c.query }) // [!code highlight]
    );
}

async function queryGithub(state: typeof RouterState.State) {
    const result = await githubAgent.invoke({
        messages: [{ role: "user", content: state.query }], // [!code highlight]
    });
    return {
        results: [{ source: "github", result: result.messages.at(-1)?.content }],
    };
}

```

```

async function queryNotion(state: typeof RouterState.State) {
  const result = await notionAgent.invoke({
    messages: [{ role: "user", content: state.query }], // [!code highlight]
  });
  return {
    results: [{ source: "notion", result: result.messages.at(-1)?.content }],
  };
}

async function querySlack(state: typeof RouterState.State) {
  const result = await slackAgent.invoke({
    messages: [{ role: "user", content: state.query }], // [!code highlight]
  });
  return {
    results: [{ source: "slack", result: result.messages.at(-1)?.content }],
  };
}

async function synthesizeResults(state: typeof RouterState.State) {
  if (state.results.length === 0) {
    return { finalAnswer: "No results found from any knowledge source." };
  }

  // Format results for synthesis
  const formatted = state.results.map(
    (r) =>
      `**From ${r.source.charAt(0).toUpperCase() + r.source.slice(1)}:**\n${r.result}`
  );

  const synthesisResponse = await routerLlm.invoke([
    {
      role: "system",
      content: `Synthesize these search results to answer the original question: "${state.query}"`
    },
    {
      role: "user", content: formatted.join("\n\n")
    },
  ]);

  return { finalAnswer: synthesisResponse.content };
}

const workflow = new StateGraph(RouterState)
  .addNode("classify", classifyQuery)
  .addNode("github", queryGithub)
  .addNode("notion", queryNotion)

```

```

.addNode("slack", querySlack)
.addNode("synthesize", synthesizeResults)
.addEdge(START, "classify")
.addConditionalEdges("classify", routeToAgents, ["github", "notion", "slack"])
.addEdge("github", "synthesize")
.addEdge("notion", "synthesize")
.addEdge("slack", "synthesize")
.addEdge("synthesize", END)
.compile();

const result = await workflow.invoke({
  query: "How do I authenticate API requests?",
});

console.log("Original query:", result.query);
console.log("\nClassifications:");
for (const c of result.classifications) {
  console.log(` ${c.source}: ${c.query}`);
}
console.log(`\n${"=".repeat(60)}\n`);
console.log("Final Answer:");
console.log(result.finalAnswer);

```

9. Advanced: Stateful routers

The router we've built so far is **stateless** (each request is handled independently with no memory between calls). For multi-turn conversations, you need a **stateful** approach.

Tool wrapper approach

The simplest way to add conversation memory is to wrap the stateless router as a tool that a conversational agent can call:

python

```

from langgraph.checkpoint.memory import InMemorySaver

@tool
def search_knowledge_base(query: str) -> str:
    """Search across multiple knowledge sources (GitHub, Notion, Slack).

    Use this to find information about code, documentation, or team discussions.
    """
    result = workflow.invoke({"query": query})
    return result["final_answer"]

conversational_agent = create_agent(
    model,

```

```

tools=[search_knowledge_base],
system_prompt=(
    "You are a helpful assistant that answers questions about our organization. "
    "Use the search_knowledge_base tool to find information across our code, "
    "documentation, and team discussions."
),
checkpointer=InMemorySaver(),
)

```

js

```

import { MemorySaver } from "@langchain/langgraph";

const searchKnowledgeBase = tool(
  async ({ query }) => {
    const result = await workflow.invoke({ query });
    return result.finalAnswer;
  },
  {
    name: "search_knowledge_base",
    description: `Search across multiple knowledge sources (GitHub, Notion, Slack).
    Use this to find information about code, documentation, or team discussions.`,
    schema: z.object({
      query: z.string().describe("The search query"),
    }),
  }
);

const conversationalAgent = createAgent({
  model: llm,
  tools: [searchKnowledgeBase],
  systemPrompt: `
    You are a helpful assistant that answers questions about our organization.
    Use the search_knowledge_base tool to find information across our code,
    documentation, and team discussions.
  `.trim(),
  checkpointer: new MemorySaver(),
});

```

This approach keeps the router stateless while the conversational agent handles memory and context. The user can have a multi-turn conversation, and the agent will call the router tool as needed.

python

```

config = {"configurable": {"thread_id": "user-123"}}

```



```

result = conversational_agent.invoke(
    {"messages": [{"role": "user", "content": "How do I authenticate API requests?"}]},
    config
)
print(result["messages"][-1].content)

result = conversational_agent.invoke(
    {"messages": [{"role": "user", "content": "What about rate limiting for those endpoints?"}]},
    config
)
print(result["messages"][-1].content)

```

js

```

const config = { configurable: { thread_id: "user-123" } };
let conversationalAgentResult = await conversationalAgent.invoke(
  {
    messages: [
      { role: "user", content: "How do I authenticate API requests?" },
    ],
  },
  config
);
console.log(conversationalAgentResult.messages.at(-1)?.content);

conversationalAgentResult = await conversationalAgent.invoke(
  {
    messages: [
      {
        role: "user",
        content: "What about rate limiting for those endpoints?",
      },
    ],
  },
  config
);
console.log(conversationalAgentResult.messages.at(-1)?.content);

```

팁

The tool wrapper approach is recommended for most use cases. It provides clean separation: the router handles multi-source querying, while the conversational agent handles context and memory.

Full persistence approach

If you need the router itself to maintain state—for example, to use previous search results in routing decisions—use [persistence](#) to store message history at the router level.

주의

Stateful routers add complexity. When routing to different agents across turns, conversations may feel inconsistent if agents have different tones or prompts. Consider the [handoffs pattern](#) or [subagents pattern](#) instead—both provide clearer semantics for multi-turn conversations with different agents.

10. Key takeaways

The router pattern excels when you have:

- **Distinct verticals:** Separate knowledge domains that each require specialized tools and prompts
- **Parallel query needs:** Questions that benefit from querying multiple sources simultaneously
- **Synthesis requirements:** Results from multiple sources need to be combined into a coherent response

The pattern has three phases: **decompose** (analyze the query and generate targeted sub-questions), **route** (execute queries in parallel), and **synthesize** (combine results).

팁

When to use the router pattern

Use the router pattern when you have multiple independent knowledge sources, need low-latency parallel queries, and want explicit control over routing logic.

For simpler cases with dynamic tool selection, consider the [subagents pattern](#). For workflows where agents need to converse with users sequentially, consider [handoffs](#).

Next steps

- Learn about [handoffs](#) for agent-to-agent conversations
- Explore the [subagents pattern](#) for centralized orchestration
- Read the [multi-agent overview](#) to compare different patterns
- Use [LangSmith](#) to debug and monitor your router

Router

langchain/multi-agent/router.mdx

In the **router** architecture, a routing step classifies input and directs it to specialized [agents](#). This is useful when you have distinct **verticals** (separate knowledge domains that each require their own agent).

```
graph LR
  A([Query]) --> B[Router]
  B --> C[Agent A]
  B --> D[Agent B]
  B --> E[Agent C]
  C --> F[Synthesize]
  D --> F
  E --> F
  F --> G([Combined answer])

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,G trigger
class B,C,D,E,F process
```

Key characteristics

- Router decomposes the query
- Zero or more specialized agents are invoked in parallel
- Results are synthesized into a coherent response

When to use

Use the router pattern when you have distinct verticals (separate knowledge domains that each require their own agent), need to query multiple sources in parallel, and want to synthesize results into a combined response.

Basic implementation

The router classifies the query and directs it to the appropriate agent(s). Use [Command](#) for single-agent routing or [Send](#) for parallel fan-out to multiple agents.

Single agent

Use `Command` to route to a single specialized agent:

python

```
from langgraph.types import Command

def classify_query(query: str) -> str:
    """Use LLM to classify query and determine the appropriate agent."""
    # Classification logic here
    ...

def route_query(state: State) -> Command:
    """Route to the appropriate agent based on query classification."""
    active_agent = classify_query(state["query"])

    # Route to the selected agent
    return Command(goto=active_agent)
```

js

```
import { z } from "zod";
import { Command } from "@langchain/langgraph";

const ClassificationResult = z.object({
  query: z.string(),
  agent: z.string(),
});

function classifyQuery(query: string): z.infer<typeof ClassificationResult> {
  // Use LLM to classify query and determine the appropriate agent
  // Classification logic here
  ...
}

function routeQuery(state: z.infer<typeof ClassificationResult>) {
  const classification = classifyQuery(state.query);

  // Route to the selected agent
  return new Command({ goto: classification.agent });
}
```

Multiple agents (parallel)

Use `Send` to fan out to multiple specialized agents in parallel:

python

```
from typing import TypedDict
from langgraph.types import Send
```

```

class ClassificationResult(TypedDict):
    query: str
    agent: str

def classify_query(query: str) -> list[ClassificationResult]:
    """Use LLM to classify query and determine which agents to invoke."""
    # Classification logic here
    ...

def route_query(state: State):
    """Route to relevant agents based on query classification."""
    classifications = classify_query(state["query"])

    # Fan out to selected agents in parallel
    return [
        Send(c["agent"], {"query": c["query"]})
        for c in classifications
    ]

```

js

```

import { z } from "zod";
import { Command } from "@langchain/langgraph";

const ClassificationResult = z.object({
  query: z.string(),
  agent: z.string(),
});

function classifyQuery(query: string): z.infer<typeof ClassificationResult>[] {
  // Use LLM to classify query and determine the appropriate agent
  // Classification logic here
  ...
}

function routeQuery(state: typeof State.State) {
  const classifications = classifyQuery(state.query);

  // Fan out to selected agents in parallel
  return classifications.map(
    (c) => new Send(c.agent, { query: c.query })
  );
}

```

For a complete implementation, see the tutorial below.

[Tutorial: Build a multi-source knowledge base with routing](#)

Build a router that queries GitHub, Notion, and Slack in parallel, then synthesizes results into a coherent answer. Covers state definition, specialized agents, parallel execution with `Send`, and result synthesis.

Stateless vs. stateful

Two approaches: * [Stateless routers](#) address each request independently * [Stateful routers](#) maintain conversation history across requests

Stateless

Each request is routed independently—no memory between calls. For multi-turn conversations, see [Stateful routers](#).

팁

Router vs. Subagents: Both patterns can dispatch work to multiple agents, but they differ in how routing decisions are made:

- **Router:** A dedicated routing step (often a single LLM call or rule-based logic) that classifies the input and dispatches to agents. The router itself typically doesn't maintain conversation history or perform multi-turn orchestration—it's a preprocessing step.
- **Subagents:** A main supervisor agent dynamically decides which [subagents](#) to call as part of an ongoing conversation. The main agent maintains context, can call multiple subagents across turns, and orchestrates complex multi-step workflows.

Use a **router** when you have clear input categories and want deterministic or lightweight classification. Use a **supervisor** when you need flexible, conversation-aware orchestration where the LLM decides what to do next based on evolving context.

Stateful

For multi-turn conversations, you need to maintain context across invocations.

Tool wrapper

The simplest approach: wrap the stateless router as a tool that a conversational agent can call. The conversational agent handles memory and context; the router stays stateless. This avoids the complexity of managing conversation history across multiple parallel agents.

python

```

@tool
def search_docs(query: str) -> str:
    """Search across multiple documentation sources."""
    result = workflow.invoke({"query": query}) # [!code highlight]
    return result["final_answer"]

# Conversational agent uses the router as a tool
conversational_agent = create_agent(
    model,
    tools=[search_docs],
    prompt="You are a helpful assistant. Use search_docs to answer questions."
)

```

js

```

const searchDocs = tool(
  async ({ query }) => {
    const result = await workflow.invoke({ query }); // [!code highlight]
    return result.finalAnswer;
  },
  {
    name: "search_docs",
    description: "Search across multiple documentation sources",
    schema: z.object({
      query: z.string().describe("The search query"),
    }),
  }
);

// Conversational agent uses the router as a tool
const conversationalAgent = createAgent({
  model,
  tools: [searchDocs],
  systemPrompt: "You are a helpful assistant. Use search_docs to answer questions.",
});

```

Full persistence

If you need the router itself to maintain state, use [persistence](#) to store message history. When routing to an agent, fetch previous messages from state and selectively include them in the agent's context—this is a lever for [context engineering](#).

주의

Stateful routers require custom history management. If the router switches between agents across turns, conversations may not feel fluid to end users when agents have

different tones or prompts. With parallel invocation, you'll need to maintain history at the router level (inputs and synthesized outputs) and leverage this history in routing logic. Consider the [handoffs pattern](#) or [subagents pattern](#) instead—both provide clearer semantics for multi-turn conversations.

Build a SQL assistant with on-demand skills

langchain/multi-agent/skills-sql-assistant.mdx

This tutorial shows how to use **progressive disclosure** - a context management technique where the agent loads information on-demand rather than upfront - to implement **skills** (specialized prompt-based instructions). The agent loads skills via tool calls, rather than dynamically changing the system prompt, discovering and loading only the skills it needs for each task.

Use case: Imagine building an agent to help write SQL queries across different business verticals in a large enterprise. Your organization might have separate datastores for each vertical, or a single monolithic database with thousands of tables. Either way, loading all schemas upfront would overwhelm the context window. Progressive disclosure solves this by loading only the relevant schema when needed. This architecture also enables different product owners and stakeholders to independently contribute and maintain skills for their specific business verticals.

What you'll build: A SQL query assistant with two skills (sales analytics and inventory management). The agent sees lightweight skill descriptions in its system prompt, then loads full database schemas and business logic through tool calls only when relevant to the user's query.

참고

For a complete example of a SQL agent with query execution, error correction, and validation, see our [SQL Agent tutorial](#). This tutorial focuses on the progressive disclosure pattern which can be applied to any domain.

팁

Progressive disclosure was popularized by Anthropic as a technique for building scalable agent skills systems. This approach uses a three-level architecture (metadata → core content → detailed resources) where agents load information only as needed. For more on this technique, see [Equipping agents for the real world with Agent Skills](#).

How it works

Here's the flow when a user asks for a SQL query:

```
%%{init: {'theme':'base', 'themeVariables': { 'primaryColor':'#E5F4FF', 'primaryTextColor':'#030710', 'primaryBorderColor':'#006DDD', 'lineColor':'#40668D', 'secondaryColor':'#F6FFDB', 'tertiaryColor':'#FDF3FF', 'tertiaryBorderColor':'#7E65AE', 'tertiaryTextColor':'#504B5F'}}}%%
```

flowchart TD

```
Start([🗨️ User: Write SQL query<br/>for high-value customers]) --> SystemPrompt[📄 Agent sees skill descriptions:<br/>• sales_analytics<br/>• inventory_management]
```

```
SystemPrompt --> Decide[😬 Need sales schema]
```

```
Decide --> LoadSkill[🔑 load_skill<br/>'sales_analytics']
```

```
LoadSkill --> Schema[📊 Schema loaded:<br/>customers, orders tables<br/>+ business logic]
```

```
Schema --> WriteQuery[👉 Agent writes SQL query<br/>using schema knowledge]
```

```
WriteQuery --> Response([✅ Returns valid SQL<br/>following business rules])
```

%% Styling for light and dark modes

```
classDef startEnd fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
```

```
classDef process fill:#FDF3FF,stroke:#7E65AE,stroke-width:2px,color:#504B5F
```

```
classDef decision fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
```

```
classDef enrichment fill:#EBD0F0,stroke:#885270,stroke-width:2px,color:#441E33
```

```
class Start,Response startEnd
```

```
class SystemPrompt,LoadSkill,WriteQuery process
```

```
class Decide decision
```

```
class Schema enrichment
```

Why progressive disclosure: - **Reduces context usage** - load only the 2-3 skills needed for a task, not all available skills - **Enables team autonomy** - different teams can develop specialized skills independently (similar to other multi-agent architectures) - **Scales efficiently** - add dozens or hundreds of skills without overwhelming context - **Simplifies conversation history** - single agent with one conversation thread

What are skills: Skills, as popularized by Claude Code, are primarily prompt-based: self-contained units of specialized instructions for specific business tasks. In Claude Code, skills are exposed as directories with files on the file system, discovered through file operations. Skills guide behavior through prompts and can provide information about tool usage or include sample code for a coding agent to execute.

팁

Skills with progressive disclosure can be viewed as a form of [RAG \(Retrieval-Augmented Generation\)](#), where each skill is a retrieval unit—though not necessarily backed by embeddings or keyword search, but by tools for browsing content (like file operations or, in this tutorial, direct lookup).

Trade-offs: - **Latency:** Loading skills on-demand requires additional tool calls, which adds latency to the first request that needs each skill - **Workflow control:** Basic implementations

rely on prompting to guide skill usage - you cannot enforce hard constraints like "always try skill A before skill B" without custom logic

팁

Implementing your own skills system

When building your own skills implementation (as we do in this tutorial), the core concept is progressive disclosure - loading information on-demand. Beyond that, you have full flexibility in implementation:

- **Storage:** databases, S3, in-memory data structures, or any backend
- **Discovery:** direct lookup (this tutorial), RAG for large skill collections, file system scanning, or API calls
- **Loading logic:** customize latency characteristics and add logic to search through skill content or rank relevance
- **Side effects:** define what happens when a skill loads, such as exposing tools associated with that skill (covered in section 8)

This flexibility lets you optimize for your specific requirements around performance, storage, and workflow control.

Setup

Installation

This tutorial requires the `langchain` package:

python

```
```bash pip pip install langchain
```

```
```bash uv
uv add langchain
```

```
```bash conda conda install langchain -c conda-forge
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```bash npm
npm install langchain
```

```
```bash yarn yarn add langchain
```

```
```bash pnpm  
pnpm add langchain
```

For more details, see our [Installation guide](#).

LangSmith

Set up [LangSmith](#) to inspect what is happening inside your agent. Then set the following environment variables:

python

```
```bash Shell export LANGSMITH_TRACING="true" export LANGSMITH_API_KEY="..."
```

```
```python Python  
import getpass  
import os  
  
os.environ["LANGSMITH_TRACING"] = "true"  
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

js

```
```bash Shell export LANGSMITH_TRACING="true" export LANGSMITH_API_KEY="..."
```

```
```typescript TypeScript  
process.env.LANGSMITH_TRACING = "true";  
process.env.LANGSMITH_API_KEY = "...";
```

Select an LLM

Select a chat model from LangChain's suite of integrations:

python

js

1. Define skills

First, define the structure for skills. Each skill has a name, a brief description (shown in the system prompt), and full content (loaded on-demand):

python

```
from typing import TypedDict

class Skill(TypedDict): # [!code highlight]
    """A skill that can be progressively disclosed to the agent."""
    name: str # Unique identifier for the skill
    description: str # 1-2 sentence description to show in system prompt
    content: str # Full skill content with detailed instructions
```

js

```
import { z } from "zod";

// A skill that can be progressively disclosed to the agent
const SkillSchema = z.object({ // [!code highlight]
  name: z.string(), // Unique identifier for the skill
  description: z.string(), // 1-2 sentence description to show in system prompt
  content: z.string(), // Full skill content with detailed instructions
});

type Skill = z.infer<typeof SkillSchema>;
```

Now define example skills for a SQL query assistant. The skills are designed to be **lightweight in description** (shown to the agent upfront) but **detailed in content** (loaded only when needed):

View complete skill definitions

python

```
SKILLS: list[Skill] = [
    {
        "name": "sales_analytics",
        "description": "Database schema and business logic for sales data analysis including customers, orders, and revenue.",
        "content": """# Sales Analytics Schema

## Tables

### customers
- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)
```

```

### orders
- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

### order_items
- item_id (PRIMARY KEY)
- order_id (FOREIGN KEY -> orders)
- product_id
- quantity
- unit_price
- discount_percent

## Business Logic

**Active customers**: status = 'active' AND signup_date <= CURRENT_DATE - INTERVAL '90
days'

**Revenue calculation**: Only count orders with status = 'completed'. Use total_amount
from orders table, which already accounts for discounts.

**Customer lifetime value (CLV)**: Sum of all completed order amounts for a customer.

**High-value orders**: Orders with total_amount > 1000

## Example Query

-- Get top 10 customers by revenue in the last quarter
SELECT
    c.customer_id,
    c.name,
    c.customer_tier,
    SUM(o.total_amount) as total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
    AND o.order_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY c.customer_id, c.name, c.customer_tier
ORDER BY total_revenue DESC
LIMIT 10;
""",
    },
    {
        "name": "inventory_management",
        "description": "Database schema and business logic for inventory tracking inclu
ding products, warehouses, and stock levels.",
        "content": """"# Inventory Management Schema

```

Tables

products

- product_id (PRIMARY KEY)
- product_name
- sku
- category
- unit_cost
- reorder_point (minimum stock level before reordering)
- discontinued (boolean)

warehouses

- warehouse_id (PRIMARY KEY)
- warehouse_name
- location
- capacity

inventory

- inventory_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- quantity_on_hand
- last_updated

stock_movements

- movement_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- movement_type (inbound/outbound/transfer/adjustment)
- quantity (positive for inbound, negative for outbound)
- movement_date
- reference_number

Business Logic

****Available stock****: quantity_on_hand from inventory table where quantity_on_hand > 0

****Products needing reorder****: Products where total quantity_on_hand across all warehouses is less than or equal to the product's reorder_point

****Active products only****: Exclude products where discontinued = true unless specifically analyzing discontinued items

****Stock valuation****: quantity_on_hand * unit_cost for each product

Example Query

```
-- Find products below reorder point across all warehouses
SELECT
    p.product_id,
```

```

        p.product_name,
        p.reorder_point,
        SUM(i.quantity_on_hand) as total_stock,
        p.unit_cost,
        (p.reorder_point - SUM(i.quantity_on_hand)) as units_to_reorder
    FROM products p
    JOIN inventory i ON p.product_id = i.product_id
    WHERE p.discontinued = false
    GROUP BY p.product_id, p.product_name, p.reorder_point, p.unit_cost
    HAVING SUM(i.quantity_on_hand) <= p.reorder_point
    ORDER BY units_to_reorder DESC;
    """,
    },
]

```

js

```

import { context } from "langchain";

const SKILLS: Skill[] = [
  {
    name: "sales_analytics",
    description:
      "Database schema and business logic for sales data analysis including customers, orders, and revenue.",
    content: context`
      # Sales Analytics Schema

## Tables

### customers
- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)

### orders
- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

### order_items
- item_id (PRIMARY KEY)
- order_id (FOREIGN KEY -> orders)

```



```

- product_id
- quantity
- unit_price
- discount_percent

## Business Logic

**Active customers**:
status = 'active' AND signup_date <= CURRENT_DATE - INTERVAL '90 days'

**Revenue calculation**:
Only count orders with status = 'completed'.
Use total_amount from orders table, which already accounts for discounts.

**Customer lifetime value (CLV)**:
Sum of all completed order amounts for a customer.

**High-value orders**:
Orders with total_amount > 1000

## Example Query

-- Get top 10 customers by revenue in the last quarter
SELECT
    c.customer_id,
    c.name,
    c.customer_tier,
    SUM(o.total_amount) as total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
AND o.order_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY c.customer_id, c.name, c.customer_tier
ORDER BY total_revenue DESC
LIMIT 10;`,
},
{
    name: "inventory_management",
    description:
        "Database schema and business logic for inventory tracking including products, ware
houses, and stock levels.",
    content: context`
# Inventory Management Schema

## Tables

### products
- product_id (PRIMARY KEY)
- product_name
- sku
- category

```

```

- unit_cost
- reorder_point (minimum stock level before reordering)
- discontinued (boolean)

### warehouses
- warehouse_id (PRIMARY KEY)
- warehouse_name
- location
- capacity

### inventory
- inventory_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- quantity_on_hand
- last_updated

### stock_movements
- movement_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- movement_type (inbound/outbound/transfer/adjustment)
- quantity (positive for inbound, negative for outbound)
- movement_date
- reference_number

## Business Logic

**Available stock**:
quantity_on_hand from inventory table where quantity_on_hand > 0

**Products needing reorder**:
Products where total quantity_on_hand across all warehouses is less
than or equal to the product's reorder_point

**Active products only**:
Exclude products where discontinued = true unless specifically analyzing discontinued i
tems

**Stock valuation**:
quantity_on_hand * unit_cost for each product

## Example Query

-- Find products below reorder point across all warehouses
SELECT
    p.product_id,
    p.product_name,
    p.reorder_point,
    SUM(i.quantity_on_hand) as total_stock,
    p.unit_cost,

```

```

        (p.reorder_point - SUM(i.quantity_on_hand)) as units_to_reorder
    FROM products p
    JOIN inventory i ON p.product_id = i.product_id
    WHERE p.discontinued = false
    GROUP BY p.product_id, p.product_name, p.reorder_point, p.unit_cost
    HAVING SUM(i.quantity_on_hand) <= p.reorder_point
    ORDER BY units_to_reorder DESC`;
    },
];

```

2. Create skill loading tool

Create a tool to load full skill content on-demand:

python

```

from langchain.tools import tool

@tool # [!code highlight]
def load_skill(skill_name: str) -> str:
    """Load the full content of a skill into the agent's context.

    Use this when you need detailed information about how to handle a specific
    type of request. This will provide you with comprehensive instructions,
    policies, and guidelines for the skill area.

    Args:
        skill_name: The name of the skill to load (e.g., "expense_reporting", "travel_booking")
    """
    # Find and return the requested skill
    for skill in SKILLS:
        if skill["name"] == skill_name:
            return f"Loaded skill: {skill_name}\n\n{skill['content']}" # [!code highlight]

    # Skill not found
    available = ", ".join(s["name"] for s in SKILLS)
    return f"Skill '{skill_name}' not found. Available skills: {available}"

```

js

```

import { tool } from "langchain";
import { z } from "zod";

const loadSkill = tool( // [!code highlight]
  async ({ skillName }) => {
    // Find and return the requested skill

```

```

const skill = SKILLS.find((s) => s.name === skillName);
if (skill) {
  return `Loaded skill: ${skillName}\n\n${skill.content}`; // [!code highlight]
}

// Skill not found
const available = SKILLS.map((s) => s.name).join(", ");
return `Skill '${skillName}' not found. Available skills: ${available}`;
},
{
  name: "load_skill",
  description: `Load the full content of a skill into the agent's context.

Use this when you need detailed information about how to handle a specific
type of request. This will provide you with comprehensive instructions,
policies, and guidelines for the skill area.`,
  schema: z.object({
    skillName: z.string().describe("The name of the skill to load"),
  }),
}
);

```

The `load_skill` tool returns the full skill content as a string, which becomes part of the conversation as a `ToolMessage`. For more details on creating and using tools, see the [Tools guide](#).

3. Build skill middleware

Create custom middleware that injects skill descriptions into the system prompt. This middleware makes skills discoverable without loading their full content upfront.

참고

This guide demonstrates creating custom middleware. For a comprehensive guide on middleware concepts and patterns, see the [custom middleware documentation](#).

python

```

from langchain.agents.middleware import ModelRequest, ModelResponse, AgentMiddleware
from langchain.messages import SystemMessage
from typing import Callable

class SkillMiddleware(AgentMiddleware): # [!code highlight]
    """Middleware that injects skill descriptions into the system prompt."""

# Register the load_skill tool as a class variable
tools = [load_skill] # [!code highlight]

```

```

def __init__(self):
    """Initialize and generate the skills prompt from SKILLS."""
    # Build skills prompt from the SKILLS list
    skills_list = []
    for skill in SKILLS:
        skills_list.append(
            f"- **{skill['name']}**: {skill['description']}"
        )
    self.skills_prompt = "\n".join(skills_list)

def wrap_model_call(
    self,
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """Sync: Inject skill descriptions into system prompt."""
    # Build the skills addendum
    skills_addendum = ( # [!code highlight]
        f"\n\n## Available Skills\n\n{self.skills_prompt}\n\n" # [!code highlight]
        "Use the load_skill tool when you need detailed information " # [!code highlight]
    )

    "about handling a specific type of request." # [!code highlight]

    # Append to system message content blocks
    new_content = list(request.system_message.content_blocks) + [
        {"type": "text", "text": skills_addendum}
    ]
    new_system_message = SystemMessage(content=new_content)
    modified_request = request.override(system_message=new_system_message)
    return handler(modified_request)

```

js

```

import { createMiddleware } from "langchain";

// Build skills prompt from the SKILLS list
const skillsPrompt = SKILLS.map(
    (skill) => ` - **${skill.name}**: ${skill.description}`
).join("\n");

const skillMiddleware = createMiddleware({ // [!code highlight]
    name: "skillMiddleware",
    tools: [loadSkill], // [!code highlight]
    wrapModelCall: async (request, handler) => {
        // Build the skills addendum
        const skillsAddendum = // [!code highlight]
            ` \n\n## Available Skills\n\n${skillsPrompt}\n\n` + // [!code highlight]

```

```

        "Use the load_skill tool when you need detailed information " + // [!code highlight]
        "about handling a specific type of request."; // [!code highlight]

// Append to system prompt
const newSystemPrompt = request.systemPrompt + skillsAddendum;

return handler({
    ...request,
    systemPrompt: newSystemPrompt,
});
},
});

```

The middleware appends skill descriptions to the system prompt, making the agent aware of available skills without loading their full content. The `load_skill` tool is registered as a class variable, making it available to the agent.

참고

Production consideration: This tutorial loads the skill list in `__init__` for simplicity. In a production system, you may want to load skills in the `before_agent` hook instead, allowing them to be refreshed periodically to reflect up-to-date changes (e.g., when new skills are added or existing ones are modified). See the [before_agent hook documentation](#) for details.

4. Create the agent with skill support

Now create the agent with the skill middleware and a checkpointer for state persistence:

python

```

from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

# Create the agent with skill support
agent = create_agent(
    model,
    system_prompt=(
        "You are a SQL query assistant that helps users "
        "write queries against business databases."
    ),
    middleware=[SkillMiddleware()], # [!code highlight]
    checkpointer=InMemorySaver(),
)

```

js

```
import { createAgent } from "langchain";
import { MemorySaver } from "@langchain/langgraph";

// Create the agent with skill support
const agent = createAgent({
  model,
  systemPrompt:
    "You are a SQL query assistant that helps users " +
    "write queries against business databases.",
  middleware: [skillMiddleware], // [!code highlight]
  checkpoint: new MemorySaver(),
});
```

The agent now has access to skill descriptions in its system prompt and can call `load_skill` to retrieve full skill content when needed. The checkpoint maintains conversation history across turns.

5. Test progressive disclosure

Test the agent with a question that requires skill-specific knowledge:

python

```
from langchain_core.utils.uuid import uuid7

# Configuration for this conversation thread
thread_id = str(uuid7())
config = {"configurable": {"thread_id": thread_id}}

# Ask for a SQL query
result = agent.invoke( # [!code highlight]
  {
    "messages": [
      {
        "role": "user",
        "content": (
          "Write a SQL query to find all customers "
          "who made orders over $1000 in the last month"
        ),
      },
    ],
  },
  config
)
```

```
# Print the conversation
for message in result["messages"]:
    if hasattr(message, 'pretty_print'):
        message.pretty_print()
    else:
        print(f"{message.type}: {message.content}")
```

js

```
// Configuration for this conversation thread
const threadId = crypto.randomUUID();
const config = { configurable: { thread_id: threadId } };

// Ask for a SQL query
const result = await agent.invoke( // [!code highlight]
{
  messages: [
    {
      role: "user",
      content:
        "Write a SQL query to find all customers " +
        "who made orders over $1000 in the last month",
    },
  ],
},
config
);

// Print the conversation
for (const message of result.messages) {
  console.log(`${message._getType()}: ${message.content}`);
}
```

Expected output:

```
===== Human Message =====

Write a SQL query to find all customers who made orders over $1000 in the last month
===== Ai Message =====

Tool Calls:
  load_skill (call_abc123)
Call ID: call_abc123
Args:
  skill_name: sales_analytics
===== Tool Message =====
Name: load_skill

Loaded skill: sales_analytics
```



```
# Sales Analytics Schema
```

```
## Tables
```

```
### customers
```

- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)

```
### orders
```

- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

```
[... rest of schema ...]
```

```
## Business Logic
```

```
**High-value orders**: Orders with `total_amount > 1000`
```

```
**Revenue calculation**: Only count orders with `status = 'completed'`
```

```
===== Ai Message =====
```

Here's a SQL query to find all customers who made orders over \$1000 in the last month:

```
\\\`sql
SELECT DISTINCT
    c.customer_id,
    c.name,
    c.email,
    c.customer_tier
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount > 1000
    AND o.status = 'completed'
    AND o.order_date >= CURRENT_DATE - INTERVAL '1 month'
ORDER BY c.customer_id;
\\\`
```

This query:

- Joins customers with their orders
- Filters for high-value orders (>\$1000) using the total_amount field
- Only includes completed orders (as per the business logic)

- Restricts to orders from the last month
- Returns distinct customers to avoid duplicates if they made multiple qualifying orders

The agent saw the lightweight skill description in its system prompt, recognized the question required sales database knowledge, called `load_skill("sales_analytics")` to get the full schema and business logic, and then used that information to write a correct query following the database conventions.

6. Advanced: Add constraints with custom state

Optional: Track loaded skills and enforce tool constraints

You can add constraints to enforce that certain tools are only available after specific skills have been loaded. This requires tracking which skills have been loaded in custom agent state.

Define custom state

First, extend the agent state to track loaded skills:

python

```
from langchain.agents.middleware import AgentState

class CustomState(AgentState): # [!code highlight]
    skills_loaded: NotRequired[list[str]] # Track which skills have been loaded # [!code highlight]
```

js

```
import { StateSchema } from "@langchain/langgraph";
import { z } from "zod";

const CustomState = new StateSchema({
  skillsloaded: z.array(z.string()).optional(), // Track which skills have been loaded
  // [!code highlight]
});
```

Update `load_skill` to modify state

Modify the `load_skill` tool to update state when a skill is loaded:

python

```
from langgraph.types import Command # [!code highlight]
from langchain.tools import tool, ToolRuntime
```

```

from langchain.messages import ToolMessage # [!code highlight]

@tool
def load_skill(skill_name: str, runtime: ToolRuntime) -> Command: # [!code highlight]
    """Load the full content of a skill into the agent's context.

    Use this when you need detailed information about how to handle a specific
    type of request. This will provide you with comprehensive instructions,
    policies, and guidelines for the skill area.

    Args:
        skill_name: The name of the skill to load
    """
    # Find and return the requested skill
    for skill in SKILLS:
        if skill["name"] == skill_name:
            skill_content = f"Loaded skill: {skill_name}\n\n{skill['content']}"

    # Update state to track loaded skill
    return Command( # [!code highlight]
        update={ # [!code highlight]
            "messages": [ # [!code highlight]
                ToolMessage( # [!code highlight]
                    content=skill_content, # [!code highlight]
                    tool_call_id=runtime.tool_call_id, # [!code highlight]
                ) # [!code highlight]
            ], # [!code highlight]
            "skills_loaded": [skill_name], # [!code highlight]
        } # [!code highlight]
    ) # [!code highlight]

    # Skill not found
    available = ", ".join(s["name"] for s in SKILLS)
    return Command(
        update={
            "messages": [
                ToolMessage(
                    content=f"Skill '{skill_name}' not found. Available skills: {available}",
                    tool_call_id=runtime.tool_call_id,
                )
            ]
        }
    )

```

js

```

import { tool, ToolMessage, type ToolRuntime } from "langchain";
import { Command } from "@langchain/langgraph"; // [!code highlight]

```

```

import { z } from "zod";

const loadSkill = tool( // [!code highlight]
  async ({ skillName }, runtime: ToolRuntime<typeof CustomState.State>) => {
    // Find and return the requested skill
    const skill = SKILLS.find((s) => s.name === skillName);

    if (skill) {
      const skillContent = `Loaded skill: ${skillName}\n\n${skill.content}`;

      // Update state to track loaded skill
      return new Command({ // [!code highlight]
        update: { // [!code highlight]
          messages: [ // [!code highlight]
            new ToolMessage({ // [!code highlight]
              content: skillContent, // [!code highlight]
              tool_call_id: runtime.toolCallId, // [!code highlight]
            }, // [!code highlight]
          ], // [!code highlight]
          skillsLoaded: [skillName], // [!code highlight]
        }, // [!code highlight]
      }); // [!code highlight]
    }

    // Skill not found
    const available = SKILLS.map((s) => s.name).join(", ");
    return new Command({
      update: {
        messages: [
          new ToolMessage({
            content: `Skill '${skillName}' not found. Available skills: ${available}`,
            tool_call_id: runtime.toolCallId,
          }),
        ],
      },
    });
  },
  {
    name: "load_skill",
    description: `Load the full content of a skill into the agent's context.`,
    schema: z.object({
      skillName: z.string().describe("The name of the skill to load"),
    }),
  }
);

```

Create constrained tool

Create a tool that's only usable after a specific skill has been loaded:

python

```
@tool
def write_sql_query( # [!code highlight]
    query: str,
    vertical: str,
    runtime: ToolRuntime,
) -> str:
    """Write and validate a SQL query for a specific business vertical.

    This tool helps format and validate SQL queries. You must load the
    appropriate skill first to understand the database schema.

    Args:
        query: The SQL query to write
        vertical: The business vertical (sales_analytics or inventory_management)
    """
    # Check if the required skill has been loaded
    skills_loaded = runtime.state.get("skills_loaded", []) # [!code highlight]

    if vertical not in skills_loaded: # [!code highlight]
        return ( # [!code highlight]
            f"Error: You must load the '{vertical}' skill first " # [!code highlight]
            f"to understand the database schema before writing queries. " # [!code highlig
ht]
            f"Use load_skill('{vertical}') to load the schema." # [!code highlight]
        ) # [!code highlight]

    # Validate and format the query
    return (
        f"SQL Query for {vertical}:\n\n"
        f"```sql\n{query}\n```\n\n"
        f"✓ Query validated against {vertical} schema\n"
        f"Ready to execute against the database."
    )
```

js

```
const writeSqlQuery = tool( // [!code highlight]
    async ({ query, vertical }, runtime: ToolRuntime<typeof CustomState.State>) => {
        // Check if the required skill has been loaded
        const skillsLoaded = runtime.state.skillsLoaded ?? []; // [!code highlight]

        if (!skillsLoaded.includes(vertical)) { // [!code highlight]
            return ( // [!code highlight]
                `Error: You must load the '${vertical}' skill first ` + // [!code highlight]
                `to understand the database schema before writing queries. ` + // [!code highligh
t]
            )
        }
    }
);
```



```
// ... rest of the middleware implementation stays the same
});
```

Create the agent with the middleware that registers the constrained tool:

python

```
agent = create_agent(
    model,
    system_prompt=(
        "You are a SQL query assistant that helps users "
        "write queries against business databases."
    ),
    middleware=[SkillMiddleware()], # [!code highlight]
    checkpointer=InMemorySaver(),
)
```

js

```
const agent = createAgent({
  model,
  systemPrompt:
    "You are a SQL query assistant that helps users " +
    "write queries against business databases.",
  middleware: [skillMiddleware], // [!code highlight]
  checkpointer: new MemorySaver(),
});
```

Now if the agent tries to use `write_sql_query` before loading the required skill, it will receive an error message prompting it to load the appropriate skill (e.g., `sales_analytics` or `inventory_management`) first. This ensures the agent has the necessary schema knowledge before attempting to validate queries.

Complete example

View complete runnable script

Here's a complete, runnable implementation combining all the pieces from this tutorial:

python

```
from langchain_core.utils.uuid import uuid7
from typing import TypedDict, NotRequired
from langchain.tools import tool
from langchain.agents import create_agent
```

```

from langchain.agents.middleware import ModelRequest, ModelResponse, AgentMiddleware
from langchain.messages import SystemMessage
from langgraph.checkpoint.memory import InMemorySaver
from typing import Callable

# Define skill structure
class Skill(TypedDict):
    """A skill that can be progressively disclosed to the agent."""
    name: str
    description: str
    content: str

# Define skills with schemas and business logic
SKILLS: list[Skill] = [
    {
        "name": "sales_analytics",
        "description": "Database schema and business logic for sales data analysis including customers, orders, and revenue.",
        "content": """# Sales Analytics Schema

## Tables

### customers
- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)

### orders
- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

### order_items
- item_id (PRIMARY KEY)
- order_id (FOREIGN KEY -> orders)
- product_id
- quantity
- unit_price
- discount_percent

## Business Logic

**Active customers**: status = 'active' AND signup_date <= CURRENT_DATE - INTERVAL '90 days'

```


****Revenue calculation****: Only count orders with status = 'completed'. Use total_amount from orders table, which already accounts for discounts.

****Customer lifetime value (CLV)****: Sum of all completed order amounts for a customer.

****High-value orders****: Orders with total_amount > 1000

Example Query

-- Get top 10 customers by revenue in the last quarter

SELECT

 c.customer_id,

 c.name,

 c.customer_tier,

 SUM(o.total_amount) as total_revenue

FROM customers c

JOIN orders o ON c.customer_id = o.customer_id

WHERE o.status = 'completed'

 AND o.order_date >= CURRENT_DATE - INTERVAL '3 months'

GROUP BY c.customer_id, c.name, c.customer_tier

ORDER BY total_revenue DESC

LIMIT 10;

""",

 },

 {

 "name": "inventory_management",

 "description": "Database schema and business logic for inventory tracking including products, warehouses, and stock levels.",

 "content": """"# Inventory Management Schema

Tables

products

- product_id (PRIMARY KEY)

- product_name

- sku

- category

- unit_cost

- reorder_point (minimum stock level before reordering)

- discontinued (boolean)

warehouses

- warehouse_id (PRIMARY KEY)

- warehouse_name

- location

- capacity

inventory

- inventory_id (PRIMARY KEY)

- product_id (FOREIGN KEY -> products)

- warehouse_id (FOREIGN KEY -> warehouses)

```

- quantity_on_hand
- last_updated

### stock_movements
- movement_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- movement_type (inbound/outbound/transfer/adjustment)
- quantity (positive for inbound, negative for outbound)
- movement_date
- reference_number

## Business Logic

**Available stock**: quantity_on_hand from inventory table where quantity_on_hand > 0

**Products needing reorder**: Products where total quantity_on_hand across all warehouses is less than or equal to the product's reorder_point

**Active products only**: Exclude products where discontinued = true unless specifically analyzing discontinued items

**Stock valuation**: quantity_on_hand * unit_cost for each product

## Example Query

-- Find products below reorder point across all warehouses
SELECT
    p.product_id,
    p.product_name,
    p.reorder_point,
    SUM(i.quantity_on_hand) as total_stock,
    p.unit_cost,
    (p.reorder_point - SUM(i.quantity_on_hand)) as units_to_reorder
FROM products p
JOIN inventory i ON p.product_id = i.product_id
WHERE p.discontinued = false
GROUP BY p.product_id, p.product_name, p.reorder_point, p.unit_cost
HAVING SUM(i.quantity_on_hand) <= p.reorder_point
ORDER BY units_to_reorder DESC;
"""
    },
]

# Create skill loading tool
@tool
def load_skill(skill_name: str) -> str:
    """Load the full content of a skill into the agent's context.

    Use this when you need detailed information about how to handle a specific type of request. This will provide you with comprehensive instructions,

```

policies, and guidelines for the skill area.

Args:

skill_name: The name of the skill to load (e.g., "sales_analytics", "inventory_management")

"""

Find and return the requested skill

for skill in SKILLS:

if skill["name"] == skill_name:

return f"Loaded skill: {skill_name}\n\n{skill['content']}"

Skill not found

available = ", ".join(s["name"] for s in SKILLS)

return f"Skill '{skill_name}' not found. Available skills: {available}"

Create skill middleware

class SkillMiddleware(AgentMiddleware):

"""Middleware that injects skill descriptions into the system prompt."""

Register the load_skill tool as a class variable

tools = [load_skill]

def __init__(self):

"""Initialize and generate the skills prompt from SKILLS."""

Build skills prompt from the SKILLS list

skills_list = []

for skill in SKILLS:

skills_list.append(

f"- **{skill['name']}**: {skill['description']}"

)

self.skills_prompt = "\n".join(skills_list)

def wrap_model_call(

self,

request: ModelRequest,

handler: Callable[[ModelRequest], ModelResponse],

) -> ModelResponse:

"""Sync: Inject skill descriptions into system prompt."""

Build the skills addendum

skills_addendum = (

f"\n\n## Available Skills\n\n{self.skills_prompt}\n\n"

"Use the load_skill tool when you need detailed information "

"about handling a specific type of request."

)

Append to system message content blocks

new_content = list(request.system_message.content_blocks) + [

{"type": "text", "text": skills_addendum}

]

new_system_message = SystemMessage(content=new_content)

modified_request = request.override(system_message=new_system_message)

```

return handler(modified_request)

# Initialize your chat model (replace with your model)
# Example: from langchain_anthropic import ChatAnthropic
# model = ChatAnthropic(model="claude-3-5-sonnet-20241022")
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.5")

# Create the agent with skill support
agent = create_agent(
    model,
    system_prompt=(
        "You are a SQL query assistant that helps users "
        "write queries against business databases."
    ),
    middleware=[SkillMiddleware()],
    checkpoint=InMemorySaver(),
)

# Example usage
if __name__ == "__main__":
    # Configuration for this conversation thread
    thread_id = str(uuid7())
    config = {"configurable": {"thread_id": thread_id}}

    # Ask for a SQL query
    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": (
                        "Write a SQL query to find all customers "
                        "who made orders over $1000 in the last month"
                    )
                },
            ]
        },
        config
    )

    # Print the conversation
    for message in result["messages"]:
        if hasattr(message, 'pretty_print'):
            message.pretty_print()
        else:
            print(f"{message.type}: {message.content}")

```

```

import {
  tool,
  createAgent,
  createMiddleware,
  ToolMessage,
  context,
  type ToolRuntime,
} from "langchain";
import { MemorySaver, Command } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { z } from "zod";

// A skill that can be progressively disclosed to the agent
const SkillSchema = z.object({
  name: z.string(), // Unique identifier for the skill
  description: z.string(), // 1-2 sentence description to show in system prompt
  content: z.string(), // Full skill content with detailed instructions
});

type Skill = z.infer<typeof SkillSchema>;

const SKILLS: Skill[] = [
  {
    name: "sales_analytics",
    description:
      "Database schema and business logic for sales data analysis including customers, orders, and revenue.",
    content: context`
      # Sales Analytics Schema

## Tables

### customers
- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)

### orders
- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

### order_items
- item_id (PRIMARY KEY)

```

```

- order_id (FOREIGN KEY -> orders)
- product_id
- quantity
- unit_price
- discount_percent

## Business Logic

**Active customers**: status = 'active' AND signup_date <= CURRENT_DATE - INTERVAL '90
days'

**Revenue calculation**:
Only count orders with status = 'completed'. Use total_amount from orders table,
which already accounts for discounts.

**Customer lifetime value (CLV)**:
Sum of all completed order amounts for a customer.

**High-value orders**:
Orders with total_amount > 1000

## Example Query
-- Get top 10 customers by revenue in the last quarter
SELECT
    c.customer_id,
    c.name,
    c.customer_tier,
    SUM(o.total_amount) as total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
AND o.order_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY c.customer_id, c.name, c.customer_tier
ORDER BY total_revenue DESC
LIMIT 10;`,
},
{
    name: "inventory_management",
    description:
        "Database schema and business logic for inventory tracking including products, ware
houses, and stock levels.",
    content: context`
# Inventory Management Schema

## Tables

### products
- product_id (PRIMARY KEY)
- product_name
- sku
- category

```

```

- unit_cost
- reorder_point (minimum stock level before reordering)
- discontinued (boolean)

### warehouses
- warehouse_id (PRIMARY KEY)
- warehouse_name
- location
- capacity

### inventory
- inventory_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- quantity_on_hand
- last_updated

### stock_movements
- movement_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- movement_type (inbound/outbound/transfer/adjustment)
- quantity (positive for inbound, negative for outbound)
- movement_date
- reference_number

## Business Logic

**Available stock**:
quantity_on_hand from inventory table where quantity_on_hand > 0

**Products needing reorder**:
Products where total quantity_on_hand across all warehouses is
less than or equal to the product's reorder_point

**Active products only**:
Exclude products where discontinued = true unless specifically
analyzing discontinued items

**Stock valuation**:
quantity_on_hand * unit_cost for each product

## Example Query

-- Find products below reorder point across all warehouses
SELECT
    p.product_id,
    p.product_name,
    p.reorder_point,
    SUM(i.quantity_on_hand) as total_stock,
    p.unit_cost,

```

```

        (p.reorder_point - SUM(i.quantity_on_hand)) as units_to_reorder
    FROM products p
    JOIN inventory i ON p.product_id = i.product_id
    WHERE p.discontinued = false
    GROUP BY p.product_id, p.product_name, p.reorder_point, p.unit_cost
    HAVING SUM(i.quantity_on_hand) <= p.reorder_point
    ORDER BY units_to_reorder DESC`;
    },
];

// const loadSkill = tool(
//   async ({ skillName }) => {
//     // Find and return the requested skill
//     const skill = SKILLS.find((s) => s.name === skillName);
//     if (skill) {
//       return `Loaded skill: ${skillName}\n\n${skill.content}`;
//     }

//     // Skill not found
//     const available = SKILLS.map((s) => s.name).join(", ");
//     return `Skill '${skillName}' not found. Available skills: ${available}`;
//   },
//   {
//     name: "load_skill",
//     description: `Load the full content of a skill into the agent's context.

// Use this when you need detailed information about how to handle a specific
// type of request. This will provide you with comprehensive instructions,
// policies, and guidelines for the skill area.`;
//     schema: z.object({
//       skillName: z.string().describe("The name of the skill to load"),
//     }),
//   }
// );

// Build skills prompt from the SKILLS list
const skillsPrompt = SKILLS.map(
  (skill) => ` - **${skill.name}**: ${skill.description}`
).join("\n");

const skillMiddleware = createMiddleware({
  name: "skillMiddleware",
  tools: [loadSkill],
  wrapModelCall: async (request, handler) => {
    // Build the skills addendum
    const skillsAddendum =
      `
\n\n## Available Skills\n\n${skillsPrompt}\n\n` +
      "Use the load_skill tool when you need detailed information " +
      "about handling a specific type of request.";

    // Append to system prompt

```



```

const newSystemPrompt = request.systemPrompt + skillsAddendum;

    return handler({
        ...request,
        systemPrompt: newSystemPrompt,
    });
},
});

const model = new ChatOpenAI({
    model: "gpt-5.4-mini",
    temperature: 0,
});

// Create the agent with skill support
const agent = createAgent({
    model,
    systemPrompt:
        "You are a SQL query assistant that helps users " +
        "write queries against business databases.",
    middleware: [skillMiddleware],
    checkpointer: new MemorySaver(),
});

// Configuration for this conversation thread
const threadId = crypto.randomUUID();
const config = { configurable: { thread_id: threadId } };

// Ask for a SQL query
const result = await agent.invoke(
    {
        messages: [
            {
                role: "user",
                content:
                    "Write a SQL query to find all customers " +
                    "who made orders over $1000 in the last month",
            },
        ],
    },
    config
);

// Print the conversation
for (const message of result.messages) {
    console.log(`${message.type}: ${message.content}`);
}

```

This complete example includes: - Skill definitions with full database schemas - The `load_skill` tool for on-demand loading - `SkillMiddleware` that injects skill descriptions into the system prompt - Agent creation with middleware and checkpointing - Example usage showing how the agent loads skills and writes SQL queries

To run this, you'll need to: 1. Install required packages: `pip install langchain langchain-openai langgraph` 2. Set your API key (e.g., `export OPENAI_API_KEY=...`) 3. Replace the model initialization with your preferred LLM provider

Implementation variations

View implementation options and trade-offs

This tutorial implemented skills as in-memory Python dictionaries loaded through tool calls. However, there are several ways to implement progressive disclosure with skills:

Storage backends: - **In-memory** (this tutorial): Skills defined as Python data structures, fast access, no I/O overhead - **File system** (Claude Code approach): Skills as directories with files, discovered via file operations like `read_file` - **Remote storage**: Skills in S3, databases, Notion, or APIs, fetched on-demand

Skill discovery (how the agent learns which skills exist): - **System prompt listing**: Skill descriptions in system prompt (used in this tutorial) - **File-based**: Discover skills by scanning directories (Claude Code approach) - **Registry-based**: Query a skill registry service or API for available skills - **Dynamic lookup**: List available skills via a tool call

Progressive disclosure strategies (how skill content is loaded): - **Single load**: Load entire skill content in one tool call (used in this tutorial) - **Paginated**: Load skill content in multiple pages/chunks for large skills - **Search-based**: Search within a specific skill's content for relevant sections (e.g., using `grep`/`read` operations on skill files) - **Hierarchical**: Load skill overview first, then drill into specific subsections

Size considerations (uncalibrated mental model - optimize for your system): - **Small skills** (< 1K tokens / ~750 words): Can be included directly in system prompt and cached with prompt caching for cost savings and faster responses - **Medium skills** (1-10K tokens / ~750-7.5K words): Benefit from on-demand loading to avoid context overhead (this tutorial) - **Large skills** (> 10K tokens / ~7.5K words, or > 5-10% of context window): Should use progressive disclosure techniques like pagination, search-based loading, or hierarchical exploration to avoid consuming excessive context

The choice depends on your requirements: in-memory is fastest but requires redeployment for skill updates, while file-based or remote storage enables dynamic skill management without code changes.

Progressive disclosure and context engineering

Combining with few-shot prompting and other techniques

Progressive disclosure is fundamentally a [context engineering technique](#) - you're managing what information is available to the agent and when. This tutorial focused on loading database schemas, but the same principles apply to other types of context.

Combining with few-shot prompting

For the SQL query use case, you could extend progressive disclosure to dynamically load **few-shot examples** that match the user's query:

Example approach: 1. User asks: "Find customers who haven't ordered in 6 months" 2. Agent loads `sales_analytics` schema (as shown in this tutorial) 3. Agent also loads 2-3 relevant example queries (via semantic search or tag-based lookup): - Query for finding inactive customers - Query with date-based filtering - Query joining customers and orders tables 4. Agent writes query using both schema knowledge AND example patterns

This combination of progressive disclosure (loading schemas on-demand) and dynamic few-shot prompting (loading relevant examples) creates a powerful context engineering pattern that scales to large knowledge bases while providing high-quality, grounded outputs.

Next steps

- Learn about [middleware](#) for more dynamic agent behaviors
- Explore [context engineering](#) techniques for managing agent context
- Explore the [handoffs pattern](#) for sequential workflows
- Read the [subagents pattern](#) for parallel task routing
- See [multi-agent patterns](#) for other approaches to specialized agents
- Use [LangSmith](#) to debug and monitor skill loading

Skills

langchain/multi-agent/skills.mdx

In the **skills** architecture, specialized capabilities are packaged as invocable "skills" that augment an [agent's](#) behavior. Skills are primarily prompt-driven specializations that an agent can invoke on-demand. For built-in skill support, see [Deep Agents](#).

팁

This pattern is conceptually identical to [Agent Skills](#) and [llms.txt](#) (introduced by Jeremy Howard), which uses tool calling for progressive disclosure of documentation. The skills pattern applies progressive disclosure to specialized prompts and domain knowledge rather than just documentation pages.

For ready-to-use skills that improve your agent's performance on LangChain ecosystem tasks, see the [LangChain Skills](#) repository.

```
graph LR
  A[User] --> B[Agent]
  B --> C[Skill A]
  B --> D[Skill B]
  B --> E[Skill C]
  B --> A

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A trigger
class B,C,D,E process
```

Key characteristics

- Prompt-driven specialization: Skills are primarily defined by specialized prompts
- Progressive disclosure: Skills become available based on context or user needs
- Team distribution: Different teams can develop and maintain skills independently
- Lightweight composition: Skills are simpler than full sub-agents
- Reference awareness: Skills can reference scripts, templates, and other resources

When to use

Use the skills pattern when you want a single [agent](#) with many possible specializations, you don't need to enforce specific constraints between skills, or different teams need to develop capabilities independently. Common examples include coding assistants (skills for different

languages or tasks), knowledge bases (skills for different domains), and creative assistants (skills for different formats).

Basic implementation

python

```
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def load_skill(skill_name: str) -> str:
    """Load a specialized skill prompt.

    Available skills:
    - write_sql: SQL query writing expert
    - review_legal_doc: Legal document reviewer

    Returns the skill's prompt and context.
    """
    # Load skill content from file/database
    ...

agent = create_agent(
    model="gpt-5.5",
    tools=[load_skill],
    system_prompt=(
        "You are a helpful assistant. "
        "You have access to two skills: "
        "write_sql and review_legal_doc. "
        "Use load_skill to access them."
    ),
)
```

js

```
import { tool, createAgent } from "langchain";
import * as z from "zod";

const loadSkill = tool(
  async ({ skillName }) => {
    // Load skill content from file/database
    return "";
  },
  {
    name: "load_skill",
    description: `Load a specialized skill.`
  }
);
```

```
Available skills:
- write_sql: SQL query writing expert
- review_legal_doc: Legal document reviewer
```

```
Returns the skill's prompt and context.`
  schema: z.object({
    skillName: z
      .string()
      .describe("Name of skill to load")
  })
}
);

const agent = createAgent({
  model: "gpt-5.5",
  tools: [loadSkill],
  systemPrompt: (
    "You are a helpful assistant. " +
    "You have access to two skills: " +
    "write_sql and review_legal_doc. " +
    "Use load_skill to access them."
  ),
});
```

For a complete implementation, see the tutorial below.

[Tutorial: Build a SQL assistant with on-demand skills](#)

Learn how to implement skills with progressive disclosure, where the agent loads specialized prompts and schemas on-demand rather than upfront.

Extending the pattern

When writing custom implementations, you can extend the basic skills pattern in several ways:

- **Dynamic tool registration:** Combine progressive disclosure with state management to register new [tools](#) as skills load. For example, loading a "database_admin" skill could both add specialized context and register database-specific tools (backup, restore, migrate). This uses the same tool-and-state mechanisms used across multi-agent patterns—tools updating state to dynamically change agent capabilities.
- **Hierarchical skills:** Skills can define other skills in a tree structure, creating nested specializations. For instance, loading a "data_science" skill might make available sub-skills like "pandas_expert", "visualization", and "statistical_analysis". Each sub-skill can be loaded independently as needed, allowing for fine-grained progressive disclosure of domain knowledge. This hierarchical approach helps manage large knowledge bases by

organizing capabilities into logical groupings that can be discovered and loaded on-demand.

- **Reference awareness:** While each skill only has one prompt, this prompt can reference the location of other assets and provide information on when the agent should use those assets. When those assets become relevant, the agent will know that those files exist and read them into memory as needed to complete tasks. This also follows the progressive disclosure pattern and limits the information in the context window.

Build a personal assistant with subagents

langchain/multi-agent/subagents-personal-assistant.mdx

Overview

The **supervisor pattern** is a [multi-agent](#) architecture where a central supervisor agent coordinates specialized worker agents. This approach excels when tasks require different types of expertise. Rather than building one agent that manages tool selection across domains, you create focused specialists coordinated by a supervisor who understands the overall workflow.

In this tutorial, you'll build a personal assistant system that demonstrates these benefits through a realistic workflow. The system will coordinate two specialists with fundamentally different responsibilities:

- A **calendar agent** that handles scheduling, availability checking, and event management.
- An **email agent** that manages communication, drafts messages, and sends notifications.

We will also incorporate [human-in-the-loop review](#) to allow users to approve, edit, and reject actions (such as outbound emails) as desired.

python

참고

If you are migrating from the [langgraph-supervisor](#) package, see [Migrate from langgraph-supervisor](#) for before-and-after patterns, including interrupt and resume flows.

Why use a supervisor?

Multi-agent architectures allow you to partition [tools](#) across workers, each with their own individual prompts or instructions. Consider an agent with direct access to all calendar and email APIs: it must choose from many similar tools, understand exact formats for each API, and handle multiple domains simultaneously. If performance degrades, it may be helpful to separate related tools and associated prompts into logical groups (in part to manage iterative improvements).

Concepts

We will cover the following concepts:

- [Multi-agent systems](#)

- [Human-in-the-loop review](#)

Setup

Installation

This tutorial requires the `langchain` package:

python

```
```bash pip pip install langchain
```

```
```bash conda
conda install langchain -c conda-forge
```

js

```
```bash npm npm install langchain
```

```
```bash yarn
yarn add langchain
```

```
```bash pnpm pnpm add langchain
```

</div>

For more details, see our [Installation guide](/oss/langchain/install).

### LangSmith

Set up [LangSmith](https://smith.langchain.com) to inspect what is happening inside your agent. Then set the following environment variables:

<div class="callout python" markdown="1">

<p class="callout-label">python</p>

```
```bash Shell
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."
```

```
```python Python import getpass import os
```

```
os.environ["LANGSMITH_TRACING"] = "true" os.environ["LANGSMITH_API_KEY"] =
getpass.getpass()
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
```bash Shell
```

```
export LANGSMITH_TRACING="true"
```

```
export LANGSMITH_API_KEY="..."
```

```
```typescript TypeScript process.env.LANGSMITH_TRACING = "true";  
process.env.LANGSMITH_API_KEY = "...";
```

```
</div>
```

```
Components
```

We will need to select a chat model from LangChain's suite of integrations:

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
</div>
```

```
<div class="callout js" markdown="1">
```

```
<p class="callout-label">js</p>
```

```
</div>
```

```
1. Define tools
```

Start by defining the tools that require structured inputs. In real applications, these would call actual APIs (Google Calendar, SendGrid, etc.). For this tutorial, you'll use stubs to demonstrate the pattern.

```
<div class="callout python" markdown="1">
```

```
<p class="callout-label">python</p>
```

```
```python
```

```
from langchain.tools import tool
```

```
@tool
```

```
def create_calendar_event(  
    title: str,
```

```
    title: str,
```

```

    start_time: str,          # ISO format: "2024-01-15T14:00:00"
    end_time: str,            # ISO format: "2024-01-15T15:00:00"
    attendees: list[str],     # email addresses
    location: str = ""
) -> str:
    """Create a calendar event. Requires exact ISO datetime format."""
    # Stub: In practice, this would call Google Calendar API, Outlook API, etc.
    return f"Event created: {title} from {start_time} to {end_time} with {len(attendees)} attendees"

@tool
def send_email(
    to: list[str], # email addresses
    subject: str,
    body: str,
    cc: list[str] = []
) -> str:
    """Send an email via email API. Requires properly formatted addresses."""
    # Stub: In practice, this would call SendGrid, Gmail API, etc.
    return f"Email sent to {', '.join(to)} - Subject: {subject}"

@tool
def get_available_time_slots(
    attendees: list[str],
    date: str, # ISO format: "2024-01-15"
    duration_minutes: int
) -> list[str]:
    """Check calendar availability for given attendees on a specific date."""
    # Stub: In practice, this would query calendar APIs
    return ["09:00", "14:00", "16:00"]

```

js

```

import { tool } from "langchain";
import { z } from "zod";

const createCalendarEvent = tool(
  async ({ title, startTime, endTime, attendees, location }) => {
    // Stub: In practice, this would call Google Calendar API, Outlook API, etc.
    return `Event created: ${title} from ${startTime} to ${endTime} with ${attendees.length} attendees`;
  },
  {
    name: "create_calendar_event",
    description: "Create a calendar event. Requires exact ISO datetime format.",
    schema: z.object({
      title: z.string(),
      startTime: z.string().describe("ISO format: '2024-01-15T14:00:00'"),
      endTime: z.string().describe("ISO format: '2024-01-15T15:00:00'"),

```

```

        attendees: z.array(z.string()).describe("email addresses"),
        location: z.string().optional(),
    })),
    }
);

const sendEmail = tool(
  async ({ to, subject, body, cc }) => {
    // Stub: In practice, this would call SendGrid, Gmail API, etc.
    return `Email sent to ${to.join(', ')} - Subject: ${subject}`;
  },
  {
    name: "send_email",
    description: "Send an email via email API. Requires properly formatted addresses.",
    schema: z.object({
      to: z.array(z.string()).describe("email addresses"),
      subject: z.string(),
      body: z.string(),
      cc: z.array(z.string()).optional(),
    })),
  }
);

const getAvailableTimeSlots = tool(
  async ({ attendees, date, durationMinutes }) => {
    // Stub: In practice, this would query calendar APIs
    return ["09:00", "14:00", "16:00"];
  },
  {
    name: "get_available_time_slots",
    description: "Check calendar availability for given attendees on a specific date.",
    schema: z.object({
      attendees: z.array(z.string()),
      date: z.string().describe("ISO format: '2024-01-15'"),
      durationMinutes: z.number(),
    })),
  }
);

```

2. Create specialized sub-agents

Next, we'll create specialized sub-agents that handle each domain.

Create a calendar agent

The calendar agent understands natural language scheduling requests and translates them into precise API calls. It handles date parsing, availability checking, and event creation.

python

```
from datetime import date

from langchain.agents import create_agent

CALENDAR_AGENT_PROMPT = (
    f"Today's date is {date.today().isoformat()}. "
    "You are a calendar scheduling assistant. "
    "Parse natural language scheduling requests (e.g., 'next Tuesday at 2pm') "
    "into proper ISO datetime formats. "
    "Use get_available_time_slots to check availability when needed. "
    "If there is no suitable time slot, stop and confirm unavailability in your response. "
    "Use create_calendar_event to schedule events. "
    "Always confirm what was scheduled in your final response."
)

calendar_agent = create_agent(
    model,
    tools=[create_calendar_event, get_available_time_slots],
    system_prompt=CALENDAR_AGENT_PROMPT,
)
```

js

```
import { createAgent } from "langchain";

const now = new Date();
const today = [
    now.getFullYear(),
    String(now.getMonth() + 1).padStart(2, "0"),
    String(now.getDate()).padStart(2, "0"),
].join("-");

const CALENDAR_AGENT_PROMPT = `
Today's date is ${today}.
You are a calendar scheduling assistant.
Parse natural language scheduling requests (e.g., 'next Tuesday at 2pm')
into proper ISO datetime formats.
Use get_available_time_slots to check availability when needed.
If there is no suitable time slot, stop and confirm unavailability in your response.
Use create_calendar_event to schedule events.
Always confirm what was scheduled in your final response.
`.trim();

const calendarAgent = createAgent({
    model: llm,
```

```
tools: [createCalendarEvent, getAvailableTimeSlots],
systemPrompt: CALENDAR_AGENT_PROMPT,
});
```

Test the calendar agent to see how it handles natural language scheduling:

python

```
query = "Schedule a team meeting next Tuesday at 2pm for 1 hour"

stream = calendar_agent.stream_events(
    {"messages": [{"role": "user", "content": query}]},
    version="v3",
)
for kind, item in stream.interleave("messages", "tool_calls"):
    if kind == "messages":
        for token in item.text:
            print(token, end="", flush=True)
    elif kind == "tool_calls":
        print(f"\nTool call: {item.tool_name}({item.input})")
        print(f"Tool result: {item.output}")
```

js

```
const query = "Schedule a team meeting next Tuesday at 2pm for 1 hour";

const stream = await calendarAgent.streamEvents(
  { messages: [{ role: "user", content: query } ] },
  { version: "v3" }
);

await Promise.all([
  (async () => {
    for await (const message of stream.messages) {
      for await (const token of message.text) {
        process.stdout.write(token);
      }
    }
  })(),
  (async () => {
    for await (const call of stream.toolCalls) {
      console.log(`\nTool call: ${call.name}(${JSON.stringify(call.input)})`);
      console.log(`Tool result: ${await call.output}`);
    }
  })(),
]);
```

```

===== Ai Message =====
Tool Calls:
  get_available_time_slots (call_EIeoeIi1hE2VmwZSfHStGmXp)
Call ID: call_EIeoeIi1hE2VmwZSfHStGmXp
Args:
  attendees: []
  date: 2024-06-18
  duration_minutes: 60
===== Tool Message =====
Name: get_available_time_slots

["09:00", "14:00", "16:00"]
===== Ai Message =====
Tool Calls:
  create_calendar_event (call_zgx3iJA66Ut0W8S3NpT93kEB)
Call ID: call_zgx3iJA66Ut0W8S3NpT93kEB
Args:
  title: Team Meeting
  start_time: 2024-06-18T14:00:00
  end_time: 2024-06-18T15:00:00
  attendees: []
===== Tool Message =====
Name: create_calendar_event

Event created: Team Meeting from 2024-06-18T14:00:00 to 2024-06-18T15:00:00 with 0 attendees
===== Ai Message =====

The team meeting has been scheduled for next Tuesday, June 18th, at 2:00 PM and will last for 1 hour. If you need to add attendees or a location, please let me know!

```

The agent parses "next Tuesday at 2pm" into ISO format ("2024-01-16T14:00:00"), calculates the end time, calls `create_calendar_event`, and returns a natural language confirmation.

Create an email agent

The email agent handles message composition and sending. It focuses on extracting recipient information, crafting appropriate subject lines and body text, and managing email communication.

python

```

EMAIL_AGENT_PROMPT = (
    "You are an email assistant. "
    "Compose professional emails based on natural language requests. "
    "Extract recipient information and craft appropriate subject lines and body text. "
    "Use send_email to send the message. "
    "Always confirm what was sent in your final response."
)

```

```
email_agent = create_agent(
    model,
    tools=[send_email],
    system_prompt=EMAIL_AGENT_PROMPT,
)
```

Test the email agent with a natural language request:

```
query = "Send the design team a reminder about reviewing the new mockups"

stream = email_agent.stream_events(
    {"messages": [{"role": "user", "content": query}]},
    version="v3",
)
for kind, item in stream.interleave("messages", "tool_calls"):
    if kind == "messages":
        for token in item.text:
            print(token, end="", flush=True)
    elif kind == "tool_calls":
        print(f"\nTool call: {item.tool_name}({item.input})")
        print(f"Tool result: {item.output}")
```

js

```
const EMAIL_AGENT_PROMPT = `
You are an email assistant.
Compose professional emails based on natural language requests.
Extract recipient information and craft appropriate subject lines and body text.
Use send_email to send the message.
Always confirm what was sent in your final response.
`.trim();

const emailAgent = createAgent({
    model: llm,
    tools: [sendEmail],
    systemPrompt: EMAIL_AGENT_PROMPT,
});
```

Test the email agent with a natural language request:

```
const query = "Send the design team a reminder about reviewing the new mockups";

const stream = await emailAgent.streamEvents(
    { messages: [{ role: "user", content: query } ] },
    { version: "v3" }
);

await Promise.all([
```



```

(async () => {
  for await (const message of stream.messages) {
    for await (const token of message.text) {
      process.stdout.write(token);
    }
  }
})();
(async () => {
  for await (const call of stream.toolCalls) {
    console.log(`\nTool call: ${call.name}(${JSON.stringify(call.input)})`);
    console.log(`Tool result: ${await call.output}`);
  }
})();
});

```

===== Ai Message =====

Tool Calls:

send_email (call_OM151FziTVY6CRZvzYfjY0Zr)

Call ID: call_OM151FziTVY6CRZvzYfjY0Zr

Args:

to: ['design-team@example.com']

subject: Reminder: Please Review the New Mockups

body: Hi Design Team,

This is a friendly reminder to review the new mockups at your earliest convenience. Your feedback is important to ensure that we stay on track with our project timeline.

Please let me know if you have any questions or need additional information.

Thank you!

Best regards,

===== Tool Message =====

Name: send_email

Email sent to design-team@example.com - Subject: Reminder: Please Review the New Mockups

===== Ai Message =====

I've sent a reminder to the design team asking them to review the new mockups. If you need any further communication on this topic, just let me know!

The agent infers the recipient from the informal request, crafts a professional subject line and body, calls `send_email`, and returns a confirmation. Each sub-agent has a narrow focus with domain-specific tools and prompts, allowing it to excel at its specific task.

3. Wrap sub-agents as tools

Now wrap each sub-agent as a tool that the supervisor can invoke. This is the key architectural step that creates the layered system. The supervisor will see high-level tools like "schedule_event", not low-level tools like "create_calendar_event".

python

```
@tool
def schedule_event(request: str) -> str:
    """Schedule calendar events using natural language.

    Use this when the user wants to create, modify, or check calendar appointments.
    Handles date/time parsing, availability checking, and event creation.

    Input: Natural language scheduling request (e.g., 'meeting with design team
    next Tuesday at 2pm')
    """
    result = calendar_agent.invoke({
        "messages": [{"role": "user", "content": request}]
    })
    return result["messages"][-1].text


@tool
def manage_email(request: str) -> str:
    """Send emails using natural language.

    Use this when the user wants to send notifications, reminders, or any email
    communication. Handles recipient extraction, subject generation, and email
    composition.

    Input: Natural language email request (e.g., 'send them a reminder about
    the meeting')
    """
    result = email_agent.invoke({
        "messages": [{"role": "user", "content": request}]
    })
    return result["messages"][-1].text
```

js

```
const scheduleEvent = tool(
  async ({ request }) => {
    const result = await calendarAgent.invoke({
      messages: [{ role: "user", content: request }]
    });
    const lastMessage = result.messages[result.messages.length - 1];
    return lastMessage.text;
  }
);
```

```

    },
    {
      name: "schedule_event",
      description: `
Schedule calendar events using natural language.

Use this when the user wants to create, modify, or check calendar appointments.
Handles date/time parsing, availability checking, and event creation.

Input: Natural language scheduling request (e.g., 'meeting with design team next Tuesday at 2pm')
      `.trim(),
      schema: z.object({
        request: z.string().describe("Natural language scheduling request"),
      }),
    }
  );

const manageEmail = tool(
  async ({ request }) => {
    const result = await emailAgent.invoke({
      messages: [{ role: "user", content: request }]
    });
    const lastMessage = result.messages[result.messages.length - 1];
    return lastMessage.text;
  },
  {
    name: "manage_email",
    description: `
Send emails using natural language.

Use this when the user wants to send notifications, reminders, or any email communication.
Handles recipient extraction, subject generation, and email composition.

Input: Natural language email request (e.g., 'send them a reminder about the meeting')
      `.trim(),
    schema: z.object({
      request: z.string().describe("Natural language email request"),
    }),
  }
);

```

The tool descriptions help the supervisor decide when to use each tool, so make them clear and specific. We return only the sub-agent's final response, as the supervisor doesn't need to see intermediate reasoning or tool calls.

4. Create the supervisor agent

Now create the supervisor that orchestrates the sub-agents. The supervisor only sees high-level tools and makes routing decisions at the domain level, not the individual API level.

python

```
SUPERVISOR_PROMPT = (
    "You are a helpful personal assistant. "
    "You can schedule calendar events and send emails. "
    "Break down user requests into appropriate tool calls and coordinate the results. "
    "When a request involves multiple actions, use multiple tools in sequence or in parallel as appropriate."
)

supervisor_agent = create_agent(
    model,
    tools=[schedule_event, manage_email],
    system_prompt=SUPERVISOR_PROMPT,
)
```

js

```
const SUPERVISOR_PROMPT = `
You are a helpful personal assistant.
You can schedule calendar events and send emails.
Break down user requests into appropriate tool calls and coordinate the results.
When a request involves multiple actions, use multiple tools in sequence or in parallel
as appropriate.
`.trim();

const supervisorAgent = createAgent({
  model: llm,
  tools: [scheduleEvent, manageEmail],
  systemPrompt: SUPERVISOR_PROMPT,
});
```

5. Use the supervisor

Now test your complete system with complex requests that require coordination across multiple domains:

Example 1: Simple single-domain request

python

```

query = "Schedule a team standup for tomorrow at 9am"

stream = supervisor_agent.stream_events(
    {"messages": [{"role": "user", "content": query}]},
    version="v3",
)
for kind, item in stream.interleave("messages", "tool_calls"):
    if kind == "messages":
        for token in item.text:
            print(token, end="", flush=True)
    elif kind == "tool_calls":
        print(f"\nTool call: {item.tool_name}({item.input})")
        print(f"Tool result: {item.output}")

```

js

```

const query = "Schedule a team standup for tomorrow at 9am";

const stream = await supervisorAgent.streamEvents(
  { messages: [{ role: "user", content: query } ] },
  { version: "v3" }
);

await Promise.all([
  (async () => {
    for await (const message of stream.messages) {
      for await (const token of message.text) {
        process.stdout.write(token);
      }
    }
  })(),
  (async () => {
    for await (const call of stream.toolCalls) {
      console.log(`\nTool call: ${call.name}(${JSON.stringify(call.input)})`);
      console.log(`Tool result: ${await call.output}`);
    }
  })(),
]);

```

```

===== Ai Message =====
Tool Calls:
  schedule_event (call_mXFJJDU8bKZadNUZPaag8Lct)
Call ID: call_mXFJJDU8bKZadNUZPaag8Lct
Args:
  request: Schedule a team standup for tomorrow at 9am with Alice and Bob.
===== Tool Message =====
Name: schedule_event

```

The team standup has been scheduled for tomorrow at 9:00 AM with Alice and Bob. If you need to make any changes or add more details, just let me know!

===== Ai Message =====

The team standup with Alice and Bob is scheduled for tomorrow at 9:00 AM. If you need any further arrangements or adjustments, please let me know!

The supervisor identifies this as a calendar task, calls `schedule_event`, and the calendar agent handles date parsing and event creation.

팁

For full transparency into the information flow, including prompts and responses for each chat model call, check out the [LangSmith trace](#) for the above run.

Example 2: Complex multi-domain request

python

```
query = (
    "Schedule a meeting with the design team next Tuesday at 2pm for 1 hour, "
    "and send them an email reminder about reviewing the new mockups."
)

stream = supervisor_agent.stream_events(
    {"messages": [{"role": "user", "content": query}]},
    version="v3",
)

for kind, item in stream.interleave("messages", "tool_calls"):
    if kind == "messages":
        for token in item.text:
            print(token, end="", flush=True)
    elif kind == "tool_calls":
        print(f"\nTool call: {item.tool_name}({item.input})")
        print(f"Tool result: {item.output}")
```

js

```
const query =
    "Schedule a meeting with the design team next Tuesday at 2pm for 1 hour, " +
    "and send them an email reminder about reviewing the new mockups.";

const stream = await supervisorAgent.streamEvents(
    { messages: [{ role: "user", content: query }] },
    { version: "v3" }
```

```

);

await Promise.all([
  (async () => {
    for await (const message of stream.messages) {
      for await (const token of message.text) {
        process.stdout.write(token);
      }
    }
  })(),
  (async () => {
    for await (const call of stream.toolCalls) {
      console.log(`\nTool call: ${call.name}(${JSON.stringify(call.input)})`);
      console.log(`Tool result: ${await call.output}`);
    }
  })(),
]);

```

===== Ai Message =====

Tool Calls:

schedule_event (call_YA68mqF0koZIItCFPx0kGQfZi)

Call ID: call_YA68mqF0koZIItCFPx0kGQfZi

Args:

request: meeting with the design team next Tuesday at 2pm for 1 hour

manage_email (call_XxqcJBvVIuKuRK794ZlzlLxx)

Call ID: call_XxqcJBvVIuKuRK794ZlzlLxx

Args:

request: send the design team an email reminder about reviewing the new mockups

===== Tool Message =====

Name: schedule_event

Your meeting with the design team is scheduled for next Tuesday, June 18th, from 2:00pm to 3:00pm. Let me know if you need to add more details or make any changes!

===== Tool Message =====

Name: manage_email

I've sent an email reminder to the design team requesting them to review the new mockups. If you need to include more information or recipients, just let me know!

===== Ai Message =====

Your meeting with the design team is scheduled for next Tuesday, June 18th, from 2:00pm to 3:00pm.

I've also sent an email reminder to the design team, asking them to review the new mockups.

Let me know if you'd like to add more details to the meeting or include additional information in the email!

The supervisor recognizes this requires both calendar and email actions, calls `schedule_event` for the meeting, then calls `manage_email` for the reminder. Each sub-agent completes its task, and the supervisor synthesizes both results into a coherent response.

참고

The supervisor dispatches tasks to subagents sequentially by default. Each tool call completes before the next one starts. However, many LLMs will issue multiple tool calls in a single response (as shown in the trace above, where both `schedule_event` and `manage_email` are called together), which the runtime executes in parallel. You can also configure explicit parallel dispatch. See the [create_supervisor reference docs](#) for details.

팁

Refer to the [LangSmith trace](#) to see the detailed information flow for the above run, including individual chat model prompts and responses.

Complete working example

Here's everything together in a runnable script:

View complete code

python

```
"""
Personal Assistant Supervisor Example

This example demonstrates the tool calling pattern for multi-agent systems.
A supervisor agent coordinates specialized sub-agents (calendar and email)
that are wrapped as tools.
"""

from datetime import date

from langchain.tools import tool
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model

# =====
# Step 1: Define low-level API tools (stubbed)
# =====

@tool
def create_calendar_event(
    title: str,
```



```

        start_time: str, # ISO format: "2024-01-15T14:00:00"
        end_time: str,   # ISO format: "2024-01-15T15:00:00"
        attendees: list[str], # email addresses
        location: str = ""
    ) -> str:
        """Create a calendar event. Requires exact ISO datetime format."""
        return f"Event created: {title} from {start_time} to {end_time} with {len(attendees)} attendees"

@tool
def send_email(
    to: list[str],      # email addresses
    subject: str,
    body: str,
    cc: list[str] = []
) -> str:
    """Send an email via email API. Requires properly formatted addresses."""
    return f"Email sent to {' '.join(to)} - Subject: {subject}"

@tool
def get_available_time_slots(
    attendees: list[str],
    date: str, # ISO format: "2024-01-15"
    duration_minutes: int
) -> list[str]:
    """Check calendar availability for given attendees on a specific date."""
    return ["09:00", "14:00", "16:00"]

# =====
# Step 2: Create specialized sub-agents
# =====

model = init_chat_model("gpt-5.5") # for example

calendar_agent = create_agent(
    model,
    tools=[create_calendar_event, get_available_time_slots],
    system_prompt=(
        f"Today's date is {date.today().isoformat()}. "
        "You are a calendar scheduling assistant. "
        "Parse natural language scheduling requests (e.g., 'next Tuesday at 2pm') "
        "into proper ISO datetime formats. "
        "Use get_available_time_slots to check availability when needed. "
        "If there is no suitable time slot, stop and confirm unavailability in your response. "
        "Use create_calendar_event to schedule events. "
        "Always confirm what was scheduled in your final response."
    )
)

email_agent = create_agent(

```

```

model,
tools=[send_email],
system_prompt=(
    "You are an email assistant. "
    "Compose professional emails based on natural language requests. "
    "Extract recipient information and craft appropriate subject lines and body text. "
    "Use send_email to send the message. "
    "Always confirm what was sent in your final response."
)
)

# =====
# Step 3: Wrap sub-agents as tools for the supervisor
# =====

@tool
def schedule_event(request: str) -> str:
    """Schedule calendar events using natural language.

    Use this when the user wants to create, modify, or check calendar appointments.
    Handles date/time parsing, availability checking, and event creation.

    Input: Natural language scheduling request (e.g., 'meeting with design team
    next Tuesday at 2pm')
    """
    result = calendar_agent.invoke({
        "messages": [{"role": "user", "content": request}]
    })
    return result["messages"][-1].text

@tool
def manage_email(request: str) -> str:
    """Send emails using natural language.

    Use this when the user wants to send notifications, reminders, or any email
    communication. Handles recipient extraction, subject generation, and email
    composition.

    Input: Natural language email request (e.g., 'send them a reminder about
    the meeting')
    """
    result = email_agent.invoke({
        "messages": [{"role": "user", "content": request}]
    })
    return result["messages"][-1].text

# =====
# Step 4: Create the supervisor agent
# =====

```

```

supervisor_agent = create_agent(
    model,
    tools=[schedule_event, manage_email],
    system_prompt=(
        "You are a helpful personal assistant. "
        "You can schedule calendar events and send emails. "
        "Break down user requests into appropriate tool calls and coordinate the results. "
        "When a request involves multiple actions, use multiple tools in sequence or in parallel as appropriate."
    )
)

# =====
# Step 5: Use the supervisor
# =====

if __name__ == "__main__":
    # Example: User request requiring both calendar and email coordination
    user_request = (
        "Schedule a meeting with the design team next Tuesday at 2pm for 1 hour, "
        "and send them an email reminder about reviewing the new mockups."
    )

    print("User Request:", user_request)
    print("\n" + "="*80 + "\n")

    stream = supervisor_agent.stream_events(
        {"messages": [{"role": "user", "content": user_request}]},
        version="v3",
    )
    for kind, item in stream.interleave("messages", "tool_calls"):
        if kind == "messages":
            for token in item.text:
                print(token, end="", flush=True)
        elif kind == "tool_calls":
            print(f"\nTool call: {item.tool_name}({item.input})")
            print(f"Tool result: {item.output}")

```

js

```

/**
 * Personal Assistant Supervisor Example
 *
 * This example demonstrates the tool calling pattern for multi-agent systems.
 * A supervisor agent coordinates specialized sub-agents (calendar and email)
 * that are wrapped as tools.
 */

```

```

import { tool, createAgent } from "langchain";
import { ChatAnthropic } from "@langchain/anthropic";
import { z } from "zod";

// =====
// Step 1: Define low-level API tools (stubbed)
// =====

const createCalendarEvent = tool(
  async ({ title, startTime, endTime, attendees, location }) => {
    // Stub: In practice, this would call Google Calendar API, Outlook API, etc.
    return `Event created: ${title} from ${startTime} to ${endTime} with ${attendees.length} attendees`;
  },
  {
    name: "create_calendar_event",
    description: "Create a calendar event. Requires exact ISO datetime format.",
    schema: z.object({
      title: z.string(),
      startTime: z.string().describe("ISO format: '2024-01-15T14:00:00'"),
      endTime: z.string().describe("ISO format: '2024-01-15T15:00:00'"),
      attendees: z.array(z.string()).describe("email addresses"),
      location: z.string().optional().default(""),
    }),
  }
);

const sendEmail = tool(
  async ({ to, subject, body, cc }) => {
    // Stub: In practice, this would call SendGrid, Gmail API, etc.
    return `Email sent to ${to.join(", ")} - Subject: ${subject}`;
  },
  {
    name: "send_email",
    description: "Send an email via email API. Requires properly formatted addresses.",
    schema: z.object({
      to: z.array(z.string()).describe("email addresses"),
      subject: z.string(),
      body: z.string(),
      cc: z.array(z.string()).optional().default([]),
    }),
  }
);

const getAvailableTimeSlots = tool(
  async ({ attendees, date, durationMinutes }) => {
    // Stub: In practice, this would query calendar APIs
    return ["09:00", "14:00", "16:00"];
  },
  {

```

```

    name: "get_available_time_slots",
    description:
        "Check calendar availability for given attendees on a specific date.",
    schema: z.object({
        attendees: z.array(z.string()),
        date: z.string().describe("ISO format: '2024-01-15'"),
        durationMinutes: z.number(),
    }),
}
);

// =====
// Step 2: Create specialized sub-agents
// =====

const llm = new ChatAnthropic({
    model: "gpt-5.5",
});

const now = new Date();
const today = [
    now.getFullYear(),
    String(now.getMonth() + 1).padStart(2, "0"),
    String(now.getDate()).padStart(2, "0"),
].join("-");

const calendarAgent = createAgent({
    model: llm,
    tools: [createCalendarEvent, getAvailableTimeSlots],
    systemPrompt: `
Today's date is ${today}.
You are a calendar scheduling assistant.
Parse natural language scheduling requests (e.g., 'next Tuesday at 2pm')
into proper ISO datetime formats.
Use get_available_time_slots to check availability when needed.
If there is no suitable time slot, stop and confirm unavailability in your response.
Use create_calendar_event to schedule events.
Always confirm what was scheduled in your final response.
`.trim(),
});

const emailAgent = createAgent({
    model: llm,
    tools: [sendEmail],
    systemPrompt: `
You are an email assistant.
Compose professional emails based on natural language requests.
Extract recipient information and craft appropriate subject lines and body text.
Use send_email to send the message.
Always confirm what was sent in your final response.
`.trim(),
});

```

```

});

// =====
// Step 3: Wrap sub-agents as tools for the supervisor
// =====

const scheduleEvent = tool(
  async ({ request }) => {
    const result = await calendarAgent.invoke({
      messages: [{ role: "user", content: request }],
    });
    const lastMessage = result.messages[result.messages.length - 1];
    return lastMessage.text;
  },
  {
    name: "schedule_event",
    description: `
Schedule calendar events using natural language.

Use this when the user wants to create, modify, or check calendar appointments.
Handles date/time parsing, availability checking, and event creation.

Input: Natural language scheduling request (e.g., 'meeting with design team next Tuesday at 2pm')
`.trim(),
    schema: z.object({
      request: z.string().describe("Natural language scheduling request"),
    }),
  }
);

const manageEmail = tool(
  async ({ request }) => {
    const result = await emailAgent.invoke({
      messages: [{ role: "user", content: request }],
    });
    const lastMessage = result.messages[result.messages.length - 1];
    return lastMessage.text;
  },
  {
    name: "manage_email",
    description: `
Send emails using natural language.

Use this when the user wants to send notifications, reminders, or any email communication.
Handles recipient extraction, subject generation, and email composition.

Input: Natural language email request (e.g., 'send them a reminder about the meeting')
`.trim(),
    schema: z.object({

```

```

        request: z.string().describe("Natural language email request"),
    })),
    }
});

// =====
// Step 4: Create the supervisor agent
// =====

const supervisorAgent = createAgent({
    model: llm,
    tools: [scheduleEvent, manageEmail],
    systemPrompt: `
You are a helpful personal assistant.
You can schedule calendar events and send emails.
Break down user requests into appropriate tool calls and coordinate the results.
When a request involves multiple actions, use multiple tools in sequence.
`.trim(),
});

// =====
// Step 5: Use the supervisor
// =====

// Example: User request requiring both calendar and email coordination
const userRequest =
    "Schedule a meeting with the design team next Tuesday at 2pm for 1 hour, " +
    "and send them an email reminder about reviewing the new mockups.";

console.log("User Request:", userRequest);
console.log(`\n${"=".repeat(80)}\n`);

const stream = await supervisorAgent.streamEvents(
    { messages: [{ role: "user", content: userRequest }] },
    { version: "v3" }
);

await Promise.all([
    (async () => {
        for await (const message of stream.messages) {
            for await (const token of message.text) {
                process.stdout.write(token);
            }
        }
    })(),
    (async () => {
        for await (const call of stream.toolCalls) {
            console.log(`\nTool call: ${call.name}${JSON.stringify(call.input)}\n`);
            console.log(`Tool result: ${await call.output}\n`);
        }
    })()
]);

```

```
})((),  
]);
```

Understanding the architecture

Your system has three layers. The bottom layer contains rigid API tools that require exact formats. The middle layer contains sub-agents that accept natural language, translate it to structured API calls, and return natural language confirmations. The top layer contains the supervisor that routes to high-level capabilities and synthesizes results.

This separation of concerns provides several benefits: each layer has a focused responsibility, you can add new domains without affecting existing ones, and you can test and iterate on each layer independently.

6. Add human-in-the-loop review

It can be prudent to incorporate [human-in-the-loop review](#) of sensitive actions. LangChain includes [built-in middleware](#) to review tool calls, in this case the tools invoked by sub-agents.

Let's add human-in-the-loop review to both sub-agents: - We configure the `create_calendar_event` and `send_email` tools to interrupt, permitting all [response types](#) (`approve`, `edit`, `reject`) - We add a [checkpoint](#) **only to the top-level agent**. This is required to pause and resume execution.

python

```
from langchain.agents import create_agent  
from langchain.agents.middleware import HumanInTheLoopMiddleware # [!code highlight]  
from langgraph.checkpoint.memory import InMemorySaver # [!code highlight]  
  
calendar_agent = create_agent(  
    model,  
    tools=[create_calendar_event, get_available_time_slots],  
    system_prompt=CALENDAR_AGENT_PROMPT,  
    middleware=[ # [!code highlight]  
        HumanInTheLoopMiddleware( # [!code highlight]  
            interrupt_on={"create_calendar_event": True}, # [!code highlight]  
            description_prefix="Calendar event pending approval", # [!code highlight]  
        ), # [!code highlight]  
    ], # [!code highlight]  
)  
  
email_agent = create_agent(  
    model,  
    tools=[send_email],  
    system_prompt=EMAIL_AGENT_PROMPT,
```



```

        middleware=[ # [!code highlight]
            HumanInTheLoopMiddleware( # [!code highlight]
                interrupt_on={"send_email": True}, # [!code highlight]
                description_prefix="Outbound email pending approval", # [!code highlight]
            ), # [!code highlight]
        ], # [!code highlight]
    )

supervisor_agent = create_agent(
    model,
    tools=[schedule_event, manage_email],
    system_prompt=SUPERVISOR_PROMPT,
    checkpoint=InMemorySaver(), # [!code highlight]
)

```

js

```

import { createAgent, humanInTheLoopMiddleware } from "langchain"; // [!code highlight]
import { MemorySaver } from "@langchain/langgraph"; // [!code highlight]

const calendarAgent = createAgent({
    model: llm,
    tools: [createCalendarEvent, getAvailableTimeSlots],
    systemPrompt: CALENDAR_AGENT_PROMPT,
    middleware: [ // [!code highlight]
        humanInTheLoopMiddleware({ // [!code highlight]
            interruptOn: { create_calendar_event: true }, // [!code highlight]
            descriptionPrefix: "Calendar event pending approval", // [!code highlight]
        }), // [!code highlight]
    ], // [!code highlight]
});

const emailAgent = createAgent({
    model: llm,
    tools: [sendEmail],
    systemPrompt: EMAIL_AGENT_PROMPT,
    middleware: [ // [!code highlight]
        humanInTheLoopMiddleware({ // [!code highlight]
            interruptOn: { send_email: true }, // [!code highlight]
            descriptionPrefix: "Outbound email pending approval", // [!code highlight]
        }), // [!code highlight]
    ], // [!code highlight]
});

const supervisorAgent = createAgent({
    model: llm,
    tools: [scheduleEvent, manageEmail],
    systemPrompt: SUPERVISOR_PROMPT,

```

```
checkpointer: new MemorySaver(), // [!code highlight]
});
```

Let's repeat the query. Note that we gather interrupt events into a list to access downstream:

python

```
query = (
    "Schedule a meeting with the design team next Tuesday at 2pm for 1 hour, "
    "and send them an email reminder about reviewing the new mockups."
)

config = {"configurable": {"thread_id": "6"}}

interrupts = []
stream = supervisor_agent.stream_events(
    {"messages": [{"role": "user", "content": query}]},
    config,
    version="v3",
)
for kind, item in stream.interleave("messages", "tool_calls"):
    if kind == "messages":
        for token in item.text:
            print(token, end="", flush=True)
    elif kind == "tool_calls":
        print(f"\nTool call: {item.tool_name}({item.input})")
if stream.interrupted:
    for interrupt_ in stream.interrupts:
        interrupts.append(interrupt_)
    print(f"\nINTERRUPTED: {interrupt_.id}")
```

js

```
const query =
    "Schedule a meeting with the design team next Tuesday at 2pm for 1 hour, " +
    "and send them an email reminder about reviewing the new mockups.";

const config = { configurable: { thread_id: "6" } };

const interrupts: any[] = [];
const stream = await supervisorAgent.streamEvents(
    { messages: [{ role: "user", content: query } ] },
    { ...config, version: "v3" }
);

await Promise.all([
    (async () => {
```

```

    for await (const message of stream.messages) {
      for await (const token of message.text) {
        process.stdout.write(token);
      }
    }
  })(),
  (async () => {
    for await (const call of stream.toolCalls) {
      console.log(`\nTool call: ${call.name}${JSON.stringify(call.input)}`);
    }
  })(),
]);
if (stream.interrupted) {
  for (const interrupt of stream.interrupts) {
    interrupts.push(interrupt);
    console.log(`\nINTERRUPTED: ${interrupt.interruptId}`);
  }
}
}

```

===== Ai Message =====

Tool Calls:

schedule_event (call_t4Wyn32ohaShpEZKuzZb183z)

Call ID: call_t4Wyn32ohaShpEZKuzZb183z

Args:

request: Schedule a meeting with the design team next Tuesday at 2pm for 1 hour.

manage_email (call_JWj4vDJ5VMnvkySymhCBm4IR)

Call ID: call_JWj4vDJ5VMnvkySymhCBm4IR

Args:

request: Send an email reminder to the design team about reviewing the new mockups before our meeting next Tuesday at 2pm.

INTERRUPTED: 4f994c9721682a292af303ec1a46abb7

INTERRUPTED: 2b56f299be313ad8bc689eff02973f16

This time we've interrupted execution. Let's inspect the interrupt events:

python

```

for interrupt_ in interrupts:
    for request in interrupt_.value["action_requests"]:
        print(f"INTERRUPTED: {interrupt_.id}")
        print(f"{request['description']}\n")

```

js

```

for (const interrupt of interrupts) {
  for (const request of interrupt.payload.actionRequests) {
    console.log(`INTERRUPTED: ${interrupt.interruptId}`);
    console.log(`${request.description}\n`);
  }
}

```

INTERRUPTED: 4f994c9721682a292af303ec1a46abb7
 Calendar event pending approval

Tool: create_calendar_event

Args: {'title': 'Meeting with the Design Team', 'start_time': '2024-06-18T14:00:00', 'end_time': '2024-06-18T15:00:00', 'attendees': ['design team']}

INTERRUPTED: 2b56f299be313ad8bc689eff02973f16
 Outbound email pending approval

Tool: send_email

Args: {'to': ['designteam@example.com'], 'subject': 'Reminder: Review New Mockups Before Meeting Next Tuesday at 2pm', 'body': "Hello Team,\n\nThis is a reminder to review the new mockups ahead of our meeting scheduled for next Tuesday at 2pm. Your feedback and insights will be valuable for our discussion and next steps.\n\nPlease ensure you've gone through the designs and are ready to share your thoughts during the meeting.\n\nThank you!\n\nBest regards,\n[Your Name]"}

We can specify decisions for each interrupt by referring to its ID using a `@[Command]`. Refer to the [human-in-the-loop guide](#) for additional details. For demonstration purposes, here we will accept the calendar event, but edit the subject of the outbound email:

python

```

from langgraph.types import Command # [!code highlight]

resume = {}
for interrupt_ in interrupts:
    if interrupt_.id == "2b56f299be313ad8bc689eff02973f16":
        # Edit email
        edited_action = interrupt_.value["action_requests"][0].copy()
        edited_action["args"]["subject"] = "Mockups reminder"
        resume[interrupt_.id] = {
            "decisions": [{"type": "edit", "edited_action": edited_action}]
        }
    else:
        resume[interrupt_.id] = {"decisions": [{"type": "approve"}]}

interrupts = []
stream = supervisor_agent.stream_events(
    Command(resume=resume), # [!code highlight]

```

```

    config,
    version="v3",
)
for kind, item in stream.interleave("messages", "tool_calls"):
    if kind == "messages":
        for token in item.text:
            print(token, end="", flush=True)
    elif kind == "tool_calls":
        print(f"\nTool call: {item.tool_name}({item.input})")
if stream.interrupted:
    for interrupt_ in stream.interrupts:
        interrupts.append(interrupt_)
        print(f"\nINTERRUPTED: {interrupt_.id}")

```

js

```

import { Command } from "@langchain/langgraph"; // [!code highlight]

const resume: Record<string, any> = {};
for (const interrupt of interrupts) {
    const actionRequest = interrupt.payload.actionRequests[0];
    if (actionRequest.name === "send_email") {
        // Edit email
        const editedAction = { ...actionRequest };
        editedAction.args.subject = "Mockups reminder";
        resume[interrupt.interruptId] = {
            decisions: [{ type: "edit", editedAction }]
        };
    } else {
        resume[interrupt.interruptId] = { decisions: [{ type: "approve" }] };
    }
}

const resumeStream = await supervisorAgent.streamEvents(
    new Command({ resume }), // [!code highlight]
    { ...config, version: "v3" }
);

await Promise.all([
    (async () => {
        for await (const message of resumeStream.messages) {
            for await (const token of message.text) {
                process.stdout.write(token);
            }
        }
    })(),
    (async () => {
        for await (const call of resumeStream.toolCalls) {
            console.log(`\nTool call: ${call.name}(${JSON.stringify(call.input)})`);
        }
    })()
]);

```

```
}
})(),
]);
```

===== Tool Message =====

Name: schedule_event

Your meeting with the design team has been scheduled for next Tuesday, June 18th, from 2:00 pm to 3:00 pm.

===== Tool Message =====

Name: manage_email

Your email reminder to the design team has been sent. Here's what was sent:

- Recipient: designteam@example.com
- Subject: Mockups reminder
- Body: A reminder to review the new mockups before the meeting next Tuesday at 2pm, with a request for feedback and readiness for discussion.

Let me know if you need any further assistance!

===== Ai Message =====

- Your meeting with the design team has been scheduled for next Tuesday, June 18th, from 2:00 pm to 3:00 pm.
- An email reminder has been sent to the design team about reviewing the new mockups before the meeting.

Let me know if you need any further assistance!

The run proceeds with our input.

7. Advanced: Control information flow

By default, sub-agents receive only the request string from the supervisor. You might want to pass additional context, such as conversation history or user preferences.

Pass additional conversational context to sub-agents

python

```
from langchain.tools import tool, ToolRuntime

@tool
def schedule_event(
    request: str,
    runtime: ToolRuntime
) -> str:
```

```

""Schedule calendar events using natural language.""
# Customize context received by sub-agent
original_user_message = next(
    message for message in runtime.state["messages"]
    if message.type == "human"
)
prompt = (
    "You are assisting with the following user inquiry:\n\n"
    f"{original_user_message.text}\n\n"
    "You are tasked with the following sub-request:\n\n"
    f"{request}"
)
result = calendar_agent.invoke({
    "messages": [{"role": "user", "content": prompt}],
})
return result["messages"][-1].text

```

js

```

import { getCurrentTaskInput } from "@langchain/langgraph";
import { type BuiltInState, HumanMessage } from "langchain";

const scheduleEvent = tool(
  async ({ request }, config) => {
    // Customize context received by sub-agent
    // Access full thread messages from the config
    const currentMessages = getCurrentTaskInput<BuiltInState>(config).messages;
    const originalUserMessage = currentMessages.find(HumanMessage.isInstance);
    const prompt = `
You are assisting with the following user inquiry:

${originalUserMessage?.content || "No context available"}

You are tasked with the following sub-request:

${request}
`.trim();

    const result = await calendarAgent.invoke({
      messages: [{ role: "user", content: prompt }],
    });
    const lastMessage = result.messages[result.messages.length - 1];
    return lastMessage.text;
  },
  {
    name: "schedule_event",
    description: "Schedule calendar events using natural language.",
    schema: z.object({
      request: z.string().describe("Natural language scheduling request"),
    })
  }
)

```

```
    }),  
  }  
);
```

This allows sub-agents to see the full conversation context, which can be useful for resolving ambiguities like "schedule it for the same time tomorrow" (referencing a previous conversation).

팁

You can see the full context received by the sub agent in the [chat model call](#) of the LangSmith trace.

Control what supervisor receives

You can also customize what information flows back to the supervisor:

python

```
import json  
  
@tool  
def schedule_event(request: str) -> str:  
    """Schedule calendar events using natural language."""  
    result = calendar_agent.invoke({  
        "messages": [{"role": "user", "content": request}]  
    })  
  
    # Option 1: Return just the confirmation message  
    return result["messages"][-1].text  
  
    # Option 2: Return structured data  
    # return json.dumps({  
    #     "status": "success",  
    #     "event_id": "evt_123",  
    #     "summary": result["messages"][-1].text  
    # })
```

js

```
const scheduleEvent = tool(  
  async ({ request }) => {  
    const result = await calendarAgent.invoke({  
      messages: [{ role: "user", content: request }]  
    });  
  });
```



```

const lastMessage = result.messages[result.messages.length - 1];

// Option 1: Return just the confirmation message
return lastMessage.text;

    // Option 2: Return structured data
    // return JSON.stringify({
    //   status: "success",
    //   event_id: "evt_123",
    //   summary: lastMessage.text
    // });
  },
  {
    name: "schedule_event",
    description: "Schedule calendar events using natural language.",
    schema: z.object({
      request: z.string().describe("Natural language scheduling request"),
    }),
  }
);

```

Important: Make sure sub-agent prompts emphasize that their final message should contain all relevant information. A common failure mode is sub-agents that perform tool calls but don't include the results in their final response.

js

팁

For a complete working example that demonstrates the full supervisor pattern with human-in-the-loop review and advanced information flow control, check out [supervisor_complete.ts](#) in the LangChain.js examples.

8. Key takeaways

The supervisor pattern creates layers of abstraction where each layer has a clear responsibility. When designing a supervisor system, start with clear domain boundaries and give each sub-agent focused tools and prompts. Write clear tool descriptions for the supervisor, test each layer independently before integration, and control information flow based on your specific needs.

팁

When to use the supervisor pattern

Use the supervisor pattern when you have multiple distinct domains (calendar, email, CRM, database), each domain has multiple tools or complex logic, you want centralized workflow control, and sub-agents don't need to converse directly with users.

For simpler cases with just a few tools, use a single agent. When agents need to have conversations with users, use [handoffs](#) instead. For peer-to-peer collaboration between agents, consider other multi-agent patterns.

Next steps

Learn about [handoffs](#) for agent-to-agent conversations, explore [context engineering](#) to fine-tune information flow, read the [multi-agent overview](#) to compare different patterns, and use [LangSmith](#) to debug and monitor your multi-agent system.

Subagents

langchain/multi-agent/subagents.mdx

In the **subagents** architecture, a central main [agent](#) (often referred to as a **supervisor**) coordinates subagents by calling them as [tools](#). The main agent decides which subagent to invoke, what input to provide, and how to combine results. By default, subagents are stateless—they don't remember past interactions, with all conversation memory maintained by the main agent. This provides [context](#) isolation: each subagent invocation works in a clean context window, preventing context bloat in the main conversation.

You can also pass the main agent's conversation history into a subagent when it needs that context. See [Subagent inputs](#).

For built-in subagent support, see [Deep Agents](#).

```
graph LR
  A[User] --> B[Main Agent]
  B --> C[Subagent A]
  B --> D[Subagent B]
  B --> E[Subagent C]
  C --> B
  D --> B
  E --> B
  B --> F[User response]

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,F trigger
class B,C,D,E process
```

Key characteristics

- Centralized control: All routing passes through the main agent
- No direct user interaction: Subagents return results to the main agent, not the user (though you can use [interrupts](#) within a subagent to allow user interaction)
- Subagents via tools: Subagents are invoked via tools
- Parallel execution: The main agent can invoke multiple subagents in a single turn

참고

Supervisor vs. Router: A supervisor agent (this pattern) is different from a [router](#). The supervisor is a full agent that maintains conversation context and dynamically decides

which subagents to call across multiple turns. A router is typically a single classification step that dispatches to agents without maintaining ongoing conversation state.

When to use

Use the subagents pattern when you have multiple distinct domains (e.g., calendar, email, CRM, database), subagents don't need to converse directly with users, or you want centralized workflow control. For simpler cases with just a few [tools](#), use a [single agent](#).

팁

Need user interaction within a subagent? While subagents typically return results to the main agent rather than conversing directly with users, you can use [interrupts](#) within a subagent to pause execution and gather user input. This is useful when a subagent needs clarification or approval before proceeding. The main agent remains the orchestrator, but the subagent can collect information from the user mid-task.

Basic implementation

The core mechanism wraps a subagent as a tool that the main agent can call:

python

```
from langchain.tools import tool
from langchain.agents import create_agent

# Create a subagent
subagent = create_agent(model="google_genai:gemin-3.6-flash", tools=[...])

# Wrap it as a tool
@tool("research", description="Research a topic and return findings")
def call_research_agent(query: str):
    result = subagent.invoke({"messages": [{"role": "user", "content": query}]})
    return result["messages"][-1].content

# Main agent with subagent as a tool
main_agent = create_agent(model="google_genai:gemin-3.6-flash", tools=[call_research_a
gent])
```

js

```
import { createAgent, tool } from "langchain";
import { z } from "zod";
```

```
// Create a subagent
const subagent = createAgent({ model: "google:gemini-3.6-flash", tools: [...] });

// Wrap it as a tool
const callResearchAgent = tool(
  async ({ query }) => {
    const result = await subagent.invoke({
      messages: [{ role: "user", content: query }]
    });
    return result.messages.at(-1)?.content;
  },
  {
    name: "research",
    description: "Research a topic and return findings",
    schema: z.object({ query: z.string() })
  }
);

// Main agent with subagent as a tool
const mainAgent = createAgent({ model: "google:gemini-3.6-flash", tools: [callResearchAgent] });
```

[Tutorial: Build a personal assistant with subagents](#)

Learn how to build a personal assistant using the subagents pattern, where a central main agent (supervisor) coordinates specialized worker agents.

Design decisions

When implementing the subagents pattern, you'll make several key design choices. This table summarizes the options—each is covered in detail in the sections below.

Decision	Options
Sync vs. async	Sync (blocking) vs. async (background)
Tool patterns	Tool per agent vs. single dispatch tool
Subagent specs	System prompt vs. enum constraint vs. tool-based discovery (single dispatch tool only)
Subagent inputs	Query only vs. full context
Subagent outputs	Subagent result vs full conversation history

Sync vs. async

Subagent execution can be **synchronous** (blocking) or **asynchronous** (background). Your choice depends on whether the main agent needs the result to continue.

Mode	Main agent behavior	Best for	Tradeoff
Sync	Waits for subagent to complete	Main agent needs result to continue	Simple, but blocks the conversation
Async	Continues while subagent runs in background	Independent tasks, user shouldn't wait	Responsive, but more complex

팁

Not to be confused with Python's `async / await`. Here, "async" means the main agent kicks off a background job (typically in a separate process or service) and continues without blocking.

Synchronous (default)

By default, subagent calls are **synchronous**: the main agent waits for each subagent to complete before continuing. Use sync when the main agent's next action depends on the subagent's result.

```
sequenceDiagram
    participant User
    participant Main Agent
    participant Research Subagent

    User->>Main Agent: "What's the weather in Tokyo?"
    Main Agent->>Research Subagent: research("Tokyo weather")
    Note over Main Agent: Waiting for result...
    Research Subagent-->>Main Agent: "Currently 72°F, sunny"
    Main Agent-->>User: "It's 72°F and sunny in Tokyo"
```

When to use sync: - Main agent needs the subagent's result to formulate its response - Tasks have order dependencies (e.g., fetch data → analyze → respond) - Subagent failures should block the main agent's response

Tradeoffs: - Simple implementation—just call and wait - User sees no response until all subagents complete - Long-running tasks freeze the conversation

Asynchronous

Use **asynchronous execution** when the subagent's work is independent—the main agent doesn't need the result to continue conversing with the user. The main agent kicks off a background job and remains responsive.

```
sequenceDiagram
    participant User
    participant Main Agent
    participant Job System
    participant Contract Reviewer

    User->>Main Agent: "Review this M&A contract"
    Main Agent->>Job System: run_agent("legal_reviewer", task)
    Job System->>Contract Reviewer: Start agent
    Job System-->>Main Agent: job_id: "job_123"
    Main Agent-->>User: "Started review (job_123)"

    Note over Contract Reviewer: Reviewing 150+ pages...

    User->>Main Agent: "What's the status?"
    Main Agent->>Job System: check_status(job_id)
    Job System-->>Main Agent: "running"
    Main Agent-->>User: "Still reviewing contract..."

    Note over Contract Reviewer: Review completes

    User->>Main Agent: "Is it done yet?"
    Main Agent->>Job System: check_status(job_id)
    Job System-->>Main Agent: "completed"
    Main Agent->>Job System: get_result(job_id)
    Job System-->>Main Agent: Contract analysis
    Main Agent-->>User: "Review complete: [findings]"
```

When to use async: - Subagent work is independent of the main conversation flow - Users should be able to continue chatting while work happens - You want to run multiple independent tasks in parallel

Three-tool pattern: 1. **Start job:** Kicks off the background task, returns a job ID 2. **Check status:** Returns current state (pending, running, completed, failed) 3. **Get result:** Retrieves the completed result

Handling job completion: When a job finishes, your application needs to notify the user. One approach: surface a notification that, when clicked, sends a `HumanMessage` like "Check job_123 and summarize the results."

Tool patterns

There are two main ways to expose subagents as tools:

Pattern	Best for	Trade-off
Tool per agent	Fine-grained control over each subagent's input/output	More setup, but more customization
Single dispatch tool	Many agents, distributed teams, convention over configuration	Simpler composition, less per-agent customization

Tool per agent

```
graph LR
    A[User] --> B[Main Agent]
    B --> C[Subagent A]
    B --> D[Subagent B]
    B --> E[Subagent C]
    C --> B
    D --> B
    E --> B
    B --> F[User response]
```

```
classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710

class A,F trigger
class B,C,D,E process
```

The key idea is wrapping subagents as tools that the main agent can call:

python

```
from langchain.tools import tool
from langchain.agents import create_agent

# Create a sub-agent
subagent = create_agent(model="...", tools=[...]) # [!code highlight]

# Wrap it as a tool # [!code highlight]
@tool("subagent_name", description="subagent_description") # [!code highlight]
def call_subagent(query: str): # [!code highlight]
    result = subagent.invoke({"messages": [{"role": "user", "content": query}]})
    return result["messages"][-1].content

# Main agent with subagent as a tool # [!code highlight]
main_agent = create_agent(model="...", tools=[call_subagent]) # [!code highlight]
```


js

```
import { createAgent, tool } from "langchain";
import * as z from "zod";

// Create a sub-agent
const subagent = createAgent({...}); // [!code highlight]

// Wrap it as a tool // [!code highlight]
const callSubagent = tool( // [!code highlight]
  async ({ query }) => { // [!code highlight]
    const result = await subagent.invoke({
      messages: [{ role: "user", content: query }]
    });
    return result.messages.at(-1)?.text;
  },
  {
    name: "subagent_name",
    description: "subagent_description",
    schema: z.object({
      query: z.string().describe("The query to send to subagent")
    })
  }
);

// Main agent with subagent as a tool // [!code highlight]
const mainAgent = createAgent({ model, tools: [callSubagent] }); // [!code highlight]
```

The main agent invokes the subagent tool when it decides the task matches the subagent's description, receives the result, and continues orchestration. See [Context engineering](#) for fine-grained control.

Single dispatch tool

An alternative approach uses a single parameterized tool to invoke ephemeral sub-agents for independent tasks. Unlike the [tool per agent](#) approach where each sub-agent is wrapped as a separate tool, this uses a convention-based approach with a single `task` tool: the task description is passed as a human message to the sub-agent, and the sub-agent's final message is returned as the tool result.

Use this approach when you want to distribute agent development across multiple teams, need to isolate complex tasks into separate context windows, need a scalable way to add new agents without modifying the coordinator, or prefer convention over customization. This approach trades flexibility in context engineering for simplicity in agent composition and strong context isolation.

```
graph LR
  A[User] --> B[Main Agent]
  B --> C{task<br/>agent_name, description}
  C -->|research| D[Research Agent]
  C -->|writer| E[Writer Agent]
  C -->|reviewer| F[Reviewer Agent]
  D --> C
  E --> C
  F --> C
  C --> B
  B --> G[User response]

classDef trigger fill:#F6FFDB,stroke:#6E8900,stroke-width:2px,color:#2E3900
classDef process fill:#E5F4FF,stroke:#006DDD,stroke-width:2px,color:#030710
classDef decision fill:#FDF3FF,stroke:#7E65AE,stroke-width:2px,color:#504B5F

class A,G trigger
class B,D,E,F process
class C decision
```

Key characteristics:

- Single task tool: One parameterized tool that can invoke any registered sub-agent by name
- Convention-based invocation: Agent selected by name, task passed as human message, final message returned as tool result
- Team distribution: Different teams can develop and deploy agents independently
- Agent discovery: Sub-agents can be discovered via system prompt (listing available agents) or through [progressive disclosure](#) (loading agent information on-demand via tools)

팁

An interesting aspect of this approach is that sub-agents may have the exact same capabilities as the main agent. In such cases, invoking a sub-agent is **really about context isolation** as the primary reason—allowing complex, multi-step tasks to run in isolated context windows without bloating the main agent's conversation history. The sub-agent completes its work autonomously and returns only a concise summary, keeping the main thread focused and efficient.

Agent registry with task dispatcher

python

```
from langchain.tools import tool
from langchain.agents import create_agent
```

```

# Sub-agents developed by different teams
research_agent = create_agent(
    model="gpt-5.5",
    prompt="You are a research specialist..."
)

writer_agent = create_agent(
    model="gpt-5.5",
    prompt="You are a writing specialist..."
)

# Registry of available sub-agents
SUBAGENTS = {
    "research": research_agent,
    "writer": writer_agent,
}

@tool
def task(
    agent_name: str,
    description: str
) -> str:
    """Launch an ephemeral subagent for a task.

    Available agents:
    - research: Research and fact-finding
    - writer: Content creation and editing
    """
    agent = SUBAGENTS[agent_name]
    result = agent.invoke({
        "messages": [
            {"role": "user", "content": description}
        ]
    })
    return result["messages"][-1].content

# Main coordinator agent
main_agent = create_agent(
    model="gpt-5.5",
    tools=[task],
    system_prompt=(
        "You coordinate specialized sub-agents. "
        "Available: research (fact-finding), "
        "writer (content creation). "
        "Use the task tool to delegate work."
    ),
)

```

js

```
import { tool, createAgent } from "langchain";
import * as z from "zod";

// Sub-agents developed by different teams
const researchAgent = createAgent({
  model: "gpt-5.5",
  prompt: "You are a research specialist...",
});

const writerAgent = createAgent({
  model: "gpt-5.5",
  prompt: "You are a writing specialist...",
});

// Registry of available sub-agents
const SUBAGENTS = {
  research: researchAgent,
  writer: writerAgent,
};

const task = tool(
  async ({ agentName, description }) => {
    const agent = SUBAGENTS[agentName];
    const result = await agent.invoke({
      messages: [
        { role: "user", content: description }
      ],
    });
    return result.messages.at(-1)?.content;
  },
  {
    name: "task",
    description: `Launch an ephemeral subagent.

Available agents:
- research: Research and fact-finding
- writer: Content creation and editing`,
    schema: z.object({
      agentName: z
        .string()
        .describe("Name of agent to invoke"),
      description: z
        .string()
        .describe("Task description"),
    }),
  }
);
```

```
// Main coordinator agent
const mainAgent = createAgent({
  model: "gpt-5.5",
  tools: [task],
  prompt: (
    "You coordinate specialized sub-agents. " +
    "Available: research (fact-finding), " +
    "writer (content creation). " +
    "Use the task tool to delegate work."
  ),
});
```

Context engineering

Control how context flows between the main agent and its subagents:

Category	Purpose	Impacts
Subagent specs	Ensure subagents are invoked when they should be	Main agent routing decisions
Subagent inputs	Ensure subagents can execute well with optimized context	Subagent performance
Subagent outputs	Ensure the supervisor can act on subagent results	Main agent performance

See also our comprehensive guide on [context engineering](#) for agents.

Subagent specs

The **names** and **descriptions** associated with subagents are the primary way the main agent knows which subagents to invoke. These are prompting levers—choose them carefully.

- **Name:** How the main agent refers to the sub-agent. Keep it clear and action-oriented (e.g., `research_agent`, `code_reviewer`).
- **Description:** What the main agent knows about the sub-agent's capabilities. Be specific about what tasks it handles and when to use it.

For the [single dispatch tool](#) design, you must additionally provide the main agent with information about the subagents it can invoke. You can provide this information in different ways based on the number of agents and whether your registry is static or dynamic:

Method	Best for	Tradeoff
System prompt enumeration	Small, static agent lists (< 10 agents)	Simple, but requires prompt updates when agents change

Method	Best for	Tradeoff
Enum constraint	Small, static agent lists (< 10 agents)	Type-safe and explicit, but requires code changes when agents change
Tool-based discovery	Large or dynamic agent registries	Flexible and scalable, but adds complexity

System prompt enumeration

List available agents directly in the main agent's system prompt. The main agent sees the list of agents and their descriptions as part of its instructions.

When to use: - You have a small, fixed set of agents (< 10) - Agent registry rarely changes - You want the simplest implementation

Example:

```
main_agent = create_agent(
    model="...",
    tools=[task],
    system_prompt=(
        "You coordinate specialized sub-agents. "
        "Available agents:\n"
        "- research: Research and fact-finding\n"
        "- writer: Content creation and editing\n"
        "- reviewer: Code and document review\n"
        "Use the task tool to delegate work."
    ),
)
```

Enum constraint on dispatch tool

Add an enum constraint to the `agent_name` parameter in your dispatch tool. This provides type safety and makes available agents explicit in the tool schema.

When to use: - You have a small, fixed set of agents (< 10) - You want type safety and explicit agent names - You prefer schema-based validation over prompt-based guidance

Example:

```
from enum import Enum

class AgentName(str, Enum):
    RESEARCH = "research"
    WRITER = "writer"
    REVIEWER = "reviewer"

@tool
```

```
def task(
    agent_name: AgentName, # Enum constraint
    description: str
) -> str:
    """Launch an ephemeral subagent for a task."""
    # ...
```

Tool-based discovery

Provide a separate tool (e.g., `list_agents` or `search_agents`) that the main agent can call to discover available agents on-demand. This enables progressive disclosure and supports dynamic registries.

When to use: - You have many agents (> 10) or a growing registry - Agent registry changes frequently or is dynamic - You want to reduce prompt size and token usage - Different teams manage different agents independently

Example:

```
@tool
def list_agents(query: str = "") -> str:
    """List available subagents, optionally filtered by query."""
    agents = search_agent_registry(query)
    return format_agent_list(agents)

@tool
def task(agent_name: str, description: str) -> str:
    """Launch an ephemeral subagent for a task."""
    # ...

main_agent = create_agent(
    model="...",
    tools=[task, list_agents],
    system_prompt="Use list_agents to discover available subagents, then use task to invoke them."
)
```

Subagent inputs

Customize what context the subagent receives to execute its task. Add input that isn't practical to capture in a static prompt—full message history, prior results, or task metadata—by pulling from the agent's state.

python

```
```python Subagent inputs example expandable from langchain.agents import AgentState
from langchain.tools import tool, ToolRuntime
```

```
class CustomState(AgentState): example_state_key: str
```

```
@tool("subagent1_name", description="subagent1_description") def
call_subagent1(query: str, runtime: ToolRuntime[None, CustomState]): # Apply any logic
needed to transform the messages into a suitable input subagent_input =
some_logic(query, runtime.state["messages"]) result = subagent1.invoke({ "messages":
subagent_input, # You could also pass other state keys here as needed. # Make sure to
define these in both the main and subagent's # state schemas. "example_state_key":
runtime.state["example_state_key"] }) return result["messages"][-1].content
```

```
</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```typescript
Subagent inputs example expandable
import { createAgent, tool, AgentState, ToolMessage } from "langchain";
import { Command } from "@langchain/langgraph";
import * as z from "zod";

// Example of passing the full conversation history to the sub agent via the state.
const callSubagent1 = tool(
  async ({query}) => {
    const state = getCurrentTaskInput<AgentState>();
    // Apply any logic needed to transform the messages into a suitable input
    const subAgentInput = someLogic(query, state.messages);
    const result = await subagent1.invoke({
      messages: subAgentInput,
      // You could also pass other state keys here as needed.
      // Make sure to define these in both the main and subagent's
      // state schemas.
      exampleStateKey: state.exampleStateKey
    });
    return result.messages.at(-1)?.content;
  },
  {
    name: "subagent1_name",
    description: "subagent1_description",
  }
);
```

`some_logic` controls how much context the subagent receives. Passing only the task description (`query`) keeps the subagent isolated. Passing the parent's message history (`runtime.state["messages"]`) instead seeds the subagent with prior work, which helps when it

needs to continue something the parent already started, like resuming a PR review or picking up a bug fix.

[Deep Agents](#) names these patterns `mode: "isolated"` (default) and `mode: "fork"`, and implements forking for you: a forked subagent receives the parent's full message history (including tool calls) and system prompt.

Mode	Subagent receives	Best for	Tradeoff
Isolated (default)	Only the task description	Focused work that needs little prior context	Context isolation, but re-derives everything the parent already did
Forked	The parent's conversation history (and, in Deep Agents, the system prompt)	Continuing a task the parent already started	Seeded with prior context, at the cost of a larger prompt and less isolation

Subagent outputs

Customize what the main agent receives back so it can make good decisions. Two strategies:

1. **Prompt the sub-agent:** Specify exactly what should be returned. A common failure mode is that the sub-agent performs tool calls or reasoning but doesn't include results in its final message—remind it that the supervisor only sees the final output.
2. **Format in code:** Adjust or enrich the response before returning it. For example, pass specific state keys back in addition to the final text using a [Command](#).

python

```
```python Subagent outputs example expandable from typing import Annotated from
langchain.agents import AgentState from langchain.tools import InjectedToolCallId from
langgraph.types import Command

@tool("subagent1_name", description="subagent1_description") def call_subagent1(
query: str, tool_call_id: Annotated[str, InjectedToolCallId],) -> Command: result =
subagent1.invoke({ "messages": [{"role": "user", "content": query}] }) return
Command(update={ # Pass back additional state from the subagent "example_state_key":
result["example_state_key"], "messages": [ToolMessage(content=result["messages"]
[-1].content, tool_call_id=tool_call_id)] })
```

</div>

<div class="callout js" markdown="1">

<p class="callout-label">js</p>

```
```typescript Subagent outputs example expandable
```

```

import { tool, ToolMessage } from "langchain";
import { Command } from "@langchain/langgraph";
import * as z from "zod";

const callSubagent1 = tool(
  async ({ query }, config) => {
    const result = await subagent1.invoke({
      messages: [{ role: "user", content: query }]
    });

    // Return a Command to update multiple state keys
    return new Command({
      update: {
        // Pass back additional state from the subagent
        exampleStateKey: result.exampleStateKey,
        messages: [
          new ToolMessage({
            content: result.messages.at(-1)?.text,
            tool_call_id: config.toolCall?.id!
          })
        ]
      }
    });
  },
  {
    name: "subagent1_name",
    description: "subagent1_description",
    schema: z.object({
      query: z.string().describe("The query to send to subagent1")
    })
  }
);

```

Checkpointing and state inspection

By default, subagents use the **inherited checkpointer** mode—each invocation starts with fresh state, supports [interrupts](#), and runs safely in parallel. If you need a subagent to maintain its own persistent conversation history across invocations, compile it with `checkerpointer=True` (continuations mode). See [subgraph persistence](#) for a full comparison of modes.

Because subagents are called inside tool functions, LangGraph cannot [statically discover](#) them. This means [get_state_with_subgraphs](#) will not return subagent state. If you need to read nested graph state (e.g., during an [interrupt](#)), invoke the subagent from a [node function](#) in a custom graph instead. See [subgraph persistence](#) for details on how each mode affects state visibility.

python

[Migrate from langgraph-supervisor](#)

The langgraph-supervisor package is no longer actively maintained. Learn how to migrate from create_supervisor to the subagents pattern, including interrupt and resume flows with external API callbacks.

langchain / test

Agent Evals

langchain/test/evals.mdx

Evaluations ("evals") measure how well your agent performs by assessing its execution trajectory, the sequence of messages and tool calls it produces. Unlike [integration tests](#) that verify basic correctness, evals score agent behavior against a reference or rubric, making them useful for catching regressions when you change prompts, tools, or models.

An evaluator is a function that takes agent outputs (and optionally reference outputs) and returns a score:

python

```
def evaluator(*, outputs: dict, reference_outputs: dict):
    output_messages = outputs["messages"]
    reference_messages = reference_outputs["messages"]
    score = compare_messages(output_messages, reference_messages)
    return {"key": "evaluator_score", "score": score}
```

js

```
function evaluator({ outputs, referenceOutputs }: {
  outputs: Record<string, any>;
  referenceOutputs: Record<string, any>;
}) {
  const outputMessages = outputs.messages;
  const referenceMessages = referenceOutputs.messages;
  const score = compareMessages(outputMessages, referenceMessages);
  return { key: "evaluator_score", score: score };
}
```

The [agentevals](#) package provides prebuilt evaluators for agent trajectories. You can evaluate by performing a **trajectory match** (deterministic comparison) or by using an **LLM judge** (qualitative assessment):

Approach	When to use
Trajectory match	You know the expected tool calls and want fast, deterministic, cost-free checks
LLM-as-judge	You want to assess overall quality and reasoning without strict expectations

Install AgentEvals

python

```
```bash pip pip install -U agentevals
```

```
```bash uv
uv add agentevals
```

js

```
npm install agentevals @langchain/core
```

Or, clone the [AgentEvals repository](#) directly.

Trajectory match evaluator

python

AgentEvals offers the `create_trajectory_match_evaluator` function to match your agent's trajectory against a reference. There are four modes:

js

AgentEvals offers the `createTrajectoryMatchEvaluator` function to match your agent's trajectory against a reference. There are four modes:

Mode	Description	Use case
strict	Exact match of message structure and tool calls in the same order (message content can differ)	Testing specific sequences (e.g., policy lookup before authorization)
unordered	Same message structure and tool calls as reference, but tool calls can happen in any order	Verifying information retrieval when order doesn't matter
subset	Agent calls only tools from reference (no extras)	Ensuring agent doesn't exceed expected scope
superset	Agent calls at least the reference tools (extras allowed)	Verifying minimum required actions are taken

The examples below share a common setup, an agent with a `get_weather` tool:

python

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from agentevals.trajectory.match import create_trajectory_match_evaluator

@tool
def get_weather(city: str):
    """Get weather information for a city."""
    return f"It's 75 degrees and sunny in {city}."

agent = create_agent("claude-sonnet-4-6", tools=[get_weather])
```

js

```
import { createAgent } from "langchain";
import { tool } from "@langchain/core/tools";
import { HumanMessage, AIMessage, ToolMessage } from "@langchain/core/messages";
import { createTrajectoryMatchEvaluator } from "agentevals";
import * as z from "zod";

const getWeather = tool(
  async ({ city }) => {
    return `It's 75 degrees and sunny in ${city}.`;
  },
  {
    name: "get_weather",
    description: "Get weather information for a city.",
    schema: z.object({ city: z.string() }),
  }
);

const agent = createAgent({
  model: "claude-sonnet-4-6",
  tools: [getWeather],
});
```

Strict match

The `strict` mode ensures trajectories contain identical messages in the same order with the same tool calls, though it allows for differences in message content. This is useful when you need to enforce a specific sequence of operations, such as requiring a policy lookup before authorizing an action.

python

```

evaluator = create_trajectory_match_evaluator( # [!code highlight]
    trajectory_match_mode="strict", # [!code highlight]
) # [!code highlight]

def test_weather_tool_called_strict():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in San Francisco?")]
    })

reference_trajectory = [
    HumanMessage(content="What's the weather in San Francisco?"),
    AIMessage(content="", tool_calls=[
        {"id": "call_1", "name": "get_weather", "args": {"city": "San Francisco"}}
    ]),
    ToolMessage(content="It's 75 degrees and sunny in San Francisco.", tool_call_id="call_1"),
    AIMessage(content="The weather in San Francisco is 75 degrees and sunny."),
]

evaluation = evaluator(
    outputs=result["messages"],
    reference_outputs=reference_trajectory
)
# {
#     'key': 'trajectory_strict_match',
#     'score': True,
#     'comment': None,
# }
assert evaluation["score"] is True

```

js

```

const evaluator = createTrajectoryMatchEvaluator({ // [!code highlight]
    trajectoryMatchMode: "strict", // [!code highlight]
}); // [!code highlight]

async function testWeatherToolCalledStrict() {
    const result = await agent.invoke({
        messages: [new HumanMessage("What's the weather in San Francisco?")]
    });

const referenceTrajectory = [
    new HumanMessage("What's the weather in San Francisco?"),
    new AIMessage({
        content: "",
        tool_calls: [
            { id: "call_1", name: "get_weather", args: { city: "San Francisco" } }
        ]
    }),
]

```



```

new ToolMessage({
  content: "It's 75 degrees and sunny in San Francisco.",
  tool_call_id: "call_1"
}),
new AIMessage("The weather in San Francisco is 75 degrees and sunny."),
];

const evaluation = await evaluator({
  outputs: result.messages,
  referenceOutputs: referenceTrajectory
});
expect(evaluation.score).toBe(true);
}

```

Unordered match

The `unordered` mode allows the same tool calls in any order. This is helpful when you want to verify that specific information was retrieved but don't care about the sequence. For example, an agent that checks both weather and events for a city with different tool calls.

python

```

@tool
def get_events(city: str):
    """Get events happening in a city."""
    return f"Concert at the park in {city} tonight."

agent = create_agent("claude-sonnet-4-6", tools=[get_weather, get_events])

evaluator = create_trajectory_match_evaluator( # [!code highlight]
    trajectory_match_mode="unordered", # [!code highlight]
) # [!code highlight]

def test_multiple_tools_any_order():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's happening in SF today?")]
    })

reference_trajectory = [
    HumanMessage(content="What's happening in SF today?"),
    AIMessage(content="", tool_calls=[
        {"id": "call_1", "name": "get_events", "args": {"city": "SF"}},
        {"id": "call_2", "name": "get_weather", "args": {"city": "SF"}},
    ]),
    ToolMessage(content="Concert at the park in SF tonight.", tool_call_id="call_1"),
    ToolMessage(content="It's 75 degrees and sunny in SF.", tool_call_id="call_2"),
    AIMessage(content="Today in SF: 75 degrees and sunny with a concert at the park tonight."),
]

```

```

evaluation = evaluator(
  outputs=result["messages"],
  reference_outputs=reference_trajectory,
)
assert evaluation["score"] is True

```

js

```

const getEvents = tool(
  async ({ city }: { city: string }) => {
    return `Concert at the park in ${city} tonight.`;
  },
  {
    name: "get_events",
    description: "Get events happening in a city.",
    schema: z.object({ city: z.string() }),
  }
);

const agent = createAgent({
  model: "claude-sonnet-4-6",
  tools: [getWeather, getEvents],
});

const evaluator = createTrajectoryMatchEvaluator({ // [!code highlight]
  trajectoryMatchMode: "unordered", // [!code highlight]
}); // [!code highlight]

async function testMultipleToolsAnyOrder() {
  const result = await agent.invoke({
    messages: [new HumanMessage("What's happening in SF today?")]
  });

  const referenceTrajectory = [
    new HumanMessage("What's happening in SF today?"),
    new AIMessage({
      content: "",
      tool_calls: [
        { id: "call_1", name: "get_events", args: { city: "SF" } },
        { id: "call_2", name: "get_weather", args: { city: "SF" } },
      ]
    }),
    new ToolMessage({
      content: "Concert at the park in SF tonight.",
      tool_call_id: "call_1"
    }),
    new ToolMessage({
      content: "It's 75 degrees and sunny in SF.",

```

```

        tool_call_id: "call_2"
    })),
    new AIMessage("Today in SF: 75 degrees and sunny with a concert at the park tonight."),
];

const evaluation = await evaluator({
    outputs: result.messages,
    referenceOutputs: referenceTrajectory,
});
expect(evaluation.score).toBe(true);
}

```

Subset and superset match

The `superset` and `subset` modes match partial trajectories. The `superset` mode verifies that the agent called at least the tools in the reference trajectory, allowing additional tool calls. The `subset` mode ensures the agent did not call any tools beyond those in the reference.

python

```

@tool
def get_detailed_forecast(city: str):
    """Get detailed weather forecast for a city."""
    return f"Detailed forecast for {city}: sunny all week."

agent = create_agent("claude-sonnet-4-6", tools=[get_weather, get_detailed_forecast])

evaluator = create_trajectory_match_evaluator( # [!code highlight]
    trajectory_match_mode="superset", # [!code highlight]
) # [!code highlight]

def test_agent_calls_required_tools_plus_extra():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in Boston?")]
    })

# Reference only requires get_weather, but agent may call additional tools
reference_trajectory = [
    HumanMessage(content="What's the weather in Boston?"),
    AIMessage(content="", tool_calls=[
        {"id": "call_1", "name": "get_weather", "args": {"city": "Boston"}},
    ]),
    ToolMessage(content="It's 75 degrees and sunny in Boston.", tool_call_id="call_1"),
    AIMessage(content="The weather in Boston is 75 degrees and sunny."),
]

evaluation = evaluator(
    outputs=result["messages"],

```

```

        reference_outputs=reference_trajectory,
    )
    assert evaluation["score"] is True

```

js

```

const getDetailedForecast = tool(
  async ({ city }: { city: string }) => {
    return `Detailed forecast for ${city}: sunny all week.`;
  },
  {
    name: "get_detailed_forecast",
    description: "Get detailed weather forecast for a city.",
    schema: z.object({ city: z.string() }),
  }
);

const agent = createAgent({
  model: "claude-sonnet-4-6",
  tools: [getWeather, getDetailedForecast],
});

const evaluator = createTrajectoryMatchEvaluator({ // [!code highlight]
  trajectoryMatchMode: "superset", // [!code highlight]
}); // [!code highlight]

async function testAgentCallsRequiredToolsPlusExtra() {
  const result = await agent.invoke({
    messages: [new HumanMessage("What's the weather in Boston?")]
  });

  const referenceTrajectory = [
    new HumanMessage("What's the weather in Boston?"),
    new AIMessage({
      content: "",
      tool_calls: [
        { id: "call_1", name: "get_weather", args: { city: "Boston" } },
      ]
    }),
    new ToolMessage({
      content: "It's 75 degrees and sunny in Boston.",
      tool_call_id: "call_1"
    }),
    new AIMessage("The weather in Boston is 75 degrees and sunny."),
  ];

  const evaluation = await evaluator({
    outputs: result.messages,
    referenceOutputs: referenceTrajectory,
  });

```

```
});  
expect(evaluation.score).toBe(true);  
}
```

정보

python

You can also set the `tool_args_match_mode` property and/or `tool_args_match_overrides` to customize how the evaluator considers equality between tool calls in the actual trajectory vs. the reference. By default, only tool calls with the same arguments to the same tool are considered equal. Visit the [repository](#) for more details.

js

You can also set the `toolArgsMatchMode` property and/or `toolArgsMatchOverrides` to customize how the evaluator considers equality between tool calls in the actual trajectory vs. the reference. By default, only tool calls with the same arguments to the same tool are considered equal. Visit the [repository](#) for more details.

LLM-as-judge evaluator

python

You can use an LLM to evaluate the agent's execution path with the `create_trajectory_llm_as_judge` function. Unlike trajectory match evaluators, it doesn't require a reference trajectory, but one can be provided if available.

js

You can use an LLM to evaluate the agent's execution path with the `createTrajectoryLLMAsJudge` function. Unlike trajectory match evaluators, it doesn't require a reference trajectory, but one can be provided if available.

Without reference trajectory

python

```
from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TRAJECTORY_ACCURACY_PROMPT  
  
evaluator = create_trajectory_llm_as_judge( # [!code highlight]
```

```

model="openai:o3-mini", # [!code highlight]
prompt=TRAJECTORY_ACCURACY_PROMPT, # [!code highlight]
) # [!code highlight]

def test_trajectory_quality():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in Seattle?")]
    })

    evaluation = evaluator(
        outputs=result["messages"],
    )
    assert evaluation["score"] is True

```

js

```

import { createTrajectoryLLMAsJudge, TRAJECTORY_ACCURACY_PROMPT } from "agentevals";

const evaluator = createTrajectoryLLMAsJudge({ // [!code highlight]
    model: "openai:o3-mini", // [!code highlight]
    prompt: TRAJECTORY_ACCURACY_PROMPT, // [!code highlight]
}); // [!code highlight]

async function testTrajectoryQuality() {
    const result = await agent.invoke({
        messages: [new HumanMessage("What's the weather in Seattle?")]
    });

    const evaluation = await evaluator({
        outputs: result.messages,
    });
    expect(evaluation.score).toBe(true);
}

```

With reference trajectory

If you have a reference trajectory, use the prebuilt `TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE` prompt:

python

```

from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE

evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE,

```

```
)
evaluation = evaluator(
    outputs=result["messages"],
    reference_outputs=reference_trajectory,
)
```

js

```
import { createTrajectoryLLMAsJudge, TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE } from
"agentevals";

const evaluator = createTrajectoryLLMAsJudge({
    model: "openai:o3-mini",
    prompt: TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE,
});

const evaluation = await evaluator({
    outputs: result.messages,
    referenceOutputs: referenceTrajectory,
});
```

정보

For more configurability over how the LLM evaluates the trajectory, visit the [repository](#).

python

Async support

All `agentevals` evaluators support Python asyncio. Async versions are available by adding `async` after `create_` in the function name.

Async judge and evaluator example

```
from agentevals.trajectory.llm import create_async_trajectory_llm_as_judge, TRAJECTORY_
ACCURACY_PROMPT
from agentevals.trajectory.match import create_async_trajectory_match_evaluator

async_judge = create_async_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)

async_evaluator = create_async_trajectory_match_evaluator(
    trajectory_match_mode="strict",
)
```

```

async def test_async_evaluation():
    result = await agent.ainvoke({
        "messages": [HumanMessage(content="What's the weather?")]
    })

    evaluation = await async_judge(outputs=result["messages"])
    assert evaluation["score"] is True

```

Run evals in LangSmith

For tracking experiments over time, log evaluator results to [LangSmith](#). First, set the required environment variables:

```

export LANGSMITH_API_KEY="your_langsmith_api_key"
export LANGSMITH_TRACING="true"

```

python

LangSmith offers two main approaches for running evaluations: [pytest](#) integration and the `evaluate` function.

Use pytest integration

```

import pytest
from langsmith import testing as t
from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TRAJECTORY_ACCURACY_PROMPT

trajectory_evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)

@pytest.mark.langsmith
def test_trajectory_accuracy():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in SF?")]
    })

    reference_trajectory = [
        HumanMessage(content="What's the weather in SF?"),
        AIMessage(content="", tool_calls=[
            {"id": "call_1", "name": "get_weather", "args": {"city": "SF"}},
        ]),
        ToolMessage(content="It's 75 degrees and sunny in SF.", tool_call_id="call_1"),
        AIMessage(content="The weather in SF is 75 degrees and sunny."),
    ]

```



```

]

t.log_inputs({})
t.log_outputs({"messages": result["messages"]})
t.log_reference_outputs({"messages": reference_trajectory})

    trajectory_evaluator(
        outputs=result["messages"],
        reference_outputs=reference_trajectory
    )

```

Run the evaluation with pytest:

```
pytest test_trajectory.py --langsmith-output
```

Use the evaluate function

Create a [LangSmith dataset](#) and use the `evaluate` function. The dataset must have the following schema:

- **input:** `{"messages": [...]}` input messages to call the agent with.
- **output:** `{"messages": [...]}` expected message history in the agent output. For trajectory evaluation, you can choose to keep only assistant messages.

```

from langsmith import Client
from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TRAJECTORY_ACCURACY_PROMPT

client = Client()

trajectory_evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)

def run_agent(inputs):
    return agent.invoke(inputs)["messages"]

experiment_results = client.evaluate(
    run_agent,
    data="your_dataset_name",
    evaluators=[trajectory_evaluator]
)

```

팁

To learn more about evaluating your agent, see the [LangSmith docs](#).

js

LangSmith offers two main approaches for running evaluations: [Vitest/Jest](#) integration and the `evaluate` function.

Use vitest/jest integration

```
import * as ls from "langsmith/vitest";
// import * as ls from "langsmith/jest";

import { createTrajectoryLLMAsJudge, TRAJECTORY_ACCURACY_PROMPT } from "agentevals";

const trajectoryEvaluator = createTrajectoryLLMAsJudge({
  model: "openai:o3-mini",
  prompt: TRAJECTORY_ACCURACY_PROMPT,
});

ls.describe("trajectory accuracy", () => {
  ls.test("accurate trajectory", {
    inputs: {
      messages: [
        { role: "user", content: "What is the weather in SF?" }
      ]
    },
    referenceOutputs: {
      messages: [
        new HumanMessage("What is the weather in SF?"),
        new AIMessage({
          content: "",
          tool_calls: [
            { id: "call_1", name: "get_weather", args: { city: "SF" } }
          ]
        }),
        new ToolMessage({
          content: "It's 75 degrees and sunny in SF.",
          tool_call_id: "call_1"
        }),
        new AIMessage("The weather in SF is 75 degrees and sunny."),
      ],
    },
  }, async ({ inputs, referenceOutputs }) => {
    const result = await agent.invoke({
      messages: [new HumanMessage("What is the weather in SF?")]
    });
  });

  ls.logOutputs({ messages: result.messages });

  await trajectoryEvaluator({
    inputs,
    outputs: result.messages,
  });
});
```

```
referenceOutputs,  
  });  
});  
});
```

Run the evaluation with your test runner:

```
vitest run test_trajectory.eval.ts  
# or  
jest test_trajectory.eval.ts
```

Use the evaluate function

Create a [LangSmith dataset](#) and use the `evaluate` function. The dataset must have the following schema:

- **input:** `{"messages": [...]}` input messages to call the agent with.
- **output:** `{"messages": [...]}` expected message history in the agent output. For trajectory evaluation, you can choose to keep only assistant messages.

```
import { evaluate } from "langsmith/evaluation";  
import { createTrajectoryLLMAsJudge, TRAJECTORY_ACCURACY_PROMPT } from "agentevals";  
  
const trajectoryEvaluator = createTrajectoryLLMAsJudge({  
  model: "openai:o3-mini",  
  prompt: TRAJECTORY_ACCURACY_PROMPT,  
});  
  
async function runAgent(inputs: any) {  
  const result = await agent.invoke(inputs);  
  return result.messages;  
}  
  
await evaluate(  
  runAgent,  
  {  
    data: "your_dataset_name",  
    evaluators: [trajectoryEvaluator],  
  }  
);
```

팁

To learn more about evaluating your agent, see the [LangSmith docs](#).

Test

langchain/test/index.mdx

Agentic applications let an LLM decide its own next steps to solve a problem. That flexibility is powerful, but the model's black-box nature makes it hard to predict how a tweak in one part of your agent will affect the whole. To build production-ready agents, thorough testing is essential.

There are a few approaches to testing your agents:

- **Unit tests** exercise small, deterministic pieces of your agent in isolation using in-memory fakes so you can assert exact behavior quickly and deterministically.
- **Integration tests** test the agent using real network calls to confirm that components work together, credentials and schemas line up, and latency is acceptable.
- **Evals** use evaluators to assess your agent's execution trajectory, either via deterministic matching or an LLM judge.

Agentic applications tend to lean more on integration because they chain multiple components together and must deal with flakiness due to the nondeterministic nature of LLMs.

팁

Run evaluations at scale, track results over time, and compare experiments with [LangSmith](#). See [Evaluate an LLM application](#) to get started.

[Unit testing](#)

Mock chat models and use in-memory persistence to test agent logic without API calls.

[Integration testing](#)

Test your agent with real LLM APIs. Organize tests, manage keys, handle flakiness, and control costs.

[Evals](#)

Evaluate agent trajectories with deterministic matching or LLM-as-judge evaluators.

Integration testing

langchain/test/integration-testing.mdx

Integration tests verify that your agent works correctly with model APIs and external services. Unlike [unit tests](#) that use fakes and mocks, integration tests make actual network calls to confirm that components work together, credentials are valid, and latency is acceptable.

Because LLM responses are nondeterministic, integration tests require different strategies than traditional software tests. This guide covers how to organize, write, and run integration tests for your agents. For general test infrastructure when contributing to LangChain itself, see [Contributing to code](#).

Separate unit and integration tests

Integration tests are slower and require API credentials, so keep them separate from unit tests. This lets you run fast unit tests on every change and reserve integration tests for CI or pre-deploy checks.

python

Use pytest markers to tag integration tests:

```
import pytest
from langchain.agents import create_agent
from langchain.messages import HumanMessage

@pytest.mark.integration
def test_agent_with_real_model():
    agent = create_agent("claude-sonnet-4-6", tools=[get_weather])
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in SF?")]
    })
    assert len(result["messages"]) > 1
```

Configure pytest to recognize the marker and exclude integration tests from default runs:

```
```.ini
pytest.ini
[pytest]
markers = integration: tests that call real LLM APIs
addopts = -m "not integration"
```

```
```.toml
pyproject.toml
[tool.pytest.ini_options]
markers = [
    "integration: tests that call real LLM APIs"
```

```
]
addopts = "-m 'not integration'"
```

Run integration tests explicitly:

```
pytest -m integration
```

js

Use a file naming convention to separate integration tests. Name integration test files `*.int.test.ts` and configure vitest to exclude them from default runs:

```
` `` `ts vitest.config.ts import { configDefaults, defineConfig } from "vitest/config";

export default defineConfig((env) => { if (env.mode === "int") { return { test: { testTimeout:
100_000, include: ["*/.int.test.ts"], setupFiles: ["dotenv/config"], }, }; }

return { test: { testTimeout: 30_000, exclude: ["*/.int.test.ts", ...configDefaults.exclude], }, };
});
```

Add scripts to `package.json`:

```
` `` `json
{
  "scripts": {
    "test": "vitest",
    "test:integration": "vitest --mode int"
  }
}
```

Run integration tests explicitly:

```
npm run test:integration
```

Manage API keys

Integration tests require real API credentials. Load them from environment variables so keys stay out of source control.

python

Use a `conftest.py` fixture to validate that required keys are available:

```
import os
import pytest

@pytest.fixture(autouse=True)
def check_api_keys():
    if not os.environ.get("OPENAI_API_KEY"):
        pytest.skip("OPENAI_API_KEY not set")
```

For local development, store keys in a `.env` file and load them with [python-dotenv](#):

```
`` bash .env OPENAI_API_KEY=sk-...
```

```
`` python conftest.py
from dotenv import load_dotenv

load_dotenv()
```

js

Add `dotenv/config` as a vitest setup file so environment variables load automatically from `.env`:

```
`` ts vitest.config.ts export default defineConfig({ test: { setupFiles: ["dotenv/config"], }, });
```

```
`` bash .env
OPENAI_API_KEY=sk-...
```

Skip tests when keys are missing:

```
import { test } from "vitest";

test.skipIf(!process.env.OPENAI_API_KEY)(
  "agent responds with tool call",
  async () => {
    // ...
  }
);
```

주의

Add `.env` to your `.gitignore` to avoid committing credentials. In CI, inject secrets through your provider's secrets management (e.g., GitHub Actions secrets).

Assert on structure, not content

LLM responses vary between runs. Instead of asserting on exact output strings, verify the structural properties of the response: message types, tool call names, argument shapes, and message count.

python

```
from langchain.agents import create_agent
from langchain.messages import AIMessage, HumanMessage

def test_agent_calls_weather_tool():
    agent = create_agent("claude-sonnet-4-6", tools=[get_weather])
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in SF?")]
    })

    messages = result["messages"]
    tool_calls = [
        tc
        for msg in messages
        if hasattr(msg, "tool_calls")
        for tc in (msg.tool_calls or [])
    ]

    assert any(tc["name"] == "get_weather" for tc in tool_calls)
    assert isinstance(messages[-1], AIMessage)
    assert len(messages[-1].content) > 0
```

js

```
test("agent calls weather tool", async () => {
    const agent = createAgent({ model: "claude-sonnet-4-6", tools: [getWeather] });
    const result = await agent.invoke({
        messages: [new HumanMessage("What's the weather in SF?")]
    });

    const aiMsg = result.messages.find(
        (m) => AIMessage.isInstance(m) && m.tool_calls?.length
    );
    expect(aiMsg).toContainToolCall({ name: "get_weather" });
    expect(result.messages.at(-1)).toBeAIMessage();
});
```

This example uses [custom test matchers](#). See the section below for setup and the full matcher reference.

팁

For more rigorous trajectory assertions, use the [AgentEvals](#) evaluators which support fuzzy matching modes like `unordered` and `superset`.

js

Use custom test matchers

`langchain` ships [custom vitest matchers](#) that make structural assertions more readable and produce clear error messages on failure. Register them once in a setup file and they become available on every `expect()` call.

Set up

Add a vitest setup file that extends `expect` with the `LangChain` matchers:

```
``ts vitest.setup.ts import { langchainMatchers } from "@langchain/core/testing";

expect.extend(langchainMatchers);
```

Reference it in your vitest config:

```
``ts vitest.config.ts
export default defineConfig({
  test: {
    setupFiles: ["vitest.setup.ts"],
  },
});
```

TypeScript types are included automatically, so no extra configuration is needed for autocomplete.

Check message types

Each message class has a corresponding matcher: `toBeHumanMessage()`, `toBeAIMessage()`, `toBeSystemMessage()`, and `toBeToolMessage()`. Call without arguments to check only the type, or pass a string to also match content:

```
const response = await agent.invoke({
  messages: [new HumanMessage("What's the weather?")]
});
const lastMessage = response.messages.at(-1);
```

```
expect(lastMessage).toBeAIMessage();
expect(lastMessage).toBeAIMessage("It's 72°F and sunny.");
```

Pass an object to match specific fields:

```
expect(lastMessage).toBeAIMessage({ name: "weather-bot" });
expect(toolMsg).toBeToolMessage({ tool_call_id: "call_1" });
```

Assert on tool calls

Three matchers cover tool call assertions on an `@[ AIMessage ]`:

```
const response = await agent.invoke({
  messages: [new HumanMessage("Weather in SF and NYC?")]
});
const aiMsg = response.messages.find(
  (m) => AIMessage.isInstance(m) && m.tool_calls?.length
);

// Check that specific tool calls are present (order-independent)
expect(aiMsg).toHaveToolCalls([
  { name: "get_weather", args: { city: "San Francisco" } },
  { name: "get_weather", args: { city: "New York" } },
]);

// Check only the count
expect(aiMsg).toHaveToolCallCount(2);

// Check that at least one tool call matches (supports .not)
expect(aiMsg).toContainToolCall({ name: "get_weather" });
expect(aiMsg).not.toContainToolCall({ name: "send_email" });
```

Assert on tool messages

`toHaveToolMessages()` takes the full message array and checks the `@[ ToolMessage ]` instances within it, in order:

```
expect(response.messages).toHaveToolMessages([
  { content: "72°F and sunny in San Francisco" },
  { content: "68°F and cloudy in New York" },
]);
```

Assert on interrupts and structured responses

`toHaveBeenInterrupted()` checks for a `__interrupt__` field in a [LangGraph interrupt](#) result. Pass a value to match the interrupt payload:

```
const result = await graph.invoke(input);

expect(result).toHaveBeenInterrupted();
expect(result).toHaveBeenInterrupted("confirm_action");
```

`toHaveStructuredResponse()` checks for a `structuredResponse` field on the result. Pass an object to match specific fields:

```
expect(result).toHaveStructuredResponse();
expect(result).toHaveStructuredResponse({ name: "Alice", age: 30 });
```

Matcher reference

Matcher	Description
<code>toBeHumanMessage(expected?)</code>	Check that the value is a <code>HumanMessage</code> . Optionally match content (string) or fields (object).
<code>toBeAIMessage(expected?)</code>	Check that the value is an <code>AIMessage</code> . Optionally match content or fields.
<code>toBeSystemMessage(expected?)</code>	Check that the value is a <code>SystemMessage</code> . Optionally match content or fields.
<code>toBeToolMessage(expected?)</code>	Check that the value is a <code>ToolMessage</code> . Optionally match content or fields like <code>tool_call_id</code> .
<code>toHaveToolCalls(expected)</code>	Check that an <code>AIMessage</code> has exactly the given tool calls (order-independent).
<code>toHaveToolCallCount(n)</code>	Check that an <code>AIMessage</code> has exactly <code>n</code> tool calls.
<code>toContainToolCall(expected)</code>	Check that an <code>AIMessage</code> contains at least one matching tool call. Supports <code>.not</code> .
<code>toHaveToolMessages(expected)</code>	Check that a message array contains the given <code>ToolMessage</code> instances, in order.
<code>toHaveBeenInterrupted(value?)</code>	Check that a result has an <code>__interrupt__</code> . Optionally match the interrupt value.
<code>toHaveStructuredResponse(expected?)</code>	Check that a result has a <code>structuredResponse</code> . Optionally match specific fields.

Reduce cost and latency

Integration tests that call LLM APIs incur real costs. A few practices help keep test suites fast and affordable:

- **Use smaller models:** `gemini-3.1-flash-lite` or equivalent for tests that only need to verify tool calling and response structure.
- **Set `maxTokens`:** Cap response length to avoid long, expensive completions.
- **Limit test scope:** Test one behavior per test. Avoid end-to-end scenarios that chain many LLM calls when a single-turn test suffices.
- **Run selectively:** Use the test separation from [above](#) to run integration tests only in CI or before deploy, not on every file save.

python

```
from langchain.agents import create_agent

agent = create_agent(
    "gemini-3.1-flash-lite",
    tools=[get_weather],
    model_kwargs={"max_tokens": 256},
)
```

js

```
const agent = createAgent({
  model: "gemini-3.1-flash-lite",
  tools: [getWeather],
  modelArgs: { maxTokens: 256 },
});
```

python

Record and replay HTTP calls

For tests that run frequently in CI, you can record HTTP interactions on the first run and replay them on subsequent runs without making real API calls. This eliminates cost and latency after the initial recording.

[vcrpy](#) records HTTP request/response pairs into YAML "cassette" files. The [pytest-recording](#) plugin integrates this with pytest.

Set up your `conftest.py` to filter sensitive information from cassettes:

```
```python conftest.py import pytest
```

```
@pytest.fixture(scope="session") def vcr_config(): return { "filter_headers": [
("authorization", "XXXX"), ("x-api-key", "XXXX"),], "filter_query_parameters": [("api_key",
"XXXX"), ("key", "XXXX"),], }
```

Configure your project to recognize the `vcr` marker:

```
```ini pytest.ini
[pytest]
markers =
    vcr: record/replay HTTP via VCR
addopts = --record-mode=once
```

```
```toml pyproject.toml [tool.pytest.ini_options] markers = [ "vcr: record/replay HTTP via
VCR"] addopts = "--record-mode=once"
```

<div class="callout info" markdown="1">

<p class="callout-label">정보</p>

The `--record-mode=once` option records HTTP interactions on the first run and replays them on subsequent runs.

</div>

Decorate your tests with the `vcr` marker:

```
```python
import pytest
from langchain.agents import create_agent
from langchain.messages import HumanMessage

@pytest.mark.vcr()
def test_agent_trajectory():
    agent = create_agent("claude-sonnet-4-6", tools=[get_weather])
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in SF?")]
    })
    assert any(
        tc["name"] == "get_weather"
        for msg in result["messages"]
        if hasattr(msg, "tool_calls")
        for tc in (msg.tool_calls or [])
    )
```

The first run makes real network calls and generates a cassette file in `tests/cassettes/`. Subsequent runs replay the recorded responses.

주의

When you modify prompts, add new tools, or change expected trajectories, your saved cassettes will become outdated and your existing tests **will fail**. Delete the

corresponding cassette files and rerun the tests to record fresh interactions.

Next steps

Learn how to evaluate agent trajectories with deterministic matching or LLM-as-judge evaluators in [Evals](#).

Unit testing

langchain/test/unit-testing.mdx

Unit tests exercise small, deterministic pieces of your agent in isolation. By replacing the real LLM with an in-memory fake (AKA fixture), you can script exact responses (text, tool calls, and errors) so tests are fast, free, and repeatable without API keys.

python

Mock chat model

LangChain provides `@[ GenericFakeChatModel ]` for mocking text responses. It accepts an iterator of responses (`@[ AIMessage ]` objects or strings) and returns one per invocation. It supports both regular and streaming usage.

```
from langchain_core.language_models.fake_chat_models import GenericFakeChatModel
from langchain.messages import AIMessage, ToolCall

model = GenericFakeChatModel(messages=iter([
    AIMessage(content="", tool_calls=[ToolCall(name="foo", args={"bar": "baz"}, id="call_1")]),
    "bar"
]))

model.invoke("hello")
# AIMessage(content='', ..., tool_calls=[{'name': 'foo', 'args': {'bar': 'baz'}, 'id': 'call_1', 'type': 'tool_call'}])
```

If we invoke the model again, it will return the next item in the iterator:

```
model.invoke("hello, again!")
# AIMessage(content='bar', ...)
```

InMemorySaver checkpointer

To enable persistence during testing, you can use the `@[ InMemorySaver ]` checkpointer. This allows you to simulate multiple turns to test state-dependent behavior:

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from langchain.messages import HumanMessage

agent = create_agent(
    model,
```

```

tools=[],
checkpointer=InMemorySaver()
)

# First invocation
agent.invoke(
    {"messages": [HumanMessage(content="I live in Sydney, Australia")]},
    config={"configurable": {"thread_id": "session-1"}}
)

# Second invocation: the first message is persisted (Sydney location), so the model returns GMT+10 time
agent.invoke(
    {"messages": [HumanMessage(content="What's my local time?")]},
    config={"configurable": {"thread_id": "session-1"}}
)

```

js

Mock chat model with `fakeModel`

`@[ fakeModel ]` is a builder-style fake chat model that lets you script exact responses (text, tool calls, errors) and assert what the model received. It extends `@[ BaseChatModel ]`, so it works anywhere a real model is expected.

```
import { fakeModel } from "langchain";
```

Quick start

Create a model, queue responses with `.respond()`, and invoke. Each `invoke()` consumes the next queued response in order:

```

import { fakeModel } from "langchain";
import { AIMessage, HumanMessage } from "@langchain/core/messages";

const model = fakeModel()
    .respond(new AIMessage("I can help with that."))
    .respond(new AIMessage("Here's what I found."))
    .respond(new AIMessage("You're welcome!"));

const r1 = await model.invoke([new HumanMessage("Can you help?")]);
// r1.content === "I can help with that."

const r2 = await model.invoke([new HumanMessage("What did you find?")]);
// r2.content === "Here's what I found."

```



```
const r3 = await model.invoke([new HumanMessage("Thanks!")]);  
// r3.content === "You're welcome!"
```

If the model is invoked more times than there are queued responses, it throws a descriptive error:

```
const model = fakeModel()  
  .respond(new AIMessage("only one"));  
  
await model.invoke([new HumanMessage("first")]); // works  
await model.invoke([new HumanMessage("second")]); // throws: "no response queued for invocation 1"
```

Tool calling responses

`.respond()` supports tool calls by passing an `@[AIMessage]` with `tool_calls`:

```
import { fakeModel } from "langchain";  
import { AIMessage, HumanMessage } from "@langchain/core/messages";  
  
const model = fakeModel()  
  .respond(new AIMessage({  
    content: "",  
    tool_calls: [  
      { name: "get_weather", args: { city: "San Francisco" }, id: "call_1", type: "tool_call" },  
    ],  
  })))  
  .respond(new AIMessage("It's 72°F and sunny in San Francisco."));  
  
const r1 = await model.invoke([new HumanMessage("What's the weather in SF?")]);  
console.log(r1.tool_calls[0].name); // "get_weather"  
  
const r2 = await model.invoke([new HumanMessage("Thanks")]);  
console.log(r2.content); // "It's 72°F and sunny in San Francisco."
```

`.respondWithTools()` is a shorthand for the same thing. Instead of constructing the full `AIMessage`, provide just the tool name and arguments:

```
// These two queue entries produce identical responses:  
  
model.respond(new AIMessage({  
  content: "",  
  tool_calls: [  
    { name: "get_weather", args: { city: "SF" }, id: "call_1", type: "tool_call" },  
  ],  
})));
```

```
// Equivalent shorthand:
model.respondWithTools([ // [!code highlight]
  { name: "get_weather", args: { city: "SF" }, id: "call_1" }, // [!code highlight]
]); // [!code highlight]
```

The `id` field is optional. If omitted, a unique ID is auto-generated.

팁

`.respond()` and `.respondWithTools()` can be mixed freely in any order. This is particularly useful for testing agentic loops where the model alternates between tool calls and text responses.

Simulate errors

Errors at specific turns

Passing an `Error` to `.respond()` makes the model throw on that specific invocation. Errors can appear at any position in the sequence:

```
import { fakeModel } from "langchain";
import { AIMessage, HumanMessage } from "@langchain/core/messages";

const model = fakeModel()
  .respond(new Error("rate limit exceeded")) // Turn 1: throws // [!code highlight]
  .respond(new AIMessage("Recovered!"));    // Turn 2: succeeds

try {
  await model.invoke([new HumanMessage("first")]);
} catch (e) {
  console.log(e.message); // "rate limit exceeded"
}

const result = await model.invoke([new HumanMessage("retry")]);
console.log(result.content); // "Recovered!"
```

Errors on every call

`.alwaysThrow()` makes every invocation throw, regardless of the queue. This is useful for testing error handling and retry logic:

```
import { fakeModel } from "langchain";
import { HumanMessage } from "@langchain/core/messages";

const model = fakeModel().alwaysThrow(new Error("service unavailable"));
```

```
await model.invoke([new HumanMessage("a")]); // throws "service unavailable"
await model.invoke([new HumanMessage("b")]); // throws "service unavailable"
```

Dynamic responses with factory functions

`.respond()` also accepts a function that computes the response based on the input messages. The function receives the full message array and returns either a `@[BaseMessage]` or an `Error`:

```
import { fakeModel } from "langchain";
import { AIMessage, HumanMessage } from "@langchain/core/messages";

const model = fakeModel()
  .respond((messages) => { // [!code highlight]
    const last = messages[messages.length - 1].text; // [!code highlight]
    return new AIMessage(`You said: ${last}`); // [!code highlight]
  }); // [!code highlight]

const result = await model.invoke([new HumanMessage("hello")]);
console.log(result.content); // "You said: hello"
```

Factory functions can also return errors:

```
import { fakeModel } from "langchain";
import { AIMessage, HumanMessage } from "@langchain/core/messages";

const model = fakeModel()
  .respond((messages) => {
    const content = messages[messages.length - 1].text;
    if (content.includes("forbidden")) {
      return new Error("Content policy violation");
    }
    return new AIMessage("OK");
  });

await model.invoke([new HumanMessage("forbidden topic")]); // throws "Content policy violation"
```

참고

Each function is a single queue entry, consumed once. To reuse the same dynamic logic for multiple turns, queue multiple `respond` function calls.

Structured output

For code that uses `.withStructuredOutput()`, configure the fake return value with `.structuredResponse()`:

```
import { fakeModel } from "langchain";
import { HumanMessage } from "@langchain/core/messages";
import { z } from "zod";

const model = fakeModel()
  .structuredResponse({ temperature: 72, unit: "fahrenheit" }); // [!code highlight]

const structured = model.withStructuredOutput(
  z.object({
    temperature: z.number(),
    unit: z.string(),
  })
);

const result = await structured.invoke([new HumanMessage("Weather?")]);
console.log(result);
// { temperature: 72, unit: "fahrenheit" }
```

The schema passed to `.withStructuredOutput()` is ignored. The model always returns the value configured with `.structuredResponse()`. This keeps tests focused on application logic rather than parsing.

Assert what the model received

`fakeModel` records every invocation, including the messages and options passed to the model. This works like a spy or mock in traditional testing frameworks:

```
import { fakeModel } from "langchain";
import { AIMessage, HumanMessage } from "@langchain/core/messages";

const model = fakeModel()
  .respond(new AIMessage("first"))
  .respond(new AIMessage("second"));

await model.invoke([new HumanMessage("question 1")]);
await model.invoke([new HumanMessage("question 2")]);

console.log(model.callCount); // 2

console.log(model.calls[0].messages[0].content); // "question 1"
console.log(model.calls[1].messages[0].content); // "question 2"
```

Calls are recorded even when the model throws:

```
import { fakeModel } from "langchain";
import { HumanMessage } from "@langchain/core/messages";

const model = fakeModel().respond(new Error("boom"));

try {
  await model.invoke([new HumanMessage("will fail")]);
} catch {
  // error handled
}

console.log(model.callCount); // 1
console.log(model.calls[0].messages[0].content); // "will fail"
```

Use with `bindTools`

Agent frameworks like LangChain agents and LangGraph call `model.bindTools(tools)` internally. `fakeModel` handles this automatically. The bound model shares the same response queue and call recording as the original, so no special setup is needed:

```
import { fakeModel } from "langchain";
import { AIMessage, HumanMessage } from "@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const searchTool = tool(async ({ query }) => `Results for: ${query}`, {
  name: "search",
  description: "Search the web",
  schema: z.object({ query: z.string() }),
});

const model = fakeModel()
  .respondWithTools([
    { name: "search", args: { query: "weather" }, id: "1" }
  ])
  .respond(new AIMessage("The weather is sunny."));

const bound = model.bindTools([searchTool]);

const r1 = await bound.invoke([new HumanMessage("weather?")]);
console.log(r1.tool_calls[0].name); // "search"

const r2 = await bound.invoke([new HumanMessage("thanks")]);
console.log(r2.content); // "The weather is sunny."

// Call recording is shared. Inspect via the original model.
console.log(model.callCount); // 2
```

Full example: test a tool-calling agent with vitest

```

import { describe, test, expect } from "vitest";
import { fakeModel } from "langchain";
import { AIMessage, HumanMessage, ToolMessage } from "@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

const getWeather = tool(
  async ({ city }) => `72°F and sunny in ${city}`,
  {
    name: "get_weather",
    description: "Get weather for a city",
    schema: z.object({ city: z.string() }),
  }
);

async function runAgent(
  model: ReturnType<typeof fakeModel>,
  input: string
) {
  const messages: any[] = [new HumanMessage(input)];
  const bound = model.bindTools([getWeather]);

  while (true) {
    const response = await bound.invoke(messages);
    messages.push(response);

    if (!response.tool_calls?.length) {
      return { messages, finalResponse: response };
    }

    for (const tc of response.tool_calls) {
      const result = await getWeather.invoke(tc.args);
      messages.push(new ToolMessage({
        content: result as string,
        tool_call_id: tc.id!,
      }));
    }
  }
}

describe("weather agent", () => {
  test("calls get_weather and returns a final answer", async () => {
    const model = fakeModel()
      .respondWithTools([
        { name: "get_weather", args: { city: "SF" }, id: "call_1" },
      ])
      .respond(new AIMessage("It's 72°F and sunny in SF!"));

    const { finalResponse } = await runAgent(model, "Weather in SF?");
  });
});

```

```

expect(finalResponse.content).toBe("It's 72°F and sunny in SF!");
expect(model.callCount).toBe(2);

const secondCall = model.calls[1].messages;
const toolMsg = secondCall.find((m: any) => m._getType() === "tool");
expect(toolMsg?.content).toContain("72°F and sunny in SF");
});

test("handles model errors gracefully", async () => {
  const model = fakeModel()
    .respond(new Error("rate limit"));

  await expect(
    runAgent(model, "Weather?")
  ).rejects.toThrow("rate limit");

  expect(model.callCount).toBe(1);
});
});

```

Next steps

Learn how to test your agent with real model provider APIs in [Integration testing](#).